Just Build It!

A Complete Two-Course C++ Programming Textbook

Lone Star College

Contents

Chapter 1 — Computing, Data, and the Software Landscape **1**
Learning Objectives . . . 1
1.1 What a Program Is . . . 1
1.2 Hardware in Brief . . . 2
1.3 Representing Data: Bits and Bytes . . . 3
1.4 Numbers in Binary, Octal, and Hexadecimal . . . 5
1.5 Representing Negative and Real Numbers . . . 7
1.6 Representing Text . . . 8
1.7 How Data Are Manipulated and Stored . . . 9
1.8 Categories of Programming Languages . . . 9
1.9 Where C++ Fits, and Why This Book Uses It . . . 11
Common Misconceptions . . . 12
Design Notes . . . 12
Grade Calculator v0.0 — Problem Statement . . . 13
Try It Yourself . . . 16
Summary . . . 17
Key Terms . . . 18

Chapter 2 — From Source Code to Running Program **20**
Learning Objectives . . . 20
2.1 Anatomy of a Simple C++ Program . . . 20
2.2 The Translation Pipeline . . . 22
2.3 Development Tools . . . 25
2.4 Setting Up Your Environment . . . 25
2.5 Editing, Compiling, and Running Your First Program . . . 26
2.6 Reading Compiler Messages . . . 27
2.7 A Template for Simple C++ Programs . . . 28
2.8 Source Formatting and Comments from Day One . . . 29
Common Errors and Warnings . . . 29
Design Notes . . . 30
Grade Calculator v0.1 — Hello, Gradebook . . . 30
Try It Yourself . . . 34
Summary . . . 36
Key Terms . . . 37

Chapter 3 — Values, Variables, and Data Types **39**
Learning Objectives . . . 39
3.1 Literal Values . . . 39

3.2 Variables and Assignment . . . 39
3.3 Identifiers and Naming . . . 41
3.4 Integer Types . . . 41
3.5 Floating-Point Types . . . 42
3.6 Characters . . . 43
3.7 Boolean Values . . . 43
3.8 Constants . . . 44
3.9 Enumerated Types . . . 44
3.10 Type Inference with `auto` . . . 44
3.11 Choosing the Right Type . . . 45
3.12 Console Input . . . 45
Common Errors and Warnings . . . 46
Design Notes . . . 46
Grade Calculator v0.2 — Naming and Storing an Assignment . . . 47
Try It Yourself . . . 52
Summary . . . 55
Key Terms . . . 55

Chapter 4 — Expressions, Arithmetic, and Errors **57**
Learning Objectives . . . 57
4.1 Expressions and Operators . . . 57
4.2 Integer and Floating-Point Arithmetic Differ . . . 58
4.3 Integer Division and Modulus . . . 58
4.4 Mixed-Type Expressions and Conversion . . . 59
4.5 Operator Precedence and Associativity . . . 59
4.6 Compound Assignment, Increment, and Decrement . . . 60
4.7 Formatting Numeric Output . . . 61
4.8 Bitwise Operators . . . 61
4.9 Kinds of Errors . . . 62
4.10 Comments and Self-Documenting Code . . . 63
Common Errors and Warnings . . . 64
Design Notes . . . 64
Grade Calculator v0.3 — Percentage and Bonus Points . . . 64
Try It Yourself . . . 71
Summary . . . 73
Key Terms . . . 74

Chapter 5 — Algorithmic Design and Program Documentation **75**
Learning Objectives . . . 75
5.1 What Makes an Algorithm . . . 75
5.2 From Requirements to Specification . . . 76
5.3 Writing Pseudocode . . . 77
5.4 Flowcharting . . . 78
5.5 Structure Charts and Top-Down Decomposition . . . 80
5.6 Desk Checking and Trace Tables . . . 81
5.7 Structured Programming . . . 81
5.8 User Interface Design for Console Programs . . . 82
5.9 Documenting Your Design . . . 83
Common Design Errors . . . 83
Design Notes . . . 83

Grade Calculator v0.4 — Designed Interface . . . 84
Try It Yourself . . . 91
Summary . . . 92
Key Terms . . . 93

Chapter 6 — Conditional Execution **94**
Learning Objectives . . . 94
6.1 The `bool` Type Revisited . . . 94
6.2 Relational Operators and Boolean Expressions . . . 94
6.3 The Simple `if` Statement . . . 95
6.4 Compound Statements and Scope . . . 95
6.5 The `if/else` Statement . . . 96
6.6 Logical Operators and Compound Conditions . . . 96
6.7 Short-Circuit Evaluation . . . 97
6.8 Nested Conditionals . . . 97
6.9 Multi-way `if/else if` Chains . . . 98
6.10 The `switch` Statement . . . 99
6.11 The Conditional Operator . . . 99
6.12 Errors in Conditional Statements . . . 100
Common Errors and Warnings . . . 101
Design Notes . . . 101
Grade Calculator v0.5 — Letter Grades . . . 102
Try It Yourself . . . 109
Summary . . . 112
Key Terms . . . 112

Chapter 7 — Iteration **114**
Learning Objectives . . . 114
7.1 The `while` Statement . . . 114
7.2 Counter-Controlled and Sentinel-Controlled Loops . . . 115
7.3 The `do/while` Statement . . . 115
7.4 The `for` Statement . . . 116
7.5 Choosing the Right Loop . . . 116
7.6 Nested Loops . . . 116
7.7 Altering Loop Flow . . . 117
7.8 Infinite Loops . . . 117
7.9 Accumulators, Counters, and Running Totals . . . 118
7.10 Loop Design and Off-by-One Errors . . . 119
Common Errors and Warnings . . . 119
Design Notes . . . 119
Grade Calculator v0.6 — Multiple Assignments . . . 120
Try It Yourself . . . 129
Summary . . . 132
Key Terms . . . 132

Chapter 8 — Using Library Functions **134**
Learning Objectives . . . 134
8.1 Functions, from the Caller's Side . . . 134
8.2 Arguments, Parameters, and Return Values . . . 134
8.3 Headers and the Standard Library . . . 135

8.4 Mathematical Functions . . . 135
8.5 Rounding, Minimum, and Maximum . . . 136
8.6 Character Functions . . . 137
8.7 Random Numbers . . . 137
8.8 Timing with `clock` . . . 138
8.9 Reading Library Documentation . . . 138
Common Errors and Warnings . . . 138
Design Notes . . . 139
Grade Calculator v0.7 — Rounding and Robustness . . . 139
Try It Yourself . . . 147
Summary . . . 150
Key Terms . . . 150

Chapter 9 — Writing Functions **152**
Learning Objectives . . . 152
9.1 Defining a Function . . . 152
9.2 Prototypes, Definitions, and Order . . . 153
9.3 Parameters and Pass by Value . . . 153
9.4 Return Values and `void` Functions . . . 154
9.5 Local Variables, Scope, and Lifetime . . . 154
9.6 Modular Design in Practice . . . 155
9.7 From Structure Chart to Function Set . . . 156
9.8 Commenting and Documenting Functions . . . 156
9.9 Testing Functions Individually . . . 157
9.10 Worked Examples . . . 157
Common Errors and Warnings . . . 158
Design Notes . . . 159
Grade Calculator v1.0 — Modular Rebuild . . . 159
Try It Yourself . . . 170
Summary . . . 173
Key Terms . . . 174

Chapter 10 — Managing Functions and Data **175**
Learning Objectives . . . 175
10.1 Global and Static Variables . . . 175
10.2 Overloaded Functions . . . 176
10.3 Default Arguments . . . 176
10.4 Recursion . . . 177
10.5 Pointers . . . 177
10.6 Reference Variables . . . 178
10.7 Pass by Reference . . . 179
10.8 Splitting a Program Across Files . . . 180
10.9 Making Functions Reusable . . . 181
Common Errors and Warnings . . . 182
Design Notes . . . 182
Grade Calculator v1.1 — Multi-File and Reusable . . . 182
Try It Yourself . . . 195
Summary . . . 198
Key Terms . . . 198

Chapter 11 — Arrays **200**
Learning Objectives . . . 200
11.1 Why Collections Are Needed . . . 200
11.2 Declaring and Initializing Arrays . . . 200
11.3 Indexing and Bounds . . . 201
11.4 Traversing an Array . . . 201
11.5 Arrays and Functions . . . 202
11.6 Parallel Arrays . . . 202
11.7 Two-Dimensional Arrays . . . 203
11.8 Replacing Code with Data . . . 204
11.9 C-Style Strings . . . 205
11.10 Command-Line Arguments *(reference only)* . . . 205
11.11 Common Array Errors . . . 206
Design Notes . . . 206
Grade Calculator v1.2 — Class Roster and Custom Letter Scale . . . 207
Try It Yourself . . . 220
Summary . . . 223
Key Terms . . . 224

Chapter 12 — Vectors and Standard Strings **225**
Learning Objectives . . . 225
12.1 The Limitations of Raw Arrays . . . 225
12.2 Declaring and Using a Vector . . . 225
12.3 Growing and Shrinking . . . 226
12.4 Vector Member Functions . . . 226
12.5 Range-Based `for` Loops . . . 227
12.6 Vectors and Functions . . . 227
12.7 Vectors of Vectors . . . 228
12.8 `std::string` as a Sequence . . . 228
12.9 String Operations . . . 229
12.10 Vectors versus Arrays . . . 229
Common Errors and Warnings . . . 230
Design Notes . . . 230
Grade Calculator v1.3 — Dynamic Roster . . . 230
Try It Yourself . . . 245
Summary . . . 248
Key Terms . . . 248

Chapter 13 — The Software Development Lifecycle **250**
Learning Objectives . . . 250
13.1 Why Process Matters . . . 250
13.2 The Five Phases . . . 251
13.3 Lifecycle Models . . . 252
13.4 Programming Methodologies . . . 253
13.5 Coding Standards . . . 254
13.6 Version Control . . . 254
Common Process Failures . . . 255
Design Notes . . . 255
Grade Calculator v2.0 — Inherited Codebase . . . 255
Try It Yourself . . . 263

Summary . . . 265
Key Terms . . . 265

Chapter 14 — Structs: Grouping Related Data **267**
Learning Objectives . . . 267
14.1 The Problem with Parallel Arrays . . . 267
14.2 Declaring and Using a `struct` . . . 268
14.3 Member Access . . . 268
14.4 Structs and Functions . . . 268
14.5 Vectors of Structs . . . 269
14.6 Nested Structs . . . 269
14.7 Designing Records from Requirements . . . 270
14.8 Structs as a Bridge to Classes . . . 270
Common Errors and Warnings . . . 271
Design Notes . . . 271
Grade Calculator v2.1 — Student Records . . . 271
Try It Yourself . . . 278
Summary . . . 281
Key Terms . . . 282

Chapter 15 — File Streams and Persistent Data **283**
Learning Objectives . . . 283
15.1 Streams as an Abstraction . . . 283
15.2 Writing to a File . . . 284
15.3 Reading from a File . . . 284
15.4 Checking Stream State . . . 284
15.5 Parsing a Line into Fields . . . 285
15.6 Converting Text to Numbers . . . 285
15.7 Building Text in Memory . . . 286
15.8 Designing a File Format . . . 286
15.9 Robust File Handling . . . 287
Common Errors and Warnings . . . 288
Design Notes . . . 288
Grade Calculator v2.2 — Save and Load . . . 288
Try It Yourself . . . 297
Summary . . . 299
Key Terms . . . 300

Chapter 16 — Debugging and Testing **301**
Learning Objectives . . . 301
16.1 Categories of Defects, Revisited . . . 301
16.2 A Systematic Debugging Process . . . 302
16.3 The Symbolic Debugger . . . 302
16.4 Debugging Run-Time Errors . . . 304
16.5 Debugging Logic Errors . . . 305
16.6 Assertions and Diagnostic Output . . . 305
16.7 Writing Test Cases . . . 306
16.8 Documenting Defects . . . 307
Common Debugging Mistakes . . . 308
Design Notes . . . 308

Grade Calculator v2.3 — Defect Hunt . . . 308
Try It Yourself . . . 314
Summary . . . 316
Key Terms . . . 316

Chapter 17 — Searching and Sorting **318**
Learning Objectives . . . 318
17.1 Why Algorithm Choice Matters . . . 318
17.2 Linear Search . . . 318
17.3 Binary Search . . . 319
17.4 Selection Sort . . . 320
17.5 Insertion Sort . . . 321
17.6 Bubble Sort and Its Reputation . . . 321
17.7 Merge Sort and Divide and Conquer . . . 322
17.8 Comparing Algorithms . . . 322
17.9 Sorting by Different Criteria . . . 323
17.10 Library Sorting and Searching . . . 323
Common Errors . . . 324
Design Notes . . . 324
Grade Calculator v2.4 — Rankings . . . 325
Try It Yourself . . . 330
Summary . . . 333
Key Terms . . . 334

Chapter 18 — Classes and Objects **335**
Learning Objectives . . . 335
18.1 From Struct to Class . . . 335
18.2 Objects, State, and Behavior . . . 335
18.3 Declaring a Class . . . 336
18.4 Member Functions . . . 337
18.5 Encapsulation . . . 338
18.6 Constructors . . . 338
18.7 Accessors and Mutators . . . 339
18.8 Class Invariants . . . 339
18.9 Identifying Classes from Requirements . . . 341
Common Errors and Warnings . . . 341
Design Notes . . . 342
Grade Calculator v2.5 — Student and GradeScale Classes . . . 342
Try It Yourself . . . 353
Summary . . . 356
Key Terms . . . 356

Chapter 19 — Refining Classes **358**
Learning Objectives . . . 358
19.1 Passing and Returning Objects . . . 358
19.2 The `this` Pointer . . . 359
19.3 Separating Declaration from Definition . . . 359
19.4 Operator Overloading . . . 360
19.5 Static Members . . . 362
19.6 What the Compiler Writes for You . . . 362

19.7 Friends . . . 362
Common Errors and Warnings . . . 363
Design Notes . . . 363
Grade Calculator v2.6 — Gradebook Class . . . 364
Try It Yourself . . . 377
Summary . . . 381
Key Terms . . . 381

Chapter 20 — Inheritance **382**
Learning Objectives . . . 382
20.1 A New Requirement: Weighted Grading . . . 382
20.2 Two Designs Compared . . . 383
20.3 Modeling "Is-A" Relationships . . . 385
20.4 Base and Derived Classes . . . 385
20.5 Constructors Under Inheritance . . . 386
20.6 `protected` Members . . . 386
20.7 Overriding Member Functions . . . 387
20.8 Multilevel Hierarchies . . . 388
20.9 Composition as an Alternative . . . 388
20.10 When Not to Inherit . . . 389
Common Errors and Warnings . . . 389
Design Notes . . . 389
Grade Calculator v3.0 — Weighted Grading . . . 390
Try It Yourself . . . 403
Summary . . . 407
Key Terms . . . 407

Chapter 21 — Polymorphism **408**
Learning Objectives . . . 408
21.1 The Problem Polymorphism Solves . . . 408
21.2 Virtual Functions . . . 409
21.3 Base Class Pointers and References . . . 409
21.4 Object Slicing . . . 409
21.5 Abstract Classes and Pure Virtual Functions . . . 410
21.6 Virtual Destructors . . . 411
21.7 Polymorphic Collections . . . 411
21.8 Runtime Type Information . . . 411
21.9 I/O Streams as a Case Study . . . 412
21.10 The Costs . . . 412
Common Errors and Warnings . . . 412
Design Notes . . . 413
Grade Calculator v3.1 — Two Schemes, Chosen at Run Time . . . 413
Try It Yourself . . . 426
Summary . . . 429
Key Terms . . . 430

Chapter 22 — Memory Management and Smart Pointers **431**
Learning Objectives . . . 431
22.1 Memory Regions . . . 431
22.2 Automatic and Dynamic Storage . . . 432

22.3 `new` and `delete` . . . 433
22.4 Three Ways to Get It Wrong . . . 433
22.5 RAII . . . 434
22.6 The Rule of Three . . . 434
22.7 Smart Pointers . . . 435
22.8 Linked Lists . . . 436
22.9 Move Semantics, Briefly . . . 436
Common Errors and Warnings . . . 437
Design Notes . . . 437
Grade Calculator v3.2 — Safe Ownership . . . 437
Try It Yourself . . . 446
Summary . . . 449
Key Terms . . . 449

Chapter 23 — Templates and the Standard Template Library **451**
Learning Objectives . . . 451
23.1 The Case for Generic Code . . . 451
23.2 Function Templates . . . 451
23.3 Class Templates . . . 452
23.4 The Standard Template Library . . . 453
23.5 Iterators . . . 453
23.6 `std::map` . . . 454
23.7 Lambda Functions . . . 455
23.8 Standard Algorithms . . . 456
23.9 Namespaces . . . 456
Common Errors and Warnings . . . 456
Design Notes . . . 457
Grade Calculator v3.3 — Generic and Indexed . . . 457
Try It Yourself . . . 467
Summary . . . 471
Key Terms . . . 471

Chapter 24 — Exceptions, Release, and the Road Ahead **473**
Learning Objectives . . . 473
24.1 Why Return Codes Are Not Enough . . . 473
24.2 `throw`, `try`, and `catch` . . . 474
24.3 Standard Exception Types . . . 474
24.4 A Custom Exception Hierarchy . . . 474
24.5 Catching Multiple Exceptions . . . 475
24.6 Stack Unwinding and Exception Safety . . . 476
24.7 When to Use Exceptions . . . 476
24.8 Program Translation, Revisited . . . 477
24.9 Preparing a Release . . . 478
Common Errors and Warnings . . . 478
Design Notes . . . 479
Grade Calculator v4.0 — Release . . . 479
The Road Ahead . . . 495
Try It Yourself . . . 495
Summary . . . 498
Key Terms . . . 499

Afterword . . . 499

Appendix A — Setting Up Your Development Environment 500
A.1 What This Book Requires . . . 500
A.2 Option 1 — The StudySite.ai Editor . . . 500
A.3 Option 2 — GitHub Codespaces . . . 501
A.4 Setting Up the Debugger . . . 502
A.5 Option 3 — VS Code on Your Own Computer . . . 503
A.6 Verifying Your Setup . . . 503
A.7 When Setup Goes Wrong . . . 504
A.8 Which Environment for Which Chapter . . . 504

Appendix B — Command Line Development 506
B.1 Moving Around . . . 506
B.2 The Compile Command . . . 506
B.3 Stopping After Each Stage . . . 507
B.4 Running Programs . . . 508
B.5 Debugging from the Command Line . . . 509
B.6 Finding Memory Errors . . . 509
B.7 Makefiles . . . 509
B.8 Version Control . . . 510
B.9 Command Reference . . . 511
B.10 Common Command Line Problems . . . 511

Appendix C — Debugger Reference Card 512
C.1 Before You Start . . . 512
C.2 Core Commands . . . 512
C.3 Breakpoints . . . 512
C.4 Inspecting State . . . 513
C.5 The Call Stack . . . 513
C.6 Watchpoints . . . 513
C.7 A Debugging Session, Start to Finish . . . 514
C.8 Debugging Techniques by Symptom . . . 514
C.9 When Not to Use a Debugger . . . 514
C.10 Common Debugger Problems . . . 515

Appendix D — Coding Standard and Style Guide 516
D.1 Layout . . . 516
D.2 Naming . . . 517
D.3 Comments . . . 518
D.4 Files and Includes . . . 519
D.5 Functions . . . 520
D.6 Variables and Types . . . 521
D.7 Classes . . . 522
D.8 Error Handling . . . 522
D.9 Output Formatting . . . 523
D.10 Portability . . . 523
D.11 What Not to Do . . . 524
D.12 Pre-Submission Checklist . . . 524
D.13 A Conforming Example . . . 525

Appendix E — Glossary **527**
A . . . 527
B . . . 528
C . . . 528
D . . . 529
E . . . 530
F . . . 531
G . . . 531
H . . . 531
I . . . 532
L . . . 532
M . . . 533
N . . . 533
O . . . 534
P . . . 534
R . . . 535
S . . . 536
T . . . 537
U . . . 538
V . . . 538
W . . . 539

Appendix F — SLO-to-Chapter Coverage Matrix **540**
Course I — Programming Fundamentals (Chapters 1–12) . . . 540
Course II — Object-Oriented Programming (Chapters 13–24) . . . 542
Where each outcome is assessed . . . 545
The traced requirement . . . 546

Chapter 1 — Computing, Data, and the Software Landscape

Learning Objectives

When you finish this chapter you will be able to:

- Explain what a program is and describe the four parts of a computer and how data moves between them. *(SLO 1.1)*
- Describe how data are represented in a computer using bits and bytes, and calculate how many distinct values a given number of bits can hold. *(SLO 1.1)*
- Convert whole numbers between decimal, binary, and hexadecimal. *(SLO 1.1)*
- Explain how negative numbers, real numbers, and text are represented, and identify the limits each representation imposes. *(SLO 1.1)*
- Distinguish machine language, assembly language, and high-level languages, and explain the trade-off each represents. *(SLO 1.2)*
- Distinguish compiled, interpreted, and hybrid languages, and describe the consequences of each for how a program is built and run. *(SLO 1.2)*
- Categorize programming languages by paradigm and select a language category appropriate to a stated purpose. *(SLO 1.2)*
- Explain where C++ sits among these categories and why this book uses it. *(SLO 1.2)*

1.1 What a Program Is

A **program** is a sequence of instructions that tells a computer how to accomplish a task.

That definition is short, and every word in it is doing work. *Sequence* means order matters: the same instructions in a different order are a different program, usually a broken one. *Instructions* means each step is something the machine can actually carry out, not a wish. And *tells a computer* means the instructions have to be expressed in a form the machine can act on, which turns out to be the central difficulty of the whole enterprise.

Here is the difficulty. The task in your head is something like "work out each student's grade." The instructions a processor can carry out are more like "copy the eight bytes at address 4000 into register 2." Between those two descriptions lies an enormous gap, and essentially everything in this book — variables, loops, functions, classes — exists to help you cross it.

You will cross it by writing **source code**: text, written by you, in a language designed to be readable by humans and translatable into machine instructions. This is the C++ source code for a complete program, and while you are not expected to understand it yet, notice that you can already guess roughly what it does:

```cpp
#include <iostream>

int main() {
    std::cout << "Grade Calculator\n";
    return 0;
}
```

That guessability is the entire point of a programming language. A processor would not accept a single character of it.

Software and hardware

Hardware is the physical machinery: the chips, boards, drives, and cables. **Software** is the instructions the hardware carries out. The distinction is real but the boundary is less crisp than it sounds — a modern processor is itself controlled by low-level software, and some software is delivered permanently fixed in a chip. For your purposes the useful version is this: hardware is what you could drop on your foot, and software is what you can change without a screwdriver.

Software divides broadly into two kinds. **System software** manages the machine itself — the operating system, device drivers, and the development tools you will meet in Chapter 2. **Application software** does something a person actually wanted done: a browser, a spreadsheet, a game, or the grade calculator you are about to build.

1.2 Hardware in Brief

You do not need a hardware course to write good programs. You do need a working mental model of where your data lives, because several things that will otherwise seem arbitrary — why numbers overflow, why files must be opened, why a variable forgets its value when the program ends — follow directly from it.

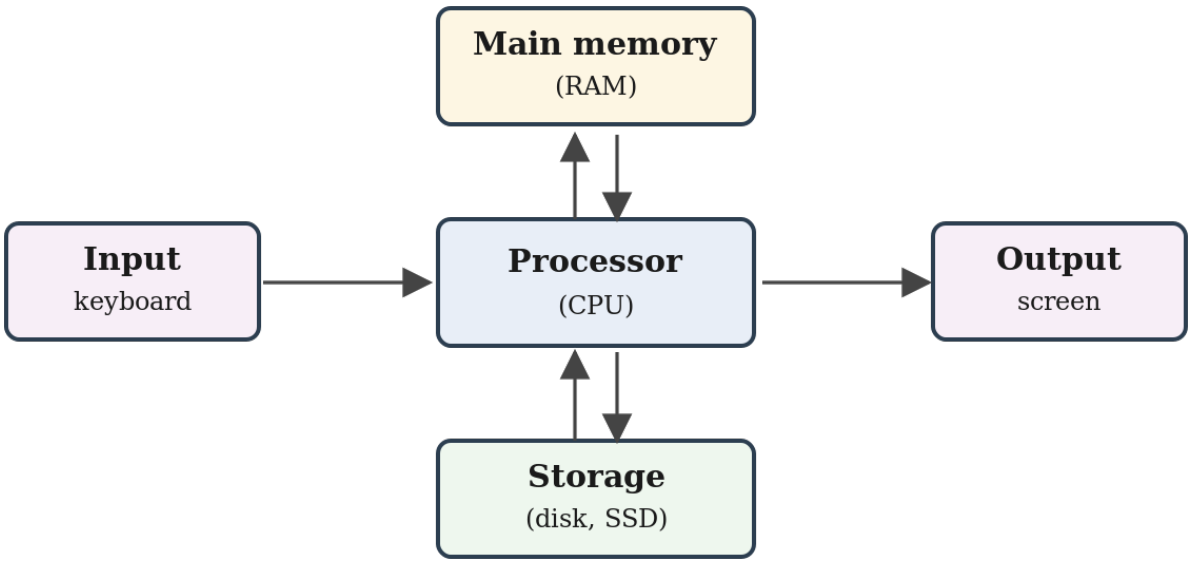


Figure 1: A block diagram showing the processor at the centre, main memory above it connected by a two-way arrow, input on the left with an arrow pointing in, output on the right with an arrow pointing out, and storage below connected to main memory by a two-way arrow.

Figure 1.1 — The four parts of a computer and how data moves between them.

Description of Figure 1.1. The processor sits at the centre. Main memory sits above it, and data flows in both directions between them. Input devices, such as a keyboard, send data one way into the processor. Output devices, such as a screen, receive data one way from the processor. Storage sits below main memory, and data flows in both directions between those two. Main memory loses its contents when power is lost; storage does not.

The processor, or CPU, carries out instructions. It does a small number of very simple things — arithmetic, comparison, moving data from one place to another, choosing which instruction to do next — at a rate of billions of operations per second. Every sophisticated behavior you have ever seen a computer produce is built from these primitives.

Main memory, or RAM, holds the data and instructions the processor is working with right now. It is fast, and it is **volatile**: when power stops, its contents are gone. Every variable you declare in this book lives in main memory, which is why a program that does not save its work to a file has nothing to show for itself once it exits. That fact will drive a real design decision in Chapter 15.

Storage — a hard disk or SSD — holds data that must survive after the power is off. It is **non-volatile** and much slower than memory. Files live here.

Input and output connect the machine to the world. In this book, input means text you type at a keyboard and output means text printed to the screen. That is a deliberate narrowing, and Section 5.8 explains why it is a reasonable place to start.

The one relationship to carry forward: **the processor works only with data in main memory**. Anything in storage must be loaded into memory before it can be used, and anything in memory must be written to storage before it will survive. Reading a file, saving your work, and the difference between the two are all consequences of that single fact.

1.3 Representing Data: Bits and Bytes

Here is the thing that surprises most people who look closely for the first time. A computer does not store numbers, or letters, or images. It stores **electrical states**, and the numbers, letters, and images you work with are conventions for interpreting those states.

The smallest unit is the **bit**, short for *binary digit*. A bit has exactly two possible states, written as `0` and `1`. Physically it might be a voltage that is high or low, or a region of a chip that is charged or not. What it *is* physically does not matter to you. What matters is that it has two states, and only two.

Two states is not much. So bits are used in groups.

Counting what a group of bits can hold

Each additional bit doubles the number of distinct patterns available, because every pattern you already had can be extended in two ways.

Bits	Distinct patterns	Calculation
1	2	2^1
2	4	2^2
3	8	2^3
4	16	2^4

Bits	Distinct patterns	Calculation
8	256	2^8
16	65,536	2^{16}
32	4,294,967,296	2^{32}
64	about 1.8×10^{19}	2^{64}

The rule is simply that ***n* bits hold 2^n distinct patterns**. Memorize that one; you will use it constantly, and it explains a great deal of otherwise mysterious behavior. When you learn in Chapter 3 that a certain kind of C++ integer can hold values from −2,147,483,648 to 2,147,483,647, you will recognize those numbers immediately: that is 2^{32} patterns, split between negative and non-negative.

The byte

A group of **eight bits** is a **byte**. The byte is the standard unit of storage — memory sizes, file sizes, and network speeds are all quoted in bytes or multiples of them. One byte holds $2^8 = 256$ distinct patterns.

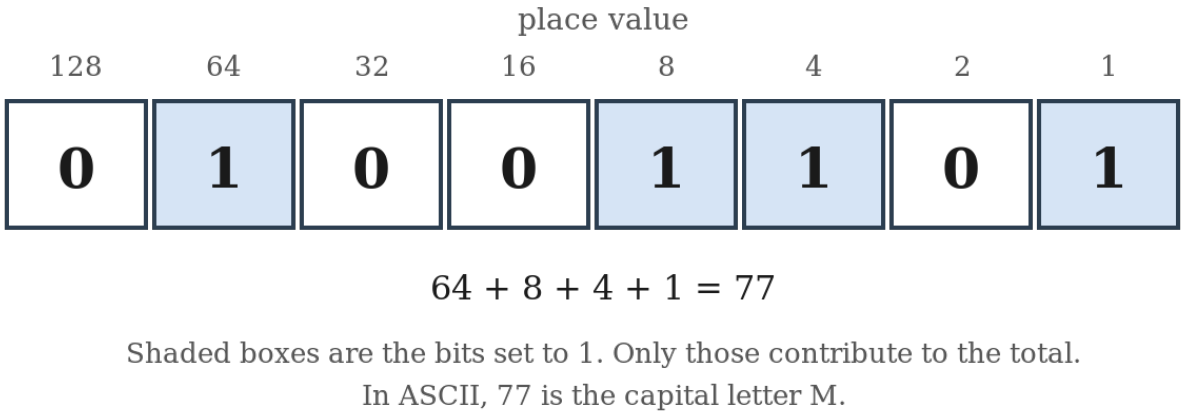


Figure 2: Eight boxes in a row representing one byte. Reading left to right the bits are 0, 1, 0, 0, 1, 1, 0, 1. Above each box is its place value: 128, 64, 32, 16, 8, 4, 2, 1. The bits set to 1 sit under place values 64, 8, 4, and 1, which add to 77.

Figure 1.2 — One byte holding the value 77.

Description of Figure 1.2. Eight bit positions are shown side by side. Reading from left to right the bits are 0, 1, 0, 0, 1, 1, 0, 1. Above each position is its place value, halving from left to right: 128, 64, 32, 16, 8, 4, 2, 1. The four positions holding a 1 are those with place values 64, 8, 4, and 1. Adding those gives 64 + 8 + 4 + 1 = 77. Under the ASCII convention described in Section 1.6, the value 77 represents the capital letter M.

Larger groupings have names too. A **word** is the amount of data a processor handles most naturally in one operation — 64 bits on most current machines. Unlike the byte, the word is not a fixed size; it depends on the processor.

For quantities of storage, note that the prefixes are used in two conflicting ways. Manufacturers usually mean powers of ten; operating systems usually mean powers of two.

Unit	Powers of ten	Powers of two
Kilobyte (KB)	1,000 bytes	1,024 bytes
Megabyte (MB)	1,000,000 bytes	1,048,576 bytes
Gigabyte (GB)	1,000,000,000 bytes	1,073,741,824 bytes

This is why a drive sold as 500 GB shows up as roughly 465 GB once your operating system has counted it. Nobody is cheating; the two are using different definitions of the same word.

Memory as addressed cells

Main memory is a long sequence of byte-sized cells, each with a number called its **address**. Addresses are consecutive whole numbers starting at zero.

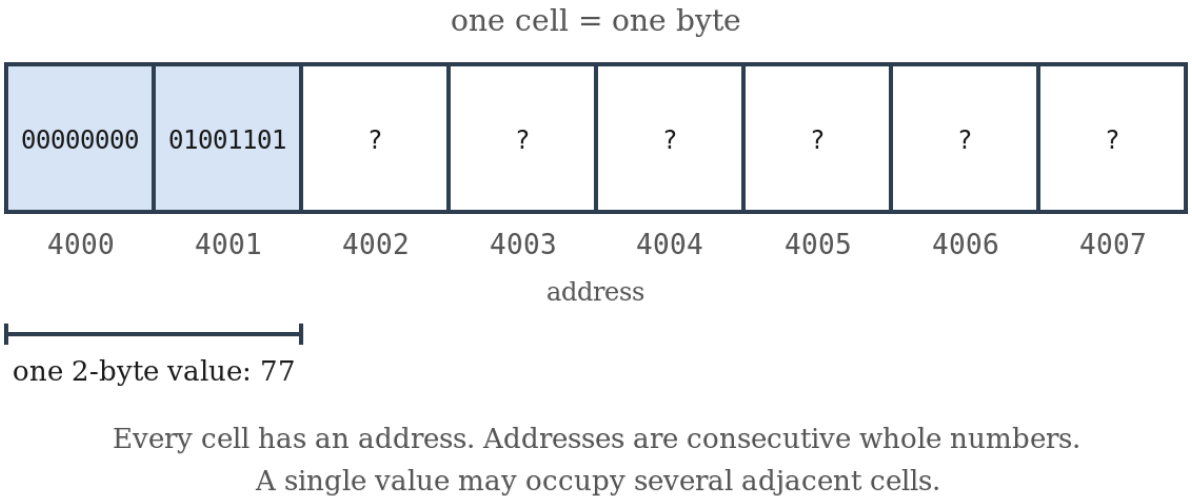


Figure 3: A row of eight cells side by side, each holding one byte, with addresses running from 4000 through 4007 beneath them. The first two cells are shaded together and bracketed as a single two-byte value.

Figure 1.3 — Main memory as a numbered sequence of cells.

Description of Figure 1.3. Eight memory cells are shown in a row. Each holds one byte. Beneath each cell is its address, running consecutively from 4000 to 4007. The cells at addresses 4000 and 4001 are shaded and bracketed together, labelled as a single value of 77 occupying two bytes. The figure notes that every cell has an address, that addresses are consecutive whole numbers, and that a single value may occupy several adjacent cells.

Two consequences matter for the rest of this book. First, **a single value may occupy several adjacent cells** — a value needing more than 256 possibilities cannot fit in one byte, so it spreads across two, four, or eight. Second, **every value in memory has an address**, and that address is itself a number that can be stored and passed around. That second fact is the whole idea behind pointers in Chapter 10. It will seem strange when you meet it. It is already true now.

1.4 Numbers in Binary, Octal, and Hexadecimal

You count in **decimal**, base ten, using ten digits. There is nothing mathematically special about ten; it is an accident of anatomy. A computer's two-state hardware makes **binary**, base two, the natural fit.

How positional notation works

In any base, a digit's contribution depends on its position, and each position is worth the base times the position to its right.

In decimal, the number 4,207 means:

Digit	4	2	0	7
Place value	1000	100	10	1
Contribution	4000	200	0	7

Total: 4000 + 200 + 0 + 7 = 4,207.

Binary works identically, except that the place values are powers of two and the only digits available are 0 and 1. The binary number 1101 means:

Digit	1	1	0	1
Place value	8	4	2	1
Contribution	8	4	0	1

Total: 8 + 4 + 0 + 1 = 13.

Converting binary to decimal

Write the place values above the digits, doubling from right to left, then add the place values that sit above a 1.

For binary 1001101, the place values from the right are 1, 2, 4, 8, 16, 32, 64:

```
 1   0   0   1   1   0   1
64  32  16   8   4   2   1
64  +  0 +  0 +  8 +  4 +  0 +  1  =  77
```

Converting decimal to binary

Repeatedly divide by two, recording each remainder, then read the remainders **from bottom to top**.

Converting 77:

Division	Quotient	Remainder
77 ÷ 2	38	1
38 ÷ 2	19	0
19 ÷ 2	9	1
9 ÷ 2	4	1
4 ÷ 2	2	0
2 ÷ 2	1	0
1 ÷ 2	0	1

Reading the remainder column from bottom to top gives 1001101, which matches the conversion above.

Hexadecimal

Binary is faithful to the hardware and miserable to read. The value 3,735,928,559 in binary is thirty-two characters of ones and zeros that no one can check at a glance.

Hexadecimal, base sixteen, solves this. It needs sixteen digits, so it borrows the letters A through F for the values ten through fifteen.

Hex	Dec	Binary	Hex	Dec	Binary
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	A	10	1010
3	3	0011	B	11	1011
4	4	0100	C	12	1100
5	5	0101	D	13	1101
6	6	0110	E	14	1110
7	7	0111	F	15	1111

Hexadecimal earns its place because of one convenient fact: **sixteen is two to the fourth power, so each hex digit corresponds to exactly four bits**. Conversion is therefore mechanical, with no arithmetic at all. Group the binary digits into fours from the right and replace each group:

```
binary   0100  1101
hex         4     D
```

So binary 01001101 is hex 4D, which is decimal 77. Two characters instead of eight, and you can convert back in your head.

You will see hexadecimal whenever raw memory is displayed — in a debugger in Chapter 16, in memory addresses in Chapter 10, and in color values on the web. **Octal**, base eight, works the same way with three bits per digit; it is largely a historical survivor, and you will meet it rarely.

1.5 Representing Negative and Real Numbers

Bits have no minus sign and no decimal point. Both have to be constructed out of conventions.

Negative numbers

The obvious approach — reserve one bit to mean "negative" — turns out to work badly. It produces two different representations of zero, and it forces subtraction to be built as a separate operation from addition.

Real machines use **two's complement**, which is a genuinely clever piece of engineering. Reserve the leftmost bit as the sign bit, but give it a **negative place value**. In an 8-bit two's complement byte, the place values are −128, 64, 32, 16, 8, 4, 2, 1.

To see it work, take the byte 11110011:

```
   1    1   1   1   0   0   1   1
-128   64  32  16   8   4   2   1
-128 +  64 + 32 + 16 +  0 +  0 +  2 +  1  =  -13
```

Two properties make this the universal choice. There is exactly one representation of zero. And ordinary binary addition produces the correct answer for negative values with no special handling, which means the same circuit does addition and subtraction.

An 8-bit two's complement byte holds values from −128 to 127. Notice the asymmetry: one more negative value than positive, because zero occupies a slot on the non-negative side. That asymmetry is real and will appear again in Chapter 3.

Real numbers

Numbers with fractional parts use **floating-point** representation, which works like scientific notation. The value 6.02×10^{23} is stored as three pieces: a sign, the digits 6.02 (the **mantissa**), and the exponent 23.

Floating-point buys enormous range — a 64-bit floating-point value covers roughly 10^{-308} to 10^{308} — and it pays for that range with **finite precision**. Only so many digits of the mantissa are kept.

This has a consequence that catches everyone once, and it is worth meeting now rather than being ambushed by it later. **Many decimal fractions cannot be represented exactly in binary.** The value 0.1 is one of them. In binary, one-tenth is a repeating fraction, exactly as one-third is a repeating decimal (0.3333…) in base ten. The computer stores the closest value it can, which is very close but not equal.

So in almost every programming language, including C++:

```
0.1 + 0.2  is not exactly equal to  0.3
```

The result is approximately 0.30000000000000004. This is not a bug in C++ or in your machine; it is a direct consequence of storing base-ten fractions in a base-two system with limited space. Chapter 4 returns to this, and Chapter 9 shows the standard technique for comparing floating-point values safely.

The practical rule you can adopt today: **use whole numbers when the quantity is genuinely whole, and never test floating-point values for exact equality.**

1.6 Representing Text

Text is stored by agreeing that particular numbers stand for particular characters.

ASCII — the American Standard Code for Information Interchange — assigns the numbers 0 through 127 to English letters, digits, punctuation, and a set of control codes. Seven bits are enough, so one character fits comfortably in a byte.

Character	ASCII value	Character	ASCII value
`A`	65	`a`	97
`B`	66	`b`	98
`M`	77	`m`	109
`Z`	90	`z`	122
`0`	48	space	32
`9`	57	newline	10

Three details in that table repay attention. The uppercase letters are consecutive, and so are the lowercase, which is why you can do arithmetic on characters in Chapter 3. Uppercase and lowercase are separated by exactly 32, which is why case conversion is a single addition. And **the character `0` is the number 48, not the number 0** — the digit you type and the quantity it names are entirely different things. That distinction causes real confusion later, and this is the moment it becomes clear.

ASCII's limitation is obvious: 128 values cannot hold Greek, Arabic, Chinese, or an emoji. **Unicode** solves this by assigning a number, called a **code point**, to every character in every writing system — over 150,000 of them so far.

Unicode says what number each character has, but not how to store it. That is the job of an **encoding**. The dominant one is **UTF-8**, which stores each character in one to four bytes: characters in the ASCII range take one

byte and have exactly their ASCII values, so every ASCII file is already a valid UTF-8 file. Characters outside that range take more.

The practical consequence for you: in UTF-8, **the number of characters in a piece of text is not necessarily the number of bytes**. For the English text in this book they are the same, which is why the programs here can treat them interchangeably. It is worth knowing that this is a simplification rather than a law.

1.7 How Data Are Manipulated and Stored

Pull the last four sections together, because there is one idea underneath all of them.

Consider a single byte containing the pattern 01001101. What does it hold?

- As an unsigned whole number, 77.
- As a two's complement signed number, 77.
- As an ASCII character, the letter `M`.
- As part of a larger value spanning several bytes, some fragment with no independent meaning.
- As a machine instruction, some operation specific to that processor.

All five readings are correct. The bits do not know which one is intended. The pattern 01001101 carries no information about its own interpretation.

What supplies the interpretation is the program. When you declare in Chapter 3 that a variable holds a whole number, you are telling the compiler how to read the bits at that address. That declaration is not a description of what is stored — it *is* the meaning. This is the reason programming languages have types at all, and it is worth carrying with you as one of the few genuinely load-bearing ideas in this chapter.

The processor's working cycle is correspondingly simple. It repeats, billions of times per second:

1. **Fetch** the next instruction from memory.
2. **Decode** it to determine what operation is required.
3. **Execute** the operation, possibly reading or writing memory.
4. Repeat.

Everything a computer does is that loop, running very fast over data whose meaning is supplied entirely by the program operating on it.

1.8 Categories of Programming Languages

Thousands of programming languages exist. They can be sorted usefully along three independent dimensions, and knowing where a language sits on each tells you most of what you need to know about when to reach for it.

1.8.1 By distance from the hardware

Machine language is what the processor actually executes: numeric codes, usually written in hexadecimal. It is the only language a processor understands, and no one writes it by choice.

```
B8 04 00 00 00
```

Assembly language replaces those numeric codes with short mnemonics, one per machine instruction. A program called an **assembler** translates it. It is readable, in the narrow sense that each line means something, but it remains tied to one specific processor family, and expressing an ordinary idea takes many lines.

```
mov eax, 4
add eax, ebx
```

High-level languages let you express operations in terms closer to the problem than to the machine. One line may correspond to dozens of machine instructions. They are portable across processors, because the translator handles the differences. C++, Python, Java, and JavaScript are all high-level languages.

```
total = total + bonusPoints;
```

The trade-off across these three is consistent: **the further you move from the hardware, the more you gain in readability, portability, and speed of development, and the less direct control you retain over exactly what the machine does.** For most work that trade is overwhelmingly worth taking, which is why high-level languages dominate.

1.8.2 By how they are translated

Compiled languages are translated in full, ahead of time, by a **compiler**, producing an executable file of machine code. The translation happens once; the result runs directly on the processor and is fast. The executable is specific to one kind of processor and operating system, so a program compiled for Windows will not run on a Mac without being recompiled. C and C++ work this way.

Interpreted languages are translated line by line, as the program runs, by an **interpreter**. Nothing is produced ahead of time. This makes the write-and-test cycle very quick and the same source runs anywhere an interpreter exists, at the cost of running more slowly, because translation happens repeatedly during execution. Python and JavaScript work this way.

Hybrid languages compile to an intermediate form called **bytecode**, which a **virtual machine** then executes. This recovers much of the speed of compilation while keeping much of the portability of interpretation. Java and C# work this way.

	Compiled	Interpreted	Hybrid
Translated	Once, before running	Continuously, while running	Once to bytecode, then run
Speed of the running program	Fastest	Slowest	Between
Speed of write-and-test	Slower — must rebuild	Fastest	Between
Portability of the result	Rebuild per platform	Runs anywhere	Runs anywhere with a VM
Errors found before running	Most	Few	Many
Examples	C, C++, Rust, Go	Python, JavaScript, Ruby	Java, C#

The row worth pausing on is the last but one. A compiler examines your entire program before it runs, so a whole class of mistakes is reported at your desk rather than discovered by a user. That is a real advantage of compiled languages for learning, and Chapter 4 makes the case in detail.

1.8.3 By paradigm

A **paradigm** is a style of organizing a program — a claim about what a program is fundamentally made of.

Procedural programming organizes a program as a sequence of steps grouped into procedures, or functions, that operate on data passed to them. Data and the code that acts on it are separate things. C is the classic example. **Chapters 2 through 12 of this book teach procedural programming**, and Course I ends with a complete procedural application.

Object-oriented programming organizes a program as a collection of objects, each bundling data together with the operations on that data. Instead of passing a student's information to a grading function, you ask a student object for its own grade. Java and C# are primarily object-oriented. **Chapters 13 through 24 teach object-oriented programming**, and Course II converts the Course I application to it — an exercise that will show you exactly what the paradigm buys.

Functional programming organizes a program as the evaluation of functions that avoid changing any stored state. Haskell is the pure example, though functional ideas now appear nearly everywhere, including in the lambda functions you will meet in Chapter 23.

These are not exclusive. Most widely used languages, C++ among them, support several, which means the paradigm is partly your choice rather than the language's decree.

1.8.4 Choosing a language for a purpose

The honest general answer is that language choice is driven as much by ecosystem, team, and existing code as by any property of the language itself. That said, the categories do predict fit.

If the priority is	Prefer	Because
Raw execution speed, tight memory control	Compiled, C or C++	No translation overhead while running; direct memory access
Getting something working quickly	Interpreted, Python	Fastest write-and-test cycle; large library ecosystem
Running unchanged across many platforms	Hybrid, Java or C#	Bytecode plus a virtual machine on each target
Running inside a web browser	JavaScript	It is what browsers execute
Catching errors before users see them	Compiled, statically typed	Whole-program checking before it ever runs
Working close to hardware	C, C++, Rust, assembly	Direct control over memory and machine behavior

A useful sanity check: a language that is a poor choice for one job may be the obvious choice for another, and "which language is best?" has no more answer than "which tool is best?"

1.9 Where C++ Fits, and Why This Book Uses It

Locate C++ on all three dimensions:

- **Distance from hardware**: high-level, but unusually close to the machine for a high-level language. It gives you direct access to memory addresses when you want it, which most high-level languages do not.
- **Translation**: compiled. Your source becomes a native executable, and the compiler checks the whole program first.
- **Paradigm**: multi-paradigm. It fully supports procedural, object-oriented, and functional styles.

Three of these properties make C++ well suited to a first course, and it is worth being explicit about them, because C++ also has a reputation for difficulty that is not entirely undeserved.

It is multi-paradigm, so one language carries both courses. You will learn procedural programming in Course I and object-oriented programming in Course II without changing languages, tools, or projects. The same Grade Calculator evolves across both. You will therefore see the difference between the paradigms clearly, because everything else stays constant.

It is compiled and statically typed, so the compiler is a teacher. Many mistakes are reported before your program ever runs, with a message pointing at the line. In an interpreted language the same mistake might surface only when a particular input reaches a particular line, possibly long after you have moved on. Compiler messages are frustrating at first and become genuinely valuable once you learn to read them, which Chapter 2 begins and Chapter 4 develops.

It does not hide the machine, so the ideas in this chapter stay visible. Bytes, addresses, and fixed-size types are things you will work with directly rather than concepts you take on faith. Chapter 10 will have you manipulate memory addresses; Chapter 22 will have you manage memory yourself before showing you the modern tools that do it for you. Languages that hide these details are pleasant to use and leave you without a model of what is actually happening.

The honest counterweight: C++ is large, it has accumulated decades of history, and it will let you make mistakes that a more protective language would prevent. This book manages that by using a modern subset — the C++17 standard — consistently, and by explaining not just what to write but why the alternatives are worse.

Common Misconceptions

There is no code yet, so there are no compiler errors yet. There are, however, four ideas from this chapter that are commonly misremembered in ways that cause trouble later.

"**The computer stores the number 77.**" It stores a bit pattern. That pattern means 77 only under a particular interpretation, and the same pattern means the letter `M` under another. The interpretation comes from your program, not from the bits. This is the point of Section 1.7 and the reason Chapter 3 spends a whole chapter on types.

"**0.1 + 0.2 equals 0.3.**" In decimal arithmetic, yes. In floating-point, no — the result is very slightly off, because 0.1 has no exact binary representation. Expecting exactness from floating-point produces bugs that appear only for certain values and are very hard to find. Never test floating-point values for equality.

"**The character `0` is the number 0.**" The character `0` is stored as 48 in ASCII. Confusing a digit character with the quantity it names is a classic source of errors when reading input, and Chapter 3 shows how to convert deliberately between them.

"**A kilobyte is 1,000 bytes.**" Sometimes. It is 1,024 bytes when your operating system says it, and 1,000 bytes when a drive manufacturer says it. Both usages are current. This is why advertised and reported drive capacities never match.

Design Notes

Two habits are worth starting now, before you write any code.

Write down what a program must do before deciding how it will do it. The temptation is to start typing. Resist it long enough to state, in ordinary sentences, what the program accepts, what it produces, and what it is not responsible for. Chapter 5 develops this into a full design method with pseudocode and flowcharts. The Grade Calculator section below is your first practice.

Say explicitly what is out of scope. A requirements document that lists only what a program *will* do leaves every reader — including you, in three weeks — to guess about the rest. Recording what you have deliberately

left out is what turns an omission into a decision. You will do this today, and you will not resolve the item you record until Chapter 20.

Grade Calculator v0.0 — Problem Statement

Throughout both courses you will build one application: a **Grade Calculator**. It grows in every chapter, and by the end of Course II it will handle multiple grading schemes, custom letter scales, file storage, and full error handling.

This chapter has no code, because you have not yet met any C++. What it has is the thing that should always come first.

Your StudySite Lab — Create the COSC 1436 Base Repository

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v0.0
- **Starting point**: No prior project files.

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

1. Use the private `COSC1436xxx-Grade-Calculator-YourLastName` repository your instructor created for you and invited you to. Do not create your own repository. Use this one repository for every chapter through Chapter 12.
2. Create `README.md` from the supplied template. Replace only `[Your name]`.
3. Create `requirements.md` by copying the supplied requirements exactly. These are the approved project requirements; do not invent or rewrite them.
4. Commit both documents as the first checkpoint. C++ coding begins in Chapter 2.

Supplied `README.md`

Copy this text and replace only `[Your name]`:

```
# Grade Calculator

A cumulative C++ project for COSC 1436.

## Student

[Your name]

## Course

COSC 1436 – Programming Fundamentals

## Project plan

This single repository contains my Grade Calculator work for Chapters 1–12.
The Chapter 12 code checkpoint will be the foundation for the separately
assigned COSC 1436 final project.
```

Supplied `requirements.md`

Copy this complete requirements document. Do not rewrite it.

```
# Grade Calculator Requirements

## Purpose

The Grade Calculator is a console application that calculates and reports
points-based course grades for a class roster.

## Inputs

The program accepts:

- the student's name;
- a user-defined letter-grade scale;
- any number of assignments;
- each assignment's name and points possible;
- each student's points earned for every assignment, including bonus points;
- a choice to drop or keep each student's lowest assignment.

## Processing

The program:

1. totals each student's earned points and possible points;
2. adds bonus points to points earned, but not to points possible;
3. optionally removes both the earned and possible points for the student's
   lowest assignment;
4. calculates the percentage using floating-point division;
5. caps the reported percentage at 100%;
6. rounds the reported percentage to one decimal place;
7. assigns a letter grade using the scale entered by the user; and
8. calculates the class average.

## Output

The program displays:

- a Grade Calculator heading;
- the active letter-grade scale;
- each student's name, percentage to one decimal place, and letter grade;
- labeled totals where required; and
- the class average.

Output must be readable, consistently aligned, and understandable without
reading the source code.

## Validation and edge cases

The program must:

- reject nonnumeric input where a number is required;
- reject negative point values;
- avoid division by zero;
- handle an empty assignment list or roster without crashing;
- reject grade-scale cutoffs that are not entered from highest to lowest; and
- ensure the scale reaches a lowest tier at 0%.

## COSC 1436 scope
```

```
This calculator uses total points only.

**Out of scope for COSC 1436:** weighted-category grading, in which categories
such as exams and homework carry different percentages of the final grade.
Weighted grading is deferred to COSC 1437.

## Storage estimate

One assignment record contains:

- assignment name: 30 characters × 1 byte = 30 bytes;
- points earned: 8 bytes;
- points possible: 8 bytes; and
- bonus points: 8 bytes.

Estimated size of one assignment record:

30 + 8 + 8 + 8 = **54 bytes**

Estimated size for 30 students with 12 assignments each:

30 × 12 × 54 = **19,440 bytes**
```

Verification

- StudySite can load from and save to the connected `COSC1436xxx-Grade-Calculator-YourLastName` repository.
- `README.md` and `requirements.md` are both present.
- There is no `main.cpp` yet.

StudySite workflow

1. Accept the GitHub invitation your instructor emailed you. It gives you access to **COSC1436xxx-Grade-Calculator-YourLastName**, the private repository already created for you in the **lscup** organization, where *YourLastName* is your own last name. Do not create your own repository.
2. In StudySite, open this chapter's **coding panel on the main stage**.
3. Click **Load from GitHub**. Connect GitHub if prompted, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `README.md` to load it into the editor.
4. Replace the README with the supplied text and create `requirements.md` by copying the complete supplied requirements. Replace only `[Your name]` in the README.
5. Chapter 1 has no compile or run step. The goal is to create the base repository and prove that StudySite can load and save its files.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Create COSC 1436 Grade Calculator base repository**.

5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- The verification list passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 1 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

No compiler needed. These use only pencil, paper, and the tables in this chapter.

1. Binary to decimal

Convert each to decimal by writing the place values above the digits and adding those above a 1.

Binary	Decimal
`1010`	?
`100000`	?
`1111`	?
`10010110`	?

Check yourself: the second one is a power of two. If your answer is not 32, recount your place values — this is the most common slip.

2. Decimal to binary

Convert using repeated division by two, reading remainders bottom to top.

Decimal	Binary
12	?
65	?
100	?
255	?

Check yourself: 255 is the largest value one byte can hold as an unsigned number. Its binary form should be eight 1s. If you got nine digits, check your division.

3. Binary to hexadecimal

Group each into fours from the right and use the table in Section 1.4.

Binary	Hex
`11111111`	?
`00010000`	?
`10101010`	?

4. Counting patterns

- How many distinct values can 12 bits hold?
- How many bits do you need to give a distinct pattern to each of 1,000 items? (Find the smallest n where $2^n \geq 1000$.)
- A student ID must distinguish 50,000 students. Is a 16-bit value enough? Show why or why not.

5. Reading a byte five ways

The byte `01000001` holds the value 65.

- What letter is this in ASCII?
- What is it in hexadecimal?
- If the leftmost bit were a two's complement sign bit, would this value be positive or negative? How can you tell without any arithmetic?
- Change only the leftmost bit to 1. As an unsigned number, what is the new value? As a two's complement signed number?

6. Predict, then reason

- Your friend says their program printed `0.30000000000000004` when it added 0.1 and 0.2, and asks whether their computer is broken. What do you tell them, and what would you suggest they do instead of testing for exact equality?
- A file contains the two characters `4` and `2`. How many bytes is the file? What decimal values do those two bytes hold? Why is that not 42?

7. Choosing a category

For each situation, name the language category you would prefer from Section 1.8.4, and give one sentence of justification.

- Software controlling an engine, which must respond within a guaranteed time
- A script to rename 400 files on your laptop, needed within the hour
- An application that must run identically on Windows, macOS, and Linux
- An interactive chart on a public web page

Summary

- A **program** is a sequence of instructions telling a computer how to accomplish a task. You write **source code**; the machine executes machine instructions; something must translate between them.
- A computer has four parts: **processor**, **main memory**, **storage**, and **input/output**. The processor works only with data in main memory. Memory is **volatile**; storage is not.
- The **bit** is the smallest unit, with two states. ***n* bits hold 2^n distinct patterns**. Eight bits make a **byte**.
- Memory is a sequence of byte-sized cells, each with a numeric **address**. A single value may span several cells, and addresses are themselves numbers that can be stored.
- **Binary** is base two, **hexadecimal** base sixteen. Each hex digit maps to exactly four bits, which is why hexadecimal is used to display raw memory.
- Negative numbers use **two's complement**, giving the leftmost bit a negative place value. **Floating-point** gives real numbers enormous range at the cost of exactness; **never test floating-point values for equality**.

- Text is stored by convention: **ASCII** for 128 characters, **Unicode** with an encoding such as **UTF-8** for everything else. The character `0` is the value 48, not 0.
- **A bit pattern has no inherent meaning.** Interpretation is supplied by the program, which is why programming languages have types.
- Languages differ in **distance from hardware** (machine, assembly, high-level), **translation method** (compiled, interpreted, hybrid), and **paradigm** (procedural, object-oriented, functional).
- **C++ is a high-level, compiled, multi-paradigm language.** This book uses it because one language carries both the procedural and object-oriented courses, the compiler catches mistakes before the program runs, and it keeps the machine visible instead of hiding it.

Key Terms

address — the number identifying one cell of main memory.

application software — software that performs a task a user wants done, as opposed to managing the machine.

ASCII — a character encoding assigning the values 0–127 to English letters, digits, punctuation, and control codes.

assembler — a program translating assembly language into machine language.

assembly language — a low-level language with one short mnemonic per machine instruction, specific to a processor family.

binary — base two, using the digits 0 and 1.

bit — the smallest unit of data, holding one of two states.

bytecode — an intermediate form produced by a hybrid language's compiler and executed by a virtual machine.

byte — eight bits; the standard unit of storage; holds 256 distinct patterns.

code point — the number Unicode assigns to a character.

compiler — a program translating an entire source program into machine code before it runs.

decimal — base ten, using the digits 0 through 9.

encoding — a scheme for storing Unicode code points as bytes; UTF-8 is the most common.

floating-point — a representation of real numbers as a sign, mantissa, and exponent, offering wide range with limited precision.

hardware — the physical components of a computer.

hexadecimal — base sixteen, using the digits 0–9 and letters A–F; each digit corresponds to four bits.

high-level language — a language whose statements describe operations in terms closer to the problem than to the machine.

input — data entering the computer from outside.

interpreter — a program translating and executing source code line by line as it runs.

machine language — the numeric instruction codes a processor executes directly.

main memory — fast, volatile storage holding data and instructions in current use; also called RAM.

mantissa — the significant digits of a floating-point value.

non-volatile — retaining contents when power is removed.

octal — base eight; each digit corresponds to three bits.

output — data leaving the computer to the outside world.

paradigm — a style of organizing a program, such as procedural, object-oriented, or functional.

processor — the component that fetches, decodes, and executes instructions; also called the CPU.

program — a sequence of instructions telling a computer how to accomplish a task.

software — the instructions a computer carries out.

source code — the text of a program as written by a programmer.

storage — non-volatile, slower memory holding data that must survive power loss.

system software — software managing the computer itself, such as an operating system.

two's complement — the standard representation of signed integers, in which the leftmost bit carries a negative place value.

Unicode — a standard assigning a code point to every character in every writing system.

UTF-8 — a Unicode encoding storing each character in one to four bytes, compatible with ASCII.

virtual machine — a program executing bytecode, allowing hybrid-language programs to run on many platforms.

volatile — losing contents when power is removed.

word — the amount of data a processor handles most naturally in one operation; commonly 64 bits.

Next: In Chapter 2 you will meet the tools that turn source code into a running program, trace exactly what happens between typing `g++` and seeing output, and write and run your first C++ program — Grade Calculator v0.1.

Chapter 2 — From Source Code to Running Program

Learning Objectives

When you finish this chapter you will be able to:

- Identify every part of a simple C++ program and explain what each one does. *(SLO 1.2)*
- Describe the four phases of program translation — preprocessing, compilation, assembly, and linking — and name the input and output of each. *(SLO 2.9)*
- Explain the difference between an error and a warning, and read a compiler message to locate a mistake in your source. *(SLO 1.2)*
- Edit, compile, and run a C++ program in the StudySite editor or GitHub Codespaces. *(SLO 1.2)*
- Apply the standard program template and the book's formatting conventions to code you write yourself. *(SLO 2.4)*
- Build and run Grade Calculator v0.1.

2.1 Anatomy of a Simple C++ Program

Chapter 1 ended with a gap: you think in tasks, the processor executes numeric instruction codes, and something has to bridge the two. This chapter is that bridge. By the end of it you will have written a program, watched four separate tools transform it, and run the result.

Start with the smallest complete C++ program that does something visible.

```
#include <iostream>

int main() {
    std::cout << "Grade Calculator\n";
    return 0;
}
```

Five lines, and each one is worth understanding now, because every program in this book — including the thousand-line version you will finish in Chapter 24 — has this same skeleton underneath it.

Figure 2.1 — The parts of a simple C++ program.

Description of Figure 2.1. The program is shown with a label pointing to each part. The line `#include <iostream>` is labelled as a preprocessor directive that brings in the input and output library. The line `int main() {` is labelled as the function where execution begins. The line `std::cout << "Hi\n";` is labelled as the statement

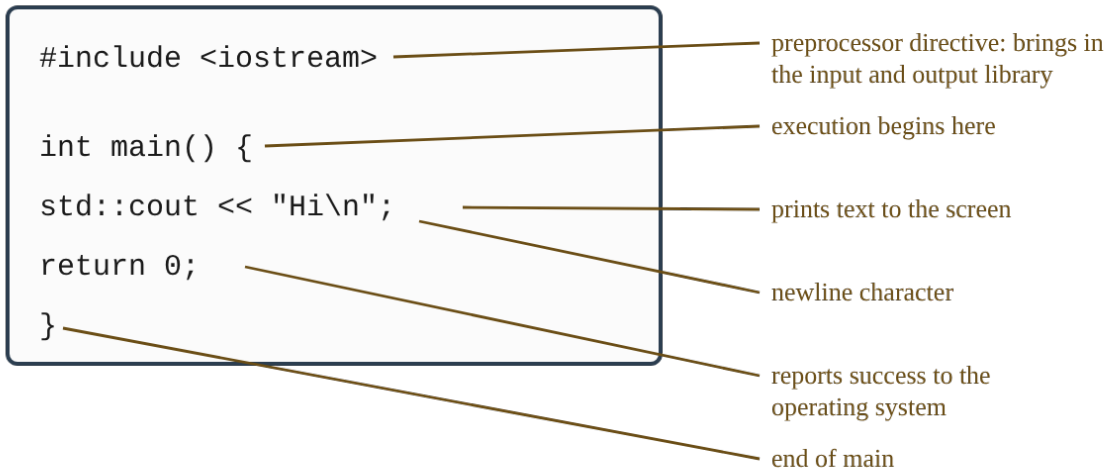


Figure 4: A four-line C++ program with labels pointing to each part, identifying the include directive, the main function, the output statement, the newline character, the return statement, and the closing brace.

that prints text, with a separate label identifying `\n` as a newline character. The line `return 0;` is labelled as reporting success to the operating system. The final `}` is labelled as the end of `main`.

`#include <iostream>`

A line beginning with `#` is a **preprocessor directive** — an instruction to a tool that runs *before* the compiler. `#include` means "insert the contents of this file here."

`iostream` is a **header** from the standard library. It declares the facilities for input and output, including `std::cout`. Without this line, the compiler has never heard of `std::cout` and will say so.

Note the angle brackets. `#include <iostream>` looks for a header the compiler supplies. `#include "gradelib.h"`, which you will write in Chapter 10, looks for a header you supply, starting in your own directory.

`int main()`

Every C++ program has exactly one function named `main`. It is where execution begins — not the first line of the file, but the first statement inside `main`. The operating system starts your program by calling it.

`int` says that `main` gives back a whole number when it finishes. The empty parentheses say it takes no information in. Chapter 9 covers functions properly; for now, treat `int main()` as the fixed phrase that marks where your program starts.

The braces

`{` and `}` mark the beginning and end of a **block** — a group of statements treated as a unit. The block after `int main()` is the body of the function: everything the program does.

Braces always come in pairs. A missing one produces a confusing error message pointing at a line far from the actual mistake, which is exactly why the indentation rules in Section 2.8 matter more than they look like they

do.

```
std::cout << "Grade Calculator\n";
```

This prints. It has four parts worth separating.

`std::cout` is the standard output stream — the screen. The `std::` prefix says `cout` comes from the standard library's namespace, `std`. Chapter 23 explains namespaces properly. For now, read `std::` as "the standard library's."

`<<` is the insertion operator. It sends what is on its right into the stream on its left. The arrows point in the direction the data flows, which is a useful mnemonic and not a coincidence.

`"Grade Calculator\n"` is a **string literal** — text in double quotes. The `\n` inside it is a single character, the **newline**, which moves output to the next line. It looks like two characters in your source and is one character in the output. Without it, the next thing printed would run on directly after `Calculator`.

`;` ends the statement. C++ requires a semicolon at the end of each statement, and forgetting one is the single most common mistake beginners make. Section 2.6 shows exactly what the compiler says when you do.

```
return 0;
```

This ends `main` and hands the value 0 back to the operating system. By long convention, **0 means success** and any other value means something went wrong. You will rarely need anything but 0 until Chapter 24, where the release build reports a failure to save.

2.2 The Translation Pipeline

You now have source code. The processor cannot execute a single character of it. Four separate programs run, in order, to fix that.

This is the material for SLO 2.9, and it is worth understanding properly rather than memorizing, because it explains several error messages that are otherwise baffling — particularly the difference between an error the compiler reports and an error the linker reports.

Figure 2.2 — The four stages of translation.

Description of Figure 2.2. Source code in `main.cpp` enters stage 1, the preprocessor, which carries out `#include` and `#define` and produces a translation unit. The translation unit enters stage 2, the compiler, which checks syntax and types and produces assembly code. The assembly enters stage 3, the assembler, which produces machine code in an object file named `main.o`. The object file, together with already-built library code such as `iostream`, enters stage 4, the linker, which joins them into an executable file named `gradecalc`. Finally the loader copies the executable into memory and runs it. A note records that a single `g++` command performs stages 1 through 4 in order.

2.2.1 Preprocessing

The **preprocessor** performs simple text substitution. It handles every line beginning with `#`: `#include` is replaced by the entire contents of the named file, and `#define` substitutes text throughout.

It does not understand C++. It does not check whether your program makes sense. It moves text around.

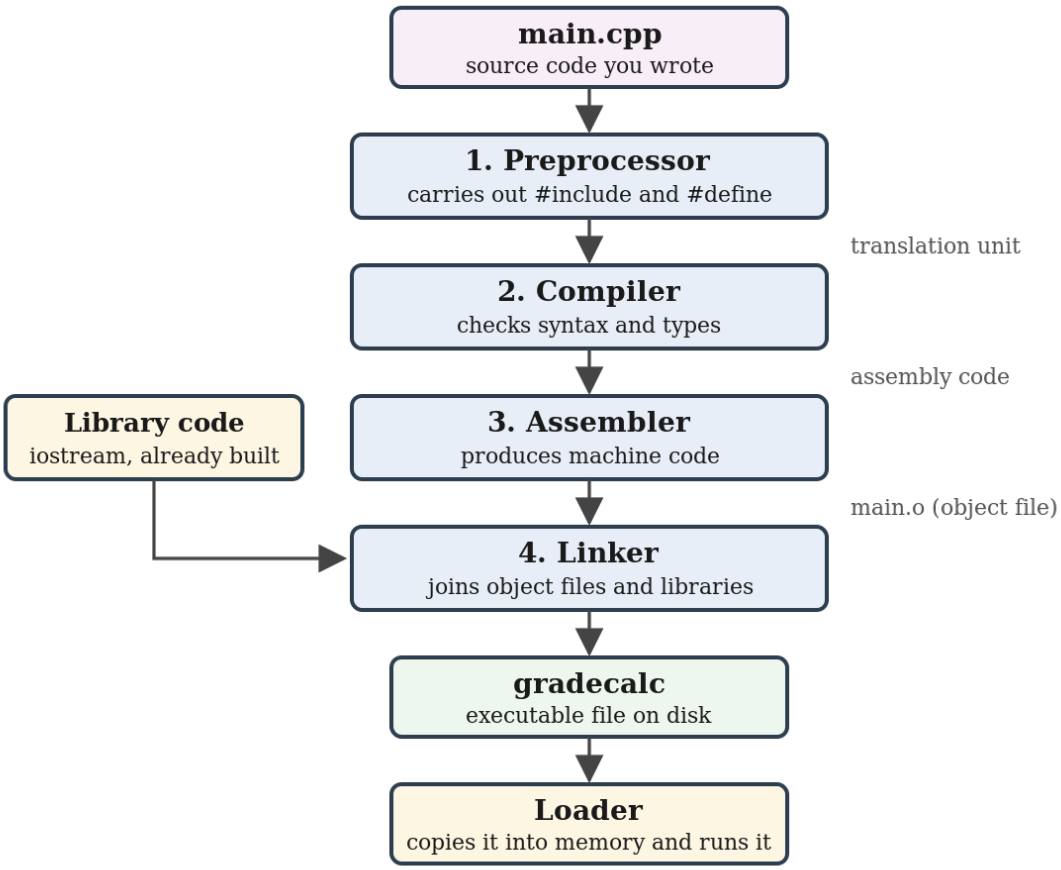


Figure 5: A vertical pipeline: source code enters the preprocessor, producing a translation unit; that enters the compiler, producing assembly code; that enters the assembler, producing an object file; the object file and library code enter the linker, producing an executable; the loader then copies the executable into memory to run it.

The result is a single expanded file called a **translation unit**, and its size is genuinely startling. Your five-line program, after preprocessing, becomes:

```
source lines:        6
after preprocessing: 32,192 lines
```

Over thirty-two thousand lines, nearly all of it the contents of `<iostream>` and everything `<iostream>` itself includes. You can see this yourself — Section 2.5 shows the command.

That number explains something practical. Compiling even a trivial C++ program takes a noticeable moment, because the compiler is not reading your five lines; it is reading thirty-two thousand.

2.2.2 Compilation to Assembly

The **compiler** is the stage that does the real work. It reads the translation unit and checks it — is the syntax valid, do the types agree, does every name refer to something that exists? This is where the great majority of your mistakes are caught, and where the error messages in Section 2.6 come from.

If the program passes, the compiler translates it into **assembly language** for your specific processor. Chapter 1 introduced assembly as one mnemonic per machine instruction; here is the beginning of `main` as the compiler wrote it on the machine used to prepare this book:

```
main:
.LFB1724:
    .cfi_startproc
    stp x29, x30, [sp, -16]!
```

You are not expected to read this, and you will never write it. Two things about it are worth noticing. Your five readable lines became roughly a hundred lines of assembly — that is the leverage a high-level language gives you. And this assembly is specific to one processor family; the same program compiled on a different kind of machine produces entirely different assembly. That is exactly the portability trade-off Chapter 1 described, now visible.

2.2.3 Assembly to Object Code

The **assembler** translates assembly language into actual machine code, producing an **object file** — `main.o` on most systems.

An object file contains real machine instructions, but it is not yet runnable. Your program refers to `std::cout`, which lives somewhere else entirely, and the object file records that reference as an unfilled blank: *"there is something called `std::cout` here, and I do not know where it is."*

2.2.4 Linking and Loading

The **linker** fills in those blanks. It gathers your object file together with the already-compiled library code — the real implementation of `std::cout` — and resolves every reference into a concrete address. The result is an **executable file** that can actually run.

This division explains an error message that confuses many beginners. If you misspell `std::cout`, the *compiler* objects, because it can see that no such name exists. But if you correctly declare a function and never write its body, the compiler is satisfied — the declaration promised the function exists — and the *linker* fails instead, saying it cannot find the definition. Same underlying mistake, two entirely different messages, from two different tools. Chapter 10 covers this properly when your program grows past one file.

Finally, when you run the executable, the operating system's **loader** copies it into main memory and directs the processor to its first instruction. That is the moment Chapter 1's diagram comes to life: your program is now data in memory, being fetched and executed.

The whole pipeline, measured

Here is what each stage produced for the five-line program:

Stage	Output	Size
Source you wrote	`main.cpp`	6 lines
After preprocessing	translation unit	32,192 lines
After compiling	assembly	101 lines
After assembling	`main.o`	2,808 bytes
After linking	executable	9,344 bytes

One command runs all four stages. You will type `g++` once and it will invoke the preprocessor, compiler, assembler, and linker in sequence. It only feels like one step. Knowing there are four is what lets you understand where a message came from when something goes wrong.

2.3 Development Tools

Four kinds of tool matter to you.

An **editor** is where you write source code. A programmer's editor understands the language: it colors syntax, matches braces, and indents automatically. The StudySite editor and VS Code both do this.

A **compiler** performs the translation just described. This book uses **g++**, the GNU C++ compiler, which is free, available on every platform, and standards-compliant.

A **debugger** lets you pause a running program and inspect it. You will not need one until Chapter 16, which teaches it properly, but it is worth knowing the category exists.

A **build tool** automates compiling projects with many files. You will not need one at all — even the largest version of the Grade Calculator builds with a single command — but Chapter 24 shows what one looks like.

An **IDE**, or integrated development environment, bundles all four into one application. VS Code, which is what GitHub Codespaces runs, is the example you will meet.

2.4 Setting Up Your Environment

You have two environments available, and both work for every program in this book.

The StudySite editor

Nothing to install. Open the editor, type your program, and run it. This is the fastest way to work through the Try It Yourself examples and is where most of your reading-and-experimenting will happen.

GitHub Codespaces

Codespaces gives you a full Linux machine in your browser, running VS Code. It takes a moment to start the first time, and in exchange you get the complete toolchain — including the debugger you will need from Chapter 16 onward, and a real terminal.

To confirm your Codespace is ready, open a terminal and type:

```
g++ --version
```

You should see a version number. Anything from g++ 9 onward supports everything in this book.

What this book assumes

Every program here is written to run identically in both environments. That means:

- **Standard library only.** No external libraries to install.
- **No project files or build systems.** One command compiles any program in this book.
- **Plain text input and output.** No graphics, no color codes, no cursor control.
- **C++17.** The compile command below selects it explicitly.

Appendix A gives step-by-step setup for both environments, plus a local VS Code installation if you prefer to work offline.

2.5 Editing, Compiling, and Running Your First Program

Three steps. Do them now rather than reading about them.

Step 1 — Edit

Create a file named `main.cpp` and type in the program from Section 2.1. **Type it; do not paste it.** Typing it is how you find out that the compiler cares about the semicolon and the quotation marks, and finding that out deliberately now is much better than discovering it accidentally later.

Step 2 — Compile

In a terminal, in the directory containing your file:

```
g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc
```

Every part of that command matters:

Part	Meaning
`g++`	the compiler
`-std=c++17`	use the C++17 standard, which this book targets
`-Wall`	report all common warnings
`-Wextra`	report additional warnings beyond `-Wall`
`main.cpp`	the source file to compile
`-o gradecalc`	name the resulting executable `gradecalc`

If it worked, nothing is printed. Silence means success. This surprises people; a compiler that has nothing to complain about says nothing at all.

The two warning flags are not optional in this book. Appendix D Section D.10 states the rule: **your code must compile with no warnings**. A warning is the compiler telling you it understood your program differently than you probably meant, and warnings you ignore become bugs you hunt.

Step 3 — Run

```
./gradecalc
```

The `./` says "in the current directory." Your output:

```
Grade Calculator
```

That is a complete C++ program, translated by four tools and executed by your processor.

Seeing the stages for yourself

Because the pipeline is real rather than a metaphor, you can stop it at any stage:

Command	Stops after	Produces
`g++ -E main.cpp -o main.ii`	preprocessing	the expanded translation unit
`g++ -S main.cpp -o main.s`	compiling	assembly code
`g++ -c main.cpp -o main.o`	assembling	an object file
`g++ main.cpp -o gradecalc`	linking	the executable

Run the first one and open `main.ii`. Scroll to the bottom and you will find your five lines, sitting under thirty-two thousand lines of standard library. It is worth doing once.

2.6 Reading Compiler Messages

Compiler messages have a reputation for being cryptic. They are actually quite structured, and learning to read them now will save you more time than any other single skill in this chapter.

Every message has the same shape:

```
file:line:column: severity: description
```

Errors and warnings are different

An **error** means the compiler could not produce a program. Nothing was built. You must fix it.

A **warning** means the compiler built your program but noticed something suspicious. The executable exists and will run. In this book you fix warnings anyway, because they are usually right.

Four real messages

Each of the following is genuine output from g++, produced by a mistake you are likely to make this week.

A missing semicolon. Remove the `;` from the end of the `std::cout` line:

```
e1.cpp: In function 'int main()':
e1.cpp:4:38: error: expected ';' before 'return'
    4 |     std::cout << "Grade Calculator\n"
      |                                      ^
```

```
      |                                       ;
    5 |     return 0;
      |     ~~~~~~
```

Read it carefully, because it teaches the most important habit in this section. The message says line 4, and the caret points past the end of line 4 — but it also *shows* you line 5 and mentions `return`. The compiler noticed the problem when it reached `return`, but the mistake is at the end of the previous line.

When a message points at a line that looks fine, check the line above it. That one rule resolves a large fraction of the confusing messages you will see.

A missing `#include`. Delete the first line:

```
e2.cpp:2:10: error: 'cout' is not a member of 'std'
    2 |     std::cout << "Grade Calculator\n";
      |          ^~~~
e2.cpp:1:1: note: 'std::cout' is defined in header '<iostream>';
             did you forget to '#include <iostream>'?
```

Modern compilers are genuinely helpful. The `note:` line diagnosed the problem and told you the fix. Always read past the first line of a message.

A misspelling. Write `std::cot` instead of `std::cout`:

```
e3.cpp:4:10: error: 'cot' is not a member of 'std'; did you mean 'oct'?
    4 |     std::cot << "Grade Calculator\n";
      |          ^~~
      |          oct
```

The compiler guessed `oct`, which is wrong — it wanted `cout`. Suggestions are suggestions, not diagnoses. Read them, then decide.

A warning, not an error. Declare a variable and never use it:

```
w1.cpp:4:9: warning: unused variable 'total' [-Wunused-variable]
    4 |     int total = 5;
      |         ^~~~~
```

The program still compiled and the executable was produced. But the compiler is pointing at something real: either you meant to use `total` and forgot, or you do not need it. Both are worth knowing. The bracketed `[-Wunused-variable]` names the specific warning, which is useful when you want to look one up.

How to work through a screen of errors

Fix the first error, then recompile. Do not read the whole list. One mistake — a missing brace especially — can produce dozens of errors, and the later ones are usually nonsense caused by the first. Fixing one line often clears twenty messages.

2.7 A Template for Simple C++ Programs

Every single-file program in the first eight chapters has this shape:

```
// Program name - brief description of what it does
// Run: click Run in StudySite and use the embedded Terminal.

#include <iostream>
```

```
int main() {

    // your statements go here

    return 0;
}
```

Start from this. As you learn more, the parts you add have predictable homes: more `#include` lines join the first, named constants go after the includes, and functions go between the includes and `main` from Chapter 9 onward.

2.8 Source Formatting and Comments from Day One

The compiler does not care about your formatting. It would accept your entire program on one line. People care, including the version of you that reads this code next month, and Chapter 13 will ask you to bring an entire codebase into conformance with a written standard.

That standard is Appendix D. Four of its rules apply to everything you write from today.

Indent with four spaces. Everything inside a pair of braces is indented one level deeper. Never use tab characters — they render at different widths in different editors, so a file that looks correct to you looks broken to everyone else.

Put the opening brace on the same line as the statement that introduces it, and the closing brace on its own line, lined up with the start of that statement.

Use blank lines to separate logical groups. One blank line, never two.

Comment why, not what. A comment restating the code is noise:

```
// Do not write this.
return 0;    // return zero
```

A comment explaining a decision earns its place:

```
// Write this.
// 0 tells the operating system the program succeeded. Chapter 24 returns a
// non-zero value here when the gradebook could not be saved.
return 0;
```

C++ has two comment forms. `//` comments run to the end of the line. `/* */` comments span from `/*` to `*/`, possibly across many lines. This book uses `//` for ordinary comments and reserves `/** */` for the documentation comments introduced in Chapter 9.

Formatting is not decoration. It is how you keep your attention on what a program does instead of on what it looks like, and it is the thing that makes a mismatched brace obvious at a glance instead of invisible.

Common Errors and Warnings

The five you are most likely to hit this week.

What you see	What it means	Fix
`error: expected ';' before ...`	A statement is missing its semicolon	Add `;` at the end of the **previous** line
`error: 'cout' is not a member of 'std'`	The compiler has not seen `<iostream>`	Add `#include <iostream>` at the top
`error: 'X' was not declared in this scope`	A name is misspelled or undeclared	Check spelling and capitalization; C++ is case-sensitive
`error: expected '}' at end of input`	A brace is unmatched	Check indentation — a misaligned block reveals the missing brace
`warning: unused variable 'x'`	You declared something you never used	Use it, or remove it

Two habits worth forming immediately:

Fix the first error, then recompile. Later errors are frequently caused by earlier ones.

Read the whole message, including `note:` lines. The compiler often tells you exactly what to do, one line below where you stopped reading.

Design Notes

Silence is success. A compiler that prints nothing has accepted your program. Get comfortable with this; it is the normal case and it is easy to misread as "nothing happened."

Compile early and often. Write a few lines, compile, run. Do not write fifty lines before the first compile. When something breaks after three lines, you know where the mistake is. When it breaks after fifty, you have fifty places to look. This habit matters more as programs grow, and it is the reason Chapter 9's refactor is done one function at a time with a recompile after each.

The build command is part of your program. Put it in a comment at the top of every file, as the template does. Six months from now you will not remember which flags a program needed, and neither will anyone else.

Grade Calculator v0.1 — Hello, Gradebook

Your first running version.

What v0.1 does

It prints the application banner and the course name, then exits cleanly. That is all — and it is a complete program, which is the point. Every version from here on will do more, and every version will still compile, run, and finish properly.

The program

```
// Grade Calculator v0.1 - Chapter 2
// Prints the application banner. Confirms the toolchain works end to end.
// Run: click Run in StudySite and use the embedded Terminal.

#include <iostream>

int main() {
```

```
    std::cout << "==================================\n";
    std::cout << "        GRADE CALCULATOR          \n";
    std::cout << "        Version 0.1               \n";
    std::cout << "==================================\n";
    std::cout << "Course: Programming Fundamentals\n";
    return 0;
}
```

Expected output

```
==================================
        GRADE CALCULATOR
        Version 0.1
==================================
Course: Programming Fundamentals
```

What to notice

Five separate `std::cout` statements, each ending in `\n`. You could combine them; Chapter 3 shows how. Kept separate, each line of output corresponds to one line of source, which makes the relationship obvious.

The file header comment records how the program is run. That is the Appendix D rule from Section 2.8, applied from the first version. In StudySite the toolchain is provided for you, so the header records the **Run** step rather than a local build command.

Nothing here computes a grade. v0.1 exists to prove your toolchain works end to end — editor, compiler, executable, output. Everything after this is built on the confidence that when something breaks, it is your program and not your setup.

Your StudySite Lab — Hello, Gradebook

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v0.1
- **Starting point**: Your Chapter 1 repository, containing `README.md` and `requirements.md` and no C++ code yet.

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

This is the first code checkpoint. The program you write here does one thing — it prints the Grade Calculator banner and exits — and that is the point. Before any grading logic exists, you confirm that the whole toolchain works end to end: you can create a source file, the compiler accepts it, the program runs, and you can commit the result to GitHub. Every later chapter adds to this same file. By the end you will have:

1. A new file `main.cpp` containing a complete, minimal C++ program: a comment header, `#include <iostream>`, an `int main()` function, and `return 0;`.
2. A five-line banner printed with `std::cout`, naming the application, its version, and the course.
3. A program that compiles with no errors and no warnings, and exits normally.

Files in this checkpoint

File	What you do to it
`main.cpp`	Create (Steps 1-3)
`README.md`, `requirements.md`	**No change** — keep them in the repository

When you are finished your project has **one** `.cpp` file.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `README.md` and `requirements.md`. Confirm the editor shows the expected file paths before editing.
4. Create the new file in StudySite's internal editor, finish the steps in order, then Run.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Create `main.cpp` with the program skeleton

> **How to create a file**: in the StudySite editor, create a new file with the exact name shown (spelling and lowercase matter, because `#include` lines refer to these names). Keep each new file open after you create it.

Create a new file named `main.cpp` and enter this code:

```
// Grade Calculator v0.1 - Chapter 2
// Prints the application banner. Confirms the toolchain works end to end.
// Build: g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc

#include <iostream>

int main() {
    return 0;
}
```

Why: Every C++ program needs exactly one `main` function — that is where execution begins. `#include <iostream>` brings in the output facilities you need in the next step; without it the compiler does not know what `std::cout` is. `return 0;` tells the operating system the program finished normally. The comment header at the top records what this version of the program does and how to build it, and you will update it in every chapter.

> ▸ **Run checkpoint**: Click **Run**. The program should compile and exit immediately without printing anything. An empty result is the correct result here — it proves the toolchain works before any output exists to distract you. If you get a compiler error, re-check the braces and the semicolon after `return 0`.

Step 2 — Print the banner

In `main.cpp`, find this line:

```
int main() {
```

Directly below it, add:

```
    std::cout << "==================================\n";
    std::cout << "        GRADE CALCULATOR          \n";
    std::cout << "        Version 0.1               \n";
    std::cout << "==================================\n";
```

Why: `std::cout` is the standard output stream, and `<<` sends each value to it in turn. The `\n` at the end of each string is the newline character; it moves output to the next line. Without it every line would run together. Note that the banner lines are printed in the order you write them — output is sequential, top to bottom, which is the single most important thing to internalize about how a program executes.

Step 3 — Add the course line

In `main.cpp`, find these two lines — the version line and the closing rule below it:

```
    std::cout << "        Version 0.1               \n";
    std::cout << "==================================\n";
```

Directly below it, add:

```
    std::cout << "Course: Programming Fundamentals\n";
```

Why: This line sits after the closing rule rather than inside the banner box, so the box stays a fixed five lines wide while the text below it can grow in later chapters. Keeping decoration and content separate is a habit worth starting now.

> ▸ **Run checkpoint**: Click **Run**. The program should print the full banner followed by the course line, then exit. Compare your output character by character with the expected output below — the = rules are 35 characters wide.

Expected output

```
==================================
        GRADE CALCULATOR
        Version 0.1
==================================
Course: Programming Fundamentals
```

The two centred lines are padded with spaces inside the string literal, so they sit under the middle of the rules. If your banner looks ragged, count the spaces in the find boxes above.

Check your work

Then confirm:

- ☐ `main.cpp` compiles with no errors and **no warnings**.
- ☐ The program prints exactly five banner lines and one course line.
- ☐ The two rule lines are the same length as each other.
- ☐ The program exits normally, with no input required.
- ☐ `README.md` and `requirements.md` are still in the repository, unchanged.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`, `README.md`, and `requirements.md`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 2 Grade Calculator v0.1**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 2 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

Each of these is a complete program. Type it, build it with the standard command, run it, and compare with the expected output before reading on.

1. Multiple lines from one statement

```cpp
#include <iostream>

int main() {
    std::cout << "Line one\nLine two\nLine three\n";
    return 0;
}
```

Expected output:

```
Line one
Line two
Line three
```

Try: Remove the middle `\n`. Predict the output before you rebuild, then check.

2. What `\n` actually does

```cpp
#include <iostream>

int main() {
    std::cout << "no newline here";
    std::cout << "so this continues the same line\n";
    return 0;
}
```

Expected output:

```
no newline hereso this continues the same line
```

Try: Add a space so the words separate properly. There are two places you could put it — does it matter which you choose?

3. Chaining insertions

```cpp
#include <iostream>

int main() {
    std::cout << "Grade" << " " << "Calculator" << "\n";
    return 0;
}
```

Expected output:

```
Grade Calculator
```

Try: Add `<< "v0.1"` before the `"\n"`. Then rewrite the whole thing as a single string literal and confirm the output is identical.

4. The return value

```cpp
#include <iostream>

int main() {
    std::cout << "Finished\n";
    return 0;
}
```

Build and run it, then immediately type:

```
echo $?
```

Expected output: `0`

Try: Change `return 0;` to `return 3;`, rebuild, rerun, and check `echo $?` again. This is the value Chapter 24 uses to report that saving failed.

5. Predict the errors

Do not build these first. Write down what the compiler will say, then build and compare.

```cpp
#include <iostream>

int main() {
    std::cout << "Missing something\n"
    return 0;
}
```

```cpp
int main() {
    std::cout << "Also missing something\n";
    return 0;
}
```

```cpp
#include <iostream>

int main() {
    std::Cout << "Capitalization matters\n";
    return 0;
}
```

For the third: the message will differ from the one in Section 2.6. Why? What is different about this mistake?

6. Watch the preprocessor work

Take any program above and run:

```
g++ -std=c++17 -E main.cpp -o main.ii
```

Open `main.ii`.

- How many lines is it? (`wc -l main.ii`)
- Where in the file is the code you wrote?
- What is all the rest of it?

Then try the same on a program with **no** `#include` line at all. How many lines now? What does that tell you about where the thirty-two thousand lines came from?

Summary

- Every C++ program has one `main` function, where execution begins, and returns `0` to report success.
- `#include` is a **preprocessor directive** that inserts the contents of a header. `<angle brackets>` find library headers; `"quotes"` find your own.
- `std::cout` is the standard output stream. `<<` sends data into it. `\n` is a single newline character. Every statement ends with `;`.
- Translation happens in **four stages**: the **preprocessor** expands directives into a translation unit; the **compiler** checks the program and emits assembly; the **assembler** produces an object file; the **linker** joins object files and libraries into an executable. The **loader** then copies it into memory to run.
- One `g++` command performs all four stages. Knowing they are separate is what lets you tell a compiler error from a linker error.
- Compile with `g++ -std=c++17 -Wall -Wextra main.cpp -o name` and run with `./name`. **Silence means success.**
- An **error** means nothing was built; a **warning** means it built but something looks wrong. Fix both.

- Compiler messages have the form `file:line:column: severity: description`. **When a message points at a line that looks fine, check the line above**. Fix the first error, then recompile.
- Indent four spaces, brace on the same line, comment *why* rather than *what*. Appendix D is the full standard; Chapter 13 will hold you to it.

Key Terms

assembler — the tool that translates assembly language into machine code in an object file.

block — a group of statements enclosed in braces and treated as a unit.

build tool — a program that automates compiling a project of many files.

comment — text in source code ignored by the compiler, written for human readers.

compiler — the tool that checks a translation unit and translates it into assembly language.

debugger — a tool for pausing a running program and inspecting its state.

editor — the program in which source code is written.

error — a problem that prevents the compiler from producing a program.

executable — a file containing machine code that the operating system can run.

header — a file, brought in with `#include`, declaring facilities defined elsewhere.

IDE — an integrated development environment, bundling editor, compiler, and debugger.

insertion operator — `<<`, which sends data into an output stream.

linker — the tool that joins object files and library code into an executable.

loader — the operating system component that copies an executable into memory and starts it.

main — the function where execution of a C++ program begins.

newline — the character `\n`, which moves output to the next line.

object file — machine code produced from one source file, not yet runnable.

preprocessor — the tool that carries out `#include`, `#define`, and other directives before compilation.

preprocessor directive — a line beginning with `#`, addressed to the preprocessor.

standard library — the collection of facilities every C++ implementation provides.

statement — a single instruction, ending in a semicolon.

std — the namespace containing the standard library.

stream — a destination or source for sequential data, such as `std::cout`.

string literal — text enclosed in double quotes.

translation unit — the single expanded file the preprocessor produces from one source file.

warning — a report of something suspicious that did not prevent compilation.

Next: Chapter 3 introduces variables and types — how to store a student's name, an assignment name, and a score, and how to choose the right type for each. Grade Calculator v0.2 will read them from you and print them back.

Chapter 3 — Values, Variables, and Data Types

Learning Objectives

When you finish this chapter you will be able to:

- Declare and initialize variables of C++'s fundamental types. *(SLO 1.3)*
- Explain what a type determines about a variable, and connect that to the bit patterns of Chapter 1. *(SLO 1.1, 1.3)*
- Choose an appropriate type for a given quantity and justify the choice. *(SLO 1.3)*
- State the range of each integer type and predict what happens when a value exceeds it. *(SLO 1.1)*
- Explain the difference between an integer and a floating-point type, and when each is appropriate. *(SLO 1.1, 1.3)*
- Use `const` to name values that must not change. *(SLO 1.3)*
- Read text and numbers from the user with `std::cin` and `std::getline`. *(SLO 1.3)*
- Build Grade Calculator v0.2.

3.1 Literal Values

Chapter 2's program printed a fixed piece of text. To calculate anything, a program needs to hold values and work with them.

The simplest values are **literals** — values written directly in your source code:

```
42          // an integer literal
84.5        // a floating-point literal
'A'         // a character literal, in single quotes
"Midterm"   // a string literal, in double quotes
true        // a boolean literal
```

Note the quotation marks carefully, because the difference is real and will bite you. **Single quotes make a character; double quotes make a string.** `'A'` is one character. `"A"` is a string that happens to contain one character. They are different types, they are stored differently, and they are not interchangeable.

Literals alone are not enough. A program that only ever prints `84.5` is not a grade calculator. It needs to hold a value that changes.

3.2 Variables and Assignment

A **variable** is a named location in memory that holds a value of a particular type.

You bring one into existence with a **declaration**:

```
double pointsEarned = 84.5;
```

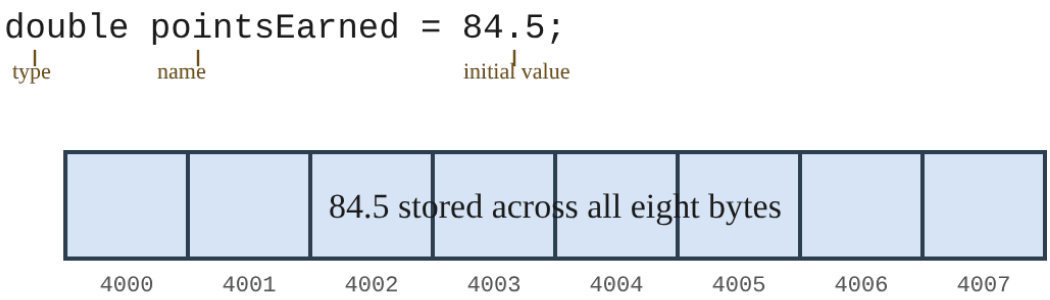


Figure 6: The declaration double pointsEarned equals 84.5 shown above eight shaded memory cells at addresses 4000 through 4007, with labels identifying the type, the name, and the initial value.

Figure 3.1 — A variable declaration and what it reserves.

Description of Figure 3.1. The declaration `double pointsEarned = 84.5;` is shown with three labels: `double` is the type, `pointsEarned` is the name, and `84.5` is the initial value. Below, eight shaded memory cells at addresses 4000 through 4007 hold the value. Notes explain that the type decides how many bytes are reserved and how their bits are read, that the name is how the program refers to this location, and that the same eight bytes read as a different type would mean something else entirely.

That last note is Chapter 1 Section 1.7 restated. A bit pattern carries no information about its own meaning. **The type is the meaning.** This is why C++ makes you state a type for every variable, and it is the single idea that makes the rest of this chapter cohere.

Assignment

Once declared, a variable's value can change:

```
double pointsEarned = 84.5;
pointsEarned = 91.0;         // the old value is gone
pointsEarned = pointsEarned + 5.0;   // now 96.0
```

The `=` sign is **assignment**, not mathematical equality. Read it as "gets" or "becomes." The line `pointsEarned = pointsEarned + 5.0;` is nonsense as an equation and perfectly sensible as an instruction: work out the right-hand side, then store the result in the variable on the left.

Always initialize

A variable declared without a value holds whatever happened to be in that memory:

```
double total;              // no. holds garbage.
double total = 0.0;        // yes.
```

An uninitialized variable produces bugs that behave differently each time you run the program, which makes them among the hardest to find. Appendix D Section D.6 states the rule without exception: **initialize every variable where you declare it**.

3.3 Identifiers and Naming

An **identifier** is a name you choose. C++ requires that it:

- contain only letters, digits, and underscores
- not begin with a digit
- not be a reserved keyword such as `int`, `return`, or `class`

C++ is **case-sensitive**: `total`, `Total`, and `TOTAL` are three different names. This is the source of a common early error, and you saw the compiler's response to it in Chapter 2.

Beyond the rules, this book follows the conventions in Appendix D Section D.2:

Kind	Convention	Example
Variable	`camelCase`	`pointsEarned`
Constant	`SCREAMING_SNAKE_CASE`	`A_CUTOFF`

Choose names that say what the value *means*. `pointsEarned` is better than `pe`, `x`, or `dblPoints`. The compiler already knows the type; the reader needs the meaning.

3.4 Integer Types

An **integer** type holds a whole number — no fractional part.

The everyday choice is `int`. On the machines this book targets it occupies 4 bytes, which by Chapter 1's rule means 2^{32} = 4,294,967,296 distinct patterns, split between negative and positive:

```
int assignmentCount = 12;
int studentId = 1001;
```

C++ provides several integer types of different sizes. These are the real measurements from the machine used to prepare this book:

Type	Bytes	Range
`short`	2	−32,768 to 32,767
`int`	4	−2,147,483,648 to 2,147,483,647
`long long`	8	about -9.2×10^{18} to 9.2×10^{18}
`unsigned int`	4	0 to 4,294,967,295

You can print these yourself; Try It Yourself example 1 shows how.

Notice that `unsigned int` uses all 32 bits for magnitude, since it gives up negative values. Same number of patterns, different split — exactly the two's complement trade-off from Chapter 1 Section 1.5.

Overflow

What happens when a value exceeds its type's range? Not an error. Not a warning at runtime. The value **wraps around**:

```
short small = 32767;      // the largest short
++small;                  // add one
```

```
short at max: 32767
after adding 1: -32768
```

Adding 1 to the largest `short` produces the *smallest* one. This is not a malfunction; it is exactly what two's complement arithmetic does when it runs out of bits, and it is the direct consequence of Chapter 1's counting rule.

The practical defense is simple: **use `int` unless you have a specific reason not to**. Its range is large enough for anything in this book. Reach for `long long` when you genuinely need bigger numbers, and reach for `short` almost never — saving two bytes is not worth the risk.

3.5 Floating-Point Types

A **floating-point** type holds a number with a fractional part.

```
double pointsEarned = 84.5;
double percentage = 91.25;
```

Type	Bytes	Approximate decimal digits of precision
`float`	4	6
`double`	8	15

Use `double`. The name means "double precision," and its extra range and accuracy cost nothing you will notice. `float` exists for situations where memory is genuinely scarce, which will not arise in this book.

The precision problem, again

Chapter 1 Section 1.5 warned that many decimal fractions cannot be represented exactly in binary. That warning becomes concrete now that you can write the code:

```
double a = 0.1;
double b = 0.2;
// a + b is 0.30000000000000004, not 0.3
```

This is not a flaw in C++ or in your machine. One-tenth in binary is a repeating fraction, exactly as one-third is a repeating decimal. The stored value is the closest representable one, which is very close and not equal.

Two rules follow, and they are worth adopting now rather than after they cost you an evening:

Never compare floating-point values with `==`. Chapter 9 shows the correct technique.

Use an integer type when the quantity is genuinely whole. A count of assignments is an `int`. A number of students is an `int`. Points earned is a `double`, because partial credit exists.

3.6 Characters

A `char` holds a single character, in one byte:

```
char letterGrade = 'A';
char separator = ',';
char newline = '\n';
```

Chapter 1 Section 1.6 explained that characters are stored as numbers under the ASCII convention. In C++ that is not a metaphor — a `char` *is* a small integer, and you can see it:

```
char c = 'A';
std::cout << "'A' as char: " << c
          << "  as int: " << static_cast<int>(c) << "\n";
std::cout << "'A' + 32 as char: " << static_cast<char>(c + 32) << "\n";
```

```
'A' as char: A  as int: 65
'A' + 32 as char: a
```

There it is: `'A'` is 65, and adding 32 gives `'a'`, because Chapter 1's ASCII table put uppercase and lowercase exactly 32 apart. `static_cast<int>(...)` is a **cast** — an explicit instruction to interpret the value as a different type. The same bits, read differently, which is Section 1.7's lesson made executable.

Escape sequences

Some characters cannot be typed directly, so they are written with a backslash:

Sequence	Meaning
`\n`	newline
`\t`	tab
`\\`	a literal backslash
`\'`	a single quote
`\"`	a double quote

Each is **one** character, despite taking two keystrokes.

Text: `std::string`

A `char` holds one character. For text, use `std::string`, which requires including `<string>`:

```
#include <string>

std::string studentName = "Ada Lovelace";
std::string assignmentName = "Midterm Exam";
```

A `std::string` holds as many characters as you need and grows automatically. Chapter 12 covers it properly. For now you need only declare one, assign to it, and print it.

3.7 Boolean Values

A `bool` holds one of exactly two values, `true` or `false`:

```
bool capAt100 = true;
bool dropLowest = false;
```

Booleans are the foundation of every decision your program makes, and Chapter 6 is built on them. Note that although a `bool` needs only one bit of information, it occupies a whole byte — memory is addressed in bytes, so a byte is the smallest thing that can have an address.

3.8 Constants

Some values must not change. Mark them `const`:

```
const double A_CUTOFF = 90.0;
const int MAX_STUDENTS = 40;
```

Attempting to assign to a `const` is a compile-time error — the compiler stops you at your desk rather than letting a bug reach a user.

`const` earns its place for two reasons. It **prevents accidental modification**, and more importantly it **names a value**. Compare:

```
if (percentage >= 90.0) { ... }        // 90.0 means what, exactly?
if (percentage >= A_CUTOFF) { ... }    // now it is obvious
```

Appendix D Section D.6 calls unnamed values *magic numbers* and asks you to avoid them. A named constant also means a policy change is one edit rather than a search through the whole program.

3.9 Enumerated Types

When a value should be one of a small fixed set, an **enumerated type** says so:

```
enum class Category { Exam, Homework, Participation };

Category kind = Category::Exam;
```

This is better than using an `int` where 0 means exam and 1 means homework, because the compiler will reject `Category::Quiz` if you never defined it, whereas it would happily accept the integer 7.

You will use enumerations lightly until Chapter 20, where assignment categories become central. `enum class` is the modern form; older code uses a plain `enum` that is more error-prone.

3.10 Type Inference with `auto`

If the compiler can work out a type from the initial value, `auto` lets it:

```
auto count = 12;          // int
auto points = 84.5;       // double
auto letter = 'A';        // char
```

`auto` is genuinely useful when a type is long and adds nothing — you will be grateful for it in Chapter 23. It is a poor choice when the explicit type carries meaning a reader needs:

```
auto total = 0;      // is this an int or a double? It is an int, and that
                     // may not be what you meant. Say what you mean.
double total = 0.0;  // no ambiguity
```

Appendix D Section D.6 states the rule: use `auto` where the type is obvious from the right-hand side and the name still carries the meaning. Until Chapter 23, write types out.

3.11 Choosing the Right Type

This is a design decision, not a lookup. For each quantity, ask what values it can take.

Quantity	Type	Why
Points earned	`double`	Partial credit exists — 8.5 out of 10
Points possible	`double`	Same reason
Bonus points	`double`	Same reason
Number of assignments	`int`	You cannot have 2.5 assignments
Student ID	`int`	An identifier, never used in arithmetic
Letter grade	`char`	Exactly one character
Student name	`std::string`	Text of any length
Assignment name	`std::string`	Text of any length
Whether to cap at 100	`bool`	Yes or no
The A cutoff	`const double`	A fixed policy value with a name

Two of these repay a second look.

Student ID is `int`, not `double`. IDs are counted and compared, never averaged. Using a floating-point type would invite meaningless operations and expose you to the precision problems of Section 3.5 for no benefit.

A name cannot be a `char`. A `char` is one character. `"Ada Lovelace"` is twelve. This sounds obvious and is a real early mistake.

3.12 Console Input

Chapter 2 covered output. Input is the mirror image, using `std::cin` and the **extraction operator** `>>`:

```
double pointsEarned = 0.0;
std::cout << "Points earned: ";
std::cin >> pointsEarned;
```

The arrows again point in the direction data flows — out of `std::cin`, into the variable.

Always **prompt before reading**. A program that waits silently looks broken.

Reading text with `std::getline`

`>>` stops at the first whitespace, which makes it useless for names:

```
std::string name;
std::cin >> name;      // "Ada Lovelace" gives you just "Ada"
```

For a whole line, use `std::getline`:

```
std::string name;
std::getline(std::cin, name);   // gets "Ada Lovelace"
```

Mixing >> and getline

Here is a trap that catches nearly everyone, and it is worth meeting deliberately.

When you type 42 and press Enter, two things go into the input buffer: the characters 42 and a newline. >> reads the number and **leaves the newline behind**. The next getline then finds that leftover newline, concludes the line is over, and hands you an empty string — without waiting for you to type anything.

```
int count = 0;
std::cin >> count;              // reads 42, leaves "\n" behind

std::string name;
std::getline(std::cin, name);   // finds "\n" immediately, returns ""
```

The fix is to discard the leftover newline:

```
#include <limits>

std::cin >> count;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::getline(std::cin, name);   // now works
```

That line says: discard characters until you have consumed a newline. It looks like an incantation, and for now you may treat it as one — but the reason is exactly as described, and knowing the reason means you will recognize the symptom when it appears in your own code.

The symptom to remember: a getline that returns an empty string without waiting for input means a stray newline was left behind by an earlier >>.

Common Errors and Warnings

What you see or observe	Cause	Fix
`error: 'x' was not declared in this scope`	Used before declaring, or misspelled	Declare it first; check capitalization
`error: expected ';'`	Missing semicolon	Add ; to the **previous** line
`error: assignment of read-only variable 'X'`	Assigned to a const	Remove const, or do not assign
`warning: unused variable 'x'`	Declared but never used	Use it or remove it
A getline returns nothing and does not wait	Leftover newline from an earlier >>	Add `std::cin.ignore(...)` — Section 3.12
`7 / 2` gives 3, not 3.5	Both operands are int, so this is integer division	Make one a double; Chapter 4 covers this fully
A large value becomes negative	Integer overflow	Use a wider type, or check the range
Two equal-looking doubles compare unequal	Floating-point imprecision	Never use == on floating-point values

---

Design Notes

Justify your types. For each variable in the Grade Calculator, you should be able to say in one sentence why it has the type it has. Section 3.11 is the model. This is a habit worth forming while programs are small.

Name constants immediately. The moment you write a number with meaning, give it a name. It costs one line and saves you from a program full of unexplained values.

Declare where you initialize. Do not declare a block of variables at the top of `main` and fill them in later. A variable that exists for twenty lines before it means anything is twenty lines of opportunity to misuse it.

Grade Calculator v0.2 — Naming and Storing an Assignment

What v0.2 does

Prompts for a student name, an assignment name, points earned, and points possible, then prints them back in a formatted summary. No calculation yet — Chapter 4 adds that.

The program

```cpp
// Grade Calculator v0.2 - Chapter 3
// Reads one named assignment and echoes a formatted summary.
// New this version: typed variables, std::getline for names.
// Run: click Run in StudySite and use the embedded Terminal.

#include <iostream>
#include <string>

int main() {
    std::cout << "=== GRADE CALCULATOR v0.2 ===\n\n";

    // Names may contain spaces, so getline is required here.
    std::string studentName;
    std::cout << "Student name: ";
    std::getline(std::cin, studentName);

    std::string assignmentName;
    std::cout << "Assignment name: ";
    std::getline(std::cin, assignmentName);

    // Points are double, not int: partial credit such as 8.5 is common.
    double pointsEarned = 0.0;
    std::cout << "Points earned: ";
    std::cin >> pointsEarned;

    double pointsPossible = 0.0;
    std::cout << "Points possible: ";
    std::cin >> pointsPossible;

    std::cout << "\n--- Summary ---\n";
    std::cout << "Student:    " << studentName << "\n";
    std::cout << "Assignment: " << assignmentName << "\n";
    std::cout << "Score:      " << pointsEarned << " / " << pointsPossible << "\n";
    return 0;
}
```

Expected output

With input `Ada Lovelace`, `Midterm Exam`, `84`, `100`:

```
=== GRADE CALCULATOR v0.2 ===

Student name: Assignment name: Points earned: Points possible:
--- Summary ---
Student:    Ada Lovelace
Assignment: Midterm Exam
Score:      84 / 100
```

What to notice

Both `getline` calls come before both `>>` calls. That ordering is deliberate. Reading the two names first, then the two numbers, means no `getline` ever follows a `>>` — so v0.2 sidesteps the trap in Section 3.12 by design rather than by patching it. Chapter 7 adds a loop that forces the two to interleave, and that is where `std::cin.ignore` appears in the project for real.

Every type is justified in a comment. Appendix D Section D.3 asks comments to explain *why*, and "points are `double` because partial credit exists" is exactly that.

No calculation yet. v0.2 reads and echoes. It is a complete program that does a small thing correctly, which is the pattern for every version in this book.

Your StudySite Lab — Store an Assignment

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v0.2
- **Starting point**: Your committed Chapter 2 v0.1 program (one file, prints a banner).

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

Chapter 2's program could only talk. This one listens. You replace the fixed banner with four prompts, store what the user types in **variables of the right type**, and echo it back as a formatted summary. The two ideas that matter here are choosing a type that fits the data, and knowing which input function to use for which kind of value. By the end you will have:

1. A shorter one-line banner, so the program's output is dominated by its work rather than its decoration.
2. Two `std::string` variables read with `std::getline`, because names contain spaces.
3. Two `double` variables read with `std::cin >>`, because scores need partial credit.
4. A `--- Summary ---` block that echoes all four values with aligned labels.

Files in this checkpoint

File	What you do to it
`main.cpp`	Edit (Steps 1-5)

When you are finished your project still has **one** `.cpp` file.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `main.cpp`. Confirm the editor shows the expected file paths before editing.
4. Continue the existing program in StudySite's internal editor, finish the steps in order, then Run.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Update the comment header and include `<string>`

In `main.cpp`, find the first two comment lines:

```
// Grade Calculator v0.1 - Chapter 2
// Prints the application banner. Confirms the toolchain works end to end.
```

Replace it with:

```
// Grade Calculator v0.2 - Chapter 3
// Reads one named assignment and echoes a formatted summary.
// New this version: typed variables, std::getline for names.
```

Now find the include line:

```
#include <iostream>
```

Directly below it, add:

```
#include <string>
```

Why: `std::string` is not built into the language the way `int` and `double` are — it comes from the standard library, and `<string>` is the header that declares it. Leaving the include out produces a confusing error about `string` not naming a type, which is one of the most common errors beginners hit. The comment header gains a "New this version" line so your file records its own history as the project grows.

Step 2 — Replace the banner with a single line

In `main.cpp`, find all five banner lines:

```
    std::cout << "====================================\n";
    std::cout << "        GRADE CALCULATOR            \n";
    std::cout << "        Version 0.1                 \n";
    std::cout << "====================================\n";
    std::cout << "Course: Programming Fundamentals\n";
```

Replace all five with this single line:

```
    std::cout << "=== GRADE CALCULATOR v0.2 ===\n\n";
```

Why: The five-line box was useful when printing was all the program did. Now that there is real output below it, a one-line header keeps the screen readable. Note the `\n\n` at the end — the second newline leaves a blank line between the header and the first prompt. Spacing is part of a usable interface, not an afterthought.

> ▸ **Run checkpoint**: Click **Run**. The program should print just the new one-line banner and exit. The prompts do not exist yet.

Step 3 — Read the two names with `std::getline`

In `main.cpp`, find the banner line:

```
    std::cout << "=== GRADE CALCULATOR v0.2 ===\n\n";
```

Directly below it, add:

```
    // Names may contain spaces, so getline is required here.
    std::string studentName;
    std::cout << "Student name: ";
    std::getline(std::cin, studentName);

    std::string assignmentName;
    std::cout << "Assignment name: ";
    std::getline(std::cin, assignmentName);
```

Why: `std::getline(std::cin, variable)` reads an entire line, spaces included, up to the point where the user pressed Enter. The alternative, `std::cin >> variable`, stops at the first space — so "Ada Lovelace" would store only "Ada" and leave "Lovelace" waiting in the input buffer to corrupt the next read. Any value that can contain a space must be read with `std::getline`. Declaring each variable immediately above the prompt that fills it keeps the two together and makes the code read top to bottom.

Step 4 — Read the two scores with `std::cin >>`

In `main.cpp`, find the line that reads the assignment name:

```
    std::getline(std::cin, assignmentName);
```

Directly below it, add:

```
    // Points are double, not int: partial credit such as 8.5 is common.
    double pointsEarned = 0.0;
    std::cout << "Points earned: ";
    std::cin >> pointsEarned;

    double pointsPossible = 0.0;
```

```
    std::cout << "Points possible: ";
    std::cin >> pointsPossible;
```

Why: These are numbers, so `std::cin >>` is correct: it skips leading whitespace and converts the digits to a `double`. The type is `double` rather than `int` because partial credit is normal — 8.5 out of 10 must be representable. Initializing each variable to `0.0` at the point of declaration means that if the read fails the variable still holds a defined value instead of garbage, which is a habit that prevents a whole class of bugs later.

Step 5 — Print the summary

In `main.cpp`, find the line that reads the points possible:

```
    std::cin >> pointsPossible;
```

Directly below it, add:

```
    std::cout << "\n--- Summary ---\n";
    std::cout << "Student:    " << studentName << "\n";
    std::cout << "Assignment: " << assignmentName << "\n";
    std::cout << "Score:      " << pointsEarned << " / " << pointsPossible << "\n";
```

Why: Echoing input back is how a user confirms the program understood them. The labels are padded with spaces so the values line up in a column — `"Student:    "` has four trailing spaces, `"Assignment: "` has one. Alignment like this is done by hand for now; Chapter 5 replaces it with a proper formatting approach.

> ▸ **Run checkpoint**: Click **Run** and type the inputs below in the Terminal, pressing Enter after each. The summary should match the expected output.

Expected output

With the inputs `Ada Lovelace`, `Analytical Engine Essay`, `8.5`, and `10`:

```
=== GRADE CALCULATOR v0.2 ===

Student name: Ada Lovelace
Assignment name: Analytical Engine Essay
Points earned: 8.5
Points possible: 10

--- Summary ---
Student:    Ada Lovelace
Assignment: Analytical Engine Essay
Score:      8.5 / 10
```

Note that `10` prints as `10`, not `10.00`. A `double` prints only as many decimals as it needs by default. Chapter 4 addresses that.

Check your work

Then confirm:

- ☐ `main.cpp` compiles with no errors and **no warnings**.
- ☐ A two-word student name is stored and echoed in full, not truncated at the space.
- ☐ A score of `8.5` is accepted and echoed as `8.5`.
- ☐ The four labels in the summary line up in a column.
- ☐ The program exits normally after printing the summary.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 3 Grade Calculator v0.2**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 3 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Discover your machine's types

```cpp
#include <iostream>
#include <limits>

int main() {
    std::cout << "int    " << sizeof(int) << " bytes  "
              << std::numeric_limits<int>::min() << " to "
              << std::numeric_limits<int>::max() << "\n";
    std::cout << "double " << sizeof(double) << " bytes  ~"
              << std::numeric_limits<double>::digits10 << " digits\n";
    std::cout << "char   " << sizeof(char) << " byte\n";
    std::cout << "bool   " << sizeof(bool) << " byte\n";
    return 0;
}
```

Expected output:

```
int    4 bytes  -2147483648 to 2147483647
double 8 bytes  ~15 digits
char   1 byte
bool   1 byte
```

Try: Add `short` and `long long`. Confirm that `int`'s range matches 2^{32} split between negative and positive, as Chapter 1 predicted.

2. Watch an integer overflow

```cpp
#include <iostream>

int main() {
    short small = 32767;
    std::cout << "short at max:   " << small << "\n";
    ++small;
    std::cout << "after adding 1: " << small << "\n";
    return 0;
}
```

Expected output:

```
short at max:   32767
after adding 1: -32768
```

Try: Predict what happens if you start at `-32768` and subtract 1. Then check.

3. Characters are numbers

```cpp
#include <iostream>

int main() {
    char c = 'A';
    std::cout << "'A' as char: " << c
              << "  as int: " << static_cast<int>(c) << "\n";
    std::cout << "'A' + 32 as char: " << static_cast<char>(c + 32) << "\n";
    return 0;
}
```

Expected output:

```
'A' as char: A  as int: 65
'A' + 32 as char: a
```

Try: Print `'Z'` as an int. Convert `'m'` to uppercase using the same 32 difference — do you add or subtract?

4. Integer division surprises

```cpp
#include <iostream>

int main() {
    int a = 7;
    int b = 2;
    std::cout << "7 / 2 as int:  " << a / b << "\n";
    std::cout << "7 % 2 as int:  " << a % b << "\n";
    std::cout << "7.0 / 2 mixed: " << 7.0 / b << "\n";
    return 0;
}
```

Expected output:

```
7 / 2 as int:  3
7 % 2 as int:  1
7.0 / 2 mixed: 3.5
```

Try: Predict `9 / 4`, `9 % 4`, and `9.0 / 4` before running them. This is the bug you will deliberately create in Chapter 4.

5. The getline trap

```cpp
#include <iostream>
#include <string>

int main() {
    int count = 0;
    std::cout << "How many assignments? ";
    std::cin >> count;

    std::string name;
    std::cout << "Your name: ";
    std::getline(std::cin, name);

    std::cout << "count=" << count << " name=[" << name << "]\n";
    return 0;
}
```

Run it and type `3`, then try to type your name.

What happens: the program never waits for your name, and prints `name=[]`.

Try: Add `#include <limits>` and insert this line after the `>>`:

```cpp
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

Rebuild and confirm it now waits.

6. Reason about types

For each, name a type and give one sentence of justification:

- The number of students in a class
- A student's grade point average
- Whether a student has submitted an assignment
- A course code such as `CS101`
- The letter grade for one assignment
- The maximum number of assignments a course may have, fixed at 20

7. Predict the errors

Do not build these first. Write down what the compiler will say.

```cpp
#include <iostream>

int main() {
    const double A_CUTOFF = 90.0;
    A_CUTOFF = 93.0;
    std::cout << A_CUTOFF << "\n";
    return 0;
}
```

```cpp
#include <iostream>

int main() {
    char name = "Ada Lovelace";
    std::cout << name << "\n";
    return 0;
}
```

Summary

- A **literal** is a value written directly in source. Single quotes make a `char`; double quotes make a string.
- A **variable** is a named memory location holding a value of a stated type. **The type is the meaning** — it decides how many bytes are reserved and how their bits are interpreted.
- `=` is **assignment**, not equality. Read it as "gets."
- **Always initialize where you declare**. Uninitialized variables hold garbage and produce bugs that differ every run.
- Identifiers are case-sensitive. Use `camelCase` for variables, `SCREAMING_SNAKE_CASE` for constants.
- **Use `int` for whole numbers** and `double` for values with fractional parts. Exceeding an integer's range causes **overflow**, which wraps around silently.
- Floating-point values are **approximations**. Never compare them with `==`.
- A `char` is one character and *is* a small integer — `'A'` is 65.
- `std::string` holds text of any length. A name cannot be a `char`.
- `const` names a value and prevents modification. Name your magic numbers.
- `std::cin >>` reads a value and stops at whitespace. `std::getline` reads a whole line. **Mixing them leaves a stray newline**, which makes the next `getline` return nothing — clear it with `std::cin.ignore`.

Key Terms

assignment — storing a new value in an existing variable, written with `=`.

bool — a type holding exactly `true` or `false`.

cast — an explicit conversion of a value to another type, written `static_cast<Type>(value)`.

char — a type holding one character in one byte; also a small integer.

const — a qualifier marking a variable as unmodifiable.

declaration — a statement bringing a variable into existence with a name and type.

double — the standard floating-point type, 8 bytes, about 15 digits of precision.

enumerated type — a type whose values are a small fixed named set.

escape sequence — a backslash followed by a character, representing one character that cannot be typed directly.

extraction operator — `>>`, which reads a value from an input stream.

float — a smaller floating-point type, 4 bytes, about 6 digits of precision.

getline — a function reading an entire line of text, including spaces.

identifier — a programmer-chosen name.

initialization — giving a variable a value at the moment it is declared.

int — the standard integer type, 4 bytes.

literal — a value written directly in source code.

magic number — an unexplained numeric value in code; avoid by naming it.

overflow — what happens when a value exceeds its type's range; the value wraps around.

std::cin — the standard input stream, normally the keyboard.

std::string — a type holding text of any length.

type — the property of a variable determining what values it can hold, how many bytes it occupies, and how its bits are interpreted.

unsigned — an integer type with no negative values, using all its bits for magnitude.

variable — a named memory location holding a value of a particular type.

Next: Chapter 4 puts these values to work. You will calculate a percentage, meet the difference between integer and floating-point division the hard way, add bonus points, and decide what your calculator should do when a bonus pushes a score past 100. Grade Calculator v0.3.

Chapter 4 — Expressions, Arithmetic, and Errors

Learning Objectives

When you finish this chapter you will be able to:

- Build expressions from values, variables, and operators, and predict their results. *(SLO 1.3)*
- Explain integer division and modulus, and recognize when integer division is producing a wrong answer. *(SLO 1.1, 1.3)*
- Predict how C++ converts types in a mixed expression, and control conversion with a cast. *(SLO 1.1)*
- Apply operator precedence and associativity, and use parentheses to make intent explicit. *(SLO 1.3)*
- Format numeric output to a chosen number of decimal places. *(SLO 1.3)*
- Distinguish compile-time errors, run-time errors, logic errors, and warnings, and describe how each is found. *(SLO 1.6)*
- Write comments that explain decisions rather than restating code. *(SLO 1.6)*
- Build Grade Calculator v0.3, including a documented bonus-point policy.

4.1 Expressions and Operators

An **expression** is anything that produces a value. A literal is an expression. A variable is an expression. Combining them with operators produces larger expressions:

```
84.5
pointsEarned
pointsEarned + bonusPoints
pointsEarned / pointsPossible * 100.0
```

An **operator** combines values. The values it works on are **operands**. C++'s arithmetic operators:

Operator	Meaning	Example	Result
`+`	addition	`7 + 2`	`9`
`-`	subtraction	`7 - 2`	`5`
`*`	multiplication	`7 * 2`	`14`
`/`	division	`7 / 2`	**see Section 4.3**
`%`	modulus (remainder)	`7 % 2`	`1`

Four of those five behave exactly as you expect. The fifth does not, and it is responsible for more early bugs than anything else in this book.

4.2 Integer and Floating-Point Arithmetic Differ

Here is the rule that matters:

> **If both operands of an operator are integers, the result is an integer. If either operand is floating-point, the result is floating-point.**

For `+`, `-`, and `*` this is harmless. For `/` it changes the answer.

4.3 Integer Division and Modulus

When both operands are integers, `/` performs **integer division**: it divides and **discards the remainder**. It does not round. It truncates.

```
int a = 7;
int b = 2;
std::cout << "7 / 2 as int:  " << a / b << "\n";
std::cout << "7 % 2 as int:  " << a % b << "\n";
std::cout << "7.0 / 2 mixed: " << 7.0 / b << "\n";
```

```
7 / 2 as int:  3
7 % 2 as int:  1
7.0 / 2 mixed: 3.5
```

`7 / 2` is 3, not 3.5 and not 4. The `.5` is thrown away. The `%` operator recovers what was discarded: `7 % 2` is 1, the remainder.

Why this matters for grades

Consider computing a percentage from 84 points out of 100:

```
int earned = 84;
int possible = 100;
double percentage = earned / possible * 100.0;
```

You would expect 84.0. You get **0.0**.

Work through it in the order C++ does. First `earned / possible` is evaluated. Both are `int`, so this is integer division: 84 divided by 100 is 0 with remainder 84, and the remainder is discarded. The result is `0`. Then `0 * 100.0` is `0.0`. Assigning to a `double` at the end cannot recover information already thrown away.

The fix is to make at least one operand floating-point before dividing:

```
double percentage = static_cast<double>(earned) / possible * 100.0;
```

Or, better, declare the variables as `double` in the first place — which is exactly what Section 3.11 concluded, because partial credit exists.

This is the single most common arithmetic bug in beginning C++. It produces no error and no warning. The program runs happily and reports the wrong grade. You will create it deliberately in the Grade Calculator section, because meeting it on purpose once is much cheaper than meeting it by accident later.

Legitimate uses of `%`

Modulus is genuinely useful, not just a curiosity:

```
if (n % 2 == 0) { /* n is even */ }
int minutes = totalSeconds % 60;
if (count % 10 == 0) { /* every tenth item */ }
```

4.4 Mixed-Type Expressions and Conversion

When operands have different types, C++ converts one before operating. The general direction is toward the type that can hold more information — an `int` mixed with a `double` becomes a `double`. This is an **implicit conversion**, because you did not ask for it.

```
int count = 3;
double average = 10.0;
double total = count * average;    // count becomes 3.0; result 30.0
```

Implicit conversion is convenient and occasionally surprising, so C++ also provides an explicit form. A **cast** states the conversion you want:

```
double exact = static_cast<double>(earned) / possible;
```

`static_cast<double>(earned)` produces a `double` copy of `earned` for this expression. The variable itself is unchanged.

Prefer `static_cast` to the older C-style form `(double)earned`. Appendix D Section D.11 explains why: `static_cast` is searchable, it is visually obvious, and it refuses conversions that are genuinely unsafe.

Conversion the other way loses information. Assigning a `double` to an `int` discards the fractional part without warning you at runtime:

```
int rounded = 91.7;    // rounded is 91, not 92 — truncation, not rounding
```

Chapter 8 introduces `std::round` for when you actually want rounding.

4.5 Operator Precedence and Associativity

Precedence decides which operator acts first when an expression contains several.

```
std::cout << 2 + 3 * 4 << "\n";      // 14, not 20
std::cout << (2 + 3) * 4 << "\n";    // 20
```

Multiplication has higher precedence than addition, so `3 * 4` happens first. Parentheses override precedence.

Associativity decides the order among operators of equal precedence.

```
std::cout << 10 - 4 - 3 << "\n";     // 3, because (10 - 4) - 3
```

Arithmetic operators are **left-associative**: they group from the left.

The operators you have met, by precedence

Highest first. Operators on the same row have equal precedence.

Precedence	Operators	Associativity
Highest	`()` grouping	—
	`++ --` (prefix), `static_cast`	right to left
	`* / %`	left to right
	`+ -`	left to right
	`<< >>` (stream)	left to right
Lowest	`= += -= *= /=`	right to left

Later chapters extend this table: Chapter 6 adds the comparison and logical operators, Chapter 7 the conditional operator.

Precedence and integer division together

Precedence and the integer-division rule interact in a way worth seeing explicitly:

```
std::cout << 84 / 100 * 100 << "\n";      // 0
std::cout << 84.0 / 100 * 100 << "\n";    // 84
```

```
84 / 100 * 100 = 0
84.0 / 100 * 100 = 84
```

Both expressions look like they should give 84. The first divides before multiplying — left to right, equal precedence — and integer division makes that first step 0. Everything after is doomed.

Use parentheses for the reader

You do not need parentheses in `a + b * c`. Add them anyway when they make intent obvious:

```
double percentage = (earned + bonus) / possible * 100.0;
```

Here they are genuinely required — without them, only `bonus` would be divided. But even where precedence already gives the right answer, parentheses cost nothing and remove doubt.

4.6 Compound Assignment, Increment, and Decrement

Adding to a variable is common enough to have a shorthand:

```
int n = 10;
n += 5;    // same as n = n + 5;   n is 15
n -= 3;    // n is 12
n *= 2;    // n is 24
n /= 4;    // n is 6
```

These are **compound assignment** operators. They are shorter and, more usefully, they name the variable once — so you cannot accidentally write `total = totl + 5`.

Adding or subtracting exactly one has its own operators:

```
++count;    // increment: add 1
--count;    // decrement: subtract 1
```

These come in two forms whose difference matters only when you use the value of the expression itself:

```
int i = 5;
int before = i++;      // postfix: use the old value, then add 1
```

```
// before is 5, i is 6

int j = 5;
int after = ++j;       // prefix: add 1, then use the new value
// after is 6, j is 6
```

```
i++ gave 5, i is now 6
++j gave 6, j is now 6
```

When you are not using the result — which is nearly always — the two behave identically, and this book writes `++count` by convention.

> **A warning worth heeding.** Do not modify a variable and also read it in the same statement, as in `std::cout << i++ << i;`. C++ does not define the order in which those happen, so the result is unpredictable and may differ between compilers. Compiling with `-Wall` reports this as `warning: operation on 'i' may be undefined [-Wsequence-point]`. Keep the increment on its own line.

4.7 Formatting Numeric Output

By default, C++ prints floating-point values with about six significant digits:

```
std::cout << 91.666666 << "\n";      // 91.6667
```

For a grade report you want control. `<iomanip>` provides **manipulators**:

```
#include <iomanip>

std::cout << std::fixed << std::setprecision(1);
std::cout << 91.666666 << "\n";      // 91.7
```

Manipulator	Effect
`std::fixed`	always use decimal notation, not scientific
`std::setprecision(n)`	with `fixed`, show exactly *n* digits after the point
`std::setw(n)`	pad the **next** value to at least *n* characters
`std::left` / `std::right`	control which side the padding goes on

```
default:            91.6667
fixed, 1 decimal:   91.7
fixed, 3 decimals:  91.667
setw(10):          [      91.7]
left setw(10):     [91.7      ]
```

Two behaviors trip people up. **`std::fixed` and `std::setprecision` stay in effect** until changed, so set them once before your report rather than repeating them. **`std::setw` applies only to the very next value**, so it must be repeated for each column.

4.8 Bitwise Operators

C++ can operate on individual bits — the bits of Chapter 1, now directly accessible.

Operator	Meaning	Example	Result
`&`	AND — 1 where both bits are 1	`12 & 10`	`8`
`|`	OR — 1 where either bit is 1	`12 | 10`	`14`
`^`	XOR — 1 where exactly one bit is 1	`12 ^ 10`	`6`
`<<`	shift left	`3 << 2`	`12`
`>>`	shift right	`12 >> 2`	`3`

Work `12 & 10` out in binary and it is obvious:

```
  1100    (12)
& 1010    (10)
  ----
  1000    (8)
```

Shifting left by n multiplies by 2^n, and shifting right divides by 2^n — which follows directly from the place values in Figure 1.2.

Reference only. The Grade Calculator never uses bitwise operators, and no exercise requires them. They are covered because they are part of the language and because they make Chapter 1's bit patterns concrete. Note that `<<` and `>>` do double duty as the stream operators — the compiler tells them apart by what is on the left.

4.9 Kinds of Errors

Programs go wrong in four distinct ways. Telling them apart is what makes debugging systematic rather than random, and it is the foundation Chapter 16 builds on.

4.9.1 Compile-time errors

The compiler cannot understand your program. **Nothing is built.**

```
std::cout << "missing semicolon\n"
return 0;
```

These are the friendliest errors you will meet: they are found before the program runs, and the compiler tells you the file and line. Chapter 2 Section 2.6 covered reading these messages.

4.9.2 Run-time errors

The program compiled and started, then failed while running.

```
int possible = 0;
int result = 100 / possible;    // integer division by zero
```

The compiler cannot catch this, because `possible` is only zero for certain inputs. Run-time errors are found by testing — particularly testing the boundary cases Chapter 16 makes systematic.

Note the asymmetry: integer division by zero crashes the program, while floating-point division by zero produces the special value `inf` and keeps going. Both are wrong; only one announces itself.

4.9.3 Logic errors

The program compiles, runs, finishes normally, and produces the **wrong answer**.

```
int earned = 84;
int possible = 100;
double percentage = earned / possible * 100.0;    // 0.0, not 84.0
```

No error. No warning. No crash. A confidently reported wrong grade.

Logic errors are the dangerous ones. The compiler cannot help, because the program is valid — it simply does not do what you meant. They are found by testing against results you worked out independently, which is why every Grade Calculator version in this book comes with expected output.

4.9.4 Compiler warnings

The program compiled, and the compiler noticed something suspicious:

```
warning: unused variable 'total' [-Wunused-variable]
```

A warning is not an error — the executable exists. But warnings are usually right, and a warning ignored today is a logic error hunted next week. Appendix D Section D.10 makes this a rule: **compile with `-Wall -Wextra` and fix every warning.**

The four compared

Kind	Found by	Program built?	Program runs?	Typical cause
Compile-time	the compiler	no	no	Syntax, misspelling, type mismatch
Run-time	testing	yes	starts, then fails	Division by zero, bad input
Logic	testing against known answers	yes	yes, finishes	Wrong formula, wrong order, integer division
Warning	the compiler	yes	yes	Something suspicious but legal

4.10 Comments and Self-Documenting Code

Chapter 2 introduced the two comment forms. Now that expressions can be subtle, here is what makes a comment worth writing.

A comment that restates the code is noise:

```
// Do not write this.
totalEarned += pointsEarned;    // add points earned to total earned
```

A comment that explains a decision earns its place:

```
// Write this.
// Bonus points are added to points EARNED but not to points POSSIBLE, so a
// percentage above 100 is possible. See CAP_AT_100 for the course policy.
totalEarned += pointsEarned + bonusPoints;
```

The best comment is often the one you did not need to write, because the code says it:

```
// Needs a comment:
double p = (e + b) / t * 100.0;
```

```
// Does not:
double percentage = (pointsEarned + bonusPoints) / pointsPossible * 100.0;
```

Good names remove the need for explanation. Comments are for the things names cannot carry: why a policy is what it is, what a formula assumes, what you deliberately chose not to do.

Common Errors and Warnings

What you see or observe	Cause	Fix
A percentage is `0` when it should not be	Integer division — both operands are `int`	Make one a `double`, or declare them `double`
A value is truncated, e.g. `91.7` becomes `91`	Assigned a `double` to an `int`	Use a `double`, or `std::round` (Chapter 8)
`2 + 3 * 4` gives 14, not 20	Precedence: `*` before `+`	Add parentheses
Output shows `91.6667` instead of `91.7`	No formatting set	`std::fixed << std::setprecision(1)`
Only the first column lines up	`std::setw` applies to one value only	Repeat `setw` per column
`warning: operation on 'i' may be undefined`	Modified and read a variable in one statement	Put the increment on its own line
Program crashes on some inputs only	Run-time error, likely integer division by zero	Check the denominator before dividing
`error: 'setprecision' was not declared`	Missing header	`#include <iomanip>`

Design Notes

Declare the type you mean. Most integer-division bugs come from declaring an `int` where the quantity was never whole. Section 3.11's justification exercise prevents this before it starts.

Guard your denominators. Before dividing, ask what happens if the denominator is zero. The Grade Calculator has a real case: an assignment worth zero points. v0.4 will report it in words rather than printing a meaningless number.

Decide policy explicitly, then write it down. You are about to make a real decision — whether a bonus may push a grade above 100. Either answer is defensible. Choosing without recording the choice is what makes code mysterious six months later.

Grade Calculator v0.3 — Percentage and Bonus Points

What v0.3 does

Reads a named assignment with points earned, points possible, and bonus points, then computes and reports a percentage to one decimal place.

The policy decision

Bonus points are added to points earned but not to points possible. That is what makes them a bonus. It also means a percentage above 100 is possible: 10 out of 10 plus 5 bonus is 150%.

Should your calculator report 150%, or cap it at 100%?

Both are real policies. Uncapped rewards exceptional work and can pull up a low average. Capped treats the maximum as a maximum. **v0.3 reports the raw value uncapped**, and Chapter 8 implements the cap once you have the tools to express it in one clean line. What matters now is that the choice is deliberate and documented.

The program

```
// Grade Calculator v0.3 - Chapter 4
// Adds bonus points and a percentage calculation.
// New this version: floating-point division, output formatting.
// Design note: bonus points are added to points EARNED but not to points
// POSSIBLE, so a percentage above 100 is possible. v0.3 reports it uncapped.
// Run: click Run in StudySite and use the embedded Terminal.

#include <iostream>
#include <iomanip>
#include <string>

int main() {
    std::cout << "=== GRADE CALCULATOR v0.3 ===\n\n";

    std::string studentName;
    std::cout << "Student name: ";
    std::getline(std::cin, studentName);

    std::string assignmentName;
    std::cout << "Assignment name: ";
    std::getline(std::cin, assignmentName);

    double pointsEarned = 0.0;
    std::cout << "Points earned: ";
    std::cin >> pointsEarned;

    double pointsPossible = 0.0;
    std::cout << "Points possible: ";
    std::cin >> pointsPossible;

    double bonusPoints = 0.0;
    std::cout << "Bonus points (0 if none): ";
    std::cin >> bonusPoints;

    double totalEarned = pointsEarned + bonusPoints;

    // Both operands are double, so this is real division, not integer division.
    double percentage = 0.0;
    if (pointsPossible > 0.0) {
        percentage = totalEarned / pointsPossible * 100.0;
    }

    std::cout << "\n--- Summary ---\n";
    std::cout << "Student:    " << studentName << "\n";
    std::cout << "Assignment: " << assignmentName << "\n";
    std::cout << "Earned:     " << totalEarned << " (" << pointsEarned
              << " + " << bonusPoints << " bonus)\n";
    std::cout << "Possible:   " << pointsPossible << "\n";
    std::cout << "Percentage: " << std::fixed << std::setprecision(1)
              << percentage << "%\n";
    return 0;
```

```
}
```

Expected output

With input `Ada Lovelace`, `Midterm Exam`, `84`, `100`, `5`:

```
--- Summary ---
Student:    Ada Lovelace
Assignment: Midterm Exam
Earned:     89 (84 + 5 bonus)
Possible:   100
Percentage: 89.0%
```

What to notice

`if (pointsPossible > 0.0)` guards the division. An assignment worth zero points has no percentage, and dividing by zero is the run-time error of Section 4.9.2. You have not formally met `if` yet — Chapter 6 covers it — but its meaning here is plain, and guarding a denominator is a habit worth starting immediately.

The comment explains why the division is safe, not what it does. That is the Section 4.10 rule.

Every variable involved in the division is `double`. Not by accident. That single decision, made in Chapter 3, is what prevents the bug you are about to create on purpose.

Your StudySite Lab — Calculate Percentages

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v0.3
- **Starting point**: Your committed Chapter 3 v0.2 program (one file, reads and echoes one assignment).

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

Your program stores numbers but does not yet do arithmetic with them. In this chapter it starts calculating: it accepts bonus points, adds them to the score, and reports a percentage formatted to one decimal place. Two traps appear here that catch nearly everyone — integer division silently throwing away the fraction, and dividing by zero — and you will handle both deliberately. By the end you will have:

1. A third numeric prompt for bonus points.
2. A `totalEarned` value combining the score and the bonus.
3. A percentage computed with real (floating-point) division, guarded so it is never computed when `pointsPossible` is zero.
4. Output formatted with `std::fixed` and `std::setprecision(1)`, so a percentage always shows exactly one decimal.

   Design note: bonus points are added to points *earned* but not to points *possible*, so a percentage above 100 is possible. This version reports it uncapped on purpose. Chapter 8 revisits that policy.

Files in this checkpoint

File	What you do to it
`main.cpp`	Edit (Steps 1-6)

When you are finished your project still has **one** `.cpp` file.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `main.cpp`. Confirm the editor shows the expected file paths before editing.
4. Continue the existing program in StudySite's internal editor, finish the steps in order, then Run.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Update the comment header and include `<iomanip>`

In `main.cpp`, find the first three comment lines:

```
// Grade Calculator v0.2 - Chapter 3
// Reads one named assignment and echoes a formatted summary.
// New this version: typed variables, std::getline for names.
```

Replace it with:

```
// Grade Calculator v0.3 - Chapter 4
// Adds bonus points and a percentage calculation.
// New this version: floating-point division, output formatting.
// Design note: bonus points are added to points EARNED but not to points
// POSSIBLE, so a percentage above 100 is possible. v0.3 reports it uncapped.
```

Now find the `<iostream>` include and add the new one directly below it:

```
#include <iostream>
```

Directly below it, add:

```
#include <iomanip>
```

Why: `std::fixed` and `std::setprecision` are declared in `<iomanip>` ("input/output manipulators"). Without the include the compiler reports that `std::setprecision` is not declared, which is the same class of error as forgetting `<string>` in Chapter 3. The design note now lives in the file header because a decision about *policy* — whether a percentage may exceed 100 — belongs with the code that implements it, where the next reader will find it.

Step 2 — Update the banner version

In `main.cpp`, find the banner line:

```
    std::cout << "=== GRADE CALCULATOR v0.2 ===\n\n";
```

Replace it with:

```
    std::cout << "=== GRADE CALCULATOR v0.3 ===\n\n";
```

Why: The banner names the version so that when a student shows you their output you can both tell which checkpoint they are running. Update it every chapter.

Step 3 — Remove two comments that have served their purpose

In `main.cpp`, find this comment and delete it:

```
    // Names may contain spaces, so getline is required here.
```

Find this comment and delete it too:

```
    // Points are double, not int: partial credit such as 8.5 is common.
```

Why: Both comments explained a decision you have now internalized — `getline` for names, `double` for points. A comment that restates what the code plainly says becomes noise, and noise is what stops people reading comments at all. Delete explanatory comments once the explanation has landed; keep the ones that record a decision you could not infer from the code, like the design note in the header.

> ▸ **Run checkpoint**: Click **Run**. The program should behave exactly as it did in Chapter 3, with the new version number in the banner. Nothing has changed functionally yet.

Step 4 — Read the bonus points

In `main.cpp`, find the three lines that read the points possible:

```
    double pointsPossible = 0.0;
    std::cout << "Points possible: ";
    std::cin >> pointsPossible;
```

Directly below it, add:

```
    double bonusPoints = 0.0;
    std::cout << "Bonus points (0 if none): ";
    std::cin >> bonusPoints;
```

Why: The prompt says `(0 if none)` because a prompt that gives no guidance leaves the user guessing whether to type `0`, press Enter, or type `none`. Only one of those works. Telling the user what to type for the empty case costs eight characters and prevents the most common input mistake.

Step 5 — Compute the total and the percentage

In `main.cpp`, find the line that reads the bonus points:

```
    std::cin >> bonusPoints;
```

Directly below it, add:

```
    double totalEarned = pointsEarned + bonusPoints;

    // Both operands are double, so this is real division, not integer division.
    double percentage = 0.0;
    if (pointsPossible > 0.0) {
        percentage = totalEarned / pointsPossible * 100.0;
    }
```

Why: Two things are happening, and both are the chapter's real content.

First, **division**. `totalEarned / pointsPossible` divides one `double` by another, which is real division and keeps the fraction. Had both values been `int`, C++ would perform *integer* division and throw the remainder away — `8 / 10` would be `0`, and the percentage would come out as `0` instead of `80`. The reason this program uses `double` for points is as much about this division as about accepting `8.5` as input.

Second, the **guard**. `if (pointsPossible > 0.0)` prevents a division by zero, which for floating-point numbers does not crash but silently produces `inf` or `nan` — values that then contaminate everything they touch and print as meaningless text. Because `percentage` was initialized to `0.0` at its declaration, skipping the calculation leaves a sensible value behind rather than garbage.

Step 6 — Report the total, the possible, and the percentage

In `main.cpp`, find the score line in the summary:

```
    std::cout << "Score:      " << pointsEarned << " / " << pointsPossible << "\n";
```

Replace that one line with these five:

```
    std::cout << "Earned:     " << totalEarned << " (" << pointsEarned
              << " + " << bonusPoints << " bonus)\n";
    std::cout << "Possible:   " << pointsPossible << "\n";
    std::cout << "Percentage: " << std::fixed << std::setprecision(1)
              << percentage << "%\n";
```

Why: The summary now shows the arithmetic rather than just the result: `Earned: 9.5 (8.5 + 1 bonus)` lets the user verify the total themselves, which is what makes output trustworthy. `std::fixed` tells the stream to use fixed rather than scientific notation, and `std::setprecision(1)` fixes the number of digits after the decimal point at one — so `80` prints as `80.0` and `83.33333` as `83.3`. Note that these two manipulators are **sticky**: once set, they affect every later floating-point value on that stream, which is why they appear on the last line rather than the first.

Note also that the statement spans two lines. A long chain of `<<` can be broken wherever you like; the compiler does not care, and the indentation lines the continuation up under the first value for readability.

- ▸ **Run checkpoint**: Click **Run** and work through the test plan below in the Terminal.

Test plan

Run the program three times with these inputs.

1. Normal case with a bonus. `Ada Lovelace`, `Analytical Engine Essay`, `8.5`, `10`, `1`:

```
--- Summary ---
Student:    Ada Lovelace
Assignment: Analytical Engine Essay
Earned:     9.5 (8.5 + 1 bonus)
Possible:   10
Percentage: 95.0%
```

2. No bonus. Same, but `0` for the bonus — `Earned: 8.5 (8.5 + 0 bonus)` and `Percentage: 85.0%`.

3. Zero points possible. Enter `0` for points possible. The percentage must print `0.0%`, not `inf` or `nan`. This is the guard from Step 5 doing its job; without it this run would produce nonsense.

Check your work

Then confirm:

- ☐ `main.cpp` compiles with no errors and **no warnings**.
- ☐ A percentage always shows exactly one digit after the decimal point.
- ☐ Entering `0` for points possible prints `0.0%` and never `inf` or `nan`.
- ☐ The `Earned:` line shows the total, the original score, and the bonus.
- ☐ A bonus large enough to exceed the points possible produces a percentage above 100, and the program still exits normally.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 4 Grade Calculator v0.3**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 4 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Integer versus floating-point division

```cpp
#include <iostream>

int main() {
    int a = 7;
    int b = 2;
    std::cout << "7 / 2 as int:  " << a / b << "\n";
    std::cout << "7 % 2 as int:  " << a % b << "\n";
    std::cout << "7.0 / 2 mixed: " << 7.0 / b << "\n";
    return 0;
}
```

Expected output:

```
7 / 2 as int:  3
7 % 2 as int:  1
7.0 / 2 mixed: 3.5
```

Try: Predict `9 / 4`, `9 % 4`, `9.0 / 4`, and `-7 / 2` before running each.

2. Precedence in action

```cpp
#include <iostream>

int main() {
    std::cout << "2 + 3 * 4        = " << 2 + 3 * 4 << "\n";
    std::cout << "(2 + 3) * 4      = " << (2 + 3) * 4 << "\n";
    std::cout << "10 - 4 - 3       = " << 10 - 4 - 3 << "\n";
    std::cout << "84 / 100 * 100   = " << 84 / 100 * 100 << "\n";
    std::cout << "84.0 / 100 * 100 = " << 84.0 / 100 * 100 << "\n";
    return 0;
}
```

Expected output:

```
2 + 3 * 4        = 14
(2 + 3) * 4      = 20
10 - 4 - 3       = 3
84 / 100 * 100   = 0
84.0 / 100 * 100 = 84
```

Try: Add parentheses to the fourth line to make it produce 84 without changing any value to a `double`. Is that a good fix? Why might declaring the values as `double` be better?

3. Formatting output

```cpp
#include <iostream>
#include <iomanip>

int main() {
    std::cout << "default:           " << 91.666666 << "\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "fixed, 1 decimal:  " << 91.666666 << "\n";
    std::cout << std::setprecision(3);
    std::cout << "fixed, 3 decimals: " << 91.666666 << "\n";
    std::cout << "setw(10):         [" << std::setw(10) << 91.7 << "]\n";
```

```
    return 0;
}
```

Expected output:

```
default:           91.6667
fixed, 1 decimal:  91.7
fixed, 3 decimals: 91.667
setw(10):         [    91.700]
```

Try: Explain why the last line shows `91.700` rather than `91.7`. Then add `std::left` before the `setw` and observe the change.

4. Compound assignment and increment

```
#include <iostream>

int main() {
    int n = 10;
    n += 5;   std::cout << "n += 5 -> " << n << "\n";
    n -= 3;   std::cout << "n -= 3 -> " << n << "\n";
    n *= 2;   std::cout << "n *= 2 -> " << n << "\n";

    int i = 5;
    int before = i++;
    std::cout << "i++ gave " << before << ", i is now " << i << "\n";

    int j = 5;
    int after = ++j;
    std::cout << "++j gave " << after << ", j is now " << j << "\n";
    return 0;
}
```

Expected output:

```
n += 5 -> 15
n -= 3 -> 12
n *= 2 -> 24
i++ gave 5, i is now 6
++j gave 6, j is now 6
```

Try: Add `n /= 5;` after the last compound assignment. `n` is 24 — what does integer division give you?

5. Bits, made visible

```
#include <iostream>

int main() {
    std::cout << "12 & 10 = " << (12 & 10) << "\n";
    std::cout << "12 | 10 = " << (12 | 10) << "\n";
    std::cout << "12 ^ 10 = " << (12 ^ 10) << "\n";
    std::cout << "3 << 2  = " << (3 << 2) << "\n";
    return 0;
}
```

Expected output:

```
12 & 10 = 8
12 | 10 = 14
12 ^ 10 = 6
3 << 2  = 12
```

Try: Write 12 and 10 in binary and verify each result by hand. Why does `3 << 2` give 12?

6. Classify the error

For each, say whether it is a compile-time error, a run-time error, a logic error, or a warning — and how you would find it.

- A semicolon is missing at the end of a statement.
- A percentage is computed as `earned / possible` where both are `int`.
- A program divides by a value the user entered as zero.
- A variable is declared and never used.
- A grade cutoff is written as `80.0` where the policy says `90.0`.
- `std::cout` is written without including `<iostream>`.

7. Find the bug by reasoning

This program reports a wrong average. Do not run it — find the error by reading:

```cpp
#include <iostream>

int main() {
    int total = 250;
    int count = 4;
    double average = total / count;
    std::cout << "Average: " << average << "\n";
    return 0;
}
```

What does it print? What should it print? What is the smallest change that fixes it? Now run it and check.

Summary

- An **expression** produces a value. **Operators** combine **operands**.
- **If both operands are integers, the result is an integer.** Integer division **truncates** — `7 / 2` is 3. `%` gives the remainder.
- Computing a percentage with `int` operands gives `0`, silently. This is the most common arithmetic bug in beginning C++.
- Mixed-type expressions **convert implicitly** toward the wider type. A **cast** — `static_cast<double>(x)` — makes conversion explicit. Assigning a `double` to an `int` truncates.
- **Precedence** decides which operator acts first; **associativity** decides among equals. Arithmetic is left-associative. Use parentheses to make intent obvious even when they are not required.
- Compound assignment (`+=`) and increment (`++`) are shorthand. Do not modify and read a variable in the same statement.
- `<iomanip>` provides `std::fixed`, `std::setprecision`, and `std::setw`. Precision persists; `setw` applies only to the next value.
- **Bitwise operators** work on individual bits. Reference only in this book.

- Four kinds of failure: **compile-time** (nothing built), **run-time** (starts, then fails), **logic** (runs and gives a wrong answer), and **warnings** (built, but suspicious). Logic errors are the dangerous ones because nothing announces them.
- Comment **why**, not what. Good names remove the need for most comments.

Key Terms

associativity — the rule deciding the order among operators of equal precedence.

bitwise operator — an operator acting on the individual bits of a value.

cast — an explicit conversion between types, written `static_cast<Type>(value)`.

compile-time error — a fault preventing the compiler from producing a program.

compound assignment — an operator such as `+=` combining an operation with assignment.

decrement — the `--` operator, subtracting one.

expression — any construct producing a value.

implicit conversion — an automatic type conversion performed by the compiler.

increment — the `++` operator, adding one.

integer division — division of two integers, discarding the remainder.

logic error — a fault in which a valid program produces a wrong result.

manipulator — a value such as `std::fixed` that changes how a stream formats output.

modulus — the `%` operator, giving the remainder of integer division.

operand — a value an operator acts upon.

operator — a symbol combining operands to produce a value.

precedence — the rule deciding which operator acts first.

run-time error — a fault occurring while the program runs.

truncation — discarding a fractional part rather than rounding.

warning — a compiler report of something suspicious but legal.

Next: Chapter 5 steps away from the keyboard. Before your calculator gets any more complicated, you will learn to design it — pseudocode, flowcharts, and structure charts — and produce the specification you will carry, and revise, for the next twenty chapters. Grade Calculator v0.4 puts that design work into the program's interface.

Chapter 5 — Algorithmic Design and Program Documentation

Learning Objectives

When you finish this chapter you will be able to:

- Define an algorithm and state the properties every algorithm must have. *(SLO 1.3)*
- Turn a requirements statement into a written specification. *(SLO 1.5)*
- Write pseudocode for a process before writing any code. *(SLO 1.4)*
- Draw a flowchart using the standard symbols, and read one written by someone else. *(SLO 1.4)*
- Draw a structure chart decomposing a program into parts. *(SLO 1.4)*
- Desk-check an algorithm with a trace table. *(SLO 1.4, 1.6)*
- Explain the three structures of structured programming. *(SLO 1.3, 1.4)*
- Apply basic user interface design to a console program. *(SLO 1.3)*
- Produce the design document you will carry, and revise, for the rest of both courses. *(SLO 1.5, 1.6)*
- Build Grade Calculator v0.4.

5.1 What Makes an Algorithm

You have written three programs by thinking about the code. That works while a program fits on one screen. It stops working shortly after.

This chapter is a deliberate pause. No new C++ appears in it. Instead you will learn to design a program before writing it — and produce a design document for the Grade Calculator that you will revise in Chapter 13, build against in Chapter 20, and formally reconcile in Chapter 24.

An **algorithm** is a finite sequence of unambiguous steps that solves a problem. Four properties are required:

Finite. It must end. A procedure that never finishes is not an algorithm, however sensible each step looks.

Unambiguous. Every step must have exactly one interpretation. “Adjust the grade appropriately” is not a step. “Add the bonus points to the points earned” is.

Effective. Each step must be something the machine can actually do.

Correct. It must produce the right answer for every valid input — not just the one you tried.

That last one is where most of the work lives. A recipe that works for 84 out of 100 and fails for 0 out of 0 is not correct, and you already met that exact case in Chapter 4.

Why design before coding

There is a temptation to skip this chapter and keep typing. Three arguments against it.

Mistakes are cheapest when they are found earliest. Fixing a design flaw on paper takes a minute. Fixing it after two hundred lines of code takes an afternoon.

Design forces the questions code lets you avoid. What should happen when points possible is zero? What if the user types a letter where a number belongs? Code lets you postpone those. A specification does not.

You cannot build what you have not described. By Chapter 24 the Grade Calculator has multiple grading schemes, custom scales, file storage, and error handling. Nobody holds that in their head. It gets built because it was written down.

5.2 From Requirements to Specification

In Chapter 1 you wrote a **requirements statement** — plain sentences describing what the calculator must do, including the scope boundary that deferred weighted grading.

A **specification** is the next step: precise enough to build from and to test against. The difference is checkability.

Requirement	Specification
"Report the grade"	"Report the percentage to one decimal place, followed by a single letter grade"
"Handle bonus points"	"Add bonus points to points earned; do not add them to points possible"
"Deal with bad input"	"If points possible is zero, display 'not available' and no percentage"

Every specification line answers a question a builder would otherwise have to guess.

A specification for Grade Calculator v0.4

Purpose. Compute and report a points-based grade for a single assignment.

Inputs.

Input	Type	Constraint
Student name	text	may contain spaces; may not be empty
Assignment name	text	may contain spaces; may not be empty
Points earned	number	zero or greater
Points possible	number	zero or greater
Bonus points	number	zero or greater; zero means none

Processing.

1. Total earned = points earned + bonus points.
2. If points possible is greater than zero, percentage = total earned ÷ points possible × 100.
3. If points possible is zero, no percentage exists.

Outputs. A report showing student name, assignment name, points earned, bonus points, total earned, points possible, and either the percentage to one decimal place or an explanation of why none is available.

Out of scope. Weighted-category grading. Letter grades (Chapter 6). Multiple assignments (Chapter 7). Multiple students (Chapter 11).

Notice that the out-of-scope section names *when* each deferred item arrives. That turns a list of absences into a plan.

5.3 Writing Pseudocode

Pseudocode describes an algorithm in structured English. It has no official syntax, which is the point — it lets you think about the process without fighting a compiler.

```
BEGIN
    DISPLAY "Student name: "
    READ studentName
    DISPLAY "Assignment name: "
    READ assignmentName
    DISPLAY "Points earned: "
    READ pointsEarned
    DISPLAY "Points possible: "
    READ pointsPossible
    DISPLAY "Bonus points: "
    READ bonusPoints

    SET totalEarned = pointsEarned + bonusPoints

    IF pointsPossible > 0 THEN
        SET percentage = totalEarned / pointsPossible * 100
        DISPLAY report WITH percentage
    ELSE
        DISPLAY report WITH "percentage not available"
    END IF
END
```

Conventions this book uses, for consistency rather than because any authority requires them:

Word	Meaning
`READ`	get a value from the user
`DISPLAY`	show something to the user
`SET`	store a value in a variable
`IF … THEN … ELSE … END IF`	choose between paths
`WHILE … END WHILE`	repeat while a condition holds
`FOR EACH … END FOR`	repeat over a collection

Two rules make pseudocode worth writing.

Indent to show structure. Everything inside an `IF` is indented. This is the same discipline as Appendix D's four-space rule, and for the same reason.

Describe the process, not the language. Write `SET percentage = total / possible * 100`, not `double percentage = total / possible * 100.0;`. The moment pseudocode becomes C++ with the semicolons removed, it stops helping you think and starts being a slower way to write code.

5.4 Flowcharting

A **flowchart** shows an algorithm as a diagram. Where pseudocode is easier to write, a flowchart is easier to *see* — branches and loops become visible shapes rather than indentation you have to track.

The symbols

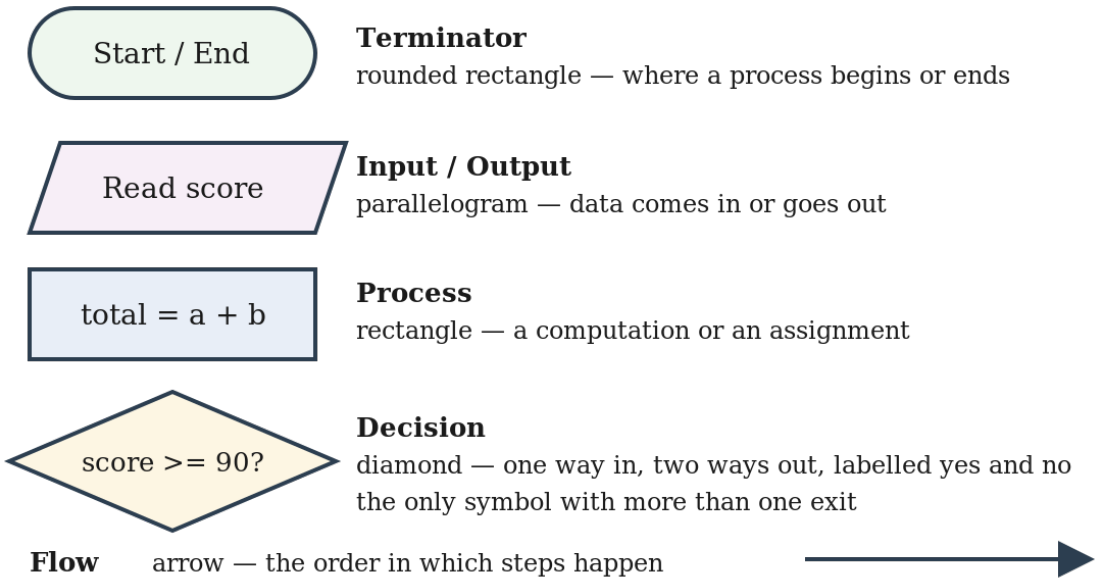


Figure 7: Five flowchart symbols with their names and meanings: a rounded rectangle for terminator, a parallelogram for input or output, a rectangle for process, a diamond for decision, and an arrow for flow.

Figure 5.1 — The five flowchart symbols used in this book.

Description of Figure 5.1. Five symbols are shown with names and meanings. A **rounded rectangle** is a *terminator*, marking where a process begins or ends. A **parallelogram** is *input or output*, where data comes in or goes out. A **rectangle** is a *process*, a computation or an assignment. A **diamond** is a *decision*, with one way in and two ways out labelled yes and no — the only symbol with more than one exit. An **arrow** shows *flow*, the order in which steps happen.

Four rules govern their use:

1. Every flowchart has exactly one **Start** and at least one **End**.
2. Every symbol except Start has at least one arrow in; every symbol except End has at least one arrow out.
3. A decision diamond has exactly **two** exits, and both are labelled.
4. Arrows never trail off. Every path reaches an End.

Rule 4 catches real design errors. A path that leads nowhere on paper is a program that hangs or crashes in practice.

A complete flowchart

Figure 5.2 — Flowchart for computing one assignment grade.

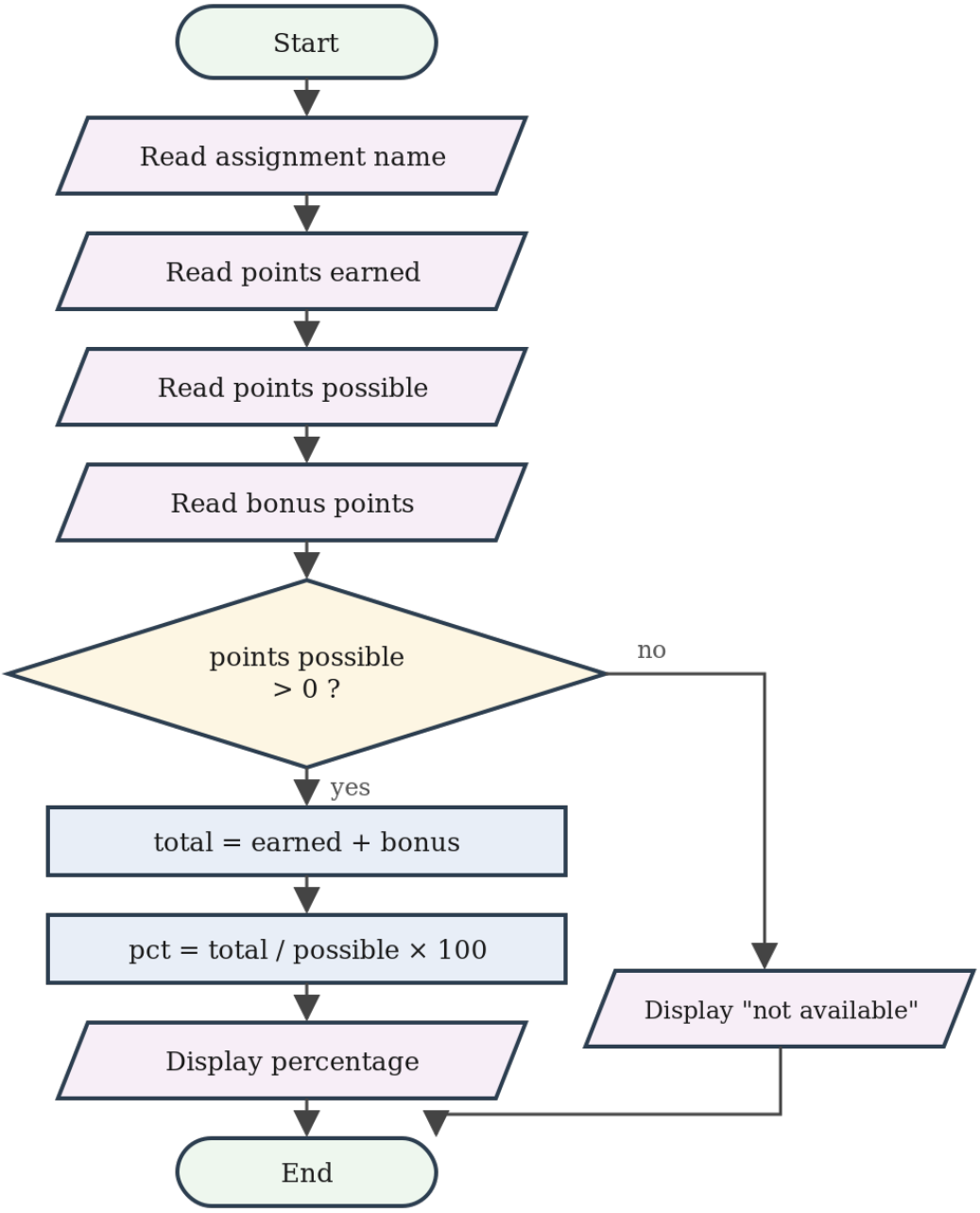


Figure 8: A flowchart with nine steps: Start, read assignment name, read points earned, read points possible, read bonus points, a decision on whether points possible is greater than zero, and two paths that both reach End.

Description of Figure 5.2. Execution begins at **Start**. Four input steps follow in sequence: read the assignment name, read points earned, read points possible, and read bonus points. A **decision** then asks whether points possible is greater than zero.

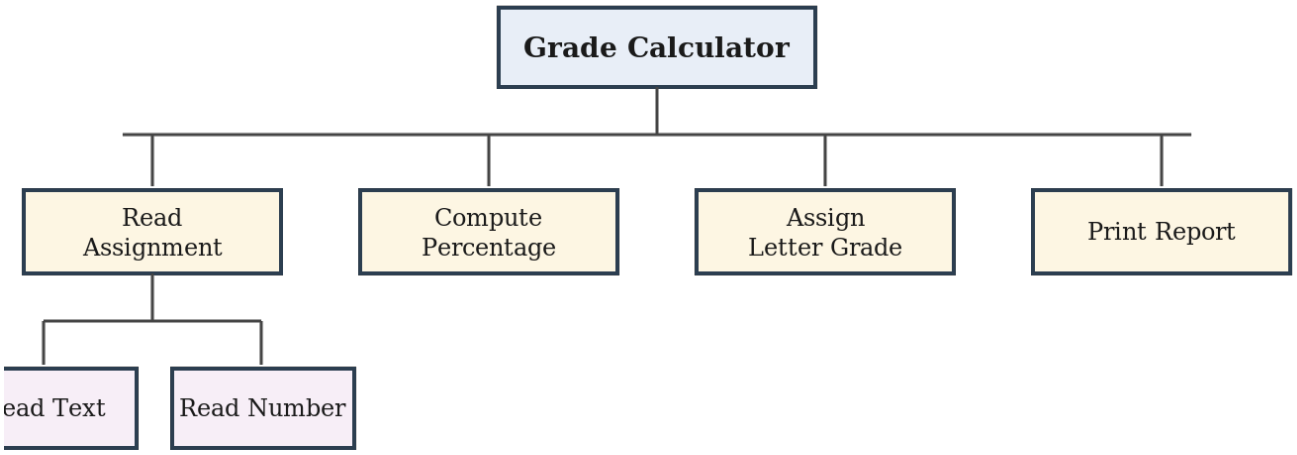
If **no**, flow moves right to an output step displaying "not available", then down and left to **End**.

If **yes**, flow continues down through two process steps — `total = earned + bonus`, then `pct = total / possible × 100` — to an output step displaying the percentage, and then to **End**.

Both paths reach End, satisfying rule 4. Compare this with the pseudocode in Section 5.3: they describe the same algorithm, and the branch that is two indented blocks in pseudocode is two visible paths here.

5.5 Structure Charts and Top-Down Decomposition

A flowchart shows *order*. A **structure chart** shows *composition* — what a program is made of, and which part uses which.



A structure chart shows what the program is made of and which part calls which.
It says nothing about the order in which they run — that is the flowchart's job.

Figure 9: A tree with Grade Calculator at the top, four children below it, and two grandchildren under the leftmost child.

Figure 5.3 — Structure chart for the Grade Calculator.

Description of Figure 5.3. At the top is **Grade Calculator**. Below it are four parts: **Read Assignment**, **Compute Percentage**, **Assign Letter Grade**, and **Print Report**. Below Read Assignment are two further parts: **Read Text** and **Read Number**. The chart shows what the program is made of and which part calls which. It says nothing about the order in which they run — that is the flowchart's job.

Top-down decomposition

A structure chart is built by **top-down decomposition**: start with the whole problem, break it into parts, and break those parts down until each is small enough to write directly.

Ask of each box: *can I describe what this does in one sentence, without using the word "and"?* If yes, it is small enough. If you need "and", it is really two boxes.

"Read an assignment and compute its percentage" is two boxes. "Read an assignment" is one.

This chart is not decoration. In Chapter 9 you will convert each box into a C++ function, and the functions you write there — `readAssignment`, `computePercentage`, `assignLetterGrade`, `printReport` — are exactly these boxes. Designing the structure now means Chapter 9's rebuild is transcription rather than invention.

5.6 Desk Checking and Trace Tables

Before writing code, you can test a design by hand. A **desk check** means walking through your algorithm with specific values and recording what happens at each step, in a **trace table**.

Trace the Section 5.3 pseudocode with points earned 84, points possible 100, bonus 5:

Step	pointsEarned	pointsPossible	bonusPoints	totalEarned	percentage	Output
Read inputs	84	100	5	—	—	prompts
SET totalEarned	84	100	5	89	—	—
IF 100 > 0	84	100	5	89	—	true, take yes
SET percentage	84	100	5	89	89.0	—
DISPLAY	84	100	5	89	89.0	`Percentage: 89.0%`

Now trace the case that broke Chapter 4 — points possible 0:

Step	pointsEarned	pointsPossible	bonusPoints	totalEarned	percentage	Output
Read inputs	0	0	0	—	—	prompts
SET totalEarned	0	0	0	0	—	—
IF 0 > 0	0	0	0	0	—	false, take no
DISPLAY	0	0	0	0	never set	`not available`

The second trace confirms the design never divides by zero. **Always desk-check the boundary cases**, not just the ordinary one — zero, the largest value, the empty input, exactly the cutoff. Chapter 16 turns this instinct into systematic testing, and Chapter 6 will show you that "exactly the cutoff" is where letter grades go wrong.

5.7 Structured Programming

Structured programming is the discipline of building programs from exactly three control structures. Every algorithm can be expressed with them, which is a genuine mathematical result and not a style preference.

Sequence — steps in order, one after another.

```
SET totalEarned = pointsEarned + bonusPoints
SET percentage = totalEarned / pointsPossible * 100
```

Selection — choosing between paths based on a condition. The diamond in a flowchart. Chapter 6.

```
IF pointsPossible > 0 THEN
    ...
ELSE
    ...
END IF
```

Repetition — doing something more than once. Chapter 7.

```
WHILE more assignments remain
    ...
END WHILE
```

Everything you write in Course I is these three, nested inside one another. That is why Chapters 6 and 7 cover selection and repetition and then Course I is essentially complete as a language matter — the remaining chapters organize what you can already express.

The historical significance is worth a sentence. Early languages relied on `goto`, which could jump anywhere and produced programs no one could follow. Structured programming's claim was that three structures suffice, and that constraining yourself to them makes programs readable and checkable. Appendix D Section D.11 excludes `goto` for exactly this reason.

5.8 User Interface Design for Console Programs

Your program's entire interface is text. That is a narrow channel, and narrow channels reward care.

Prompts

A prompt should say what is wanted, and any restriction on it:

```
Points possible:
Points possible (must be greater than 0):
```

The second tells the user the rule *before* they break it. Prompts should end with a space so typed input does not touch the text.

Reports

Aligned columns are scannable; ragged ones are not:

```
Student         Ada Lovelace
Assignment      Midterm Exam
Points earned   84.0
Bonus points    5.0
Total earned    89.0
Points possible 100.0
Percentage      89.0%
```

Group related values, label everything, and use consistent decimal places. `std::setw` from Chapter 4 does the alignment.

Messages when something is missing

Do not print a misleading value. Explain:

```
Percentage      not available
  An assignment worth 0 points has no percentage,
  because a score cannot be divided by zero.
```

A user who sees `0.0%` concludes the student scored zero. A user who sees this understands what happened. This principle scales all the way to Chapter 24's exception messages.

Accessibility in text interfaces

Your output may be read by a screen reader, so:

Do not use layout alone to carry meaning. Label every value. A number in a column with no label means nothing when read aloud.

Do not use color or symbols as the only signal. No ANSI color codes — Appendix D Section D.10 forbids them anyway for portability, and the same rule serves accessibility.

Keep output linear. Text is read top to bottom. Do not build anything that depends on visual scanning.

Use ordinary words in messages. "Points possible must be greater than zero" beats "invalid input: err 3".

5.9 Documenting Your Design

Your design document has five parts. You will revise it repeatedly; keep it in a file beside your code.

Part	Contents
Requirements statement	Plain sentences: what it must do, from Chapter 1
Specification	Precise inputs, processing, outputs, constraints
Pseudocode	The algorithm in structured English
Flowchart	The algorithm as a diagram
Structure chart	What the program is made of

Date every revision and say what changed. By Chapter 24 this document will have a history, and that history is the evidence for SLO 2.1.

Common Design Errors

Error	Why it matters	How to catch it
An ambiguous step	Two builders produce two programs	Read each step aloud — could it mean two things?
A path that reaches no End	The program hangs or crashes	Trace every arrow to an End
A decision with an unlabelled exit	Nobody knows which way is which	Label both exits, always
No boundary cases considered	Fails on zero, empty, or exactly-at-cutoff	Desk-check at least three cases
Pseudocode written in C++	You are coding, not designing	Remove all syntax; describe the process
A structure chart box needing "and"	The box is really two boxes	Split it
A specification that cannot be checked	You cannot tell whether you succeeded	Rewrite until each line is testable

---

Design Notes

Design is not a phase you finish. You will revise this document in Chapter 13, extend it in Chapter 20, and reconcile it in Chapter 24. A design that never changes usually means nobody consulted it.

Record what you decided not to do. The out-of-scope section is what makes weighted grading a scope decision rather than an oversight — a distinction Chapter 13 will make you defend.

Test the design before the code. A desk check costs minutes. Finding the same flaw after implementation costs an afternoon.

Grade Calculator v0.4 — Designed Interface

This chapter produces two deliverables. One is documents. The other is a running program, because every chapter ships one.

Deliverable 1 — the design document

Produce all five parts from Section 5.9 for the full points-based Grade Calculator you will build across Course I. Use the specification in Section 5.2 as your model and extend it to cover:

- a user-defined letter grade scale (arriving in Chapter 11)
- multiple assignments per student (Chapter 7)
- multiple students (Chapter 11)

Your out-of-scope section must still contain the weighted-grading paragraph from Chapter 1, unchanged. You will not resolve it until Chapter 21.

Deliverable 2 — the program

v0.4 computes exactly what v0.3 did. What changes is the interface, applying Section 5.8 to the parts you can already control.

```
// Grade Calculator v0.4 - Chapter 5
// Same computation as v0.3, redesigned interface.
// New this version: prompts state units and ranges, aligned report columns,
// a plain-language message when points possible is zero.
// Run: click Run in StudySite and use the embedded Terminal.

#include <iostream>
#include <iomanip>
#include <string>

int main() {
    std::cout << "====================================\n";
    std::cout << "  GRADE CALCULATOR  v0.4\n";
    std::cout << "  Points-based grading\n";
    std::cout << "====================================\n\n";
    std::cout << "Enter the assignment details below.\n\n";

    std::string studentName;
    std::cout << "Student name               : ";
    std::getline(std::cin, studentName);

    std::string assignmentName;
    std::cout << "Assignment name            : ";
    std::getline(std::cin, assignmentName);

    double pointsEarned = 0.0;
    std::cout << "Points earned (e.g. 42.5)  : ";
    std::cin >> pointsEarned;

    double pointsPossible = 0.0;
```

```
    std::cout << "Points possible (must be >0): ";
    std::cin >> pointsPossible;

    double bonusPoints = 0.0;
    std::cout << "Bonus points (0 for none)   : ";
    std::cin >> bonusPoints;

    double totalEarned = pointsEarned + bonusPoints;

    std::cout << "\n-------------------------------------\n";
    std::cout << "  GRADE REPORT\n";
    std::cout << "-------------------------------------\n";
    std::cout << std::left << std::setw(16) << "Student"    << studentName << "\n";
    std::cout << std::left << std::setw(16) << "Assignment" << assignmentName << "\n";
    std::cout << std::left << std::setw(16) << "Points earned"
              << std::fixed << std::setprecision(1) << pointsEarned << "\n";
    std::cout << std::left << std::setw(16) << "Bonus points" << bonusPoints << "\n";
    std::cout << std::left << std::setw(16) << "Total earned" << totalEarned << "\n";
    std::cout << std::left << std::setw(16) << "Points possible" << pointsPossible << "\n";
    std::cout << "-------------------------------------\n";

    // A zero denominator is explained, not silently reported as 0%.
    if (pointsPossible > 0.0) {
        double percentage = totalEarned / pointsPossible * 100.0;
        std::cout << std::left << std::setw(16) << "Percentage"
                  << percentage << "%\n";
    } else {
        std::cout << "Percentage      not available\n";
        std::cout << "  An assignment worth 0 points has no percentage,\n";
        std::cout << "  because a score cannot be divided by zero.\n";
    }
    std::cout << "-------------------------------------\n";
    return 0;
}
```

Expected output

With input `Ada Lovelace`, `Midterm Exam`, `84`, `100`, `5`:

```
-------------------------------------
  GRADE REPORT
-------------------------------------
Student         Ada Lovelace
Assignment      Midterm Exam
Points earned   84.0
Bonus points    5.0
Total earned    89.0
Points possible 100.0
-------------------------------------
Percentage      89.0%
-------------------------------------
```

With points possible `0`:

```
-------------------------------------
Percentage      not available
  An assignment worth 0 points has no percentage,
  because a score cannot be divided by zero.
-------------------------------------
```

What to notice

Each prompt states its rule. `(must be >0)` and `(0 for none)` tell the user the constraint before they violate it.

Every value is labelled. `std::setw(16)` with `std::left` aligns the labels; the label is what makes the report readable aloud as well as on screen.

The zero case is explained in sentences. Compare with printing `0.0%`, which is not merely unhelpful but actively misleading.

The computation is unchanged from v0.3. Design work is real work even when the arithmetic does not move.

Your StudySite Lab — Design the Interface

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v0.4
- **Starting point**: Your committed Chapter 4 v0.3 program (one file, computes a percentage).

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

This chapter adds no new calculation. The arithmetic at the end is identical to Chapter 4's. What changes is everything around it: the prompts say what units and ranges they expect, the report lines up in columns that stay aligned whatever the values, and the zero-points case explains itself in plain English instead of quietly printing `0.0%`. Documentation and interface design are the chapter's subject, and the exercise is to improve a program's usability without touching what it computes. By the end you will have:

1. A four-line banner naming the application, its version, and its grading model.
2. Five prompts that state their units, an example, or their valid range, with the colons aligned in a column.
3. A `GRADE REPORT` block using `std::left` and `std::setw` so labels and values align regardless of length.
4. An `else` branch that explains *why* there is no percentage when an assignment is worth zero points.

Files in this checkpoint

File	What you do to it
`main.cpp`	Edit (Steps 1-5)

When you are finished your project still has **one** `.cpp` file.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.

- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `main.cpp`. Confirm the editor shows the expected file paths before editing.
4. Continue the existing program in StudySite's internal editor, finish the steps in order, then Run.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Update the comment header

In `main.cpp`, find the five header comment lines:

```
// Grade Calculator v0.3 - Chapter 4
// Adds bonus points and a percentage calculation.
// New this version: floating-point division, output formatting.
// Design note: bonus points are added to points EARNED but not to points
// POSSIBLE, so a percentage above 100 is possible. v0.3 reports it uncapped.
```

Replace it with:

```
// Grade Calculator v0.4 - Chapter 5
// Same computation as v0.3, redesigned interface.
// New this version: prompts state units and ranges, aligned report columns,
// a plain-language message when points possible is zero.
```

Why: The design note about uncapped percentages goes away because this version no longer reports a bare percentage in the zero case — the new `else` branch in Step 5 replaces that behavior with an explanation. A header comment is only useful while it is true, so a note that no longer describes the code is worse than no note at all. The replacement records what this version actually changed: the interface, not the computation.

Step 2 — Replace the one-line banner with a titled block

In `main.cpp`, find the banner line:

```
    std::cout << "=== GRADE CALCULATOR v0.3 ===\n\n";
```

Replace it with these five lines:

```
    std::cout << "====================================\n";
    std::cout << "  GRADE CALCULATOR  v0.4\n";
    std::cout << "  Points-based grading\n";
    std::cout << "====================================\n\n";
    std::cout << "Enter the assignment details below.\n\n";
```

Why: A program's first screen should answer "what is this and what does it want from me". The banner names the application and the grading model, and the line below it tells the user what is about to happen. The `\n\n` on the closing rule and on the instruction line creates breathing room, so the prompts that follow are visually separate from the header rather than crowding it.

Step 3 — Rewrite the five prompts

In `main.cpp`, find the student-name prompt:

```
    std::cout << "Student name: ";
```

Replace it with:

```
    std::cout << "Student name               : ";
```

Find the assignment-name prompt:

```
    std::cout << "Assignment name: ";
```

Replace it with:

```
    std::cout << "Assignment name            : ";
```

Find the points-earned prompt:

```
    std::cout << "Points earned: ";
```

Replace it with:

```
    std::cout << "Points earned (e.g. 42.5)  : ";
```

Find the points-possible prompt:

```
    std::cout << "Points possible: ";
```

Replace it with:

```
    std::cout << "Points possible (must be >0): ";
```

Find the bonus-points prompt:

```
    std::cout << "Bonus points (0 if none): ";
```

Replace it with:

```
    std::cout << "Bonus points (0 for none)  : ";
```

Why: Each prompt now carries the information the user needs *before* typing, not after getting it wrong. `(e.g. 42.5)` shows that decimals are accepted. `(must be >0)` states the constraint the Step 5 guard enforces. `(0 for none)` removes the ambiguity about the empty case.

The colons are also padded to the same column, which is why each prompt string has a different number of trailing spaces. Aligned prompts are easier to scan and make the program look deliberate rather than assembled. Count the spaces carefully when you copy — the alignment is the point of the step.

> ▸ **Run checkpoint**: Click **Run**. The banner and all five prompts should look new, but the report at the end is still Chapter 4's `--- Summary ---` block. That is next.

Step 4 — Replace the summary with an aligned report

In `main.cpp`, find this comment:

```
    // Both operands are double, so this is real division, not integer division.
```

Delete that line and everything below it, down to and including this line:

```
    double percentage = 0.0;
```

In the space where those two lines were, enter:

```
    std::cout << "\n-------------------------------------\n";
    std::cout << "  GRADE REPORT\n";
    std::cout << "-------------------------------------\n";
    std::cout << std::left << std::setw(16) << "Student"    << studentName << "\n";
    std::cout << std::left << std::setw(16) << "Assignment" << assignmentName << "\n";
    std::cout << std::left << std::setw(16) << "Points earned"
              << std::fixed << std::setprecision(1) << pointsEarned << "\n";
    std::cout << std::left << std::setw(16) << "Bonus points" << bonusPoints << "\n";
    std::cout << std::left << std::setw(16) << "Total earned" << totalEarned << "\n";
    std::cout << std::left << std::setw(16) << "Points possible" << pointsPossible << "\n";
    std::cout << "-------------------------------------\n";

    // A zero denominator is explained, not silently reported as 0%.
```

Why: This is the chapter's main technique. `std::setw(16)` sets the **field width** for the next value only: the label is printed into a sixteen-character space. `std::left` makes it left-aligned within that space instead of right-aligned, which is the default. Together they guarantee the values all start at column 17, no matter whether the label is `Student` or `Points possible`.

`std::setw` applies to exactly one value and then resets, which is why it is repeated on every line — unlike `std::fixed` and `std::setprecision`, which are sticky and so are set once, on the first numeric line. That asymmetry between sticky and one-shot manipulators is worth remembering; it is a common source of confusion.

Notice the report also breaks `Total earned` out as its own row rather than showing the arithmetic inline the way Chapter 4 did. In a column layout, one value per row reads better than a parenthetical.

Step 5 — Explain the zero-points case

In `main.cpp`, find the line that computes the percentage inside the `if`:

```
        percentage = totalEarned / pointsPossible * 100.0;
```

Replace that single line with this:

```
        double percentage = totalEarned / pointsPossible * 100.0;
        std::cout << std::left << std::setw(16) << "Percentage"
                  << percentage << "%\n";
    } else {
        std::cout << "Percentage      not available\n";
        std::cout << "  An assignment worth 0 points has no percentage,\n";
        std::cout << "  because a score cannot be divided by zero.\n";
```

Now find the old summary block, which the new report has replaced, and delete all of it:

```
    std::cout << "\n--- Summary ---\n";
    std::cout << "Student:    " << studentName << "\n";
    std::cout << "Assignment: " << assignmentName << "\n";
    std::cout << "Earned:     " << totalEarned << " (" << pointsEarned
              << " + " << bonusPoints << " bonus)\n";
    std::cout << "Possible:   " << pointsPossible << "\n";
    std::cout << "Percentage: " << std::fixed << std::setprecision(1)
              << percentage << "%\n";
```

Finally, the report needs a closing rule below the percentage. Find the end of `main`:

```
    return 0;
```

Directly above it, add:

```
    std::cout << "-----------------------------------\n";
```

Why: Chapter 4 guarded against division by zero but reported the result as `0.0%`, which is a lie: an assignment worth zero points does not have a percentage of zero, it has no percentage at all. Saying `not available` and then explaining why is honest, and it teaches the user something instead of leaving them to wonder why their score vanished.

Note also that `percentage` is now declared *inside* the `if` block, rather than before it. It is only needed there, and a variable declared in the smallest scope that needs it cannot be accidentally used — or accidentally trusted — anywhere else. Chapter 4 had to declare it outside and initialize it to `0.0` precisely because the old code printed it unconditionally; now that printing happens only where the value exists, the initialization is unnecessary.

▸ **Run checkpoint**: Click **Run** and work through the test plan below in the Terminal.

Test plan

1. Normal case. `Ada Lovelace`, `Analytical Engine Essay`, `42.5`, `50`, `2.5`:

```
-----------------------------------
  GRADE REPORT
-----------------------------------
Student         Ada Lovelace
Assignment      Analytical Engine Essay
Points earned   42.5
Bonus points    2.5
Total earned    45.0
Points possible 50.0
-----------------------------------
Percentage      90.0%
-----------------------------------
```

Check that every value starts in the same column, including after the long label `Points possible`.

2. Zero points possible. Enter `0` for points possible. The report should end with the `not available` message and its two explanatory lines, and no percentage.

3. A long name. Enter a student name of twenty or more characters. The label column must stay sixteen wide and the value must still start at column 17 — `setw` pads short labels but never truncates long values.

Check your work

Then confirm:

- ☐ `main.cpp` compiles with no errors and **no warnings**.
- ☐ Every prompt states a unit, an example, or a valid range.
- ☐ The prompt colons line up in a single column.
- ☐ All report values start in the same column, for both short and long labels.
- ☐ Entering `0` for points possible prints the `not available` explanation and no percentage.
- ☐ The computed percentage is identical to what Chapter 4 produced for the same inputs — the interface changed, the arithmetic did not.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 5 Grade Calculator v0.4**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 5 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

These are design exercises. Only the last needs a compiler.

1. Ambiguous or unambiguous?

For each, say whether it could be an algorithm step. Rewrite the bad ones.

- Add the bonus points to the points earned.
- Give partial credit if the answer is close.
- If the percentage is 90 or above, the letter grade is A.
- Handle invalid input appropriately.
- Divide the total points earned by the total points possible.
- Round the result nicely.

2. Pseudocode for a letter grade

Write pseudocode that takes a percentage and displays a letter grade: A for 90 and above, B for 80–89.9, C for 70–79.9, D for 60–69.9, F below 60.

Then desk-check it with 95, 90, 89.9, 60, and 59.9. **The value 90 is the interesting one** — does your pseudocode give it an A or a B? Chapter 6 shows that this exact boundary is where letter-grade code most often goes wrong.

3. Flowchart the same thing

Draw a flowchart for your Exercise 2 pseudocode. Use the five symbols from Figure 5.1, label both exits of every decision, and verify every path reaches End.

How many decisions did you need for five letter grades? Why not five?

4. Decompose a problem

Draw a structure chart for a program that reads a class roster from a file, computes each student's grade, sorts by grade, and prints a report.

Aim for three levels. Check each box against the one-sentence-without-"and" test.

5. Trace a design flaw

This pseudocode is wrong. Find the error by desk-checking it with a percentage of 85.

```
BEGIN
    READ percentage
    IF percentage >= 60 THEN
        SET letter = "D"
    ELSE IF percentage >= 70 THEN
        SET letter = "C"
    ELSE IF percentage >= 80 THEN
        SET letter = "B"
    ELSE IF percentage >= 90 THEN
        SET letter = "A"
    ELSE
        SET letter = "F"
    END IF
    DISPLAY letter
END
```

What letter does 85 get? What should it get? What is wrong, and what is the smallest fix?

6. Rewrite a bad interface

This program's output is technically correct and hard to use:

```
Enter:
84
Enter:
100
Result: 84
```

Rewrite the prompts and the report following Section 5.8. State what each change improves.

7. Specify before you build

Write a specification — inputs with constraints, processing, outputs, and out-of-scope — for a program that computes the average of several test scores and reports whether the student passed, where passing is 60%.

Then answer: what should it do if no scores are entered at all? Your specification must say. That question is the whole point of the exercise.

Summary

- An **algorithm** is a finite sequence of unambiguous, effective steps that solves a problem correctly for **every** valid input.
- A **requirements statement** says what a program must do. A **specification** says it precisely enough to build from and test against.

- **Pseudocode** describes an algorithm in structured English. Indent to show structure; describe the process, not the language.
- A **flowchart** uses five symbols: terminator, input/output, process, decision, and flow. Decisions have exactly two labelled exits, and every path must reach an End.
- A **structure chart** shows what a program is made of and which part calls which — composition, not order. It is built by **top-down decomposition**, and in Chapter 9 each box becomes a function.
- A **desk check** with a **trace table** tests a design by hand. Always trace the boundary cases.
- **Structured programming** builds everything from **sequence**, **selection**, and **repetition**.
- Console interface design: prompts state their constraints, reports use aligned labelled columns, missing values are explained rather than faked, and output stays plain, linear, and labelled so a screen reader can convey it fully.
- The design document has five parts, and it gets revised — in Chapters 13, 20, and 24.

Key Terms

algorithm — a finite sequence of unambiguous, effective steps that solves a problem.

decision — a flowchart symbol, drawn as a diamond, with one entry and two labelled exits.

desk check — testing an algorithm by hand with specific values.

flowchart — a diagram showing the steps of an algorithm and the order in which they run.

process — a flowchart symbol, drawn as a rectangle, representing a computation or assignment.

pseudocode — a description of an algorithm in structured English.

repetition — a control structure that performs steps more than once.

requirements statement — a plain-language description of what a program must do.

selection — a control structure that chooses between paths based on a condition.

sequence — a control structure in which steps happen one after another in order.

specification — a precise, checkable description of inputs, processing, and outputs.

structure chart — a diagram showing what a program is composed of and which part calls which.

structured programming — building programs from sequence, selection, and repetition only.

terminator — a flowchart symbol, drawn as a rounded rectangle, marking a start or end.

top-down decomposition — breaking a problem into parts, and those parts into smaller parts.

trace table — a table recording the value of each variable at each step of a desk check.

Next: Chapter 6 gives you the C++ for **selection** — the diamond in your flowchart. You will write the letter-grade logic you designed here, and discover exactly what goes wrong at a boundary such as 90.0. Grade Calculator v0.5.

Chapter 6 — Conditional Execution

Learning Objectives

When you finish this chapter you will be able to:

- Write Boolean expressions using the relational operators. *(SLO 1.3)*
- Use `if`, `if/else`, and multi-way `if/else if` chains to choose between paths. *(SLO 1.3, 1.4)*
- Combine conditions with `&&`, `||`, and `!`, and explain short-circuit evaluation. *(SLO 1.3)*
- Explain how a block determines the scope of a variable. *(SLO 1.3)*
- Write a `switch` statement and say when it is a better choice than an `if/else if` chain. *(SLO 1.3)*
- Identify and fix the common faults in conditional logic, including boundary errors and floating-point comparison. *(SLO 1.6, 1.7)*
- Build Grade Calculator v0.5, and explain why a percentage that displays as 90.0 may still receive a B.

6.1 The `bool` Type Revisited

Chapter 5 identified **selection** as one of the three structures of structured programming, and drew it as a diamond. This chapter is the C++ for that diamond.

Everything rests on `bool`, introduced in Section 3.7. A `bool` holds exactly `true` or `false`.

```
bool capAt100 = true;
bool dropLowest = false;
```

Printing a `bool` shows `1` or `0` by default. `std::boolalpha` makes it readable, which is useful while you are learning:

```
std::cout << std::boolalpha;
std::cout << (5 > 3) << "\n";      // true
```

6.2 Relational Operators and Boolean Expressions

A **relational operator** compares two values and produces a `bool`.

Operator	Meaning	Example	Result
`>`	greater than	`5 > 3`	`true`
`<`	less than	`5 < 3`	`false`
`>=`	greater than or equal to	`5 >= 5`	`true`
`<=`	less than or equal to	`3 <= 5`	`true`

Operator	Meaning	Example	Result
`==`	equal to	`5 == 3`	`false`
`!=`	not equal to	`5 != 3`	`true`

An expression producing a `bool` is a **Boolean expression** — the *condition* a selection tests.

> **`=` and `==` are different, and confusing them is legal C++.** A single `=` assigns; a double `==` compares. `if (x = 3)` stores 3 in `x` and is always true. Compiling with `-Wall` warns about this, which is one more reason Appendix D Section D.10 requires you to fix warnings.

6.3 The Simple `if` Statement

```
if (condition) {
    // runs only when condition is true
}
```

```
if (pointsPossible > 0.0) {
    percentage = totalEarned / pointsPossible * 100.0;
}
```

You have used this since Chapter 4 to guard a division. Now it is formal.

The parentheses around the condition are required. The braces are not, strictly — but Appendix D Section D.1 requires them anyway, and the reason is worth restating:

```
// Do not write this.
if (percentage >= 90.0)
    letter = 'A';
```

The day someone adds a second line to that body and forgets to add braces, the second line runs unconditionally, the program still compiles, and the resulting logic error is invisible. Braces from the start make it impossible.

> **A semicolon after the condition is a silent disaster.** `if (x > 5);` is legal: the `;` is an empty statement forming the entire body, so the block that follows always runs. The compiler accepts it. Section 6.12 returns to this.

6.4 Compound Statements and Scope

A **block** is statements enclosed in braces, treated as one statement.

A block also defines **scope** — the region where a name exists:

```
if (pointsPossible > 0.0) {
    double percentage = totalEarned / pointsPossible * 100.0;
    std::cout << percentage << "\n";      // fine
}
std::cout << percentage << "\n";          // error: does not exist here
```

`percentage` is created on entering the block and destroyed on leaving it. Using it outside is a compile-time error.

The fix is to declare it where it needs to live:

```
double percentage = 0.0;
if (pointsPossible > 0.0) {
    percentage = totalEarned / pointsPossible * 100.0;
}
std::cout << percentage << "\n";          // fine
```

Declare a variable in the smallest scope that will hold it. Appendix D Section D.6 puts it this way: a variable that exists longer than it means anything is an opportunity to misuse it.

6.5 The `if/else` Statement

```
if (pointsPossible > 0.0) {
    std::cout << "Percentage: " << percentage << "%\n";
} else {
    std::cout << "Percentage: not available\n";
}
```

Exactly one branch runs — never both, never neither. This is the flowchart diamond of Figure 5.1 with both exits labelled.

6.6 Logical Operators and Compound Conditions

Three operators combine Boolean expressions.

Operator	Name	True when
`&&`	AND	both operands are true
`||`	OR	at least one operand is true
`!`	NOT	the operand is false

```
if (percentage >= 80.0 && percentage < 90.0) {
    letter = 'B';
}

if (pointsPossible <= 0.0 || pointsEarned < 0.0) {
    std::cout << "Invalid assignment data.\n";
}

if (!scaleIsValid) {
    std::cout << "Please define a grade scale first.\n";
}
```

Precedence

`!` binds tightest, then `&&`, then `||`. All relational operators bind tighter than any of them, which is why this works without parentheses:

```
if (percentage >= 80.0 && percentage < 90.0)
```

Both comparisons happen before the `&&`. Where a mix of `&&` and `||` appears, add parentheses — Appendix D asks for clarity over minimalism:

```
if ((a && b) || c)      // clearer than a && b || c
```

The chained comparison trap

Mathematics writes `0 ≤ score ≤ 100`. C++ does not work that way, and the result is not what you would guess:

```
if (0 <= score <= 100)      // WRONG - and always true
```

C++ evaluates left to right. `0 <= score` produces a `bool`, which converts to 1 or 0. Then `1 <= 100` is compared — and that is true no matter what `score` was. A score of 150 or −50 both pass.

Compiling with `-Wall` catches it:

```
warning: comparison of constant '100' with boolean expression is always true
         [-Wbool-compare]
```

The correct form states both comparisons:

```
if (0 <= score && score <= 100)
```

6.7 Short-Circuit Evaluation

C++ stops evaluating a compound condition as soon as the answer is settled. This is **short-circuit evaluation**.

For `&&`, if the left operand is false, the right is never evaluated — the result is already false. For `||`, if the left is true, the right is never evaluated.

This is not an optimization detail you can ignore. It is a tool:

```
if (pointsPossible > 0.0 && totalEarned / pointsPossible > 0.9) {
    // ...
}
```

If `pointsPossible` is zero, the left operand is false and **the division never happens**. Reverse the order and the program divides by zero.

Order your conditions so the check that makes the rest safe comes first. This pattern appears throughout the Grade Calculator, and it is the reason a great many potential run-time errors never occur.

6.8 Nested Conditionals

An `if` inside another `if`:

```
if (pointsPossible > 0.0) {
    if (totalEarned > pointsPossible) {
        std::cout << "Note: bonus points pushed this above 100%.\n";
    } else {
        std::cout << "Score is within the normal range.\n";
    }
} else {
    std::cout << "No percentage is available.\n";
}
```

Nesting is sometimes necessary and quickly becomes hard to read. Two levels is usually fine. Three is a signal to reconsider — either combine conditions with `&&`, or extract the inner logic into a function once you reach Chapter 9.

6.9 Multi-way `if/else if` Chains

For more than two mutually exclusive outcomes, chain the tests:

```
char letter = 'F';
if (percentage >= 90.0) {
    letter = 'A';
} else if (percentage >= 80.0) {
    letter = 'B';
} else if (percentage >= 70.0) {
    letter = 'C';
} else if (percentage >= 60.0) {
    letter = 'D';
}
```

The first true condition wins, and the rest are skipped. That is what makes this correct despite each condition looking incomplete. A percentage of 95 satisfies `>= 90`, gets an `A`, and never reaches the `>= 80` test.

Order matters absolutely

Reverse the chain and it breaks completely:

```
// WRONG
if (percentage >= 60.0) {
    letter = 'D';
} else if (percentage >= 70.0) {
    letter = 'C';
    // ...
}
```

Now 95 satisfies `>= 60` first and gets a `D`. Nothing below is ever reached. This is Exercise 5 from Chapter 5, and it compiles without complaint — a logic error of exactly the kind Section 4.9.3 warned about.

Order a chain from most restrictive to least. For grade cutoffs, that means highest first.

Why `else` matters here

Without `else`, each `if` is independent and every test runs:

```
// WRONG
if (percentage >= 90.0) { letter = 'A'; }
if (percentage >= 80.0) { letter = 'B'; }
if (percentage >= 70.0) { letter = 'C'; }
```

A 95 sets `letter` to `A`, then overwrites it with `B`, then `C`. Everyone gets a C. The `else` is what makes the branches mutually exclusive.

6.10 The `switch` Statement

When you compare one value against a list of **exact** constants, `switch` is often clearer:

```
switch (letter) {
    case 'A':
        std::cout << "Excellent\n";
        break;
    case 'B':
        std::cout << "Good\n";
        break;
    case 'C':
        std::cout << "Satisfactory\n";
        break;
    default:
        std::cout << "Needs improvement\n";
        break;
}
```

`break` exits the switch. Without it, execution **falls through** into the next case and keeps going — a real source of bugs, and occasionally useful when several cases share behavior:

```
switch (letter) {
    case 'A':
    case 'B':
        std::cout << "Passing with distinction\n";
        break;
    default:
        std::cout << "Other\n";
        break;
}
```

When `switch` will not work

`switch` requires **integral** values — `int`, `char`, `bool`, or an enumeration — compared against compile-time constants. It cannot test ranges and cannot switch on a `double`.

So this is impossible:

```
switch (percentage) {          // error: percentage is a double
    case >= 90.0:              // error: cases must be constants, not ranges
```

Grade cutoffs are ranges over a `double`, so letter-grade logic must use an `if/else if` chain. `switch` earns its place for menu choices, where you compare one `char` against a fixed list — which is exactly how the Grade Calculator uses it from Chapter 9 onward.

6.11 The Conditional Operator

For choosing between two *values*, C++ offers a compact form:

```
condition ? valueIfTrue : valueIfFalse
```

```
double reported = capAt100 ? std::min(raw, 100.0) : raw;
```

That line appears in Grade Calculator v0.7 and expresses the whole cap policy in one readable statement.

Use it for short value selections. Do not use it to choose between actions, and never nest it — an `if/else` is clearer and Appendix D prefers clarity.

6.12 Errors in Conditional Statements

Six faults account for most conditional bugs. Four are silent.

= instead of ==

```
if (letter = 'A') {     // assigns, then is always true
```

Caught by `-Wall`. Fix your warnings.

A stray semicolon

```
if (percentage >= 90.0);
{
    letter = 'A';       // always runs
}
```

The `;` is the entire body. The block that follows is unconditional. **Silent.**

Reversed chain order

Covered in Section 6.9. Everyone gets a D. **Silent.**

Missing `else`

Also Section 6.9. Later assignments overwrite earlier ones. **Silent.**

Comparing floating-point values with ==

```
double a = 0.1 + 0.2;
if (a == 0.3) { ... }      // false
```

```
0.1+0.2 == 0.3 ?      false
0.1+0.2 is exactly 0.30000000000000004
```

Chapters 1 and 3 warned about this; here is where it bites. **Never test floating-point values with == or `!=`.** Compare with <, >, <=, >=, or against a tolerance — Chapter 9 shows the standard technique. **Silent.**

Boundary errors — and one worth dwelling on

Does a percentage of exactly 90.0 earn an A or a B? With `>= 90.0`, an A. With `> 90.0`, a B. Both compile. Only one matches your policy.

Now the case that catches nearly everyone. Consider a student whose percentage is 89.95:

```
double percentage = 89.95;
std::cout << std::fixed << std::setprecision(1);
std::cout << "Percentage: " << percentage << "\n";
std::cout << "Gets an A?  " << std::boolalpha << (percentage >= 90.0) << "\n";
```

```
Percentage: 90.0
Gets an A?  false
```

The report says 90.0 and the student gets a B.

Nothing here is broken. `setprecision(1)` rounds *for display only* — it does not change the stored value, which is still 89.95 and genuinely less than 90. But try explaining that to the student holding a printout that says 90.0.

This is a design decision disguised as a formatting detail, and you must choose:

1. **Grade the stored value.** 89.95 is a B, and the report should show more decimal places so the number matches the grade.
2. **Grade the rounded value.** Round to one decimal place *first*, then compare — so the displayed 90.0 is the number actually graded.

Chapter 8 implements option 2 with `std::round`, and from v0.7 onward the Grade Calculator rounds once and grades what it displays. It is worth noticing that the fix belongs in the design, not in the comparison.

Common Errors and Warnings

What you see or observe	Cause	Fix
`warning: suggest parentheses around assignment used as truth value`	`=` where `==` was meant	Use `==`
`warning: comparison ... is always true [-Wbool-compare]`	Chained comparison such as `0 <= x <= 100`	Write `0 <= x && x <= 100`
A branch always runs	Stray `;` after the condition	Delete the semicolon
Everyone gets the same grade	Chain ordered least-restrictive first	Order highest cutoff first
Later branches overwrite earlier ones	Missing `else` between tests	Chain with `else if`
Two equal-looking values compare unequal	`==` on floating-point values	Use `<`, `>`, or a tolerance
`error: 'x' was not declared in this scope`	Used a variable outside its block	Declare it in an enclosing scope
`error: switch quantity not an integer`	Switched on a `double`	Use an `if`/`else if` chain
A case runs and the next one also runs	Missing `break`	Add `break` to each case

---

Design Notes

Order conditions so the safe one comes first. Short-circuit evaluation makes `possible > 0.0 && earned / possible > 0.9` safe. The reverse order divides by zero.

Use `else if` for mutually exclusive outcomes. Independent `if` statements let more than one branch fire, which is almost never what a grade scale means.

Decide your boundaries explicitly and write them down. Is 90.0 an A? Is 89.95? Your specification from Chapter 5 should answer both. If it does not, revise it now.

Test at the boundary, on both sides. For a cutoff of 90, test 89.9, 90.0, and 90.1. Chapter 16 makes this systematic; Chapter 5's desk-check habit makes it early.

Grade Calculator v0.5 — Letter Grades

What v0.5 does

Everything v0.4 did, plus a letter grade from a fixed scale: A at 90, B at 80, C at 70, D at 60, F below.

The program

```
// Grade Calculator v0.5 - Chapter 6
// Adds a letter grade from a fixed 90/80/70/60 scale.
// New this version: multi-way if/else if chain, named constants.
// Limitation: the cutoffs are hard-coded. Chapter 11 makes them user-defined.
// Run: click Run in StudySite and use the embedded Terminal.

#include <iostream>
#include <iomanip>
#include <string>

// Named constants: changing a cutoff means editing one line, not hunting
// for a magic number inside the conditional chain.
const double A_CUTOFF = 90.0;
const double B_CUTOFF = 80.0;
const double C_CUTOFF = 70.0;
const double D_CUTOFF = 60.0;

int main() {
    std::cout << "=== GRADE CALCULATOR v0.5 ===\n\n";

    std::string studentName;
    std::cout << "Student name               : ";
    std::getline(std::cin, studentName);

    std::string assignmentName;
    std::cout << "Assignment name            : ";
    std::getline(std::cin, assignmentName);

    double pointsEarned = 0.0;
    std::cout << "Points earned              : ";
    std::cin >> pointsEarned;

    double pointsPossible = 0.0;
    std::cout << "Points possible (must be >0): ";
    std::cin >> pointsPossible;

    double bonusPoints = 0.0;
    std::cout << "Bonus points (0 for none)  : ";
    std::cin >> bonusPoints;

    double totalEarned = pointsEarned + bonusPoints;

    std::cout << "\n--- GRADE REPORT ---\n";
    std::cout << "Student:    " << studentName << "\n";
    std::cout << "Assignment: " << assignmentName << "\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Score:      " << totalEarned << " / " << pointsPossible << "\n";

    if (pointsPossible > 0.0) {
        double percentage = totalEarned / pointsPossible * 100.0;

        // Order matters: the first true condition wins, so cutoffs are
```

```
        // tested from highest to lowest.
        char letter = 'F';
        if (percentage >= A_CUTOFF) {
            letter = 'A';
        } else if (percentage >= B_CUTOFF) {
            letter = 'B';
        } else if (percentage >= C_CUTOFF) {
            letter = 'C';
        } else if (percentage >= D_CUTOFF) {
            letter = 'D';
        }

        std::cout << "Percentage: " << percentage << "%\n";
        std::cout << "Grade:      " << letter << "\n";
        if (percentage > 100.0) {
            std::cout << "Note: bonus points pushed this above 100%.\n";
        }
    } else {
        std::cout << "Percentage: not available (0 points possible)\n";
        std::cout << "Grade:      not available\n";
    }
    return 0;
}
```

Expected output

With `Ada Lovelace`, `Midterm Exam`, `84`, `100`, `5`:

```
--- GRADE REPORT ---
Student:    Ada Lovelace
Assignment: Midterm Exam
Score:      89.0 / 100.0
Percentage: 89.0%
Grade:      B
```

What to notice

The cutoffs are named constants. Changing the A cutoff to 93 is one edit. If the numbers were written inline, it would be a search through the chain — and Appendix D Section D.6 calls those magic numbers for exactly this reason.

`letter` starts at `'F'` and the chain has no final `else`. Anything failing every test keeps the initial value. This is deliberate: the F case needs no test, because F *is* the default.

The chain runs highest to lowest. Reverse it and everyone passes with a D.

Bonus above 100 is reported, not hidden. The note tells the user why an unusual number appeared.

The limitation

Every cutoff is compiled into the program. An instructor wanting a plus-minus scale, or a pass/fail scale, cannot have one without editing and rebuilding.

Look at the chain and ask: *what would it take to let the user define their own scale?* With what you know now, the answer is a longer chain — and a longer chain still fixed at compile time. There is no arrangement of `if/else if` that accepts a scale from the user.

That is not a failure of your code. It is a limit of the tool, and Chapter 11 provides the missing one. The cutoffs will become **data** rather than code, the chain will collapse into a short loop, and the same program will handle any scale. Keep this chain in mind; you will delete it with some satisfaction.

Your StudySite Lab — Assign Letter Grades

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v0.5
- **Starting point**: Your committed Chapter 5 v0.4 program (one file, aligned report).

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

A percentage is a number; a grade is a judgment. This chapter adds the judgment: a four-way decision that turns the percentage into a letter using a fixed 90/80/70/60 scale. The technique is the `if/else if` chain, and the thing to understand about it is that **order matters** — the first true condition wins and the rest are never tested. By the end you will have:

1. Four named constants at file scope holding the grade cutoffs.
2. A `char letter` variable set by an `if/else if` chain testing the cutoffs from highest to lowest.
3. `Percentage` and `Grade` lines in the report, plus a note when bonus points push the score above 100%.
4. A simplified report layout, so the two new rows fit without the block growing unwieldy.

   Limitation, by design: the cutoffs are hard-coded in this version. That is a real constraint, not an oversight — Chapter 11 replaces them with a scale the user defines at run time.

Files in this checkpoint

File	What you do to it
`main.cpp`	Edit (Steps 1-6)

When you are finished your project still has **one** `.cpp` file.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `main.cpp`. Confirm the editor shows the expected file paths before editing.
4. Continue the existing program in StudySite's internal editor, finish the steps in order, then Run.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Update the comment header

In `main.cpp`, find the four header comment lines:

```
// Grade Calculator v0.4 - Chapter 5
// Same computation as v0.3, redesigned interface.
// New this version: prompts state units and ranges, aligned report columns,
// a plain-language message when points possible is zero.
```

Replace it with:

```
// Grade Calculator v0.5 - Chapter 6
// Adds a letter grade from a fixed 90/80/70/60 scale.
// New this version: multi-way if/else if chain, named constants.
// Limitation: the cutoffs are hard-coded. Chapter 11 makes them user-defined.
```

Why: The `Limitation:` line is the important addition. Recording a known constraint, and where it gets fixed, is how you keep a limitation from being mistaken for a bug by the next person to read the file — including you, three chapters from now.

Step 2 — Add the grade cutoffs as named constants

In `main.cpp`, find the last include line:

```
#include <string>
```

Directly below it, add:

```
// Named constants: changing a cutoff means editing one line, not hunting
// for a magic number inside the conditional chain.
const double A_CUTOFF = 90.0;
const double B_CUTOFF = 80.0;
const double C_CUTOFF = 70.0;
const double D_CUTOFF = 60.0;
```

Why: `90.0` written directly inside an `if` is a **magic number**: a value whose meaning depends on where it appears. `A_CUTOFF` says what it is. The practical payoff is that changing the scale means editing one line instead of searching the conditional chain for every occurrence, and getting it wrong in one place is no longer possible.

`const` tells the compiler the value never changes, so an accidental assignment becomes a compile error rather than a silent bug. These are declared *outside* `main`, at file scope, because in Chapter 9 the grading logic moves into its own function and will need to see them too.

Step 3 — Simplify the banner

In `main.cpp`, find the five banner lines:

```
    std::cout << "====================================\n";
    std::cout << "  GRADE CALCULATOR  v0.4\n";
    std::cout << "  Points-based grading\n";
    std::cout << "====================================\n\n";
    std::cout << "Enter the assignment details below.\n\n";
```

Replace all five with this single line:

```
    std::cout << "=== GRADE CALCULATOR v0.5 ===\n\n";
```

Then find the points-earned prompt and shorten it:

```
    std::cout << "Points earned (e.g. 42.5)   : ";
```

Replace it with:

```
    std::cout << "Points earned               : ";
```

Why: The report is about to gain two rows and a conditional note. Trimming the header and the longest prompt keeps the whole interaction on one screen, which matters more for usability than a decorative banner does. The prompt colons stay aligned — check that your edit preserves the column.

Step 4 — Simplify the report block

In `main.cpp`, find the eleven-line report block:

```
    std::cout << "\n-------------------------------------\n";
    std::cout << "  GRADE REPORT\n";
    std::cout << "-------------------------------------\n";
    std::cout << std::left << std::setw(16) << "Student"    << studentName << "\n";
    std::cout << std::left << std::setw(16) << "Assignment" << assignmentName << "\n";
    std::cout << std::left << std::setw(16) << "Points earned"
              << std::fixed << std::setprecision(1) << pointsEarned << "\n";
    std::cout << std::left << std::setw(16) << "Bonus points" << bonusPoints << "\n";
    std::cout << std::left << std::setw(16) << "Total earned" << totalEarned << "\n";
    std::cout << std::left << std::setw(16) << "Points possible" << pointsPossible << "\n";
    std::cout << "-------------------------------------\n";
```

Replace all eleven lines with these five:

```
    std::cout << "\n--- GRADE REPORT ---\n";
    std::cout << "Student:    " << studentName << "\n";
    std::cout << "Assignment: " << assignmentName << "\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Score:      " << totalEarned << " / " << pointsPossible << "\n";
```

Then find this comment and delete it:

```
    // A zero denominator is explained, not silently reported as 0%.
```

Why: The `std::setw` column layout from Chapter 5 is replaced by a compact label style, and the six separate point rows collapse into one `Score:` line showing total over possible. The grading result, not the breakdown of points, is what this version is about.

Note that `std::fixed` and `std::setprecision(1)` now sit on a line of their own, with no value after them. That is legal and often clearer: they configure the stream, and every floating-point value printed after them

is affected. Setting stream state separately from printing data is worth preferring once more than one or two values are involved.

> ▸ **Run checkpoint**: Click **Run**. The program should work exactly as in Chapter 5, with the simpler report. No letter grade yet.

Step 5 — Add the if/else if chain

In `main.cpp`, find the two lines that print the percentage inside the `if`:

```
        std::cout << std::left << std::setw(16) << "Percentage"
                  << percentage << "%\n";
```

Replace both lines with this:

```
        // Order matters: the first true condition wins, so cutoffs are
        // tested from highest to lowest.
        char letter = 'F';
        if (percentage >= A_CUTOFF) {
            letter = 'A';
        } else if (percentage >= B_CUTOFF) {
            letter = 'B';
        } else if (percentage >= C_CUTOFF) {
            letter = 'C';
        } else if (percentage >= D_CUTOFF) {
            letter = 'D';
        }

        std::cout << "Percentage: " << percentage << "%\n";
        std::cout << "Grade:      " << letter << "\n";
        if (percentage > 100.0) {
            std::cout << "Note: bonus points pushed this above 100%.\n";
        }
```

Why: This is the chapter's core idea, and there are three things in it.

Order matters. The chain tests the highest cutoff first. In an `if`/`else if` chain, evaluation stops at the first condition that is true, so a percentage of 95 matches `>= A_CUTOFF` and the B, C, and D tests never run. Reverse the order and every passing score would match `>= D_CUTOFF` first and come out as a D. The ordering is not stylistic — it is the logic.

The default comes first. `letter` is initialized to `'F'` before any test. That means the chain needs no final `else`: anything that fails all four tests keeps the initial value. Initializing to the fallback and then overwriting it is a common, readable pattern for this shape of decision.

A `char` holds one character. `'A'` in single quotes is a `char` — a single character. `"A"` in double quotes would be a string, and assigning it to a `char` is a compile error. The distinction catches people out constantly.

The nested `if (percentage > 100.0)` prints a note only when the bonus pushed the score past full marks. The percentage above 100 is still reported as-is — the Chapter 4 policy stands — but now the output says why it looks unusual.

Step 6 — Update the zero-points branch

In `main.cpp`, find the three lines in the `else` branch:

```
        std::cout << "Percentage      not available\n";
        std::cout << "  An assignment worth 0 points has no percentage,\n";
        std::cout << "  because a score cannot be divided by zero.\n";
```

Replace all three with these two:

```
        std::cout << "Percentage: not available (0 points possible)\n";
        std::cout << "Grade:      not available\n";
```

Then find the closing rule after the `else` block and delete it:

```
    std::cout << "-----------------------------------\n";
```

Why: The `else` branch now has to report *two* missing values, not one — there is no percentage and therefore no grade either. Saying so for both keeps the report's shape the same in each branch, so a reader always finds a `Percentage:` line and a `Grade:` line whatever the input. The reason is now given parenthetically rather than on its own lines, matching the compact style of the rest of the report.

▸ **Run checkpoint**: Click **Run** and work through the test plan below in the Terminal.

Test plan

Run the program once per row. All use `Ada Lovelace` and `Essay`, with points possible `100` and bonus `0` unless stated.

Points earned	Percentage	Expected grade	Why this case
`95`	95.0%	`A`	Comfortably above the A cutoff
`90`	90.0%	`A`	**Exactly** on the cutoff — `>=` must include it
`89.9`	89.9%	`B`	Just below; proves the boundary is not off by a hair
`80`	80.0%	`B`	Exactly on the B cutoff
`70`	70.0%	`C`	Exactly on the C cutoff
`60`	60.0%	`D`	Exactly on the D cutoff
`59.9`	59.9%	`F`	Below every cutoff — the initialized default
`0`	0.0%	`F`	Zero is a valid score, not an error

Then two more:

Above 100%. Points earned `100`, bonus `5`: the grade is `A`, the percentage reads `105.0%`, and the note about bonus points appears.

Zero points possible. Enter `0` for points possible: both `Percentage:` and `Grade:` report `not available`, and no letter is printed.

> The boundary rows are the ones that matter. An off-by-one error in a grade scale is invisible on a score of 95 and obvious on a score of exactly 90 — which is the score a student is most likely to argue about.

Check your work

Then confirm:

- ☐ `main.cpp` compiles with no errors and **no warnings**.

- ☐ Every row of the table above produces the expected grade.
- ☐ A score of exactly 90 is an `A`, not a `B`.
- ☐ A score below 60 is an `F` with no final `else` in your chain.
- ☐ The cutoffs appear only in the four `const double` declarations, not as numbers inside the `if` chain.
- ☐ A bonus above 100% prints the note and still reports a grade.
- ☐ Zero points possible reports both values as `not available`.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 6 Grade Calculator v0.5**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 6 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Relational and logical operators

```cpp
#include <iostream>

int main() {
    std::cout << std::boolalpha;
    std::cout << "5 > 3         " << (5 > 3) << "\n";
    std::cout << "5 == 3        " << (5 == 3) << "\n";
    std::cout << "true && false " << (true && false) << "\n";
    std::cout << "true || false " << (true || false) << "\n";
    std::cout << "!true         " << (!true) << "\n";
    return 0;
}
```

Expected output:

```
5 > 3         true
5 == 3        false
true && false false
true || false true
!true         false
```

Try: Predict `(5 > 3) && (2 > 4)` and `!(5 == 5)` before running them.

2. The boundary that looks wrong

```cpp
#include <iostream>
#include <iomanip>

int main() {
    double percentage = 89.95;
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Percentage: " << percentage << "\n";
    std::cout << std::boolalpha;
    std::cout << "Gets an A?  " << (percentage >= 90.0) << "\n";
    return 0;
}
```

Expected output:

```
Percentage: 90.0
Gets an A?  false
```

Try: Change the precision to 4 and rerun. Does the report now agree with the grade? Which fix would you rather ship — showing more decimals, or rounding before comparing?

3. Floating-point equality

```cpp
#include <iostream>
#include <iomanip>

int main() {
    double a = 0.1 + 0.2;
    std::cout << std::boolalpha;
    std::cout << "0.1+0.2 == 0.3 ? " << (a == 0.3) << "\n";
    std::cout << std::setprecision(17);
    std::cout << "0.1+0.2 is exactly " << a << "\n";
    return 0;
}
```

Expected output:

```
0.1+0.2 == 0.3 ? false
0.1+0.2 is exactly 0.30000000000000004
```

Try: Test whether the difference between them is smaller than 0.000001. Does that comparison give the answer you wanted?

4. Short-circuit evaluation protects you

```cpp
#include <iostream>

int main() {
    double possible = 0.0;
    double earned = 50.0;

    if (possible > 0.0 && earned / possible > 0.9) {
        std::cout << "High score\n";
    } else {
        std::cout << "Not computed - possible is zero\n";
```

```
    }
    return 0;
}
```

Expected output:

```
Not computed - possible is zero
```

Try: Swap the two conditions so the division comes first. What happens now, and why did the original order prevent it?

5. A menu with `switch`

```
#include <iostream>
#include <string>

int main() {
    std::cout << "1) Add  2) Report  3) Quit\nChoice: ";
    std::string line;
    std::getline(std::cin, line);
    char choice = line.empty() ? '?' : line[0];

    switch (choice) {
        case '1': std::cout << "Adding\n";    break;
        case '2': std::cout << "Reporting\n"; break;
        case '3': std::cout << "Goodbye\n";   break;
        default:  std::cout << "Please enter 1, 2, or 3\n"; break;
    }
    return 0;
}
```

Try: Remove the `break` from case 1 and enter `1`. What happens? That is fall-through.

6. Find the bug by reading

Each compiles. Each is wrong. Say what happens and why.

```
if (percentage >= 60.0)      { letter = 'D'; }
else if (percentage >= 70.0) { letter = 'C'; }
else if (percentage >= 80.0) { letter = 'B'; }
else if (percentage >= 90.0) { letter = 'A'; }
```

```
if (percentage >= 90.0) { letter = 'A'; }
if (percentage >= 80.0) { letter = 'B'; }
if (percentage >= 70.0) { letter = 'C'; }
```

```
if (percentage >= 90.0);
{
    letter = 'A';
}
```

```
if (0 <= percentage <= 100) {
    std::cout << "valid\n";
}
```

7. Design and implement

Write a program reading a percentage and reporting a **plus-minus** letter grade: A at 93, A− at 90, B+ at 87, B at 83, B− at 80, and F below 80.

Write the `if/else if` chain, then test 93, 92.9, 90, 89.9, 87, 80, and 79.9.

Then answer: how many lines is your chain? How many would a full A-through-F plus-minus scale need? Now reconsider the question from the Grade Calculator section — what would it take to let a user *define* this scale without touching the code?

Summary

- A **relational operator** compares values and produces a `bool`. `=` assigns; `==` compares.
- `if` runs a block when a condition is true; `if/else` chooses exactly one of two paths. **Always use braces.**
- A **block** defines **scope**. A variable declared inside one does not exist outside it.
- `&&`, `||`, and `!` combine conditions. **Chained comparisons such as `0 <= x <= 100` do not work** — write `0 <= x && x <= 100`.
- **Short-circuit evaluation** stops as soon as the answer is known. Put the check that makes the rest safe first.
- A multi-way **`if/else if` chain** handles mutually exclusive outcomes. **The first true condition wins**, so order from most restrictive to least. Without `else`, later branches overwrite earlier ones.
- **`switch`** compares one integral value against exact constants. It cannot test ranges or `doubles`, so letter-grade logic needs a chain. Each case needs `break`.
- The **conditional operator** `? :` selects between two values.
- Four conditional errors are **silent**: a stray semicolon, reversed chain order, a missing `else`, and `==` on floating-point values.
- **A value that displays as 90.0 may still be less than 90.0.** `setprecision` rounds the display, not the value. Decide whether you grade the stored number or the rounded one, and write the decision down.

Key Terms

block — statements enclosed in braces and treated as one statement.

Boolean expression — an expression producing `true` or `false`.

condition — the Boolean expression a selection statement tests.

conditional operator — `? :`, which selects between two values.

fall-through — continuing into the next `case` because `break` was omitted.

logical operator — `&&`, `||`, or `!`, combining Boolean expressions.

nested conditional — a conditional statement inside another.

relational operator — an operator comparing two values: `<`, `>`, `<=`, `>=`, `==`, `!=`.

scope — the region of a program in which a name exists.

selection — a control structure choosing between paths.

short-circuit evaluation — stopping evaluation of a compound condition once the result is determined.

switch — a statement comparing one integral value against a list of constants.

Next: Chapter 7 gives you **repetition**, the third structure from Chapter 5. Your calculator will accept as many assignments as a student has, accumulating totals as it goes — and you will see that points-based grading *is* the accumulator pattern. Grade Calculator v0.6.

Chapter 7 — Iteration

Learning Objectives

When you finish this chapter you will be able to:

- Write `while`, `do/while`, and `for` loops, and choose appropriately among them. *(SLO 1.3, 1.4)*
- Distinguish counter-controlled from sentinel-controlled repetition. *(SLO 1.4)*
- Use accumulators and counters to build running totals. *(SLO 1.3, 1.7)*
- Write and trace nested loops. *(SLO 1.4)*
- Use `break` and `continue`, and explain why they should be used sparingly. *(SLO 1.3)*
- Diagnose infinite loops and off-by-one errors. *(SLO 1.6)*
- Build Grade Calculator v0.6, and explain why points-based grading *is* the accumulator pattern.

7.1 The `while` Statement

Chapter 5 named three control structures. You have sequence and selection. This chapter is **repetition**, the last one — and after it, Course I is essentially complete as a language matter. Everything remaining organizes what you can already express.

The simplest loop:

```
while (condition) {
    // repeats while the condition is true
}
```

```
int i = 1;
while (i <= 5) {
    std::cout << i << " ";
    ++i;
}
```

```
1 2 3 4 5
```

The condition is tested **before** each pass. If it is false initially, the body never runs at all.

Three things must be present, and forgetting any one of them is a bug:

1. **Initialize** something before the loop — here, `i = 1`.
2. **Test** it in the condition — `i <= 5`.
3. **Change** it inside the body — `++i`.

Omit step 3 and `i` stays 1 forever. The condition never becomes false. That is an **infinite loop**, and Section 7.8 covers it.

7.2 Counter-Controlled and Sentinel-Controlled Loops

Loops divide into two kinds by how they know when to stop.

Counter-controlled repetition runs a known number of times:

```
int count = 1;
while (count <= assignmentCount) {
    // process assignment number `count`
    ++count;
}
```

Sentinel-controlled repetition runs until a special value appears. You do not know in advance how many passes there will be:

```
std::string name;
std::cout << "Assignment name (or 'done'): ";
std::getline(std::cin, name);

while (name != "done") {
    // process this assignment
    std::cout << "Assignment name (or 'done'): ";
    std::getline(std::cin, name);
}
```

The value `"done"` is the **sentinel** — a value that means "stop", chosen so it cannot be confused with real data. An assignment is unlikely to be named `done`; a sentinel of `0` for a score would be a poor choice, since 0 is a real score.

Notice the shape of the sentinel loop: **read once before the loop, then read again at the end of the body**. That first read is called a *priming read*, and without it the condition has nothing to test on the first pass. Forgetting it is a common early mistake.

Grade Calculator v0.6 uses exactly this structure, because a student may have any number of assignments.

7.3 The `do/while` Statement

A `do/while` tests the condition **after** the body, so the body always runs at least once:

```
int n = 100;
do {
    std::cout << "body ran with n=" << n << "\n";
} while (n < 5);
```

```
body ran with n=100
```

The condition was false from the start, and the body still ran once.

Note the semicolon after `while (n < 5);` — it is required here and forbidden after a plain `while`, which is a small inconsistency worth remembering.

Use `do/while` when the action must happen at least once: prompting for input you will validate, showing a menu before asking for a choice. Use `while` otherwise. In practice `while` is far more common.

7.4 The `for` Statement

When a loop is counter-controlled, `for` gathers all three parts into one line:

```
for (initialization; condition; update) {
    // body
}
```

```
for (int k = 1; k <= 5; ++k) {
    std::cout << k << " ";
}
```

```
1 2 3 4 5
```

This is exactly the `while` loop from Section 7.1, rearranged. The advantage is that all three parts sit together where you can see them, so a missing update is obvious rather than hidden twenty lines down.

The loop variable declared in the initialization exists **only inside the loop** — that is Section 6.4's scope rule, and it is a feature. It cannot leak, and you can reuse `k` in the next loop without conflict.

Counting from zero

C++ programmers conventionally count from 0:

```
for (int k = 0; k < 5; ++k) {      // runs 5 times: 0, 1, 2, 3, 4
```

Note `<` rather than `<=`. With `k < 5` starting from 0, the loop runs exactly 5 times. This convention exists because array indices start at 0, as Chapter 11 will show, so `for (int k = 0; k < size; ++k)` visits every element exactly once.

7.5 Choosing the Right Loop

Use	When
`for`	You know how many passes, or you are stepping through a range
`while`	Repetition depends on a condition, and may not run at all
`do/while`	The body must run at least once

All three are interchangeable in principle. Choosing the one that matches your intent tells a reader what you meant.

7.6 Nested Loops

A loop inside a loop. The inner loop completes fully for each single pass of the outer:

```
for (int r = 1; r <= 3; ++r) {
    for (int c = 1; c <= 3; ++c) {
        std::cout << r * c << "\t";
    }
    std::cout << "\n";
}
```

```
1   2   3
2   4   6
3   6   9
```

The outer loop runs 3 times; the inner runs 3 times per outer pass; the body executes 9 times. **The pass counts multiply.**

Nested loops become the natural tool in Chapter 11, where a roster of students each having several assignments is a two-dimensional structure: outer loop over students, inner loop over assignments.

7.7 Altering Loop Flow

Two statements change the normal course of a loop.

`break` exits the loop immediately:

```
while (true) {
    std::string line = readLine("Choice: ");
    if (line == "quit") {
        break;
    }
    // handle the choice
}
```

`continue` skips the rest of the current pass and goes to the next:

```
for (int k = 0; k < count; ++k) {
    if (pointsPossible[k] <= 0.0) {
        continue;          // skip ungraded items
    }
    // process this one
}
```

Both are legitimate and both are easy to overuse. A loop with several `break`s and `continue`s scattered through it has many exit points, and following it means holding all of them in mind at once.

Prefer a clear condition to a `break` where you can. `while (name != "done")` says what it means; `while (true)` with a `break` buried inside does not.

> **`goto` is excluded from this book.** C++ has it. It can jump anywhere, which is precisely what structured programming was invented to prevent, and Appendix D Section D.11 excludes it. Every use has a clearer structured form.

7.8 Infinite Loops

An **infinite loop** never terminates. The three common causes:

The update is missing.

```
int i = 1;
while (i <= 5) {
    std::cout << i << " ";      // no ++i — runs forever
}
```

The update moves the wrong way.

```
for (int k = 5; k > 0; ++k) {   // k grows; condition never fails
```

The condition can never become false.

```
double x = 0.1;
while (x != 1.0) {              // floating-point never lands exactly on 1.0
    x += 0.1;
}
```

That third one is Chapter 6 Section 6.12 again: **never use == or != on floating-point values**. Written as `while (x < 1.0)` it terminates.

If your program hangs, press **Ctrl+C** to stop it. Then check: is the loop variable updated? Does the update move toward the condition failing? Can the condition become false at all?

A trace table of the first three passes finds nearly every infinite loop in under a minute.

7.9 Accumulators, Counters, and Running Totals

This section is the heart of the chapter for the Grade Calculator.

An **accumulator** is a variable that builds a total across passes. A **counter** counts passes. Both must be **initialized before the loop** — an uninitialized accumulator starts at garbage, and Section 3.2 warned about exactly this.

```
double totalEarned = 0.0;      // accumulator
double totalPossible = 0.0;    // accumulator
int assignmentCount = 0;       // counter

// inside the loop, for each assignment:
totalEarned += pointsEarned + bonusPoints;
totalPossible += pointsPossible;
++assignmentCount;
```

Trace it with three assignments — 9/10, 84/100, and 18/20:

Pass	earned	possible	totalEarned	totalPossible	percentage so far
start	—	—	0	0	—
1	9	10	9	10	90.00%
2	84	100	93	110	84.55%
3	18	20	111	130	85.38%

```
after item 1: 9/10 = 90%
after item 2: 93/110 = 84.5455%
after item 3: 111/130 = 85.3846%
```

Why this matters more than it looks

Look at that final row. The course percentage is `totalEarned / totalPossible × 100`. Two running totals, one division.

Points-based grading *is* the accumulator pattern. There is nothing else to it. This is why Course I can build a genuinely useful grade calculator with only the tools you now have, and why the procedural code in Chapters 9 through 12 is honest rather than a straw man.

It is also worth noticing what this pattern *cannot* do. Every assignment contributes to one pair of totals, so every point counts the same regardless of what kind of work it came from. If exams were meant to count 50% of the course and homework 30%, two running totals could not express it — you would need separate totals per category, combined by weight afterward.

That is weighted grading, the requirement your Chapter 1 specification deferred. Nothing in Course I will need it. Chapter 20 is where the accumulator pattern finally runs out, and the answer to what replaces it is the reason Course II exists.

7.10 Loop Design and Off-by-One Errors

An **off-by-one error** runs a loop one time too many or too few. It is the most common loop bug, and it is silent.

```
for (int k = 0; k < 5; ++k)     // runs 5 times: 0 1 2 3 4
for (int k = 0; k <= 5; ++k)    // runs 6 times: 0 1 2 3 4 5
```

```
k < 5 runs:  0 1 2 3 4
k <= 5 runs: 0 1 2 3 4 5
```

One character apart. In Chapter 11 that extra pass reads past the end of an array, which is a genuine defect and the first seeded bug in Chapter 16's debugging lab.

Trace the first and last passes by hand. Write down what the loop variable is on the first pass and on the last, and confirm both are what you intended. Two lines of a trace table catch nearly every off-by-one error, and they cost far less than finding one later.

For the standard idiom, memorize the pairing: **start at 0, test with <, and the loop runs exactly as many times as the limit.**

Common Errors and Warnings

What you see or observe	Cause	Fix
The program hangs	Infinite loop	Ctrl+C; check the update moves toward the condition failing
The loop runs one time too many	Off-by-one, <= where < was meant	Trace the first and last passes
The loop never runs	Condition false initially	Check the initial value; consider `do/while`
The total is nonsense	Accumulator not initialized	Set it to 0 before the loop
The total keeps growing across students	Accumulator not reset	Reinitialize inside the outer loop
`while (x != 1.0)` never ends	`!=` on a floating-point value	Use < or >
A `getline` after the loop returns nothing	Leftover newline from >>	`std::cin.ignore(...)` — Section 3.12
`error: 'k' was not declared in this scope`	Used a `for` variable after the loop	Declare it before the loop
The loop body runs once, always	Stray `;` after `while (...)`	Delete the semicolon

---

Design Notes

Initialize accumulators immediately before the loop, not at the top of the function. The distance between initialization and use is where reset bugs live.

Choose a sentinel that cannot be real data. `"done"` is a safe assignment name. `0` is a real score and a terrible sentinel.

Trace before you run. Three rows of a trace table is faster than three rebuild cycles.

Prefer a meaningful condition to `while (true)` plus `break`. The condition documents the loop's purpose in the place a reader looks first.

Grade Calculator v0.6 — Multiple Assignments

What v0.6 does

Accepts any number of named assignments using a sentinel, accumulating totals as it goes, reporting the running percentage after each entry and a final course grade at the end.

This is the first version that handles a realistic amount of data.

The program

```
// Grade Calculator v0.6 - Chapter 7
// Accepts many named assignments until a sentinel, then reports the course grade.
// New this version: sentinel-controlled while loop, accumulators.
// Key idea: points-based grading IS the accumulator pattern - a running total
// of points earned divided by a running total of points possible.
// Run: click Run in StudySite and use the embedded Terminal.

#include <iostream>
#include <iomanip>
#include <limits>
#include <string>

const double A_CUTOFF = 90.0;
const double B_CUTOFF = 80.0;
const double C_CUTOFF = 70.0;
const double D_CUTOFF = 60.0;
const std::string SENTINEL = "done";

int main() {
    std::cout << "=== GRADE CALCULATOR v0.6 ===\n\n";

    std::string studentName;
    std::cout << "Student name: ";
    std::getline(std::cin, studentName);

    std::cout << "\nEnter assignments one at a time.\n";
    std::cout << "Type " << SENTINEL << " as the assignment name when finished.\n\n";

    // Accumulators: these two running totals are the entire grading algorithm.
    double totalEarned = 0.0;
    double totalPossible = 0.0;
    int assignmentCount = 0;

    std::string assignmentName;
    std::cout << "Assignment name (or " << SENTINEL << "): ";
    std::getline(std::cin, assignmentName);
```

```
    while (assignmentName != SENTINEL) {
        double pointsEarned = 0.0;
        std::cout << "  Points earned   : ";
        std::cin >> pointsEarned;

        double pointsPossible = 0.0;
        std::cout << "  Points possible : ";
        std::cin >> pointsPossible;

        double bonusPoints = 0.0;
        std::cout << "  Bonus points    : ";
        std::cin >> bonusPoints;

        // After reading a number with >>, the newline is still in the buffer.
        // Discard it, or the next getline returns an empty string.
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        totalEarned += pointsEarned + bonusPoints;
        totalPossible += pointsPossible;
        ++assignmentCount;

        std::cout << std::fixed << std::setprecision(1);
        if (totalPossible > 0.0) {
            std::cout << "  Running total: " << totalEarned << " / " << totalPossible
                      << " (" << totalEarned / totalPossible * 100.0 << "%)\n\n";
        } else {
            std::cout << "  Running total: " << totalEarned << " / 0.0\n\n";
        }

        std::cout << "Assignment name (or " << SENTINEL << "): ";
        std::getline(std::cin, assignmentName);
    }

    std::cout << "\n--- COURSE REPORT ---\n";
    std::cout << "Student:     " << studentName << "\n";
    std::cout << "Assignments: " << assignmentCount << "\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Total:       " << totalEarned << " / " << totalPossible << "\n";

    if (assignmentCount == 0) {
        std::cout << "No assignments entered, so there is no course grade yet.\n";
    } else if (totalPossible > 0.0) {
        double percentage = totalEarned / totalPossible * 100.0;
        char letter = 'F';
        if (percentage >= A_CUTOFF)      { letter = 'A'; }
        else if (percentage >= B_CUTOFF) { letter = 'B'; }
        else if (percentage >= C_CUTOFF) { letter = 'C'; }
        else if (percentage >= D_CUTOFF) { letter = 'D'; }
        std::cout << "Percentage:  " << percentage << "%\n";
        std::cout << "Grade:       " << letter << "\n";
    } else {
        std::cout << "All assignments were worth 0 points, so no percentage exists.\n";
    }
    return 0;
}
```

Expected output

With `Ada`, then `Homework 1` 9/10 bonus 0, `Midterm` 84/100 bonus 5, then `done`:

```
--- COURSE REPORT ---
Student:     Ada
Assignments: 2
Total:       98.0 / 110.0
Percentage:  89.1%
Grade:       B
```

What to notice

The `std::cin.ignore` line is finally necessary. Chapter 3 Section 3.12 explained the `>>`-then-`getline` trap, and v0.2 avoided it by ordering the reads so it never arose. A loop forces them to interleave: three `>>` reads, then a `getline` at the bottom. Without the `ignore`, that `getline` receives the leftover newline, the sentinel test sees an empty string rather than a name, and the loop exits after one assignment. Remove the line and watch it happen — it is worth thirty seconds.

Two accumulators and a counter, all initialized before the loop. They are declared immediately above it, per the Design Notes.

The priming read is before the loop, and the matching read is the last statement of the body. Both prompts are identical, which is how a sentinel loop should look.

Three cases at the end, not one. No assignments entered, all assignments worth zero points, and the normal case. Chapter 5's desk-check habit is what surfaces the first two.

Your StudySite Lab — Process Multiple Assignments

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v0.6
- **Starting point**: Your committed Chapter 6 v0.5 program (one file, one assignment, letter grade).

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

Until now the program graded exactly one assignment. A course has many, and you do not know in advance how many — so this chapter introduces the loop that keeps asking until the user says stop, and the **accumulator** pattern that builds a result across the iterations. The key insight is worth stating plainly: points-based grading *is* the accumulator pattern. A course grade is a running total of points earned divided by a running total of points possible. By the end you will have:

1. A `SENTINEL` constant holding the word `done`, used both in the prompts and in the loop condition.
2. Three accumulators declared before the loop: `totalEarned`, `totalPossible`, and `assignmentCount`.
3. A `while` loop that reads one assignment per iteration and reports a running total after each.
4. A `std::cin.ignore(...)` call that fixes the classic bug where a `getline` after a numeric read returns an empty string.
5. A `COURSE REPORT` that grades the accumulated totals, and says so plainly when no assignments were entered at all.

Files in this checkpoint

File	What you do to it
`main.cpp`	Edit (Steps 1-7)

When you are finished your project still has **one** `.cpp` file.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `main.cpp`. Confirm the editor shows the expected file paths before editing.
4. Continue the existing program in StudySite's internal editor, finish the steps in order, then Run.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Update the header and add `<limits>`

In `main.cpp`, find the four header comment lines:

```
// Grade Calculator v0.5 - Chapter 6
// Adds a letter grade from a fixed 90/80/70/60 scale.
// New this version: multi-way if/else if chain, named constants.
// Limitation: the cutoffs are hard-coded. Chapter 11 makes them user-defined.
```

Replace it with:

```
// Grade Calculator v0.6 - Chapter 7
// Accepts many named assignments until a sentinel, then reports the course grade.
// New this version: sentinel-controlled while loop, accumulators.
// Key idea: points-based grading IS the accumulator pattern - a running total
// of points earned divided by a running total of points possible.
```

Now find the `<iomanip>` include and add this below it:

```
#include <iomanip>
```

Directly below it, add:

```
#include <limits>
```

Why: `<limits>` provides `std::numeric_limits`, which Step 4 uses to clear the input buffer. The includes are kept in alphabetical order — `iomanip`, `limits`, `string` — which is a convention worth following: it makes a missing include easy to spot and avoids pointless diffs when two people add one.

Step 2 — Add the sentinel constant

In `main.cpp`, find the two-line comment above the cutoffs and delete it:

```
// Named constants: changing a cutoff means editing one line, not hunting
// for a magic number inside the conditional chain.
```

Then find the `D_CUTOFF` line:

```
const double D_CUTOFF = 60.0;
```

Directly below it, add:

```
const std::string SENTINEL = "done";
```

Why: The sentinel — the value that means "no more input" — is defined once as a constant rather than written as `"done"` in three places. That matters more than it looks: the word appears in two prompts *and* in the loop condition, and if those ever disagree the loop becomes impossible to exit. One constant makes that bug unreachable, and changing the sentinel to `quit` later is a one-line edit.

The comment above the cutoffs goes because you now know why named constants exist; the point was made in Chapter 6 and the code no longer needs to argue for itself.

Step 3 — Update the banner and the student prompt

In `main.cpp`, find the banner line:

```
    std::cout << "=== GRADE CALCULATOR v0.5 ===\n\n";
```

Replace it with:

```
    std::cout << "=== GRADE CALCULATOR v0.6 ===\n\n";
```

Find the student-name prompt and shorten it:

```
    std::cout << "Student name               : ";
```

Replace it with:

```
    std::cout << "Student name: ";
```

Why: The student name is now asked once, before the loop, while the assignment prompts repeat inside it. Those two groups no longer line up in one column, so the long padding on this prompt has nothing left to align with.

Step 4 — Add the instructions and the accumulators

In `main.cpp`, find the line that declares the assignment name:

```
    std::string assignmentName;
```

Directly above it, add:

```
    std::cout << "\nEnter assignments one at a time.\n";
    std::cout << "Type " << SENTINEL << " as the assignment name when finished.\n\n";

    // Accumulators: these two running totals are the entire grading algorithm.
    double totalEarned = 0.0;
    double totalPossible = 0.0;
    int assignmentCount = 0;
```

Why: The two instruction lines tell the user how to stop *before* they are trapped in a loop with no visible exit. Printing the sentinel from the constant means the instructions can never drift out of step with the code.

The three accumulators are the chapter's real content, and where they are declared is the whole trick. They live **outside** the loop, because a variable declared inside a loop body is created fresh on every iteration and destroyed at the end of it — it cannot remember anything. Declared outside, each one persists across every iteration and accumulates. Each starts at zero, the identity for addition, so the first `+=` produces the right answer with no special case for the first assignment.

`assignmentCount` is an `int` rather than a `double` because it counts discrete things; there is no such thing as 2.5 assignments.

Step 5 — Turn the single read into a loop

In `main.cpp`, find the assignment-name prompt:

```
    std::cout << "Assignment name             : ";
```

Replace it with:

```
    std::cout << "Assignment name (or " << SENTINEL << "): ";
```

Now find the whole block from the points-earned declaration down to the `Score:` line — nineteen lines in all:

```
    double pointsEarned = 0.0;
    std::cout << "Points earned               : ";
    std::cin >> pointsEarned;

    double pointsPossible = 0.0;
    std::cout << "Points possible (must be >0): ";
    std::cin >> pointsPossible;

    double bonusPoints = 0.0;
    std::cout << "Bonus points (0 for none)   : ";
    std::cin >> bonusPoints;

    double totalEarned = pointsEarned + bonusPoints;

    std::cout << "\n--- GRADE REPORT ---\n";
    std::cout << "Student:    " << studentName << "\n";
    std::cout << "Assignment: " << assignmentName << "\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Score:      " << totalEarned << " / " << pointsPossible << "\n";
```

Replace all of it with this:

```
    while (assignmentName != SENTINEL) {
        double pointsEarned = 0.0;
        std::cout << "  Points earned   : ";
        std::cin >> pointsEarned;

        double pointsPossible = 0.0;
        std::cout << "  Points possible : ";
        std::cin >> pointsPossible;

        double bonusPoints = 0.0;
        std::cout << "  Bonus points    : ";
        std::cin >> bonusPoints;

        // After reading a number with >>, the newline is still in the buffer.
        // Discard it, or the next getline returns an empty string.
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        totalEarned += pointsEarned + bonusPoints;
        totalPossible += pointsPossible;
        ++assignmentCount;

        std::cout << std::fixed << std::setprecision(1);
        if (totalPossible > 0.0) {
            std::cout << "  Running total: " << totalEarned << " / " << totalPossible
                      << " (" << totalEarned / totalPossible * 100.0 << "%)\n\n";
        } else {
            std::cout << "  Running total: " << totalEarned << " / 0.0\n\n";
        }

        std::cout << "Assignment name (or " << SENTINEL << "): ";
        std::getline(std::cin, assignmentName);
    }

    std::cout << "\n--- COURSE REPORT ---\n";
    std::cout << "Student:     " << studentName << "\n";
    std::cout << "Assignments: " << assignmentCount << "\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Total:       " << totalEarned << " / " << totalPossible << "\n";
```

Why: This is the largest single edit in the course so far, so read what changed rather than just copying it.

The loop shape is "read once, then loop". The first `getline(std::cin, assignmentName)` happens *before* the `while`, and the last statement *inside* the loop reads the next one. That arrangement — called a priming read — is what lets the condition be tested before the body runs even once. Type `done` at the very first prompt and the body never executes, which is exactly right: zero assignments is a legitimate input, not an error.

The accumulators do the work. `totalEarned += pointsEarned + bonusPoints;` means "add this to what you already have". `++assignmentCount;` adds one. The per-assignment variables are declared inside the loop because each iteration genuinely wants a fresh one; the totals are outside because they must survive.

`std::cin.ignore(...)` fixes a real bug. After `std::cin >> bonusPoints` reads a number, the Enter key the user pressed is *still sitting in the input buffer*. The next `getline` finds that leftover newline immediately, decides the line is over, and hands back an empty string — so the loop condition compares `""` to `"done"`, finds them different, and loops forever asking for points. The `ignore` call discards everything up to and including the next newline, clearing the way. `std::numeric_limits<std::streamsize>::max()` is simply "however much there might be". **Mixing `>>` and `getline` without this call is one of the most common bugs in beginning C++**, and the symptom — a prompt that seems to skip itself — looks nothing like its cause.

The running total is feedback. Printing progress after each assignment lets the user catch a typo at the point they made it, rather than discovering a wrong course grade at the end. It needs the same zero-denominator guard as before, because the first assignment could be worth zero points.

> **Do not click Run yet.** The grading section below still refers to `pointsPossible`, which now lives inside the loop and is not visible outside it. The program will not compile until Step 6 replaces that section. This is the expected mid-restructure state — finish Steps 6 and 7, then Run.

Step 6 — Grade the accumulated totals

In `main.cpp`, find the `if` that opens the grading section, together with the comment below it:

```
    if (pointsPossible > 0.0) {
        double percentage = totalEarned / pointsPossible * 100.0;

        // Order matters: the first true condition wins, so cutoffs are
        // tested from highest to lowest.
```

Replace those five lines with these four:

```
    if (assignmentCount == 0) {
        std::cout << "No assignments entered, so there is no course grade yet.\n";
    } else if (totalPossible > 0.0) {
        double percentage = totalEarned / totalPossible * 100.0;
```

Now find the if/else if chain and the lines that print the percentage and grade:

```
        if (percentage >= A_CUTOFF) {
            letter = 'A';
        } else if (percentage >= B_CUTOFF) {
            letter = 'B';
        } else if (percentage >= C_CUTOFF) {
            letter = 'C';
        } else if (percentage >= D_CUTOFF) {
            letter = 'D';
        }

        std::cout << "Percentage: " << percentage << "%\n";
        std::cout << "Grade:      " << letter << "\n";
        if (percentage > 100.0) {
            std::cout << "Note: bonus points pushed this above 100%.\n";
        }
```

Replace all fifteen lines with these six:

```
        if (percentage >= A_CUTOFF)      { letter = 'A'; }
        else if (percentage >= B_CUTOFF) { letter = 'B'; }
        else if (percentage >= C_CUTOFF) { letter = 'C'; }
        else if (percentage >= D_CUTOFF) { letter = 'D'; }
        std::cout << "Percentage:  " << percentage << "%\n";
        std::cout << "Grade:       " << letter << "\n";
```

Why: The decision now has **three** outcomes, not two, and they must be tested in this order. `assignmentCount == 0` comes first because "nothing was entered" is a different situation from "something was entered but it was worth no points", and the user deserves to be told which one happened. Only after both of those are ruled out is a percentage meaningful.

The chain itself is unchanged in logic — same cutoffs, same highest-first order, same `'F'` default — but it now divides the *totals* rather than one assignment's points, and it is written in the compact one-line-per-branch

form. Both forms are correct; this one shows the parallel structure of the four branches at a glance, which is easier to check for an ordering mistake.

The note about exceeding 100% is dropped: across a whole course, a bonus on one assignment rarely pushes the total over, and the line was becoming noise.

Step 7 — Update the zero-points message

In `main.cpp`, find the two lines in the final `else`:

```
        std::cout << "Percentage: not available (0 points possible)\n";
        std::cout << "Grade:      not available\n";
```

Replace both with this one line:

```
        std::cout << "All assignments were worth 0 points, so no percentage exists.\n";
```

Why: The message now describes the course-wide situation — *all* the assignments were worth zero — rather than a single assignment's points possible. A message that names the actual condition is the difference between a user understanding the program and distrusting it.

▸ **Run checkpoint**: Click **Run** and work through the test plan below in the Terminal.

Test plan

1. **Three assignments**. Student `Ada Lovelace`, then:

Assignment	Earned	Possible	Bonus
`Essay`	45	50	0
`Quiz`	18	20	2
`Project`	88	100	0

then `done`. Totals should be `153.0 / 170.0`, percentage `90.0%`, grade `A`, and `Assignments: 3`. Check that the running total after each row matches what you would compute by hand.

2. **The empty case — the important one**. Enter a student name, then type `done` at the very first assignment prompt. The report must show `Assignments: 0` and the message about no course grade yet. The program must **not** crash, print `nan`, or report an `F`.

3. **The `getline` bug, deliberately**. Enter one assignment, and at the assignment-name prompt for the second one, type a name with a space in it such as `Final Exam`. It must be stored in full. If you see the prompt flash past without waiting for input, or the loop will not exit on `done`, the `std::cin.ignore(...)` line from Step 5 is missing or misplaced.

4. **All assignments worth zero**. One assignment with points possible `0`, then `done`: the report should say all assignments were worth 0 points.

Check your work

Then confirm:

- ☐ `main.cpp` compiles with no errors and **no warnings**.
- ☐ Typing `done` at the first prompt gives `Assignments: 0` and no grade.
- ☐ A multi-word assignment name entered after the first iteration is stored in full.
- ☐ The loop exits on `done` every time, not only the first.

- ☐ The running total after each assignment matches the final total.
- ☐ The word `done` appears in the program only as `SENTINEL`, never as a literal.
- ☐ The three accumulators are declared before the loop, not inside it.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 7 Grade Calculator v0.6**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 7 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. The same loop, three ways

```cpp
#include <iostream>

int main() {
    std::cout << "while: ";
    int i = 1;
    while (i <= 5) { std::cout << i << " "; ++i; }

    std::cout << "\nfor:   ";
    for (int k = 1; k <= 5; ++k) { std::cout << k << " "; }

    std::cout << "\ndo:    ";
    int j = 1;
    do { std::cout << j << " "; ++j; } while (j <= 5);

    std::cout << "\n";
    return 0;
}
```

Expected output:

```
while: 1 2 3 4 5
for:   1 2 3 4 5
do:    1 2 3 4 5
```

Try: Change all three to start at 5 and count down to 1.

2. `do/while` runs at least once

```cpp
#include <iostream>

int main() {
    int n = 100;

    while (n < 5) {
        std::cout << "while body ran\n";
    }

    do {
        std::cout << "do body ran with n=" << n << "\n";
    } while (n < 5);

    return 0;
}
```

Expected output:

```
do body ran with n=100
```

Try: Explain in one sentence why the `while` produced nothing.

3. Off by one

```cpp
#include <iostream>

int main() {
    std::cout << "k < 5:  ";
    for (int k = 0; k < 5; ++k) { std::cout << k << " "; }
    std::cout << "\nk <= 5: ";
    for (int k = 0; k <= 5; ++k) { std::cout << k << " "; }
    std::cout << "\n";
    return 0;
}
```

Expected output:

```
k < 5:  0 1 2 3 4
k <= 5: 0 1 2 3 4 5
```

Try: Write a loop printing exactly ten values starting at 1. Which comparison did you use, and why?

4. The accumulator pattern

```cpp
#include <iostream>
#include <iomanip>

int main() {
    double earned[3]   = {9.0, 84.0, 18.0};
    double possible[3] = {10.0, 100.0, 20.0};

    double totalEarned = 0.0;
    double totalPossible = 0.0;
```

```
    std::cout << std::fixed << std::setprecision(2);
    for (int k = 0; k < 3; ++k) {
        totalEarned   += earned[k];
        totalPossible += possible[k];
        std::cout << "after item " << k + 1 << ": "
                  << totalEarned << "/" << totalPossible << " = "
                  << totalEarned / totalPossible * 100.0 << "%\n";
    }
    return 0;
}
```

Expected output:

```
after item 1: 9.00/10.00 = 90.00%
after item 2: 93.00/110.00 = 84.55%
after item 3: 111.00/130.00 = 85.38%
```

Try: Move the two accumulator initializations *inside* the loop. What happens, and what does the final percentage become? This is the reset bug from Common Errors.

5. Nested loops

```
#include <iostream>

int main() {
    for (int r = 1; r <= 3; ++r) {
        for (int c = 1; c <= 3; ++c) {
            std::cout << r * c << "\t";
        }
        std::cout << "\n";
    }
    return 0;
}
```

Expected output:

```
1   2   3
2   4   6
3   6   9
```

Try: Extend to 5 by 5. How many times does the inner body run in total? Verify by adding a counter.

6. Find the infinite loop

Each of these never ends. Say why, and give the smallest fix. Do not run them without being ready to press Ctrl+C.

```
int i = 1;
while (i <= 5) {
    std::cout << i << " ";
}
```

```
for (int k = 5; k > 0; ++k) {
    std::cout << k << " ";
}
```

```
double x = 0.0;
while (x != 1.0) {
    x += 0.1;
}
```

```
int count = 0;
while (count < 10);
{
    ++count;
}
```

7. Design a loop from a specification

Write a program that reads scores until the user enters `-1`, then reports how many scores were entered, their total, their average, the highest, and the lowest.

Before you write code:

- What is your sentinel, and why is `-1` a safe choice here when `0` would not be?
- How do you initialize "highest" and "lowest" so the first real score sets them correctly?
- What should the program report if the very first entry is `-1`?

Answer all three in writing, then implement it and test the no-scores case first.

Summary

- **Repetition** is the third structure of structured programming. Every loop needs three parts: **initialize**, **test**, and **change**.
- `while` tests before the body. `do/while` tests after, so the body always runs at least once. `for` gathers all three parts on one line and suits counter-controlled loops.
- **Counter-controlled** loops run a known number of times; **sentinel-controlled** loops run until a special value appears. A sentinel loop needs a **priming read** before it and a matching read at the end of the body.
- Choose a sentinel that cannot be mistaken for real data.
- **Nested loops** multiply their pass counts.
- `break` exits a loop; `continue` skips to the next pass. Prefer a clear condition where you can. `goto` is excluded from this book.
- **Infinite loops** come from a missing update, an update in the wrong direction, or a condition that can never be false — including `!=` on floating-point values.
- An **accumulator** builds a total across passes; a **counter** counts them. Both must be initialized before the loop, and reset in the right place when nested.
- **Points-based grading is the accumulator pattern**: two running totals and one division. It is also the reason weighted grading cannot be expressed this way — which is Chapter 20's problem.
- **Off-by-one errors** are the commonest loop bug and are silent. Trace the first and last passes. The standard idiom is: start at 0, test with <.

Key Terms

accumulator — a variable that builds a running total across loop passes.

break — a statement exiting a loop immediately.

continue — a statement skipping the rest of the current pass.

counter — a variable counting the number of loop passes.

counter-controlled — repetition running a known number of times.

do/while — a loop testing its condition after the body, so the body always runs once.

for — a loop gathering initialization, condition, and update into one line.

infinite loop — a loop whose condition never becomes false.

iteration — one pass through a loop body; also, repetition in general.

loop control variable — the variable a loop tests and updates.

nested loop — a loop contained within another loop.

off-by-one error — a loop running one pass too many or too few.

priming read — an input read before a sentinel loop, so the condition has something to test.

sentinel — a special input value signalling the end of data.

sentinel-controlled — repetition running until a sentinel value appears.

while — a loop testing its condition before each pass.

Next: Chapter 8 introduces the standard library's functions — rounding, minimum and maximum, character tests, and random numbers. You will replace your hand-written rounding with `std::round` and finally implement the bonus-point cap policy you chose in Chapter 4. Grade Calculator v0.7.

Chapter 8 — Using Library Functions

Learning Objectives

When you finish this chapter you will be able to:

- Call a function, and identify its arguments and return value. *(SLO 1.3)*
- Include the correct header for a standard library facility. *(SLO 1.3)*
- Use the mathematical functions in `<cmath>`, including the rounding family. *(SLO 1.3, 1.7)*
- Use `std::min` and `std::max` to express a clamping policy in one line. *(SLO 1.7)*
- Use the character classification functions in `<cctype>`. *(SLO 1.3)*
- Generate pseudorandom numbers with `<random>`. *(SLO 1.7)*
- Read a function's documentation and determine how to call it. *(SLO 1.7)*
- Build Grade Calculator v0.7, resolving the 89.95 problem from Chapter 6.

8.1 Functions, from the Caller's Side

You have been calling functions since Chapter 3 — `std::getline` is one. This chapter makes the idea explicit and puts the standard library to work. Chapter 9 shows you how to write your own.

A **function** is a named piece of code that performs a task. You **call** it by writing its name followed by parentheses:

```
double root = std::sqrt(16.0);
```

Three things are happening. `std::sqrt` is the function's name. `16.0` is the **argument** — the information you hand it. And the function **returns** a value, 4.0, which is stored in `root`.

The essential idea is that **you do not need to know how it works**. Somebody has already written a correct, tested square-root routine. Using it is faster than writing one, and more importantly it is more likely to be right. This is what a library is for.

8.2 Arguments, Parameters, and Return Values

Three terms that are easy to blur:

An **argument** is the value you pass at the call site. A **parameter** is the name the function uses for it internally. A **return value** is the result handed back.

```
double area = std::pow(radius, 2.0);
```

`radius` and `2.0` are arguments. Inside `std::pow`, they have parameter names you never see. The result comes back and is stored.

Some functions take several arguments, separated by commas, and **order matters**: `std::pow(2.0, 10.0)` is 1024, while `std::pow(10.0, 2.0)` is 100.

Some functions return nothing. `std::getline(std::cin, name)` does its work by modifying `name` rather than returning a value.

A returned value that you do not use is discarded. Writing `std::sqrt(16.0);` on a line by itself computes the root and throws it away. That is legal, useless, and usually a mistake.

8.3 Headers and the Standard Library

The **standard library** is the collection of facilities every C++ implementation provides. To use one, include its **header**:

Header	Provides
`<iostream>`	`std::cin`, `std::cout`
`<string>`	`std::string`, `std::getline`
`<iomanip>`	`std::setw`, `std::setprecision`, `std::fixed`
`<cmath>`	mathematical functions
`<algorithm>`	`std::min`, `std::max`, and much more
`<cctype>`	character classification
`<random>`	pseudorandom number generation
`<limits>`	type limits, and `std::cin.ignore`'s argument

Appendix D Section D.4 gives the rule: **include what you use**. If your file names `std::sqrt`, include `<cmath>` — even if some other header you included happens to provide it. Relying on an indirect include means your file breaks when an unrelated header changes.

The headers beginning with `c` — `<cmath>`, `<cctype>` — come from the C language. That is why their functions have terse names like `pow` and `isalpha` rather than the fuller style of the rest of the library.

8.4 Mathematical Functions

`<cmath>` provides the mathematics.

Function	Returns	Example	Result
`std::sqrt(x)`	square root	`std::sqrt(16.0)`	`4.0`
`std::pow(x, y)`	x raised to y	`std::pow(2.0, 10.0)`	`1024.0`
`std::abs(x)`	absolute value	`std::abs(-7.5)`	`7.5`
`std::round(x)`	nearest whole number	`std::round(89.95)`	`90.0`
`std::floor(x)`	largest whole number ≤ x	`std::floor(89.95)`	`89.0`
`std::ceil(x)`	smallest whole number ≥ x	`std::ceil(89.95)`	`90.0`

```
sqrt(16)     4.0000
pow(2,10)    1024.0000
abs(-7.5)    7.5000
```

Trigonometric and logarithmic functions are there too — `std::sin`, `std::cos`, `std::log`, `std::exp` — and none of them appear in this book's project.

8.5 Rounding, Minimum, and Maximum

Two small tools that resolve two problems you have been carrying.

Rounding to a chosen number of decimal places

`std::round` rounds to a whole number. To round to one decimal place, use the standard trick: **multiply**, **round**, **divide**.

```
double rounded = std::round(value * 10.0) / 10.0;
```

For two decimal places, use 100.0. For three, 1000.0.

The 89.95 problem, resolved

Chapter 6 Section 6.12 left you with a genuine dilemma. A percentage of 89.95 *displays* as `90.0` under `setprecision(1)`, but compares as less than 90.0 — so the report says 90.0 and the student gets a B.

Two policies were on the table. Grade the stored value and show more decimals, or **round first and grade what you display**. Here is the second, working:

```
double percentage = 89.95;
double rounded = std::round(percentage * 10.0) / 10.0;

std::cout << "raw     >= 90.0 ? " << (percentage >= 90.0) << "\n";
std::cout << "rounded >= 90.0 ? " << (rounded >= 90.0) << "\n";
```

```
raw     >= 90.0 ?    false
rounded >= 90.0 ?    true
```

Rounding once, early, and then using the rounded value for **both** the display and the comparison means the number shown is the number graded. No student can ever hold a printout that disagrees with their letter.

From v0.7 onward the Grade Calculator does exactly this. Notice where the fix lives: not in the comparison, and not in the formatting, but in a decision to compute the reported value once and use it everywhere. That is a design fix, which is what Chapter 6 predicted.

Clamping with `std::min` and `std::max`

From `<algorithm>`:

```
std::min(150.0, 100.0)     // 100.0
std::max(0.0, -5.0)        // 0.0
```

```
min(150.0,100.0) 100.0
max(0.0,-5.0)    0.0
```

`std::min` is exactly what the bonus-point cap needs. Chapter 4 asked you to decide whether a bonus may push a percentage past 100. If your answer was no, the entire policy is one line:

```
double reported = capAt100 ? std::min(raw, 100.0) : raw;
```

Compare with the hand-written form:

```
double reported = raw;
if (capAt100 && raw > 100.0) {
    reported = 100.0;
}
```

Four lines against one, and the one-line version names the operation. **Both arguments to `std::min` must be the same type** — `std::min(raw, 100)` fails to compile because `raw` is a `double` and `100` is an `int`. Write `100.0`.

8.6 Character Functions

`<cctype>` classifies and converts single characters.

Function	True when
`std::isdigit(c)`	c is `'0'` through `'9'`
`std::isalpha(c)`	c is a letter
`std::isspace(c)`	c is a space, tab, or newline
`std::isupper(c)` / `std::islower(c)`	c is upper or lower case
`std::toupper(c)` / `std::tolower(c)`	returns the converted character

```
isdigit('7') true
isalpha('7') false
toupper('a') A
```

Two quirks catch people. **These functions return an `int`, not a `bool`** — zero for false, nonzero for true. `if (std::isdigit(c))` works fine, but comparing to `true` is unreliable.

And **`std::toupper` returns an `int`**, so printing it directly shows a number:

```
std::cout << std::toupper('a');                    // prints 65
std::cout << static_cast<char>(std::toupper('a'));   // prints A
```

Both are consequences of Section 3.6: a `char` *is* a small integer, and these C-inherited functions work in integers.

You can accept either case for a menu choice with one call:

```
if (std::toupper(choice) == 'Y') { ... }
```

8.7 Random Numbers

Computers produce **pseudorandom** numbers — a sequence that looks random but is generated deterministically from a starting **seed**. Same seed, same sequence.

The modern approach uses `<random>`:

```
#include <random>

std::random_device seedSource;
std::mt19937 generator(seedSource());
std::uniform_int_distribution<int> roll(1, 6);

int value = roll(generator);      // 1 through 6, each equally likely
```

Three pieces: a **seed source**, a **generator** producing raw random bits, and a **distribution** shaping them into the range you want.

This is more setup than the older `rand()` you may see elsewhere, and it is worth it — `rand()` has genuinely poor statistical properties and awkward range behavior. Appendix D excludes it.

Seeding fixed makes results reproducible, which is exactly what you want while testing:

```
std::mt19937 generator(12345);    // same sequence every run
```

The Grade Calculator does not use random numbers. They appear here because generating test data is a real need, and because Chapter 16 will have you produce repeatable test cases.

8.8 Timing with `clock`

`<ctime>` lets you measure elapsed processor time:

```
#include <ctime>

std::clock_t start = std::clock();
// ... work ...
double seconds = static_cast<double>(std::clock() - start) / CLOCKS_PER_SEC;
```

Chapter 17 uses this to compare linear and binary search on the same data — which turns a claim about efficiency into a measurement.

8.9 Reading Library Documentation

You cannot memorize the standard library, and nobody does. What you need is the ability to read a function's description and work out how to call it.

Documentation gives you a **signature**:

```
double round(double x);
```

Read it in three parts. The **return type** comes first — `double`, so it gives back a floating-point number. The **name** is next. The **parameters** are in parentheses — one `double`.

That signature answers three questions: what do I pass, what do I get back, and what header do I need. When you look up a function, get those three and you can use it.

Two habits are worth forming. **Check the header** — documentation always names it, and a missing include is the most common error when trying something new. And **check the edge cases** — what does this return for zero, for a negative value, for an empty string? Documentation usually says, and it is the part people skip.

Common Errors and Warnings

What you see	Cause	Fix
`error: 'sqrt' was not declared in this scope`	Missing `#include <cmath>`	Add the header
`error: 'min' was not declared in this scope`	Missing `#include <algorithm>`	Add the header
`error: no matching function for call to 'min(double, int)'`	Arguments have different types	Write `100.0`, not `100`
`std::toupper('a')` prints `65`	It returns an `int`	Wrap in `static_cast<char>(...)`
`std::round(89.95)` gives `90`, not `89.95`	`round` goes to a whole number	Multiply by 10, round, divide by 10
Random values are the same every run	Fixed seed	Seed from `std::random_device`
Random values differ every run during testing	Seeded from `random_device`	Use a fixed seed while testing
A computed value is discarded	Return value not stored	Assign it to something

---

Design Notes

Prefer a library function to your own. `std::round` is correct, tested, and instantly recognizable. Your hand-rolled version is none of those things.

Round once, as early as the value is final. Rounding in two places invites the two results to disagree. The Grade Calculator computes a rounded percentage once and uses it for the report and the letter grade both.

Let a function name a policy. `std::min(raw, 100.0)` says *capped at 100* more clearly than four lines of `if`.

Include what you use. One header per facility you name, grouped and ordered per Appendix D Section D.4.

Grade Calculator v0.7 — Rounding and Robustness

What v0.7 does

Everything v0.6 did, with two corrections: the percentage is rounded once and graded as rounded, and the bonus-point cap policy from Chapter 4 is finally implemented.

The program

```
// Grade Calculator v0.7 - Chapter 8
// Correct rounding and a documented bonus-point cap policy.
// New this version: std::round, std::min, library functions replace hand-rolled logic.
//
// POLICY DECISION (documented per Chapter 4): this gradebook CAPS the reported
// percentage at 100. Bonus points can rescue a low score but cannot produce a
// grade above the maximum. Set CAP_AT_100 to false for uncapped reporting.
//
// Run: click Run in StudySite and use the embedded Terminal.

#include <cmath>
#include <iostream>
#include <iomanip>
#include <limits>
#include <string>

const double A_CUTOFF = 90.0;
```

```
const double B_CUTOFF = 80.0;
const double C_CUTOFF = 70.0;
const double D_CUTOFF = 60.0;
const std::string SENTINEL = "done";
const bool CAP_AT_100 = true;

int main() {
    std::cout << "=== GRADE CALCULATOR v0.7 ===\n\n";

    std::string studentName;
    std::cout << "Student name: ";
    std::getline(std::cin, studentName);

    std::cout << "\nType " << SENTINEL << " as the assignment name when finished.\n\n";

    double totalEarned = 0.0;
    double totalPossible = 0.0;
    int assignmentCount = 0;

    std::string assignmentName;
    std::cout << "Assignment name (or " << SENTINEL << "): ";
    std::getline(std::cin, assignmentName);

    while (assignmentName != SENTINEL) {
        double pointsEarned = 0.0;
        std::cout << "  Points earned   : ";
        std::cin >> pointsEarned;

        double pointsPossible = 0.0;
        std::cout << "  Points possible : ";
        std::cin >> pointsPossible;

        double bonusPoints = 0.0;
        std::cout << "  Bonus points    : ";
        std::cin >> bonusPoints;
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        totalEarned += pointsEarned + bonusPoints;
        totalPossible += pointsPossible;
        ++assignmentCount;

        std::cout << "  Recorded.\n\n";
        std::cout << "Assignment name (or " << SENTINEL << "): ";
        std::getline(std::cin, assignmentName);
    }

    std::cout << "\n--- COURSE REPORT ---\n";
    std::cout << "Student:     " << studentName << "\n";
    std::cout << "Assignments: " << assignmentCount << "\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Total:       " << totalEarned << " / " << totalPossible << "\n";

    if (assignmentCount == 0 || totalPossible <= 0.0) {
        std::cout << "No course grade is available yet.\n";
        return 0;
    }

    double rawPercentage = totalEarned / totalPossible * 100.0;

    // std::min expresses the cap policy in one readable line.
```

```
    double reported = CAP_AT_100 ? std::min(rawPercentage, 100.0) : rawPercentage;

    // Round to one decimal place: multiply, round, divide. Doing this once here
    // means the displayed number and the graded number are always the same.
    double rounded = std::round(reported * 10.0) / 10.0;

    char letter = 'F';
    if (rounded >= A_CUTOFF)      { letter = 'A'; }
    else if (rounded >= B_CUTOFF) { letter = 'B'; }
    else if (rounded >= C_CUTOFF) { letter = 'C'; }
    else if (rounded >= D_CUTOFF) { letter = 'D'; }

    std::cout << "Percentage:  " << rounded << "%\n";
    std::cout << "Grade:       " << letter << "\n";
    if (CAP_AT_100 && rawPercentage > 100.0) {
        std::cout << "Note: raw score was " << rawPercentage
                  << "%, capped at 100% by course policy.\n";
    }
    return 0;
}
```

Expected output

With `Ada`, then `Homework 1 10/10 bonus 5`, then `done`:

```
--- COURSE REPORT ---
Student:     Ada
Assignments: 1
Total:       15.0 / 10.0
Percentage:  100.0%
Grade:       A
Note: raw score was 150.0%, capped at 100% by course policy.
```

What to notice

The rounded value is used for both the report and the letter grade. That single decision closes the 89.95 problem. Trace it: 89.95 rounds to 90.0, which satisfies `>= A_CUTOFF`, so the student sees 90.0 and receives an A. The number displayed is the number graded.

The cap is one line, and it is conditional on a named constant. Changing course policy means editing `CAP_AT_100` — one word — rather than rewriting logic.

The cap is applied before rounding. Order matters here: capping at 100 and then rounding gives 100.0; rounding 150.0 first and then capping gives the same answer in this case but not in every case. Applying the policy to the raw value, then rounding for presentation, is the order that always makes sense.

The raw value is still reported when it was capped. Hiding the fact that a cap applied would leave a user puzzled about their missing bonus.

The guard returns early. `if (assignmentCount == 0 || totalPossible <= 0.0) { ...; return 0; }` handles the impossible cases first and lets the main logic run unindented — the guard-clause pattern from Appendix D Section D.5.

Your StudySite Lab — Round and Cap the Grade

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v0.7

- **Starting point**: Your committed Chapter 7 v0.6 program (one file, sentinel loop, accumulators).

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

Chapter 4 left a question open: should bonus points be able to push a grade above 100%? This chapter answers it — and, more importantly, answers it *in one place*, with the decision written down. You will also fix a subtle unfairness in how the grade is decided, by rounding the percentage **before** comparing it to the cutoffs rather than only when printing it. Both changes are made with standard library functions instead of hand-written logic. By the end you will have:

1. A `CAP_AT_100` constant, and a header comment recording the policy it implements and how to reverse it.
2. `std::min` applied to the raw percentage, expressing the cap in one line.
3. `std::round` used to round to one decimal place, so the number displayed and the number graded are the same number.
4. An early `return 0;` for the no-grade cases, which flattens the nested decision that was starting to get hard to follow.
5. A note reporting the uncapped score when the cap actually applies.

Files in this checkpoint

File	What you do to it
`main.cpp`	Edit (Steps 1-6)

When you are finished your project still has **one** `.cpp` file.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `main.cpp`. Confirm the editor shows the expected file paths before editing.

4. Continue the existing program in StudySite's internal editor, finish the steps in order, then Run.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Document the policy and include `<cmath>`

In `main.cpp`, find the five header comment lines:

```
// Grade Calculator v0.6 - Chapter 7
// Accepts many named assignments until a sentinel, then reports the course grade.
// New this version: sentinel-controlled while loop, accumulators.
// Key idea: points-based grading IS the accumulator pattern - a running total
// of points earned divided by a running total of points possible.
```

Replace it with:

```
// Grade Calculator v0.7 - Chapter 8
// Correct rounding and a documented bonus-point cap policy.
// New this version: std::round, std::min, library functions replace hand-rolled logic.
//
// POLICY DECISION (documented per Chapter 4): this gradebook CAPS the reported
// percentage at 100. Bonus points can rescue a low score but cannot produce a
// grade above the maximum. Set CAP_AT_100 to false for uncapped reporting.
//
```

Now find the `<iostream>` include and add this above it:

```
#include <iostream>
```

Directly above it, add:

```
#include <cmath>
```

Why: `<cmath>` declares `std::round`; `std::min` comes from `<algorithm>`, but in practice it is reachable here through the other headers — on a strict compiler you would add `<algorithm>` explicitly, and it is good practice to do so if yours complains.

The header block is where the **policy decision** is recorded, and this is the chapter's quieter lesson. Capping at 100 is a choice, not a fact: a different course could reasonably decide the other way. Code that silently embodies a policy is code nobody can question or change safely. Naming the decision, saying who it came from, and saying exactly what to flip to reverse it turns an invisible assumption into a documented one.

Step 2 — Add the policy constant and update the banner

In `main.cpp`, find the `SENTINEL` line:

```
const std::string SENTINEL = "done";
```

Directly below it, add:

```
const bool CAP_AT_100 = true;
```

Then find the banner line:

```
    std::cout << "=== GRADE CALCULATOR v0.6 ===\n\n";
```

Replace it with:

```
    std::cout << "=== GRADE CALCULATOR v0.7 ===\n\n";
```

Why: `const bool CAP_AT_100 = true;` is the switch the header comment promises. A named boolean constant is better than writing `true` inside the calculation for the same reason `A_CUTOFF` beat `90.0`: the name says what the value *means*, and changing the policy is a one-line edit in an obvious place rather than a hunt through the logic.

Step 3 — Tidy the instructions and remove two stale comments

In `main.cpp`, find the two instruction lines above the accumulators:

```
    std::cout << "\nEnter assignments one at a time.\n";
    std::cout << "Type " << SENTINEL << " as the assignment name when finished.\n\n";
```

Replace both with this one line:

```
    std::cout << "\nType " << SENTINEL << " as the assignment name when finished.\n\n";
```

Find the comment above the accumulators and delete it:

```
    // Accumulators: these two running totals are the entire grading algorithm.
```

Then find the blank line and two-line comment above the `std::cin.ignore` call and delete all three lines:

```
        // After reading a number with >>, the newline is still in the buffer.
        // Discard it, or the next getline returns an empty string.
```

Why: All three were teaching comments that have done their job. The accumulator pattern and the `>>`/`getline` buffer problem were Chapter 7's subject; by now they are part of how you read this code rather than something it has to explain. The `ignore` call stays, of course — only the commentary goes. Leaving the call tucked directly under the read it belongs to actually makes the pairing clearer than the comment did.

Step 4 — Replace the running total with a brief acknowledgement

In `main.cpp`, find the running-total block inside the loop — seven lines, plus the blank line below it:

```
        std::cout << std::fixed << std::setprecision(1);
        if (totalPossible > 0.0) {
            std::cout << "  Running total: " << totalEarned << " / " << totalPossible
                      << " (" << totalEarned / totalPossible * 100.0 << "%)\n\n";
        } else {
            std::cout << "  Running total: " << totalEarned << " / 0.0\n\n";
        }
```

Replace all of it with this single line:

```
        std::cout << "  Recorded.\n\n";
```

Why: The running total duplicated the percentage calculation, and from this chapter on that calculation acquires a cap and a rounding rule. Two copies of a rule that is about to get more complicated is exactly the kind of duplication that drifts out of step — one copy gets the cap, the other does not, and the running total starts contradicting the final report. Removing the second copy now is cheaper than keeping them synchronized, and `Recorded.` still gives the user the confirmation they need that the entry was accepted.

> ▸ **Run checkpoint**: Click **Run**. The program should behave as it did in Chapter 7, except that each assignment is acknowledged with `Recorded.` instead of a running total. The grading at the end is unchanged so far.

Step 5 — Flatten the decision with an early return

In `main.cpp`, find the start of the grading section — the `if` testing the count, through the line that computes the percentage:

```
    if (assignmentCount == 0) {
        std::cout << "No assignments entered, so there is no course grade yet.\n";
    } else if (totalPossible > 0.0) {
        double percentage = totalEarned / totalPossible * 100.0;
```

Replace those four lines with these six:

```
    if (assignmentCount == 0 || totalPossible <= 0.0) {
        std::cout << "No course grade is available yet.\n";
        return 0;
    }

    double rawPercentage = totalEarned / totalPossible * 100.0;
```

Why: This is a **guard clause**: handle the cases that have no answer, return immediately, and let the rest of the function assume the normal case. Compare the two shapes. Before, the real work sat indented inside an `else if`, with its conditions held in your head while you read it. After, the impossible cases are dealt with in four lines at the top and everything below runs at one level of indentation with nothing left to remember.

The two no-grade conditions also combine with `||` into one branch, because they produce the same outcome. Chapter 7 separated them to explain the difference; now that the distinction has been made, one message for "no grade available" is simpler and the user loses nothing.

Note that `rawPercentage` is the name, not `percentage`. From here there are *three* related numbers — raw, capped, rounded — and giving each its own exact name is what keeps the next three steps readable.

Step 6 — Cap, round, then grade

In `main.cpp`, find everything from the `char letter` declaration down to the closing brace of the final `else` — ten lines:

```
        char letter = 'F';
        if (percentage >= A_CUTOFF)      { letter = 'A'; }
        else if (percentage >= B_CUTOFF) { letter = 'B'; }
        else if (percentage >= C_CUTOFF) { letter = 'C'; }
        else if (percentage >= D_CUTOFF) { letter = 'D'; }
        std::cout << "Percentage:  " << percentage << "%\n";
        std::cout << "Grade:       " << letter << "\n";
    } else {
        std::cout << "All assignments were worth 0 points, so no percentage exists.\n";
    }
```

Replace all of it with this:

```
    // std::min expresses the cap policy in one readable line.
    double reported = CAP_AT_100 ? std::min(rawPercentage, 100.0) : rawPercentage;

    // Round to one decimal place: multiply, round, divide. Doing this once here
    // means the displayed number and the graded number are always the same.
    double rounded = std::round(reported * 10.0) / 10.0;

    char letter = 'F';
    if (rounded >= A_CUTOFF)      { letter = 'A'; }
    else if (rounded >= B_CUTOFF) { letter = 'B'; }
    else if (rounded >= C_CUTOFF) { letter = 'C'; }
```

```
    else if (rounded >= D_CUTOFF) { letter = 'D'; }

    std::cout << "Percentage:  " << rounded << "%\n";
    std::cout << "Grade:       " << letter << "\n";
    if (CAP_AT_100 && rawPercentage > 100.0) {
        std::cout << "Note: raw score was " << rawPercentage
                  << "%, capped at 100% by course policy.\n";
    }
```

Why: Three ideas, in the order the code applies them.

`std::min` expresses the cap. `std::min(rawPercentage, 100.0)` is "whatever is smaller". Written by hand it would be an `if` with a reassignment, four lines instead of one, with room for a `>` that should have been `>=`. The conditional operator `? :` then selects between capped and uncapped reporting based on the policy constant: read it as "if `CAP_AT_100`, use the capped value, otherwise use the raw one".

`std::round` makes the grade fair. This is the real bug fix, and it is worth understanding. Previously `std::setprecision(1)` rounded the number *for display only* — a percentage of 89.96 printed as `90.0` but was compared to the cutoffs as 89.96, and so came out a `B`. A student seeing `90.0%` next to a `B` has a legitimate complaint. `std::round(reported * 10.0) / 10.0` is the standard idiom for rounding to one decimal: scale up, round to the nearest whole number, scale back down. Doing it **once**, **here**, **before the comparison** guarantees the number shown and the number graded are the same number.

The note reports what the cap hid. When the cap applies, the student should be able to see the uncapped figure and the reason it was not used. `CAP_AT_100 && rawPercentage > 100.0` is true only when the cap actually changed something — with the policy off, or a score under 100, no note appears.

▸ **Run checkpoint**: Click **Run** and work through the test plan below in the Terminal.

Test plan

All rows use student `Ada Lovelace` and a single assignment named `Essay`, then `done`.

Earned	Possible	Bonus	Percentage	Grade	What it proves
`90`	`100`	`0`	90.0%	`A`	Exact cutoff still works
`89.96`	`100`	`0`	90.0%	**`A`**	**The rounding fix** — this was a `B` before
`89.94`	`100`	`0`	89.9%	`B`	Rounds down, so correctly a `B`
`100`	`100`	`10`	100.0%	`A`	Cap applies; note shows raw 110.0%
`50`	`100`	`45`	95.0%	`A`	Bonus rescues a low score, under the cap
`0`	`100`	`0`	0.0%	`F`	Zero is valid

The second row is the point of the chapter. Run it on your Chapter 7 program if you still have it and you will get a `B`; here you get an `A`, because the grade is now decided from the same rounded number the student sees.

Then two structural cases:

No assignments. Type `done` immediately: `No course grade is available yet.` and a clean exit via the guard clause.

All worth zero points. One assignment with possible `0`, then `done`: the same message, because the guard now covers both conditions.

Check your work

Then confirm:

- ☐ `main.cpp` compiles with no errors and **no warnings**.
- ☐ A raw percentage of 89.96 reports `90.0%` **and** grade `A`.
- ☐ A raw percentage of 89.94 reports `89.9%` and grade `B`.
- ☐ A score above 100% is reported as `100.0%` with the note naming the raw figure.
- ☐ Setting `CAP_AT_100` to `false`, rebuilding, and re-running the 110% case reports `110.0%` with no note — then set it back to `true`.
- ☐ The percentage is rounded once, before the grade comparison, not only in the output.
- ☐ Both no-grade cases exit through the single guard clause.

Save this checkpoint

> **IMPORTANT — commit to save your work:** StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 8 Grade Calculator v0.7**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 8 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. The rounding family

```cpp
#include <cmath>
#include <iomanip>
#include <iostream>

int main() {
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "round(89.95) " << std::round(89.95) << "\n";
    std::cout << "floor(89.95) " << std::floor(89.95) << "\n";
    std::cout << "ceil(89.95)  " << std::ceil(89.95) << "\n";
```

```
    return 0;
}
```

Expected output:

```
round(89.95) 90.0
floor(89.95) 89.0
ceil(89.95)  90.0
```

Try: Predict all three for `-2.5` before running. Does `round` go toward zero or away from it?

2. Rounding to one decimal place

```
#include <cmath>
#include <iomanip>
#include <iostream>

int main() {
    double p = 89.95;
    double rounded = std::round(p * 10.0) / 10.0;

    std::cout << std::boolalpha << std::fixed << std::setprecision(1);
    std::cout << "value shown:      " << rounded << "\n";
    std::cout << "raw >= 90.0 ?     " << (p >= 90.0) << "\n";
    std::cout << "rounded >= 90.0 ? " << (rounded >= 90.0) << "\n";
    return 0;
}
```

Expected output:

```
value shown:      90.0
raw >= 90.0 ?     false
rounded >= 90.0 ? true
```

Try: Change `p` to `89.94` and rerun. Which line changes, and is that the behavior you want?

3. Clamping

```
#include <algorithm>
#include <iomanip>
#include <iostream>

int main() {
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "min(150.0, 100.0) " << std::min(150.0, 100.0) << "\n";
    std::cout << "max(0.0, -5.0)    " << std::max(0.0, -5.0) << "\n";
    return 0;
}
```

Expected output:

```
min(150.0, 100.0) 100.0
max(0.0, -5.0)    0.0
```

Try: Change `100.0` to `100` and rebuild. Read the error carefully — it is the type-mismatch message from Common Errors. Then clamp a value into the range 0 to 100 using both functions in one expression.

4. Character classification

```
#include <cctype>
#include <iostream>

int main() {
    std::cout << std::boolalpha;
    std::cout << "isdigit('7') " << (std::isdigit('7') != 0) << "\n";
    std::cout << "isalpha('7') " << (std::isalpha('7') != 0) << "\n";
    std::cout << "toupper('a') " << static_cast<char>(std::toupper('a')) << "\n";
    return 0;
}
```

Expected output:

```
isdigit('7') true
isalpha('7') false
toupper('a') A
```

Try: Remove the `static_cast` from the last line. Why does it print a number? Chapter 3 Section 3.6 has the answer.

5. Reproducible random numbers

```
#include <iostream>
#include <random>

int main() {
    std::mt19937 generator(12345);          // fixed seed
    std::uniform_int_distribution<int> score(60, 100);

    for (int k = 0; k < 5; ++k) {
        std::cout << score(generator) << " ";
    }
    std::cout << "\n";
    return 0;
}
```

Try: Run it three times. The values are identical every time. Now change the seed to `54321` and run again. Then replace the fixed seed with `std::random_device{}()` and run three times. Which behavior do you want while testing, and which in a finished program?

6. Read a signature

Each of these is a real standard library signature. For each, say what you would pass, what you would get back, and which header you need.

```
double pow(double base, double exponent);
int toupper(int c);
const double& min(const double& a, const double& b);
size_t string::length() const;
```

7. Replace hand-written code with library calls

Rewrite each using a standard library function, and name the header.

```
double result = value;
if (value < 0.0) {
```

```
    result = -value;
}
```

```
double capped = raw;
if (capped > 100.0) {
    capped = 100.0;
}
```

```
double whole = static_cast<int>(value + 0.5);
```

For the third one: does your library replacement behave identically for negative values? Test it with `-2.5` and explain any difference.

Summary

- A **function** is a named piece of code performing a task. You pass **arguments** and usually receive a **return value**. You do not need to know how it works.
- Include the **header** for every facility you name. `<cmath>`, `<algorithm>`, `<cctype>`, `<random>`, `<ctime>`.
- `<cmath>` provides `sqrt`, `pow`, `abs`, and the rounding family: `round`, `floor`, `ceil`.
- **Round to a chosen number of decimal places** by multiplying, rounding, and dividing.
- **Round once, then use the rounded value for both display and comparison**. This resolves the 89.95 problem from Chapter 6: the number shown becomes the number graded.
- `std::min` and `std::max` from `<algorithm>` express clamping in one line. Both arguments must be the same type.
- `<cctype>` classifies characters. These functions return `int`, not `bool`, and `toupper` needs a cast before printing.
- `<random>` produces **pseudorandom** numbers from a **seed**, a **generator**, and a **distribution**. A fixed seed gives reproducible results, which is what testing wants.
- Reading a **signature** tells you what to pass, what you get back, and which header you need.

Key Terms

argument — a value passed to a function at the call site.

call — to invoke a function by name.

distribution — an object shaping raw random values into a desired range.

function — a named piece of code performing a task.

generator — an object producing a sequence of pseudorandom values.

header — a file declaring library facilities, brought in with `#include`.

parameter — the name a function uses internally for an argument it receives.

pseudorandom — generated deterministically from a seed but statistically resembling randomness.

return value — the result a function hands back to its caller.

seed — the starting value determining a pseudorandom sequence.

signature — a function's return type, name, and parameter list.

standard library — the collection of facilities every C++ implementation provides.

Next: Chapter 9 turns the tables — instead of calling functions, you will write them. The Grade Calculator is rebuilt from the structure chart you drew in Chapter 5, one function per box, and gains a menu so it no longer runs once and exits. Grade Calculator v1.0.

Chapter 9 — Writing Functions

Learning Objectives

When you finish this chapter you will be able to:

- Define a function with parameters and a return value. *(SLO 1.4)*
- Explain the difference between a prototype and a definition, and say when each is needed. *(SLO 1.4)*
- Explain pass by value and predict what a function can and cannot change. *(SLO 1.4)*
- Write `void` functions and explain when a return value is unnecessary. *(SLO 1.4)*
- Describe the scope and lifetime of a local variable. *(SLO 1.3)*
- Convert a structure chart into a set of functions. *(SLO 1.4, 1.5)*
- Write documentation comments in the form Appendix D requires. *(SLO 1.6)*
- Test a function individually with hand-computed cases. *(SLO 1.6)*
- Build Grade Calculator v1.0 — the modular rebuild, with a menu.

9.1 Defining a Function

Chapter 8 had you calling functions other people wrote. Now you write your own, and the Grade Calculator gets its first real structure.

A **function definition** has four parts:

```
double computePercentage(double earned, double possible) {
    return earned / possible * 100.0;
}
```

Part	In the example
Return type	`double`
Name	`computePercentage`
Parameter list	`(double earned, double possible)`
Body	the block, containing `return`

Call it exactly as you called library functions:

```
double pct = computePercentage(84.0, 100.0);    // 84.0
```

The value passed at the call site is an **argument**. The name inside the function is a **parameter**. Chapter 8 Section 8.2 drew that distinction from the caller's side; now you are on the other side of it.

9.2 Prototypes, Definitions, and Order

C++ reads a file top to bottom. A function must be **declared** before it is called.

The simplest approach is to define every function above `main`:

```
#include <iostream>

double computePercentage(double earned, double possible) {
    return earned / possible * 100.0;
}

int main() {
    std::cout << computePercentage(84.0, 100.0) << "\n";
    return 0;
}
```

Sometimes that ordering is inconvenient. A **prototype** declares a function without defining it — the signature followed by a semicolon:

```
#include <iostream>

double computePercentage(double earned, double possible);    // prototype

int main() {
    std::cout << computePercentage(84.0, 100.0) << "\n";
    return 0;
}

double computePercentage(double earned, double possible) {   // definition
    return earned / possible * 100.0;
}
```

A prototype is a promise: *this function exists somewhere with this signature.* The compiler accepts calls on that promise. If nobody keeps it — if no definition is ever supplied — the **linker** fails, not the compiler.

That is the division from Chapter 2 Section 2.2.4, now something you can trigger deliberately. It is worth doing once: write a prototype, call it, define nothing, and read the message. It looks nothing like a compiler error, and knowing why saves confusion later.

Prototypes become essential in Chapter 10, when your program spans several files.

9.3 Parameters and Pass by Value

By default C++ uses **pass by value**: the function receives a *copy* of each argument. Changing the parameter does not affect the caller's variable.

```
#include <iostream>

void tryToChange(double points) {
    points = 999.0;                          // changes the copy only
    std::cout << "inside the function: " << points << "\n";
}

int main() {
    double score = 84.0;
    tryToChange(score);
```

```
    std::cout << "back in main:       " << score << "\n";
    return 0;
}
```

```
inside the function: 999
back in main:       84
```

The copy really did change — the function is not failing to assign. The assignment simply had no effect on `score`, because the function was never given `score` in the first place. It was given a copy.

This is a safety feature. A function cannot corrupt the caller's data by accident, so you can call one without worrying about what it might do to your variables.

It is also a limitation. A function that genuinely needs to modify its caller's variable cannot do it this way. Chapter 10 introduces **pass by reference** for exactly that case, and the Grade Calculator needs it for input validation.

9.4 Return Values and `void` Functions

A function returns a value with `return`:

```
char assignLetterGrade(double percentage) {
    if (percentage >= 90.0) { return 'A'; }
    if (percentage >= 80.0) { return 'B'; }
    if (percentage >= 70.0) { return 'C'; }
    if (percentage >= 60.0) { return 'D'; }
    return 'F';
}
```

`return` exits the function immediately. That is why this version needs no `else` — reaching the second `if` already means the first was false. Compare with the chained form in Chapter 6 Section 6.9; both are correct, and this one is shorter because `return` does the work `else` was doing.

A function that does something rather than computing something has return type **`void`**:

```
void printReport(const std::string& student, double percentage, char letter) {
    std::cout << "Student:    " << student << "\n";
    std::cout << "Percentage: " << percentage << "%\n";
    std::cout << "Grade:      " << letter << "\n";
}
```

A `void` function needs no `return` statement, though a bare `return;` may be used to exit early.

> **`main` is special**. It returns `int`, and reaching its closing brace without a `return` implicitly returns 0. Every other non-`void` function must return a value on every path — failing to do so is undefined behavior, and `-Wall` warns about it.

9.5 Local Variables, Scope, and Lifetime

A variable declared inside a function is **local** to it. Chapter 6 Section 6.4 introduced scope for blocks; the same rule applies here, with one addition.

Scope is *where* a name is visible. **Lifetime** is *when* the variable exists.

```
double computePercentage(double earned, double possible) {
    double raw = earned / possible * 100.0;    // created here
    return std::round(raw * 10.0) / 10.0;
}                                              // destroyed here
```

`raw` exists only while the function runs. Each call creates a fresh one; nothing carries over between calls.

Two functions may use the same local name without interfering:

```
double computePercentage(double earned, double possible) {
    double result = earned / possible * 100.0;
    return result;
}

char assignLetterGrade(double percentage) {
    char result = 'F';                     // a different variable entirely
    // ...
    return result;
}
```

This independence is what makes functions genuinely modular. You can write, read, and test one without holding the rest of the program in your head.

9.6 Modular Design in Practice

Here is the rebuild, and the discipline that makes it safe.

Appendix D Section D.5 states the rule: **a function does one thing**. If you find yourself writing a comment that says "now for the second part," that is a second function asking to exist. If you cannot describe what a function does in one sentence without the word "and", split it.

Refactor incrementally, never in one leap

The completeness rule for this book says no chapter leaves the project in a state that does not build. That constrains *how* you refactor, not just what you end up with.

Extract one function, recompile, and rerun. Then the next.

1. Copy the letter-grade chain out of `main` into a new function.
2. Replace the original code with a call.
3. Rebuild. Run. Confirm the output is unchanged.
4. Only then extract the next one.

At every point in that sequence you have a working program. If step 3 fails, you know the cause is the twenty lines you just moved.

The tempting alternative — rewrite everything, then compile — produces a screen of errors with no way to tell which change caused which. Appendix D Section D.5 and the Chapter 2 habit of compiling early and often are the same advice, and this is where it starts to pay.

9.7 From Structure Chart to Function Set

In Chapter 5 you drew a structure chart. Figure 5.3 had four boxes under the root: **Read Assignment**, **Compute Percentage**, **Assign Letter Grade**, and **Print Report**.

Those boxes are your functions. The conversion is mechanical:

Structure chart box	Function
Read Assignment	`void readAssignment(double&, double&, int&)`
Compute Percentage	`double computePercentage(double, double)`
Assign Letter Grade	`char assignLetterGrade(double)`
Print Report	`void printReport(const std::string&, double, double, int)`

This is why Chapter 5 was worth the effort. The design decisions — what the parts are, which calls which — were made on paper when they were cheap to change. Chapter 9 is transcription.

Notice how the return types fall out of the box names. A box that *computes* something returns a value. A box that *does* something is `void`. If a box needs both, it is probably two boxes.

9.8 Commenting and Documenting Functions

Appendix D Section D.3 requires a documentation comment on every function another file can call. The form:

```
/**
 * Converts points to a percentage, capped and rounded by course policy.
 *
 * @param  earned    total points earned, including any bonus
 * @param  possible  total points possible
 * @return the percentage, rounded to one decimal place
 * @pre    possible > 0; the caller must check this first
 */
double computePercentage(double earned, double possible);
```

Tag	Use
`@param name`	What one parameter means, including restrictions on its value
`@return`	What the return value means
`@throws Type`	An exception this may raise — from Chapter 24
`@pre`	A precondition the caller must satisfy, which the function does not check

`@pre` deserves attention. `computePercentage` divides by `possible`. It could check for zero itself, but then every caller would have to interpret whatever it returned in that case. Stating the precondition instead makes the contract explicit: **the caller checks, the function assumes**.

Either choice is defensible. What matters is writing down which one you made — an unstated assumption is a defect waiting for the right input.

9.9 Testing Functions Individually

A function small enough to describe in one sentence is small enough to test on its own. Doing that is what makes modular design pay.

Write a short `main` that calls one function with values whose answers you worked out by hand:

```
#include <algorithm>
#include <cmath>
#include <iostream>

double computePercentage(double earned, double possible) {
    double raw = earned / possible * 100.0;
    return std::round(std::min(raw, 100.0) * 10.0) / 10.0;
}

char assignLetterGrade(double percentage) {
    if (percentage >= 90.0) { return 'A'; }
    if (percentage >= 80.0) { return 'B'; }
    if (percentage >= 70.0) { return 'C'; }
    if (percentage >= 60.0) { return 'D'; }
    return 'F';
}

int main() {                        // a temporary main, used only for testing
    std::cout << computePercentage(84.0, 100.0) << "\n";     // expect 84
    std::cout << computePercentage(0.0, 100.0) << "\n";      // expect 0
    std::cout << computePercentage(15.0, 10.0) << "\n";      // expect 100
    std::cout << assignLetterGrade(90.0) << "\n";            // expect A
    std::cout << assignLetterGrade(89.9) << "\n";            // expect B
    return 0;
}
```

```
84
0
100
A
B
```

Choose cases at the boundaries, exactly as Chapter 5's desk-check habit taught: zero, the cutoff, just below the cutoff, and the impossible-looking case. The ordinary case in the middle almost never finds a bug.

Chapter 16 turns this into a systematic practice with a proper test harness. For now, hand-checking each function as you write it is enough — and it is far more than most people do.

9.10 Worked Examples

Four small functions worth having.

Comparing floating-point values safely

Chapters 1, 3, and 6 all warned that == is unreliable on floating-point values. Here is the standard technique, finally:

```
/**
 * Reports whether two floating-point values are close enough to treat as equal.
 * @param  a          first value
```

```
 * @param  b          second value
 * @param  tolerance  how close counts as equal; defaults to one millionth
 */
bool nearlyEqual(double a, double b, double tolerance = 0.000001) {
    return std::abs(a - b) < tolerance;
}
```

```
0.1+0.2 == 0.3           false
nearlyEqual(0.1+0.2,0.3) true
nearlyEqual(0.3, 0.4)    false
```

Never `a == b`. Always "are they closer together than I care about?"

Validating a menu choice

```
/** Returns the first character of a line, or '?' if the line was empty. */
char readMenuChoice() {
    std::cout << "1) Add assignment   2) View report   3) Quit\n";
    std::cout << "Choice: ";
    std::string line;
    std::getline(std::cin, line);
    return line.empty() ? '?' : line[0];
}
```

Reading the whole line with `getline` and taking its first character sidesteps the `>>`-and-`getline` trap from Section 3.12 entirely. There is no `>>`, so there is no stray newline.

Clamping into a range

```
/** Restricts a value to lie between low and high inclusive. */
double clamp(double value, double low, double high) {
    return std::min(std::max(value, low), high);
}
```

Two library calls from Chapter 8, composed.

Applying a bonus

```
/** Adds bonus points to a raw score. Bonus defaults to none. */
double applyBonus(double pointsEarned, double bonusPoints = 0.0) {
    return pointsEarned + bonusPoints;
}
```

The `= 0.0` is a **default argument**: callers may omit it. `applyBonus(84.0)` and `applyBonus(84.0, 0.0)` are equivalent. Chapter 10 covers default arguments properly; this one is here because v1.5 will use it to extend an interface without breaking existing calls.

Common Errors and Warnings

What you see	Cause	Fix
`error: 'computePercentage' was not declared in this scope`	Called before declaring	Define it above `main`, or add a prototype

What you see	Cause	Fix
`undefined reference to 'computePercentage'`	Prototype without a definition — a **linker** error	Write the function body
`warning: no return statement in function returning non-void`	A path falls off the end	Return a value on every path
`warning: parameter 'x' set but not used`	Assigned to a parameter, changing only the copy	Use a reference (Chapter 10) or return a value
The caller's variable is unchanged	Pass by value	Return a value, or use a reference
`error: too few arguments to function`	Wrong number of arguments	Check the signature
`error: 'result' was not declared in this scope`	Used a local outside its function	Return it instead
Two `doubles` that should match compare unequal	`==` on floating-point	Use `nearlyEqual`

Design Notes

One function, one job, one sentence. If the sentence needs "and", split the function.

Extract one function at a time and rebuild between each. The project is never allowed to be broken between two chapters, and it should not be broken between two edits either.

Document the contract, especially preconditions. `@pre possible > 0` tells the caller what they are responsible for. Without it, both sides assume the other checked.

Test each function as you write it, with boundary values you computed by hand.

Write functions that return values where you can. They are easier to test, easier to reason about, and cannot surprise their callers.

Grade Calculator v1.0 — Modular Rebuild

What v1.0 does

Everything v0.7 did, rebuilt from functions derived from the Chapter 5 structure chart — plus a **menu loop**, so the program no longer runs once and exits.

This is the first version that behaves like an application. The menu shell it gains here remains in every version through Chapter 24.

Behavior worth stating plainly

v1.0 does not store individual assignments; it keeps running totals, exactly as v0.7 did. Arrays do not arrive until Chapter 11. What changes here is **structure**, not capability — and that is the point. A refactor that alters behavior is a defect, which Chapter 13 will make you prove.

The program

```
// Grade Calculator v1.0 - Chapter 9
// Same behavior as v0.7, rebuilt from documented functions, plus a menu loop.
// New this version: user-defined functions, a menu that keeps the program running.
```

```cpp
// Note: individual assignments are not stored yet (no arrays until Chapter 11),
// so the program keeps running totals only.
// Run: click Run in StudySite and use the embedded Terminal.

#include <cmath>
#include <iostream>
#include <iomanip>
#include <limits>
#include <string>

const double A_CUTOFF = 90.0;
const double B_CUTOFF = 80.0;
const double C_CUTOFF = 70.0;
const double D_CUTOFF = 60.0;
const bool CAP_AT_100 = true;

/**
 * Adds bonus points to a raw score.
 * @param  pointsEarned  raw points earned, must be >= 0
 * @param  bonusPoints   bonus to add, must be >= 0
 * @return the combined total
 */
double applyBonus(double pointsEarned, double bonusPoints) {
    return pointsEarned + bonusPoints;
}

/**
 * Converts points to a percentage, capped by course policy.
 * @param  earned    total points earned including bonus
 * @param  possible  total points possible, must be > 0
 * @return percentage rounded to one decimal place
 * @pre    possible > 0; the caller must check this first
 */
double computePercentage(double earned, double possible) {
    double raw = earned / possible * 100.0;
    double reported = CAP_AT_100 ? std::min(raw, 100.0) : raw;
    return std::round(reported * 10.0) / 10.0;
}

/**
 * Maps a percentage to a letter grade on the fixed course scale.
 * @param  percentage  a value from 0 upward
 * @return one of A, B, C, D, F
 */
char assignLetterGrade(double percentage) {
    if (percentage >= A_CUTOFF) { return 'A'; }
    if (percentage >= B_CUTOFF) { return 'B'; }
    if (percentage >= C_CUTOFF) { return 'C'; }
    if (percentage >= D_CUTOFF) { return 'D'; }
    return 'F';
}

/**
 * Prompts for and reads one assignment, updating the running totals.
 * @param  totalEarned    running total, updated in place
 * @param  totalPossible  running total, updated in place
 * @param  count          assignment count, updated in place
 */
void readAssignment(double& totalEarned, double& totalPossible, int& count) {
    std::string assignmentName;
```

```
    std::cout << "  Assignment name : ";
    std::getline(std::cin, assignmentName);

    double pointsEarned = 0.0;
    std::cout << "  Points earned   : ";
    std::cin >> pointsEarned;

    double pointsPossible = 0.0;
    std::cout << "  Points possible : ";
    std::cin >> pointsPossible;

    double bonusPoints = 0.0;
    std::cout << "  Bonus points    : ";
    std::cin >> bonusPoints;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    totalEarned += applyBonus(pointsEarned, bonusPoints);
    totalPossible += pointsPossible;
    ++count;
    std::cout << "  Recorded: " << assignmentName << "\n\n";
}

/**
 * Prints the full course report for the current totals.
 */
void printReport(const std::string& student, double earned, double possible, int count) {
    std::cout << "\n--- COURSE REPORT ---\n";
    std::cout << "Student:     " << student << "\n";
    std::cout << "Assignments: " << count << "\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Total:       " << earned << " / " << possible << "\n";

    if (count == 0 || possible <= 0.0) {
        std::cout << "No course grade is available yet.\n\n";
        return;
    }
    double percentage = computePercentage(earned, possible);
    std::cout << "Percentage:  " << percentage << "%\n";
    std::cout << "Grade:       " << assignLetterGrade(percentage) << "\n\n";
}

/**
 * Displays the menu and returns the user's choice as a character.
 */
char readMenuChoice() {
    std::cout << "1) Add assignment   2) View report   3) Quit\n";
    std::cout << "Choice: ";
    std::string line;
    std::getline(std::cin, line);
    return line.empty() ? '?' : line[0];
}

int main() {
    std::cout << "=== GRADE CALCULATOR v1.0 ===\n\n";

    std::string studentName;
    std::cout << "Student name: ";
    std::getline(std::cin, studentName);
    std::cout << "\n";
```

```
    double totalEarned = 0.0;
    double totalPossible = 0.0;
    int assignmentCount = 0;
    bool running = true;

    while (running) {
        char choice = readMenuChoice();
        switch (choice) {
            case '1':
                readAssignment(totalEarned, totalPossible, assignmentCount);
                break;
            case '2':
                printReport(studentName, totalEarned, totalPossible, assignmentCount);
                break;
            case '3':
                running = false;
                break;
            default:
                std::cout << "Please enter 1, 2, or 3.\n\n";
                break;
        }
    }

    printReport(studentName, totalEarned, totalPossible, assignmentCount);
    std::cout << "Goodbye.\n";
    return 0;
}
```

Expected output

With `Ada`, then `1` and `Homework 1 9/10 bonus 0`, then `1` and `Midterm 84/100 bonus 5`, then `2`, then `3`:

```
--- COURSE REPORT ---
Student:     Ada
Assignments: 2
Total:       98.0 / 110.0
Percentage:  89.1%
Grade:       B

Goodbye.
```

What to notice

Each function came from a box in Figure 5.3. `readAssignment`, `computePercentage`, `assignLetterGrade`, `printReport`. The design work was done four chapters ago.

`readAssignment` takes references — `double&` — and this is new. Chapter 10 explains the syntax properly. What matters now is that pass by value (Section 9.3) could not do this job: the function must update the caller's running totals, and a copy cannot. This is the limitation of Section 9.3 arriving exactly where predicted.

`printReport` calls `computePercentage`, which calls nothing. That layering is the structure chart, alive in the code.

The menu uses `switch` on a `char`. Chapter 6 Section 6.10 said `switch` cannot handle grade ranges but suits menu choices exactly. Here is the case it was made for.

`readMenuChoice` uses `getline`, not `>>`. No `>>` means no stray newline, so the trap from Section 3.12 never arises in the menu.

`printReport` is called twice — from the menu and again on exit. That is the value of a function: one definition, several uses, no duplication.

Your StudySite Lab — Rebuild with Functions

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v1.0
- **Starting point**: Your committed Chapter 8 v0.7 program (one file, one long `main`).

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

Your `main` has grown to seventy lines and does five unrelated jobs. This chapter rebuilds the same behavior out of six small functions, each with one job and a documentation comment stating what it takes, what it returns, and what it assumes. Then it adds the thing functions make possible: a **menu loop**, so the program keeps running and the user chooses what to do next instead of answering a fixed sequence of questions once.

Nothing about the grading changes. The output for the same inputs is identical. What changes is that the program becomes something you can read a piece at a time. By the end you will have:

1. `applyBonus`, `computePercentage`, and `assignLetterGrade` — three pure calculations that take values and return values.
2. `readAssignment` — reads one assignment and updates the running totals through **reference parameters**.
3. `printReport` and `readMenuChoice` — one prints, one asks.
4. A `main` of about thirty lines: set up, loop on a menu with a `switch`, print a final report, exit.

   Note: individual assignments are still not stored — there are no arrays until Chapter 11 — so the program keeps running totals only. Adding an assignment and viewing the report many times is now possible; listing the assignments is not, yet.

Files in this checkpoint

File	What you do to it
`main.cpp`	Edit (Steps 1-9)

When you are finished your project still has **one** `.cpp` file.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.

- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `main.cpp`. Confirm the editor shows the expected file paths before editing.
4. Continue the existing program in StudySite's internal editor, finish the steps in order, then Run.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Update the comment header

In `main.cpp`, find the eight header comment lines above the `Build:` line:

```
// Grade Calculator v0.7 - Chapter 8
// Correct rounding and a documented bonus-point cap policy.
// New this version: std::round, std::min, library functions replace hand-rolled logic.
//
// POLICY DECISION (documented per Chapter 4): this gradebook CAPS the reported
// percentage at 100. Bonus points can rescue a low score but cannot produce a
// grade above the maximum. Set CAP_AT_100 to false for uncapped reporting.
//
```

Replace it with:

```
// Grade Calculator v1.0 - Chapter 9
// Same behavior as v0.7, rebuilt from documented functions, plus a menu loop.
// New this version: user-defined functions, a menu that keeps the program running.
// Note: individual assignments are not stored yet (no arrays until Chapter 11),
// so the program keeps running totals only.
```

Why: The policy block moves out of the header because the policy is about to live inside `computePercentage`, documented on the function that implements it. A comment belongs as close as possible to the code it describes. What replaces it is the new limitation worth recording: no storage of individual assignments yet.

Step 2 — Write `applyBonus`

> **Where these go**: each function in Steps 2-7 is added **above** `int main()` and below the constants. Add them in order, so each new one appears directly above `int main()` and below the one you added before it. A function must be declared before the code that calls it, and `main` calls all of them.

In `main.cpp`, find the start of `main`:

```
int main() {
```

Directly above it, add:

```
/**
 * Adds bonus points to a raw score.
 * @param  pointsEarned  raw points earned, must be >= 0
```

```
 * @param  bonusPoints   bonus to add, must be >= 0
 * @return the combined total
 */
double applyBonus(double pointsEarned, double bonusPoints) {
    return pointsEarned + bonusPoints;
}
```

Why: This is the smallest useful function in the program — one line of arithmetic — and it is worth writing precisely because it is small. The name says what the operation *means*, where `pointsEarned + bonusPoints` only says what it does. If the bonus policy ever changes, there is now exactly one place to change it.

Read the documentation comment carefully, because every function from here on has one. `@param` describes each input and any constraint on it; `@return` describes what comes back. Writing this before the body forces you to decide what the function is responsible for, which is most of the work of designing one.

Step 3 — Write `computePercentage`

Again find the start of `main`:

```
int main() {
```

Directly above it, add:

```
/**
 * Converts points to a percentage, capped by course policy.
 * @param  earned    total points earned including bonus
 * @param  possible  total points possible, must be > 0
 * @return percentage rounded to one decimal place
 * @pre    possible > 0; the caller must check this first
 */
double computePercentage(double earned, double possible) {
    double raw = earned / possible * 100.0;
    double reported = CAP_AT_100 ? std::min(raw, 100.0) : raw;
    return std::round(reported * 10.0) / 10.0;
}
```

Why: The cap and the rounding from Chapter 8 move in here, unchanged. Now there is one place in the program that knows how a percentage is produced, and the policy constant is read at exactly one site.

Note the `@pre` line — a **precondition**. This function divides by `possible`, so it requires `possible > 0`, and it states that the *caller* must check. That is a real design decision: the alternative is for the function to check and return some sentinel value, which pushes the problem to every caller instead. Either is defensible; what matters is writing down which one you chose, because a caller cannot guess.

Step 4 — Write `assignLetterGrade`

Find the start of `main` once more:

```
int main() {
```

Directly above it, add:

```
/**
 * Maps a percentage to a letter grade on the fixed course scale.
 * @param  percentage  a value from 0 upward
 * @return one of A, B, C, D, F
 */
char assignLetterGrade(double percentage) {
    if (percentage >= A_CUTOFF) { return 'A'; }
```

```
    if (percentage >= B_CUTOFF) { return 'B'; }
    if (percentage >= C_CUTOFF) { return 'C'; }
    if (percentage >= D_CUTOFF) { return 'D'; }
    return 'F';
}
```

Why: Compare this to the `if`/`else if` chain it replaces. There is no `else if` and no `letter` variable — each test simply **returns** when it matches. `return` exits the function immediately, so reaching the second `if` already proves the first was false. The `else` is implied by control flow.

This is the clearest form this logic can take, and it is only available because the code is in a function. Inside `main` you had to assign to a variable and carry it along; here the answer leaves the moment it is known. The final `return 'F';` is the default, reached only when all four tests fail.

Step 5 — Write `readAssignment`

Find the start of `main`:

```
int main() {
```

Directly above it, add:

```
/**
 * Prompts for and reads one assignment, updating the running totals.
 * @param  totalEarned    running total, updated in place
 * @param  totalPossible  running total, updated in place
 * @param  count          assignment count, updated in place
 */
void readAssignment(double& totalEarned, double& totalPossible, int& count) {
    std::string assignmentName;
    std::cout << "  Assignment name : ";
    std::getline(std::cin, assignmentName);

    double pointsEarned = 0.0;
    std::cout << "  Points earned   : ";
    std::cin >> pointsEarned;

    double pointsPossible = 0.0;
    std::cout << "  Points possible : ";
    std::cin >> pointsPossible;

    double bonusPoints = 0.0;
    std::cout << "  Bonus points    : ";
    std::cin >> bonusPoints;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    totalEarned += applyBonus(pointsEarned, bonusPoints);
    totalPossible += pointsPossible;
    ++count;
    std::cout << "  Recorded: " << assignmentName << "\n\n";
}
```

Why: This is the chapter's most important new idea: the `&` in `double& totalEarned`. That is a **reference parameter**, and it means the function receives the caller's actual variable rather than a copy of its value. `totalEarned += ...` inside this function changes `main`'s `totalEarned`.

Without references this function could not work. A normal parameter is a copy; adding to the copy would update something that ceases to exist when the function returns, and `main`'s totals would stay at zero. The three accumulators must be references because updating them *is* the function's job.

Notice also that this function calls `applyBonus` rather than adding the bonus itself, and that the prompts and the `std::cin.ignore` call moved in here wholesale. Everything about reading one assignment now lives in one place.

Step 6 — Write `printReport`

Find the start of `main`:

```
int main() {
```

Directly above it, add:

```
/**
 * Prints the full course report for the current totals.
 */
void printReport(const std::string& student, double earned, double possible, int count) {
    std::cout << "\n--- COURSE REPORT ---\n";
    std::cout << "Student:     " << student << "\n";
    std::cout << "Assignments: " << count << "\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Total:       " << earned << " / " << possible << "\n";

    if (count == 0 || possible <= 0.0) {
        std::cout << "No course grade is available yet.\n\n";
        return;
    }
    double percentage = computePercentage(earned, possible);
    std::cout << "Percentage:  " << percentage << "%\n";
    std::cout << "Grade:       " << assignLetterGrade(percentage) << "\n\n";
}
```

Why: `const std::string& student` is a reference too, but for the opposite reason. Copying a string means copying its characters, which is wasted work; passing a reference avoids that. The `const` then promises the function will not modify it — so the reference is purely an efficiency measure, with no change in meaning. **`const` reference for input you only read, plain reference for values you intend to change** is the convention worth internalizing.

The guard clause from Chapter 8 survives as an early `return` from this function, and the function calls `computePercentage` and `assignLetterGrade` rather than repeating their logic. This is what the rebuild buys: the report reads like a description of itself.

Step 7 — Write `readMenuChoice`

Find the start of `main` for the last time:

```
int main() {
```

Directly above it, add:

```
/**
 * Displays the menu and returns the user's choice as a character.
 */
char readMenuChoice() {
    std::cout << "1) Add assignment   2) View report   3) Quit\n";
    std::cout << "Choice: ";
    std::string line;
    std::getline(std::cin, line);
    return line.empty() ? '?' : line[0];
}
```

Why: This reads a whole line with `getline` and returns only its first character. Reading the line — rather than a single `char` with `>>` — is what keeps the input buffer clean, so the `getline` inside `readAssignment` is not derailed by a leftover newline. It is the Chapter 7 lesson applied structurally: handle input a line at a time and the buffer problem stops recurring.

`line.empty() ? '?' : line[0]` guards the case where the user just presses Enter. Reading `line[0]` of an empty string is undefined behavior — it might appear to work, which is worse than crashing. Returning `'?'` sends it to the `default` branch and the program asks again.

> ▸ **Run checkpoint**: Click **Run**. The program should behave exactly as it did in Chapter 8 — the six new functions are defined but nothing calls them yet. This checkpoint proves all six compile before you rewrite `main` around them.

Step 8 — Rebuild `main` around the menu

In `main.cpp`, find the first line inside `main`:

```
    std::cout << "=== GRADE CALCULATOR v0.7 ===\n\n";
```

Delete that line and everything below it (about seventy lines), down to and including this line:

```
    return 0;
```

In the space where that code was, enter this — the whole of the new `main`:

```
    std::cout << "=== GRADE CALCULATOR v1.0 ===\n\n";

    std::string studentName;
    std::cout << "Student name: ";
    std::getline(std::cin, studentName);
    std::cout << "\n";

    double totalEarned = 0.0;
    double totalPossible = 0.0;
    int assignmentCount = 0;
    bool running = true;

    while (running) {
        char choice = readMenuChoice();
        switch (choice) {
            case '1':
                readAssignment(totalEarned, totalPossible, assignmentCount);
                break;
            case '2':
                printReport(studentName, totalEarned, totalPossible, assignmentCount);
                break;
            case '3':
                running = false;
                break;
            default:
                std::cout << "Please enter 1, 2, or 3.\n\n";
                break;
        }
    }

    printReport(studentName, totalEarned, totalPossible, assignmentCount);
    std::cout << "Goodbye.\n";
    return 0;
```

Why: `main` is now a description of the program rather than the program itself: greet, set up the totals, loop

until the user quits, print a final report. Every line of detail lives in a function with a name.

The **menu loop** is the new behavior. `while (running)` keeps going until something sets `running` to `false`, and the `switch` dispatches on the character returned by `readMenuChoice`. A `switch` is the natural fit when you are choosing among a fixed set of discrete values; it reads better than four chained `if`s comparing the same variable.

Two details that matter. Every `case` ends with `break` — without it, control **falls through** into the next case and runs its code too, which is a classic bug and the reason `break` is not optional here. And `default` catches everything unmatched, so an unexpected keystroke produces a helpful message rather than silence.

The final `printReport` call after the loop means the student always sees their grade on the way out, whether or not they chose option 2.

Step 9 — Remove the now-unused sentinel

In `main.cpp`, find the `SENTINEL` constant and delete it:

```
const std::string SENTINEL = "done";
```

Why: The menu replaced the sentinel: the user now chooses `3` to quit instead of typing `done` as an assignment name. A constant nothing references is dead weight that invites the next reader to wonder what it is for. Deleting code that has stopped being used is part of finishing a change, not an afterthought.

▸ **Run checkpoint**: Click **Run** and work through the test plan below in the Terminal.

Test plan

1. **The menu works**. Enter student `Ada Lovelace`. You should see the menu. Choose `2` straight away: the report shows `Assignments: 0` and `No course grade is available yet.` Then choose `3`: the report prints once more, followed by `Goodbye.`

2. **Add, view, add, view**. Restart and choose `1`, entering `Essay`, `45`, `50`, `0`. Choose `2` — total `45.0 / 50.0`, percentage `90.0%`, grade `A`. Choose `1` again with `Quiz`, `10`, `50`, `0`. Choose `2` — total `55.0 / 100.0`, `55.0%`, grade `F`. Being able to view a report *between* entries is the new capability this chapter unlocked.

3. **Invalid input**. At the menu, type `x` and press Enter: `Please enter 1, 2, or 3.` and the menu returns. Then press Enter on its own with no text: the same message, via the `'?'` from `readMenuChoice`. Neither should crash or skip a prompt.

4. **Multi-word names still work**. Choose `1` and enter `Final Research Paper` as the assignment name. It must be stored in full and echoed by the `Recorded:` line.

5. **Identical results**. Run the same inputs you used in Chapter 8 and confirm the percentage and grade are the same. This chapter restructured the program; it must not have changed its answers.

Check your work

Then confirm:

- ☐ `main.cpp` compiles with no errors and **no warnings**.
- ☐ `main` is about thirty lines and contains no arithmetic.
- ☐ Each of the six functions has a documentation comment.
- ☐ The three accumulator parameters of `readAssignment` are references (`double&`, `int&`), and the totals in `main` actually change.
- ☐ `printReport` takes the student name as a `const std::string&`.
- ☐ Every `case` in the `switch` ends with `break`.

- ☐ Choosing `3` prints the report and then `Goodbye.`
- ☐ `SENTINEL` no longer appears anywhere in the file.
- ☐ The same inputs produce the same grade as your Chapter 8 program.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 9 Grade Calculator v1.0**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 9 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Your first function

```cpp
#include <iostream>

double computePercentage(double earned, double possible) {
    return earned / possible * 100.0;
}

int main() {
    std::cout << computePercentage(84.0, 100.0) << "\n";
    std::cout << computePercentage(9.0, 10.0) << "\n";
    return 0;
}
```

Expected output:

```
84
90
```

Try: Add a third call with `possible` of `0`. What happens? Then add a `@pre` comment saying whose responsibility that is.

2. Pass by value

```
#include <iostream>

double doubled(double points) {
    points = points * 2.0;
    return points;
}

int main() {
    double score = 84.0;
    double result = doubled(score);
    std::cout << "returned: " << result << "\n";
    std::cout << "original: " << score << "\n";
    return 0;
}
```

Expected output:

```
returned: 168
original: 84
```

Try: Explain in one sentence why `score` is unchanged. What would you have to do to change it? Chapter 10 has the answer.

3. Comparing floating-point values safely

```
#include <cmath>
#include <iostream>

bool nearlyEqual(double a, double b, double tolerance = 0.000001) {
    return std::abs(a - b) < tolerance;
}

int main() {
    double a = 0.1 + 0.2;
    std::cout << std::boolalpha;
    std::cout << "0.1+0.2 == 0.3           " << (a == 0.3) << "\n";
    std::cout << "nearlyEqual(0.1+0.2,0.3) " << nearlyEqual(a, 0.3) << "\n";
    std::cout << "nearlyEqual(0.3, 0.4)    " << nearlyEqual(0.3, 0.4) << "\n";
    return 0;
}
```

Expected output:

```
0.1+0.2 == 0.3           false
nearlyEqual(0.1+0.2,0.3) true
nearlyEqual(0.3, 0.4)    false
```

Try: Call it with a tolerance of `0.2` comparing 0.3 and 0.4. What does that tell you about choosing a tolerance?

4. `return` exits immediately

```
#include <iostream>

char assignLetterGrade(double percentage) {
    if (percentage >= 90.0) { return 'A'; }
    if (percentage >= 80.0) { return 'B'; }
    if (percentage >= 70.0) { return 'C'; }
```

```
    return 'F';
}

int main() {
    std::cout << assignLetterGrade(95.0) << "\n";
    std::cout << assignLetterGrade(85.0) << "\n";
    std::cout << assignLetterGrade(50.0) << "\n";
    return 0;
}
```

Expected output:

```
A
B
F
```

Try: Add `std::cout << "checking B\n";` before the second `if`. Call with 95.0. Does the message print? Why not?

5. `void` and early return

```
#include <iostream>

void describe(int count) {
    if (count == 0) {
        std::cout << "No assignments yet.\n";
        return;                    // exit early
    }
    std::cout << count << " assignment(s) recorded.\n";
}

int main() {
    describe(0);
    describe(3);
    return 0;
}
```

Expected output:

```
No assignments yet.
3 assignment(s) recorded.
```

Try: Remove the `return;`. What does `describe(0)` print now, and why is that wrong?

6. Decompose a long function

This works and does too much. Split it into three functions, each describable in one sentence without "and". Then write a documentation comment for each.

```
#include <iostream>

int main() {
    double earned = 0.0;
    double possible = 0.0;
    std::cout << "Points earned: ";
    std::cin >> earned;
    std::cout << "Points possible: ";
    std::cin >> possible;
```

```
    double pct = 0.0;
    if (possible > 0.0) {
        pct = earned / possible * 100.0;
    }

    char letter = 'F';
    if (pct >= 90.0)      { letter = 'A'; }
    else if (pct >= 80.0) { letter = 'B'; }
    else if (pct >= 70.0) { letter = 'C'; }
    else if (pct >= 60.0) { letter = 'D'; }

    std::cout << "Percentage: " << pct << "%\n";
    std::cout << "Grade: " << letter << "\n";
    return 0;
}
```

7. Trigger a linker error deliberately

```
#include <iostream>

double mysteryFunction(double x);      // prototype, no definition

int main() {
    std::cout << mysteryFunction(3.0) << "\n";
    return 0;
}
```

Build it. **The message is not a compiler error** — it comes from the linker, and looks completely different from anything in Chapter 2.

- Which of the four translation stages produced it?
- Why did the compiler not object?
- What is the fix?

This is Chapter 2 Section 2.2.4 made real, and it is worth two minutes.

Summary

- A **function definition** has a return type, a name, a parameter list, and a body.
- A **prototype** declares a function without defining it. A prototype with no definition passes the compiler and **fails at the linker**.
- **Pass by value** gives the function a copy. Changing a parameter does not affect the caller. Chapter 10 introduces references for when it must.
- `return` produces a value **and exits immediately**, which is why a chain of `if` statements with `return` needs no `else`.
- A **`void`** function does something rather than computing something. `return;` exits it early.
- **Local variables** have the scope and lifetime of their function. Two functions may reuse a name freely.
- **A function does one thing**, describable in one sentence without "and".
- **Refactor one function at a time, rebuilding between each.** The program should never be broken between two edits.
- A **structure chart box becomes a function.** Boxes that compute return values; boxes that act are `void`.

- Document every function with `@param`, `@return`, and especially `@pre` — preconditions say who is responsible for checking.
- **Test each function individually with boundary values** you computed by hand.
- Compare floating-point values with a **tolerance**, never with ==.

Key Terms

default argument — a parameter value used when the caller omits that argument.

definition — a function's full declaration together with its body.

lifetime — the period during which a variable exists.

local variable — a variable declared inside a function, existing only while it runs.

modular design — organizing a program as small parts, each doing one thing.

parameter — the name a function uses for a value it receives.

pass by value — passing a copy of an argument, so the caller's variable is unaffected.

precondition — a requirement the caller must satisfy, stated with `@pre`.

prototype — a declaration of a function's signature without its body.

return — a statement producing a function's result and exiting it.

signature — a function's name and parameter types.

void — the return type of a function that returns no value.

Next: Chapter 10 splits the Grade Calculator across three files, introduces references so a function can genuinely modify its caller's variables, and makes input validation robust enough that no typed input can crash the program. Grade Calculator v1.1.

Chapter 10 — Managing Functions and Data

Learning Objectives

When you finish this chapter you will be able to:

- Explain why global variables are avoided, and when `static` is appropriate. *(SLO 1.4)*
- Write overloaded functions and default arguments. *(SLO 1.4, 1.7)*
- Write a simple recursive function and identify its base case. *(SLO 1.4)*
- Explain what a pointer holds, and use the address-of and dereference operators. *(SLO 1.1, 1.4)*
- Use reference parameters so a function can modify its caller's variable. *(SLO 1.4)*
- Explain why large objects are passed by `const` reference. *(SLO 1.4)*
- Split a program across a header and two implementation files, with a header guard. *(SLO 1.4, 1.6)*
- Build Grade Calculator v1.1 — three files, and input that cannot crash.

10.1 Global and Static Variables

Chapter 9 gave you functions with local variables. This chapter is about the machinery that makes a *program* out of a collection of functions.

A **global variable** is declared outside every function and visible to all of them:

```
#include <iostream>

double totalEarned = 0.0;    // global — avoid this

void addPoints(double p) {
    totalEarned += p;        // no parameter needed
}
```

It looks convenient. It is not.

Any function can change a global, so no function can be understood alone. When `totalEarned` holds a wrong value, the suspects are every function in the program. Testing one function in isolation becomes impossible, because its behavior depends on state some other function may have left behind.

Appendix D Section D.11 excludes globals. **Pass what a function needs and return what it produces**. The `const` values at the top of the Grade Calculator — `A_CUTOFF`, `CAP_AT_100` — are not exceptions to this rule: they cannot be modified, so they cannot be the source of a surprise.

`static` local variables

A `static` local keeps its value between calls:

```
int nextId() {
    static int counter = 1000;    // initialized once, ever
    return ++counter;
}
```

Each call returns 1001, 1002, 1003. The variable is created once and survives after the function returns, but remains invisible outside it.

This is a narrower and safer tool than a global — the state is hidden inside the one function that uses it. Use it sparingly. A function with `static` state is not testable in isolation either, because calling it twice gives different answers.

10.2 Overloaded Functions

C++ allows several functions with the same name, provided their **parameter lists differ**:

```
int overloaded(int a)       { return a * 2; }
double overloaded(double a) { return a * 2.5; }
```

```
overloaded(3)   = 6
overloaded(3.0) = 7.5
```

The compiler chooses by the argument types. This is **overload resolution**, and it happens entirely at compile time.

Overloading earns its place when several functions do conceptually the same thing to different types:

```
std::string formatGrade(double percentage);   // "89.1%"
std::string formatGrade(char letter);         // "B"
```

One name, one idea, two representations. The caller does not have to remember `formatGradeAsPercentage` and `formatGradeAsLetter`.

> **The return type is not part of the signature.** Two functions differing *only* in return type will not compile — the compiler has no way to tell which one a call means.

10.3 Default Arguments

A parameter may have a default the caller can omit:

```
double applyBonus(double pointsEarned, double bonusPoints = 0.0) {
    return pointsEarned + bonusPoints;
}
```

```
applyBonus(84.0, 5.0);    // 89.0
applyBonus(84.0);         // 84.0 — bonus defaults to 0.0
```

Two rules govern them. **Defaults must come last** — you cannot default a parameter and then require one after it, because a caller omitting the first would have no way to supply the second. And **the default belongs in the declaration**, which means in the header when a function is declared there, not repeated in the definition.

Default arguments are how you extend an interface without breaking existing calls. Adding `bonusPoints = 0.0` to a function that previously took one argument leaves every old call compiling and behaving exactly as

before. v1.1 uses this.

10.4 Recursion

A **recursive** function calls itself. Every recursive function needs two things:

- a **base case** that returns without recursing, and
- a **recursive case** that moves closer to the base case.

```
int factorial(int n) {
    return (n <= 1) ? 1 : n * factorial(n - 1);
}
```

```
factorial(5) = 120
```

Trace it: `factorial(5)` needs `factorial(4)`, which needs `factorial(3)`, down to `factorial(1)`, which returns 1 without recursing. The results then multiply back up: 1, 2, 6, 24, 120.

Omit the base case and the function recurses forever, consuming memory until the program crashes with a *stack overflow*. That is the recursive equivalent of Chapter 7's infinite loop, and it fails harder.

Recursion suits problems that are naturally self-similar — tree structures, and the merge sort in Chapter 17. For anything a loop expresses naturally, use a loop: it is clearer and cheaper. The Grade Calculator never needs recursion, and this section exists because it appears throughout computing and you should recognize it.

10.5 Pointers

Chapter 1 Section 1.3 said that every value in memory has an **address**, and that the address is itself a number that can be stored. A **pointer** is a variable that stores one.

```
double score = 84.0;
double* ptr = &score;      // ptr holds the address of score
```

Two operators:

Operator	Name	Meaning
&	address-of	gives the address of a variable
*	dereference	gives the value at an address

```
std::cout << score;      // 84 — the value
std::cout << ptr;        // an address, e.g. 0x7ffd4c
std::cout << *ptr;       // 84 — the value at that address
```

* appears in two roles, which is a genuine source of confusion. In a *declaration*, `double* ptr` means "ptr is a pointer to double." In an *expression*, `*ptr` means "the value ptr points at." Same symbol, different jobs.

> **A pointer that points at nothing should be `nullptr`.** Dereferencing a null or uninitialized pointer is a run-time error that usually crashes the program. Chapter 22 covers pointer safety properly.

10.6 Reference Variables

A **reference** is another name for an existing variable:

```
double score = 84.0;
double& alias = score;     // alias IS score

alias = 91.0;
std::cout << score;        // 91 — they are the same variable
```

```
score  = 84
*ptr   = 84
alias  = 84
after alias = 91.0, score = 91
```

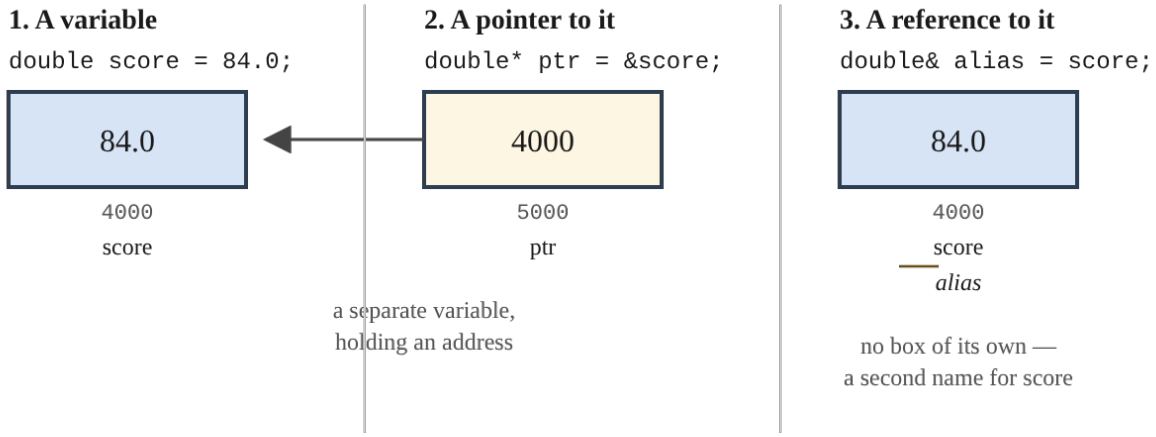


Figure 10: Three panels showing a variable score at address 4000 holding 84.0, a pointer ptr at address 5000 holding the value 4000 with an arrow to score, and a reference alias drawn as a second label on the same box at address 4000.

Figure 10.1 — A variable, a pointer to it, and a reference to it.

Description of Figure 10.1. Three panels. **Panel 1** shows a variable `score` at address 4000 holding 84.0. **Panel 2** shows a pointer `ptr`, itself a variable at address 5000, holding the value 4000, with an arrow pointing back to `score` — a separate variable that stores an address. **Panel 3** shows a reference `alias` with no box of its own, drawn as a second label attached to the same box at address 4000 — another name for an existing variable, occupying no storage. Notes record that a pointer stores an address and can be redirected or left empty, while a reference is bound to one variable when created and can never be moved. Both let a function reach the caller's variable; references are simpler, so prefer them.

	Pointer	Reference
Is a separate variable	yes	no
Must be initialized	no	**yes**
Can be reassigned to another variable	yes	no
Can be empty	yes, `nullptr`	no

	Pointer	Reference
Syntax to use the value	`*ptr`	just the name

A reference is simpler in every respect that matters here, which is why it is the tool for the next section.

10.7 Pass by Reference

Chapter 9 Section 9.3 showed the limitation: pass by value gives a function a copy, so it cannot change the caller's variable. Here is the fix.

Via references

Add `&` to the parameter type:

```
void byReference(double& p) { p = 999.0; }

double s = 84.0;
byReference(s);
std::cout << s;      // 999
```

```
after byValue:     84
after byPointer:   999
after byReference: 999
```

The parameter `p` is not a copy — it is another name for `s`. The call site looks identical to a pass-by-value call, which is worth noting: **you cannot tell from the call whether a function may modify its argument**. You have to read the signature. That is a real cost, and the reason a function that modifies a parameter should say so in its name and its documentation.

Via pointers

The same job with a pointer requires more punctuation:

```
void byPointer(double* p) { *p = 999.0; }

byPointer(&s);       // the & is required here
```

Prefer references. Use a pointer only when the argument might legitimately be absent, since a pointer can be `nullptr` and a reference cannot.

`const` references for large objects

There is a second, more common reason to use a reference — efficiency:

```
void printReport(const std::string& student, double percentage);
```

Passing a `std::string` by value copies every character. Passing by reference copies nothing. The `const` promises the function will not modify it, so you get the speed of a reference with the safety of a copy.

Appendix D Section D.5 gives the rule: **pass large objects by `const` reference; pass small values — `int`, `double`, `char`, `bool` — by value**. A reference to a `double` is no cheaper than the `double` itself.

You have been using `const std::string&` since v1.0 without explanation. This is it.

10.8 Splitting a Program Across Files

One file works until it does not. Splitting a program lets you find things, compile only what changed, and reuse code across projects.

The standard division is **declarations in a header, definitions in an implementation file**.

The header — `gradelib.h`

```
#ifndef GRADELIB_H
#define GRADELIB_H

#include <string>

const double A_CUTOFF = 90.0;

/** Adds bonus points to a raw score. Bonus defaults to none. */
double applyBonus(double pointsEarned, double bonusPoints = 0.0);

/** Maps a percentage to a letter grade. */
char assignLetterGrade(double percentage);

#endif
```

The implementation — `gradelib.cpp`

```
#include "gradelib.h"

double applyBonus(double pointsEarned, double bonusPoints) {
    return pointsEarned + bonusPoints;      // no default repeated here
}

char assignLetterGrade(double percentage) {
    if (percentage >= A_CUTOFF) { return 'A'; }
    return 'F';
}
```

The user — `main.cpp`

```
#include "gradelib.h"
#include <iostream>

int main() {
    std::cout << assignLetterGrade(95.0) << "\n";
    return 0;
}
```

Building it

```
g++ -std=c++17 -Wall -Wextra main.cpp gradelib.cpp -o gradecalc
```

Both `.cpp` files are named. Headers are never compiled directly — they are pulled in by `#include`, exactly as Chapter 2 Section 2.2.1 described.

Header guards

Every header is wrapped in a **header guard**:

```
#ifndef GRADELIB_H
#define GRADELIB_H
// ...
#endif
```

The first time the preprocessor sees this file, `GRADELIB_H` is not defined, so it defines it and includes the contents. Any later time, `GRADELIB_H` *is* defined, so everything between is skipped.

Without a guard, a header included twice — easily done indirectly — declares everything twice, and the compiler reports errors about redefinition that point at a file you never edited.

Appendix D Section D.4 names the guard after the file in capitals, and prefers guards to `#pragma once` because guards are standard C++ and `#pragma once` is not.

Include order

Appendix D Section D.4 groups includes, most specific first, with blank lines between:

```
#include "gradelib.h"      // the header this file implements

#include <iostream>        // standard library
#include <string>
```

Putting the file's own header first is deliberate: if that header is missing an include it needs, this order makes the compiler say so immediately rather than hiding the problem.

10.9 Making Functions Reusable

A function is reusable when it depends only on its parameters. That means:

- **No globals.** Its behavior must not depend on hidden state.
- **No input or output**, unless that is its job. A function that computes *and* prints can only be used where you want both.
- **A clear contract**, documented with `@param`, `@return`, and `@pre`.
- **A general name.** `computePercentage` travels; `doTheMidtermThing` does not.

Compare:

```
// Not reusable: prints, and depends on a global.
void showGrade() {
    std::cout << totalEarned / totalPossible * 100.0 << "\n";
}

// Reusable: takes what it needs, returns what it computes.
double computePercentage(double earned, double possible);
```

The second can be tested with hand-computed values, called from a report, called from a test harness, and moved into another program unchanged.

Common Errors and Warnings

What you see	Cause	Fix
`undefined reference to 'f'`	Declared in the header, not defined; or the `.cpp` was left off the build	Define it; list every `.cpp` on the command line
`error: redefinition of 'X'`	Header included twice without a guard	Add a header guard
`error: 'A_CUTOFF' was not declared`	Header not included where needed	`#include "gradelib.h"`
`error: default argument given for parameter 1`	Default repeated in the definition	Put it only in the declaration
The caller's variable is unchanged	Passed by value	Add `&` to the parameter type
`error: invalid initialization of reference`	Passed a literal to a non-`const` reference	Pass a variable, or take `const&`
Program crashes on dereference	Null or uninitialized pointer	Initialize; check for `nullptr`
Stack overflow	Recursion with no reachable base case	Add or fix the base case
`error: call of overloaded 'f(...)' is ambiguous`	Argument types match two overloads equally	Cast the argument, or rename one

---

Design Notes

No globals. Pass what a function needs; return what it produces.

Prefer references to pointers. Use a pointer only when the value may legitimately be absent.

`const` reference for anything large, by value for anything small.

Name a function that modifies its argument so the modification is expected. `readAssignment(totalEarned, ...)` reads as changing something. `check(x)` does not.

One header, one implementation file, one purpose. Related functions together; unrelated functions apart.

Grade Calculator v1.1 — Multi-File and Reusable

What v1.1 does

Everything v1.0 did, split across three files, with input validation that no typed input can defeat.

The header — `gradelib.h`

```
// gradelib.h - Grade Calculator v1.1 - Chapter 10
// Declarations for the grading library. Separating declarations from
// definitions lets main.cpp use these functions without seeing their code.
#ifndef GRADELIB_H
#define GRADELIB_H

#include <string>

const double A_CUTOFF = 90.0;
const double B_CUTOFF = 80.0;
const double C_CUTOFF = 70.0;
const double D_CUTOFF = 60.0;
const bool CAP_AT_100 = true;
```

```
/** Adds bonus points to a raw score. Bonus defaults to none. */
double applyBonus(double pointsEarned, double bonusPoints = 0.0);

/** Converts points to a capped, rounded percentage. Requires possible > 0. */
double computePercentage(double earned, double possible);

/** Maps a percentage to a letter grade on the fixed course scale. */
char assignLetterGrade(double percentage);

/** Overload 1: formats a percentage for display, e.g. "89.1%". */
std::string formatGrade(double percentage);

/** Overload 2: formats a letter grade for display, e.g. "B". */
std::string formatGrade(char letter);

/**
 * Reads a non-negative number from the user, re-prompting until valid.
 * @param  prompt  text shown to the user
 * @param  value   receives the validated number
 * @return true if a value was read, false if input closed unexpectedly
 */
bool readNonNegative(const std::string& prompt, double& value);

/** Reads a whole line of text, re-prompting if it is empty. */
std::string readNonEmptyLine(const std::string& prompt);

#endif
```

The implementation — `gradelib.cpp`

```
// gradelib.cpp - Grade Calculator v1.1 - Chapter 10
// Definitions for the grading library declared in gradelib.h.
#include "gradelib.h"

#include <cmath>
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>

double applyBonus(double pointsEarned, double bonusPoints) {
    return pointsEarned + bonusPoints;
}

double computePercentage(double earned, double possible) {
    double raw = earned / possible * 100.0;
    double reported = CAP_AT_100 ? std::min(raw, 100.0) : raw;
    return std::round(reported * 10.0) / 10.0;
}

char assignLetterGrade(double percentage) {
    if (percentage >= A_CUTOFF) { return 'A'; }
    if (percentage >= B_CUTOFF) { return 'B'; }
    if (percentage >= C_CUTOFF) { return 'C'; }
    if (percentage >= D_CUTOFF) { return 'D'; }
    return 'F';
}

std::string formatGrade(double percentage) {
```

```
    std::ostringstream out;
    out << std::fixed << std::setprecision(1) << percentage << "%";
    return out.str();
}

std::string formatGrade(char letter) {
    return std::string(1, letter);
}

bool readNonNegative(const std::string& prompt, double& value) {
    while (true) {
        std::cout << prompt;
        if (!(std::cin >> value)) {
            if (std::cin.eof()) {
                return false;   // input closed; caller decides what to do
            }
            // Not a number. Clear the error state and discard the bad input,
            // otherwise every later read fails too.
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "  That is not a number. Please try again.\n";
            continue;
        }
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        if (value < 0.0) {
            std::cout << "  Points cannot be negative. Please try again.\n";
            continue;
        }
        return true;
    }
}

std::string readNonEmptyLine(const std::string& prompt) {
    std::string line;
    while (true) {
        std::cout << prompt;
        if (!std::getline(std::cin, line)) {
            return "";
        }
        if (!line.empty()) {
            return line;
        }
        std::cout << "  This cannot be blank. Please try again.\n";
    }
}
```

The program — `main.cpp`

```
// main.cpp - Grade Calculator v1.1 - Chapter 10
// Same behavior as v1.0, now split across three files with validated input.
// Run: click Run in StudySite and use the embedded Terminal.
#include "gradelib.h"

#include <iomanip>
#include <iostream>
#include <string>

void readAssignment(double& totalEarned, double& totalPossible, int& count) {
    std::string name = readNonEmptyLine("  Assignment name : ");
```

```
    double earned = 0.0;
    double possible = 0.0;
    double bonus = 0.0;
    if (!readNonNegative("  Points earned   : ", earned))   { return; }
    if (!readNonNegative("  Points possible : ", possible)) { return; }
    if (!readNonNegative("  Bonus points    : ", bonus))    { return; }

    totalEarned += applyBonus(earned, bonus);
    totalPossible += possible;
    ++count;
    std::cout << "  Recorded: " << name << "\n\n";
}

void printReport(const std::string& student, double earned, double possible, int count) {
    std::cout << "\n--- COURSE REPORT ---\n";
    std::cout << "Student:     " << student << "\n";
    std::cout << "Assignments: " << count << "\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "Total:       " << earned << " / " << possible << "\n";

    if (count == 0 || possible <= 0.0) {
        std::cout << "No course grade is available yet.\n\n";
        return;
    }
    double percentage = computePercentage(earned, possible);
    std::cout << "Percentage:  " << formatGrade(percentage) << "\n";
    std::cout << "Grade:       " << formatGrade(assignLetterGrade(percentage)) << "\n\n";
}

int main() {
    std::cout << "=== GRADE CALCULATOR v1.1 ===\n\n";
    std::string studentName = readNonEmptyLine("Student name: ");
    std::cout << "\n";

    double totalEarned = 0.0;
    double totalPossible = 0.0;
    int assignmentCount = 0;
    bool running = true;

    while (running) {
        std::cout << "1) Add assignment   2) View report   3) Quit\nChoice: ";
        std::string line;
        if (!std::getline(std::cin, line)) { break; }
        char choice = line.empty() ? '?' : line[0];

        switch (choice) {
            case '1': readAssignment(totalEarned, totalPossible, assignmentCount); break;
            case '2': printReport(studentName, totalEarned, totalPossible, assignmentCount); break;
            case '3': running = false; break;
            default:  std::cout << "Please enter 1, 2, or 3.\n\n"; break;
        }
    }

    printReport(studentName, totalEarned, totalPossible, assignmentCount);
    std::cout << "Goodbye.\n";
    return 0;
}
```

Expected output

Keep `main.cpp`, `gradelib.h`, and `gradelib.cpp` all open in the editor so **Run** compiles the whole program.

With `Ada`, then `1` and `Homework 1` with `abc` typed for points earned, then `9`, `10`, `0`, then `2`, then `3`:

```
  That is not a number. Please try again.

--- COURSE REPORT ---
Student:     Ada
Assignments: 1
Total:       9.0 / 10.0
Percentage:  90.0%
Grade:       A

Goodbye.
```

The `abc` was rejected, the prompt repeated, and the program carried on. **No typed input can crash v1.1.**

What to notice

`readNonNegative` takes `double& value` and returns `bool`. This is the pattern for a function that must produce two things: the value comes back through the reference, and success or failure comes back as the return value. It is exactly the case Section 9.3 said pass by value could not handle.

`std::cin.clear()` then `std::cin.ignore(...)`. When >> fails on non-numeric input, the stream enters a failed state and **stays there** — every later read fails instantly. `clear()` resets the state; `ignore()` discards the offending characters. Omit either one and the program spins forever printing the error message. Try it.

`formatGrade` is overloaded, once for `double` and once for `char`. One name, one idea.

`applyBonus` has a default argument in the header only. Repeating it in `gradelib.cpp` is an error, which is a rule people forget exactly once.

Every function parameter that is a `std::string` is `const std::string&`. No copying, no modification.

`gradelib.cpp` has no `main`, and `main.cpp` has no grading logic. Either file could be replaced without touching the other, which is what Section 10.9 means by reusable.

Your StudySite Lab — Organize Reusable Functions

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v1.1
- **Starting point**: Your committed Chapter 9 v1.0 program (one file, six functions, menu loop).

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

One file with six functions works, but the grading rules and the program that uses them are tangled together. This chapter separates them into a reusable **library**: a header file declaring what the library offers, and an implementation file containing the code. `main.cpp` then uses those functions without seeing how they work.

While the functions are moving, two new capabilities go in that belong to the library rather than to `main`: **input validation** that re-prompts instead of accepting nonsense, and **overloaded** formatting functions. By the end you will have:

1. `gradelib.h` — the constants, seven function declarations, and an include guard.
2. `gradelib.cpp` — the definitions, including two new validating readers that re-prompt until the input is usable.
3. Two functions both named `formatGrade`, distinguished only by their parameter type — an **overload**.
4. A default argument on `applyBonus`, so a call that omits the bonus means "no bonus".
5. A `main.cpp` of about seventy lines that includes `"gradelib.h"` and contains only the program's flow.

   The build command changes this chapter. Two `.cpp` files must be compiled together. In StudySite, both must be open in the editor when you click **Run**; a `.cpp` file that is not open is not compiled, and the linker error you get names the missing function, not the missing tab.

Files in this checkpoint

File	What you do to it
`gradelib.h`	Create (Step 1)
`gradelib.cpp`	Create (Step 2)
`main.cpp`	Edit (Steps 3-8)

When you are finished your project has **three** files.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `main.cpp`. Confirm the editor shows the expected file paths before editing.
4. Create the two new files in StudySite's internal editor, then finish the steps in order. Keep all three files open, and click **Run** at the ▸ Run checkpoint.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Create `gradelib.h`

> **How to create a file**: in the StudySite editor, create a new file with the exact name shown (spelling and lowercase matter, because `#include` lines refer to these names). Keep each new file open after you create it.

Create a new file named `gradelib.h` and enter this:

```
// gradelib.h - Grade Calculator v1.1 - Chapter 10
// Declarations for the grading library. Separating declarations from
// definitions lets main.cpp use these functions without seeing their code.
#ifndef GRADELIB_H
#define GRADELIB_H

#include <string>

const double A_CUTOFF = 90.0;
const double B_CUTOFF = 80.0;
const double C_CUTOFF = 70.0;
const double D_CUTOFF = 60.0;
const bool CAP_AT_100 = true;

/** Adds bonus points to a raw score. Bonus defaults to none. */
double applyBonus(double pointsEarned, double bonusPoints = 0.0);

/** Converts points to a capped, rounded percentage. Requires possible > 0. */
double computePercentage(double earned, double possible);

/** Maps a percentage to a letter grade on the fixed course scale. */
char assignLetterGrade(double percentage);

/** Overload 1: formats a percentage for display, e.g. "89.1%". */
std::string formatGrade(double percentage);

/** Overload 2: formats a letter grade for display, e.g. "B". */
std::string formatGrade(char letter);

/**
 * Reads a non-negative number from the user, re-prompting until valid.
 * @param  prompt  text shown to the user
 * @param  value   receives the validated number
 * @return true if a value was read, false if input closed unexpectedly
 */
bool readNonNegative(const std::string& prompt, double& value);

/** Reads a whole line of text, re-prompting if it is empty. */
std::string readNonEmptyLine(const std::string& prompt);

#endif
```

Why: A header is a **contract**. It says what the library provides — names, parameter types, return types — without saying how any of it works. `main.cpp` includes this file and can then call the functions; it never needs to see their bodies.

Three things in it are new.

The include guard. `#ifndef GRADELIB_H` / `#define GRADELIB_H` / `#endif` means "if this has not been included yet, include it; otherwise skip". Without it, a file that includes the header twice gets two copies of every declaration and the compiler reports redefinition errors. Every header you ever write needs a guard.

The default argument. `double applyBonus(double pointsEarned, double bonusPoints = 0.0);` means `applyBonus(45.0)` is legal and means `applyBonus(45.0, 0.0)`. The default goes in the **declaration only**, never in the definition — repeating it in `gradelib.cpp` is a compile error.

The overload. Two functions are named `formatGrade`: one takes a `double`, the other a `char`. That is allowed because they differ in parameter type, and the compiler picks the right one from the argument you pass. Overloading lets one concept — "format this for display" — keep one name instead of needing `formatPercentage` and `formatLetter`.

Step 2 — Create `gradelib.cpp`

Create a new file named `gradelib.cpp` and enter this:

```
// gradelib.cpp - Grade Calculator v1.1 - Chapter 10
// Definitions for the grading library declared in gradelib.h.
#include "gradelib.h"

#include <cmath>
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>

double applyBonus(double pointsEarned, double bonusPoints) {
    return pointsEarned + bonusPoints;
}

double computePercentage(double earned, double possible) {
    double raw = earned / possible * 100.0;
    double reported = CAP_AT_100 ? std::min(raw, 100.0) : raw;
    return std::round(reported * 10.0) / 10.0;
}

char assignLetterGrade(double percentage) {
    if (percentage >= A_CUTOFF) { return 'A'; }
    if (percentage >= B_CUTOFF) { return 'B'; }
    if (percentage >= C_CUTOFF) { return 'C'; }
    if (percentage >= D_CUTOFF) { return 'D'; }
    return 'F';
}

std::string formatGrade(double percentage) {
    std::ostringstream out;
    out << std::fixed << std::setprecision(1) << percentage << "%";
    return out.str();
}

std::string formatGrade(char letter) {
    return std::string(1, letter);
}

bool readNonNegative(const std::string& prompt, double& value) {
    while (true) {
        std::cout << prompt;
        if (!(std::cin >> value)) {
            if (std::cin.eof()) {
                return false;   // input closed; caller decides what to do
            }
            // Not a number. Clear the error state and discard the bad input,
```

```
            // otherwise every later read fails too.
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "  That is not a number. Please try again.\n";
            continue;
        }
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        if (value < 0.0) {
            std::cout << "  Points cannot be negative. Please try again.\n";
            continue;
        }
        return true;
    }
}

std::string readNonEmptyLine(const std::string& prompt) {
    std::string line;
    while (true) {
        std::cout << prompt;
        if (!std::getline(std::cin, line)) {
            return "";
        }
        if (!line.empty()) {
            return line;
        }
        std::cout << "  This cannot be blank. Please try again.\n";
    }
}
```

Why: This is the implementation side of the contract. The first three functions are moved from `main.cpp` unchanged, which is the point — separating declaration from definition does not alter behavior.

`#include "gradelib.h"` uses **quotes**, not angle brackets. Quotes mean "look next to this file first" and are for your own headers; angle brackets are for the standard library. An implementation file always includes its own header, so the compiler can check the definitions match the declarations.

The two reader functions are the chapter's new behavior, and `readNonNegative` repays careful reading. `if (!(std::cin >> value))` detects a **failed** read — the user typed `abc` where a number was expected. When that happens the stream enters an error state and *stays* there: every later read fails instantly, which is why a program without this handling appears to hang or loop forever printing prompts. The recovery is two steps and both are required: `std::cin.clear()` resets the error flags, and `std::cin.ignore(...)` discards the bad characters still sitting in the buffer. Clear without ignore and the same bad input fails again on the next attempt.

`std::cin.eof()` is checked separately and returns `false` to the caller. End-of-input is not a typo — it means there is no more input coming, ever, so re-prompting would loop forever. Distinguishing "bad input, ask again" from "no input left, give up" is what makes the loop safe.

`formatGrade(double)` uses a `std::ostringstream` — a stream that writes into a string instead of to the screen. It lets the function apply `std::fixed` and `std::setprecision(1)` and hand back `"89.1%"` as a value, so formatting is decided in one place rather than at every call site. `formatGrade(char)` uses `std::string(1, letter)`, which builds a string of one character.

Step 3 — Update `main.cpp`'s header and includes

In `main.cpp`, find the comment header and the five include lines — twelve lines in all:

```
// Grade Calculator v1.0 - Chapter 9
// Same behavior as v0.7, rebuilt from documented functions, plus a menu loop.
// New this version: user-defined functions, a menu that keeps the program running.
// Note: individual assignments are not stored yet (no arrays until Chapter 11),
// so the program keeps running totals only.
// Build: g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc

#include <cmath>
#include <iostream>
#include <iomanip>
#include <limits>
#include <string>
```

Replace all twelve with these eight:

```
// main.cpp - Grade Calculator v1.1 - Chapter 10
// Same behavior as v1.0, now split across three files with validated input.
// Build: g++ -std=c++17 -Wall -Wextra main.cpp gradelib.cpp -o gradecalc
#include "gradelib.h"

#include <iomanip>
#include <iostream>
#include <string>
```

Why: `main.cpp` now includes `"gradelib.h"` first, then only the standard headers it still needs itself. `<cmath>` and `<limits>` are gone because the code that used them moved to `gradelib.cpp` — an include that is no longer needed should go, or the file's dependencies become fiction.

The `Build:` line is updated to name both `.cpp` files. That is the command the autograder and your local toolchain will use, and it is the first time this project needs more than one source file on the command line.

Step 4 — Remove what moved to the library

In `main.cpp`, find the first constant:

```
const double A_CUTOFF = 90.0;
```

Delete that line and everything below it (the five constants and the three calculation functions with their comments), stopping just **above** this line, which you keep:

```
void readAssignment(double& totalEarned, double& totalPossible, int& count) {
```

Next, find the documentation comment above `printReport` and delete those three lines:

```
/**
 * Prints the full course report for the current totals.
 */
```

Finally, find `readMenuChoice` with its comment and the blank line above it, and delete all of it:

```
/**
 * Displays the menu and returns the user's choice as a character.
 */
char readMenuChoice() {
    std::cout << "1) Add assignment   2) View report   3) Quit\n";
    std::cout << "Choice: ";
    std::string line;
    std::getline(std::cin, line);
    return line.empty() ? '?' : line[0];
}
```

Why: Every one of these now lives in the library, and a function defined in two places is a **link error** — the compiler accepts each file alone and the linker then finds two definitions of `applyBonus` and refuses to produce a program. This step is not tidying; the program will not build until it is done.

The constants move to the header rather than the implementation file because `main.cpp` might need them too, and anything a user of the library can see belongs in the contract.

`readMenuChoice` is deleted outright rather than moved. It is about this program's menu, not about grading, so it has no business in a reusable grading library. Step 7 folds it into `main`. Deciding what does *not* belong in a library is as much a part of designing one as deciding what does.

The documentation comments on `readAssignment` and `printReport` go too: those two functions are private to `main.cpp` and their one-line declarations are no longer published anywhere, so the full `@param` blocks would be describing an interface nobody else can use.

Step 5 — Rewrite `readAssignment` to validate its input

In `main.cpp`, find the whole of `readAssignment`:

```
void readAssignment(double& totalEarned, double& totalPossible, int& count) {
    std::string assignmentName;
    std::cout << "  Assignment name : ";
    std::getline(std::cin, assignmentName);

    double pointsEarned = 0.0;
    std::cout << "  Points earned   : ";
    std::cin >> pointsEarned;

    double pointsPossible = 0.0;
    std::cout << "  Points possible : ";
    std::cin >> pointsPossible;

    double bonusPoints = 0.0;
    std::cout << "  Bonus points    : ";
    std::cin >> bonusPoints;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    totalEarned += applyBonus(pointsEarned, bonusPoints);
    totalPossible += pointsPossible;
    ++count;
    std::cout << "  Recorded: " << assignmentName << "\n\n";
}
```

Replace all of it with this:

```
void readAssignment(double& totalEarned, double& totalPossible, int& count) {
    std::string name = readNonEmptyLine("  Assignment name : ");

    double earned = 0.0;
    double possible = 0.0;
    double bonus = 0.0;
    if (!readNonNegative("  Points earned   : ", earned))   { return; }
    if (!readNonNegative("  Points possible : ", possible)) { return; }
    if (!readNonNegative("  Bonus points    : ", bonus))    { return; }

    totalEarned += applyBonus(earned, bonus);
    totalPossible += possible;
    ++count;
    std::cout << "  Recorded: " << name << "\n\n";
}
```

Why: Twenty-three lines become fifteen, and the new version is harder to break. Each raw `std::cin >>` is replaced by a call to `readNonNegative`, which will not return until it has a usable number — so a typo costs the user a re-prompt instead of corrupting the totals.

The `if (!readNonNegative(...)) { return; }` pattern handles the end-of-input case: if input has closed there is nothing more to read, so the function gives up without recording a partial assignment. Returning early rather than carrying on with half-filled values is what keeps the totals trustworthy.

Note `applyBonus(earned, bonus)` is still called explicitly with both arguments. The default argument added in Step 1 is available — `applyBonus(earned)` would compile — but passing the bonus the user actually entered is clearer here.

Step 6 — Use the formatting overloads in `printReport`

In `main.cpp`, find the two lines that print the percentage and grade:

```
    std::cout << "Percentage:  " << percentage << "%\n";
    std::cout << "Grade:       " << assignLetterGrade(percentage) << "\n\n";
```

Replace both with these:

```
    std::cout << "Percentage:  " << formatGrade(percentage) << "\n";
    std::cout << "Grade:       " << formatGrade(assignLetterGrade(percentage)) << "\n\n";
```

Why: Both calls are spelled `formatGrade`, and the compiler chooses between the two overloads by argument type: `percentage` is a `double` so it gets the `"89.1%"` version, while `assignLetterGrade(percentage)` returns a `char` so it gets the one-character version. **This is overload resolution, and it happens at compile time** — there is no run-time cost to having two functions share a name.

The `"%"` also disappears from this line, because `formatGrade(double)` now supplies it. Formatting rules living in one place is the whole reason to pass values through a formatter rather than assembling strings at each call site.

Step 7 — Inline the menu prompt into `main`

In `main.cpp`, find the banner line and the three lines that read the student name:

```
    std::cout << "=== GRADE CALCULATOR v1.0 ===\n\n";

    std::string studentName;
    std::cout << "Student name: ";
    std::getline(std::cin, studentName);
```

Replace those five lines with these two:

```
    std::cout << "=== GRADE CALCULATOR v1.1 ===\n\n";
    std::string studentName = readNonEmptyLine("Student name: ");
```

Then find the menu read and the whole `switch`:

```
        char choice = readMenuChoice();
        switch (choice) {
            case '1':
                readAssignment(totalEarned, totalPossible, assignmentCount);
                break;
            case '2':
                printReport(studentName, totalEarned, totalPossible, assignmentCount);
```

```
            break;
        case '3':
            running = false;
            break;
        default:
            std::cout << "Please enter 1, 2, or 3.\n\n";
            break;
    }
```

Replace all fifteen lines with these eleven:

```
    std::cout << "1) Add assignment   2) View report   3) Quit\nChoice: ";
    std::string line;
    if (!std::getline(std::cin, line)) { break; }
    char choice = line.empty() ? '?' : line[0];

    switch (choice) {
        case '1': readAssignment(totalEarned, totalPossible, assignmentCount); break;
        case '2': printReport(studentName, totalEarned, totalPossible, assignmentCount); break;
        case '3': running = false; break;
        default:  std::cout << "Please enter 1, 2, or 3.\n\n"; break;
    }
```

Why: The student name now comes from `readNonEmptyLine`, so a student who presses Enter without typing is asked again instead of getting a blank report. Three lines collapse into one, and the validation is free because the library provides it.

The menu read moves inline because `readMenuChoice` is gone. The important addition is `if (!std::getline(std::cin, line)) { break; }` — if input closes while the menu is waiting, the loop exits rather than spinning forever on a dead stream. The earlier version would have looped printing the menu with no way to stop.

The `switch` is reformatted to one line per case. The logic is identical; the parallel structure is simply easier to scan when each case is a single short statement.

> **The intermediate states in Steps 3-7 do not compile.** Between removing a function from `main.cpp` and the library providing it, names are temporarily missing or doubly defined. That is expected — finish all five steps, then Run.
>
> ▸ **Run checkpoint**: Click **Run** with all three files open. Then work through the test plan below in the Terminal.

Test plan

1. **It still works.** Student `Ada Lovelace`, option `1` with `Essay`, `45`, `50`, `0`, then option `2`. Percentage `90.0%`, grade `A` — identical to Chapter 9 for the same input.

2. **Bad input is rejected, not absorbed.** Choose `1`, and at `Points earned` type `abc`. You should see `That is not a number. Please try again.` and the same prompt once more. Then type `45` and continue normally. In Chapter 9 this input would have broken every later read.

3. **Negative points are rejected.** At `Points earned` type `-5`: `Points cannot be negative. Please try again.`

4. **Blank input is rejected.** At the student-name prompt press Enter with no text: `This cannot be blank. Please try again.` Same at an assignment name.

5. **The library is genuinely separate.** Open `main.cpp` and search for `A_CUTOFF`, `std::round`, and `std::min`. None should appear — they live in the library now.

6. The missing-file error, on purpose. Close the `gradelib.cpp` tab and click **Run**. You should get a *linker* error naming `applyBonus` or another library function as undefined — not a compiler error. Learn to recognize it: it means a `.cpp` file is missing from the build, not that your code is wrong. Reopen the tab and Run again.

Check your work

Then confirm:

- ☐ All three files compile together with no errors and **no warnings**.
- ☐ `gradelib.h` has an include guard.
- ☐ The default argument on `applyBonus` appears in `gradelib.h` only, not in `gradelib.cpp`.
- ☐ Both `formatGrade` functions exist and the program calls both.
- ☐ Typing a letter where a number is expected re-prompts and then works.
- ☐ Typing a negative number re-prompts.
- ☐ A blank student name or assignment name re-prompts.
- ☐ `main.cpp` contains no cutoffs and no arithmetic on percentages.
- ☐ `readMenuChoice` no longer exists anywhere.
- ☐ The same inputs produce the same grade as your Chapter 9 program.

Save this checkpoint

> **IMPORTANT — commit to save your work:** StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`, `gradelib.h`, and `gradelib.cpp` — all three must be open. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 10 Grade Calculator v1.1**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 10 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Three ways to pass an argument

```cpp
#include <iostream>

void byValue(double p) {
    p = 999.0;                    // the copy changes; the caller's does not
    (void)p;                      // tell the compiler the assignment is the point
```

```
}
void byPointer(double* p)   { *p = 999.0; }
void byReference(double& p) {  p = 999.0; }

int main() {
    double s = 84.0;
    byValue(s);     std::cout << "after byValue:     " << s << "\n";
    byPointer(&s);  std::cout << "after byPointer:   " << s << "\n";
    s = 84.0;
    byReference(s); std::cout << "after byReference: " << s << "\n";
    return 0;
}
```

Expected output:

```
after byValue:     84
after byPointer:   999
after byReference: 999
```

Try: Which call site tells you the function might modify the argument? What does that suggest about naming such functions?

2. Pointers and references side by side

```
#include <iostream>

int main() {
    double score = 84.0;
    double* ptr = &score;
    double& alias = score;

    std::cout << "score = " << score << "\n";
    std::cout << "*ptr  = " << *ptr << "\n";
    std::cout << "alias = " << alias << "\n";

    alias = 91.0;
    std::cout << "after alias = 91.0, score = " << score << "\n";
    return 0;
}
```

Expected output:

```
score = 84
*ptr  = 84
alias = 84
after alias = 91.0, score = 91
```

Try: Add `std::cout << ptr;` — you will see an address. Then set `*ptr = 100.0;` and print `score` and `alias`. All three names refer to one location.

3. Overloading

```
#include <iostream>

int overloaded(int a)       { return a * 2; }
double overloaded(double a) { return a * 2.5; }

int main() {
```

```
    std::cout << "overloaded(3)   = " << overloaded(3) << "\n";
    std::cout << "overloaded(3.0) = " << overloaded(3.0) << "\n";
    return 0;
}
```

Expected output:

```
overloaded(3)   = 6
overloaded(3.0) = 7.5
```

Try: Add `char overloaded(char a);` and call `overloaded('A')`. Then try adding a version differing only in return type and read the error.

4. Default arguments

```
#include <iostream>

double applyBonus(double pointsEarned, double bonusPoints = 0.0) {
    return pointsEarned + bonusPoints;
}

int main() {
    std::cout << applyBonus(84.0, 5.0) << "\n";
    std::cout << applyBonus(84.0) << "\n";
    return 0;
}
```

Expected output:

```
89
84
```

Try: Move the default to the *first* parameter instead and read the error. Why must defaults come last?

5. Recursion

```
#include <iostream>

int factorial(int n) {
    return (n <= 1) ? 1 : n * factorial(n - 1);
}

int main() {
    std::cout << "factorial(5) = " << factorial(5) << "\n";
    return 0;
}
```

Expected output:

```
factorial(5) = 120
```

Try: Write the same thing as a loop. Which is clearer? Now remove the base case, rebuild, and run — be ready to see it crash.

6. Split a program yourself

Take any single-file program from Chapter 9 and split it into `mathlib.h`, `mathlib.cpp`, and `main.cpp`. Include a header guard.

Then, deliberately:

- Build with only `main.cpp` and identify which stage produced the error.
- Remove the header guard and include the header twice.
- Repeat a default argument in both the header and the `.cpp`.

Three errors, three distinct messages. Being able to recognize them on sight is worth ten minutes now.

7. Reason about design

For each, say whether it should take its parameter by value, by reference, or by `const` reference — and why.

- A function computing the square root of a number
- A function printing a student's name
- A function that fills a name with input from the user
- A function checking whether a percentage is passing
- A function adding an assignment to a running total

Summary

- **Avoid global variables**. Any function can change them, so no function can be reasoned about alone. Pass what a function needs; return what it produces.
- A **`static` local** keeps its value between calls, hidden inside one function. Use it sparingly.
- **Overloaded** functions share a name and differ in parameters. The return type is not part of the signature.
- **Default arguments** must come last and belong in the declaration only. They extend an interface without breaking existing calls.
- A **recursive** function calls itself and needs a **base case**. Prefer a loop where a loop is natural.
- A **pointer** stores an address. `&` takes an address; `*` dereferences one. A pointer may be `nullptr`.
- A **reference** is another name for an existing variable. It must be initialized and can never be rebound.
- **Pass by reference** lets a function modify its caller's variable. Prefer references to pointers.
- **Pass large objects by `const` reference; small values by value.**
- Split a program into a **header** with declarations and an **implementation file** with definitions. Guard every header. List every `.cpp` on the build command.
- A function is **reusable** when it depends only on its parameters, does one thing, and documents its contract.

Key Terms

address-of operator — `&`, giving the address of a variable.

base case — the condition under which a recursive function returns without recursing.

dereference operator — `*`, giving the value stored at an address.

global variable — a variable declared outside every function, visible to all.

header guard — `#ifndef`/`#define`/`#endif` preventing a header from being processed twice.

null pointer — a pointer holding `nullptr`, pointing at nothing.

overload resolution — the compiler's choice among overloaded functions based on argument types.

overloading — defining several functions with one name and different parameter lists.

pass by reference — passing a reference so a function can modify the caller's variable.

pointer — a variable holding the address of another variable.

recursion — a function calling itself.

reference — an alternative name for an existing variable.

stack overflow — a crash caused by recursion that never reaches its base case.

static local variable — a local variable retaining its value between calls.

Next: Chapter 11 finally gives your calculator more than one student. Arrays hold a whole roster — and, more importantly, they let the hard-coded grade cutoffs from Chapter 6 become **data the user supplies**, collapsing that conditional chain into a short loop. Grade Calculator v1.2.

Chapter 11 — Arrays

Learning Objectives

When you finish this chapter you will be able to:

- Declare, initialize, and index an array. *(SLO 2.6)*
- Traverse an array with a loop and explain the standard idiom. *(SLO 1.4, 2.6)*
- Pass an array to a function together with its size. *(SLO 1.4, 2.6)*
- Use parallel arrays to associate related values. *(SLO 2.6)*
- Declare and traverse a two-dimensional array. *(SLO 2.6)*
- Explain what happens when an index goes out of bounds. *(SLO 1.6)*
- Replace a hard-coded conditional chain with a table-driven loop. *(SLO 1.7)*
- Build Grade Calculator v1.2 — a class roster, and a grade scale the user defines.

11.1 Why Collections Are Needed

Your calculator handles one student and keeps running totals. To handle a class, you would need a variable per student — and you do not know how many students there are when you write the program.

```
double student1 = 0.0;
double student2 = 0.0;
double student3 = 0.0;      // and then what?
```

An **array** holds many values of one type under a single name, reached by number.

There is a second problem this chapter solves, and it is the more interesting one. In Chapter 6 you built a letter-grade chain and the chapter ended by pointing out its limitation: every cutoff is compiled into the program, so no user can define their own scale. That limitation is not fixed by writing a longer chain. It is fixed by turning the cutoffs into **data**, and arrays are what make data-driven code possible.

11.2 Declaring and Initializing Arrays

```
double scores[5];                              // 5 doubles, uninitialized
double scores[5] = {9.0, 84.0, 18.0, 45.0, 30.0};   // 5 doubles, initialized
double scores[5] = {0.0};                      // all five set to 0.0
double scores[] = {9.0, 84.0, 18.0};           // size 3, deduced
```

The number in brackets is the **size**, and it must be a compile-time constant:

```
const int MAX_STUDENTS = 40;
double totals[MAX_STUDENTS];      // fine

int count = 0;
std::cin >> count;
double totals[count];             // NOT standard C++
```

That last restriction is a real limitation — you must decide a maximum in advance and hope it is enough. Chapter 12 removes it with `std::vector`.

Always initialize. `double scores[5];` contains garbage, and Section 3.2's warning applies with five times the force.

11.3 Indexing and Bounds

Reach an element with a **subscript**:

```
double scores[5] = {9.0, 84.0, 18.0, 45.0, 30.0};

std::cout << scores[0];      // 9.0  — the FIRST element
std::cout << scores[4];      // 30.0 — the LAST element
scores[2] = 20.0;            // assign to the third
```

Indices start at 0. An array of size 5 has valid indices 0, 1, 2, 3, 4. There is no `scores[5]`.

This is why Chapter 7 Section 7.4 taught you to count from zero with <:

```
for (int k = 0; k < 5; ++k)      // visits 0,1,2,3,4 — every element, exactly once
```

Out of bounds

C++ does not check subscripts. Reading or writing past the end compiles, runs, and does something undefined:

```
double scores[5];
scores[5] = 100.0;      // one past the end — no error, no warning
```

It may appear to work. It may corrupt another variable. It may crash. It may behave differently on different days.

This is the single most dangerous thing in this chapter. An out-of-bounds write is the first seeded defect in Chapter 16's debugging lab, precisely because it is invisible until it is catastrophic.

Two defenses. **Use `< size`, never `<= size`.** And **check any index that came from outside your program** before using it.

11.4 Traversing an Array

The standard idiom:

```
const int COUNT = 5;
double scores[COUNT] = {9.0, 84.0, 18.0, 45.0, 30.0};

double total = 0.0;
for (int k = 0; k < COUNT; ++k) {
```

```
    total += scores[k];
}
```

This is Chapter 7's accumulator pattern with an array supplying the values. The loop and the array fit together so naturally that the pairing is worth memorizing: **`for (int k = 0; k < size; ++k)` visits every element exactly once**.

Finding the largest follows the same shape:

```
double highest = scores[0];              // start with the first, not with 0.0
for (int k = 1; k < COUNT; ++k) {        // start at 1 — element 0 is already in
    if (scores[k] > highest) {
        highest = scores[k];
    }
}
```

Starting `highest` at `scores[0]` rather than `0.0` matters: with all-negative data, starting at zero gives an answer that is not in the array at all.

11.5 Arrays and Functions

An array does **not** carry its size. Pass both:

```
double sum(const double values[], int count) {
    double total = 0.0;
    for (int k = 0; k < count; ++k) {
        total += values[k];
    }
    return total;
}

double total = sum(scores, COUNT);
```

Two things differ from every other parameter you have met.

Arrays are passed by reference automatically — the function receives the array's address, not a copy. So a function *can* modify the caller's array, unlike the `double` in Chapter 9 Section 9.3. That is efficient and slightly dangerous.

`const` prevents modification. `const double values[]` promises the function will not change it, and the compiler enforces the promise. Use `const` on every array parameter you do not intend to modify.

Because the size does not travel with the array, `count` must be correct. Pass the wrong count and the function walks off the end.

11.6 Parallel Arrays

To associate several pieces of information about the same thing, use **parallel arrays** — separate arrays where the same index refers to the same item:

```
std::string studentNames[3] = {"Ada", "Grace", "Alan"};
double studentTotals[3]     = {89.0, 91.5, 72.0};
char studentLetters[3]      = {'B', 'A', 'C'};
```

```
// index 1 is Grace, 91.5, 'A'
```

This works and has a real weakness: **nothing enforces the correspondence**. Sort one array and not the others and every student gets someone else's grade. Add a student to two of the three and the arrays drift out of step permanently.

Chapter 14 replaces parallel arrays with a `struct`, which makes that class of bug structurally impossible. For Course I, parallel arrays are the tool available — and knowing exactly why they are risky is what makes Chapter 14 land.

11.7 Two-Dimensional Arrays

A roster is naturally a grid: students down, assignments across.

```
const int MAX_STUDENTS = 3;
const int MAX_ASSIGNMENTS = 4;

double pointsEarned[MAX_STUDENTS][MAX_ASSIGNMENTS] = {{0.0}};

pointsEarned[1][2] = 20.0;     // student 1, assignment 2
```

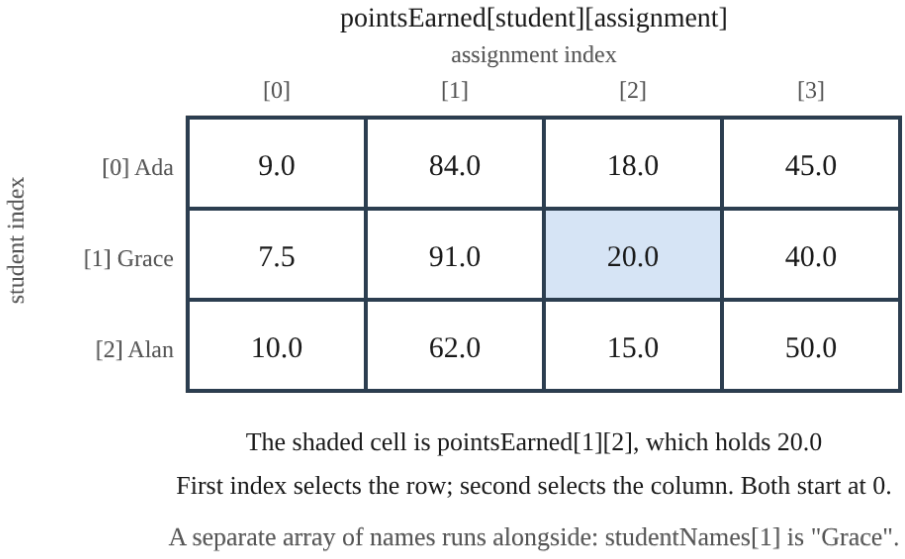


Figure 11: A grid of three rows and four columns holding points earned, with rows labelled as students Ada, Grace and Alan, columns labelled as assignment indices 0 to 3, and the cell at row 1 column 2 highlighted.

Figure 11.1 — A two-dimensional array of points earned.

Description of Figure 11.1. A grid with three rows and four columns. Rows are students, indexed 0 (Ada), 1 (Grace), and 2 (Alan). Columns are assignments, indexed 0 through 3.

	[0]	[1]	[2]	[3]
[0] Ada	9.0	84.0	18.0	45.0
[1] Grace	7.5	91.0	**20.0**	40.0

	[0]	[1]	[2]	[3]
[2] Alan	10.0	62.0	15.0	50.0

The highlighted cell is `pointsEarned[1][2]`, holding 20.0. The first index selects the row, the second the column, and both start at 0. A separate array of names runs alongside: `studentNames[1]` is `"Grace"`.

Traversing with nested loops

Chapter 7 Section 7.6 introduced nested loops. This is what they were for:

```
for (int s = 0; s < studentCount; ++s) {
    double total = 0.0;
    for (int a = 0; a < assignmentCount; ++a) {
        total += pointsEarned[s][a];
    }
    std::cout << studentNames[s] << ": " << total << "\n";
}
```

The outer loop walks students; the inner walks that student's assignments. Reverse them and you total per assignment instead — same data, different question.

Arrays are stored in row-major order, meaning row 0 sits entirely in memory before row 1 begins. That matters for performance in large programs and not at all here.

Higher dimensions

C++ allows more:

```
double scores[4][30][10];      // sections, students, assignments
```

Three dimensions is occasionally useful and rapidly hard to reason about. Two is enough for this book.

11.8 Replacing Code with Data

This section is the chapter's real lesson.

Here is the letter-grade logic from Chapter 6:

```
char letter = 'F';
if (percentage >= 90.0)      { letter = 'A'; }
else if (percentage >= 80.0) { letter = 'B'; }
else if (percentage >= 70.0) { letter = 'C'; }
else if (percentage >= 60.0) { letter = 'D'; }
```

The cutoffs and letters are **written into the control flow**. To use a different scale you must edit and recompile.

Now put the same information in parallel arrays:

```
double cutoffs[5] = {90.0, 80.0, 70.0, 60.0, 0.0};
char letters[5]   = {'A',  'B',  'C',  'D',  'F'};
int tierCount = 5;

char letterFor(double percentage, const double cutoffs[], const char letters[], int count) {
    for (int k = 0; k < count; ++k) {
        if (percentage >= cutoffs[k]) {
```

```
            return letters[k];
        }
    }
    return '?';
}
```

Compare them honestly:

	Chapter 6 chain	Table-driven loop
Lines of logic	grows with every tier	5, always
Adding a tier	edit code, recompile	add one array element
A plus-minus scale	rewrite the chain	supply different data
A scale the user types in	**impossible**	just fill the arrays
Testing	test every branch	test one loop

The loop is shorter, it never grows, and — decisively — **the data can come from anywhere**, including the user.

That last row is what makes this a turning point rather than a tidying-up. There is no arrangement of `if/else if` that accepts a grading scale at run time. The moment the cutoffs became an array, an entire category of feature became possible.

This pattern — replacing branching code with a table of data — recurs throughout the book. In Chapter 14 the two parallel arrays become one array of `struct`. In Chapter 18 that becomes a `GradeScale` class that validates itself. The idea does not change; only the container does.

11.9 C-Style Strings

Before `std::string`, C++ inherited C's approach: an array of `char` ending with a **null terminator**, the character `'\0'`.

```
char name[6] = {'A', 'd', 'a', '\0'};
char name[] = "Ada";                    // same thing; the '\0' is added for you
```

`"Ada"` occupies four bytes, not three. Every string literal you have written since Chapter 2 has been a C string.

They are awkward. The size is fixed, `=` does not copy them, `==` compares addresses rather than contents, and forgetting the terminator makes functions read off the end of the array.

Use `std::string`. C strings are covered because you will meet them in older code and in library interfaces, and because they explain what a string literal actually is.

11.10 Command-Line Arguments *(reference only)*

A program can receive information from the command line through a different form of `main`:

```
#include <iostream>

int main(int argc, char* argv[]) {
    for (int k = 0; k < argc; ++k) {
        std::cout << "argv[" << k << "] = " << argv[k] << "\n";
    }
```

```
    return 0;
}
```

`argc` is the number of arguments and `argv` is an array of C strings holding them. `argv[0]` is always the program's own name.

```
./gradecalc period3 --verbose

argv[0] = ./gradecalc
argv[1] = period3
argv[2] = --verbose
```

This belongs in the arrays chapter because `argv` is an array of C strings — both concepts from this chapter, combined.

It is reference material only. Every program in this book is menu-driven and reads its input interactively from `std::cin`. The Grade Calculator never reads command-line arguments, and no exercise requires them. You should recognize `int main(int argc, char* argv[])` when you see it elsewhere, and that is all this section is for.

11.11 Common Array Errors

What you see or observe	Cause	Fix
Values are garbage	Array not initialized	`double a[5] = {0.0};`
The program crashes, or a nearby variable changes	Index out of bounds	Use `< size`, never `<= size`
The loop misses the last element	Started at 1, or used `< size - 1`	Start at 0, test `< size`
The loop runs one time too many	`<= size`	Use `< size`
The wrong grade goes to the wrong student	Parallel arrays out of step	Update all of them together; Chapter 14 fixes this properly
`error: array must be initialized with a brace-enclosed initializer`	Assigned an array with =	Copy element by element
`error: array bound is not an integer constant`	Size from a variable	Use a `const int`, or `std::vector`
A function sums the wrong number of elements	Wrong count passed	Pass the same constant used to declare it
Comparing two C strings with == is always false	Comparing addresses	Use `std::string`

Design Notes

Always pass the size with the array. An array does not know how big it is, and a function that guesses is a defect waiting for the wrong input.

Mark array parameters `const` unless you intend to modify them. Arrays pass by reference, so without `const` any function can rewrite your data.

Prefer data to branching. If a conditional chain encodes a table, make it a table. The code shrinks and the capability grows.

Treat parallel arrays as a known risk. Every operation must touch all of them. Write a comment saying so, and look forward to Chapter 14.

Grade Calculator v1.2 — Class Roster and Custom Letter Scale

What v1.2 does

Two features arrive together, because both are the same idea: replacing code with data.

A **multi-student roster**, stored in a two-dimensional array of points with parallel arrays of names. And a **user-defined letter grade scale** — an instructor can now type in A/B/C/D/F, a plus-minus scale, or pass/fail, without the program changing.

The program

```
// Grade Calculator v1.2 - Chapter 11
// Multi-student roster on arrays, plus a USER-DEFINED letter grade scale.
// New this version: 2D arrays, parallel arrays, table-driven grade lookup.
//
// Central lesson: the hard-coded if/else chain of v1.1 becomes a loop over a
// table of data. Changing the grading scheme no longer means changing code.
//
// Run: click Run in StudySite and use the embedded Terminal.

#include <cmath>
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>

const int MAX_STUDENTS = 40;
const int MAX_ASSIGNMENTS = 20;
const int MAX_TIERS = 12;
const bool CAP_AT_100 = true;

// ---------- input helpers ----------

double readNonNegative(const std::string& prompt) {
    double value = 0.0;
    while (true) {
        std::cout << prompt;
        if (!(std::cin >> value)) {
            if (std::cin.eof()) { return 0.0; }
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "  That is not a number. Please try again.\n";
            continue;
        }
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        if (value < 0.0) {
            std::cout << "  Value cannot be negative. Please try again.\n";
            continue;
        }
        return value;
    }
}
```

```
std::string readLine(const std::string& prompt) {
    std::cout << prompt;
    std::string line;
    std::getline(std::cin, line);
    return line;
}

// ---------- grade scale (parallel arrays) ----------

/**
 * Reads a custom grade scale from the user into two parallel arrays.
 * Tiers must be entered highest cutoff first. Validated on entry.
 * @return the number of tiers actually read
 */
int readGradeScale(double cutoffs[], char letters[]) {
    std::cout << "\n--- Define your grade scale ---\n";
    std::cout << "Enter tiers from highest to lowest. The last tier should be 0.\n";
    std::cout << "Example: A 90, B 80, C 70, D 60, F 0\n\n";

    int count = 0;
    while (count < MAX_TIERS) {
        std::string prompt = "Tier " + std::to_string(count + 1) + " letter (or 'done'): ";
        std::string letterText = readLine(prompt);
        if (letterText == "done" || letterText.empty()) { break; }

        double cutoff = readNonNegative("  Minimum percentage for this tier: ");

        // Validation: cutoffs must strictly descend, or lookup breaks.
        if (count > 0 && cutoff >= cutoffs[count - 1]) {
            std::cout << "  Cutoff must be lower than the previous tier ("
                      << cutoffs[count - 1] << "). Tier not added.\n";
            continue;
        }
        letters[count] = letterText[0];
        cutoffs[count] = cutoff;
        ++count;
    }

    // A scale that does not reach 0 leaves low scores unclassified.
    if (count == 0 || cutoffs[count - 1] > 0.0) {
        std::cout << "\nNote: your scale did not reach 0, so an 'F' at 0 was added.\n";
        if (count < MAX_TIERS) {
            letters[count] = 'F';
            cutoffs[count] = 0.0;
            ++count;
        }
    }
    return count;
}

void useDefaultScale(double cutoffs[], char letters[], int& count) {
    const char defLetters[] = {'A', 'B', 'C', 'D', 'F'};
    const double defCutoffs[] = {90.0, 80.0, 70.0, 60.0, 0.0};
    count = 5;
    for (int i = 0; i < count; ++i) {
        letters[i] = defLetters[i];
        cutoffs[i] = defCutoffs[i];
    }
}
```

```
/**
 * Table-driven lookup. Compare with the if/else chain in v1.1: this version
 * works for any scale the user defines, and is shorter.
 */
char letterFor(double percentage, const double cutoffs[], const char letters[], int count) {
    for (int i = 0; i < count; ++i) {
        if (percentage >= cutoffs[i]) {
            return letters[i];
        }
    }
    return '?';
}

double computePercentage(double earned, double possible) {
    if (possible <= 0.0) { return 0.0; }
    double raw = earned / possible * 100.0;
    double reported = CAP_AT_100 ? std::min(raw, 100.0) : raw;
    return std::round(reported * 10.0) / 10.0;
}

// ---------- main ----------

int main() {
    std::cout << "=== GRADE CALCULATOR v1.2 ===\n";

    // Parallel arrays: names[i] belongs with scores[i][...]
    std::string studentNames[MAX_STUDENTS];
    std::string assignmentNames[MAX_ASSIGNMENTS];
    double pointsEarned[MAX_STUDENTS][MAX_ASSIGNMENTS] = {{0.0}};
    double pointsPossible[MAX_ASSIGNMENTS] = {0.0};
    int studentCount = 0;
    int assignmentCount = 0;

    double cutoffs[MAX_TIERS] = {0.0};
    char letters[MAX_TIERS] = {'\0'};
    int tierCount = 0;

    std::string useCustom = readLine("Define a custom grade scale? (y/n): ");
    if (!useCustom.empty() && (useCustom[0] == 'y' || useCustom[0] == 'Y')) {
        tierCount = readGradeScale(cutoffs, letters);
    } else {
        useDefaultScale(cutoffs, letters, tierCount);
        std::cout << "Using the default scale: A 90, B 80, C 70, D 60, F 0.\n";
    }

    std::cout << "\n--- Enter assignments ---\n";
    while (assignmentCount < MAX_ASSIGNMENTS) {
        std::string name = readLine("Assignment name (or 'done'): ");
        if (name == "done" || name.empty()) { break; }
        assignmentNames[assignmentCount] = name;
        pointsPossible[assignmentCount] = readNonNegative("  Points possible: ");
        ++assignmentCount;
    }

    std::cout << "\n--- Enter students ---\n";
    while (studentCount < MAX_STUDENTS) {
        std::string name = readLine("Student name (or 'done'): ");
        if (name == "done" || name.empty()) { break; }
        studentNames[studentCount] = name;
        for (int a = 0; a < assignmentCount; ++a) {
```

```
            std::string prompt = "  " + assignmentNames[a] + " points earned (incl. bonus): ";
            pointsEarned[studentCount][a] = readNonNegative(prompt);
        }
        ++studentCount;
    }

    // ---------- report ----------
    std::cout << "\n====================================\n";
    std::cout << "  CLASS REPORT\n";
    std::cout << "====================================\n";
    std::cout << std::fixed << std::setprecision(1);

    double classTotal = 0.0;
    for (int s = 0; s < studentCount; ++s) {
        double earned = 0.0;
        double possible = 0.0;
        for (int a = 0; a < assignmentCount; ++a) {
            earned += pointsEarned[s][a];
            possible += pointsPossible[a];
        }
        double pct = computePercentage(earned, possible);
        classTotal += pct;
        std::cout << std::left << std::setw(20) << studentNames[s]
                  << std::right << std::setw(8) << pct << "%   "
                  << letterFor(pct, cutoffs, letters, tierCount) << "\n";
    }

    if (studentCount > 0) {
        std::cout << "------------------------------------\n";
        std::cout << std::left << std::setw(20) << "CLASS AVERAGE"
                  << std::right << std::setw(8) << classTotal / studentCount << "%\n";
    } else {
        std::cout << "No students entered.\n";
    }

    std::cout << "\nPer-assignment averages:\n";
    for (int a = 0; a < assignmentCount; ++a) {
        double sum = 0.0;
        for (int s = 0; s < studentCount; ++s) { sum += pointsEarned[s][a]; }
        double avg = (studentCount > 0) ? sum / studentCount : 0.0;
        std::cout << "  " << std::left << std::setw(20) << assignmentNames[a]
                  << avg << " / " << pointsPossible[a] << "\n";
    }
    return 0;
}
```

Expected output

With a custom scale of A 93, B 85, C 77, F 0; assignments `Homework 1` (10 points) and `Midterm` (100 points); students Ada (10, 90) and Alan (8, 70):

```
====================================
  CLASS REPORT
====================================
Ada                   90.9%   B
Alan                  70.9%   F
------------------------------------
CLASS AVERAGE         80.9%

Per-assignment averages:
```

```
Homework 1          9.0 / 10.0
Midterm             80.0 / 100.0
```

Note Alan's F. Under the default scale, 70.9% is a C. Under the custom scale, where C requires 77, it is an F. **The program did not change — the data did.**

What to notice

`letterFor` is five lines and never grows. It handles a five-tier scale, a twelve-tier plus-minus scale, and a two-tier pass/fail scale identically. Put it beside the chain in v1.1 and the difference in *capability*, not just length, is the point.

Validation moved from the compiler to run time. When cutoffs were code, a wrong order was a bug you introduced. Now a user can type them out of order, so `readGradeScale` checks that each is lower than the last and rejects the ones that are not. **Data from outside the program must be validated inside it.**

The scale is repaired if it does not reach 0. Without a bottom tier, a low percentage matches nothing and `letterFor` returns `'?'`. Rather than let that happen, the program adds an F at 0 and says so.

Two nested loops, two different questions. Students-outer gives per-student totals; assignments-outer gives per-assignment averages. Same array, transposed traversal.

Every array parameter that is not modified is `const`.

The fixed maxima are a real limitation. `MAX_STUDENTS = 40` means a class of 41 does not fit. Chapter 12 removes the limit entirely.

Your StudySite Lab — Build a Class Roster

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v1.2
- **Starting point**: Your committed Chapter 10 v1.1 program (three files, validated input).

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

Two limits fall in this chapter. The program has only ever held **one** student, and its grade scale has only ever been the one written into the code. Arrays fix the first: a 2D array stores every student's score on every assignment. A **table** of cutoffs and letters fixes the second, and that is the chapter's real idea — the `if`/`else if` chain from Chapter 6 becomes a loop over data, so changing the grading scheme no longer means changing code.

By the end you will have:

1. `MAX_TIERS` in the library, and three new library functions: `useDefaultScale`, `readGradeScale`, and `letterFor`.
2. A grade scale held in **parallel arrays** — `cutoffs[i]` and `letters[i]` describe the same tier — that the user can define at run time, with validation that the cutoffs descend.
3. `assignLetterGrade` and the four `*_CUTOFF` constants **gone**, replaced by the table.
4. A roster in `main.cpp`: a 2D array of earned points, parallel arrays of student and assignment names, and a class report with per-student grades, a class average, and per-assignment averages.

> **Limitation, by design**: the arrays are fixed size — 40 students, 20 assignments, 12 tiers. Chapter 12 removes all three limits with `std::vector`.

Files in this checkpoint

File	What you do to it
`gradelib.h`	Edit (Steps 1-3)
`gradelib.cpp`	Edit (Steps 4-6)
`main.cpp`	Edit (Steps 7-8)

When you are finished your project still has **three** files.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `main.cpp`, `gradelib.h`, and `gradelib.cpp`. Confirm the editor shows the expected file paths before editing.
4. Continue the existing project in StudySite's internal editor. Keep all three files open, and click **Run** at the ▸ Run checkpoint.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Replace the fixed cutoffs with a tier limit

In `gradelib.h`, update the first comment line:

```
// gradelib.h - Grade Calculator v1.1 - Chapter 10
```

Replace it with:

```
// gradelib.h - Grade Calculator v1.2 - Chapter 11
```

Then find the four cutoff constants and delete all four:

```
const double A_CUTOFF = 90.0;
const double B_CUTOFF = 80.0;
const double C_CUTOFF = 70.0;
const double D_CUTOFF = 60.0;
```

Find the `CAP_AT_100` constant:

```
const bool CAP_AT_100 = true;
```

Directly below it, add:

```
// The most tiers a grade scale may hold. Chapter 12 removes this limit.
const int MAX_TIERS = 12;
```

Why: `A_CUTOFF` and friends were an improvement on magic numbers in Chapter 6, but they are still *code*. A grade scale is **data**: this course uses 90/80/70/60, another might use 93/85/77/70, and a program that needs recompiling to change that is the wrong shape. From here the cutoffs live in arrays the user can fill, and the constants have nothing left to do.

`MAX_TIERS` replaces them with a different kind of limit — how many tiers the fixed-size array can hold. That is a real constraint of arrays, and naming it is how the limit stays in one place.

Step 2 — Document the new `computePercentage` contract

In `gradelib.h`, find the one-line comment above `computePercentage`:

```
/** Converts points to a capped, rounded percentage. Requires possible > 0. */
```

Replace it with this fuller block:

```
/**
 * Converts points to a capped, rounded percentage.
 * @param  earned    total points earned including bonus
 * @param  possible  total points possible
 * @return percentage rounded to one decimal place, or 0.0 when possible <= 0
 */
```

Why: Chapter 10's version carried a precondition — the caller had to ensure `possible > 0`. That worked when one caller existed. A roster has many: every student row calls it, and any of them could total zero possible points if every assignment is worth 0. Pushing the check into the function is the right call once the number of callers grows, and the documented contract changes from "you must check" to "returns 0.0 in that case". Step 4 implements it.

Step 3 — Swap the fixed lookup for the table-driven one

In `gradelib.h`, find the `assignLetterGrade` declaration with its comment and the blank line below it, and delete all three lines:

```
/** Maps a percentage to a letter grade on the fixed course scale. */
char assignLetterGrade(double percentage);
```

Then find the last line of the file:

```
#endif
```

Directly above it, add:

```
/**
 * Fills a scale with the default tiers: A 90, B 80, C 70, D 60, F 0.
 * @param  cutoffs  receives the minimum percentage for each tier
 * @param  letters  receives the letter for each tier
 * @param  count    receives the number of tiers written
 */
void useDefaultScale(double cutoffs[], char letters[], int& count);

/**
 * Reads a custom grade scale from the user into two parallel arrays, highest
 * cutoff first. A tier whose cutoff is not below the previous one is refused,
 * and an 'F' at 0 is appended if the scale does not reach 0.
 * @return the number of tiers stored, never more than MAX_TIERS
 */
int readGradeScale(double cutoffs[], char letters[]);

/**
 * Table-driven lookup: the letter of the first tier the percentage reaches.
 * @pre    cutoffs[0..count-1] strictly descend
 * @return that tier's letter, or '?' if the scale does not reach this value
 */
char letterFor(double percentage, const double cutoffs[], const char letters[], int count);
```

Why: These three declarations are the chapter's new interface, and the shape of `letterFor` is worth reading closely:

```
char letterFor(double percentage, const double cutoffs[], const char letters[], int count);
```

An array parameter does **not** carry its own length in C++ — `cutoffs[]` decays to a pointer, and `sizeof` inside the function would tell you the size of a pointer, not the array. So the count must be passed alongside it. That is why every one of these functions takes or returns a `count`, and it is the single most important thing to understand about passing arrays to functions.

`const` on the two lookup arrays promises `letterFor` will not modify them. `useDefaultScale` and `readGradeScale` take them non-`const`, because filling them in *is* their job.

Step 4 — Guard the division in `computePercentage`

In `gradelib.cpp`, update the first comment line:

```
// gradelib.cpp - Grade Calculator v1.1 - Chapter 10
```

Replace it with:

```
// gradelib.cpp - Grade Calculator v1.2 - Chapter 11
```

Then find the start of `computePercentage`:

```
double computePercentage(double earned, double possible) {
```

Directly below it, add:

```
    // A roster can produce a zero denominator (every assignment worth 0, or
    // every assignment dropped), so the guard lives here rather than in every
    // caller.
    if (possible <= 0.0) { return 0.0; }
```

Why: This is the precondition from Step 2 turned into code. Returning `0.0` for a zero denominator means a student whose assignments are all worth nothing shows 0.0% rather than `inf` or `nan` — and because the check lives here, every caller in the roster loop gets it for free.

Step 5 — Remove `assignLetterGrade`

In `gradelib.cpp`, find the whole of `assignLetterGrade` and the blank line below it, and delete all eight lines:

```
char assignLetterGrade(double percentage) {
    if (percentage >= A_CUTOFF) { return 'A'; }
    if (percentage >= B_CUTOFF) { return 'B'; }
    if (percentage >= C_CUTOFF) { return 'C'; }
    if (percentage >= D_CUTOFF) { return 'D'; }
    return 'F';
}
```

Why: Its replacement arrives in the next step. Deleting it now, in the same chapter, is deliberate: a program with two grading functions — one fixed, one table-driven — invites a caller to use the wrong one, and the fixed one would silently ignore whatever scale the user defined.

Step 6 — Add the three scale functions

Go to the very end of `gradelib.cpp`. After the last line, leave one blank line and add all three functions:

```
void useDefaultScale(double cutoffs[], char letters[], int& count) {
    const char defLetters[] = {'A', 'B', 'C', 'D', 'F'};
    const double defCutoffs[] = {90.0, 80.0, 70.0, 60.0, 0.0};
    count = 5;
    for (int i = 0; i < count; ++i) {
        letters[i] = defLetters[i];
        cutoffs[i] = defCutoffs[i];
    }
}

int readGradeScale(double cutoffs[], char letters[]) {
    std::cout << "\n--- Define your grade scale ---\n";
    std::cout << "Enter tiers from highest to lowest. The last tier should be 0.\n";
    std::cout << "Example: A 90, B 80, C 70, D 60, F 0\n\n";

    int count = 0;
    while (count < MAX_TIERS) {
        std::string prompt = "Tier " + std::to_string(count + 1) + " letter (or 'done'): ";
        std::string letterText = readNonEmptyLine(prompt);
        if (letterText == "done" || letterText.empty()) { break; }

        double cutoff = 0.0;
        if (!readNonNegative("  Minimum percentage for this tier: ", cutoff)) { break; }

        // Validation: cutoffs must strictly descend, or the lookup breaks.
        if (count > 0 && cutoff >= cutoffs[count - 1]) {
            std::cout << "  Cutoff must be lower than the previous tier ("
                      << cutoffs[count - 1] << "). Tier not added.\n";
            continue;
        }
        letters[count] = letterText[0];
        cutoffs[count] = cutoff;
        ++count;
    }

    // A scale that does not reach 0 leaves low scores unclassified.
    if (count == 0 || cutoffs[count - 1] > 0.0) {
        std::cout << "\nNote: your scale did not reach 0, so an 'F' at 0 was added.\n";
        if (count < MAX_TIERS) {
            letters[count] = 'F';
```

```
            cutoffs[count] = 0.0;
            ++count;
        }
    }
    return count;
}

char letterFor(double percentage, const double cutoffs[], const char letters[], int count) {
    for (int i = 0; i < count; ++i) {
        if (percentage >= cutoffs[i]) {
            return letters[i];
        }
    }
    return '?';
}
```

Why: Three things to notice.

`useDefaultScale` copies from local arrays. It writes the course default into the caller's arrays element by element. The `const char defLetters[] = {...}` initializer form is how you write a small fixed table inline.

`readGradeScale` validates that cutoffs descend. `if (count > 0 && cutoff >= cutoffs[count - 1])` refuses a tier that is not strictly below the one before it. This is not fussiness: `letterFor` returns the *first* tier the percentage reaches, so an out-of-order table silently returns wrong letters rather than failing. The function also appends an `F` at 0 if the user's scale stops short, because a scale that never reaches 0 leaves low percentages with no letter at all — `letterFor` would return `'?'`.

`letterFor` is the payoff. Compare it with the chain it replaces:

```
for (int i = 0; i < count; ++i) {
    if (percentage >= cutoffs[i]) { return letters[i]; }
}
return '?';
```

Four lines, and it works for *any* scale with any number of tiers. The Chapter 6 chain was eight lines and worked for exactly one scale. **Replacing branching logic with a loop over data is one of the most valuable moves in programming**, and this is the clearest example of it in the book.

> **Do not click Run yet.** `main.cpp` still calls `assignLetterGrade`, which no longer exists, so the link will fail. Finish Steps 7 and 8 first.

Step 7 — Rewrite the top of `main.cpp`

In `main.cpp`, find the comment header and the include lines — eight lines in all:

```
// main.cpp - Grade Calculator v1.1 - Chapter 10
// Same behavior as v1.0, now split across three files with validated input.
// Build: g++ -std=c++17 -Wall -Wextra main.cpp gradelib.cpp -o gradecalc
#include "gradelib.h"

#include <iomanip>
#include <iostream>
#include <string>
```

Replace all eight with these sixteen:

```
// main.cpp - Grade Calculator v1.2 - Chapter 11
// Multi-student roster on arrays, plus a USER-DEFINED letter grade scale.
// New this version: 2D arrays, parallel arrays, table-driven grade lookup.
```

```
//
// Central lesson: the hard-coded if/else chain of v1.1 becomes a loop over a
// table of data. Changing the grading scheme no longer means changing code.
//
// Build: g++ -std=c++17 -Wall -Wextra main.cpp gradelib.cpp -o gradecalc
#include "gradelib.h"

#include <iomanip>
#include <iostream>
#include <string>

const int MAX_STUDENTS = 40;
const int MAX_ASSIGNMENTS = 20;
```

Why: `MAX_STUDENTS` and `MAX_ASSIGNMENTS` live in `main.cpp`, not the library, because they describe *this program's* capacity rather than anything about grading. `MAX_TIERS` went in the header instead, because the library's own `readGradeScale` needs it. Deciding which constant belongs on which side of the interface is the same judgment as deciding which functions belong there.

Step 8 — Replace the body with the roster

In `main.cpp`, find the start of `readAssignment`:

```
void readAssignment(double& totalEarned, double& totalPossible, int& count) {
```

Delete that line and everything below it (both helper functions and the whole of the old `main`), down to and including this line:

```
    return 0;
```

In the space where that code was, enter this:

```
int main() {
    std::cout << "=== GRADE CALCULATOR v1.2 ===\n";

    // Parallel arrays: studentNames[s] belongs with pointsEarned[s][...]
    std::string studentNames[MAX_STUDENTS];
    std::string assignmentNames[MAX_ASSIGNMENTS];
    double pointsEarned[MAX_STUDENTS][MAX_ASSIGNMENTS] = {{0.0}};
    double pointsPossible[MAX_ASSIGNMENTS] = {0.0};
    int studentCount = 0;
    int assignmentCount = 0;

    double cutoffs[MAX_TIERS] = {0.0};
    char letters[MAX_TIERS] = {'\0'};
    int tierCount = 0;

    std::string useCustom = readNonEmptyLine("Define a custom grade scale? (y/n): ");
    if (!useCustom.empty() && (useCustom[0] == 'y' || useCustom[0] == 'Y')) {
        tierCount = readGradeScale(cutoffs, letters);
    } else {
        useDefaultScale(cutoffs, letters, tierCount);
        std::cout << "Using the default scale: A 90, B 80, C 70, D 60, F 0.\n";
    }

    std::cout << "\n--- Enter assignments ---\n";
    while (assignmentCount < MAX_ASSIGNMENTS) {
        std::string name = readNonEmptyLine("Assignment name (or 'done'): ");
        if (name == "done" || name.empty()) { break; }
        assignmentNames[assignmentCount] = name;
```

```
        if (!readNonNegative("  Points possible: ", pointsPossible[assignmentCount])) { break; }
        ++assignmentCount;
    }

    std::cout << "\n--- Enter students ---\n";
    while (studentCount < MAX_STUDENTS) {
        std::string name = readNonEmptyLine("Student name (or 'done'): ");
        if (name == "done" || name.empty()) { break; }
        studentNames[studentCount] = name;
        for (int a = 0; a < assignmentCount; ++a) {
            std::string prompt = "  " + assignmentNames[a] + " points earned (incl. bonus): ";
            if (!readNonNegative(prompt, pointsEarned[studentCount][a])) { break; }
        }
        ++studentCount;
    }

    // ---------- report ----------
    std::cout << "\n====================================\n";
    std::cout << "  CLASS REPORT\n";
    std::cout << "====================================\n";
    std::cout << std::fixed << std::setprecision(1);

    double classTotal = 0.0;
    for (int s = 0; s < studentCount; ++s) {
        double earned = 0.0;
        double possible = 0.0;
        for (int a = 0; a < assignmentCount; ++a) {
            earned += pointsEarned[s][a];
            possible += pointsPossible[a];
        }
        double pct = computePercentage(earned, possible);
        classTotal += pct;
        std::cout << std::left << std::setw(20) << studentNames[s]
                  << std::right << std::setw(9) << formatGrade(pct) << "   "
                  << formatGrade(letterFor(pct, cutoffs, letters, tierCount)) << "\n";
    }

    if (studentCount > 0) {
        std::cout << "------------------------------------\n";
        std::cout << std::left << std::setw(20) << "CLASS AVERAGE"
                  << std::right << std::setw(9)
                  << formatGrade(classTotal / studentCount) << "\n";
    } else {
        std::cout << "No students entered.\n";
    }

    std::cout << "\nPer-assignment averages:\n";
    for (int a = 0; a < assignmentCount; ++a) {
        double sum = 0.0;
        for (int s = 0; s < studentCount; ++s) { sum += pointsEarned[s][a]; }
        double avg = (studentCount > 0) ? sum / studentCount : 0.0;
        std::cout << "  " << std::left << std::setw(20) << assignmentNames[a]
                  << avg << " / " << pointsPossible[a] << "\n";
    }
    return 0;
```

Why: `readAssignment` and `printReport` go because both assumed a single student. What replaces them is the chapter's data structure work.

The 2D array. `double pointsEarned[MAX_STUDENTS][MAX_ASSIGNMENTS]` is a grid: `pointsEarned[s][a]` is stu-

dent `s`'s score on assignment `a`. The `= {{0.0}}` initializer zeroes the whole thing — without it the array holds whatever was in that memory, and a student who skipped an assignment would get a garbage score.

Parallel arrays. `studentNames[s]` describes the same student as `pointsEarned[s][...]`, and `assignmentNames[a]` and `pointsPossible[a]` the same assignment. Nothing in the language enforces that correspondence — it is a convention you maintain, which is exactly why the comment naming it is there. Insert into one and forget the other and the data silently misaligns. Chapter 14 replaces this arrangement with `structs`, which make the grouping real.

Nested loops. The report's outer loop walks students; the inner loop walks that student's assignments to total them. The per-assignment averages at the end invert the nesting — outer over assignments, inner over students — to sum a *column* of the grid instead of a row. Which index is on the outside determines what you are summing, and getting that backwards is a classic error.

Counts, not capacities. Every loop runs to `studentCount` or `assignmentCount`, never to `MAX_STUDENTS` or `MAX_ASSIGNMENTS`. The arrays are 40 long but only the first `studentCount` entries hold real data; reading past the count gives you the zeroes from initialization and a wrong class average. **With fixed arrays you always track how many entries are actually in use** — the array cannot tell you.

> ▸ **Run checkpoint**: Click **Run** with all three files open, then work through the test plan below in the Terminal.

Test plan

1. Default scale, two assignments, two students. Answer `n` to the custom scale, then `Essay` / `50`, `Quiz` / `20`, `done`; then `Ada Lovelace` with `45` and `18`, `Grace Hopper` with `30` and `10`, `done`. Expected:

```
====================================
  CLASS REPORT
====================================
Ada Lovelace           90.0%   A
Grace Hopper           57.1%   F
------------------------------------
CLASS AVERAGE          73.5%

Per-assignment averages:
  Essay               37.5 / 50.0
  Quiz                14.0 / 20.0
```

Check the arithmetic by hand: Ada has 63 of 70 possible, which is 90.0%.

2. A custom scale. Answer `y`, then enter `A` 85, `B` 70, `C` 50, `done`. Note the message that an `F` at 0 was added — your scale stopped at 50, so anything below it had no letter. Then one assignment worth `100`, and students scoring `88` and `60`. With this scale 88 is an `A` and 60 is a `C`; with the default scale they would be a `B` and a `D`. **Same program, same data, different scheme, no recompile** — that is the chapter.

3. Cutoffs must descend. Answer `y`, enter `A` 80, then `B` 90. The tier is refused with a message naming the previous cutoff. Enter `B` 70 instead and carry on.

4. Bad input still recovers. At any points prompt type `abc`, then `-5`, then a real number. The library's validation from Chapter 10 is unchanged and still handles both.

5. No students. Answer `n`, enter one assignment, then `done` immediately at the student prompt. The report prints `No students entered.` and the program exits normally — no division by zero.

Check your work

Then confirm:

- ☐ All three files compile together with no errors and **no warnings**.
- ☐ `A_CUTOFF` through `D_CUTOFF` and `assignLetterGrade` no longer appear anywhere in the project.
- ☐ A custom scale changes the letters awarded without any code change.
- ☐ A tier whose cutoff is not below the previous one is refused.
- ☐ A scale that does not reach 0 gets an `F` at 0 appended, with a message.
- ☐ Per-student percentages match a hand calculation of their row.
- ☐ Per-assignment averages match a hand calculation of their column.
- ☐ Entering no students prints `No students entered.` and exits cleanly.
- ☐ Every loop runs to a count, never to `MAX_STUDENTS` or `MAX_ASSIGNMENTS`.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`, `gradelib.h`, and `gradelib.cpp` — all three must be open. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 11 Grade Calculator v1.2**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 11 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Declaring, indexing, traversing

```
#include <iostream>

int main() {
    const int COUNT = 5;
    double scores[COUNT] = {9.0, 84.0, 18.0, 45.0, 30.0};

    std::cout << "first: " << scores[0] << "\n";
    std::cout << "last:  " << scores[COUNT - 1] << "\n";

    double total = 0.0;
    for (int k = 0; k < COUNT; ++k) {
        total += scores[k];
```

```
    }
    std::cout << "total: " << total << "\n";
    return 0;
}
```

Expected output:

```
first: 9
last:  30
total: 186
```

Try: Change `k < COUNT` to `k <= COUNT` and run it several times. Does the total change between runs? What does that tell you?

2. Finding the largest

```
#include <iostream>

int main() {
    const int COUNT = 5;
    double scores[COUNT] = {9.0, 84.0, 18.0, 45.0, 30.0};

    double highest = scores[0];
    for (int k = 1; k < COUNT; ++k) {
        if (scores[k] > highest) { highest = scores[k]; }
    }
    std::cout << "highest: " << highest << "\n";
    return 0;
}
```

Expected output:

```
highest: 84
```

Try: Change all five values to negative numbers, then change `highest = scores[0]` to `highest = 0.0`. What answer do you get, and why is it wrong?

3. Arrays and functions

```
#include <iostream>

double sum(const double values[], int count) {
    double total = 0.0;
    for (int k = 0; k < count; ++k) { total += values[k]; }
    return total;
}

int main() {
    double scores[4] = {10.0, 20.0, 30.0, 40.0};
    std::cout << "sum: " << sum(scores, 4) << "\n";
    std::cout << "wrong count: " << sum(scores, 6) << "\n";
    return 0;
}
```

Expected output: the first line is `sum: 100`. The second reads past the end and prints an unpredictable value.

Try: Remove `const` from the parameter and add `values[0] = 0.0;` inside `sum`. Print `scores[0]` in `main` afterward. What does that tell you about how arrays are passed?

4. Two-dimensional traversal

```
#include <iomanip>
#include <iostream>

int main() {
    const int STUDENTS = 3;
    const int ASSIGNMENTS = 2;
    double points[STUDENTS][ASSIGNMENTS] = {{9.0, 84.0}, {7.5, 91.0}, {10.0, 62.0}};

    for (int s = 0; s < STUDENTS; ++s) {
        double total = 0.0;
        for (int a = 0; a < ASSIGNMENTS; ++a) { total += points[s][a]; }
        std::cout << "student " << s << " total: " << total << "\n";
    }
    for (int a = 0; a < ASSIGNMENTS; ++a) {
        double total = 0.0;
        for (int s = 0; s < STUDENTS; ++s) { total += points[s][a]; }
        std::cout << "assignment " << a << " total: " << total << "\n";
    }
    return 0;
}
```

Expected output:

```
student 0 total: 93
student 1 total: 98.5
student 2 total: 72
assignment 0 total: 26.5
assignment 1 total: 237
```

Try: Add a per-student average. Which loop does that belong in?

5. Table-driven grade lookup

```
#include <iostream>

char letterFor(double percentage, const double cutoffs[], const char letters[], int count) {
    for (int k = 0; k < count; ++k) {
        if (percentage >= cutoffs[k]) { return letters[k]; }
    }
    return '?';
}

int main() {
    double cutoffs[5] = {90.0, 80.0, 70.0, 60.0, 0.0};
    char letters[5]   = {'A',  'B',  'C',  'D',  'F'};

    double tests[5] = {95.0, 85.0, 75.0, 65.0, 30.0};
    for (int k = 0; k < 5; ++k) {
        std::cout << tests[k] << " -> " << letterFor(tests[k], cutoffs, letters, 5) << "\n";
    }
    return 0;
}
```

Expected output:

```
95 -> A
85 -> B
75 -> C
```

```
65 -> D
30 -> F
```

Try: Change the arrays to a plus-minus scale with eight tiers, updating the count. **You did not touch `letterFor`.** Now do the same to the Chapter 6 chain and compare the effort.

6. Parallel arrays drift

```cpp
#include <iostream>
#include <string>

int main() {
    std::string names[3] = {"Ada", "Grace", "Alan"};
    double totals[3]     = {89.0, 91.5, 72.0};

    // Someone "fixes" the order of one array and not the other.
    std::string temp = names[0];
    names[0] = names[2];
    names[2] = temp;

    for (int k = 0; k < 3; ++k) {
        std::cout << names[k] << ": " << totals[k] << "\n";
    }
    return 0;
}
```

Expected output:

```
Alan: 89
Grace: 91.5
Ada: 72
```

Try: Every grade is now attached to the wrong student, and the program reported it confidently. Write one sentence describing what would have to be true for this bug to be impossible. Chapter 14 makes it so.

7. Reason about arrays

- An array is declared `double a[8];`. What are the valid indices? What is `a[8]`?
- Why must a function receiving an array also receive its size?
- Why does adding `const` to an array parameter matter more than adding it to a `double` parameter?
- A grading scale has 12 tiers. How many lines does the table-driven lookup need? How many does an `if/else if` chain need?
- Give one thing parallel arrays cannot guarantee that a single array of records could.

Summary

- An **array** holds many values of one type under one name, reached by **index**. Indices start at **0**; an array of size n has valid indices 0 through $n-1$.
- The size must be a **compile-time constant**. Always initialize.
- **C++ does not check bounds.** Going past the end is undefined behavior — it may work, corrupt data, or crash. Use `< size`, never `<= size`.
- `for (int k = 0; k < size; ++k)` visits every element exactly once.

- **Arrays pass by reference automatically** and do not carry their size, so always pass the count. Mark array parameters `const` unless you intend to modify them.
- **Parallel arrays** associate values by shared index and are fragile: nothing enforces the correspondence. Chapter 14 replaces them with `struct`.
- A **two-dimensional array** is a grid; nested loops traverse it, and swapping the loops answers a different question.
- **Replacing a conditional chain with a table of data** shrinks the code and, decisively, allows the data to come from the user. This is the chapter's central lesson and it recurs in Chapters 14 and 18.
- **Data from outside the program must be validated inside it.** What the compiler used to guarantee is now your job.
- **C strings** are `char` arrays ending in `'\0'`. Use `std::string`. **Command-line arguments** are reference material only.

Key Terms

array — a collection of values of one type, stored under one name and reached by index.

bounds — the valid range of indices for an array, 0 through size−1.

C string — a `char` array terminated by the null character `'\0'`.

element — one value stored in an array.

index — the number identifying an element's position; also called a subscript.

null terminator — the `'\0'` character marking the end of a C string.

out of bounds — an index outside the valid range; undefined behavior.

parallel arrays — separate arrays in which the same index refers to related values.

row-major order — the layout in which each row of a two-dimensional array is stored contiguously.

subscript — the bracketed index used to reach an element.

table-driven — organized so that behavior is determined by data rather than by branching code.

two-dimensional array — an array of arrays, forming a grid.

Next: Chapter 12 removes the fixed maxima. `std::vector` grows to fit any number of students, assignments, or grade tiers, `std::string` replaces the last of the fixed-length text, and the calculator gains a drop-lowest feature. That version is the Course I capstone. Grade Calculator v1.3.

Chapter 12 — Vectors and Standard Strings

Learning Objectives

When you finish this chapter you will be able to:

- Explain the limitations of raw arrays that `std::vector` removes. *(SLO 2.6)*
- Declare a vector, add and remove elements, and query its size. *(SLO 1.3, 2.6)*
- Traverse a vector with an indexed loop and with a range-based `for`. *(SLO 1.3)*
- Pass vectors to functions by value and by `const` reference, and say which to use. *(SLO 1.4)*
- Use a vector of vectors as a grid. *(SLO 2.6)*
- Use `std::string` operations to inspect and build text. *(SLO 1.3)*
- Choose between an array and a vector for a given purpose. *(SLO 1.7)*
- Build Grade Calculator v1.3 — the Course I capstone.

12.1 The Limitations of Raw Arrays

Chapter 11's calculator works and is hemmed in by three restrictions, all from the same source: an array's size is fixed when the program is compiled.

A fixed maximum. `MAX_STUDENTS = 40` means a class of 41 does not fit, and a class of 5 wastes room for 35.

The size does not travel with the array. Every function taking an array needs a separate count parameter, and passing the wrong one walks off the end.

No bounds checking, ever. `scores[99]` on a five-element array compiles and does something undefined.

`std::vector` fixes all three. It grows and shrinks as needed, it knows its own size, and it offers a checked way to access elements.

12.2 Declaring and Using a Vector

Include `<vector>`. The type of the elements goes in angle brackets:

```
#include <vector>

std::vector<double> scores;                    // empty
std::vector<double> preset(3, 10.0);           // three elements, each 10.0
std::vector<double> listed = {9.0, 84.0, 18.0}; // three given values
std::vector<std::string> names;                // a vector of strings
```

`std::vector<double>` reads as "vector of double". The angle-bracket notation is a **template**, and Chapter 23 explains how it works. For now, treat `std::vector<T>` as the way to say "a growable collection of T".

Indexing looks exactly like an array:

```
scores[0] = 9.0;
std::cout << scores[1];
```

12.3 Growing and Shrinking

The essential difference:

```
std::vector<double> scores;          // size 0

scores.push_back(9.0);               // size 1
scores.push_back(84.0);              // size 2
scores.push_back(18.0);              // size 3

scores.pop_back();                   // size 2
```

```
empty:  size=0 empty=true
after 3 push_back: size=3
after pop_back: size=2
```

No maximum is declared anywhere. The vector obtains more memory as it needs it. A class of 41 students is no different from a class of 4.

That memory management is real work happening behind the scenes, and Chapter 22 shows what it involves. For now the useful fact is that it is handled correctly and you do not have to think about it.

12.4 Vector Member Functions

A vector is an **object**, and the functions it carries are called with a dot. Chapter 18 explains what that means properly; the syntax is worth using now.

Call	Does
`v.size()`	how many elements
`v.empty()`	true when size is 0
`v.push_back(x)`	add `x` at the end
`v.pop_back()`	remove the last element
`v.front()` / `v.back()`	first / last element
`v.clear()`	remove everything
`v[k]`	element `k`, **unchecked**
`v.at(k)`	element `k`, **checked**

```
front=9 back=18 [1]=84
```

`at()` versus `[]`

```
scores[99];       // undefined behavior — may crash, may not
scores.at(99);    // throws std::out_of_range — a definite, catchable error
```

```
at(99) threw: out_of_range
```

`at()` checks the index; `[]` does not. Checking costs a little time, which is why both exist.

Use `at()` when the index came from outside your program — from a user, a file, a calculation you are not certain of. Use `[]` inside a loop you control, such as `for (k = 0; k < v.size(); ++k)`, where the index is correct by construction.

Chapter 24 will show what `throws` means and how to catch it.

A signed/unsigned trap

`size()` returns an unsigned type, so comparing it with a signed `int` produces a warning under `-Wall`:

```
for (int k = 0; k < scores.size(); ++k)            // warning: signed/unsigned comparison
for (std::size_t k = 0; k < scores.size(); ++k)    // correct
```

`std::size_t` is the type vectors use for sizes. Appendix D Section D.6 requires it, and the alternative in the next section avoids the question entirely.

12.5 Range-Based `for` Loops

When you want every element and do not need the index:

```
for (double s : scores) {
    std::cout << s << " ";
}
```

```
range-based for: 9 84
```

Read `:` as "in". No index, no size, no comparison — so no off-by-one error and no signed/unsigned warning is possible.

To modify elements, take a reference:

```
for (double& s : scores) {
    s += 5.0;                // modifies the vector
}
```

For anything larger than a number, take a `const` reference to avoid copying — the same rule as Chapter 10 Section 10.7:

```
for (const std::string& name : studentNames) {
    std::cout << name << "\n";
}
```

Prefer the range-based form when you do not need the index. Use an indexed loop when you do — for example when walking two parallel vectors together.

12.6 Vectors and Functions

Unlike arrays, vectors are passed **by value** by default — which copies every element:

```
double sum(std::vector<double> values);              // copies the whole vector
double sum(const std::vector<double>& values);       // copies nothing
```

Always take a `const` reference unless you intend to modify:

```
double sum(const std::vector<double>& values) {
    double total = 0.0;
    for (double v : values) { total += v; }
    return total;
}
```

To modify, take a plain reference:

```
void addScore(std::vector<double>& values, double score) {
    values.push_back(score);
}
```

No count parameter is needed, because the vector carries its own size. That eliminates an entire class of bug from Chapter 11.

12.7 Vectors of Vectors

A grid is a vector whose elements are vectors:

```
std::vector<std::vector<double>> grid(2, std::vector<double>(3, 0.0));
grid[1][2] = 7.5;
```

```
grid[1][2]=7.5 rows=2 cols=3
```

`grid.size()` is the number of rows; `grid[0].size()` is the number of columns in row 0.

Unlike a two-dimensional array, **the rows need not be the same length**. That is exactly what a roster needs: students may have different numbers of assignments recorded, and a vector of vectors accommodates that without wasted space or special cases.

Growing it is straightforward:

```
std::vector<std::vector<double>> roster;
roster.push_back({9.0, 84.0});          // a student with two scores
roster.push_back({7.5, 91.0, 18.0});    // a student with three
```

12.8 `std::string` as a Sequence

You have used `std::string` since Chapter 3 to hold text. It is also a sequence of characters, with much of the same interface as a vector:

```
std::string name = "Ada Lovelace";

name.length()      // 12  (size() works too)
name[0]            // 'A'
name.back()        // 'e'
name.empty()       // false
```

```
length=12 [0]=A back=e
```

Chapter 11 Section 11.9 described C strings — fixed-length `char` arrays with a null terminator. `std::string` removes every one of their problems: it grows, `=` copies, `==` compares contents, and it cannot run off its end unnoticed.

12.9 String Operations

Call	Does
`s.length()` / `s.size()`	number of characters
`s.substr(start, count)`	a piece of the string
`s.find(x)`	position of `x`, or `std::string::npos`
`s + t`	joined
`s += t`	append
`s == t`	compare contents
`s.empty()`	true when there are no characters

```
substr(0,3)=Ada
find(' ')=3
find('z')=npos (not found)
concat: Ada Lovelace Jr.
```

`npos`

`find` returns `std::string::npos` when the text is not present. Test for it explicitly:

```
if (name.find('z') == std::string::npos) {
    std::cout << "not found\n";
}
```

`npos` is a very large number, **not** −1 in any useful sense. Testing `if (pos < 0)` fails to detect it and is a real bug.

Splitting on a separator

Combining `find` and `substr` splits text:

```
std::string full = "Ada Lovelace";
std::size_t space = full.find(' ');
if (space != std::string::npos) {
    std::string first = full.substr(0, space);
    std::string last  = full.substr(space + 1);
}
```

Chapter 15 uses this pattern to read comma-separated gradebook files.

12.10 Vectors versus Arrays

	Raw array	`std::vector`
Size fixed at compile time	yes	no

	Raw array	`std::vector`
Knows its own size	no	`size()`
Grows and shrinks	no	`push_back`, `pop_back`
Bounds checking available	no	`at()`
Needs a count parameter	yes	no
Passing copies by default	no, passes by reference	yes — use `const&`

Use `std::vector`. In this book, from here on, arrays appear only where a fixed size is genuinely part of the problem.

Chapter 11 was not wasted effort. Vectors are built on arrays, indexing works identically, the loop idiom is the same, and the out-of-bounds danger still exists whenever you use `[]`. Understanding arrays is what makes vectors comprehensible rather than magical.

Common Errors and Warnings

What you see	Cause	Fix
`warning: comparison of integer expressions of different signedness`	`int k` compared with `size()`	Use `std::size_t`, or a range-based `for`
Program crashes on `v[k]`	Index out of range	Use `at()`, or check the index
`terminate called after throwing 'std::out_of_range'`	`at()` caught a bad index — working as designed	Fix the index
A function does not change the vector	Passed by value	Take `std::vector<T>&`
The program is slow with large data	Copying vectors by value	Take `const std::vector<T>&`
`error: 'vector' was not declared`	Missing `#include <vector>`	Add the header
`if (s.find(x) < 0)` never fires	`npos` is huge, not negative	Compare to `std::string::npos`
`v.front()` crashes	The vector is empty	Check `empty()` first
Modifying `s` in `for (double s : v)` changes nothing	`s` is a copy	Use `for (double& s : v)`

Design Notes

Take `const std::vector<T>&` unless you intend to modify. Vectors copy by default and can be large.

Use `at()` for indices from outside your program; `[]` inside loops you control.

Prefer a range-based `for` when you do not need the index. It cannot be off by one.

Never store the size in a separate variable. `v.size()` is always correct; a copy of it goes stale the moment anything is added.

Grade Calculator v1.3 — Dynamic Roster

What v1.3 does

Everything v1.2 did, with **every fixed limit removed** — any number of students, assignments, and grade tiers — plus a **drop-lowest** feature.

This is the **Course I capstone**. It is a complete, genuinely useful points-based grading application, and nothing in Course II is required to make it work. Course II makes it general.

The program

```
// Grade Calculator v1.3 - Chapter 12 - COURSE I FINAL
// Unlimited roster, assignments, and grade tiers using std::vector.
// New this version: std::vector replaces fixed arrays, drop-lowest feature.
//
// Complete Course I feature set: named assignments, bonus points, custom
// letter scale, multi-student roster, class statistics, drop lowest.
// Grading model: points-based only. Weighted grading arrives in Chapter 20.
//
// Run: click Run in StudySite and use the embedded Terminal.

#include <cmath>
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>
#include <vector>

const bool CAP_AT_100 = true;

// ---------- input helpers ----------

double readNonNegative(const std::string& prompt) {
    double value = 0.0;
    while (true) {
        std::cout << prompt;
        if (!(std::cin >> value)) {
            if (std::cin.eof()) { return 0.0; }
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "  That is not a number. Please try again.\n";
            continue;
        }
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        if (value < 0.0) {
            std::cout << "  Value cannot be negative. Please try again.\n";
            continue;
        }
        return value;
    }
}

std::string readLine(const std::string& prompt) {
    std::cout << prompt;
    std::string line;
    std::getline(std::cin, line);
    return line;
}

bool readYesNo(const std::string& prompt) {
    std::string answer = readLine(prompt);
    return !answer.empty() && (answer[0] == 'y' || answer[0] == 'Y');
}

// ---------- grade scale ----------
```

```
std::vector<double> gradeCutoffs;
std::vector<char> gradeLetters;

void useDefaultScale() {
    gradeCutoffs = {90.0, 80.0, 70.0, 60.0, 0.0};
    gradeLetters = {'A', 'B', 'C', 'D', 'F'};
}

void readGradeScale() {
    gradeCutoffs.clear();
    gradeLetters.clear();
    std::cout << "\n--- Define your grade scale ---\n";
    std::cout << "Enter tiers highest first. Type 'done' to finish.\n\n";

    while (true) {
        std::string letterText = readLine("Tier letter (or 'done'): ");
        if (letterText == "done" || letterText.empty()) { break; }
        double cutoff = readNonNegative("  Minimum percentage: ");
        if (!gradeCutoffs.empty() && cutoff >= gradeCutoffs.back()) {
            std::cout << "  Cutoff must be lower than " << gradeCutoffs.back()
                      << ". Tier not added.\n";
            continue;
        }
        gradeLetters.push_back(letterText[0]);
        gradeCutoffs.push_back(cutoff);
    }

    if (gradeCutoffs.empty() || gradeCutoffs.back() > 0.0) {
        std::cout << "Note: scale did not reach 0, so an 'F' at 0 was added.\n";
        gradeLetters.push_back('F');
        gradeCutoffs.push_back(0.0);
    }
}

char letterFor(double percentage) {
    for (std::size_t i = 0; i < gradeCutoffs.size(); ++i) {
        if (percentage >= gradeCutoffs[i]) { return gradeLetters[i]; }
    }
    return '?';
}

double computePercentage(double earned, double possible) {
    if (possible <= 0.0) { return 0.0; }
    double raw = earned / possible * 100.0;
    double reported = CAP_AT_100 ? std::min(raw, 100.0) : raw;
    return std::round(reported * 10.0) / 10.0;
}

int main() {
    std::cout << "=== GRADE CALCULATOR v1.3 ===\n";
    std::cout << "    Course I final version\n\n";

    if (readYesNo("Define a custom grade scale? (y/n): ")) {
        readGradeScale();
    } else {
        useDefaultScale();
        std::cout << "Using default scale: A 90, B 80, C 70, D 60, F 0.\n";
    }

    // Assignments: name and points possible, any number of them.
```

```
    std::vector<std::string> assignmentNames;
    std::vector<double> pointsPossible;

    std::cout << "\n--- Enter assignments ---\n";
    while (true) {
        std::string name = readLine("Assignment name (or 'done'): ");
        if (name == "done" || name.empty()) { break; }
        assignmentNames.push_back(name);
        pointsPossible.push_back(readNonNegative("  Points possible: "));
    }

    // Students: name plus one earned-points entry per assignment.
    std::vector<std::string> studentNames;
    std::vector<std::vector<double>> earned;

    std::cout << "\n--- Enter students ---\n";
    while (true) {
        std::string name = readLine("Student name (or 'done'): ");
        if (name == "done" || name.empty()) { break; }
        studentNames.push_back(name);

        std::vector<double> row;
        for (std::size_t a = 0; a < assignmentNames.size(); ++a) {
            row.push_back(readNonNegative("  " + assignmentNames[a] + " points (incl. bonus): "));
        }
        earned.push_back(row);
    }

    bool dropLowest = false;
    if (assignmentNames.size() > 1) {
        dropLowest = readYesNo("\nDrop each student's lowest assignment? (y/n): ");
    }

    std::cout << "\n====================================\n";
    std::cout << "  CLASS REPORT\n";
    std::cout << "====================================\n";
    std::cout << std::fixed << std::setprecision(1);

    double classTotal = 0.0;
    for (std::size_t s = 0; s < studentNames.size(); ++s) {
        double totalEarned = 0.0;
        double totalPossible = 0.0;
        for (std::size_t a = 0; a < assignmentNames.size(); ++a) {
            totalEarned += earned[s][a];
            totalPossible += pointsPossible[a];
        }

        // Dropping in a points-based scheme removes BOTH the earned points and
        // the possible points. Removing only one would distort the result.
        if (dropLowest && !assignmentNames.empty()) {
            std::size_t worst = 0;
            double worstRatio = 2.0;
            for (std::size_t a = 0; a < assignmentNames.size(); ++a) {
                if (pointsPossible[a] <= 0.0) { continue; }
                double ratio = earned[s][a] / pointsPossible[a];
                if (ratio < worstRatio) { worstRatio = ratio; worst = a; }
            }
            if (worstRatio <= 1.0) {
                totalEarned -= earned[s][worst];
                totalPossible -= pointsPossible[worst];
```

```
            }
        }

        double pct = computePercentage(totalEarned, totalPossible);
        classTotal += pct;
        std::cout << std::left << std::setw(20) << studentNames[s]
                  << std::right << std::setw(8) << pct << "%   "
                  << letterFor(pct) << "\n";
    }

    if (!studentNames.empty()) {
        std::cout << "-----------------------------------\n";
        std::cout << std::left << std::setw(20) << "CLASS AVERAGE"
                  << std::right << std::setw(8)
                  << classTotal / studentNames.size() << "%\n";
    } else {
        std::cout << "No students entered.\n";
    }
    return 0;
}
```

Expected output

Default scale; assignments `Homework 1` (10), `Midterm` (100), `Quiz` (20); Ada scoring 10, 90, 5 and Alan scoring 8, 70, 20; drop-lowest **on**:

```
===================================
  CLASS REPORT
===================================
Ada                    90.9%   A
Alan                   93.3%   A
-----------------------------------
CLASS AVERAGE          92.1%
```

Alan's 93.3% deserves a look. His raw total is 98 out of 130, which is 75.4%. Dropping his worst assignment by ratio — the Midterm, at 70/100 — leaves 28 out of 30, or 93.3%. Dropping one assignment moved him from a C to an A, which is exactly why drop-lowest policies matter and why the arithmetic must be right.

What to notice

Not one fixed maximum anywhere. `MAX_STUDENTS`, `MAX_ASSIGNMENTS`, and `MAX_TIERS` are gone. The program handles four students or four hundred.

`std::vector<std::vector<double>> earned` replaces the 2D array, and each row is built with `push_back` as the student's scores are entered.

Drop-lowest removes both the earned and the possible points. This is the subtle part. In a points-based scheme, an assignment contributes to both running totals, so dropping it must remove it from both. Remove only the earned points and the student is penalized for an assignment they no longer have.

Worst is chosen by ratio, not by raw points. A 60/100 exam is a better performance than 2/10 on a quiz, even though 60 is a bigger number. Comparing `earned / possible` compares like with like.

Every loop uses `std::size_t`, matching what `size()` returns and avoiding the signed/unsigned warning.

The one thing v1.3 still cannot do

Look at how the course percentage is computed: two running totals, one division. Every point counts the same regardless of which assignment it came from.

That is a design decision, and it is correct for a points-based scheme. It also means **weighted grading is impossible here**. If exams were meant to count 50% of the course and homework 30%, no arrangement of two running totals could express it — you would need separate totals per category, combined by weight afterward.

That requirement has been in your specification since Chapter 1, marked out of scope. Course I is now finished and has never needed it. Chapter 13 analyzes it properly, and Chapters 20 and 21 build it.

Your StudySite Lab — Complete the Dynamic Gradebook

- **Course**: COSC 1436 — Programming Fundamentals
- **Project checkpoint**: v1.3
- **Starting point**: Your committed Chapter 11 v1.2 program (three files, fixed-size roster).

  One-repository rule: Continue in the same COSC 1436 Grade Calculator repository through Chapter 12. Do not create a chapter folder or a new repository. Each chapter replaces or extends the current working program.

What you will build

This is the Course I capstone, and it removes the last artificial limit in the program. Chapter 11's arrays hold 40 students, 20 assignments and 12 tiers because somebody had to pick a number. `std::vector` picks no number: it grows as you add to it, it knows its own size, and it frees its own memory. Every `MAX_` constant and every separate `count` variable disappears with it.

Then one real feature: **drop each student's lowest assignment**, which is both genuinely useful and a good lesson in getting a policy right — dropping an assignment in a points-based scheme has to remove the points possible as well as the points earned.

By the end you will have:

1. The grade scale as `std::vector<double>` and `std::vector<char>`, with no tier limit and no `count` parameter.
2. `readYesNo` in the library, replacing the hand-rolled y/n checks.
3. A roster of `std::vector<std::string>` and `std::vector<std::vector<double>>` — a vector of rows — with no student or assignment limit.
4. An optional drop-lowest pass, offered only when there is more than one assignment to drop.

   Course I complete. After this checkpoint the program has the full Course I feature set: named assignments, bonus points, a custom letter scale, a multi-student roster, class statistics, and drop-lowest. The grading model is points-based only; weighted grading arrives in Chapter 20.

Files in this checkpoint

File	What you do to it
`gradelib.h`	Edit (Steps 1-3)
`gradelib.cpp`	Edit (Steps 4-6)
`main.cpp`	Edit (Steps 7-8)

When you are finished your project still has **three** files.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1436xxx-Grade-Calculator-YourLastName**, and click `main.cpp`, `gradelib.h`, and `gradelib.cpp`. Confirm the editor shows the expected file paths before editing.
4. Continue the existing project in StudySite's internal editor. Keep all three files open, and click **Run** at the ▸ Run checkpoint.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Step 1 — Include `<vector>` and drop the tier limit

In `gradelib.h`, find the three header comment lines:

```
// gradelib.h - Grade Calculator v1.2 - Chapter 11
// Declarations for the grading library. Separating declarations from
// definitions lets main.cpp use these functions without seeing their code.
```

Replace it with:

```
// gradelib.h - Grade Calculator v1.3 - Chapter 12 - COURSE I FINAL
// Declarations for the grading library. The fixed-size arrays of v1.2 are
// gone: the scale is now a pair of std::vectors with no tier limit.
```

Then find the include and add the new one below it:

```
#include <string>
```

Directly below it, add:

```
#include <vector>
```

Now find the `MAX_TIERS` comment and constant, and the blank line below them, and delete all three lines:

```
// The most tiers a grade scale may hold. Chapter 12 removes this limit.
const int MAX_TIERS = 12;
```

Why: `MAX_TIERS` existed only because an array has to be declared with a size before you know how much you need. A vector does not, so the constant has nothing left to describe. **Deleting a limit is the clearest possible evidence that a data structure changed** — if you find yourself keeping a `MAX_` constant alongside a vector, something has not actually been converted.

`<vector>` must be included wherever `std::vector` appears in a declaration, and here that is the header itself, because the scale functions now take vectors as parameters.

Step 2 — Add `readYesNo` to the library

In `gradelib.h`, find the `readNonEmptyLine` declaration:

```
std::string readNonEmptyLine(const std::string& prompt);
```

Directly below it, add:

```
/** Reads a line and reports whether it begins with y or Y. */
bool readYesNo(const std::string& prompt);
```

Why: `main.cpp` has been writing `!answer.empty() && (answer[0] == 'y' || answer[0] == 'Y')` by hand, and this chapter adds a second place that needs it. Two copies of a condition is the point at which it should become a function — not because it is long, but because two copies can disagree. The library is the right home: asking a yes/no question is input handling, which is what the other readers in there do.

Step 3 — Convert the scale declarations to vectors

In `gradelib.h`, find the `useDefaultScale` comment block and declaration:

```
 * Fills a scale with the default tiers: A 90, B 80, C 70, D 60, F 0.
 * @param  cutoffs  receives the minimum percentage for each tier
 * @param  letters  receives the letter for each tier
 * @param  count    receives the number of tiers written
 */
void useDefaultScale(double cutoffs[], char letters[], int& count);
```

Replace all six lines with these four:

```
 * Replaces the scale with the default tiers: A 90, B 80, C 70, D 60, F 0.
 * Both vectors are resized to match, so no count has to be tracked.
 */
void useDefaultScale(std::vector<double>& cutoffs, std::vector<char>& letters);
```

Find the `readGradeScale` comment block and declaration:

```
 * Reads a custom grade scale from the user into two parallel arrays, highest
 * cutoff first. A tier whose cutoff is not below the previous one is refused,
 * and an 'F' at 0 is appended if the scale does not reach 0.
 * @return the number of tiers stored, never more than MAX_TIERS
 */
int readGradeScale(double cutoffs[], char letters[]);
```

Replace them with:

```
 * Reads a custom grade scale from the user, highest cutoff first, into two
 * parallel vectors. A tier whose cutoff is not below the previous one is
 * refused, and an 'F' at 0 is appended if the scale does not reach 0. There
 * is no limit on the number of tiers.
 */
void readGradeScale(std::vector<double>& cutoffs, std::vector<char>& letters);
```

Find the `letterFor` precondition line and declaration:

```
 * @pre    cutoffs[0..count-1] strictly descend
 * @return that tier's letter, or '?' if the scale does not reach this value
 */
char letterFor(double percentage, const double cutoffs[], const char letters[], int count);
```

Replace them with:

```
 * @pre    cutoffs strictly descend, and letters is the same size
 * @return that tier's letter, or '?' if the scale does not reach this value
 */
char letterFor(double percentage, const std::vector<double>& cutoffs,
               const std::vector<char>& letters);
```

Why: Compare the two signatures of `letterFor`:

```
char letterFor(double pct, const double cutoffs[], const char letters[], int count);
char letterFor(double pct, const std::vector<double>& cutoffs,
               const std::vector<char>& letters);
```

The `count` parameter is gone, and that is the whole argument for vectors. An array cannot tell a function how long it is, so the length had to travel separately — and nothing stopped a caller passing the wrong one. A vector carries its own size, so `cutoffs.size()` is always right and the two can never disagree.

`readGradeScale` also changes from returning `int` to returning `void`, for the same reason: it used to report how many tiers it stored, and now the vector knows.

Both vectors are still passed **by reference**. `const&` on the lookup parameters avoids copying the whole vector while promising not to modify it; plain `&` on the two filling functions is what lets them write into the caller's vectors. The Chapter 10 rule holds: `const&` for input you read, plain `&` for what you change.

Step 4 — Update the implementation header and comment

In `gradelib.cpp`, update the first comment line:

```
// gradelib.cpp - Grade Calculator v1.2 - Chapter 11
```

Replace it with:

```
// gradelib.cpp - Grade Calculator v1.3 - Chapter 12 - COURSE I FINAL
```

Then find these two comment lines inside `computePercentage`:

```
    // every assignment dropped), so the guard lives here rather than in every
    // caller.
```

Replace it with:

```
    // the only assignment dropped), so the guard lives here rather than in
    // every caller.
```

Why: The guard's reasoning gains a case. In Chapter 11 a zero denominator meant every assignment was worth nothing; now dropping the only assignment a student has produces the same situation. Same code, one more way to reach it, so the comment says so.

Step 5 — Add `readYesNo` and convert `useDefaultScale`

In `gradelib.cpp`, find the whole of `useDefaultScale`:

```
void useDefaultScale(double cutoffs[], char letters[], int& count) {
    const char defLetters[] = {'A', 'B', 'C', 'D', 'F'};
    const double defCutoffs[] = {90.0, 80.0, 70.0, 60.0, 0.0};
    count = 5;
    for (int i = 0; i < count; ++i) {
        letters[i] = defLetters[i];
        cutoffs[i] = defCutoffs[i];
    }
}
```

Replace all nine lines with these — `readYesNo` first, then the new `useDefaultScale`:

```
bool readYesNo(const std::string& prompt) {
    std::string answer = readNonEmptyLine(prompt);
    return !answer.empty() && (answer[0] == 'y' || answer[0] == 'Y');
}

void useDefaultScale(std::vector<double>& cutoffs, std::vector<char>& letters) {
    cutoffs = {90.0, 80.0, 70.0, 60.0, 0.0};
    letters = {'A', 'B', 'C', 'D', 'F'};
}
```

Why: `useDefaultScale` goes from eight lines to three. The array version had to write each element in a loop and set a count; the vector version assigns a whole **initializer list** in one statement — `cutoffs = {90.0, 80.0, 70.0, 60.0, 0.0};` — and the vector resizes itself to fit. There is no count to set, because there is no count.

`readYesNo` delegates to `readNonEmptyLine`, so an empty answer is re-prompted rather than silently treated as "no". That is a small behavioral improvement the refactor buys for free.

Step 6 — Convert `readGradeScale` and `letterFor`

In `gradelib.cpp`, find the whole of `readGradeScale` — thirty-six lines:

```
int readGradeScale(double cutoffs[], char letters[]) {
    std::cout << "\n--- Define your grade scale ---\n";
    std::cout << "Enter tiers from highest to lowest. The last tier should be 0.\n";
    std::cout << "Example: A 90, B 80, C 70, D 60, F 0\n\n";

    int count = 0;
    while (count < MAX_TIERS) {
        std::string prompt = "Tier " + std::to_string(count + 1) + " letter (or 'done'): ";
        std::string letterText = readNonEmptyLine(prompt);
        if (letterText == "done" || letterText.empty()) { break; }

        double cutoff = 0.0;
        if (!readNonNegative("  Minimum percentage for this tier: ", cutoff)) { break; }

        // Validation: cutoffs must strictly descend, or the lookup breaks.
        if (count > 0 && cutoff >= cutoffs[count - 1]) {
            std::cout << "  Cutoff must be lower than the previous tier ("
                      << cutoffs[count - 1] << "). Tier not added.\n";
            continue;
        }
        letters[count] = letterText[0];
        cutoffs[count] = cutoff;
        ++count;
    }

    // A scale that does not reach 0 leaves low scores unclassified.
    if (count == 0 || cutoffs[count - 1] > 0.0) {
```

```
        std::cout << "\nNote: your scale did not reach 0, so an 'F' at 0 was added.\n";
        if (count < MAX_TIERS) {
            letters[count] = 'F';
            cutoffs[count] = 0.0;
            ++count;
        }
    }
    return count;
}
```

Replace all of it with this:

```
void readGradeScale(std::vector<double>& cutoffs, std::vector<char>& letters) {
    cutoffs.clear();
    letters.clear();
    std::cout << "\n--- Define your grade scale ---\n";
    std::cout << "Enter tiers highest first. Type 'done' to finish.\n\n";

    while (true) {
        std::string letterText = readNonEmptyLine("Tier letter (or 'done'): ");
        if (letterText == "done" || letterText.empty()) { break; }

        double cutoff = 0.0;
        if (!readNonNegative("  Minimum percentage: ", cutoff)) { break; }

        // Validation: cutoffs must strictly descend, or the lookup breaks.
        if (!cutoffs.empty() && cutoff >= cutoffs.back()) {
            std::cout << "  Cutoff must be lower than " << cutoffs.back()
                      << ". Tier not added.\n";
            continue;
        }
        letters.push_back(letterText[0]);
        cutoffs.push_back(cutoff);
    }

    // A scale that does not reach 0 leaves low scores unclassified.
    if (cutoffs.empty() || cutoffs.back() > 0.0) {
        std::cout << "Note: scale did not reach 0, so an 'F' at 0 was added.\n";
        letters.push_back('F');
        cutoffs.push_back(0.0);
    }
}
```

Then find the whole of `letterFor`:

```
char letterFor(double percentage, const double cutoffs[], const char letters[], int count) {
    for (int i = 0; i < count; ++i) {
        if (percentage >= cutoffs[i]) {
            return letters[i];
        }
    }
    return '?';
}
```

Replace it with:

```
char letterFor(double percentage, const std::vector<double>& cutoffs,
               const std::vector<char>& letters) {
    for (std::size_t i = 0; i < cutoffs.size(); ++i) {
        if (percentage >= cutoffs[i]) { return letters[i]; }
    }
```

```
    return '?';
}
```

Why: `readGradeScale` loses six lines and gains unlimited tiers. Three specific changes are worth naming:

- `while (count < MAX_TIERS)` becomes `while (true)`. The loop no longer has a capacity to run out of; only the user's `done` ends it.
- `cutoffs[count] = cutoff; ++count;` becomes `cutoffs.push_back(cutoff);`. **`push_back` is the vector idiom**: it appends and grows the vector as needed, with no index to maintain and no risk of writing past the end.
- `cutoffs[count - 1]` becomes `cutoffs.back()`, and `count > 0` becomes `!cutoffs.empty()`. Both say what they mean instead of doing arithmetic on an index — and `cutoffs[count - 1]` with `count == 0` would read `cutoffs[-1]`, which is undefined behavior the old code avoided only because of the `count > 0` guard sitting in front of it.

`letterFor` loops to `cutoffs.size()` with a `std::size_t` index. `size()` returns an unsigned type, so comparing it against a signed `int` is exactly the kind of mismatch `-Wall -Wextra` warns about — using `std::size_t` for the index is what keeps the build clean.

> **Do not click Run yet.** `main.cpp` still declares the scale as arrays and calls the old signatures, so it will not compile against the new library. Finish Steps 7 and 8 first.

Step 7 — Convert the top of `main.cpp`

In `main.cpp`, find the comment header, the includes, and the two capacity constants — sixteen lines:

```
// main.cpp - Grade Calculator v1.2 - Chapter 11
// Multi-student roster on arrays, plus a USER-DEFINED letter grade scale.
// New this version: 2D arrays, parallel arrays, table-driven grade lookup.
//
// Central lesson: the hard-coded if/else chain of v1.1 becomes a loop over a
// table of data. Changing the grading scheme no longer means changing code.
//
// Build: g++ -std=c++17 -Wall -Wextra main.cpp gradelib.cpp -o gradecalc
#include "gradelib.h"

#include <iomanip>
#include <iostream>
#include <string>

const int MAX_STUDENTS = 40;
const int MAX_ASSIGNMENTS = 20;
```

Replace all sixteen with these fifteen:

```
// main.cpp - Grade Calculator v1.3 - Chapter 12 - COURSE I FINAL
// Unlimited roster, assignments, and grade tiers using std::vector.
// New this version: std::vector replaces fixed arrays, drop-lowest feature.
//
// Complete Course I feature set: named assignments, bonus points, custom
// letter scale, multi-student roster, class statistics, drop lowest.
// Grading model: points-based only. Weighted grading arrives in Chapter 20.
//
// Build: g++ -std=c++17 -Wall -Wextra main.cpp gradelib.cpp -o gradecalc
#include "gradelib.h"

#include <iomanip>
#include <iostream>
```

```
#include <string>
#include <vector>
```

Why: `MAX_STUDENTS` and `MAX_ASSIGNMENTS` follow `MAX_TIERS` out of the program. The roster is about to become vectors, so there is no capacity to declare — the only limit left is the machine's memory.

Step 8 — Convert the roster and add drop-lowest

In `main.cpp`, find the start of `main`:

```
int main() {
```

Delete that line and everything below it (the whole of the old `main`), down to and including this line:

```
    return 0;
```

In the space where that code was, enter this:

```
int main() {
    std::cout << "=== GRADE CALCULATOR v1.3 ===\n";
    std::cout << "    Course I final version\n\n";

    std::vector<double> cutoffs;
    std::vector<char> letters;

    if (readYesNo("Define a custom grade scale? (y/n): ")) {
        readGradeScale(cutoffs, letters);
    } else {
        useDefaultScale(cutoffs, letters);
        std::cout << "Using default scale: A 90, B 80, C 70, D 60, F 0.\n";
    }

    // Assignments: name and points possible, any number of them.
    std::vector<std::string> assignmentNames;
    std::vector<double> pointsPossible;

    std::cout << "\n--- Enter assignments ---\n";
    while (true) {
        std::string name = readNonEmptyLine("Assignment name (or 'done'): ");
        if (name == "done" || name.empty()) { break; }
        double possible = 0.0;
        if (!readNonNegative("  Points possible: ", possible)) { break; }
        assignmentNames.push_back(name);
        pointsPossible.push_back(possible);
    }

    // Students: name plus one earned-points entry per assignment.
    std::vector<std::string> studentNames;
    std::vector<std::vector<double>> earned;

    std::cout << "\n--- Enter students ---\n";
    while (true) {
        std::string name = readNonEmptyLine("Student name (or 'done'): ");
        if (name == "done" || name.empty()) { break; }
        studentNames.push_back(name);

        std::vector<double> row;
        for (std::size_t a = 0; a < assignmentNames.size(); ++a) {
            double value = 0.0;
            readNonNegative("  " + assignmentNames[a] + " points (incl. bonus): ", value);
            row.push_back(value);
```

```
        }
        earned.push_back(row);
    }

    bool dropLowest = false;
    if (assignmentNames.size() > 1) {
        dropLowest = readYesNo("\nDrop each student's lowest assignment? (y/n): ");
    }

    std::cout << "\n====================================\n";
    std::cout << "  CLASS REPORT\n";
    std::cout << "====================================\n";
    std::cout << std::fixed << std::setprecision(1);

    double classTotal = 0.0;
    for (std::size_t s = 0; s < studentNames.size(); ++s) {
        double totalEarned = 0.0;
        double totalPossible = 0.0;
        for (std::size_t a = 0; a < assignmentNames.size(); ++a) {
            totalEarned += earned[s][a];
            totalPossible += pointsPossible[a];
        }

        // Dropping in a points-based scheme removes BOTH the earned points and
        // the possible points. Removing only one would distort the result.
        if (dropLowest && !assignmentNames.empty()) {
            std::size_t worst = 0;
            double worstRatio = 2.0;
            for (std::size_t a = 0; a < assignmentNames.size(); ++a) {
                if (pointsPossible[a] <= 0.0) { continue; }
                double ratio = earned[s][a] / pointsPossible[a];
                if (ratio < worstRatio) { worstRatio = ratio; worst = a; }
            }
            if (worstRatio <= 1.0) {
                totalEarned -= earned[s][worst];
                totalPossible -= pointsPossible[worst];
            }
        }

        double pct = computePercentage(totalEarned, totalPossible);
        classTotal += pct;
        std::cout << std::left << std::setw(20) << studentNames[s]
                  << std::right << std::setw(9) << formatGrade(pct) << "   "
                  << formatGrade(letterFor(pct, cutoffs, letters)) << "\n";
    }

    if (!studentNames.empty()) {
        std::cout << "------------------------------------\n";
        std::cout << std::left << std::setw(20) << "CLASS AVERAGE"
                  << std::right << std::setw(9)
                  << formatGrade(classTotal / studentNames.size()) << "\n";
    } else {
        std::cout << "No students entered.\n";
    }
    return 0;
```

Why: Read this in three parts.

The roster becomes vectors. `std::vector<std::vector<double>> earned` is a **vector of vectors** — `earned[s]` is student `s`'s whole row, and `earned[s][a]` is one score, exactly as the 2D array indexed. The difference is that

each row is built with `push_back` as the scores are read and then appended with `earned.push_back(row)`, so the structure is exactly as big as the data. Note also that there is no longer any `= {{0.0}}` initializer: a vector starts empty and every element is created by being pushed, so there is no uninitialized memory to worry about.

Every loop bound comes from the data. `studentNames.size()`, `assignmentNames.size()`. No parallel `count` variables, and no way for a count to drift out of step with its array — the bug that fixed-size arrays invite and vectors make impossible.

Drop-lowest, and why it removes two numbers. The loop finds the assignment with the worst **ratio** of earned to possible, not the lowest raw score: 8/10 is a worse performance than 40/100, even though 8 is the smaller number. Assignments worth 0 points are skipped, since their ratio is undefined. Then:

```
totalEarned -= earned[s][worst];
totalPossible -= pointsPossible[worst];
```

Both subtractions are required. Removing only the earned points would be equivalent to scoring zero on the assignment — the opposite of dropping it — and removing only the possible points would hand the student free marks. In a points-based scheme, dropping means the assignment never existed, and that means removing it from both sides of the division.

The offer is guarded by `assignmentNames.size() > 1`, because dropping the only assignment leaves nothing to grade.

> ▸ **Run checkpoint**: Click **Run** with all three files open, then work through the test plan below in the Terminal.

Test plan

1. Same answers as Chapter 11. Answer `n`, then `Essay` / `50`, `Quiz` / `20`, `done`; `Ada Lovelace` with `45` and `18`, `Grace Hopper` with `30` and `10`, `done`; then `n` to dropping. Ada should be `90.0%` `A` and Grace `57.1%` `F`, with a class average of `73.5%` — identical to Chapter 11. The data structures changed; the answers must not.

2. Drop-lowest, verified by hand. One student, two assignments: `Essay` / `50` and `Quiz` / `20`; `Ada` scoring `40` and `19`; then `y`.

Essay is 40/50 = 0.80, Quiz is 19/20 = 0.95, so Essay is dropped. What remains is 19 of 20, which is `95.0%` and an `A`. Without the drop it would be 59 of 70 = `84.3%` and a `B`. Confirm you get `95.0%`.

3. Ratio, not raw score. One student, two assignments: `Final` / `100` and `Pop quiz` / `10`; scores `85` and `6`. Raw scores say the pop quiz (6) is lowest; ratios say 0.60 against 0.85, so the pop quiz is correctly the one dropped, leaving 85/100 = `85.0%`. Had the program compared raw scores it would have reached the same answer here — construct a case where it would not, such as `Final` / `100` scoring `60` and `Quiz` / `10` scoring `9`: ratios 0.60 and 0.90 drop the Final, leaving `90.0%`.

4. More than twelve tiers. Answer `y` and enter thirteen or more tiers with strictly descending cutoffs — `A+` 97, `A` 93, `A-` 90, `B+` 87, and so on down. Chapter 11 would have stopped accepting them at twelve. All of them should now be accepted.

5. One assignment: no drop offer. Enter a single assignment. The drop-lowest question must not appear at all.

6. No students. `done` immediately at the student prompt prints `No students entered.` and exits cleanly.

Check your work

Then confirm:

- ☐ All three files compile together with no errors and **no warnings**.

- ☐ `MAX_TIERS`, `MAX_STUDENTS` and `MAX_ASSIGNMENTS` appear nowhere in the project.
- ☐ No `count` variable tracks the length of anything; every bound comes from `.size()`.
- ☐ A scale with more than twelve tiers is accepted.
- ☐ The same inputs give the same percentages and letters as your Chapter 11 program.
- ☐ Drop-lowest removes the assignment with the worst **ratio**, and removes its points possible as well as its points earned.
- ☐ With one assignment, the drop-lowest question is not asked.
- ☐ Entering no students prints `No students entered.` and exits cleanly.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is `main.cpp`, `gradelib.h`, and `gradelib.cpp` — all three must be open. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1436xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 12 Grade Calculator v1.3**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1436xxx-Grade-Calculator-YourLastName** contains the Chapter 12 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Growing a vector

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<double> scores;
    std::cout << std::boolalpha;
    std::cout << "size=" << scores.size() << " empty=" << scores.empty() << "\n";

    scores.push_back(9.0);
    scores.push_back(84.0);
    scores.push_back(18.0);
    std::cout << "after 3 push_back: size=" << scores.size() << "\n";
    std::cout << "front=" << scores.front() << " back=" << scores.back() << "\n";

    scores.pop_back();
    std::cout << "after pop_back: size=" << scores.size() << "\n";
```

```
    return 0;
}
```

Expected output:

```
size=0 empty=true
after 3 push_back: size=3
front=9 back=18
after pop_back: size=2
```

Try: Call `front()` on an empty vector. What happens? Add an `empty()` check.

2. Two ways to traverse

```
#include <iostream>
#include <vector>

int main() {
    std::vector<double> scores = {9.0, 84.0, 18.0};

    for (std::size_t k = 0; k < scores.size(); ++k) {
        std::cout << k << ": " << scores[k] << "\n";
    }
    for (double s : scores) {
        std::cout << s << " ";
    }
    std::cout << "\n";
    return 0;
}
```

Expected output:

```
0: 9
1: 84
2: 18
9 84 18
```

Try: Change `std::size_t k` to `int k` and rebuild. Read the warning. Then double every score using a range-based loop — you will need `double&`.

3. `at()` versus `[]`

```
#include <iostream>
#include <stdexcept>
#include <vector>

int main() {
    std::vector<double> scores = {9.0, 84.0};
    try {
        std::cout << scores.at(99) << "\n";
    } catch (const std::out_of_range&) {
        std::cout << "at(99) threw out_of_range\n";
    }
    return 0;
}
```

Expected output:

```
at(99) threw out_of_range
```

Try: Replace `at(99)` with `scores[99]`. Run it several times. Does it crash? Print a value? Both are undefined behavior — which of the two versions would you rather ship?

4. A vector of vectors

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<std::vector<double>> roster;
    roster.push_back({9.0, 84.0});
    roster.push_back({7.5, 91.0, 18.0});

    for (std::size_t s = 0; s < roster.size(); ++s) {
        std::cout << "student " << s << " has " << roster[s].size() << " scores: ";
        for (double v : roster[s]) { std::cout << v << " "; }
        std::cout << "\n";
    }
    return 0;
}
```

Expected output:

```
student 0 has 2 scores: 9 84
student 1 has 3 scores: 7.5 91 18
```

Try: Explain why a two-dimensional array could not hold this data without waste.

5. Strings as sequences

```cpp
#include <iostream>
#include <string>

int main() {
    std::string name = "Ada Lovelace";
    std::cout << "length      " << name.length() << "\n";
    std::cout << "first char  " << name[0] << "\n";
    std::cout << "substr(0,3) " << name.substr(0, 3) << "\n";
    std::cout << "find(' ')   " << name.find(' ') << "\n";
    std::cout << "concat      " << name + " Jr." << "\n";
    return 0;
}
```

Expected output:

```
length      12
first char  A
substr(0,3) Ada
find(' ')   3
concat      Ada Lovelace Jr.
```

Try: Extract the last name using `find` and `substr` together, without hard-coding the position 4.

6. Splitting text

Write a program that reads a full name and prints the first and last names on separate lines.

Then answer: what should it do if the input has no space at all? What if it has two spaces? Your program should not crash on either. Test both before you consider it finished.

7. Convert array code to vectors

Rewrite this using `std::vector`, removing the count parameter and the fixed maximum:

```
#include <iostream>

const int MAX = 100;

double average(const double values[], int count) {
    if (count == 0) { return 0.0; }
    double total = 0.0;
    for (int k = 0; k < count; ++k) { total += values[k]; }
    return total / count;
}

int main() {
    double scores[MAX] = {9.0, 84.0, 18.0};
    std::cout << average(scores, 3) << "\n";
    return 0;
}
```

How many parameters did the function lose? What class of bug did that eliminate?

Summary

- `std::vector` removes the three limits of raw arrays: it **grows**, it **knows its own size**, and it offers **bounds-checked access**.
- Declare with `std::vector<T>`. Add with `push_back`, remove with `pop_back`, query with `size()` and `empty()`.
- `v[k]` is unchecked; `v.at(k)` throws `std::out_of_range`. Use `at()` for indices from outside your program.
- `size()` returns an unsigned type — use `std::size_t` for loop indices, or a **range-based `for`**, which cannot be off by one.
- **Vectors copy by value.** Take `const std::vector<T>&` unless you intend to modify. **No count parameter is ever needed.**
- A **vector of vectors** is a grid whose rows may differ in length.
- `std::string` is a sequence of characters with `length`, `substr`, `find`, `+`, and `==`. `find` returns `std::string::npos` when nothing matches — never test for a negative value.
- **Use vectors.** Arrays remain worth understanding because vectors are built on them and `[]` is still unchecked.
- **Grade Calculator v1.3 completes Course I**: named assignments, bonus points, a user-defined letter scale, an unlimited roster, class statistics, and drop-lowest — all on a single points-based grading model.

Key Terms

at() — a bounds-checked element access that throws on an invalid index.

member function — a function belonging to an object, called with a dot.

npos — the value `find` returns when nothing matches.

pop_back — removes the last element of a vector.

push_back — adds an element to the end of a vector.

range-based for — a loop visiting every element of a collection without an index.

size_t — the unsigned type used for container sizes and indices.

std::string — the standard type for text of any length.

std::vector — the standard growable sequence container.

substr — a function returning part of a string.

template — a type parameterized by another type, as in `std::vector<double>`.

Next: Course II begins. Chapter 13 steps back from code to ask how software is actually built and maintained — planning, analysis, design, development, and maintenance — and has you write a maintenance plan for the application you just inherited from yourself. Its headline item is the requirement you have deferred since Chapter 1. Grade Calculator v2.0.

Chapter 13 — The Software Development Lifecycle

Learning Objectives

When you finish this chapter you will be able to:

- Identify and explain the five phases of a software development lifecycle. *(SLO 2.1)*
- Describe the inputs and outputs of each phase. *(SLO 2.1)*
- Compare waterfall, iterative, and agile lifecycle models. *(SLO 2.1)*
- Distinguish procedural, structured, and object-oriented programming methodologies. *(SLO 2.8)*
- Explain why coding standards matter and apply one to an existing codebase. *(SLO 2.4)*
- Explain why a behavior-preserving refactor must be verified, not assumed. *(SLO 2.1, 2.4)*
- Describe what version control provides and why it matters in maintenance.
- Build Grade Calculator v2.0 and write a maintenance plan for the code you inherited.

13.1 Why Process Matters

Course I taught you to write programs. Course II is about building **software** — which differs in ways that only become visible at a certain size.

A program is something you write, run, and are finished with. Software is something that gets used, misunderstood, extended, fixed, handed to someone else, and still running years later. The techniques in this course — structs, classes, inheritance, polymorphism, exceptions — exist because of that difference. They cost effort up front and repay it in change.

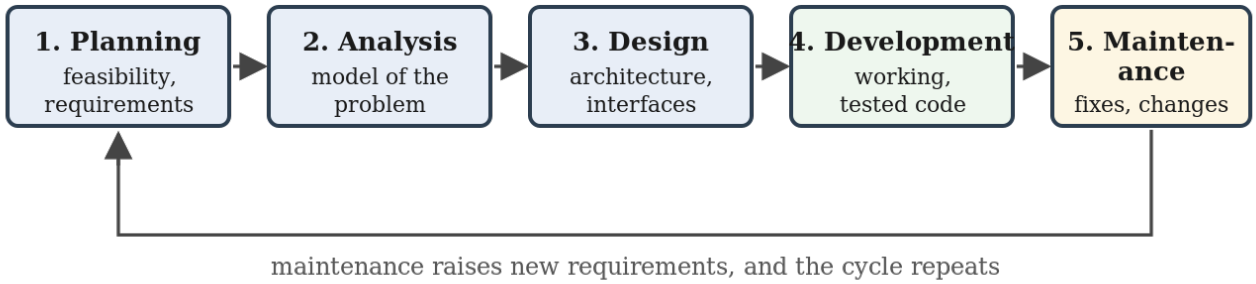
Course II opens here rather than with a new language feature because that framing changes what the rest of the material is *for*.

You are now in maintenance

Look at your position honestly. You have inherited a working application — Grade Calculator v1.3 — written by someone who is no longer available: yourself, twelve chapters ago. You did not write it recently. You do not remember every decision. There is a design document, of unknown accuracy.

That is the normal condition of professional software work. Most programmers spend most of their time changing code they did not write. This chapter is about doing that deliberately.

13.2 The Five Phases



Most of a program's life is spent in phase 5.
The Grade Calculator you inherit in this chapter is already there.

A defect found in planning costs minutes. The same defect found in maintenance costs orders of magnitude more.

Figure 12: Five boxes in a ring connected by arrows in order: Planning, Analysis, Design, Development, and Maintenance, with a long return arrow from Maintenance back to Planning.

Figure 13.1 — The five phases of the software development lifecycle.

Description of Figure 13.1. Five phases connected in order, with an arrow returning from the last to the first.

Phase	Question it answers	Output
1. Planning	Should this be built at all?	Feasibility assessment, requirements
2. Analysis	What exactly must it do?	A model of the problem
3. Design	How will it be built?	Architecture, interfaces, data structures
4. Development	Build it	Working, tested code
5. Maintenance	Keep it working and current	Fixes and enhancements

Maintenance raises new requirements, so the cycle repeats. Most of a program's life is spent in phase 5 — and the Grade Calculator is already there. A defect found in planning costs minutes; the same defect found in maintenance costs orders of magnitude more.

Planning

Decide whether the work is worth doing. Gather requirements from the people who will use the thing. Estimate cost and time.

Your Chapter 1 requirements statement was planning. So was the sentence deferring weighted grading — **a scope decision is a planning decision**, and recording it is what separates a decision from an oversight.

Analysis

Understand the problem thoroughly before deciding on a solution. What are the real-world things involved? What rules govern them? What can go wrong?

For a gradebook: what is an assignment? Can points possible be zero? Can a percentage exceed 100? Every one of those questions cost you a chapter when it arrived late.

Analysis describes the **problem**, not the solution. That distinction is easy to lose and worth guarding.

Design

Decide how to build it. Chapter 5's structure charts and pseudocode were design. In Course II design also means deciding what the **objects** are — which is Chapter 18.

Development

Write the code, integrate the pieces, test as you go. This is what Course I taught, and it is one phase of five.

Maintenance

Four kinds of work happen here, and they are worth naming because people tend to think only of the first:

Kind	Meaning	Example
Corrective	Fixing defects	The 89.95 boundary problem
Adaptive	Responding to a changed environment	A new compiler, a new file format
Perfective	Adding capability users want	Weighted grading
Preventive	Making future change easier	Refactoring; converting to classes

Perfective and preventive work dominate. Most maintenance is not fixing what is broken; it is changing what works.

13.3 Lifecycle Models

The phases are the same. The models differ in how you move through them.

Waterfall

Each phase completes before the next begins. Simple to plan and to audit.

It assumes you know all the requirements at the start, which is rarely true. And a mistake in analysis is not discovered until development, by which point it is expensive. Waterfall suits work where requirements genuinely are fixed and the cost of failure is high.

Iterative

Go through all five phases, produce something working, then go through them again adding more.

This book is iterative. Every chapter produces a complete running program that does less than the next one. That is not a teaching convenience — it is a lifecycle model, and the completeness rule stated in the front matter is what makes it one. Twenty-three working versions, each a full pass through design, development, and test.

Agile

Iterative, with short cycles, continuous user feedback, and a willingness to change requirements as understanding improves. Favors working software over documentation, and responding to change over following a plan.

Agile does not mean no design. It means designing continuously rather than once. The mistake agile is often used to justify — skipping analysis and design entirely — produces exactly the code Chapter 20 will show you the cost of.

	Waterfall	Iterative	Agile
Requirements fixed up front	yes	mostly	no
Working software available	at the end	after each cycle	continuously
Cost of a late change	very high	moderate	low
Suits	fixed, well-understood work	most projects	evolving requirements

---

13.4 Programming Methodologies

SLO 2.8 asks you to distinguish these, and Course II turns on the difference.

Unstructured

Early programs used `goto` to jump anywhere. Control flow became impossible to follow — the term was "spaghetti code." Appendix D Section D.11 excludes `goto` for this reason.

Procedural

Organize as a sequence of steps grouped into procedures operating on data passed to them. **Data and the code acting on it are separate.**

Course I is procedural. `computePercentage` receives points and returns a percentage; the points live elsewhere.

Structured

Procedural, restricted to **sequence, selection, and repetition** — Chapter 5 Section 5.7. Every program in this book is structured.

Object-oriented

Organize as **objects**, each bundling data with the operations on that data. Rather than passing a student's information to a grading function, you ask a student object for its grade.

Course II is object-oriented, and Chapters 14 and 18 make the transition concrete.

Which is better?

Neither, and the question is worth resisting.

Procedural code is appropriate when the data structures are simple and the operations are few. Course I's calculator is genuinely well served by it. Nothing in v1.3 would be improved by classes.

Object-oriented code earns its cost when a program has many kinds of thing, or when behavior must vary by kind. That second condition is precisely what weighted grading introduces, and it is why Course II's payoff arrives in Chapter 20 rather than being asserted in Chapter 14.

13.5 Coding Standards

A **coding standard** is a written agreement about how code will be formatted, named, and documented. Appendix D is this book's.

Consistency is not aesthetics. It is about **where your attention goes**. In a codebase with one style, an unusual construct stands out because it is unusual. In a codebase with five styles, everything looks unusual and nothing stands out.

This is SLO 2.4, and it is assessed by a style audit at every milestone from here on.

Refactoring

Refactoring means changing a program's internal structure without changing what it does. Every conversion in Course II is a refactor: parallel arrays to structs, structs to classes, one file to many.

The defining property is what makes it dangerous:

> **A refactor that changes behavior is a defect.**

And the only way to know whether behavior changed is to check. Not to be careful — to *check*. That is why v2.0's task is to make a substantial change and then prove, by comparing output, that nothing observable moved.

This is the discipline behind Chapter 9's rule about extracting one function at a time, and it becomes essential as the changes get larger.

Technical debt

Technical debt is the accumulated cost of shortcuts. Code that works but is hard to change is borrowed time, and the interest is paid by every future modification.

Some debt is worth taking deliberately. The Grade Calculator has carried an explicit debt since Chapter 1 — the deferred weighted-grading requirement — and it was the right call, because taking it on early would have cost Course I its clarity. The distinction that matters is between debt you chose and recorded, and debt you accumulated without noticing.

13.6 Version Control

A **version control system** records every change to a codebase, with who made it and why.

It gives you four things maintenance depends on:

- **History**. What changed, when, and by whom.
- **Undo**. Return to any earlier working state.
- **Branching**. Try a change without endangering working code.
- **Collaboration**. Several people editing without overwriting each other.

Git is the dominant system, and GitHub Codespaces has it available. Three commands cover most use:

```
git add .                       # stage your changes
git commit -m "Add drop-lowest"  # record them with a message
git log --oneline               # see the history
```

Commit each Grade Calculator version separately, with a message saying what changed. By Chapter 24 you will have a history of twelve versions — which is itself evidence for SLO 2.1, and considerably more convincing than a claim that you followed a process.

A good commit message says *why*, not *what*. The difference is exactly the one Appendix D Section D.3 draws about comments.

Common Process Failures

Failure	Consequence	Prevention
Coding before analyzing	Building the wrong thing correctly	Write a specification first
Undocumented scope decisions	Later readers cannot tell omission from choice	Record what is out of scope, and why
Refactoring without verifying	Silent behavior changes	Compare output before and after
Refactoring and adding features at once	You cannot tell which change broke it	Refactor, verify, *then* add
Ignoring the coding standard	Nothing looks unusual, so nothing stands out	Audit at every milestone
Undocumented technical debt	Shortcuts become permanent	Write down what you deferred
No version control	No undo, no history, no evidence	Commit every version

---

Design Notes

Analysis describes the problem; design describes the solution. Mixing them produces a solution to a problem nobody confirmed.

A scope decision is only a decision if it is written down. Otherwise it is an omission with a story attached.

Verify behavior-preserving changes. "I was careful" is not evidence.

Separate refactoring from feature work. Two changes at once means two candidate causes when something breaks.

Grade Calculator v2.0 — Inherited Codebase

Two deliverables. One is a document; one is a running program, because every chapter ships one.

Deliverable 1 — the maintenance plan

You have inherited v1.3. Write a maintenance and enhancement plan containing:

1. **What it does.** A summary of current capability, written by reading the code, not by remembering.

2. **What it does not do.** Current limitations, including the deferred weighted-grading requirement from Chapter 1.

3. **A prioritized backlog** for the term, each item classified as corrective, adaptive, perfective, or preventive:

Item	Kind	Why
Conform to Appendix D	preventive	Consistency makes everything else cheaper
Replace parallel arrays with records	preventive	Removes a class of bug — Chapter 14
Save and load gradebooks	perfective	Work is lost when the program exits
Fix defects found by testing	corrective	Chapter 16
Sort and search the roster	perfective	Chapter 17
Convert to classes	preventive	Chapter 18
Add weighted grading	**perfective**	**The deferred requirement — Chapters 20–21**

4. **An analysis of weighted grading.** This is the substantial part. Answer, in writing:

   - What is weighted-category grading, precisely?
   - How does it differ from points-based grading, arithmetically?
   - Which parts of v1.3 would have to change, and which would not?
   - What could go wrong — what if weights do not total 100%, or a category has no assignments yet?

Write this **before** you know how you will implement it. That is the position a maintenance programmer occupies, and doing analysis without a solution in hand is a skill worth practicing deliberately.

Chapter 20 will show you the answer. Compare it with what you wrote.

Deliverable 2 — the program

v2.0 brings the entire Course I codebase into conformance with **Appendix D**, and adds an About screen.

The behavior is deliberately unchanged.

What conforming involves

Work through Appendix D Section D.12's checklist:

- Four-space indentation, no tabs
- Braces on every block, opening brace on the same line
- No line over 90 characters
- `camelCase` variables, `SCREAMING_SNAKE_CASE` constants
- No magic numbers
- A file header with purpose and build command
- Documentation comments with `@param`, `@return`, `@pre`
- Comments explaining *why*, not *what*
- No commented-out code
- Compiles clean under `-Wall -Wextra`

The About screen

One small visible addition, so you finish the chapter with something new you can see:

```
// ------------------------------------------------------------------------
//  showAbout - prints version and build information.
// ------------------------------------------------------------------------
void showAbout() {
    std::cout << "\n-------------------------------------\n";
    std::cout << "  Grade Calculator\n";
    std::cout << "  Version : 2.0\n";
```

```
    std::cout << "  Course  : Object-Oriented Programming\n";
    std::cout << "  Built   : " << __DATE__ << " " << __TIME__ << "\n";
    std::cout << "  Scheme  : points-based only\n";
    std::cout << "------------------------------------\n\n";
}
```

`__DATE__` and `__TIME__` are filled in by the **preprocessor** — Chapter 2 Section 2.2.1 — so the banner reports when the executable was built. That is genuinely useful when someone reports a bug in a version you cannot identify.

Note the last line: `Scheme : points-based only`. The program tells its user what it does not do.

Verifying that nothing changed

This is the chapter's real lesson, and it is a procedure, not an intention.

1. Write down a fixed script of test input covering every case in your Chapter 12 test plan — the exact keystrokes, in order, so the run is repeatable.
2. Load the **unmodified** Chapter 12 starter, click **Run**, and type that input into the embedded Terminal. Copy the resulting course report into `baseline-output.md` in your repository.
3. Make your conformance changes. Click **Run** again and type the *same* input.
4. **Compare the two reports, line by line.** They must be identical apart from the banner.

Recording the baseline in the repository before you touch the code is what makes this a comparison rather than a recollection. A remembered output is not evidence; a committed one is.

Two identical reports are your evidence.

If they differ, you introduced a defect while "only" reformatting. Find it. That is the entire point of the exercise — and it is why "a refactor that changes behavior is a defect" is a rule rather than a slogan.

Your StudySite Lab — Establish the COSC 1437 Baseline

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v2.0
- **Starting point**: The private `COSC1437xxx-Grade-Calculator-YourLastName` repository your instructor created for you in the `lscup` organization and invited you to, and the complete Chapter 12 v1.3 solution supplied in this lab.

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

1. Create `main.cpp` with the complete supplied Chapter 12 starter code before making Chapter 13 changes. This is your Course I program consolidated into one file — see the note in Step 1.
2. Run the starter unchanged and confirm the complete points-based Grade Calculator works.
3. Create `maintenance-plan.md` identifying the coding-standard cleanup, behavior-preservation plan, test evidence, and weighted-grading requirement deferred to Chapters 20–21.
4. Bring the file's top header comment into conformance with the Appendix D file-header standard, without changing any grade calculation.
5. Add a `showAbout()` function that prints the program's version, course, and build date/time, and call it once at program startup.

6. Keep all Chapter 13–24 work in this same COSC 1437 repository.

This checkpoint **retires nothing** from Chapter 12. Every function you built in v1.3 — `readNonNegative`, `readLine`, `readYesNo`, `useDefaultScale`, `readGradeScale`, `letterFor`, and `computePercentage` — carries over unchanged, and no existing prompt or grade calculation changes. There is also still **no menu** in v2.0; the menu loop does not arrive until Chapter 15. The only code removed is `main`'s original two-line startup banner (`=== GRADE CALCULATOR v1.3 ===` / `Course I final version`); the only code added is the new `showAbout()` function that replaces it.

Build it: step by step

Each step below shows the actual code for that piece. Type it in as you go — don't wait until the end to test.

Step 1 — Create `main.cpp` with your consolidated Chapter 12 program Create `main.cpp` in the StudySite coding panel with the complete code below. Run this starter unchanged before beginning the Chapter 13 changes — this is the baseline you will diff your Chapter 13 output against.

> **Why one file, when Chapter 12 ended with three?** Your Course I program finished as `main.cpp`, `gradelib.h` and `gradelib.cpp` — Chapter 10 split it that way and Chapters 11 and 12 built on it. The code below is that same program consolidated back into a single file, and it behaves identically: run it against your Chapter 12 program with the same input and the output matches line for line.
>
> Course II starts consolidated on purpose. Chapters 13 to 18 change *what the program is made of* — a coding standard, a debugger, tests, then classes and objects — and following those changes is easier in one file. Chapter 19 then splits the program into files again, this time around the classes you will have built by then, which is a more durable division than a general-purpose `gradelib`. You are not losing the Chapter 10 lesson; you are about to apply it to a better set of boundaries.

```
// Grade Calculator v1.3 - Chapter 12 - COURSE I FINAL
// Unlimited roster, assignments, and grade tiers using std::vector.
// New this version: std::vector replaces fixed arrays, drop-lowest feature.
//
// Complete Course I feature set: named assignments, bonus points, custom
// letter scale, multi-student roster, class statistics, drop lowest.
// Grading model: points-based only. Weighted grading arrives in Chapter 20.
//
// Run: click Run in StudySite and use the embedded Terminal.

#include <cmath>
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>
#include <vector>

const bool CAP_AT_100 = true;

// ---------- input helpers ----------

double readNonNegative(const std::string& prompt) {
    double value = 0.0;
    while (true) {
        std::cout << prompt;
        if (!(std::cin >> value)) {
            if (std::cin.eof()) { return 0.0; }
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

```
            std::cout << "  That is not a number. Please try again.\n";
            continue;
        }
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        if (value < 0.0) {
            std::cout << "  Value cannot be negative. Please try again.\n";
            continue;
        }
        return value;
    }
}

std::string readLine(const std::string& prompt) {
    std::cout << prompt;
    std::string line;
    std::getline(std::cin, line);
    return line;
}

bool readYesNo(const std::string& prompt) {
    std::string answer = readLine(prompt);
    return !answer.empty() && (answer[0] == 'y' || answer[0] == 'Y');
}

// ---------- grade scale ----------

std::vector<double> gradeCutoffs;
std::vector<char> gradeLetters;

void useDefaultScale() {
    gradeCutoffs = {90.0, 80.0, 70.0, 60.0, 0.0};
    gradeLetters = {'A', 'B', 'C', 'D', 'F'};
}

void readGradeScale() {
    gradeCutoffs.clear();
    gradeLetters.clear();
    std::cout << "\n--- Define your grade scale ---\n";
    std::cout << "Enter tiers highest first. Type 'done' to finish.\n\n";

    while (true) {
        std::string letterText = readLine("Tier letter (or 'done'): ");
        if (letterText == "done" || letterText.empty()) { break; }
        double cutoff = readNonNegative("  Minimum percentage: ");
        if (!gradeCutoffs.empty() && cutoff >= gradeCutoffs.back()) {
            std::cout << "  Cutoff must be lower than " << gradeCutoffs.back()
                      << ". Tier not added.\n";
            continue;
        }
        gradeLetters.push_back(letterText[0]);
        gradeCutoffs.push_back(cutoff);
    }

    if (gradeCutoffs.empty() || gradeCutoffs.back() > 0.0) {
        std::cout << "Note: scale did not reach 0, so an 'F' at 0 was added.\n";
        gradeLetters.push_back('F');
        gradeCutoffs.push_back(0.0);
    }
}
```

```
char letterFor(double percentage) {
    for (std::size_t i = 0; i < gradeCutoffs.size(); ++i) {
        if (percentage >= gradeCutoffs[i]) { return gradeLetters[i]; }
    }
    return '?';
}

double computePercentage(double earned, double possible) {
    if (possible <= 0.0) { return 0.0; }
    double raw = earned / possible * 100.0;
    double reported = CAP_AT_100 ? std::min(raw, 100.0) : raw;
    return std::round(reported * 10.0) / 10.0;
}

int main() {
    std::cout << "=== GRADE CALCULATOR v1.3 ===\n";
    std::cout << "    Course I final version\n\n";

    if (readYesNo("Define a custom grade scale? (y/n): ")) {
        readGradeScale();
    } else {
        useDefaultScale();
        std::cout << "Using default scale: A 90, B 80, C 70, D 60, F 0.\n";
    }

    // Assignments: name and points possible, any number of them.
    std::vector<std::string> assignmentNames;
    std::vector<double> pointsPossible;

    std::cout << "\n--- Enter assignments ---\n";
    while (true) {
        std::string name = readLine("Assignment name (or 'done'): ");
        if (name == "done" || name.empty()) { break; }
        assignmentNames.push_back(name);
        pointsPossible.push_back(readNonNegative("  Points possible: "));
    }

    // Students: name plus one earned-points entry per assignment.
    std::vector<std::string> studentNames;
    std::vector<std::vector<double>> earned;

    std::cout << "\n--- Enter students ---\n";
    while (true) {
        std::string name = readLine("Student name (or 'done'): ");
        if (name == "done" || name.empty()) { break; }
        studentNames.push_back(name);

        std::vector<double> row;
        for (std::size_t a = 0; a < assignmentNames.size(); ++a) {
            row.push_back(readNonNegative("  " + assignmentNames[a] + " points (incl. bonus): "));
        }
        earned.push_back(row);
    }

    bool dropLowest = false;
    if (assignmentNames.size() > 1) {
        dropLowest = readYesNo("\nDrop each student's lowest assignment? (y/n): ");
    }

    std::cout << "\n====================================\n";
```

```
    std::cout << "  CLASS REPORT\n";
    std::cout << "======================================\n";
    std::cout << std::fixed << std::setprecision(1);

    double classTotal = 0.0;
    for (std::size_t s = 0; s < studentNames.size(); ++s) {
        double totalEarned = 0.0;
        double totalPossible = 0.0;
        for (std::size_t a = 0; a < assignmentNames.size(); ++a) {
            totalEarned += earned[s][a];
            totalPossible += pointsPossible[a];
        }

        // Dropping in a points-based scheme removes BOTH the earned points and
        // the possible points. Removing only one would distort the result.
        if (dropLowest && !assignmentNames.empty()) {
            std::size_t worst = 0;
            double worstRatio = 2.0;
            for (std::size_t a = 0; a < assignmentNames.size(); ++a) {
                if (pointsPossible[a] <= 0.0) { continue; }
                double ratio = earned[s][a] / pointsPossible[a];
                if (ratio < worstRatio) { worstRatio = ratio; worst = a; }
            }
            if (worstRatio <= 1.0) {
                totalEarned -= earned[s][worst];
                totalPossible -= pointsPossible[worst];
            }
        }

        double pct = computePercentage(totalEarned, totalPossible);
        classTotal += pct;
        std::cout << std::left << std::setw(20) << studentNames[s]
                  << std::right << std::setw(8) << pct << "%   "
                  << letterFor(pct) << "\n";
    }

    if (!studentNames.empty()) {
        std::cout << "--------------------------------------\n";
        std::cout << std::left << std::setw(20) << "CLASS AVERAGE"
                  << std::right << std::setw(8)
                  << classTotal / studentNames.size() << "%\n";
    } else {
        std::cout << "No students entered.\n";
    }
    return 0;
}
```

Step 2 — Reformat the file header comment to the Appendix D standard Appendix D requires the file containing `main` to open with a boxed header naming its purpose, what changed, how to verify the change, and the build command. Replace the six-line comment at the very top of the starter (the `// Grade Calculator v1.3 - Chapter 12 ...` block) with this box. Nothing else in the file's includes or code changes in this step.

```
// ============================================================================
//  Grade Calculator  v2.0                                        Chapter 13
// ----------------------------------------------------------------------------
//  Purpose : Points-based gradebook. Inherited from Course I (v1.3).
//  Change  : Conformed to the course coding standard (Appendix D).
//            Added an About screen. BEHAVIOR IS OTHERWISE UNCHANGED.
//  Verify  : run v1.3 and v2.0 with the same input and diff the output.
```

```
//            A refactor that changes behavior is a defect.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc
// ============================================================================
```

That is the entire coding-standard change for this checkpoint. The rest of v1.3 — four-space indentation, `camelCase` names, the `CAP_AT_100` constant instead of a magic number, and the existing `// ---------- input helpers ----------` / `// ---------- grade scale ----------` section banners — already matched Appendix D, so none of it needs to move or be renamed.

Step 3 — Add `showAbout()` and call it from `main` Add this new function directly above `main` (after `computePercentage`):

```
// ----------------------------------------------------------------------------
//  showAbout - prints version and build information.
// ----------------------------------------------------------------------------
void showAbout() {
    std::cout << "\n-------------------------------------\n";
    std::cout << "  Grade Calculator\n";
    std::cout << "  Version : 2.0\n";
    std::cout << "  Course  : Object-Oriented Programming\n";
    std::cout << "  Built   : " << __DATE__ << " " << __TIME__ << "\n";
    std::cout << "  Scheme  : points-based only\n";
    std::cout << "-------------------------------------\n\n";
}
```

Then delete `main`'s two original `std::cout` banner lines and call the new function in their place, as the very first statement in `main`:

```
int main() {
    showAbout();

    if (readYesNo("Define a custom grade scale? (y/n): ")) {
```

`showAbout` is a **screen**, not a menu — it always prints once, at startup, immediately before the existing "Define a custom grade scale?" prompt. A menu that lets the user choose it on demand does not exist until Chapter 15.

Step 4 — Write `maintenance-plan.md` Document, in your own words and based on the code you actually have:

- **What v1.3 does** — a summary of current capability, written by reading the code, not from memory.
- **What it does not do** — current limitations, including the deferred weighted-grading requirement.
- **A prioritized backlog** for the term, with each item classified as corrective, adaptive, perfective, or preventive (conform to Appendix D, replace parallel arrays with records, save/load gradebooks, fix defects found by testing, sort/search the roster, convert to classes, and add weighted grading).
- **A written analysis of weighted grading** — what it is, how it differs arithmetically from points-based grading, which parts of v1.3 would have to change, and what could go wrong (weights not totaling 100%, a category with no assignments yet). Write this before you know how you will implement it.

Verification

- The supplied v1.3 starter compiles and runs, unmodified, before you touch anything (Step 1).
- After Steps 2–3, the same input produces the same grades, letters, and class average as the unmodified v1.3 run — compare the two Terminal outputs line by line; they should differ only in the startup banner.
- The program compiles under `-std=c++17 -Wall -Wextra` with no warnings.

- Running the program prints the About banner (version, course, build date and time, and grading scheme) exactly once, at startup, immediately before the "Define a custom grade scale?" prompt.
- `maintenance-plan.md` includes the prioritized backlog table and the weighted-grading analysis.

StudySite workflow

1. Accept the GitHub invitation your instructor emailed you. It gives you access to **COSC1437xxx-Grade-Calculator-YourLastName**, the private repository already created for you in the **lscup** organization, where *YourLastName* is your own last name. Do not create your own repository, and do not continue Chapter 13 in the COSC 1436 repository.
2. In StudySite, open this chapter's **coding panel on the main stage**.
3. Click **Load from GitHub**. Connect GitHub if prompted, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click `README.md` to load it into the editor.
4. Complete Step 1 above: create `main.cpp` with the complete Chapter 12 starter code. Click **Run**, read the output in the embedded Terminal, and confirm the unchanged v1.3 program works.
5. Save this untouched baseline first: click **Save to GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName** and **main**, enter **Add Chapter 12 starter for COSC 1437**, and click **Commit**. Confirm the commit succeeds.
6. Complete Steps 2–4 above in the same coding panel. Keep every source and documentation file needed for this checkpoint open.
7. Click **Run** again. Fix every compiler error and warning, then complete the verification list.
8. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work:** StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 13 Grade Calculator v2.0**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- The verification list passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 13 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

No new C++ this chapter. These exercises are the professional skills SLO 2.1 and 2.8 name.

1. Classify maintenance work

Corrective, adaptive, perfective, or preventive?

- Fixing the 89.95 boundary problem from Chapter 6
- Adding weighted-category grading
- Converting parallel arrays to structs
- Updating the build command for a new compiler version
- Renaming variables to match Appendix D
- Adding a save-to-file feature
- Fixing a crash when the roster is empty

2. Which model?

Waterfall, iterative, or agile — and why?

- Flight control software, certified before deployment
- A startup's first product, with requirements still being discovered
- A university gradebook replacing a spreadsheet, with a fixed August deadline
- This textbook's Grade Calculator

3. Analysis without solution

Without writing any code, answer for weighted grading:

- Given Exams 50%, Homework 30%, Participation 20%, how is a course percentage computed from a list of assignments?
- What happens if a student has no participation assignments yet?
- What if the weights total 90% instead of 100%?
- What if an assignment has no category?
- Does bonus grading still work the same way? Does drop-lowest?

Your answers are the analysis phase. Keep them; Chapter 20 will let you check your reasoning.

4. Audit a codebase

Take one of your Chapter 11 or 12 programs and audit it against Appendix D Section D.12. List every violation. Do not fix them yet — just count them.

How many did you find? Would you have noticed any of them without the checklist?

5. Practice a verified refactor

Take a working program of your own. Rename every variable to be more descriptive. Change nothing else.

Then prove behavior did not change: capture output before and after with identical input, and `diff` them.

Now do it again, but deliberately introduce one small behavior change — swap two output lines. Confirm `diff` catches it. **The technique is only valuable if you have seen it detect something**.

6. Write a commit history

Initialize a Git repository containing your Grade Calculator. Commit v1.3, then v2.0.

Write both messages saying *why* rather than *what*. Then read them and ask: would these tell a stranger — or you in six months — what happened?

7. Procedural or object-oriented?

For each, say which methodology suits it better and why:

- Converting temperatures between Celsius and Fahrenheit
- A game with many kinds of character sharing some behavior
- Computing statistics for a list of numbers
- A drawing program handling circles, rectangles, and polygons
- Grade Calculator v1.3, with one grading scheme
- Grade Calculator v3.1, with three grading schemes

The last two are the same application. What changed between them, and why does that change the answer?

Summary

- **Software** differs from a program in that it is maintained, extended, and outlives its author's memory. Course II's techniques exist for that difference.
- The **lifecycle** has five phases: **planning**, **analysis**, **design**, **development**, and **maintenance**. Most of a program's life is maintenance.
- **Analysis describes the problem; design describes the solution.** Keeping them separate prevents solving a problem nobody confirmed.
- Maintenance is **corrective**, **adaptive**, **perfective**, or **preventive**. Most is not fixing defects.
- **Waterfall** completes each phase before the next. **Iterative** repeats all phases, producing working software each cycle. **Agile** shortens the cycle and expects requirements to change. **This book is iterative.**
- Methodologies: **procedural** separates data from the code acting on it; **structured** restricts control flow to sequence, selection, and repetition; **object-oriented** bundles data with its operations. Procedural is genuinely appropriate for simple data and few operations — which is why Course I is not a straw man.
- A **coding standard** matters because consistency is what lets unusual code stand out.
- **A refactor that changes behavior is a defect** — and the only way to know is to compare output, not to be careful.
- **Technical debt** you chose and recorded is a decision. Debt you accumulated without noticing is a problem.
- **Version control** provides history, undo, branching, and collaboration. Commit each version with a message saying why.

Key Terms

adaptive maintenance — changes responding to a changed environment.

agile — a lifecycle model with short cycles, continuous feedback, and changing requirements.

analysis — the phase that models the problem precisely.

coding standard — a written agreement on formatting, naming, and documentation.

corrective maintenance — fixing defects.

design — the phase deciding how a system will be built.

development — the phase in which code is written, integrated, and tested.

iterative model — a lifecycle repeating all phases, producing working software each cycle.

maintenance — the phase in which working software is kept correct and current.

object-oriented programming — organizing a program as objects bundling data with operations.

perfective maintenance — adding capability users want.

planning — the phase deciding whether and what to build.

preventive maintenance — changes making future modification easier.

procedural programming — organizing a program as procedures operating on separately held data.

refactoring — changing internal structure without changing behavior.

software development lifecycle — the phases through which software is planned, built, and maintained.

structured programming — restricting control flow to sequence, selection, and repetition.

technical debt — the accumulated future cost of shortcuts taken now.

version control — a system recording every change to a codebase.

waterfall model — a lifecycle completing each phase before beginning the next.

Next: Chapter 14 begins the technical work of Course II. Every parallel-array pair in your calculator collapses into a single record, and the class of bug where a grade attaches to the wrong student becomes structurally impossible rather than merely unlikely. Grade Calculator v2.1.

Chapter 14 — Structs: Grouping Related Data

Learning Objectives

When you finish this chapter you will be able to:

- Explain the specific weakness of parallel arrays that a record removes. *(SLO 2.2, 2.6)*
- Declare a `struct` and access its members. *(SLO 2.2)*
- Pass structs to and from functions, by value and by `const` reference. *(SLO 2.2)*
- Build and traverse a vector of structs. *(SLO 2.2, 2.6)*
- Nest structs to model composite information. *(SLO 2.2)*
- Design a set of records from a requirements statement. *(SLO 2.2)*
- Explain how a struct differs from a class, and why structs come first. *(SLO 2.2, 2.8)*
- Build Grade Calculator v2.1, replacing every parallel-array pair with a record.

14.1 The Problem with Parallel Arrays

Chapter 11 introduced parallel arrays and flagged their weakness. Chapter 13 put "replace parallel arrays with records" at the top of your maintenance backlog as *preventive* work. This chapter does it.

Here is the weakness, made concrete:

```
std::vector<std::string> names  = {"Ada", "Grace", "Alan"};
std::vector<double>      totals = {89.0, 91.5, 72.0};
```

The correspondence between `names[1]` and `totals[1]` exists only in your head. Nothing in the program records it. So:

- Sort one and not the other, and every grade attaches to the wrong student.
- Add to one and not the other, and they drift permanently out of step.
- Remove from the middle of one, and everything after it shifts by one.
- Pass one to a function without the other, and the function cannot know who a value belongs to.

Chapter 12's Exercise 6 demonstrated this: three students, one reordering, every grade wrong, and the program reported it confidently.

None of these are hard to avoid. All of them are easy to forget. That is the distinction worth holding on to — the goal is not to be more careful, but to make the mistake impossible to express.

14.2 Declaring and Using a `struct`

A **struct** groups related values into a single type:

```
struct Assignment {
    std::string name;
    double pointsPossible = 0.0;
};
```

That declares a new type named `Assignment`. The values inside are its **members**.

Note the semicolon after the closing brace. It is required, and forgetting it produces a confusing error pointing at the *next* line.

Create one and use it:

```
Assignment hw;
hw.name = "Homework 1";
hw.pointsPossible = 10.0;
```

Or initialize it directly:

```
Assignment hw{"Homework 1", 10.0};
```

```
hw.name=Homework 1 possible=10
```

Members are given values in declaration order. **Give members default values where you declare them**, as `= 0.0` does above — Appendix D Section D.6's initialization rule applies to members exactly as it does to variables.

14.3 Member Access

The dot operator reaches a member:

```
hw.pointsPossible = 20.0;
std::cout << hw.name;
double half = hw.pointsPossible / 2.0;
```

You have used this syntax since Chapter 12 — `scores.size()`, `name.length()`. Those were member *functions* on objects. This is member *data* on a struct. Chapter 18 unifies the two, which is exactly why structs come first.

14.4 Structs and Functions

A struct is a value like any other. It can be passed, returned, and assigned:

```
double total(const Student& s) {
    double t = 0.0;
    for (const Score& sc : s.scores) {
        t += sc.pointsEarned + sc.bonusPoints;
    }
    return t;
}
```

Chapter 10 Section 10.7's rule applies: **pass by `const` reference**. A struct may be large — this one contains a `std::string` and a `std::vector` — and copying it costs real work.

To modify, take a plain reference:

```
void addScore(Student& s, double earned, double bonus) {
    s.scores.push_back({earned, bonus});
}
```

Unlike arrays, structs **assign and copy properly**:

```
Student a = b;         // a full, independent copy — every member
```

This is one of the quiet advantages of records. Two parallel arrays cannot be copied as a unit; a struct can.

14.5 Vectors of Structs

The essential move:

```
std::vector<Student> roster;
roster.push_back({"Ada", 1001, {}});
roster.push_back({"Grace", 1002, {{10.0, 0.0}}});
```

One vector. One `push_back`. Every field travels together, permanently.

Now look at what happens when you sort:

```
std::sort(roster.begin(), roster.end(),
          [](const Student& a, const Student& b) { return total(a) > total(b); });
```

```
1001 Ada 94
1002 Grace 10
```

Every student's name, ID, and scores moved together, because they were never separate. **The bug from Chapter 12's Exercise 6 cannot be written here.** There is no second array to forget.

That is what "structurally impossible" means, and it is worth being precise about the claim. The struct did not make you more careful. It removed the thing you had to be careful about.

14.6 Nested Structs

A struct member may itself be a struct:

```
struct Score {
    double pointsEarned = 0.0;
    double bonusPoints  = 0.0;
};

struct Student {
    std::string name;
    int id = 0;
    std::vector<Score> scores;      // a vector of structs, inside a struct
};
```

Reach through with repeated dots:

```
ada.scores[0].bonusPoints = 1.0;
```

Read it left to right: the student `ada`, her `scores`, the first one, its `bonusPoints`.

Nesting lets a record mirror the shape of the real thing. A student *has* scores; a score *has* earned and bonus points. When the data structure matches the problem, code that walks it tends to read plainly.

14.7 Designing Records from Requirements

Deciding what the records are is a **design** activity — Chapter 13's phase 3.

The technique is simple and effective: **read your requirements statement and look for the nouns**.

From the Grade Calculator specification: *student*, *assignment*, *points earned*, *points possible*, *bonus points*, *grade scale*, *tier*, *cutoff*, *letter*.

Group nouns that always travel together:

Record	Members	Why grouped
`Assignment`	name, points possible	An assignment always has both
`Score`	points earned, bonus points	One student's result on one assignment
`Student`	name, ID, scores	A student has an identity and results
`GradeTier`	cutoff, letter	A cutoff is meaningless without its letter

That last one deserves attention. `GradeTier` replaces the two parallel vectors — `gradeCutoffs` and `gradeLetters` — that Chapter 11 introduced. They were the most fragile thing in v1.3, because a mismatch between them silently assigns wrong letters to everyone.

A useful test: if two values must always be updated together, they belong in one record.

14.8 Structs as a Bridge to Classes

A struct holds data. It has no behavior — the functions operating on it live outside:

```
struct Student { std::string name; int id = 0; std::vector<Score> scores; };

double total(const Student& s);          // outside the struct
double percentage(const Student& s);     // outside the struct
```

That is the **procedural** methodology from Chapter 13 Section 13.4: data and the code acting on it are separate.

Two consequences follow, and they are what Chapter 18 addresses.

Nothing protects the members. Any code can write `s.id = -5;` or leave `scores` inconsistent. The record groups the data but does not defend it.

The related functions are not attached to it. `total` and `percentage` are obviously about `Student`, but nothing in the program says so. They sit among every other function in the file.

A **class** fixes both: it bundles the data *with* the functions, and it can make members private so only those functions may touch them.

Structs come first because they are the smaller step. You get grouping now, and protection in Chapter 18, and you will be able to say precisely what the second step bought you.

In C++ a `struct` and a `class` are the same construct with one difference: members of a struct are public by default, members of a class are private by default. Everything in Chapter 18 could be written with `struct`. The convention this book follows — and Appendix D Section D.7 states — is `struct` for plain data with no invariants, `class` when there is behavior or something to protect.

Common Errors and Warnings

What you see	Cause	Fix
`error: expected ';' after struct definition`	Missing semicolon after the closing brace	Add `;`
`error: 'name' was not declared in this scope`	Used a member without the object	Write `s.name`, not `name`
`error: 'struct Student' has no member named 'Name'`	Wrong capitalization	C++ is case-sensitive
Members contain garbage	No default values given	`double x = 0.0;` in the struct
The program is slow with a large roster	Passing structs by value	Take `const Student&`
A function's changes do not stick	Passed by value	Take `Student&`
`error: too many initializers`	More values than members	Match the member list
Values land in the wrong members	Initializer order does not match declaration order	Assign member by member

---

Design Notes

If two values must always change together, put them in one record. That is the test.

Give every member a default value where it is declared.

Pass structs by `const` reference unless modifying.

Name records after the real-world thing they represent. `Student`, `Assignment`, `GradeTier` — nouns from the requirements, not invented abbreviations.

Keep records small and specific. A struct with fifteen unrelated members is a bag, not a record.

Grade Calculator v2.1 — Student Records

What v2.1 does

Everything v1.3 did, with every parallel-array pair replaced by a record. Behavior is unchanged; **structure is the change**, and this is a preventive-maintenance item from your Chapter 13 backlog.

The records

```
struct Assignment {
    std::string name;
    double pointsPossible = 0.0;
};
```

```
struct Score {
    double pointsEarned = 0.0;
    double bonusPoints  = 0.0;
};

struct Student {
    std::string name;
    int id = 0;
    std::vector<Score> scores;   // one entry per assignment, same order
};

struct GradeTier {
    double cutoff = 0.0;
    char   letter = 'F';
};
```

What each replaces

v1.3	v2.1
`std::vector<std::string> assignmentNames` + `std::vector<double> pointsPossible`	`std::vector<Assignment>`
`std::vector<std::string> studentNames` + `std::vector<std::vector<double>> earned`	`std::vector<Student>`
`std::vector<double> gradeCutoffs` + `std::vector<char> gradeLetters`	`std::vector<GradeTier>`

Six containers become three. More importantly, three opportunities for drift become zero.

The grading functions

```
std::vector<GradeTier> defaultScale() {
    return { {90.0, 'A'}, {80.0, 'B'}, {70.0, 'C'}, {60.0, 'D'}, {0.0, 'F'} };
}

char letterFor(double percentage, const std::vector<GradeTier>& scale) {
    for (const GradeTier& tier : scale) {
        if (percentage >= tier.cutoff) { return tier.letter; }
    }
    return '?';
}
```

Compare `letterFor` with v1.2's four-parameter version:

```
// v1.2 — two arrays and a count, all of which must agree
char letterFor(double percentage, const double cutoffs[], const char letters[], int count);

// v2.1 — one thing
char letterFor(double percentage, const std::vector<GradeTier>& scale);
```

Four parameters become two. The count is gone because the vector knows its size; the second array is gone because the tiers are one thing. Three ways to pass inconsistent arguments have disappeared from the signature.

Computing a student's percentage

```
double studentPercentage(const Student& student,
                         const std::vector<Assignment>& assignments,
                         bool dropLowest) {
    double earned = 0.0;
    double possible = 0.0;
    for (std::size_t a = 0; a < assignments.size() && a < student.scores.size(); ++a) {
        earned   += student.scores[a].pointsEarned + student.scores[a].bonusPoints;
        possible += assignments[a].pointsPossible;
    }
    if (dropLowest && assignments.size() > 1) {
        std::size_t worst = 0;
        double worstRatio = 2.0;
        for (std::size_t a = 0; a < assignments.size() && a < student.scores.size(); ++a) {
            if (assignments[a].pointsPossible <= 0.0) { continue; }
            double ratio = (student.scores[a].pointsEarned + student.scores[a].bonusPoints)
                         / assignments[a].pointsPossible;
            if (ratio < worstRatio) { worstRatio = ratio; worst = a; }
        }
        if (worstRatio <= 1.0) {
            earned   -= student.scores[worst].pointsEarned + student.scores[worst].bonusPoints;
            possible -= assignments[worst].pointsPossible;
        }
    }
    return computePercentage(earned, possible);
}
```

Expected output

Default scale; assignments `Homework 1` (10) and `Midterm` (100); Ada with 9 + 90 (bonus 5 on the midterm) and Alan with 8 + 70; drop-lowest off:

```
====================================
  CLASS REPORT
====================================
1001  Ada                   94.5%   A
1002  Alan                  70.9%   C
------------------------------------
CLASS AVERAGE               82.7%
```

What to notice

Student IDs appeared. Adding a field to a record is one line. Adding a parallel array would have meant finding every place the other arrays are touched and touching this one too — which is exactly the maintenance cost records remove.

`student.scores[a].pointsEarned` reads as a path. The student, her scores, the one for assignment `a`, the points earned. The expression describes the data.

The `&& a < student.scores.size()` guard. A student may have fewer scores recorded than there are assignments. The struct did not eliminate the need to check bounds — it eliminated the possibility of two containers being *silently* out of step. Chapter 18's `Student` class will enforce the correspondence properly.

Nothing about the grading arithmetic changed. This is a preventive refactor, and Chapter 13's rule applies: verify it by comparing output.

Your StudySite Lab — Replace Parallel Arrays with Structs

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v2.1
- **Starting point**: The working Chapter 13 v2.0 program.

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

1. Add `Assignment`, `Score`, `Student`, and `GradeTier` structs.
2. Replace the parallel `gradeCutoffs`/`gradeLetters` vectors with `defaultScale()` and `readGradeScale()` functions that build and return a `std::vector<GradeTier>` instead of writing to globals.
3. Change `letterFor` to take a `const std::vector<GradeTier>& scale` parameter instead of reading the (now-deleted) global scale vectors.
4. Replace the parallel `assignmentNames`/`pointsPossible` vectors with a `std::vector<Assignment>`, and the parallel `studentNames`/`earned` vectors with a `std::vector<Student>`, each carrying an auto-assigned `id` and a `std::vector<Score>`.
5. Extract the drop-lowest earned/possible math that lived inline in `main`'s report loop into a new `studentPercentage(const Student&, const std::vector<Assignment>&, bool dropLowest)` function.

This checkpoint also **retires the `showAbout()` About screen** — the startup banner becomes a single line, `=== GRADE CALCULATOR v2.1 (struct records) ===`. Two behaviors change along with the structs, not just their internal representation, so name them rather than assuming "preserve behavior": each student's score entry is now **two** prompts per assignment — "points earned" then "bonus points" — instead of v2.0's one combined "points (incl. bonus)" prompt, and the class report gains a new left column showing each student's auto-assigned ID (starting at 1001) before the name. `readYesNo`, `readNonNegative`, `readLine`, and `computePercentage` all carry over unchanged.

Build it: step by step

Each step below shows the actual code for that piece. Type it in as you go — don't wait until the end to test. Where a step **replaces** something from v2.0, the old code is named so you know what to remove.

Step 1 — The four structs Add these right after `const bool CAP_AT_100 = true;`, before the input helpers.

```
struct Assignment {
    std::string name;
    double pointsPossible = 0.0;
};

struct Score {
    double pointsEarned = 0.0;
    double bonusPoints  = 0.0;
};

struct Student {
    std::string name;
    int id = 0;
    std::vector<Score> scores;   // one entry per assignment, same order
};

struct GradeTier {
```

```
    double cutoff = 0.0;
    char   letter = 'F';
};
```

`Score` is new — v2.0 had no record for one assignment's result, only a bare `double` sitting in a parallel `earned` vector. The other three replace a pair of parallel containers each. Give every member a default value, as shown — Appendix D's initialization rule applies to struct members exactly as it does to variables.

Step 2 — Grade scale functions return a vector instead of writing globals Delete the two global vectors and rewrite `useDefaultScale`/`readGradeScale` so each **returns** a `std::vector<GradeTier>` instead of mutating shared state:

```
// DELETE these two globals:
// std::vector<double> gradeCutoffs;
// std::vector<char> gradeLetters;

std::vector<GradeTier> defaultScale() {
    return { {90.0, 'A'}, {80.0, 'B'}, {70.0, 'C'}, {60.0, 'D'}, {0.0, 'F'} };
}

std::vector<GradeTier> readGradeScale() {
    std::vector<GradeTier> scale;
    std::cout << "\n--- Define your grade scale ---\n";
    std::cout << "Enter tiers highest first. Type 'done' to finish.\n\n";

    while (true) {
        std::string letterText = readLine("Tier letter (or 'done'): ");
        if (letterText == "done" || letterText.empty()) { break; }
        double cutoff = readNonNegative("  Minimum percentage: ");
        if (!scale.empty() && cutoff >= scale.back().cutoff) {
            std::cout << "  Cutoff must be lower than " << scale.back().cutoff
                      << ". Tier not added.\n";
            continue;
        }
        scale.push_back({cutoff, letterText[0]});
    }
    if (scale.empty() || scale.back().cutoff > 0.0) {
        std::cout << "Note: scale did not reach 0, so an 'F' at 0 was added.\n";
        scale.push_back({0.0, 'F'});
    }
    return scale;
}
```

Renamed: `useDefaultScale` (void, wrote to globals) becomes `defaultScale` (returns a value). `readGradeScale` keeps its name but changes from `void` to a function returning `std::vector<GradeTier>`.

```
char letterFor(double percentage, const std::vector<GradeTier>& scale) {
    for (const GradeTier& tier : scale) {
        if (percentage >= tier.cutoff) { return tier.letter; }
    }
    return '?';
}
```

Step 3 — `letterFor` takes the scale as a parameter Compare with v2.0's version, which read the two now-deleted globals by name and indexed them in lockstep: `for (std::size_t i = 0; i < gradeCutoffs.size(); ++i) { if (percentage >= gradeCutoffs[i]) { return gradeLetters[i]; } }`. The new version cannot desynchronize a cutoff from its letter — there is only one thing to index into.

Step 4 — `studentPercentage`: pull the drop-lowest math out of `main` `computePercentage` carries over unchanged and is now called from inside this new function instead of directly from `main`.

```
double studentPercentage(const Student& student,
                         const std::vector<Assignment>& assignments,
                         bool dropLowest) {
    double earned = 0.0;
    double possible = 0.0;
    for (std::size_t a = 0; a < assignments.size() && a < student.scores.size(); ++a) {
        earned   += student.scores[a].pointsEarned + student.scores[a].bonusPoints;
        possible += assignments[a].pointsPossible;
    }
    if (dropLowest && assignments.size() > 1) {
        std::size_t worst = 0;
        double worstRatio = 2.0;
        for (std::size_t a = 0; a < assignments.size() && a < student.scores.size(); ++a) {
            if (assignments[a].pointsPossible <= 0.0) { continue; }
            double ratio = (student.scores[a].pointsEarned + student.scores[a].bonusPoints)
                         / assignments[a].pointsPossible;
            if (ratio < worstRatio) { worstRatio = ratio; worst = a; }
        }
        if (worstRatio <= 1.0) {
            earned   -= student.scores[worst].pointsEarned + student.scores[worst].bonusPoints;
            possible -= assignments[worst].pointsPossible;
        }
    }
    return computePercentage(earned, possible);
}
```

This is the same earned/possible loop and worst-assignment search that used to live directly inside `main`'s report `for` loop, indexed into the `earned`/`pointsPossible` parallel containers. Nothing about the arithmetic changes — only where it lives and what it reads.

Step 5 — Delete `showAbout` Delete the whole `showAbout()` function. In `main`, replace the `showAbout();` call with one line:

```
std::cout << "=== GRADE CALCULATOR v2.1 (struct records) ===\n";
```

The About screen — version, course, build date/time, scheme — is gone. There is no struct-based replacement for it in v2.1; it is simply retired.

Step 6 — Rewrite the assignment and student entry loops in `main` Replace the parallel-vector declarations and both entry loops with:

```
std::vector<Assignment> assignments;
std::cout << "\n--- Enter assignments ---\n";
while (true) {
    std::string name = readLine("Assignment name (or 'done'): ");
    if (name == "done" || name.empty()) { break; }
    Assignment a;
    a.name = name;
    a.pointsPossible = readNonNegative("  Points possible: ");
    assignments.push_back(a);
}

std::vector<Student> roster;
int nextId = 1001;
std::cout << "\n--- Enter students ---\n";
while (true) {
```

```
    std::string name = readLine("Student name (or 'done'): ");
    if (name == "done" || name.empty()) { break; }
    Student s;
    s.name = name;
    s.id = nextId++;
    for (const Assignment& a : assignments) {
        Score sc;
        sc.pointsEarned = readNonNegative("  " + a.name + " points earned: ");
        sc.bonusPoints  = readNonNegative("  " + a.name + " bonus points: ");
        s.scores.push_back(sc);
    }
    roster.push_back(s);
}
```

Gone: `std::vector<std::string> assignmentNames`, `std::vector<double> pointsPossible`, `std::vector<std::string> studentNames`, and `std::vector<std::vector<double>> earned`. Notice the single combined "points (incl. bonus)" prompt from v2.0 is now two prompts per assignment, one per `Score` member, and each student gets an `id` the moment they're added. The `dropLowest` line right after the student loop is unchanged from v2.0 except that it now tests `assignments.size()` instead of `assignmentNames.size()`.

```
double classTotal = 0.0;
for (const Student& s : roster) {
    double pct = studentPercentage(s, assignments, dropLowest);
    classTotal += pct;
    std::cout << std::left << std::setw(6) << s.id
              << std::setw(20) << s.name
              << std::right << std::setw(8) << pct << "%   "
              << letterFor(pct, scale) << "\n";
}
if (!roster.empty()) {
    std::cout << "-----------------------------------\n";
    std::cout << std::left << std::setw(26) << "CLASS AVERAGE"
              << std::right << std::setw(8) << classTotal / roster.size() << "%\n";
} else {
    std::cout << "No students entered.\n";
}
```

Step 7 — Rewrite the report loop The report no longer computes earned/possible totals inline — it calls `studentPercentage`. `letterFor` now takes `scale` as its second argument. The new `std::setw(6) << s.id` column pushes the name and total further right, which is why `"CLASS AVERAGE"` widens from `setw(20)` to `setw(26)` to keep the divider and the average aligned under the report above it.

Verification

- Student names cannot be reordered separately from their scores.
- A grade letter cannot exist without its cutoff.
- Given the same assignment points, student scores, and drop-lowest choice, v2.1 computes the same percentages and letters as v2.0 — remember that the score-entry prompts themselves changed (earned and bonus are now entered separately, so re-derive equivalent totals when comparing runs).
- Each student is assigned a unique ID starting at 1001, in entry order.
- The program builds without warnings.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click each source, header, or documentation file needed for this chapter. Confirm the editor shows the expected file paths before editing.
4. Work through Steps 1–7 above in StudySite's internal editor, in order. Click **Run** after Steps 1–5 (once you have the four structs, the scale functions returning a vector, the parameterized `letterFor`, and `studentPercentage` in place, and `showAbout` deleted) even though `main`'s entry loops haven't changed yet — the file should still compile. Then apply Steps 6 and 7 together, since the rewritten report loop calls `studentPercentage` and the new `letterFor`.
5. Click **Run**. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted.
6. Fix every compiler error and warning, then complete the verification list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work:** StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 14 Grade Calculator v2.1**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- The verification list passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 14 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion. —

Try It Yourself

1. A first struct

```cpp
#include <iostream>
#include <string>

struct Assignment {
    std::string name;
    double pointsPossible = 0.0;
};
```

```
int main() {
    Assignment hw;
    hw.name = "Homework 1";
    hw.pointsPossible = 10.0;

    Assignment exam{"Midterm", 100.0};

    std::cout << hw.name << " is worth " << hw.pointsPossible << "\n";
    std::cout << exam.name << " is worth " << exam.pointsPossible << "\n";
    return 0;
}
```

Expected output:

```
Homework 1 is worth 10
Midterm is worth 100
```

Try: Remove the `= 0.0` default, declare an `Assignment` without initializing, and print `pointsPossible`. What do you get?

2. Structs in functions

```
#include <iostream>
#include <string>

struct Assignment {
    std::string name;
    double pointsPossible = 0.0;
};

void describe(const Assignment& a) {
    std::cout << a.name << ": " << a.pointsPossible << " points\n";
}

void doubleValue(Assignment& a) {
    a.pointsPossible *= 2.0;
}

int main() {
    Assignment hw{"Homework 1", 10.0};
    describe(hw);
    doubleValue(hw);
    describe(hw);
    return 0;
}
```

Expected output:

```
Homework 1: 10 points
Homework 1: 20 points
```

Try: Remove the `&` from `doubleValue`'s parameter. What changes, and why?

3. Nested structs and a vector of structs

```
#include <iostream>
#include <string>
#include <vector>
```

```
struct Score   { double pointsEarned = 0.0; double bonusPoints = 0.0; };
struct Student { std::string name; int id = 0; std::vector<Score> scores; };

double total(const Student& s) {
    double t = 0.0;
    for (const Score& sc : s.scores) { t += sc.pointsEarned + sc.bonusPoints; }
    return t;
}

int main() {
    Student ada;
    ada.name = "Ada";
    ada.id = 1001;
    ada.scores.push_back({9.0, 1.0});
    ada.scores.push_back({84.0, 0.0});

    std::cout << ada.name << " (" << ada.id << ") total = " << total(ada) << "\n";
    std::cout << "first bonus: " << ada.scores[0].bonusPoints << "\n";
    return 0;
}
```

Expected output:

```
Ada (1001) total = 94
first bonus: 1
```

Try: Add a third score and rerun. How many lines did you change?

4. Sorting keeps records intact

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

struct Student { std::string name; int id = 0; double total = 0.0; };

int main() {
    std::vector<Student> roster = {
        {"Ada", 1001, 89.0}, {"Grace", 1002, 91.5}, {"Alan", 1003, 72.0}
    };

    std::sort(roster.begin(), roster.end(),
              [](const Student& a, const Student& b) { return a.total > b.total; });

    for (const Student& s : roster) {
        std::cout << s.id << " " << s.name << " " << s.total << "\n";
    }
    return 0;
}
```

Expected output:

```
1002 Grace 91.5
1001 Ada 89
1003 Alan 72
```

Try: Compare this with Chapter 12's Exercise 6, where sorting one parallel array scrambled the data. What would you have to do here to produce that bug?

5. Replace parallel arrays

Rewrite this using a struct. Then say which specific bug becomes impossible.

```
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> titles = {"Intro", "Advanced"};
    std::vector<int> credits = {3, 4};
    std::vector<char> grades = {'A', 'B'};

    for (std::size_t k = 0; k < titles.size(); ++k) {
        std::cout << titles[k] << " " << credits[k] << " " << grades[k] << "\n";
    }
    return 0;
}
```

6. Design records from requirements

For a library system: *A member has a name, a membership number, and a list of borrowed books. A book has a title, an author, an ISBN, and a due date when borrowed. A due date has a day, a month, and a year.*

Write the structs. Which are nested? Which values must always change together?

7. Reason about records

- Why does a struct need a semicolon after its closing brace?
- Why pass a struct by `const` reference rather than by value?
- What is the test for whether two values belong in the same record?
- A struct groups data but does not protect it. Give one specific way a caller could put a `Student` into an invalid state.
- What is the only real difference between `struct` and `class` in C++?

Summary

- **Parallel arrays** keep their correspondence only in the programmer's head. A **struct** records it in the program.
- A struct declares a new type whose **members** are accessed with the dot operator. The declaration ends with a **semicolon**.
- **Give every member a default value where it is declared.**
- Structs are passed and returned like any value. **Pass by `const` reference** unless modifying. Unlike arrays, structs **copy properly**.
- A **vector of structs** keeps every field of every record together permanently. Sorting cannot scramble them, because there is nothing to keep in step.
- Structs **nest**, letting a record mirror the shape of the real thing.
- Design records by finding the **nouns** in your requirements. **If two values must always be updated together, they belong in one record.**
- A struct groups data but does not **protect** it, and the functions that operate on it live outside. Chapter 18's **class** fixes both.

- In C++ `struct` and `class` differ only in default access. Use `struct` for plain data, `class` when there is behavior or an invariant to defend.

Key Terms

aggregate initialization — initializing a struct's members with a brace-enclosed list in declaration order.

dot operator — `.`, used to reach a member of a struct or object.

member — a variable declared inside a struct or class.

nested struct — a struct used as the type of a member of another struct.

record — a group of related values treated as one item; in C++, a struct.

struct — a type grouping related values under one name.

Next: Chapter 15 makes your gradebook survive being closed. File streams save the roster, the assignments, and the custom grade scale to a text file and read them back — and you will design the file format yourself, including deciding whether it can accommodate a feature you have not built yet. Grade Calculator v2.2.

Chapter 15 — File Streams and Persistent Data

Learning Objectives

When you finish this chapter you will be able to:

- Explain what a stream abstracts, and why files and the console share an interface. *(SLO 2.1)*
- Write to a file with `std::ofstream` and read from one with `std::ifstream`. *(SLO 2.6)*
- Check stream state and handle a file that is missing or malformed. *(SLO 2.1)*
- Read a file line by line and split each line into fields. *(SLO 2.6)*
- Convert text to numbers safely with `std::stod` and `std::stoi`. *(SLO 2.6)*
- Use `std::stringstream` to build and parse text in memory. *(SLO 2.6)*
- Design a documented file format, and judge whether it can accommodate future data. *(SLO 2.1, 2.4)*
- Build Grade Calculator v2.2 — a gradebook that survives being closed.

15.1 Streams as an Abstraction

Chapter 1 Section 1.2 said main memory is **volatile**: when your program ends, everything it held is gone. Every version of the Grade Calculator so far has thrown away the entire roster on exit.

Files fix that, and this chapter is item three on your Chapter 13 backlog — *perfective* maintenance, adding capability users want.

A **stream** is a sequence of data flowing to or from somewhere. You have used two since Chapter 2:

```
std::cout << "text";     // a stream to the screen
std::cin >> value;       // a stream from the keyboard
```

File streams use the **same operators**:

```
std::ofstream out("gradebook.csv");
out << "text";                       // a stream to a file

std::ifstream in("gradebook.csv");
in >> value;                          // a stream from a file
```

That sameness is the point of the abstraction. Code written against a stream does not care what is on the other end, which means the reporting code you already have could write to a file with almost no change. Chapter 19 exploits this properly with `operator<<`.

Type	Header	Direction
`std::ofstream`	`<fstream>`	output — write to a file
`std::ifstream`	`<fstream>`	input — read from a file

Type	Header	Direction
`std::ostringstream`	`<sstream>`	output — build a string in memory
`std::istringstream`	`<sstream>`	input — read from a string in memory

15.2 Writing to a File

```
#include <fstream>

std::ofstream out("gradebook.csv");
out << "STUDENT,1001,Ada Lovelace\n";
out << "WORK,1001,Midterm,84,100,5\n";
```

The file is created if it does not exist and **emptied if it does**. That second behavior deserves respect — opening for output destroys the previous contents immediately, before you write anything.

The file closes automatically when the stream goes out of scope. You may close it explicitly with `out.close()`, and it is worth doing when you need to reopen the file for reading in the same function.

Everything from Chapter 4 works: `std::fixed`, `std::setprecision`, `std::setw` all apply to file streams exactly as to `std::cout`.

15.3 Reading from a File

```
std::ifstream in("gradebook.csv");

std::string line;
while (std::getline(in, line)) {
    // process one line
}
```

`std::getline(in, line)` reads one line and returns the stream, which converts to `false` when there is nothing more. That makes it the standard idiom for reading a whole file — and it is Chapter 7's sentinel loop, with end-of-file as the sentinel.

15.4 Checking Stream State

A file may not open. It may not exist, or you may lack permission. Always check:

```
std::ifstream in("gradebook.csv");
if (!in) {
    std::cout << "No file named 'gradebook.csv' was found.\n";
    return;
}
```

```
opened nope.csv? false
!missing evaluates to true
```

A stream converts to `false` when something has gone wrong. `!in` is the idiomatic test.

Test	True when
`in.is_open()`	the file opened successfully
`in.eof()`	reading reached end of file
`in.fail()`	an operation failed, such as reading text into a number
`in.good()`	nothing has gone wrong
`!in`	the stream is in a failed or ended state

Failing to check is the most common file-handling defect, and the symptom is misleading: a program that reads a missing file simply sees no data and reports an empty gradebook, exactly as though the file were empty. The user concludes their work was lost.

15.5 Parsing a Line into Fields

Reading a line gives you text. Splitting it is the next step, and `std::istringstream` makes it straightforward:

```
#include <sstream>

std::vector<std::string> splitCsv(const std::string& line) {
    std::vector<std::string> fields;
    std::istringstream in(line);
    std::string field;
    while (std::getline(in, field, ',')) {
        fields.push_back(field);
    }
    return fields;
}
```

The third argument to `getline` is the **delimiter** — the character that ends each piece. Reading a string as though it were a stream is a genuinely useful trick, and it appears again whenever text needs structure.

```
1: (skipped comment)
2: tag=STUDENT fields=3
3: tag=WORK fields=6
```

> **This simple splitter does not handle fields containing commas.** A student named `Lovelace, Ada` would split into two fields and corrupt the record. Real CSV handles this with quoting. This book's format documentation states the restriction explicitly, which is the honest alternative to pretending the problem does not exist.

15.6 Converting Text to Numbers

Fields arrive as text. Converting is a place where bad data becomes a crash if you are careless:

Function	Converts to
`std::stod(s)`	`double`
`std::stoi(s)`	`int`

```
double points = std::stod("84.5");     // 84.5
```

```
stod("abc") threw invalid_argument
stod("84.5") = 84.5
```

These throw when the text is not a number. `std::stod("abc")` raises `std::invalid_argument`; a number too large raises `std::out_of_range`. An unhandled exception terminates the program.

Catch them:

```
try {
    points = std::stod(field);
} catch (const std::invalid_argument&) {
    std::cout << "Line " << lineNumber << ": expected a number.\n";
}
```

Chapter 24 covers exceptions properly. What you need here is the shape — `try`, then `catch` — and the reason: **a file is outside your program, so nothing guarantees its contents are what you expect.** That is Chapter 11's lesson about validating external data, arriving in a new form.

15.7 Building Text in Memory

`std::ostringstream` builds a string using stream operators:

```
#include <sstream>

std::ostringstream out;
out << std::fixed << std::setprecision(1) << percentage << "%";
std::string text = out.str();
```

This is how `formatGrade` was implemented in Chapter 10 without explanation. It is the clean way to produce formatted text when you need a `std::string` rather than immediate output.

15.8 Designing a File Format

This is the design work of the chapter, and it is more consequential than the code.

Requirements for a gradebook format

Human-readable. You should be able to open it and see what is wrong. Binary formats are compact and useless when something breaks.

Line-oriented. One record per line, so a corrupt line damages one record rather than everything.

Self-describing. A reader should be able to tell what each line is without counting positions.

Extensible. New kinds of record should be addable without invalidating existing files.

A format that meets them

```
# Grade Calculator gradebook file, version 1
# Section tags allow new record types to be added later.
NEXTID,1002
TIER,90,A
```

```
TIER,80,B
TIER,0,F
ASSIGNMENT,Homework 1,10
STUDENT,1001,Ada,9,1
```

Each line begins with a **tag** naming the record type. Lines beginning with `#` are comments and are skipped. A reader dispatches on the tag and ignores tags it does not recognize.

That last property is what makes the format extensible. Adding a `CATEGORY` record later does not break a reader that has never heard of one — it skips the line and carries on.

The question you cannot answer yet

Your Chapter 13 maintenance plan analyzed weighted grading. Weighted grading needs category names and weights stored somewhere.

Can this format hold them? Think about it before reading on, and write down your answer.

The tag design says yes: add `CATEGORY,Exam,50` lines, and old readers skip them while new readers use them. If instead you had chosen a fixed-column format — "field 4 is always points possible" — adding data would mean changing every position and invalidating every existing file.

You will find out in Chapter 21 whether your format actually was extensible. That is a real test of a design decision made twelve chapters earlier, and it is worth making deliberately rather than by luck.

Documenting the format

Write it down, in a file beside your code:

```
GRADEBOOK FILE FORMAT, version 1
--------------------------------
Plain text, one record per line, fields separated by commas.
Lines beginning with # are comments. Blank lines are ignored.
Unrecognized tags are skipped, so new record types may be added.

RESTRICTION: fields may not contain commas.

NEXTID,<n>                      the next student ID to assign
TIER,<cutoff>,<letter>          one grade scale tier, highest first
ASSIGNMENT,<name>,<possible>    one assignment
STUDENT,<id>,<name>,<e1>,<b1>,<e2>,<b2>,...
                                one student, then earned/bonus pairs
```

Appendix D Section D.3 asks you to document decisions. **A file format is a promise to every future version of your program**, and an undocumented one is a promise nobody can keep.

15.9 Robust File Handling

Four rules, each from a real failure mode.

Check that the file opened. Report it clearly if not.

Report the line number on any error. `"Cannot read 'gradebook.csv' at line 14: expected a number."` is actionable. `"parse error"` is not.

Decide what a malformed line means, and be consistent. Two defensible policies: skip the bad line and carry on, or refuse the whole file. v2.2 skips and warns; Chapter 24's release build refuses, because a partly-loaded gradebook is worse than none.

Never destroy existing data on a failed load. Build the new gradebook in a separate variable and install it only on success. Loading a corrupt file should leave you with what you had, not with nothing.

Common Errors and Warnings

What you see or observe	Cause	Fix
The program reports an empty gradebook	The file did not open and nobody checked	Test `if (!in)`
`error: 'ifstream' was not declared`	Missing `#include <fstream>`	Add the header
Previous data vanished	Opening for output truncates	Check before overwriting
`terminate called after throwing 'std::invalid_argument'`	`stod` on non-numeric text	Wrap in `try`/`catch`
Only the first field is read	Used `>>` instead of `getline` with a delimiter	`std::getline(ss, field, ',')`
A name with a comma splits into two fields	Format restriction	Document it, or quote fields
Reading gets one extra empty record	Trailing newline at end of file	Skip empty lines
The file is written but is empty	Stream not flushed or closed	Let it go out of scope, or `close()`
`error: 'stod' was not declared`	Missing `#include <string>`	Add the header

Design Notes

Check every file operation. The failure mode is silence, which is worse than a crash.

Report the line number. Users cannot fix what they cannot find.

Design formats to be readable and extensible. Tags over positions.

Build the replacement, then install it. A failed load must leave the previous state untouched.

Document the format, including its restrictions. The commas-in-fields limitation is real; saying so is the honest choice.

Grade Calculator v2.2 — Save and Load

What v2.2 does

Everything v2.1 did, plus: the roster, the assignments, and the custom grade scale all persist to `gradebook.csv` and load automatically at startup. A menu-driven loop replaces the run-once flow.

An instructor's custom grading scale now survives between sessions, which is the feature that makes the calculator usable as an actual tool rather than a demonstration.

Saving

```
bool saveGradebook(const Gradebook& book, const std::string& filename) {
    std::ofstream out(filename);
    if (!out) { return false; }

    out << "# Grade Calculator gradebook file, version 1\n";
    out << "# Section tags allow new record types to be added later.\n";
    out << "NEXTID," << book.nextId << "\n";
    for (const GradeTier& t : book.scale) {
        out << "TIER," << t.cutoff << "," << t.letter << "\n";
    }
    for (const Assignment& a : book.assignments) {
        out << "ASSIGNMENT," << a.name << "," << a.pointsPossible << "\n";
    }
    for (const Student& s : book.roster) {
        out << "STUDENT," << s.id << "," << s.name;
        for (const Score& sc : s.scores) {
            out << "," << sc.pointsEarned << "," << sc.bonusPoints;
        }
        out << "\n";
    }
    return out.good();
}
```

Loading

```
bool loadGradebook(Gradebook& book, const std::string& filename, std::string& message) {
    std::ifstream in(filename);
    if (!in) {
        message = "No file named '" + filename + "' was found.";
        return false;
    }

    Gradebook loaded;              // build separately; install only on success
    std::string line;
    int lineNumber = 0;
    while (std::getline(in, line)) {
        ++lineNumber;
        if (line.empty() || line[0] == '#') { continue; }
        std::vector<std::string> f = splitCsv(line);
        if (f.empty()) { continue; }

        try {
            if (f[0] == "NEXTID" && f.size() >= 2) {
                loaded.nextId = std::stoi(f[1]);
            } else if (f[0] == "TIER" && f.size() >= 3) {
                loaded.scale.push_back({std::stod(f[1]), f[2].empty() ? '?' : f[2][0]});
            } else if (f[0] == "ASSIGNMENT" && f.size() >= 3) {
                loaded.assignments.push_back({f[1], std::stod(f[2])});
            } else if (f[0] == "STUDENT" && f.size() >= 3) {
                Student s;
                s.id = std::stoi(f[1]);
                s.name = f[2];
                for (std::size_t i = 3; i + 1 < f.size(); i += 2) {
                    s.scores.push_back({std::stod(f[i]), std::stod(f[i + 1])});
                }
                loaded.roster.push_back(s);
            } else {
```

```
                message = "Unrecognized record on line " + std::to_string(lineNumber)
                        + "; the rest of the file was still loaded.";
            }
        } catch (...) {
            message = "Malformed number on line " + std::to_string(lineNumber)
                    + "; that record was skipped.";
        }
    }

    if (loaded.scale.empty()) {
        loaded.scale = { {90.0,'A'}, {80.0,'B'}, {70.0,'C'}, {60.0,'D'}, {0.0,'F'} };
        message = "File had no grade scale; the default scale was used.";
    }
    book = loaded;                  // install, now that it is known good
    if (message.empty()) { message = "Loaded " + filename + "."; }
    return true;
}
```

A saved file

```
# Grade Calculator gradebook file, version 1
# Section tags allow new record types to be added later.
NEXTID,1002
TIER,90,A
TIER,80,B
TIER,70,C
TIER,60,D
TIER,0,F
ASSIGNMENT,Homework 1,10
STUDENT,1001,Ada,9,1
```

Expected behavior

Add an assignment and a student, save, quit, and restart:

```
=== GRADE CALCULATOR v2.2 (persistent) ===

Loaded gradebook.csv.

1) Add assignment  2) Add student  3) Report  4) Save  5) Quit
```

The report shows the student entered in the previous session.

What to notice

`Gradebook loaded;` then `book = loaded;` — the whole reason for the temporary. A file that fails partway leaves `book` untouched. Assigning directly into `book` as you parse would leave a half-loaded gradebook if line 14 were malformed.

Every record type is checked for field count before its fields are used. `f.size() >= 3` before touching `f[2]` prevents an out-of-range access on a truncated line — Chapter 11's bounds lesson, applied to data from outside the program.

Unrecognized tags produce a warning, not a failure. That is what makes the format extensible, and it is the property Chapter 21 will test.

A missing grade scale is repaired with the default. A gradebook with no scale cannot assign letters at all, so a sensible default beats an unusable file — and the message says what happened.

`catch (...)` catches everything. That is a blunt instrument, appropriate here because any conversion failure means the same thing. Chapter 24 replaces it with specific exception types and better messages.

Adding an assignment gives every existing student a zero score for it, keeping `scores` the same length as `assignments`. That correspondence is exactly what Chapter 18's `Student` class will enforce rather than maintain by hand.

Your StudySite Lab — Save and Load the Gradebook

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v2.2
- **Starting point**: The working Chapter 14 v2.1 program.

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

1. Add a `Gradebook` struct that bundles the scale, assignments, and roster together, and give it up to `saveGradebook`/`loadGradebook`.
2. Add file-stream support that saves the grade scale, assignments, and students to `gradebook.csv`.
3. Load `gradebook.csv` at startup when it exists; parse into a temporary `Gradebook` and replace the active one only after the whole file is valid.
4. Replace the single batch-entry pass from v2.1 with a menu loop: add assignment, add student, print report, save, quit.
5. Create `gradebook-format.md` documenting every record tag, its field order, and the restriction on commas in text fields.

This checkpoint also **retires two Chapter 14 features**: drop-lowest, and the interactive custom-grade-scale prompt (`readGradeScale`). Persistence and the menu loop are the point of this chapter; either could return later as its own addition, but neither is part of v2.2. A saved file can still carry a custom scale — `loadGradebook` reads whatever `TIER` rows are in the file — there's just no more interactive prompt to define one from a blank slate.

Build it: step by step

Each step below shows the actual code for that piece. Type it in as you go — don't wait until the end to test. Where a step **replaces** something from v2.1, the old code is named so you know what to remove.

Step 1 — New includes, constants, and the `Gradebook` struct Add `<fstream>` and `<sstream>` to your includes, add `DEFAULT_FILE`, and add a `Gradebook` struct that owns everything `main` used to hold in separate local variables.

```
#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>
#include <string>
#include <vector>
```

```
const bool CAP_AT_100 = true;
const std::string DEFAULT_FILE = "gradebook.csv";

struct Assignment { std::string name; double pointsPossible = 0.0; };
struct Score      { double pointsEarned = 0.0; double bonusPoints = 0.0; };
struct Student    { std::string name; int id = 0; std::vector<Score> scores; };
struct GradeTier  { double cutoff = 0.0; char letter = 'F'; };

struct Gradebook {
    std::vector<Assignment> assignments;
    std::vector<Student>    roster;
    std::vector<GradeTier>  scale;
    int nextId = 1001;
};
```

`Score`, `Assignment`, `Student`, and `GradeTier` are unchanged from Chapter 14. `Gradebook` is new — it's the one thing you'll save, load, and pass around instead of three separate vectors and an `int nextId`.

Delete four functions carried over from Chapter 14 that nothing in v2.2 calls: `readYesNo`, `defaultScale`, `readGradeScale`, and `computePercentage`. `readYesNo` supported the old yes/no prompts, which the menu loop replaces. `defaultScale` and `readGradeScale` supported the interactive custom-scale prompt, which v2.2 drops (see above). `computePercentage` is folded directly into the new `percentageOf` in Step 5. Keep `readNonNegative` and `readLine` exactly as they were in v2.1 — both are still used.

Step 2 — A CSV line splitter Every load needs to turn one text line back into fields. Add this above the save/load functions:

```
/** Splits one CSV line on commas. Fields containing commas are not supported;
 *  the format documentation states this restriction explicitly. */
std::vector<std::string> splitCsv(const std::string& line) {
    std::vector<std::string> fields;
    std::istringstream in(line);
    std::string field;
    while (std::getline(in, field, ',')) { fields.push_back(field); }
    return fields;
}
```

Step 3 — `saveGradebook` Writes the whole `Gradebook` as tagged CSV rows: one `NEXTID` line, one `TIER` line per grade tier, one `ASSIGNMENT` line per assignment, one `STUDENT` line per student (with that student's scores trailing on the same line).

```
bool saveGradebook(const Gradebook& book, const std::string& filename) {
    std::ofstream out(filename);
    if (!out) { return false; }

    out << "# Grade Calculator gradebook file, version 1\n";
    out << "# Section tags allow new record types to be added later.\n";
    out << "NEXTID," << book.nextId << "\n";
    for (const GradeTier& t : book.scale) {
        out << "TIER," << t.cutoff << "," << t.letter << "\n";
    }
    for (const Assignment& a : book.assignments) {
        out << "ASSIGNMENT," << a.name << "," << a.pointsPossible << "\n";
    }
    for (const Student& s : book.roster) {
        out << "STUDENT," << s.id << "," << s.name;
        for (const Score& sc : s.scores) {
```

```
            out << "," << sc.pointsEarned << "," << sc.bonusPoints;
        }
        out << "\n";
    }
    return out.good();
}
```

Step 4 — `loadGradebook` This is the step most students under-build: it reads into a **local** `Gradebook loaded`, not directly into `book`. Only if the whole file parses does `book = loaded;` run at the end. A bad line partway through a bad file must never leave `book` half-overwritten.

```
/** Loads a gradebook. Returns false and leaves book untouched on failure. */
bool loadGradebook(Gradebook& book, const std::string& filename, std::string& message) {
    std::ifstream in(filename);
    if (!in) {
        message = "No file named '" + filename + "' was found.";
        return false;
    }

    Gradebook loaded;
    std::string line;
    int lineNumber = 0;
    while (std::getline(in, line)) {
        ++lineNumber;
        if (line.empty() || line[0] == '#') { continue; }
        std::vector<std::string> f = splitCsv(line);
        if (f.empty()) { continue; }

        try {
            if (f[0] == "NEXTID" && f.size() >= 2) {
                loaded.nextId = std::stoi(f[1]);
            } else if (f[0] == "TIER" && f.size() >= 3) {
                loaded.scale.push_back({std::stod(f[1]), f[2].empty() ? '?' : f[2][0]});
            } else if (f[0] == "ASSIGNMENT" && f.size() >= 3) {
                loaded.assignments.push_back({f[1], std::stod(f[2])});
            } else if (f[0] == "STUDENT" && f.size() >= 3) {
                Student s;
                s.id = std::stoi(f[1]);
                s.name = f[2];
                for (std::size_t i = 3; i + 1 < f.size(); i += 2) {
                    s.scores.push_back({std::stod(f[i]), std::stod(f[i + 1])});
                }
                loaded.roster.push_back(s);
            } else {
                message = "Unrecognized record on line " + std::to_string(lineNumber)
                        + "; the rest of the file was still loaded.";
            }
        } catch (...) {
            message = "Malformed number on line " + std::to_string(lineNumber)
                    + "; that record was skipped.";
        }
    }

    if (loaded.scale.empty()) {
        loaded.scale = { {90.0,'A'}, {80.0,'B'}, {70.0,'C'}, {60.0,'D'}, {0.0,'F'} };
        message = "File had no grade scale; the default scale was used.";
    }
    book = loaded;
    if (message.empty()) { message = "Loaded " + filename + "."; }
```

```
    return true;
}
```

Step 5 — Grading functions, carried over and trimmed `letterFor` carries over from Chapter 14 with only its parameter names tidied up (`percentage`→`pct`, `tier`→`t`) — the logic is unchanged. `studentPercentage` from v2.1 becomes `percentageOf`: the same earned/possible math, now computing the rounded percentage inline instead of calling the `computePercentage` helper you deleted in Step 1, and with its `dropLowest` parameter and the worst-assignment search removed, because drop-lowest isn't part of v2.2. `printReport` is new: it's the class-report loop pulled out of `main` so the menu's `3) Report` option can call it on demand instead of it running once at the end of the program.

```
char letterFor(double pct, const std::vector<GradeTier>& scale) {
    for (const GradeTier& t : scale) { if (pct >= t.cutoff) { return t.letter; } }
    return '?';
}

double percentageOf(const Student& s, const std::vector<Assignment>& as) {
    double earned = 0.0;
    double possible = 0.0;
    for (std::size_t a = 0; a < as.size() && a < s.scores.size(); ++a) {
        earned   += s.scores[a].pointsEarned + s.scores[a].bonusPoints;
        possible += as[a].pointsPossible;
    }
    if (possible <= 0.0) { return 0.0; }
    double raw = earned / possible * 100.0;
    return std::round((CAP_AT_100 ? std::min(raw, 100.0) : raw) * 10.0) / 10.0;
}

void printReport(const Gradebook& book) {
    std::cout << "\n====================================\n  CLASS REPORT\n";
    std::cout << "====================================\n";
    std::cout << std::fixed << std::setprecision(1);
    if (book.roster.empty()) { std::cout << "No students on the roster.\n\n"; return; }
    for (const Student& s : book.roster) {
        double pct = percentageOf(s, book.assignments);
        std::cout << std::left << std::setw(6) << s.id << std::setw(20) << s.name
                  << std::right << std::setw(8) << pct << "%   "
                  << letterFor(pct, book.scale) << "\n";
    }
    std::cout << "\n";
}
```

Step 6 — Rewrite `main` as a menu loop This is the biggest structural change in the chapter. Delete the entire body of your v2.1 `main` — the custom-grade-scale prompt, the assignment-entry loop, the student-entry loop, the drop-lowest prompt, and the final report — and replace it with this:

```
int main() {
    std::cout << "=== GRADE CALCULATOR v2.2 (persistent) ===\n\n";

    Gradebook book;
    book.scale = { {90.0,'A'}, {80.0,'B'}, {70.0,'C'}, {60.0,'D'}, {0.0,'F'} };

    std::string message;
    if (loadGradebook(book, DEFAULT_FILE, message)) {
        std::cout << message << "\n\n";
    } else {
        std::cout << message << " Starting a new gradebook.\n\n";
```

```
    }

    bool running = true;
    while (running) {
        std::cout << "1) Add assignment  2) Add student  3) Report  "
                     "4) Save  5) Quit\nChoice: ";
        std::string line;
        if (!std::getline(std::cin, line)) { break; }
        char choice = line.empty() ? '?' : line[0];

        if (choice == '1') {
            Assignment a;
            a.name = readLine("  Assignment name: ");
            if (a.name.empty()) { continue; }
            a.pointsPossible = readNonNegative("  Points possible: ");
            book.assignments.push_back(a);
            for (Student& s : book.roster) { s.scores.push_back({0.0, 0.0}); }
            std::cout << "  Added.\n\n";
        } else if (choice == '2') {
            Student s;
            s.name = readLine("  Student name: ");
            if (s.name.empty()) { continue; }
            s.id = book.nextId++;
            for (const Assignment& a : book.assignments) {
                Score sc;
                sc.pointsEarned = readNonNegative("  " + a.name + " points earned: ");
                sc.bonusPoints  = readNonNegative("  " + a.name + " bonus: ");
                s.scores.push_back(sc);
            }
            book.roster.push_back(s);
            std::cout << "  Added as ID " << s.id << ".\n\n";
        } else if (choice == '3') {
            printReport(book);
        } else if (choice == '4') {
            std::cout << (saveGradebook(book, DEFAULT_FILE)
                          ? "  Saved to " + DEFAULT_FILE + ".\n\n"
                          : "  Could not write " + DEFAULT_FILE + ".\n\n");
        } else if (choice == '5') {
            running = false;
        } else {
            std::cout << "  Please enter 1-5.\n\n";
        }
    }

    if (saveGradebook(book, DEFAULT_FILE)) {
        std::cout << "Gradebook saved. Goodbye.\n";
    } else {
        std::cout << "Warning: could not save. Goodbye.\n";
    }
    return 0;
}
```

Notice the program now **loads on startup and saves on exit automatically**, in addition to the explicit `4) Save` option — so a student who forgets to save before quitting doesn't lose the session.

Step 7 — Write `gradebook-format.md` Document the file format your code just implemented: the `NEXTID`, `TIER`, `ASSIGNMENT`, and `STUDENT` tags, the field order for each, and the rule that comma-containing text fields aren't supported. Base it on what `saveGradebook` actually writes — don't describe a format your code doesn't produce.

Verification

- A save-then-load round trip reproduces the same gradebook.
- A missing file is handled without a crash (a first run with no `gradebook.csv` starts a new, empty gradebook).
- A nonnumeric field, truncated student row, unknown tag, and missing grade tiers are each rejected or repaired without crashing.
- A failed load does not destroy the active in-memory data.
- Quitting from the menu, and letting the program exit any other way, both leave a valid `gradebook.csv` behind.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click each source, header, or documentation file needed for this chapter. Confirm the editor shows the expected file paths before editing.
4. Work through Steps 1–7 above in StudySite's internal editor, in order. Click **Run** after Steps 1–4 (once you have `Gradebook`, `splitCsv`, `saveGradebook`, and `loadGradebook` in place) even though `main` hasn't changed yet — the file should still compile. Then apply Steps 5 and 6 together, since `main` calls `printReport`.
5. Click **Run**. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted.
6. Fix every compiler error and warning, then complete the verification list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor, including `gradebook-format.md`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 15 Grade Calculator v2.2**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- The verification list passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 15 checkpoint, including `gradebook-format.md`.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Write and read a file

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    {
        std::ofstream out("demo.txt");
        out << "first line\n";
        out << "second line\n";
    }   // the stream closes here

    std::ifstream in("demo.txt");
    if (!in) {
        std::cout << "could not open demo.txt\n";
        return 1;
    }
    std::string line;
    while (std::getline(in, line)) {
        std::cout << "read: " << line << "\n";
    }
    return 0;
}
```

Expected output:

```
read: first line
read: second line
```

Try: Change the filename in the `ifstream` to something that does not exist. Confirm the check fires. Then remove the check and see what the program reports instead — that silence is the failure mode from Section 15.4.

2. Splitting a line into fields

```
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

std::vector<std::string> splitCsv(const std::string& line) {
    std::vector<std::string> fields;
    std::istringstream in(line);
    std::string field;
    while (std::getline(in, field, ',')) { fields.push_back(field); }
    return fields;
}

int main() {
    std::vector<std::string> f = splitCsv("STUDENT,1001,Ada Lovelace,84,5");
    std::cout << "fields: " << f.size() << "\n";
    for (const std::string& s : f) { std::cout << "[" << s << "]\n"; }
    return 0;
}
```

Expected output:

```
fields: 5
[STUDENT]
[1001]
[Ada Lovelace]
[84]
[5]
```

Try: Split `"STUDENT,1001,Lovelace, Ada,84"`. How many fields now? This is the restriction from Section 15.5 — see it happen.

3. Converting text safely

```cpp
#include <iostream>
#include <stdexcept>
#include <string>

int main() {
    std::cout << "stod(\"84.5\") = " << std::stod("84.5") << "\n";
    try {
        std::cout << std::stod("abc") << "\n";
    } catch (const std::invalid_argument&) {
        std::cout << "stod(\"abc\") threw invalid_argument\n";
    }
    return 0;
}
```

Expected output:

```
stod("84.5") = 84.5
stod("abc") threw invalid_argument
```

Try: Remove the `try`/`catch` and rerun. The program terminates — that is an unhandled exception, and Chapter 24 explains what happened.

4. Building text in memory

```cpp
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>

std::string formatGrade(double percentage) {
    std::ostringstream out;
    out << std::fixed << std::setprecision(1) << percentage << "%";
    return out.str();
}

int main() {
    std::cout << formatGrade(89.666) << "\n";
    std::cout << "[" << formatGrade(100.0) << "]\n";
    return 0;
}
```

Expected output:

```
89.7%
[100.0%]
```

Try: Change it to pad the result to eight characters using `std::setw`.

5. A round trip

Write a program that saves three names and scores to a file, then reads them back and prints them. Confirm what comes out matches what went in.

Then, deliberately: edit the file by hand to remove a comma from one line. What does your program do? Make it report the line number rather than crashing or silently skipping.

6. Judge a format

Which of these gradebook formats is more extensible, and why? What breaks in the weaker one if a category and weight must be added to each assignment?

```
Format A:
Ada,1001,84,100,5
Grace,1002,91,100,0

Format B:
STUDENT,1001,Ada
WORK,1001,Midterm,84,100,5
STUDENT,1002,Grace
WORK,1002,Midterm,91,100,0
```

7. Reason about files

- Why must you check whether a file opened, when a missing file produces no error message?
- Why build the loaded data in a temporary and assign it at the end?
- What is the advantage of a tag at the start of each line over fixed field positions?
- Why report the line number in a parse error?
- Your program writes `gradebook.csv` and a user opens it in a spreadsheet, edits it, and saves. What could go wrong, and what does that suggest about documenting the format?

Summary

- A **stream** is a sequence of data flowing to or from somewhere. File streams use the same `<<` and `>>` as the console, which is what makes the abstraction valuable.
- `std::ofstream` writes; `std::ifstream` reads; both need `<fstream>`. **Opening for output empties the file immediately**.
- `while (std::getline(in, line))` is the standard idiom for reading a whole file.
- **Always check that a file opened**. The failure mode is silence — an empty result indistinguishable from an empty file.
- Split a line into fields with `std::istringstream` and `std::getline` with a **delimiter**.
- `std::stod` and `std::stoi` convert text to numbers and **throw** on bad input. A file is outside your program, so its contents must be validated.
- `std::ostringstream` builds formatted text in memory.
- Design formats to be **human-readable**, **line-oriented**, **self-describing**, and **extensible**. Tags beat fixed positions because unknown tags can be skipped.
- **Document the format, including its restrictions**.
- Build loaded data separately and install it only on success, so a failed load never destroys what you had.

Key Terms

delimiter — the character separating fields in a line of text.

file stream — a stream connected to a file.

ifstream — an input file stream.

ofstream — an output file stream.

parsing — extracting structured values from text.

persistence — storing data so it survives after a program ends.

stream state — the flags recording whether a stream has failed or reached its end.

stringstream — a stream reading from or writing to a string in memory.

tag — a leading field identifying what kind of record a line holds.

truncate — to empty a file when opening it for output.

Next: Chapter 16 promotes debugging to a first-class skill. You will learn the symbolic debugger — breakpoints, stepping, watches, and the call stack — and then use it on a broken build of the Grade Calculator containing three seeded defects that are invisible on ordinary input. Grade Calculator v2.3.

Chapter 16 — Debugging and Testing

Learning Objectives

When you finish this chapter you will be able to:

- Follow a systematic debugging process rather than guessing. *(SLO 2.7)*
- Set breakpoints and step through a running program. *(SLO 2.7)*
- Inspect variables and read the call stack. *(SLO 2.7)*
- Use conditional breakpoints and watch expressions. *(SLO 2.7)*
- Diagnose run-time and logic errors with a debugger. *(SLO 2.7)*
- Use assertions and diagnostic output appropriately. *(SLO 2.7)*
- Write test cases that target boundaries rather than ordinary values. *(SLO 2.1, 2.4)*
- Write a defect report and a regression test. *(SLO 2.4)*
- Build Grade Calculator v2.3 by finding and fixing three seeded defects.

16.1 Categories of Defects, Revisited

Chapter 4 Section 4.9 named four ways programs go wrong. That was a taxonomy. This chapter is the tooling.

Debugging is promoted to a full chapter here, before the object-oriented material, because you will need it for the rest of the book. A debugger is not a last resort for desperate situations. It is the ordinary way to find out what a program is actually doing rather than what you assume it is doing.

Recall the four categories:

Kind	Found by	Difficulty
Compile-time	the compiler tells you the line	easiest
Run-time	the program crashes at a specific point	moderate
Logic	nothing announces it	**hardest**
Warning	the compiler tells you	easy, if you read them

Logic errors are what this chapter is really about. The program compiles, runs, finishes, and reports a wrong grade. No message, no crash, no clue.

16.2 A Systematic Debugging Process

The instinct when something is wrong is to change code and see whether it helps. That is guessing, and it fails in a specific way: you often "fix" the symptom by breaking something else, and you never learn what was wrong.

Six steps instead:

1. **Reproduce it.** Find input that produces the fault reliably. A defect you cannot reproduce cannot be verified as fixed.

2. **Narrow it.** Find the smallest input that still fails. If a 30-student roster misreports, does a 2-student roster? Does one student?

3. **Form a hypothesis.** State what you think is wrong, specifically enough to test. "Something is wrong with the grade scale" is not a hypothesis. "`letterFor` returns the wrong letter when the percentage exactly equals a cutoff" is.

4. **Test the hypothesis.** Use the debugger to check. This is where breakpoints go.

5. **Fix it.** Change the cause, not the symptom.

6. **Verify, and write a regression test.** Confirm the fix works, confirm nothing else broke, and write a test that would have caught it — so it cannot come back silently.

Step 3 is where the discipline lives. **Before you look, predict what you will see.** If the debugger shows what you expected, your hypothesis survives and you move on. If it shows something else, you have learned something real. Either way you have made progress, which is more than guessing offers.

16.3 The Symbolic Debugger

A **symbolic debugger** runs your program under supervision, letting you pause it and inspect it in terms of *your* names — your variables, your functions, your line numbers. "Symbolic" is that last part: without it you would be reading machine registers.

This book uses the debugger in **GitHub Codespaces**, which is VS Code. The concepts transfer to any debugger; only the keystrokes differ.

Build with debug information

The debugger needs a map from machine code back to your source. Add `-g`:

```
g++ -std=c++17 -Wall -Wextra -g main.cpp -o gradecalc
```

Without `-g`, you can still run the program but the debugger cannot show you your variable names or lines. **A build you intend to debug must include `-g`.**

16.3.1 Breakpoints

A **breakpoint** tells the debugger to pause when execution reaches a particular line.

To set one in VS Code, click in the narrow gutter to the left of a line number; a red dot appears. Start debugging with **F5**. The program runs normally until it reaches that line, then stops with everything still in memory and available to inspect.

Set a breakpoint where you can first observe the problem, not at the top of `main`. If a grade is wrong, break at the line that computes it.

16.3.2 Stepping

Once paused, four commands move you forward:

Command	Key	Does
Continue	F5	run until the next breakpoint
Step Over	F10	execute this line; do not enter functions it calls
Step Into	F11	execute this line, entering the first function it calls
Step Out	Shift+F11	finish the current function and return to its caller

The distinction that matters is **Step Over** versus **Step Into**. If a line calls `computePercentage` and you trust it, step *over* — the call happens and you stay put. If you suspect it, step *into* and watch it work.

A useful strategy: step over everything until you see a value go wrong, then rerun and step *into* the call that produced it. Repeating this narrows the fault to one function quickly, without reading the whole program.

16.3.3 Inspecting Variables

Paused, you can see everything:

- The **Variables** panel lists locals and their current values automatically.
- **Hovering** over a name in the editor shows its value.
- The **Watch** panel evaluates expressions you supply, updating as you step.

Watch expressions are the underused one. You can watch `scale.size()`, or `earned / possible`, or a comparison such as `pct >= 90.0` — and see it as `true` or `false` at each step. **Watching the exact condition you suspect is often faster than reasoning about it.**

16.3.4 The Call Stack

The **call stack** shows how execution reached the current line: which function called which.

Figure 16.1 — The call stack when a defect is reached.

Description of Figure 16.1. Four **stack frames**, drawn bottom to top. At the bottom is `main()`, holding `choice = '3'`. Above it is `printReport(book)`, holding `index = 0` and `book.size() = 3`. Above that is `percentageOf(student, work)`, holding `earned = 9.0` and `possible = 10.0`. At the top, where execution is paused, is `letterFor(pct, scale)`, holding `pct = 90.0`, `i = 5`, and `scale.size() = 5`. The top frame is where the program is now; reading downward shows how it got there.

Each **frame** holds one function call's local variables. Clicking a frame switches the Variables panel to *that* function's locals, so you can inspect the caller's state without restarting.

The call stack answers a question that is otherwise very hard: **"how did I get here?"** When a function misbehaves only sometimes, the stack shows which caller produced the bad call — and looking at that caller's variables usually explains why.

Look closely at the top frame in Figure 16.1. The loop index `i` is 5 and `scale.size()` is also 5. That is a valid index range of 0 to 4, and `i` is 5. **The defect is visible in the frame**, without reading a line of code.

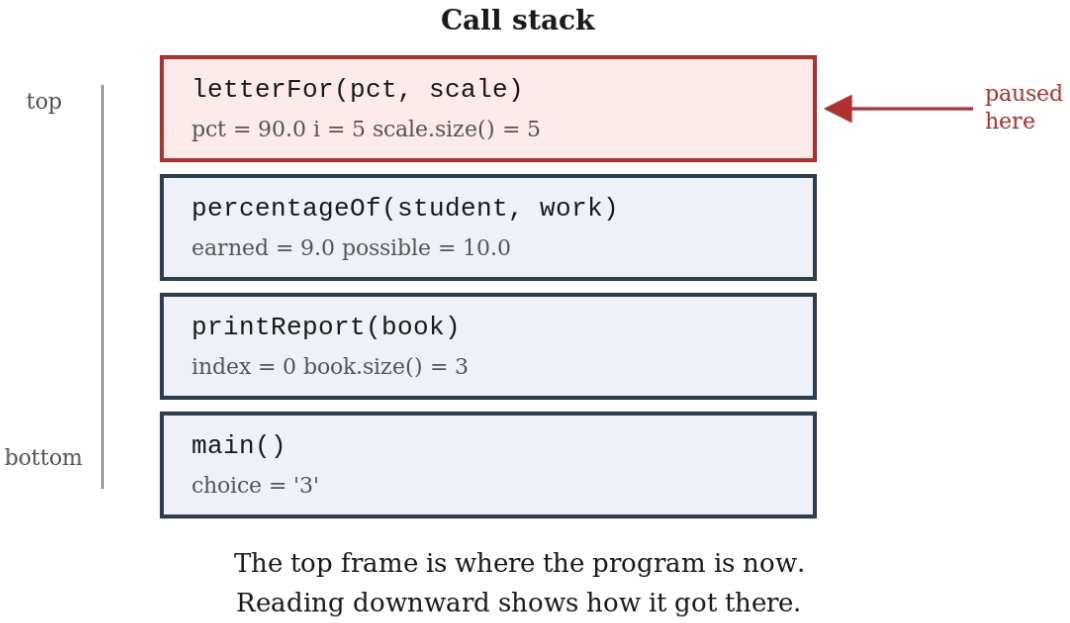


Figure 13: A stack of four frames drawn bottom to top: main at the bottom, then printReport, then percentageOf, and letterFor at the top where execution is paused.

16.3.5 Conditional Breakpoints and Watchpoints

A plain breakpoint inside a loop over 300 students fires 300 times. A **conditional breakpoint** fires only when a condition holds.

In VS Code, right-click a breakpoint and choose *Edit Breakpoint*, then enter a condition:

```
student.id == 1042
percentage >= 89.0 && percentage <= 91.0
i >= scale.size()
```

That last one is a debugging technique worth remembering: **break when the thing you believe is impossible happens**. If the breakpoint never fires, your belief was right. If it fires, you are standing exactly where the defect is.

A **watchpoint**, or data breakpoint, pauses when a *variable changes* rather than when a line is reached. It answers the other hard question — "what changed this?" — which is otherwise a search through everything that could touch it.

16.4 Debugging Run-Time Errors

A crash gives you a location for free. Run under the debugger and the program stops at the failing line with the call stack intact.

Common causes and what to look for:

Symptom	Likely cause	Check
Crash on `v[k]`	index out of range	`k` and `v.size()` in the Variables panel
Crash on `*ptr`	null or dangling pointer	is `ptr` `nullptr` or nonsense?
Crash on division	integer division by zero	the denominator
Crash deep in library code	bad argument from your code	the call stack — find *your* nearest frame

That last row is worth emphasizing. A crash inside `std::vector`'s internals is almost never a bug in `std::vector`. **Walk down the call stack to the first frame that is your code**, and look at what it passed.

16.5 Debugging Logic Errors

No crash, no location. The technique is bisection.

1. Identify a place where a value is definitely **right** — usually just after input.
2. Identify a place where it is definitely **wrong** — usually the report.
3. Set a breakpoint halfway between, and check.
4. Whichever half contains the transition, repeat there.

Each step halves the search. A fault anywhere in a thousand lines is located in about ten checks.

Section 16.2's discipline applies at every step: **predict the value before you look**. A value that matches your expectation moves the boundary. A value that does not is the answer.

16.6 Assertions and Diagnostic Output

Not every problem needs a debugger.

Assertions

An **assertion** states something you believe must be true. If it is not, the program stops immediately and tells you where:

```
#include <cassert>

double computePercentage(double earned, double possible) {
    assert(possible > 0.0);          // the caller's precondition
    return earned / possible * 100.0;
}
```

Assertions document assumptions **and check them**. Chapter 9 Section 9.8 had you write `@pre possible > 0` in a comment; an assertion is the same statement, enforced.

They are removed when the program is compiled with `NDEBUG` defined, so they cost nothing in a release build. **Assert what must never happen; do not assert on user input** — bad input is expected, and belongs in validation or, from Chapter 24, in an exception.

Diagnostic output

Printing values is legitimate and sometimes faster than a debugger, especially for a fault that appears once in a thousand iterations:

```
std::cout << "[debug] pct=" << pct << " i=" << i
          << " size=" << scale.size() << "\n";
```

Two rules. **Mark it clearly** — a `[debug]` prefix makes it findable. And **remove it before you commit**; Appendix D Section D.11 excludes leftover debugging output for the same reason it excludes commented-out code.

A debugger is better when you need to look around; printing is better when you need to see a pattern over many iterations.

16.7 Writing Test Cases

A **test case** is an input with a known expected result, worked out independently of the program.

That last clause is the whole discipline. If you compute the expected answer *by running the program*, you have tested nothing — you have recorded what it does.

Boundary testing

Bugs live at boundaries. Ordinary values almost never find them.

For a grade cutoff of 90:

Test	Why
89.9	just below
90.0	**exactly at**
90.1	just above
89.95	rounds to 90.0 — Chapter 6's problem

For a roster:

Test	Why
0 students	empty case
1 student	minimum non-empty
many students	ordinary

For points possible:

Test	Why
0	division by zero
positive	ordinary
earned > possible	bonus pushing past 100

Every one of these has already caused a problem in this book. That is not coincidence — it is what boundary means.

A simple test harness

You do not need a framework:

```
int checksRun = 0;
int checksFailed = 0;

void check(bool condition, const std::string& label) {
    ++checksRun;
    if (condition) {
        std::cout << "  PASS  " << label << "\n";
    } else {
        ++checksFailed;
        std::cout << "  FAIL  " << label << "\n";
    }
}
```

Then state each expectation as a line of code:

```
check(letterFor(90.0, scale) == 'A', "90.0 is an A, not a B");
check(letterFor(0.0, scale)  == 'F', "0.0 is an F, no out-of-range read");
```

Two dozen of these run in a second and are worth more than an hour of manual clicking.

Regression tests

A **regression test** is a test written in response to a specific defect, which fails before the fix and passes after.

Its purpose is not to find the bug — the bug is already found. Its purpose is to ensure the bug **cannot return unnoticed**. Defects have a way of coming back when someone later "simplifies" the fix without knowing why it was there.

Every defect you fix should leave a regression test behind. That is step 6 of Section 16.2, and it is the step people skip.

16.8 Documenting Defects

A **defect report** records what was wrong so the work is not repeated. Six fields:

Field	Contents
Symptom	What was observed
Reproduction	The exact input that triggers it
Diagnosis	The actual cause, in the code
Fix	What changed
Verification	How the fix was confirmed
Regression test	The test that would have caught it

An example, from a defect you are about to find:

> **Symptom.** A student with exactly 90.0% receives a B instead of an A. **Reproduction.** One assignment, 9 points earned out of 10, default scale. **Diagnosis.** `letterFor` compares with `>` instead of `>=`, so a percentage exactly at a cutoff falls through to the next tier. **Fix.** Changed `>` to `>=` in `letterFor`. **Verification.** 90.0 now returns A; 89.9 still returns B. **Regression test.** `check(letterFor(90.0, scale) == 'A', "90.0 is an A, not a B");`

Common Debugging Mistakes

Mistake	Why it fails	Instead
Changing code before understanding	Fixes the symptom, hides the cause	Form a hypothesis first
Debugging without `-g`	No names, no line numbers	Rebuild with `-g`
Breaking at the top of `main`	Hundreds of steps to reach the fault	Break where you can observe it
Ignoring the call stack	Missing how you got here	Read it every time you pause
Testing only ordinary values	Boundaries are where bugs are	Test at and beside every boundary
Computing expected results with the program	Tests only what it does, not what is right	Work them out by hand
Fixing without a regression test	The defect returns silently	Write the test
Leaving debug output in	Clutter, and it eventually ships	Remove before committing

---

Design Notes

Predict before you look. A debugger that confirms your expectation moves you forward; one that surprises you has found something.

Break on the impossible. A conditional breakpoint on a condition you believe cannot occur is free if you are right and decisive if you are wrong.

Test at boundaries. Ordinary values are the least informative input you can choose.

Every fix leaves a test behind. Otherwise you are relying on nobody ever touching that line again.

Grade Calculator v2.3 — Defect Hunt

The lab

You will seed **three defects into your own working v2.2**, one at a time, observe each failure, and then restore the correct code. These are the only intentionally broken lines in the book, and you introduce them deliberately so that you know what a correct program looked like ten seconds earlier.

Work on a copy of your v2.2 code in the StudySite editor, and do not commit a seeded defect to GitHub. The three defects are:

1. change a grade-cutoff comparison from `>=` to `>`;
2. add bonus points to *possible* points as well as earned points; and
3. allow the grade-scale loop to read index `size()`.

Each defect is invisible on ordinary input and obvious on boundary input. That is precisely why they are worth finding with a debugger rather than by reading.

Symptoms

Seed a defect, click **Run**, and try these in the Terminal:

1. Enter one assignment worth 10 points, and a student earning **exactly 9** with no bonus. The percentage shows 90.0%. What letter?

2. Enter a student earning **8 points with 2 bonus** on a 10-point assignment. What percentage is reported? Compute it by hand first.
3. Enter a grade scale with **no tier at 0** — say A at 90 and B at 80 only — then a student scoring 50%. What happens?

Your StudySite Lab — Debug and Add Regression Tests

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v2.3
- **Starting point**: The working Chapter 15 v2.2 program.

One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

1. Seed the same three defects described below into a copy of your own working v2.2 code, so you observe each failure before you fix it.
2. Find and fix **Defect 1 (D1)**, an off-by-one loop bound, and **Defect 3 (D3)**, a `>` used where `>=` is required — both in `letterFor`.
3. Find and fix **Defect 2 (D2)**, bonus points wrongly added to points possible as well as points earned, in `percentageOf`.
4. Add a `check`/`runTests` regression-test harness that exercises all three defects plus the zero-points-possible boundary, and wire a new `5) Run tests` menu option to call it.
5. Renumber the menu so `6) Quit` replaces the old `5) Quit`, and update the invalid-choice prompt to `1-6`.
6. Create `defect-reports.md` recording the symptom, reproduction steps, cause, correction, and regression test for each of the three defects.

Nothing from v2.2 is **retired** this chapter, but two things change that are easy to miss. First, the menu grows a line — quitting moves from choice `5` to choice `6`, and `5` now runs the tests — so if you type the menu loop by hand rather than copying it, don't leave a stray `else if (choice == '5') { running = false; }` behind. Second, the reference solution's own header comment says "Run the tests from menu option 6" — that line in the reference file is wrong about its own code. The code beneath it wires `'5'` to `runTests()` and `'6'` to quitting. Write your own header comment to say it correctly: **5 runs the tests, 6 quits.**

Build it: step by step

Each step below shows the actual broken code to type in and watch fail, then the fix. Build with `-g` so the debugger has your names and line numbers, and rebuild after each step — don't wait until the end to test.

Step 1 — Reproduce and fix D1 and D3 in `letterFor` Temporarily change your working `letterFor` to this broken version:

```
char letterFor(double pct, const std::vector<GradeTier>& scale) {
    for (std::size_t i = 0; i <= scale.size(); ++i) {
        if (pct > scale[i].cutoff) { return scale[i].letter; }
    }
    return '?';
}
```

Rebuild, run, and reproduce **D3** first: add one assignment worth 10 points, add a student who earns exactly 9 points with no bonus (90.0%), and print the report. You should see an A; the broken build prints a B. Set a

breakpoint on the `if (pct > scale[i].cutoff)` line, watch `pct` and `scale[i].cutoff` in the Watch panel, and step through with F10. When `i` is 0 (the 90.0-cutoff tier), `pct` and `scale[i].cutoff` are both `90.0` — and `pct > scale[i].cutoff` evaluates `false`, because they are *equal*, not greater. The loop falls through to the B tier instead of stopping here. That confirms D3: the comparison needs `>=`.

Now reproduce **D1** without changing anything back yet: add a student who earns 0 points on that same assignment (0.0%). The default scale's lowest tier is `{0.0, 'F'}`, so the same `>` bug means `pct > scale[i].cutoff` is `false` for that tier too — the loop never returns inside the vector and walks off the end. Move your breakpoint's watch to `i` and `scale.size()` instead, and step through with F10: `i` climbs 0, 1, 2, 3, 4, and the loop condition `i <= scale.size()` still lets it run once more at `i == 5`, which equals `scale.size()`. At that point `scale[i]` reads past the end of the vector — undefined behavior. Depending on what happens to be in memory there, you might see a crash, a garbage letter, or by chance a plausible one; none of those outcomes is correct, and none is guaranteed to repeat.

Fix both defects at once by restoring the range-for loop with `>=`:

```
char letterFor(double pct, const std::vector<GradeTier>& scale) {
    for (const GradeTier& t : scale) { if (pct >= t.cutoff) { return t.letter; } }
    return '?';
}
```

Rebuild and repeat both inputs: 9/10 should now report A, and 0/10 should report F, every time.

Step 2 — Reproduce and fix D2 in `percentageOf` Temporarily change `percentageOf`'s accumulation line so bonus points count toward `possible` as well as `earned`:

```
double percentageOf(const Student& s, const std::vector<Assignment>& as) {
    double earned = 0.0;
    double possible = 0.0;
    for (std::size_t a = 0; a < as.size() && a < s.scores.size(); ++a) {
        earned   += s.scores[a].pointsEarned + s.scores[a].bonusPoints;
        possible += as[a].pointsPossible + s.scores[a].bonusPoints;
    }
    if (possible <= 0.0) { return 0.0; }
    double raw = earned / possible * 100.0;
    return std::round((CAP_AT_100 ? std::min(raw, 100.0) : raw) * 10.0) / 10.0;
}
```

Reproduce: one assignment worth 10 points, a student who earns 8 points with 2 bonus. By hand, 8 + 2 earned out of 10 possible should cap at 100%. The broken build reports 83.3% instead. Set a breakpoint on the `possible += ...` line inside the loop, and watch `possible` and `s.scores[a].bonusPoints`. Step over the line once with F10: `possible` jumps from `0.0` to `12.0`, not `10.0` — the bonus leaked into the denominator. That confirms D2.

Fix it by dropping the bonus term from the `possible` accumulation — bonus only ever helps `earned`:

```
possible += as[a].pointsPossible;
```

Rebuild and repeat 8+2-out-of-10; you should now see exactly 100.0%.

Step 3 — Add the regression-test harness Above `main`, add the `check` helper and a `runTests` function that exercises all three defects plus one extra boundary — an assignment worth zero points, which must not divide by zero:

```
int checksRun = 0;
int checksFailed = 0;

void check(bool condition, const std::string& label) {
```

```
    ++checksRun;
    if (condition) {
        std::cout << "  PASS  " << label << "\n";
    } else {
        ++checksFailed;
        std::cout << "  FAIL  " << label << "\n";
    }
}

void runTests() {
    std::cout << "\n--- Regression tests ---\n";
    std::vector<GradeTier> scale = { {90.0,'A'}, {80.0,'B'}, {70.0,'C'}, {60.0,'D'}, {0.0,'F'} };

    // D3: exactly at a cutoff must take the higher grade.
    check(letterFor(90.0, scale) == 'A', "D3 90.0 is an A, not a B");
    check(letterFor(80.0, scale) == 'B', "D3 80.0 is a B, not a C");
    check(letterFor(60.0, scale) == 'D', "D3 60.0 is a D, not an F");

    // D1: a percentage below every cutoff must not read past the end.
    check(letterFor(0.0, scale)  == 'F', "D1 0.0 is an F, no out-of-range read");
    check(letterFor(-5.0, scale) == '?', "D1 below scale returns '?', no crash");

    // D2: bonus raises earned points only, never points possible.
    std::vector<Assignment> as = { {"HW1", 10.0} };
    Student withBonus;
    withBonus.scores.push_back({8.0, 2.0});     // 8 earned + 2 bonus out of 10
    check(percentageOf(withBonus, as) == 100.0, "D2 8+2 bonus of 10 is 100%, not 83.3%");

    Student noBonus;
    noBonus.scores.push_back({8.0, 0.0});
    check(percentageOf(noBonus, as) == 80.0,    "D2 8 of 10 with no bonus is 80%");

    // Boundary: an assignment worth zero points must not divide by zero.
    std::vector<Assignment> zero = { {"Ungraded", 0.0} };
    Student s2;
    s2.scores.push_back({0.0, 0.0});
    check(percentageOf(s2, zero) == 0.0, "zero points possible does not crash");

    std::cout << "  " << (checksRun - checksFailed) << " of " << checksRun
              << " checks passed.\n\n";
    checksRun = 0;
    checksFailed = 0;
}
```

If you run this with either defect still in place, one or more lines print `FAIL` instead of `PASS` — that is the point of a regression test: it fails before the fix and passes after. Once both fixes from Steps 1 and 2 are in place, every line should print `PASS`.

Step 4 — Rewire the menu: 5 runs tests, 6 quits Change the menu prompt, add a `'5'` branch that calls `runTests()`, move quitting to `'6'`, and update the invalid-input message:

```
std::cout << "1) Add assignment  2) Add student  3) Report  "
             "4) Save  5) Run tests  6) Quit\nChoice: ";
...
} else if (choice == '5') {
    runTests();
} else if (choice == '6') {
    running = false;
} else {
```

```
    std::cout << "  Please enter 1-6.\n\n";
}
```

If you are working from a printed or reference copy of `main.cpp`, note that its own header comment claims "Run the tests from menu option 6." That comment is wrong about its own code. The code beneath it wires `'5'` to `runTests()` and `'6'` to quitting — write your own header comment correctly.

Verification

- Before you apply the fixes, each seeded defect makes at least one `runTests()` check print `FAIL`: D3 fails the exact-cutoff checks, D1 fails (or crashes on) the `0.0` and below-scale checks, and D2 fails the bonus check.
- After both fixes, choosing `5) Run tests` prints `8 of 8 checks passed.` with no `FAIL` lines.
- A student scoring exactly at a cutoff (e.g., 90.0%) receives the higher letter grade, not the lower one.
- A student scoring at or below the lowest cutoff (0.0%) still returns `F`, and a percentage below the whole scale returns `?` — neither reads past the end of the scale vector.
- A bonus that brings a student exactly to an assignment's points possible caps the percentage at 100%, never inflating the denominator.
- Choosing `5` runs the tests and `6` quits; there is no longer a `5) Quit` option.
- The normal Grade Calculator menu (add assignment, add student, report, save) still works.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click each source, header, or documentation file needed for this chapter. Confirm the editor shows the expected file paths before editing.
4. Work through Steps 1–4 above in StudySite's internal editor, in order: seed and fix D1/D3 in `letterFor` first and confirm both test inputs behave correctly; then seed and fix D2 in `percentageOf` and confirm the bonus case; then add the `check`/`runTests` harness; then rewire the menu, since the new `5) Run tests` line depends on `runTests` already existing. Click **Run** after each step.
5. Click **Run**. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted.
6. Fix every compiler error and warning, then complete the verification list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor, including `defect-reports.md`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 16 Grade Calculator v2.3**.

5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- The verification list passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 16 checkpoint, including `defect-reports.md`.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

The expected result

Your v2.3 should produce this from the test menu option:

```
--- Regression tests ---
  PASS  D3 90.0 is an A, not a B
  PASS  D3 80.0 is a B, not a C
  PASS  D3 60.0 is a D, not an F
  PASS  D1 0.0 is an F, no out-of-range read
  PASS  D1 below scale returns '?', no crash
  PASS  D2 8+2 bonus of 10 is 100%, not 83.3%
  PASS  D2 8 of 10 with no bonus is 80%
  PASS  zero points possible does not crash
  8 of 8 checks passed.
```

What to notice about these three defects

All three pass ordinary input. A student with 84 out of 100 and no bonus receives a B from the broken build and from the fixed one. Any amount of casual testing would miss all three.

Two of them produce plausible wrong answers. The bonus defect reports 83.3% instead of 100% — a number that looks like a grade, on a report that looks correct. Nothing announces it. This is the logic error of Chapter 4 Section 4.9.3, and it is why grades computed by software need tests rather than confidence.

One of them is undefined behavior that may not crash. The out-of-range read may return a plausible letter, a nonsense letter, or crash — possibly differently on different days. Chapter 11 Section 11.3 warned that this is the most dangerous thing in that chapter. Here it is.

Additional exercises

4. **Find a defect by bisection.** Have a classmate change one character in your working v2.3 without telling you where. Locate it using Section 16.5's halving technique. Count the steps.
5. **Use a watchpoint.** Set one on `totalEarned` and run a three-assignment entry. How many times does it change? Is that what you expected?
6. **Add an assertion** to `computePercentage` stating its precondition. Deliberately violate it and observe what an assertion failure looks like.
7. **Extend the harness.** Add checks for: an empty roster, a student with no assignments, a bonus pushing past 100 with the cap on, and a percentage of exactly 89.95. All should pass; if any fails, you have found a fourth defect.

Try It Yourself

1. A test harness

```cpp
#include <iostream>
#include <string>

int checksRun = 0;
int checksFailed = 0;

void check(bool condition, const std::string& label) {
    ++checksRun;
    if (condition) {
        std::cout << "  PASS  " << label << "\n";
    } else {
        ++checksFailed;
        std::cout << "  FAIL  " << label << "\n";
    }
}

int main() {
    check(2 + 2 == 4, "arithmetic works");
    check(2 + 2 == 5, "this one should fail");
    std::cout << "  " << (checksRun - checksFailed) << " of "
              << checksRun << " checks passed.\n";
    return 0;
}
```

Expected output:

```
  PASS  arithmetic works
  FAIL  this one should fail
  1 of 2 checks passed.
```

Try: Make `main` return 1 when any check fails. Why would that matter to an automated build?

2. Boundary testing a grade scale

```cpp
#include <iostream>
#include <string>
#include <vector>

struct GradeTier { double cutoff = 0.0; char letter = 'F'; };

char letterFor(double pct, const std::vector<GradeTier>& scale) {
    for (const GradeTier& t : scale) {
        if (pct >= t.cutoff) { return t.letter; }
    }
    return '?';
}

int main() {
    std::vector<GradeTier> scale = {{90.0,'A'},{80.0,'B'},{70.0,'C'},{60.0,'D'},{0.0,'F'}};
    double tests[7] = {90.1, 90.0, 89.9, 80.0, 60.0, 0.0, -1.0};
    for (double t : tests) {
        std::cout << t << " -> " << letterFor(t, scale) << "\n";
```

```
    }
    return 0;
}
```

Expected output:

```
90.1 -> A
90 -> A
89.9 -> B
80 -> B
60 -> D
0 -> F
-1 -> ?
```

Try: Change `>=` to `>` and rerun. Four lines change — every test that sits *exactly* on a cutoff, including 0.0, which now matches no tier at all and returns `?`. Identify all four and explain why the other three are unaffected.

3. Assertions

```
#include <cassert>
#include <iostream>

double computePercentage(double earned, double possible) {
    assert(possible > 0.0);
    return earned / possible * 100.0;
}

int main() {
    std::cout << computePercentage(84.0, 100.0) << "\n";
    return 0;
}
```

Expected output:

```
84
```

Try: Call it with `possible` of 0. Observe the assertion message — it names the file, line, and failed condition. Then rebuild with `-DNDEBUG` and call it again. What happens now, and what does that tell you about relying on assertions for user input?

4. Read a call stack

Build this with `-g`, set a breakpoint on the marked line, run it, and read the call stack.

```
#include <iostream>

double level3(double x) {
    double result = x * 2.0;     // <-- breakpoint here
    return result;
}
double level2(double x) { return level3(x + 1.0); }
double level1(double x) { return level2(x * 10.0); }

int main() {
    std::cout << level1(4.0) << "\n";
    return 0;
}
```

Expected output: `82`

Try: When paused, how many frames are on the stack? Click each and note the value of `x`. Can you explain how 4.0 became 41.0 by reading only the stack?

5. Bisection in practice

Take a working program of at least 60 lines. Have someone change one character — a `<` to `<=`, a `+` to `-`.

Find it by bisection. Record how many breakpoints you needed. Then find it by reading the code, and compare the time.

6. Write a defect report

Using the six fields from Section 16.8, write a full report for the 89.95 problem from Chapter 6 Section 6.12 — including the regression test that would have caught it.

7. Design test cases from a specification

A student passes if their course percentage is 60% or higher. Bonus points count toward the percentage. A student with no assignments has no grade.

List every test case you would write. Aim for at least eight, and justify each in a few words. At minimum you should cover: exactly 60, just below 60, no assignments, one assignment worth zero points, and a bonus that pushes a failing score to passing.

Summary

- Debugging is a **systematic process**: reproduce, narrow, hypothesize, test, fix, verify with a regression test. **Predict before you look**.
- A **symbolic debugger** pauses a running program and shows it in terms of your own names. Build with **`-g`**.
- A **breakpoint** pauses at a line. **Step Over** executes without entering calls; **Step Into** enters them; **Step Out** finishes the current function.
- The **Variables** panel and **Watch** expressions show current values. Watching the exact condition you suspect is often faster than reasoning about it.
- The **call stack** shows how execution reached here. Each **frame** holds one call's locals. When a crash is deep in library code, walk down to your nearest frame.
- **Conditional breakpoints** fire only when a condition holds — including conditions you believe are impossible.
- Find logic errors by **bisection** between a known-good point and a known-bad one.
- **Assertions** state and check what must never happen. Do not assert on user input.
- A **test case** has an expected result **worked out independently of the program**.
- **Bugs live at boundaries** — zero, empty, exactly at a cutoff, one past the end.
- A **regression test** fails before a fix and passes after, ensuring the defect cannot return silently. **Every fix leaves one behind**.

Key Terms

assertion — a checked statement of something that must be true, removed in release builds.

breakpoint — an instruction to pause execution at a particular line.

call stack — the sequence of function calls leading to the current point.

conditional breakpoint — a breakpoint that pauses only when a condition holds.

defect report — a record of a fault's symptom, cause, fix, and verification.

regression test — a test written for a specific defect, failing before the fix and passing after.

stack frame — one function call's entry on the call stack, holding its local variables.

step into — execute the current line, entering any function it calls.

step out — finish the current function and return to its caller.

step over — execute the current line without entering functions it calls.

symbolic debugger — a debugger presenting a program in terms of its source names.

test case — an input paired with an independently determined expected result.

watch expression — an expression the debugger evaluates and displays as you step.

watchpoint — a breakpoint that fires when a variable's value changes.

Next: Chapter 17 puts your roster in order. Sorting and searching are the first real algorithms in this book, and you will implement them, trace them by hand, and measure the difference between a linear scan and a binary search on your own data. Grade Calculator v2.4.

Chapter 17 — Searching and Sorting

Learning Objectives

When you finish this chapter you will be able to:

- Implement and trace a linear search. *(SLO 2.5)*
- Implement and trace a binary search, and state its precondition. *(SLO 2.5)*
- Implement selection sort and insertion sort, and trace each pass. *(SLO 2.5)*
- Describe merge sort and the divide-and-conquer idea. *(SLO 2.5)*
- Compare algorithms by counting operations rather than by timing alone. *(SLO 2.5)*
- Sort by different criteria using a comparison function. *(SLO 2.5, 2.8)*
- Use `std::sort` and `std::find` from the standard library. *(SLO 2.5)*
- Build Grade Calculator v2.4 — a sortable, searchable roster.

17.1 Why Algorithm Choice Matters

Item five on your Chapter 13 backlog: sort and search the roster. *Perfective* maintenance.

Two things make this chapter different from what came before. These are the first named **algorithms** in the book — general procedures with known properties, not code written for one situation. And it is the first time the *choice* between two correct programs matters, because both give the right answer and one takes far longer.

With 30 students, any approach works. With 30,000, the difference is the difference between instant and unusable. Knowing which is which is the skill.

17.2 Linear Search

Look at each element in turn until you find what you want:

```
int linearSearchById(const std::vector<Student>& roster, int id, int& comparisons) {
    comparisons = 0;
    for (std::size_t i = 0; i < roster.size(); ++i) {
        ++comparisons;
        if (roster[i].id == id) {
            return static_cast<int>(i);
        }
    }
    return -1;      // not found
}
```

Returning −1 for "not found" is a common convention. It works only because a valid index is never negative — and Chapter 24 replaces it with something better, because a caller who forgets to check −1 will use it as an index.

Linear search works on any data, in any order. That is its whole appeal, and it is not a small one.

Case	Comparisons
Found first	1
Found in the middle	about $n/2$
Found last, or absent	n

Doubling the roster doubles the work. That relationship — work proportional to n — is what makes it a **linear** search.

17.3 Binary Search

If the data is **sorted**, you can do far better. Look at the middle. If it is what you want, stop. If your target is smaller, everything to the right is eliminated; if larger, everything to the left is. Repeat on what remains.

```
int binarySearchById(const std::vector<Student>& roster, int id, int& comparisons) {
    comparisons = 0;
    int low = 0;
    int high = static_cast<int>(roster.size()) - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        ++comparisons;
        if (roster[mid].id == id)     { return mid; }
        else if (roster[mid].id < id) { low = mid + 1; }
        else                          { high = mid - 1; }
    }
    return -1;
}
```

The precondition

Binary search requires sorted data. On unsorted data it does not merely run slowly — it gives wrong answers, reporting that a present element is absent. That is a logic error of the Chapter 4 Section 4.9.3 kind: silent and confident.

This is a `@pre` in the sense of Chapter 9 Section 9.8, and it should be documented:

```
/**
 * @pre roster is sorted by id, ascending. Results are meaningless otherwise.
 */
```

Why `low + (high - low) / 2`

The obvious midpoint is `(low + high) / 2`. It is subtly wrong: for very large values, `low + high` can overflow — Chapter 3 Section 3.4's wraparound, in a real algorithm. The form above cannot overflow and is worth writing by habit.

The trade-off

Each comparison eliminates half of what remains, so the work grows with the *logarithm* of *n*:

Roster size	Linear (worst)	Binary (worst)
10	10	4
100	100	7
1,000	1,000	10
1,000,000	1,000,000	20

A million students, twenty comparisons. That is not a small improvement; it is a different category of thing.

But look at a small case measured honestly. Searching for all five keys in a five-element vector:

```
linear search, all 5 keys: 15 comparisons
binary search, all 5 keys: 11 comparisons
```

Barely better — and that ignores the cost of sorting in the first place. **Binary search only pays when the data is already sorted, or when you will search many times.** Sorting once to search once is worse than not sorting at all.

17.4 Selection Sort

Repeatedly find the smallest remaining element and move it into place.

```
void selectionSort(std::vector<int>& a) {
    for (std::size_t i = 0; i + 1 < a.size(); ++i) {
        std::size_t best = i;
        for (std::size_t j = i + 1; j < a.size(); ++j) {
            if (a[j] < a[best]) { best = j; }
        }
        if (best != i) { std::swap(a[i], a[best]); }
    }
}
```

Traced on `64 25 12 22 11`:

Pass	Result	What happened
start	`64 25 12 22 11`	
1	`11 25 12 22 64`	smallest (11) swapped to front
2	`11 12 25 22 64`	smallest of the rest (12) into place
3	`11 12 22 25 64`	22 into place
4	`11 12 22 25 64`	already correct

```
start:  64 25 12 22 11
pass 1: 11 25 12 22 64
pass 2: 11 12 25 22 64
pass 3: 11 12 22 25 64
pass 4: 11 12 22 25 64
```

The left portion is sorted and final; the right is unexamined. That invariant is what makes it easy to trace by hand, which is why this book starts here.

Selection sort makes $n(n-1)/2$ comparisons **regardless of the input**. Already-sorted data takes just as long as random data. It does, however, make at most $n-1$ swaps, which matters when moving elements is expensive.

17.5 Insertion Sort

Take each element and insert it into its correct position among those already handled — the way most people sort a hand of cards.

```
void insertionSort(std::vector<int>& a) {
    for (std::size_t i = 1; i < a.size(); ++i) {
        int key = a[i];
        std::size_t j = i;
        while (j > 0 && a[j - 1] > key) {
            a[j] = a[j - 1];
            --j;
        }
        a[j] = key;
    }
}
```

Same data:

Pass	Result	What happened
start	64 25 12 22 11	
1	25 64 12 22 11	25 inserted before 64
2	12 25 64 22 11	12 inserted at the front
3	12 22 25 64 11	22 inserted after 12
4	11 12 22 25 64	11 inserted at the front

```
start:  64 25 12 22 11
pass 1: 25 64 12 22 11
pass 2: 12 25 64 22 11
pass 3: 12 22 25 64 11
pass 4: 11 12 22 25 64
```

Here the left portion is sorted **among itself** but not final — later elements still move into it.

Insertion sort's advantage is that it **adapts to its input**. On nearly-sorted data the inner `while` barely runs, and the whole thing approaches n comparisons. On reversed data it is as bad as selection sort. Real data is often nearly sorted, which makes this a better default than its reputation suggests.

17.6 Bubble Sort and Its Reputation

Bubble sort repeatedly walks the data swapping adjacent out-of-order pairs, so large values "bubble" to the end.

It is famous because it is easy to explain, and it is not recommended. It does the same number of comparisons as selection sort while making far more swaps. Insertion sort is simpler to write correctly and faster on nearly every input.

It is mentioned here so you recognize the name, and so you know that meeting it in older code is not evidence that it was a good choice.

17.7 Merge Sort and Divide and Conquer

Selection and insertion sort both do work proportional to n^2. Doubling the data quadruples the time. For 30 students that is nothing; for 30,000 it is fatal.

Merge sort does better with a different strategy — **divide and conquer**:

1. If the data has 0 or 1 elements, it is already sorted. *(base case)*
2. Otherwise, split it in half.
3. Sort each half — by the same method, recursively.
4. **Merge** the two sorted halves into one sorted whole.

Step 4 is where the work happens, and it is easy: with two sorted lists, repeatedly take the smaller of the two front elements.

This is the recursion from Chapter 10 Section 10.4 doing something a loop cannot express as naturally. The splitting is self-similar all the way down, and the base case is reached because each split is strictly smaller.

Merge sort does work proportional to $n \log n$:

Size	n^2 (roughly)	$n \log n$ (roughly)
100	10,000	700
1,000	1,000,000	10,000
10,000	100,000,000	130,000

At 10,000 elements the difference is a factor of several hundred.

Merge sort's cost is memory: merging needs somewhere to put the result. Selection and insertion sort work in place.

17.8 Comparing Algorithms

The honest way to compare algorithms is to **count operations**, not to time them. Timing depends on the machine, the compiler, and what else is running; a comparison count is a property of the algorithm.

Algorithm	Comparisons (worst)	Best case	Extra memory	Adapts to input
Linear search	n	1	none	—
Binary search	$\log_2 n$	1	none	requires sorted data
Selection sort	$n^2/2$	$n^2/2$	none	**no**
Insertion sort	$n^2/2$	n	none	**yes**
Merge sort	$n \log n$	$n \log n$	n	no

Two habits follow.

Instrument your code to count. The `comparisons` reference parameter in Sections 17.2 and 17.3 exists for this. A measured count settles an argument that estimation cannot.

Measure on your actual data. Merge sort beats insertion sort asymptotically and loses on small or nearly-sorted inputs, which is why real library implementations switch between strategies by size.

17.9 Sorting by Different Criteria

A roster might be sorted by name, by percentage, or by ID. Writing three sorts would be three chances to get it wrong.

Instead, pass the comparison in. A **function pointer** holds the address of a function:

```
typedef bool (*StudentComparer)(const Student&, const Student&,
                                const std::vector<Assignment>&);

bool byName(const Student& a, const Student& b, const std::vector<Assignment>&) {
    return a.name < b.name;
}

bool byId(const Student& a, const Student& b, const std::vector<Assignment>&) {
    return a.id < b.id;
}
```

Then one sort serves all orderings:

```
void selectionSort(std::vector<Student>& roster,
                   const std::vector<Assignment>& as,
                   StudentComparer comesFirst) {
    for (std::size_t i = 0; i + 1 < roster.size(); ++i) {
        std::size_t best = i;
        for (std::size_t j = i + 1; j < roster.size(); ++j) {
            if (comesFirst(roster[j], roster[best], as)) { best = j; }
        }
        if (best != i) { std::swap(roster[i], roster[best]); }
    }
}
```

The algorithm and the ordering are now separate concerns. Adding a new ordering means writing one small function; the sort is untouched.

Note that `byName` and `byId` ignore their third parameter — it is unnamed for exactly that reason, which is how you tell the compiler an unused parameter is deliberate. They must accept it because they all need one signature.

> This is worth remembering. In Chapter 20 you will meet the same problem in a larger form: several ways to compute a grade, and code that should not have to know which. The solution there is a class hierarchy, and function pointers are its ancestor.

17.10 Library Sorting and Searching

You will rarely write a sort in production. `<algorithm>` provides tested implementations:

```
#include <algorithm>

std::sort(v.begin(), v.end());                                  // ascending
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; }); // descending
```

```
ascending:  11 12 22 25 64
descending: 64 25 22 12 11
```

`v.begin()` and `v.end()` are **iterators** marking a range; Chapter 23 covers them. The `[](int a, int b) { return a > b; }` is a **lambda** — an unnamed function written where it is used. Also Chapter 23; for now, read it as a comparison function without a name.

Also available:

Call	Does
`std::find(v.begin(), v.end(), x)`	linear search
`std::binary_search(v.begin(), v.end(), x)`	binary search on sorted data
`std::min_element` / `std::max_element`	smallest / largest
`std::reverse(v.begin(), v.end())`	reverse in place

`std::sort` is typically a hybrid, switching strategies by size, and is faster than anything you would write by hand.

So why implement sorts yourself? Because the reasoning transfers. You will choose between algorithms, recognize when a precondition is violated, and estimate whether an approach will scale — and none of that comes from calling a library function.

Common Errors

What you see or observe	Cause	Fix
Binary search reports absent items as present, or vice versa	Data not sorted	Sort first; document the precondition
Sort runs forever	Comparison not strict — returns true for equal items	Use `<`, never `<=`
The last element is never sorted	Loop bound off by one	Trace the first and last passes
Crash during sort	Index out of range in the inner loop	Check `j < size`
Sorted names but wrong grades	Sorting parallel arrays separately	Sort records — Chapter 14
`std::sort` will not compile	No `<` for your type	Supply a comparison function
Binary search never terminates	`low`/`high` not moving	`mid + 1` and `mid - 1`, not `mid`
Results differ between runs	Comparison depends on something unstable	Make the ordering total and deterministic

---

Design Notes

State and document preconditions. Binary search on unsorted data is a silent wrong answer, not a slow one.

Count operations rather than guessing. Instrument the code; the number ends the argument.

Separate the algorithm from the ordering. One sort plus several comparison functions beats several sorts.

Sort records, never parallel arrays. Chapter 14's lesson, and sorting is exactly where it bites.

Use the library. Implement sorts to learn; call `std::sort` to ship.

Grade Calculator v2.4 — Rankings

What v2.4 does

Everything v2.3 did, plus: sort the roster by name, percentage, or ID with the ordering chosen at run time; find a student by ID; and report the comparison counts for linear and binary search on the same data.

The searches, instrumented

Both take a `comparisons` reference so their cost can be measured rather than assumed:

```
int linearSearchById(const std::vector<Student>& roster, int id, int& comparisons);
int binarySearchById(const std::vector<Student>& roster, int id, int& comparisons);
```

The menu handler runs both and reports both counts:

```
int linearCost = 0;
int found = linearSearchById(book.roster, id, linearCost);

// Binary search needs sorted data, so sort a copy first.
std::vector<Student> sorted = book.roster;
selectionSort(sorted, book.assignments, byId);
int binaryCost = 0;
binarySearchById(sorted, id, binaryCost);

std::cout << "  Comparisons - linear: " << linearCost
          << ", binary: " << binaryCost << "\n";
```

Note what that comparison honestly shows. Sorting a copy to run one binary search is far more work than the linear scan it replaced. On a roster of three students, both report one comparison. The measurement demonstrates the technique and simultaneously demonstrates that it is not worth it at this size — which is the more useful lesson.

Expected output

Three students entered as Zoe, Ada, Max; sorted by name; then a search for ID 1002:

```
1002  Ada                    90.0%   A
1003  Max                    70.0%   C
1001  Zoe                    50.0%   F

  Found: Ada (90.0%)
  Comparisons - linear: 1, binary: 1
```

The roster is alphabetical, and every student's ID, name, and scores moved together — Chapter 14's records earning their keep.

The grade scale as a search problem

Look again at `letterFor`. It walks a list of tiers from the top until it finds one the percentage reaches. **That is a linear search.**

For five tiers it is exactly right — a binary search would be more code, more chances to be wrong, and no faster. For a fifty-tier scale it would be worth reconsidering.

This is the chapter's judgment in miniature: **the better algorithm is the one that fits the size of your problem.** Recognizing that `letterFor` is a linear search is what lets you make that call deliberately rather than by default.

Your StudySite Lab — Sort and Search the Roster

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v2.4
- **Starting point**: The working Chapter 16 v2.3 program.

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

1. Add a `StudentComparer` function-pointer type and three comparator functions — `byName`, `byId`, and `byPercentageDescending` — so one sort routine can serve every ordering the menu offers.
2. Add `selectionSort`, a selection-sort routine that sorts a `std::vector<Student>` in place using whichever comparator is passed to it.
3. Add `linearSearchById` and `binarySearchById`. Both search by student ID and both count their own comparisons through an `int&` output parameter, so the two techniques can be measured, not just trusted.
4. Add two new menu options and renumber `Quit`:
   - `6) Sort roster` — asks which key to sort by (name, percentage, or ID) and calls `selectionSort` with the matching comparator.
   - `7) Find by ID` — runs `linearSearchById` on the roster as it stands, then runs `binarySearchById` on a **sorted copy** of the roster (never the live one), and prints both comparison counts.
   - `Quit` moves from choice `6` to choice `8`.

Nothing from v2.3 is retired, renamed, or changed behaviorally — this checkpoint is purely additive. The only things a returning user will notice are the two-line menu (it no longer fits on one line) and that `Quit` is now `8` instead of `6`.

Build it: step by step

Each step below shows the actual code for that piece. Type it in as you go — don't wait until the end to test. All of it goes after `runTests()` and before `main()`.

Step 1 — The comparator type and three comparator functions A comparison function pointer lets one sort routine serve every ordering instead of writing `sortByName`, `sortById`, and `sortByPercentage` as three separate copies of the same loop.

```
// A comparison function pointer lets one sort routine serve every ordering.
typedef bool (*StudentComparer)(const Student&, const Student&,
                                const std::vector<Assignment>&);

bool byName(const Student& a, const Student& b, const std::vector<Assignment>&) {
    return a.name < b.name;
}

bool byId(const Student& a, const Student& b, const std::vector<Assignment>&) {
    return a.id < b.id;
}

bool byPercentageDescending(const Student& a, const Student& b,
                            const std::vector<Assignment>& as) {
    return percentageOf(a, as) > percentageOf(b, as);
}
```

Each comparator answers one question: "does `a` come before `b`?" `byName` and `byId` ignore the assignment list entirely (hence the unnamed third parameter) because a name or an ID needs no calculation. `byPercentageDescending` uses it, because a percentage has to be computed from scores — and it uses `>` rather than `<` so that the *highest* percentage sorts first, matching how a ranked list is normally read.

```
/**
 * Selection sort: repeatedly find the smallest remaining element and swap it
 * into place. Simple to trace by hand, which is why Chapter 17 starts here.
 * Comparisons: n(n-1)/2 regardless of the input order.
 */
void selectionSort(std::vector<Student>& roster,
                   const std::vector<Assignment>& as,
                   StudentComparer comesFirst) {
    for (std::size_t i = 0; i + 1 < roster.size(); ++i) {
        std::size_t best = i;
        for (std::size_t j = i + 1; j < roster.size(); ++j) {
            if (comesFirst(roster[j], roster[best], as)) { best = j; }
        }
        if (best != i) { std::swap(roster[i], roster[best]); }
    }
}
```

Step 2 — `selectionSort` The outer loop fixes one position at a time; the inner loop scans everything still unsorted to find what belongs there. That inner scan runs for every position regardless of how the data already looked, so selection sort always does the same amount of work: `n(n-1)/2` comparisons for `n` students, best case and worst case alike — $O(n^2)$. It is never the fastest sort, but it is the easiest to trace by hand, which is exactly why it is the one to learn first. Passing in `comesFirst` is what lets the same function sort by name one moment and by percentage the next — the sort logic never changes, only which comparator it's handed.

```
/** Linear search: works on any order, examines up to n elements. */
int linearSearchById(const std::vector<Student>& roster, int id, int& comparisons) {
    comparisons = 0;
    for (std::size_t i = 0; i < roster.size(); ++i) {
        ++comparisons;
        if (roster[i].id == id) { return static_cast<int>(i); }
    }
    return -1;
}

/**
 * Binary search: requires the roster to be sorted by ID first.
 * That precondition is the whole trade-off - it is far faster, but only
 * if someone has paid the cost of sorting.
 */
int binarySearchById(const std::vector<Student>& roster, int id, int& comparisons) {
    comparisons = 0;
    int low = 0;
    int high = static_cast<int>(roster.size()) - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        ++comparisons;
        if (roster[mid].id == id)      { return mid; }
        else if (roster[mid].id < id)  { low = mid + 1; }
        else                           { high = mid - 1; }
    }
    return -1;
```

```
}
```

Step 3 — `linearSearchById` and `binarySearchById` Linear search makes no assumption about ordering, so it can run on the roster exactly as the user left it — worst case, it examines every student. Binary search is faster (worst case around $\log_2 n$ comparisons instead of n), but only because it throws away half the remaining range on every step, and that trick only works if the range is sorted by the field being searched. Hand it unsorted data and it will confidently return the wrong answer, or none at all. That precondition — sorted first, searched second — is the entire trade-off this step is teaching, not just "a second way to search."

Step 4 — Two new menu options, and `Quit` moves to `8` The menu prompt grows to two lines and gains two entries before `Quit`:

```
std::cout << "1) Add assignment  2) Add student  3) Report  4) Save\n"
             "5) Run tests  6) Sort roster  7) Find by ID  8) Quit\nChoice: ";
```

Insert these two branches immediately before the old `Quit` branch, and renumber that branch from `'6'` to `'8'`:

```
} else if (choice == '6') {
    std::cout << "  Sort by: 1) name  2) percentage  3) ID\n  Choice: ";
    std::string s;
    std::getline(std::cin, s);
    StudentComparer cmp = byName;
    if (!s.empty() && s[0] == '2')      { cmp = byPercentageDescending; }
    else if (!s.empty() && s[0] == '3') { cmp = byId; }
    selectionSort(book.roster, book.assignments, cmp);
    std::cout << "  Roster sorted.\n\n";
} else if (choice == '7') {
    std::cout << "  Student ID: ";
    std::string s;
    std::getline(std::cin, s);
    int id = 0;
    try { id = std::stoi(s); }
    catch (...) { std::cout << "  Not a valid ID.\n\n"; continue; }

    int linearCost = 0;
    int found = linearSearchById(book.roster, id, linearCost);

    // Binary search needs sorted data, so sort a copy first.
    std::vector<Student> sorted = book.roster;
    selectionSort(sorted, book.assignments, byId);
    int binaryCost = 0;
    binarySearchById(sorted, id, binaryCost);

    if (found >= 0) {
        std::cout << "  Found: " << book.roster[found].name << " ("
                  << percentageOf(book.roster[found], book.assignments) << "%)\n";
    } else {
        std::cout << "  No student with ID " << id << ".\n";
    }
    std::cout << "  Comparisons - linear: " << linearCost
              << ", binary: " << binaryCost << "\n\n";
} else if (choice == '8') {
    running = false;
```

Look closely at option 7: it builds `std::vector<Student> sorted = book.roster;` and sorts *that copy* by ID, leaving `book.roster` exactly as the user left it. This is deliberate. If binary search sorted the live roster as a side effect, "Find by ID" would silently reorder the class list every time it ran — a search should never rearrange the thing it searches. Copying first means the roster the user sees in the next report is unaffected by whatever

order they last searched in, and it lets the linear search (on the original order) and the binary search (on the sorted copy) be timed fairly against each other on the same underlying data. Finally, update the fallback error message from `"  Please enter 1-6.\n\n"` to `"  Please enter 1-8.\n\n"` to match the new range of choices.

Verification

- Sorting by name, by percentage, and by ID each produce a correctly ordered roster — check at least five students per order.
- Searching for an ID that exists on the roster finds the right student by both linear and binary search; searching for an ID that does not exist reports "not found" from both.
- The printed comparison counts are sane: binary search's count should generally be lower than linear search's on a roster of any real size, and binary search must never report *more* comparisons than linear search's worst case for the same target.
- After a sort-then-search round trip, `3) Report` still shows every student's correct percentage and letter grade — sorting rearranges the roster, it must never corrupt or drop a student's data.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click each source, header, or documentation file needed for this chapter. Confirm the editor shows the expected file paths before editing.
4. Work through Steps 1–4 above in StudySite's internal editor, in order. Add Steps 1–3 first and click **Run** — the file should still compile even though `main` hasn't changed yet. Then apply Step 4 and click **Run** again.
5. Click **Run**. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted.
6. Fix every compiler error and warning, then complete the verification list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work:** StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 17 Grade Calculator v2.4**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- The verification list passes.

- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 17 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Selection sort, pass by pass

```cpp
#include <iostream>
#include <string>
#include <vector>

void show(const std::vector<int>& v, const std::string& label) {
    std::cout << label;
    for (int x : v) { std::cout << x << " "; }
    std::cout << "\n";
}

int main() {
    std::vector<int> a = {64, 25, 12, 22, 11};
    show(a, "start:  ");
    for (std::size_t i = 0; i + 1 < a.size(); ++i) {
        std::size_t best = i;
        for (std::size_t j = i + 1; j < a.size(); ++j) {
            if (a[j] < a[best]) { best = j; }
        }
        if (best != i) { std::swap(a[i], a[best]); }
        show(a, "pass " + std::to_string(i + 1) + ": ");
    }
    return 0;
}
```

Expected output:

```
start:  64 25 12 22 11
pass 1: 11 25 12 22 64
pass 2: 11 12 25 22 64
pass 3: 11 12 22 25 64
pass 4: 11 12 22 25 64
```

Try: Run it on already-sorted data. How many passes? How many comparisons? What does that say about selection sort?

2. Insertion sort, pass by pass

```cpp
#include <iostream>
#include <string>
#include <vector>

void show(const std::vector<int>& v, const std::string& label) {
    std::cout << label;
    for (int x : v) { std::cout << x << " "; }
    std::cout << "\n";
}

int main() {
    std::vector<int> b = {64, 25, 12, 22, 11};
```

```
    show(b, "start:  ");
    for (std::size_t i = 1; i < b.size(); ++i) {
        int key = b[i];
        std::size_t j = i;
        while (j > 0 && b[j - 1] > key) { b[j] = b[j - 1]; --j; }
        b[j] = key;
        show(b, "pass " + std::to_string(i) + ": ");
    }
    return 0;
}
```

Expected output:

```
start:  64 25 12 22 11
pass 1: 25 64 12 22 11
pass 2: 12 25 64 22 11
pass 3: 12 22 25 64 11
pass 4: 11 12 22 25 64
```

Try: Run it on `{11, 12, 22, 25, 64}`. Add a counter to the inner `while`. How many times does it run now, and why is that insertion sort's advantage?

3. Counting comparisons

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> sorted = {11, 12, 22, 25, 64};
    int lin = 0;
    int bin = 0;

    for (int x : sorted) {
        for (std::size_t k = 0; k < sorted.size(); ++k) {
            ++lin;
            if (sorted[k] == x) { break; }
        }
    }
    for (int x : sorted) {
        int lo = 0;
        int hi = 4;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            ++bin;
            if (sorted[mid] == x)      { break; }
            else if (sorted[mid] < x)  { lo = mid + 1; }
            else                       { hi = mid - 1; }
        }
    }
    std::cout << "linear: " << lin << " comparisons\n";
    std::cout << "binary: " << bin << " comparisons\n";
    return 0;
}
```

Expected output:

```
linear: 15 comparisons
binary: 11 comparisons
```

Try: Extend to 100 sorted values. Now what are the counts? At roughly what size does binary search become

clearly worthwhile?

4. Binary search on unsorted data

```cpp
#include <iostream>
#include <vector>

bool binarySearch(const std::vector<int>& v, int target) {
    int lo = 0;
    int hi = static_cast<int>(v.size()) - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (v[mid] == target)     { return true; }
        else if (v[mid] < target) { lo = mid + 1; }
        else                      { hi = mid - 1; }
    }
    return false;
}

int main() {
    std::vector<int> unsorted = {64, 25, 12, 22, 11};
    std::cout << std::boolalpha;
    std::cout << "is 11 present? " << binarySearch(unsorted, 11) << "\n";
    std::cout << "(11 is at index 4)\n";
    return 0;
}
```

Expected output:

```
is 11 present? false
(11 is at index 4)
```

Try: Test every value in the vector. How many are found? This is a **wrong answer**, not a slow one — write one sentence on why that is worse.

5. Sorting by different criteria

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

struct Student { std::string name; int id = 0; double pct = 0.0; };

int main() {
    std::vector<Student> roster = {
        {"Zoe", 1001, 50.0}, {"Ada", 1002, 90.0}, {"Max", 1003, 70.0}
    };

    std::sort(roster.begin(), roster.end(),
              [](const Student& a, const Student& b) { return a.name < b.name; });
    for (const Student& s : roster) { std::cout << s.name << " "; }
    std::cout << "\n";

    std::sort(roster.begin(), roster.end(),
              [](const Student& a, const Student& b) { return a.pct > b.pct; });
    for (const Student& s : roster) { std::cout << s.name << " "; }
    std::cout << "\n";
```

```
    return 0;
}
```

Expected output:

```
Ada Max Zoe
Ada Max Zoe
```

Try: The two orderings coincide for this data. Change one student's percentage so they differ, and confirm. Then sort by ID descending.

6. Trace by hand first

Before running anything, trace **selection sort** on `{5, 2, 9, 1, 7}`, writing the vector after each pass. Then trace **insertion sort** on the same data.

Which reaches a fully sorted state sooner? Which does less work overall? Now verify with code.

7. Reason about algorithms

- Why does binary search require sorted data, and what happens if it is not?
- Why is `low + (high - low) / 2` preferred to `(low + high) / 2`?
- Selection sort makes the same number of comparisons on sorted and reversed input. Why?
- Insertion sort does not. Why?
- You must search a 50-element roster once. Sort then binary search, or just scan? Justify it.
- You must search the same roster 10,000 times. Same question, different answer — why?

Summary

- **Linear search** examines elements in order and works on any data. Work grows with n.
- **Binary search** halves the remaining range each step, and work grows with $\log n$ — but it **requires sorted data**, and on unsorted data it gives silent wrong answers.
- Use `low + (high - low) / 2` for the midpoint to avoid overflow.
- **Selection sort** repeatedly moves the smallest remaining element into place. Its cost is the same for every input.
- **Insertion sort** inserts each element into the sorted portion. It **adapts** — nearly-sorted data is nearly free.
- **Bubble sort** is famous, not recommended.
- **Merge sort** divides, sorts each half recursively, and merges. Work grows with $n \log n$, at the cost of extra memory.
- **Compare algorithms by counting operations**, not by timing. Instrument your code.
- Pass a **comparison function** so one sort serves many orderings, separating the algorithm from the ordering.
- `std::sort`, `std::find`, and `std::binary_search` are tested and fast. Implement sorts to learn; call the library to ship.
- **The better algorithm is the one that fits the size of your problem.** For a five-tier grade scale, a linear scan is right.

Key Terms

binary search — a search that halves the remaining range each step; requires sorted data.

comparison function — a function defining an ordering, passed to a sorting routine.

divide and conquer — solving a problem by splitting it, solving the parts, and combining.

function pointer — a variable holding the address of a function.

insertion sort — a sort that inserts each element into the sorted portion; adapts to input.

linear search — a search examining elements one at a time in order.

merge — combining two sorted sequences into one sorted sequence.

merge sort — a divide-and-conquer sort with $n \log n$ comparisons.

precondition — a requirement that must hold before an algorithm is used.

selection sort — a sort that repeatedly moves the smallest remaining element into place.

stability — whether a sort preserves the relative order of equal elements.

Next: Chapter 18 begins the object-oriented material in earnest. Your structs gain behavior and become classes, and a `GradeScale` that validates itself in its constructor makes an invalid grading scale impossible to construct — closing a category of defect you have been checking for by hand since Chapter 11. Grade Calculator v2.5.

Chapter 18 — Classes and Objects

Learning Objectives

When you finish this chapter you will be able to:

- Explain the difference between a class and an object. *(SLO 2.2)*
- Declare a class with private data and public member functions. *(SLO 2.2)*
- Explain encapsulation and why data members are private. *(SLO 2.2, 2.3)*
- Write constructors, including a default and a parameterized one. *(SLO 2.2)*
- Write accessors and mutators, and mark non-modifying functions `const`. *(SLO 2.2)*
- State a class invariant and enforce it in the constructor. *(SLO 2.2, 2.8)*
- Identify candidate classes from a requirements statement. *(SLO 2.2)*
- Build Grade Calculator v2.5, in which an invalid grade scale becomes impossible to construct.

18.1 From Struct to Class

Chapter 14 gave you records. Chapter 14 Section 14.8 also named what they do not do:

- **Nothing protects the members**. Any code can write `s.id = -5;`.
- **The related functions live outside**, among every other function in the file.

A **class** fixes both. This is item six on your Chapter 13 backlog — *preventive* maintenance, making future change cheaper.

Figure 18.1 — A struct compared with a class holding the same data.

Description of Figure 18.1. The left panel shows `struct Student` with three public members — `name`, `id`, `scores` — and, drawn separately outside the struct in a dashed box, the free functions `total` and `percentage`. A note records that any code can write `s.id = -5;`.

The right panel shows `class Student` with the same information, arranged differently: a `public:` section listing `addAssignment`, `totalEarned`, and `percentage`, and a `private:` section holding `name_`, `id_`, and `work_`, all inside one boundary. A note records that outside code can only call the functions.

The data did not change. What can reach it did.

18.2 Objects, State, and Behavior

An **object** bundles **state** — the data it holds — with **behavior** — the operations on that data.

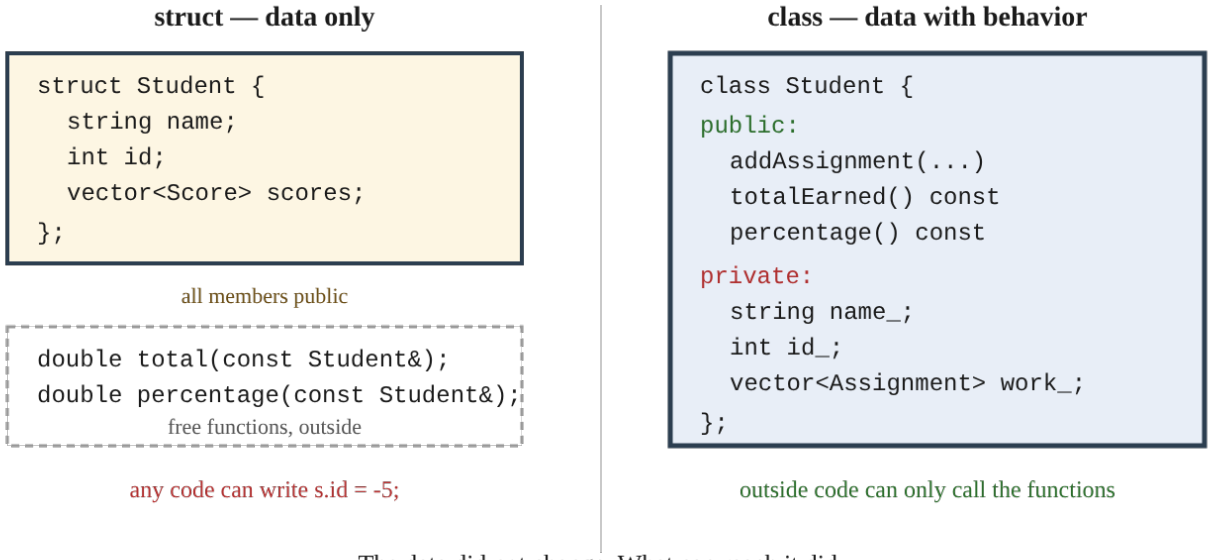


Figure 14: Two panels: on the left a struct Student with three public members and separate free functions outside it; on the right a class Student with the same members marked private and the member functions enclosed inside the boundary.

A **class** describes what objects of a kind look like and can do. An **object** is one actual instance:

```
Student ada("Ada Lovelace", 1001);      // one object
Student grace("Grace Hopper", 1002);    // another, independent
```

A class is a blueprint; objects are the buildings. Each object has its own copy of the data.

You have used objects since Chapter 12. `std::vector<double> scores;` creates an object; `scores.push_back(9.0)` calls a member function on it; the vector's internal storage is state you never see. Everything in this chapter is how that was built.

18.3 Declaring a Class

```
class Student {
public:
    Student() = default;
    Student(const std::string& name, int id);

    const std::string& name() const { return name_; }
    int id() const { return id_; }

    void addAssignment(const Assignment& a);
    double totalEarned() const;
    double percentage() const;

private:
    std::string name_;
    int id_ = 0;
```

```
    std::vector<Assignment> assignments_;
};
```

Three things to note.

Access specifiers. `public:` members are reachable by any code. `private:` members are reachable only from inside the class. Everything after a specifier has that access until the next one.

Ordering. Appendix D Section D.7 requires `public` first, then `protected`, then `private` — because the interface is what a reader wants and the implementation is what they can skip.

Trailing underscores. `name_`, `id_` — Appendix D Section D.2's convention, so you can tell a member from a local at every point of use, and so a constructor parameter can share its member's name without ambiguity.

> `struct` and `class` are the same construct in C++, differing only in default access: struct members are public, class members are private. Everything here could be written with `struct`. The convention this book follows is `struct` for plain data with no invariants, `class` when there is behavior or something to protect.

18.4 Member Functions

A **member function** is declared inside the class and operates on the object it is called on:

```
double Student::totalEarned() const {
    double sum = 0.0;
    for (const Assignment& a : assignments_) {
        sum += a.totalEarned();
    }
    return sum;
}
```

`Student::` says which class this belongs to. Inside, `assignments_` refers to **this object's** member — no parameter needed, because the object is implicit.

Compare with the free function from Chapter 14:

```
double total(const Student& s);      // Chapter 14: takes the student
double Student::totalEarned() const; // Chapter 18: is part of the student
```

Called as `ada.totalEarned()`, which reads as asking the object for something rather than performing an operation on it.

`const` member functions

A member function that does not modify the object is marked `const`:

```
double totalEarned() const;    // does not change the object
void addAssignment(const Assignment& a);   // does
```

This matters more than it looks. **A `const` object can only have its `const` member functions called**. Since Chapter 10 you have been passing things as `const Student&`, so without `const` member functions, a `const Student` would be nearly useless.

Appendix D Section D.5 asks you to do this from the start: retrofitting `const` onto a finished class is tedious, and until it is done, `const` references are unusable.

18.5 Encapsulation

Encapsulation is bundling data with the functions that operate on it, and restricting access to the data.

The word people reach for is "hiding," which undersells it. The point is not secrecy. **The point is that a class can guarantee things about itself that a struct cannot.**

With a struct:

```
Student s;
s.id = -5;                    // nonsense, and nothing stops it
s.scores.clear();             // now scores and assignments disagree
```

With a class, those members are private. The only way to change a `Student` is through its member functions, and those can check.

Two practical consequences:

You can change the implementation freely. If `Student` stores a cached total instead of recomputing it, no calling code changes — the interface is the same. With public members, every user of `scores` is a user of your implementation.

Bugs have a small number of suspects. When a `Student` holds a wrong value, the only code that could have caused it is `Student`'s own member functions. That is a handful of functions instead of the whole program — the same argument Chapter 10 Section 10.1 made against globals, applied at a smaller scale.

18.6 Constructors

A **constructor** runs when an object is created, and its job is to leave the object in a valid state.

```
class Student {
public:
    Student() = default;                                  // default constructor
    Student(const std::string& name, int id);             // parameterized
    // ...
};

Student::Student(const std::string& name, int id)
    : name_(name), id_(id) {}
```

The `: name_(name), id_(id)` is an **initializer list**, and it runs before the constructor body. Prefer it to assignment inside the body — it initializes members directly rather than default-constructing them and then overwriting.

`= default` asks the compiler to generate the obvious default constructor. Combined with member defaults declared in the class, that is usually all you need:

```
private:
    std::string name_;
    int id_ = 0;
    std::vector<Assignment> assignments_;
```

`explicit`

A single-argument constructor should usually be `explicit`:

```
explicit GradeScale(const std::vector<Tier>& requested);
```

Without it, the compiler will silently convert a `std::vector<Tier>` into a `GradeScale` anywhere one is expected — including places you did not intend. Appendix D Section D.7 requires `explicit` unless the implicit conversion is genuinely wanted.

Destructors

A **destructor** runs when an object is destroyed:

```
~Student();
```

For a class holding only `std::string` and `std::vector`, you do not need one — those clean up after themselves. Destructors become essential in Chapter 22, when a class owns memory directly.

18.7 Accessors and Mutators

An **accessor** reads state; a **mutator** changes it:

```
const std::string& name() const { return name_; }     // accessor
void setName(const std::string& n) { name_ = n; }     // mutator
```

A caution worth taking seriously: **an accessor and mutator for every member is a struct with extra steps.**

```
// This class protects nothing.
class Student {
public:
    int getId() const { return id_; }
    void setId(int id) { id_ = id; }      // still allows id = -5
private:
    int id_ = 0;
};
```

Ask what the class is *for*. A `Student` needs `addAssignment` and `percentage` — operations meaningful in the problem. It does not need `setScores`, because nothing in the problem replaces a student's entire score list.

Provide the operations the problem needs, not one pair per member. That distinction is what separates object-oriented design from struct-with-ceremony.

18.8 Class Invariants

An **invariant** is something that is always true of an object, from construction until destruction.

This is the most valuable idea in the chapter.

Since Chapter 11, your grade scale has needed three properties: cutoffs strictly descending, no negative cutoffs, and a bottom tier at 0 so every percentage maps to a letter. You have been checking these by hand in `readGradeScale`, and every future path that builds a scale would need the same checks.

State it once, as an invariant, and enforce it in the constructor:

```
/**
 * An ordered set of grade cutoffs.
 * INVARIANT: cutoffs strictly descend and the lowest tier is 0, so every
 * percentage from 0 upward maps to exactly one letter. The constructor
 * establishes this, and no member function can break it.
 */
class GradeScale {
public:
    struct Tier { double cutoff = 0.0; char letter = 'F'; };

    GradeScale() {
        tiers_ = { {90.0,'A'}, {80.0,'B'}, {70.0,'C'}, {60.0,'D'}, {0.0,'F'} };
    }

    explicit GradeScale(const std::vector<Tier>& requested) {
        for (const Tier& t : requested) {
            if (t.cutoff < 0.0) { continue; }
            if (!tiers_.empty() && t.cutoff >= tiers_.back().cutoff) { continue; }
            tiers_.push_back(t);
        }
        if (tiers_.empty() || tiers_.back().cutoff > 0.0) {
            tiers_.push_back({0.0, 'F'});
        }
    }

    char letterFor(double percentage) const {
        for (const Tier& t : tiers_) {
            if (percentage >= t.cutoff) { return t.letter; }
        }
        return tiers_.back().letter;
    }

private:
    std::vector<Tier> tiers_;
};
```

Feed it deliberate nonsense — out of order, negative, no floor tier:

```
GradeScale bad({{90.0,'A'}, {95.0,'X'}, {-5.0,'Q'}, {85.0,'B'}});
```

```
repaired tiers=3
  96 -> A
  90 -> A
  86 -> B
  10 -> F
```

The `X` tier was rejected because 95 is not below 90. The `Q` tier was rejected as negative. A floor tier was added. **What survived is a valid scale**, and there was never a moment when the object was invalid.

What the invariant buys

Look at `letterFor` again. It has **no error handling**. No check that the scale is non-empty, no `'?'` return for a percentage below every tier, no defensive anything.

It does not need any, because the constructor guarantees the scale is usable. Compare with Chapter 11's version, which returned `'?'` when nothing matched — a value every caller then had to handle.

An invariant established in the constructor is a fact the rest of your code may rely on without checking. That is the entire return on encapsulation, and it is why the members must be private: if a caller could write

`tiers_` directly, the guarantee would be worthless.

Repair or reject?

This version *repairs* bad input. That is one policy. The alternative is to refuse — to make constructing an invalid `GradeScale` impossible rather than silently corrected.

Repair is friendlier and hides the fact that the user's input was wrong. Rejection is honest and requires a way to report the failure, which you do not have until Chapter 24.

Chapter 24 switches this class to rejection, throwing `InvalidScaleError`. Notice that the *invariant* does not change — only what happens when someone violates it. That is a sign the invariant was the right thing to state.

18.9 Identifying Classes from Requirements

Chapter 14 Section 14.7 found records by looking for **nouns**. Classes are found the same way, with an added question.

For each noun, ask: **does it have behavior, or rules about what values are valid?**

Noun	Behavior or rules?	Verdict
Assignment	computes its own total and ratio	class
Student	accumulates work, computes a percentage	class
Grade scale	must be ordered and complete	class — the invariant demands it
Tier	a cutoff and a letter, nothing more	**struct**, nested inside `GradeScale`
Score	two numbers	struct — or fold into `Assignment`

`GradeScale::Tier` staying a struct is deliberate. It has no behavior and no invariant of its own; it exists only as part of a scale. Making it a class would add ceremony and protect nothing. **Not everything should be a class.**

Note also that `Tier` is declared *inside* `GradeScale`. A tier has no meaning apart from a scale, and nesting says so — Appendix D Section D.4's guidance about small helper types.

Common Errors and Warnings

What you see	Cause	Fix
`error: 'name_' is private within this context`	Outside code touching a private member	Add an accessor, or a real operation
`error: passing 'const Student' as 'this' discards qualifiers`	Non-`const` function called on a `const` object	Mark the function `const`
`error: no matching function for call to 'Student::Student()'`	No default constructor, but one is needed	Add `Student() = default;`
`error: expected ';' after class definition`	Missing semicolon after `}`	Add `;`
Members hold garbage	Constructor did not initialize them	Use an initializer list, or member defaults
A conversion happens unexpectedly	Single-argument constructor not `explicit`	Mark it `explicit`
`error: 'class Student' has no member named 'Name'`	Wrong capitalization	C++ is case-sensitive
An object reaches an invalid state	The invariant is not enforced	Enforce it in the constructor; keep members private

---

Design Notes

Data members are always private. A public member is a promise you cannot take back.

Mark every non-modifying member function `const`, from the start.

State the invariant in a comment above the class, and enforce it in the constructor. Then trust it everywhere else.

Provide the operations the problem needs, not a getter and setter per member.

Not everything is a class. Plain data with no rules stays a struct.

Grade Calculator v2.5 — Student and GradeScale Classes

What v2.5 does

The v2.4 data model rebuilt with classes: `Assignment`, `Student`, and `GradeScale` become classes with private data, and the grading math is unchanged. Because each `Student` now owns its own `Assignment` objects instead of sharing one course-wide list, assignments are entered per student (when you add the student) rather than in a separate "add an assignment" step. See "A restructure worth noting" below.

v2.5 also starts from a deliberately smaller core — Add student, Report, Custom scale, and Quit — so the class design gets your full attention. Saving and loading, the regression tests, and sort and find are set aside for this checkpoint; sorting returns in Chapter 19, lookup by ID in Chapter 23, and saving and loading in Chapter 24.

A restructure worth noting

Each `Student` now owns its own `Assignment` objects, where previously assignments were a separate list and students held parallel scores:

```
class Assignment {
public:
    Assignment(const std::string& name, double earned, double possible, double bonus = 0.0);
    const std::string& name() const { return name_; }
    double totalEarned() const;      // earned + bonus
    double ratio() const;            // fraction of possible, or 0
private:
    std::string name_;
    double pointsEarned_   = 0.0;
    double pointsPossible_ = 0.0;
    double bonusPoints_    = 0.0;
};
```

This removes the correspondence problem entirely. A student's assignments cannot be out of step with a separate list, because there is no separate list. It also makes `Student::percentage()` self-contained:

```
double Student::percentage() const {
    double possible = totalPossible();
    if (possible <= 0.0) { return 0.0; }
    double raw = totalEarned() / possible * 100.0;
    double reported = CAP_AT_100 ? std::min(raw, 100.0) : raw;
```

```
    return std::round(reported * 10.0) / 10.0;
}
```

No parameters. The student has everything it needs.

GradeScale enforces its invariant

The class from Section 18.8, used directly:

```
GradeScale scale;                   // default: A/B/C/D/F, valid by construction
scale = GradeScale(userTiers);      // custom: repaired to valid, or built valid
```

`readGradeScale`'s hand-written validation from Chapter 11 is **gone**. It moved into the constructor, where it cannot be bypassed.

Expected output

Add Ada with `HW1` 9/10 bonus 1 and `Midterm` 84/100 bonus 5, view the report, then set a custom scale of A 93, B 85, and a deliberately invalid tier `ZZZ` at 99:

```
  Scale: A >= 90   B >= 80   C >= 70   D >= 60   F >= 0
1001  Ada                     90.0%   A
  Scale accepted.
  Scale: A >= 93   B >= 85   F >= 0
1001  Ada                     90.0%   B
```

Three things happened. The `ZZZ` tier at 99 was **silently rejected**, because 99 is not below 93. A floor tier `F` at 0 was **added**, because the user's scale did not reach 0. And the same 90.0% became a **B** under the stricter scale — the data changed, not the code.

What to notice

`letterFor` has no error handling and needs none. The invariant guarantees a usable scale.

The validation code exists once. In v1.3 it lived in `readGradeScale`. Any second way of building a scale — from a file, from a default, from a copy — would have needed its own copy of the checks. Now every path goes through the constructor.

`std::min` needs `<algorithm>`, and `std::round` needs `<cmath>`. Appendix D Section D.4: include what you use.

The class is longer than the struct. That is the cost, and it is real. What you get for it is that a whole category of wrong state can no longer be expressed.

Your StudySite Lab — Introduce Classes and Invariants

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v2.5
- **Starting point**: Your committed Chapter 17 v2.4 program (`main.cpp`).

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

In this lab you rebuild the Grade Calculator's data around **classes** instead of structs. By the end you will have:

1. An `Assignment` class that holds one graded item — its name, points earned, points possible, and bonus — behind `private` data and `const` accessors. The old `Score` struct is folded into it.
2. A `GradeScale` class whose constructor **enforces an invariant**: cutoffs strictly descend and the lowest tier is 0. Whatever tiers a user types, the constructor repairs them into a usable scale, so no other code ever has to check.
3. A `Student` class that **owns its own assignments** and computes its own percentage with a `percentage()` member that needs no parameters.
4. A smaller, cleaner `main` with four menu options: **Add student**, **Report**, **Custom scale**, and **Quit**.

What is different about v2.5

v2.5 is a *rebuild*, not an add-on, so a few things from Chapter 17 change on purpose. Read this before you start so nothing surprises you.

- **Assignments are entered per student**. In v2.4 you added assignments once for the whole course, and each student held a list of scores that lined up with that list by position. In v2.5 each `Student` owns its own `Assignment` objects, so you enter a student's assignments right after the student's name. The separate *Add assignment* menu option goes away.
- **The `Gradebook` struct goes away**. `main` holds a `GradeScale`, a `std::vector<Student>`, and a next-ID counter directly. (Chapter 19 brings back a `Gradebook` — as a real class.)
- **v2.5 starts from a smaller core**. Saving and loading (Chapter 15), the regression tests (Chapter 16), and sort and find (Chapter 17) are set aside in this checkpoint so you can concentrate on classes. They are not lost: your Chapter 17 commit keeps them in your repository's history. Sorting returns in Chapter 19, Find by ID in Chapter 23, and saving and loading in Chapter 24 — each rebuilt on top of the classes you write today.
- **The grading math does not change**. The same scores still produce the same percentage and letter grade.

Files in this checkpoint

File	What you do to it
`main.cpp`	Edit. Everything in this lab happens in this one file.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

  Heads-up: Steps 2 through 7 take the program apart before Step 8 puts `main` back together, so the program will **not** compile again until the end of Step 8. That is expected. Do not try to fix compiler errors in the middle — finish the steps in order, then Run.

Build it: step by step

Step 1 — Update the header comment Every version of the program opens with a comment block that says what changed. Update it so the file describes v2.5.

At the very top of `main.cpp`, find the header comment:

```
// ============================================================================
//  Grade Calculator  v2.4                                        Chapter 17
// ----------------------------------------------------------------------------
//  Change  : Sorting and searching. The roster can be sorted by name, by
//            percentage, or by ID, with the comparison chosen at run time.
//            Search by ID is implemented both linearly and with binary search
//            so the two can be compared on the same data.
//  Format  : documented in gradebook-format.md (section-tagged CSV, chosen so
//            new record types can be added later without breaking old files).
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc
// ============================================================================
```

Replace it with:

```
// ============================================================================
//  Grade Calculator  v2.5                                        Chapter 18
// ----------------------------------------------------------------------------
//  Change  : Structs become classes with private data and behavior.
//            GradeScale validates its own tiers in the constructor, so an
//            invalid scale can no longer be constructed at all. The loose
//            validation functions of Chapter 11 are gone.
//            Each Student now owns its own Assignment records.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc
// ============================================================================
```

Why: the header is the first thing a reader (including future you) sees. It should describe the code that is actually in the file.

Step 2 — Update the `#include` list and constants v2.5 no longer reads or writes files, so it no longer needs `<fstream>`, `<sstream>`, or the `DEFAULT_FILE` constant. It does need `<algorithm>`, because `Student::percentage()` will call `std::min`.

Just below the header comment, find the `#include` lines and the two constants:

```
#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>
#include <string>
#include <vector>

const bool CAP_AT_100 = true;
const std::string DEFAULT_FILE = "gradebook.csv";
```

Replace it with:

```
#include <algorithm>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>
```

```
#include <vector>

const bool CAP_AT_100 = true;
```

Why: include what you use, and only what you use (Appendix D, Section D.4). `CAP_AT_100` stays — the course still caps a reported percentage at 100.

Step 3 — Replace the structs with an `Assignment` class Chapter 17's data lived in five structs: `Assignment` (a name and points possible), `Score` (points earned and bonus), `Student` (which held a list of `Scores`), `GradeTier`, and `Gradebook`. In v2.5 an assignment belongs to exactly one student, so it can hold *everything* about itself — and `Score` is no longer needed.

Find the first struct, just below `CAP_AT_100`:

```
struct Assignment { std::string name; double pointsPossible = 0.0; };
```

Delete that line and everything below it (the `Score`, `Student`, `GradeTier`, and `Gradebook` structs), down to and including this line:

```
};
```

In the space where the five structs were, enter the `Assignment` class:

```
// ----------------------------------------------------------------------------
//  Assignment - one graded item belonging to one student.
// ----------------------------------------------------------------------------
class Assignment {
public:
    Assignment() = default;
    Assignment(const std::string& name, double earned, double possible, double bonus = 0.0)
        : name_(name), pointsEarned_(earned), pointsPossible_(possible), bonusPoints_(bonus) {}

    const std::string& name() const { return name_; }
    double pointsEarned() const     { return pointsEarned_; }
    double pointsPossible() const   { return pointsPossible_; }
    double bonusPoints() const      { return bonusPoints_; }

    /** Points credited to the student: raw score plus any bonus. */
    double totalEarned() const { return pointsEarned_ + bonusPoints_; }

    /** Fraction of possible points earned, or 0 for an ungraded item. */
    double ratio() const {
        return pointsPossible_ > 0.0 ? totalEarned() / pointsPossible_ : 0.0;
    }

private:
    std::string name_;
    double pointsEarned_   = 0.0;
    double pointsPossible_ = 0.0;
    double bonusPoints_    = 0.0;
};
```

Look at what changed:

- The data members are **`private`** and end in an underscore (`name_`, `pointsEarned_`) — the naming convention from Appendix D that marks a member variable. Code outside the class can no longer change them directly.
- The **constructor** uses a member initializer list to set all four values at once. `bonus` has a default of `0.0`, so a caller can leave it out.

- Every accessor ends in **`const`**, which promises the compiler it does not change the object.
- `totalEarned()` and `ratio()` are **behavior that lives with the data**. Bonus is added to points earned, never to points possible — the D2 rule from Chapter 16, now written once in the class that owns the numbers.

Step 4 — Add the `GradeScale` class and its invariant In v2.4 the scale was a plain `std::vector<GradeTier>`, and nothing stopped the program from holding a broken one (tiers out of order, no 0 tier). The `GradeScale` class makes a broken scale impossible to build.

Find the comment line that now sits just below the `Assignment` class:

```
// ---------- input helpers ----------
```

Directly above it, add:

```
// ---------------------------------------------------------------------------
//  GradeScale - an ordered set of cutoffs and letters.
//  INVARIANT: cutoffs strictly descend and the lowest tier is 0.
//  The constructor enforces this, so no other code needs to check it.
// ---------------------------------------------------------------------------
class GradeScale {
public:
    struct Tier {
        double cutoff = 0.0;
        char   letter = 'F';
    };

    /** Builds the default A/B/C/D/F scale. */
    GradeScale() {
        tiers_ = { {90.0,'A'}, {80.0,'B'}, {70.0,'C'}, {60.0,'D'}, {0.0,'F'} };
    }

    /**
     * Builds a scale from user-supplied tiers, repairing what it can.
     * Out-of-order tiers are dropped; a missing floor tier is added.
     * The result is always a usable scale - that is the invariant.
     */
    explicit GradeScale(const std::vector<Tier>& requested) {
        for (const Tier& t : requested) {
            if (t.cutoff < 0.0) { continue; }
            if (!tiers_.empty() && t.cutoff >= tiers_.back().cutoff) { continue; }
            tiers_.push_back(t);
        }
        if (tiers_.empty() || tiers_.back().cutoff > 0.0) {
            tiers_.push_back({0.0, 'F'});
        }
    }

    /** Every percentage from 0 upward maps to a letter. Cannot fail. */
    char letterFor(double percentage) const {
        for (const Tier& t : tiers_) {
            if (percentage >= t.cutoff) { return t.letter; }
        }
        return tiers_.back().letter;
    }

    std::size_t tierCount() const { return tiers_.size(); }
    const std::vector<Tier>& tiers() const { return tiers_; }

    void describe() const {
        std::cout << "  Scale: ";
```

```
        for (const Tier& t : tiers_) {
            std::cout << t.letter << " >= " << std::fixed << std::setprecision(0)
                      << t.cutoff << "   ";
        }
        std::cout << "\n";
    }

private:
    std::vector<Tier> tiers_;
};
```

Study the second constructor — it is the heart of this chapter:

- `GradeScale()` with no arguments builds the default A/B/C/D/F scale.
- `explicit GradeScale(const std::vector<Tier>& requested)` walks the tiers the user asked for and **keeps only the ones that preserve the invariant**: a negative cutoff is skipped, and a cutoff that is not lower than the previous kept tier is skipped. If the result does not reach 0, an `F` tier at 0 is added.
- Because *every* `GradeScale` passes through one of these constructors, the invariant is always true. That is why `letterFor` has no error handling: it cannot be handed a bad scale.
- `Tier` is a small struct **nested inside** the class, so its full name is `GradeScale::Tier`. It replaces the old `GradeTier` struct.
- `describe()` prints the scale on one line, so `main` does not need to know how a scale is stored.

Step 5 — Add the `Student` class A `Student` now owns a `std::vector<Assignment>`. Everything needed to compute a grade is inside the object.

Find the same comment line again — it is now just below the `GradeScale` class:

```
// ---------- input helpers ----------
```

Directly above it, add:

```
// ---------------------------------------------------------------------------
//  Student - owns a name, an ID, and its own assignments.
// ---------------------------------------------------------------------------
class Student {
public:
    Student() = default;
    Student(const std::string& name, int id) : name_(name), id_(id) {}

    const std::string& name() const { return name_; }
    int id() const { return id_; }
    const std::vector<Assignment>& assignments() const { return assignments_; }

    void addAssignment(const Assignment& a) { assignments_.push_back(a); }

    double totalEarned() const {
        double sum = 0.0;
        for (const Assignment& a : assignments_) { sum += a.totalEarned(); }
        return sum;
    }

    double totalPossible() const {
        double sum = 0.0;
        for (const Assignment& a : assignments_) { sum += a.pointsPossible(); }
        return sum;
    }
```

```
    /** Points-based course percentage, capped and rounded by course policy. */
    double percentage() const {
        double possible = totalPossible();
        if (possible <= 0.0) { return 0.0; }
        double raw = totalEarned() / possible * 100.0;
        double reported = CAP_AT_100 ? std::min(raw, 100.0) : raw;
        return std::round(reported * 10.0) / 10.0;
    }

private:
    std::string name_;
    int id_ = 0;
    std::vector<Assignment> assignments_;
};
```

Compare `percentage()` with Chapter 17's `percentageOf(student, assignments)`. The old function needed the course-wide assignment list passed in, because a student only held scores. The new member function takes **no parameters** — the student has everything it needs. It still applies the same course policy: skip a zero total to avoid dividing by zero, cap at 100 when `CAP_AT_100` is `true`, and round to one decimal place.

`addAssignment` is the **only** way to put work into a student, so there is exactly one place to look if a student's work ever looks wrong.

Step 6 — Update the input-helpers banner Your `readNonNegative` and `readLine` functions do not change. Only the comment above them changes, to match the banner style of the classes above it.

Find the comment line:

```
// ---------- input helpers ----------
```

Replace it with:

```
// ------------------------------------------------------------------------
//  Input helpers
// ------------------------------------------------------------------------
```

Step 7 — Remove the code that v2.5 sets aside Everything between `readLine` and `main` belonged to the struct-based design: the CSV helpers and `saveGradebook`/`loadGradebook` (Chapter 15), the free functions `letterFor`, `percentageOf`, and `printReport`, the regression tests (Chapter 16), and the sort and search functions (Chapter 17). The classes now do the grading work, and the other features are set aside for this checkpoint (see *What is different about v2.5*).

Find the line just below the end of `readLine`:

```
// ---------- CSV helpers ----------
```

Delete that line and everything below it (all the CSV, persistence, grading, test, sorting, and searching code), stopping just **above** this line, which you keep:

```
int main() {
```

When you are done, the closing `}` of `readLine` should be followed by one blank line and then `int main() {`.

Why: in Chapter 17 this code worked on `Student::scores` and the shared `book.assignments` list. Neither exists any more, so keeping it would only produce compiler errors. Your Chapter 17 commit still has every line of it if you want to look back.

Step 8 — Rewrite `main` around the new classes `main` gets simpler: three local variables replace the `Gradebook` struct, and the menu has four options.

Find the start of `main`:

```
int main() {
```

Delete that line and everything below it, all the way to the final closing `}` at the very end of the file.

In its place, enter the new `main`:

```
int main() {
    std::cout << "=== GRADE CALCULATOR v2.5 (classes) ===\n\n";

    GradeScale scale;   // default scale; invariant guaranteed by construction
    scale.describe();

    std::vector<Student> roster;
    int nextId = 1001;

    bool running = true;
    while (running) {
        std::cout << "\n1) Add student  2) Report  3) Custom scale  4) Quit\nChoice: ";
        std::string line;
        if (!std::getline(std::cin, line)) { break; }
        char choice = line.empty() ? '?' : line[0];

        if (choice == '1') {
            std::string name = readLine("  Student name: ");
            if (name.empty()) { continue; }
            Student s(name, nextId++);
            while (true) {
                std::string an = readLine("    Assignment name (or 'done'): ");
                if (an == "done" || an.empty()) { break; }
                double earned   = readNonNegative("      Points earned  : ");
                double possible = readNonNegative("      Points possible: ");
                double bonus    = readNonNegative("      Bonus points   : ");
                s.addAssignment(Assignment(an, earned, possible, bonus));
            }
            roster.push_back(s);
            std::cout << "  Added " << s.name() << " as ID " << s.id() << ".\n";

        } else if (choice == '2') {
            std::cout << "\n====================================\n  CLASS REPORT\n";
            std::cout << "====================================\n";
            std::cout << std::fixed << std::setprecision(1);
            if (roster.empty()) { std::cout << "No students yet.\n"; continue; }
            for (const Student& s : roster) {
                std::cout << std::left << std::setw(6) << s.id()
                          << std::setw(20) << s.name()
                          << std::right << std::setw(8) << s.percentage() << "%   "
                          << scale.letterFor(s.percentage()) << "\n";
            }

        } else if (choice == '3') {
            std::vector<GradeScale::Tier> requested;
            std::cout << "  Enter tiers highest first, 'done' to finish.\n";
            while (true) {
                std::string letterText = readLine("    Letter (or 'done'): ");
                if (letterText == "done" || letterText.empty()) { break; }
                double cutoff = readNonNegative("      Minimum percentage: ");
```

```
                requested.push_back({cutoff, letterText[0]});
            }
            // Any bad input is repaired by the constructor, not by this code.
            scale = GradeScale(requested);
            std::cout << "  Scale accepted.\n";
            scale.describe();

        } else if (choice == '4') {
            running = false;
        } else {
            std::cout << "  Please enter 1-4.\n";
        }
    }
    std::cout << "\nGoodbye.\n";
    return 0;
}
```

Walk through what each part does:

- **Startup**. `GradeScale scale;` builds the default scale — valid by construction. `scale.describe()` prints it so the user can see it.
- **Option 1 — Add student**. After reading the name, `Student s(name, nextId++)` creates the student with the next ID. The inner loop reads one assignment at a time (name, points earned, points possible, bonus) until the user types `done` or presses Enter on an empty name, adding each one with `s.addAssignment(...)`. Only then is the finished student added to the roster.
- **Option 2 — Report**. Each row calls `s.id()`, `s.name()`, and `s.percentage()` — accessors, not data members — and `scale.letterFor(...)` for the letter.
- **Option 3 — Custom scale**. The loop collects tiers into a `std::vector<GradeScale::Tier>` without checking them, then `scale = GradeScale(requested);` builds a new scale. **The constructor does the repairing** — this code contains no validation at all.
- **Option 4 — Quit** ends the loop. Because there is no file to save in v2.5, the program simply says goodbye.

▸ **Run checkpoint**: Click **Run**. The program should compile with no errors and no warnings and show the v2.5 banner, the default scale, and the four-option menu. If you see errors, the message names a line — compare that area with the code boxes above.

Check your work

Run the program and type the inputs below in the Terminal. Your output should match.

1. Choose `1` (Add student). Enter the name `Ada`. Add two assignments: `HW1` with points earned `9`, points possible `10`, bonus `1`; and `Midterm` with `84`, `100`, bonus `5`. Type `done`. The program should print `Added Ada as ID 1001.`
2. Choose `2` (Report). You should see:

   ```
   ====================================
     CLASS REPORT
   ====================================
   1001  Ada                    90.0%   A
   ```

   Check it by hand: (9 + 1 + 84 + 5) ÷ (10 + 100) = 99 ÷ 110 = 90.0%.
3. Choose `3` (Custom scale). Enter `A` with `93`, `B` with `85`, and then `ZZZ` with `99`. Type `done`. You should see:

   ```
     Scale accepted.
     Scale: A >= 93   B >= 85   F >= 0
   ```

The `ZZZ` tier was dropped because 99 is not below 85, and an `F` tier at 0 was added because your scale did not reach 0. The constructor repaired the input — no error, no crash.

4. Choose `2` again. Ada's 90.0% is now a `B` under the stricter scale. The data did not change; only the scale did.
5. Choose `4` (Quit). The program prints `Goodbye.` and ends normally.

Then confirm:

- ☐ The program compiles with no errors **and no warnings**.
- ☐ `Assignment`, `GradeScale`, and `Student` have only `private` data members; `main` reaches their data only through member functions.
- ☐ Every accessor that does not change its object is marked `const`.
- ☐ The structs `Score`, `GradeTier`, and `Gradebook` no longer appear anywhere in `main.cpp`.
- ☐ The menu shows exactly four options: Add student, Report, Custom scale, Quit.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click `main.cpp` to open it. Confirm the editor shows the expected file paths before editing.
4. Work through Steps 1–8 above in StudySite's internal editor, in order. Click **Run** at the ▸ Run checkpoint at the end of Step 8.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — for this chapter that is just `main.cpp`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 18 Grade Calculator v2.5**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 18 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. A first class

```cpp
#include <iostream>
#include <string>

class Assignment {
public:
    Assignment(const std::string& name, double earned, double possible)
        : name_(name), earned_(earned), possible_(possible) {}

    const std::string& name() const { return name_; }
    double ratio() const { return possible_ > 0.0 ? earned_ / possible_ : 0.0; }

private:
    std::string name_;
    double earned_ = 0.0;
    double possible_ = 0.0;
};

int main() {
    Assignment hw("Homework 1", 9.0, 10.0);
    std::cout << hw.name() << " ratio " << hw.ratio() << "\n";
    return 0;
}
```

Expected output:

```
Homework 1 ratio 0.9
```

Try: Add `hw.earned_ = 100.0;` to `main` and read the error. That error is encapsulation working.

2. `const` member functions

```cpp
#include <iostream>
#include <string>

class Student {
public:
    explicit Student(const std::string& name) : name_(name) {}
    const std::string& name() const { return name_; }
    void rename(const std::string& n) { name_ = n; }
private:
    std::string name_;
};

void show(const Student& s) {
    std::cout << s.name() << "\n";        // works: name() is const
}

int main() {
    Student ada("Ada");
    show(ada);
    ada.rename("Ada Lovelace");
    show(ada);
```

```
    return 0;
}
```

Expected output:

```
Ada
Ada Lovelace
```

Try: Add `s.rename("X");` inside `show` and read the error. Then remove `const` from `name()` and see `show` stop compiling. Explain the connection.

3. An invariant enforced

```
#include <iostream>
#include <vector>

class GradeScale {
public:
    struct Tier { double cutoff = 0.0; char letter = 'F'; };

    GradeScale() { tiers_ = {{90.0,'A'},{80.0,'B'},{70.0,'C'},{60.0,'D'},{0.0,'F'}}; }

    explicit GradeScale(const std::vector<Tier>& requested) {
        for (const Tier& t : requested) {
            if (t.cutoff < 0.0) { continue; }
            if (!tiers_.empty() && t.cutoff >= tiers_.back().cutoff) { continue; }
            tiers_.push_back(t);
        }
        if (tiers_.empty() || tiers_.back().cutoff > 0.0) { tiers_.push_back({0.0,'F'}); }
    }

    char letterFor(double pct) const {
        for (const Tier& t : tiers_) { if (pct >= t.cutoff) { return t.letter; } }
        return tiers_.back().letter;
    }
    std::size_t tierCount() const { return tiers_.size(); }

private:
    std::vector<Tier> tiers_;
};

int main() {
    GradeScale d;
    std::cout << "default tiers=" << d.tierCount()
              << " 95->" << d.letterFor(95) << " 0->" << d.letterFor(0) << "\n";

    GradeScale bad({{90.0,'A'},{95.0,'X'},{-5.0,'Q'},{85.0,'B'}});
    std::cout << "repaired tiers=" << bad.tierCount() << "\n";
    for (double p : {96.0, 90.0, 86.0, 10.0}) {
        std::cout << "  " << p << " -> " << bad.letterFor(p) << "\n";
    }
    return 0;
}
```

Expected output:

```
default tiers=5 95->A 0->F
repaired tiers=3
  96 -> A
  90 -> A
```

```
  86 -> B
  10 -> F
```

Try: Work out by hand which of the four requested tiers survived and why. Then construct a scale from an empty vector — what do you get, and does `letterFor` still work?

4. Constructors and initializer lists

```cpp
#include <iostream>
#include <string>

class Course {
public:
    Course() = default;
    Course(const std::string& title, int credits)
        : title_(title), credits_(credits) {}

    void describe() const {
        std::cout << "[" << title_ << "] " << credits_ << " credits\n";
    }

private:
    std::string title_ = "Untitled";
    int credits_ = 0;
};

int main() {
    Course a;
    Course b("Programming Fundamentals", 4);
    a.describe();
    b.describe();
    return 0;
}
```

Expected output:

```
[Untitled] 0 credits
[Programming Fundamentals] 4 credits
```

Try: Remove `Course() = default;` and rebuild. Which line fails, and why does supplying one constructor remove the free one?

5. Convert a struct to a class

Take this struct and make it a class with a meaningful invariant: a bank balance may never be negative.

```cpp
struct Account {
    std::string owner;
    double balance = 0.0;
};
```

Provide `deposit` and `withdraw`. What should `withdraw` do when the amount exceeds the balance? Your answer is a design decision — write it down, and note that Chapter 24 gives you a better option than the ones available now.

6. Identify the classes

For a library system: *A member has a name, a number, and a list of borrowed books. A book has a title, an author, and an availability status. A member may borrow at most five books. A book cannot be borrowed by two members at once.*

Which nouns become classes and which stay structs? **What are the invariants?** Where is each enforced?

7. Reason about encapsulation

- Why must data members be private for an invariant to mean anything?
- What is the difference between a class and an object?
- Why should non-modifying member functions be `const`, and what breaks if they are not?
- Why is a class with a getter and setter for every member no better than a struct?
- `GradeScale::letterFor` has no error handling. Why is that safe here and would not be in Chapter 11?

Summary

- A **class** describes a kind of object; an **object** is one instance with its own state.
- **Encapsulation** bundles data with behavior and restricts access. Data members are **private**; the interface is public.
- A **member function** operates on the object it is called on. Mark non-modifying ones **`const`**, from the start.
- A **constructor** leaves a new object in a valid state. Prefer an **initializer list** to assignment in the body. Mark single-argument constructors **`explicit`**.
- **An invariant is something always true of an object.** State it above the class and enforce it in the constructor.
- **An invariant established in the constructor is a fact the rest of your code may rely on without checking** — which is why `letterFor` needs no error handling.
- **A getter and setter for every member protects nothing.** Provide the operations the problem needs.
- Find classes by looking for nouns with **behavior or rules.** Plain data with neither stays a **struct**.
- The point of privacy is not secrecy. It is that a class can **guarantee** things about itself that a struct cannot.

Key Terms

accessor — a member function that reads state without modifying it.

access specifier — `public`, `protected`, or `private`, controlling what may reach a member.

class — a type bundling data with the functions that operate on it.

const member function — a member function that does not modify the object.

constructor — a function run when an object is created, establishing its invariant.

destructor — a function run when an object is destroyed.

encapsulation — bundling data with behavior and restricting access to the data.

explicit — a qualifier preventing implicit conversion through a constructor.

initializer list — the `: member(value)` syntax initializing members before the constructor body.

instance — one object of a class.

invariant — a property that is always true of an object.

member function — a function declared inside a class, operating on an object of it.

mutator — a member function that changes state.

object — an instance of a class, holding its own state.

state — the data an object holds.

Next: Chapter 19 finishes the conversion. A `Gradebook` class takes ownership of the roster and the scale, `operator<<` prints an entire report with a single `<<`, and every class moves into its own header and implementation file pair. Grade Calculator v2.6.

Chapter 19 — Refining Classes

Learning Objectives

When you finish this chapter you will be able to:

- Pass and return objects efficiently and safely. *(SLO 2.2)*
- Explain what `this` refers to inside a member function. *(SLO 2.2)*
- Separate a class declaration from its definitions across a header and implementation file. *(SLO 2.2, 2.4)*
- Overload operators, including `<<`, `[]`, and `<`. *(SLO 2.2, 2.3)*
- Explain what `static` members are and when they are appropriate. *(SLO 2.2)*
- Describe what the compiler generates when you write nothing. *(SLO 2.2)*
- Explain what `friend` does and why it is used sparingly. *(SLO 2.2)*
- Build Grade Calculator v2.6 — one class per file, and a whole report printed with one `<<`.

19.1 Passing and Returning Objects

Chapter 10 Section 10.7's rule applies to objects, and now matters more, because objects can be large.

```
void printReport(const Gradebook& book);      // yes — no copy, cannot modify
void printReport(Gradebook book);             // no  — copies the entire roster
```

Take `const&` unless you intend to modify. A `Gradebook` holding 300 students copies 300 students otherwise.

Returning is different. Returning by value is normal and efficient:

```
Student Gradebook::studentAt(std::size_t i) const {
    return roster_[i];        // fine — the compiler avoids the copy
}
```

Modern C++ elides the copy in almost every such case. What you must **never** do is return a reference to something local:

```
const Student& bad() {
    Student s("Ada", 1001);
    return s;                 // s is destroyed; the reference dangles
}
```

That compiles, warns under `-Wall`, and produces undefined behavior. Returning a reference is only safe when the object outlives the call — a member, or something the caller owns.

19.2 The `this` Pointer

Inside a member function, `this` is a pointer to the object the function was called on:

```
void Student::rename(const std::string& name) {
    this->name_ = name;       // same as name_ = name
}
```

You rarely need it, because unqualified member names already refer to the current object. Two cases where it earns its place:

Returning the object itself, which enables chaining:

```
Student& Student::addAssignment(const Assignment& a) {
    assignments_.push_back(a);
    return *this;             // note the *, giving the object not the pointer
}

ada.addAssignment(hw).addAssignment(exam);   // chained
```

Distinguishing a member from a parameter of the same name. Appendix D's trailing underscore makes this unnecessary, which is one reason for the convention.

19.3 Separating Declaration from Definition

Chapter 10 Section 10.8 split functions across files. Classes split the same way, and Appendix D Section D.4 sets the rule: **one class, one header, one implementation file, all lowercase.**

The header — `gradescale.h`

```
// gradescale.h - Grade Calculator v2.6 - Chapter 19
#ifndef GRADESCALE_H
#define GRADESCALE_H

#include <iosfwd>
#include <vector>

/**
 * An ordered set of grade cutoffs.
 * INVARIANT: cutoffs strictly descend and the lowest tier is 0, so every
 * percentage from 0 upward maps to exactly one letter.
 */
class GradeScale {
public:
    struct Tier {
        double cutoff = 0.0;
        char   letter = 'F';
    };

    GradeScale();
    explicit GradeScale(const std::vector<Tier>& requested);

    char letterFor(double percentage) const;
    std::size_t tierCount() const { return tiers_.size(); }
    const std::vector<Tier>& tiers() const { return tiers_; }
```

```
private:
    std::vector<Tier> tiers_;
};

std::ostream& operator<<(std::ostream& out, const GradeScale& scale);

#endif
```

The implementation — `gradescale.cpp`

```
// gradescale.cpp - Grade Calculator v2.6 - Chapter 19
#include "gradescale.h"

#include <iomanip>
#include <ostream>

GradeScale::GradeScale() {
    tiers_ = { {90.0,'A'}, {80.0,'B'}, {70.0,'C'}, {60.0,'D'}, {0.0,'F'} };
}

char GradeScale::letterFor(double percentage) const {
    for (const Tier& t : tiers_) {
        if (percentage >= t.cutoff) { return t.letter; }
    }
    return tiers_.back().letter;
}
```

Each definition is prefixed with `GradeScale::` to say which class it belongs to.

What goes where. Very short accessors — one line, no logic — may stay in the header. Anything with real logic goes in the `.cpp`. The header should read as a summary of what the class *is*, uncluttered by how it works.

`<iosfwd>`

Note the header includes `<iosfwd>` rather than `<ostream>`. It declares that `std::ostream` exists without pulling in the whole stream library — enough to declare `operator<<`, and much less for the compiler to read in every file that includes yours.

The `.cpp` includes the full `<ostream>`, because that is where the definition actually uses it.

Headers should include as little as possible. Every file including your header pays for whatever it includes.

19.4 Operator Overloading

C++ lets you define what operators mean for your own types. Used well, this makes your classes read like built-in ones. Used badly, it makes code cryptic.

The rule: overload an operator only when its meaning is obvious. `+` on a `Money` class is obvious. `+` on a `Student` is not.

Stream output with `operator<<`

The most valuable overload by far:

```
std::ostream& operator<<(std::ostream& out, const Student& s) {
    out << std::fixed << std::setprecision(1)
        << std::left  << std::setw(6)  << s.id()
        << std::setw(14) << s.name()
        << std::right << std::setw(7)  << s.percentage() << "%";
    return out;
}
```

Now a `Student` prints like anything else:

```
std::cout << ada << "\n";
```

Three details make it work:

It is a free function, not a member. The left operand is the stream, not your object, so it cannot be a member of your class.

It takes and returns `std::ostream&`. Returning the stream is what allows chaining — `out << a << b` works because the first `<<` returns the stream for the second.

The object parameter is `const&`. Printing does not modify.

Because it takes any `std::ostream`, the same function writes to `std::cout`, to a file, or to a `std::ostringstream`. **One definition, every destination** — which is Chapter 15 Section 15.1's abstraction paying off.

Indexing with `operator[]`

```
const Student& Gradebook::operator[](std::size_t index) const {
    return roster_[index];
}
```

```
std::cout << book[1] << "\n";
```

A `Gradebook` now indexes like a vector. Note the **precondition**: this does not check bounds, matching what `[]` means everywhere else in C++. Document it, and provide an `at()` if you want checking — Chapter 12 Section 12.4's distinction, applied to your own class.

Comparison with `operator<`

```
bool Student::operator<(const Student& other) const {
    return percentage() > other.percentage();   // highest first
}
```

This one deserves a warning. **`operator<` here means "sorts before", not "is less than"** — a student with a *higher* percentage sorts first. That is a legitimate ordering and a surprising definition of `<`.

Chapter 23 shows the alternative: pass a lambda to `std::sort` stating the ordering where it is used. **When an operator's meaning is not obvious, a named function or an explicit comparison is better.**

Member or free function?

Overload as	When	Examples
Member	the left operand is your class	`[]`, `<`, `+=`
Free function	the left operand is something else	`<<`, `>>`

What you cannot do

You cannot invent new operators, change precedence, or change the number of operands. ** is not an operator in C++ and cannot be made one.

19.5 Static Members

A **static member** belongs to the class rather than to any object — one copy shared by all.

```
class Student {
public:
    static int totalCreated();
private:
    static int count_;        // declared here
};

int Student::count_ = 0;      // defined once, in the .cpp
```

Static data members are declared in the class and **defined once** in the implementation file. Forgetting the definition produces a linker error, which is Chapter 2 Section 2.2.4's division appearing again.

A **static member function** has no object and therefore no `this`. It can only touch static members.

Use static members sparingly. Chapter 10 Section 10.1's argument against globals applies: shared mutable state makes a class harder to test, because two tests can affect each other. A counter of objects created is a reasonable use; shared configuration usually is not.

19.6 What the Compiler Writes for You

If you write none of these, the compiler generates them:

Generated	Does
Default constructor	default-constructs each member — **only if you declare no other constructor**
Copy constructor	copies each member
Copy assignment	assigns each member
Destructor	destroys each member

For a class holding `std::string` and `std::vector`, **the generated versions are correct**, because those members copy and clean up properly themselves. `Student`, `Assignment`, and `GradeScale` all need nothing.

Two things to remember. **Declaring any constructor removes the free default one** — which is why `Student() = default;` appears explicitly when you also want a parameterized constructor. And the generated versions stop being correct as soon as a class manages a resource directly, which is Chapter 22's subject.

19.7 Friends

A **friend** function is granted access to a class's private members:

```
class Student {
    friend std::ostream& operator<<(std::ostream&, const Student&);
private:
    std::string name_;
};
```

`friend` deliberately breaks encapsulation, so it needs justification. The usual one is `operator<<`, which must be a free function but wants to print private state.

Often you do not need it. If the class already has public accessors, `operator<<` can use those:

```
std::ostream& operator<<(std::ostream& out, const Student& s) {
    out << s.id() << " " << s.name();     // public accessors — no friend needed
    return out;
}
```

Prefer accessors to friendship. Grade Calculator v2.6 uses no `friend` at all. The keyword is covered because you will meet it, and because knowing when *not* to reach for it is the useful part.

Common Errors and Warnings

What you see	Cause	Fix
`undefined reference to 'Student::name()'`	Declared in the header, not defined	Define it in the `.cpp` with `Student::`
`undefined reference to 'Student::count_'`	Static member declared, never defined	Define it once in the `.cpp`
`error: no match for 'operator<<'`	No overload for your type	Write one, or print members individually
`warning: reference to local variable returned`	Returning a reference to a local	Return by value
`error: passing 'const X' discards qualifiers`	Non-`const` function on a `const` object	Mark it `const`
`error: 'name_' is private within this context`	Free function touching private data	Use an accessor, or `friend` if justified
Chained `<<` fails to compile	`operator<<` returns `void`	Return `std::ostream&`
Every file recompiles after a small change	Header includes too much	Use `<iosfwd>`; move includes to the `.cpp`
`error: redefinition of class`	Missing header guard	Add one

Design Notes

Take objects by `const&`; return by value. Never return a reference to a local.

One class, one header, one implementation file. Headers summarize; implementations explain.

Headers include as little as possible. `<iosfwd>` over `<ostream>` when a declaration is all you need.

Overload an operator only when its meaning is obvious. `operator<<` almost always is; `operator<` on a `Student` is not.

Prefer accessors to `friend`.

Grade Calculator v2.6 — Gradebook Class

What v2.6 does

Everything v2.5 did, reorganized: a `Gradebook` class owns the roster, the scale, the course name, and the next-ID counter; reports print through `operator<<`; and every class lives in its own header and implementation file. It also adds one feature, **Sort by grade**, built on `Student::operator<` and `std::sort`.

The file layout

```
assignment.h    assignment.cpp
gradescale.h    gradescale.cpp
student.h       student.cpp
gradebook.h     gradebook.cpp
main.cpp
```

Every `.cpp` file has to reach the compiler. In StudySite that means keeping `main.cpp`, `assignment.cpp`, `gradescale.cpp`, `student.cpp`, and `gradebook.cpp` — and their headers — open in the editor before you click **Run**. A file that is not open is a file that is not built, and the linker error you get names the missing function, not the missing tab.

Nine files where there was one. That is the cost, and it buys something specific: each class can be read, changed, and reasoned about without the others in front of you.

The Gradebook class

```
class Gradebook {
public:
    explicit Gradebook(const std::string& courseName = "Untitled Course");

    void addStudent(const Student& s);
    void setScale(const GradeScale& scale) { scale_ = scale; }

    const GradeScale& scale() const { return scale_; }
    const std::string& courseName() const { return courseName_; }
    std::size_t size() const { return roster_.size(); }
    bool empty() const { return roster_.empty(); }

    /** Retrieves a student by position. Precondition: index < size(). */
    const Student& operator[](std::size_t index) const { return roster_[index]; }

    int claimNextId() { return nextId_++; }

    double classAverage() const;
    void sortByPercentage();

private:
    std::string courseName_;
    std::vector<Student> roster_;
    GradeScale scale_;
    int nextId_ = 1001;
};

std::ostream& operator<<(std::ostream& out, const Gradebook& book);
```

`Gradebook` owns everything a gradebook needs. In v2.5, `main` held a roster, a scale, and a next-ID counter as separate variables and passed them around. Now they are one thing, and the ID counter cannot drift out of step

with the roster.

The whole report, one operator

```
std::ostream& operator<<(std::ostream& out, const Gradebook& book) {
    out << "\n====================================\n";
    out << "  " << book.courseName() << "\n";
    out << "  " << book.scale() << "\n";
    out << "====================================\n";

    if (book.empty()) {
        out << "No students on the roster.\n";
        return out;
    }
    for (std::size_t i = 0; i < book.size(); ++i) {
        const Student& s = book[i];
        out << s << "   " << book.scale().letterFor(s.percentage()) << "\n";
    }
    out << "------------------------------------\n";
    out << std::fixed << std::setprecision(1)
        << std::left << std::setw(26) << "CLASS AVERAGE"
        << std::right << std::setw(8) << book.classAverage() << "%\n";
    return out;
}
```

And in `main`:

```
std::cout << book << "\n";      // one operator prints everything
```

Notice the nesting. `operator<<` for `Gradebook` uses `operator<<` for `Student`, which uses `operator<<` for `GradeScale`. **Each class knows how to print itself, and nothing knows how to print anything else.**

Sorting through `operator<`

```
void Gradebook::sortByPercentage() {
    std::sort(roster_.begin(), roster_.end());   // uses Student::operator<
}
```

One line, because `Student` defines its own ordering.

This is also where the concern from Section 19.4 becomes concrete. Reading `std::sort(roster_.begin(), roster_.end())` tells you the roster gets sorted; it does not tell you *by what*. You have to go and read `Student::operator<` — and discover it means "highest percentage first", which is not what `<` usually suggests.

Chapter 23 replaces this with a lambda stating the ordering at the call site. **Keep this version in mind for the comparison.**

Expected output

Zoe with 5/10 and Ada with 9/10 plus 1 bonus, reported, then sorted, then reported again:

```
  Scale: A >= 90   B >= 80   C >= 70   D >= 60   F >= 0
1001  Zoe                     50.0%   F
1002  Ada                    100.0%   A
CLASS AVERAGE                 75.0%

  Scale: A >= 90   B >= 80   C >= 70   D >= 60   F >= 0
1002  Ada                    100.0%   A
```

```
1001  Zoe                   50.0%   F
CLASS AVERAGE               75.0%
```

What to notice

`main` shrank considerably. It reads input, calls member functions, and prints. All the logic moved into the classes that own the data.

`main` never touches a roster directly. It cannot — `roster_` is private. Every change goes through `addStudent` or `sortByPercentage`, so the list of things that could corrupt it is exactly two functions.

No `friend` anywhere. Every `operator<<` uses public accessors.

Headers use `<iosfwd>`. Changing `student.cpp` does not force `gradebook.cpp` to recompile, because `student.h` pulls in almost nothing.

Every accessor is `const`. Without that, `operator<<` taking `const Gradebook&` would not compile.

Your StudySite Lab — Split Classes into Files

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v2.6
- **Starting point**: Your committed Chapter 18 v2.5 program (`main.cpp`).

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

In Chapter 18 all three classes lived in `main.cpp`. In this lab you give each class its own **header** (`.h`) and **implementation** (`.cpp`) file, add a `Gradebook` class, and teach every class to print itself with **`operator<<`**. By the end you will have:

1. `assignment.h`/`.cpp`, `gradescale.h`/`.cpp`, and `student.h`/`.cpp` — the Chapter 18 classes, split into declaration and definition, each header protected by an **include guard**.
2. An `operator<<` for `Assignment`, `GradeScale`, and `Student`, written using only each class's public accessors — no `friend` needed.
3. `Student::operator<`, which orders students by percentage, highest first.
4. A new `Gradebook` class (`gradebook.h`/`.cpp`) that owns the course name, the roster, the grade scale, and the next-ID counter, and prints a whole class report with a single `operator<<`.
5. A short `main.cpp` with five menu options: **Add student**, **Report**, **Sort by grade** (new), **Custom scale**, and **Quit**.

Files in this checkpoint

File	What you do to it
`assignment.h`, `assignment.cpp`	Create (Step 1)
`gradescale.h`, `gradescale.cpp`	Create (Step 2)
`student.h`, `student.cpp`	Create (Step 3)
`gradebook.h`, `gradebook.cpp`	Create (Step 4)
`main.cpp`	Edit (Steps 5–7)

When you are finished your project has **nine files**. Every one of them must be open in the StudySite editor when you click **Run** — a `.cpp` file that is not open is not compiled, and the linker error you get names the missing function, not the missing tab.

> **How to create a file**: in the StudySite editor, create a new file with the exact name shown (spelling and lowercase matter, because `#include` lines refer to these names). Keep each new file open after you create it.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

> **Heads-up**: Steps 1–4 create the new files; Steps 5–7 remove the old class code from `main.cpp`. Do **not** click Run until the checkpoint at the end of Step 7 — until then, the classes exist in two places at once.

Build it: step by step

Step 1 — Create `assignment.h` and `assignment.cpp` A **header** declares what a class looks like; its **implementation file** defines the function bodies. Other files `#include` the header, never the `.cpp`.

Create a new file named `assignment.h` and enter this code:

```
// assignment.h - Grade Calculator v2.6 - Chapter 19
#ifndef ASSIGNMENT_H
#define ASSIGNMENT_H

#include <iosfwd>
#include <string>

/** One graded item belonging to one student. */
class Assignment {
public:
    Assignment() = default;
    Assignment(const std::string& name, double earned, double possible, double bonus = 0.0);

    const std::string& name() const { return name_; }
    double pointsEarned() const     { return pointsEarned_; }
    double pointsPossible() const   { return pointsPossible_; }
    double bonusPoints() const      { return bonusPoints_; }

    double totalEarned() const;
    double ratio() const;

private:
    std::string name_;
    double pointsEarned_   = 0.0;
```

```
    double pointsPossible_ = 0.0;
    double bonusPoints_    = 0.0;
};

std::ostream& operator<<(std::ostream& out, const Assignment& a);

#endif
```

Notice:

- `#ifndef ASSIGNMENT_H` / `#define ASSIGNMENT_H` / `#endif` is an **include guard**. If two files both include `assignment.h`, the class is still defined only once.
- Short accessors stay in the header. `totalEarned()` and `ratio()` are now only **declared** here (they end in `;`); their bodies move to the `.cpp`.
- The header includes `<iosfwd>`, not `<iostream>`. `<iosfwd>` only *announces* that `std::ostream` exists, which is all a declaration like `operator<<(std::ostream&, ...)` needs. Files that include this header compile faster and do not drag in all of `<iostream>`.
- `operator<<` is declared **outside** the class. It is a free function that uses the class's public accessors.

Create a new file named `assignment.cpp` and enter this code:

```
// assignment.cpp - Grade Calculator v2.6 - Chapter 19
#include "assignment.h"

#include <iomanip>
#include <ostream>

Assignment::Assignment(const std::string& name, double earned, double possible, double bonus)
    : name_(name), pointsEarned_(earned), pointsPossible_(possible), bonusPoints_(bonus) {}

double Assignment::totalEarned() const {
    return pointsEarned_ + bonusPoints_;
}

double Assignment::ratio() const {
    return pointsPossible_ > 0.0 ? totalEarned() / pointsPossible_ : 0.0;
}

std::ostream& operator<<(std::ostream& out, const Assignment& a) {
    out << std::fixed << std::setprecision(1)
        << std::left << std::setw(18) << a.name()
        << std::right << std::setw(7) << a.totalEarned()
        << " / " << std::setw(6) << a.pointsPossible();
    if (a.bonusPoints() > 0.0) {
        out << "  (+" << a.bonusPoints() << " bonus)";
    }
    return out;
}
```

Each definition outside the class is written with the class name and the **scope resolution operator**: `Assignment::totalEarned()`. The constructor's default argument (`bonus = 0.0`) appears only in the header, never again here.

`operator<<` prints an assignment as one aligned line — name, total earned, and points possible, plus a bonus note when there is one — and **returns the stream**, which is what lets you chain `<<` calls.

Step 2 — Create `gradescale.h` and `gradescale.cpp` Create a new file named `gradescale.h` and enter this code:

```
// gradescale.h - Grade Calculator v2.6 - Chapter 19
#ifndef GRADESCALE_H
#define GRADESCALE_H

#include <iosfwd>
#include <vector>

/**
 * An ordered set of grade cutoffs.
 * INVARIANT: cutoffs strictly descend and the lowest tier is 0, so every
 * percentage from 0 upward maps to exactly one letter. The constructor
 * establishes this, and no member function can break it.
 */
class GradeScale {
public:
    struct Tier {
        double cutoff = 0.0;
        char   letter = 'F';
    };

    GradeScale();
    explicit GradeScale(const std::vector<Tier>& requested);

    char letterFor(double percentage) const;
    std::size_t tierCount() const { return tiers_.size(); }
    const std::vector<Tier>& tiers() const { return tiers_; }

private:
    std::vector<Tier> tiers_;
};

std::ostream& operator<<(std::ostream& out, const GradeScale& scale);

#endif
```

Create a new file named `gradescale.cpp` and enter this code:

```
// gradescale.cpp - Grade Calculator v2.6 - Chapter 19
#include "gradescale.h"

#include <iomanip>
#include <ostream>

GradeScale::GradeScale() {
    tiers_ = { {90.0,'A'}, {80.0,'B'}, {70.0,'C'}, {60.0,'D'}, {0.0,'F'} };
}

GradeScale::GradeScale(const std::vector<Tier>& requested) {
    for (const Tier& t : requested) {
        if (t.cutoff < 0.0) { continue; }
        if (!tiers_.empty() && t.cutoff >= tiers_.back().cutoff) { continue; }
        tiers_.push_back(t);
    }
    if (tiers_.empty() || tiers_.back().cutoff > 0.0) {
        tiers_.push_back({0.0, 'F'});
    }
}

char GradeScale::letterFor(double percentage) const {
    for (const Tier& t : tiers_) {
        if (percentage >= t.cutoff) { return t.letter; }
```

```
    }
    return tiers_.back().letter;
}

std::ostream& operator<<(std::ostream& out, const GradeScale& scale) {
    out << "Scale: ";
    for (const GradeScale::Tier& t : scale.tiers()) {
        out << t.letter << " >= " << std::fixed << std::setprecision(0)
            << t.cutoff << "   ";
    }
    return out;
}
```

Two changes from Chapter 18 besides the split:

- The invariant now lives in a **comment above the class**, where anyone reading the header will see it. The constructor logic that enforces it is unchanged.
- Chapter 18's `describe()` member is gone. **`operator<<` replaces it**, so printing a scale is `std::cout << scale` — the same way you print anything else. Unlike `describe()`, it does not add `"   "` in front or a newline at the end; the caller decides the layout.

Step 3 — Create `student.h` and `student.cpp` Create a new file named `student.h` and enter this code:

```
// student.h - Grade Calculator v2.6 - Chapter 19
#ifndef STUDENT_H
#define STUDENT_H

#include "assignment.h"

#include <iosfwd>
#include <string>
#include <vector>

class Student {
public:
    Student() = default;
    Student(const std::string& name, int id);

    const std::string& name() const { return name_; }
    int id() const { return id_; }
    const std::vector<Assignment>& assignments() const { return assignments_; }

    void addAssignment(const Assignment& a);

    double totalEarned() const;
    double totalPossible() const;
    double percentage() const;

    /** Orders students by percentage, highest first. Enables sorting. */
    bool operator<(const Student& other) const;

private:
    std::string name_;
    int id_ = 0;
    std::vector<Assignment> assignments_;
};

std::ostream& operator<<(std::ostream& out, const Student& s);
```

```
#endif
```

Create a new file named `student.cpp` and enter this code:

```
// student.cpp - Grade Calculator v2.6 - Chapter 19
#include "student.h"

#include <algorithm>
#include <cmath>
#include <iomanip>
#include <ostream>

namespace {
    const bool CAP_AT_100 = true;
}

Student::Student(const std::string& name, int id) : name_(name), id_(id) {}

void Student::addAssignment(const Assignment& a) {
    assignments_.push_back(a);
}

double Student::totalEarned() const {
    double sum = 0.0;
    for (const Assignment& a : assignments_) { sum += a.totalEarned(); }
    return sum;
}

double Student::totalPossible() const {
    double sum = 0.0;
    for (const Assignment& a : assignments_) { sum += a.pointsPossible(); }
    return sum;
}

double Student::percentage() const {
    double possible = totalPossible();
    if (possible <= 0.0) { return 0.0; }
    double raw = totalEarned() / possible * 100.0;
    double reported = CAP_AT_100 ? std::min(raw, 100.0) : raw;
    return std::round(reported * 10.0) / 10.0;
}

bool Student::operator<(const Student& other) const {
    return percentage() > other.percentage();   // highest first
}

std::ostream& operator<<(std::ostream& out, const Student& s) {
    out << std::fixed << std::setprecision(1)
        << std::left << std::setw(6) << s.id()
        << std::setw(20) << s.name()
        << std::right << std::setw(8) << s.percentage() << "%";
    return out;
}
```

Notice:

- `student.h` includes `"assignment.h"` — a `Student` holds a `std::vector<Assignment>`, so the compiler must see the full `Assignment` class first.
- `CAP_AT_100` moved out of `main.cpp` and into an **unnamed namespace** in `student.cpp`, the only file that uses it. An unnamed namespace makes the name private to that one file.

- **`operator<`** is a member function. It answers "does this student come before `other`?" and it means *higher percentage first* — not ordinary numeric less-than. That is the convenience and the risk of overloading an operator; Chapter 23 revisits it.
- `operator<<` prints ID, name, and percentage. It leaves out the letter grade, because a `Student` does not know which scale is in use.

Step 4 — Create the `Gradebook` class In Chapter 18, `main` held a scale, a roster, and a next-ID counter as three separate variables. A `Gradebook` class gathers them into one object that guards them.

Create a new file named `gradebook.h` and enter this code:

```
// gradebook.h - Grade Calculator v2.6 - Chapter 19
#ifndef GRADEBOOK_H
#define GRADEBOOK_H

#include "gradescale.h"
#include "student.h"

#include <iosfwd>
#include <string>
#include <vector>

/** Owns the roster and the grading scale, and produces reports. */
class Gradebook {
public:
    explicit Gradebook(const std::string& courseName = "Untitled Course");

    void addStudent(const Student& s);
    void setScale(const GradeScale& scale) { scale_ = scale; }

    const GradeScale& scale() const { return scale_; }
    const std::string& courseName() const { return courseName_; }
    std::size_t size() const { return roster_.size(); }
    bool empty() const { return roster_.empty(); }

    /** Retrieves a student by position. Precondition: index < size(). */
    const Student& operator[](std::size_t index) const { return roster_[index]; }

    int nextId() const { return nextId_; }
    int claimNextId() { return nextId_++; }

    double classAverage() const;
    void sortByPercentage();

private:
    std::string courseName_;
    std::vector<Student> roster_;
    GradeScale scale_;
    int nextId_ = 1001;
};

std::ostream& operator<<(std::ostream& out, const Gradebook& book);

#endif
```

- All the data is `private`. The roster can change only through `addStudent` and `sortByPercentage`, so those are the only two places that could ever corrupt it.
- `operator[]` returns a `const Student&` for a position. Its comment states the **precondition** (`index < size()`); it does not check.

- `claimNextId()` returns the next ID and advances the counter in one call, so an ID can never be handed out twice.

Create a new file named `gradebook.cpp` and enter this code:

```
// gradebook.cpp - Grade Calculator v2.6 - Chapter 19
#include "gradebook.h"

#include <algorithm>
#include <iomanip>
#include <ostream>

Gradebook::Gradebook(const std::string& courseName) : courseName_(courseName) {}

void Gradebook::addStudent(const Student& s) {
    roster_.push_back(s);
}

double Gradebook::classAverage() const {
    if (roster_.empty()) { return 0.0; }
    double sum = 0.0;
    for (const Student& s : roster_) { sum += s.percentage(); }
    return sum / static_cast<double>(roster_.size());
}

void Gradebook::sortByPercentage() {
    std::sort(roster_.begin(), roster_.end());   // uses Student::operator<
}

std::ostream& operator<<(std::ostream& out, const Gradebook& book) {
    out << "\n====================================\n";
    out << "  " << book.courseName() << "\n";
    out << "  " << book.scale() << "\n";
    out << "====================================\n";

    if (book.empty()) {
        out << "No students on the roster.\n";
        return out;
    }
    for (std::size_t i = 0; i < book.size(); ++i) {
        const Student& s = book[i];
        out << s << "   " << book.scale().letterFor(s.percentage()) << "\n";
    }
    out << "------------------------------------\n";
    out << std::fixed << std::setprecision(1)
        << std::left << std::setw(26) << "CLASS AVERAGE"
        << std::right << std::setw(8) << book.classAverage() << "%\n";
    return out;
}
```

- `sortByPercentage()` is one line: `std::sort` puts the roster in order using `Student::operator<`, so it sorts highest percentage first.
- `operator<<` for `Gradebook` prints the whole report. It prints the course name, then the scale **using the scale's own <<**, then each student **using the student's own <<**, followed by the letter grade, and finally a class average. Each class knows how to print itself, and nothing knows how to print anything else.

Step 5 — Update the header comment in `main.cpp` Now switch to `main.cpp`. From here on, `main.cpp` only runs the menu.

At the top of `main.cpp`, find the header comment:

```
// ============================================================================
//  Grade Calculator  v2.5                                        Chapter 18
// ----------------------------------------------------------------------------
//  Change  : Structs become classes with private data and behavior.
//            GradeScale validates its own tiers in the constructor, so an
//            invalid scale can no longer be constructed at all. The loose
//            validation functions of Chapter 11 are gone.
//            Each Student now owns its own Assignment records.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc
// ============================================================================
```

Replace it with:

```
// ============================================================================
//  Grade Calculator  v2.6                                        Chapter 19
// ----------------------------------------------------------------------------
//  Change  : One header/implementation pair per class. Reports print through
//            operator<<, so a whole Gradebook is written with a single <<.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp assignment.cpp gradescale.cpp
//            student.cpp gradebook.cpp -o gradecalc
// ============================================================================
```

The new `Build` line lists all five `.cpp` files — every one must be compiled and linked together.

Step 6 — Remove the classes from `main.cpp` The classes now live in their own files, so `main.cpp` must stop defining them. It includes `gradebook.h` instead, which in turn includes the other headers.

Just below the header comment, find the first `#include` line:

```
#include <algorithm>
```

Delete that line and everything below it (the remaining `#include` lines, `CAP_AT_100`, all three classes, and the *Input helpers* banner), stopping just **above** this line, which you keep:

```
double readNonNegative(const std::string& prompt) {
```

In the space where that code was, enter:

```
#include "gradebook.h"

#include <iostream>
#include <limits>
#include <string>
```

Afterward, `main.cpp` should begin with the header comment, then these four `#include` lines, then `readNonNegative`.

Why: a class must be defined exactly once in the whole program. If `main.cpp` kept its own copy, the program would have two definitions of each class. Quoted includes (`"gradebook.h"`) name your own files; angle-bracket includes (`<iostream>`) name the standard library.

Step 7 — Rewrite `main` to use a `Gradebook` Find the start of `main`:

```
int main() {
```

Delete that line and everything below it, all the way to the final closing `}` at the very end of the file.

In its place, enter the new `main`:

```
int main() {
    std::cout << "=== GRADE CALCULATOR v2.6 (one class per file) ===\n";

    Gradebook book("Programming Fundamentals");
    bool running = true;

    while (running) {
        std::cout << "\n1) Add student  2) Report  3) Sort by grade  "
                     "4) Custom scale  5) Quit\nChoice: ";
        std::string line;
        if (!std::getline(std::cin, line)) { break; }
        char choice = line.empty() ? '?' : line[0];

        if (choice == '1') {
            std::string name = readLine("  Student name: ");
            if (name.empty()) { continue; }
            Student s(name, book.claimNextId());
            while (true) {
                std::string an = readLine("    Assignment name (or 'done'): ");
                if (an == "done" || an.empty()) { break; }
                double earned   = readNonNegative("      Points earned  : ");
                double possible = readNonNegative("      Points possible: ");
                double bonus    = readNonNegative("      Bonus points   : ");
                s.addAssignment(Assignment(an, earned, possible, bonus));
            }
            book.addStudent(s);
            std::cout << "  Added " << s.name() << " as ID " << s.id() << ".\n";

        } else if (choice == '2') {
            std::cout << book << "\n";       // one operator<< prints everything

        } else if (choice == '3') {
            book.sortByPercentage();
            std::cout << "  Sorted, highest first.\n";

        } else if (choice == '4') {
            std::vector<GradeScale::Tier> requested;
            std::cout << "  Enter tiers highest first, 'done' to finish.\n";
            while (true) {
                std::string letterText = readLine("    Letter (or 'done'): ");
                if (letterText == "done" || letterText.empty()) { break; }
                double cutoff = readNonNegative("      Minimum percentage: ");
                requested.push_back({cutoff, letterText[0]});
            }
            book.setScale(GradeScale(requested));
            std::cout << "  " << book.scale() << "\n";

        } else if (choice == '5') {
            running = false;
        } else {
            std::cout << "  Please enter 1-5.\n";
        }
    }
    std::cout << "\nGoodbye.\n";
    return 0;
}
```

What changed:

- `Gradebook book("Programming Fundamentals");` replaces the three loose variables from Chapter 18.

- **Add student** gets its ID from `book.claimNextId()` and stores the student with `book.addStudent(s)`.
- **Report** is now one line: `std::cout << book << "\n";`.
- **Sort by grade** (new, option 3) calls `book.sortByPercentage()`.
- **Custom scale** (now option 4) builds the scale exactly as before and hands it to `book.setScale(...)`, then prints it with `<<`.
- **Quit** moves to option 5.

▸ **Run checkpoint**: Make sure all nine files are open in the editor, then click **Run**. The program should compile with no errors and no warnings and show the v2.6 banner and the five-option menu. An `undefined reference` error means a `.cpp` file is not open or a function name is misspelled; a `redefinition` error means a class is still defined in `main.cpp`.

Check your work

Run the program and type the inputs below in the Terminal.

1. Choose `1`. Add `Zoe` with one assignment: `HW1`, earned `5`, possible `10`, bonus `0`. Type `done`.
2. Choose `1`. Add `Ada` with one assignment: `HW1`, earned `9`, possible `10`, bonus `1`. Type `done`.
3. Choose `2` (Report). You should see:

```
====================================
  Programming Fundamentals
  Scale: A >= 90   B >= 80   C >= 70   D >= 60   F >= 0
====================================
1001  Zoe                     50.0%   F
1002  Ada                    100.0%   A
------------------------------------
CLASS AVERAGE                 75.0%
```

4. Choose `3` (Sort by grade). The program prints `Sorted, highest first.`
5. Choose `2` again. Ada (1002) is now listed first, Zoe second, and the class average is still 75.0%.
6. Choose `5` (Quit). The program prints `Goodbye.`

Then confirm:

☐ All nine files are in the project, and the program compiles with no errors and no warnings.
☐ Every header starts with an include guard (`#ifndef` / `#define`) and ends with `#endif`.
☐ `main.cpp` no longer contains `class Assignment`, `class GradeScale`, `class Student`, or `CAP_AT_100`.
☐ The report is printed by `std::cout << book`, and no `operator<<` uses `friend`.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click `main.cpp` to open it. You will create the eight new files in the editor during the lab. Confirm the editor shows the expected file paths before editing.
4. Work through Steps 1–7 above in StudySite's internal editor, in order. Keep every file open, and click **Run** at the ▸ Run checkpoint at the end of Step 7.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.

6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — all nine files: `main.cpp` plus the four `.h` and four `.cpp` files you created. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 19 Grade Calculator v2.6**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 19 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. `operator<<` for your own class

```cpp
#include <iomanip>
#include <iostream>
#include <ostream>
#include <string>

class Student {
public:
    Student(const std::string& n, int i, double p) : name_(n), id_(i), pct_(p) {}
    const std::string& name() const { return name_; }
    int id() const { return id_; }
    double percentage() const { return pct_; }
private:
    std::string name_;
    int id_ = 0;
    double pct_ = 0.0;
};

std::ostream& operator<<(std::ostream& out, const Student& s) {
    out << std::fixed << std::setprecision(1)
        << std::left << std::setw(6) << s.id()
        << std::setw(14) << s.name()
        << std::right << std::setw(7) << s.percentage() << "%";
    return out;
}
```

```
int main() {
    Student ada("Ada", 1002, 90.0);
    std::cout << ada << "\n";
    std::cout << "twice: " << ada << " and " << ada << "\n";
    return 0;
}
```

Expected output:

```
1002  Ada             90.0%
twice: 1002  Ada              90.0% and 1002  Ada              90.0%
```

Try: Change the return type to `void` and rebuild. Which line fails, and why does returning the stream matter?

2. Nested `operator<<` and `operator[]`

```
#include <iomanip>
#include <iostream>
#include <ostream>
#include <string>
#include <vector>

class Student {
public:
    Student(const std::string& n, int i, double p) : name_(n), id_(i), pct_(p) {}
    const std::string& name() const { return name_; }
    int id() const { return id_; }
    double percentage() const { return pct_; }
private:
    std::string name_;
    int id_ = 0;
    double pct_ = 0.0;
};

std::ostream& operator<<(std::ostream& out, const Student& s) {
    out << std::fixed << std::setprecision(1)
        << std::left << std::setw(6) << s.id() << std::setw(14) << s.name()
        << std::right << std::setw(7) << s.percentage() << "%";
    return out;
}

class Gradebook {
public:
    explicit Gradebook(const std::string& c) : course_(c) {}
    void add(const Student& s) { roster_.push_back(s); }
    std::size_t size() const { return roster_.size(); }
    const Student& operator[](std::size_t i) const { return roster_[i]; }
    const std::string& course() const { return course_; }
    double average() const {
        if (roster_.empty()) { return 0.0; }
        double t = 0.0;
        for (const Student& s : roster_) { t += s.percentage(); }
        return t / static_cast<double>(roster_.size());
    }
private:
    std::string course_;
    std::vector<Student> roster_;
};
```

```
std::ostream& operator<<(std::ostream& out, const Gradebook& b) {
    out << "== " << b.course() << " ==\n";
    for (std::size_t i = 0; i < b.size(); ++i) { out << b[i] << "\n"; }
    out << std::fixed << std::setprecision(1) << "AVERAGE " << b.average() << "%\n";
    return out;
}

int main() {
    Gradebook book("Programming Fundamentals");
    book.add({"Zoe", 1001, 50.0});
    book.add({"Ada", 1002, 90.0});
    std::cout << book;
    std::cout << "\nsingle student: " << book[1] << "\n";
    return 0;
}
```

Expected output:

```
== Programming Fundamentals ==
1001  Zoe              50.0%
1002  Ada              90.0%
AVERAGE 70.0%

single student: 1002  Ada              90.0%
```

Try: Write the same output to a file with `std::ofstream out("r.txt"); out << book;`. How many lines did you change in `operator<<`?

3. Returning a reference to a local

```
#include <iostream>
#include <string>

const std::string& bad() {
    std::string local = "temporary";
    return local;                 // local is destroyed on return
}

int main() {
    std::cout << "built\n";
    return 0;
}
```

Try: Build it and read the warning. Then call `bad()` and print the result. What happens? Now explain why returning by value is safe where this is not.

4. `this` and chaining

```
#include <iostream>
#include <string>
#include <vector>

class Roster {
public:
    Roster& add(const std::string& name) {
        names_.push_back(name);
        return *this;
    }
```

```
    void show() const {
        for (const std::string& n : names_) { std::cout << n << " "; }
        std::cout << "\n";
    }
private:
    std::vector<std::string> names_;
};

int main() {
    Roster r;
    r.add("Ada").add("Grace").add("Alan");
    r.show();
    return 0;
}
```

Expected output:

```
Ada Grace Alan
```

Try: Change `return *this;` to `return this;` and read the error. What is the difference between the two?

5. Static members

```
#include <iostream>

class Counter {
public:
    Counter() { ++count_; }
    static int created() { return count_; }
private:
    static int count_;
};

int Counter::count_ = 0;

int main() {
    Counter a;
    Counter b;
    Counter c;
    std::cout << "created: " << Counter::created() << "\n";
    return 0;
}
```

Expected output:

```
created: 3
```

Try: Delete the line `int Counter::count_ = 0;` and rebuild. Which tool reports the error — compiler or linker? Chapter 2 Section 2.2.4 explains why.

6. Split a class across files

Take the `Student` class from Exercise 1 and split it into `student.h` and `student.cpp` with a header guard. Put the `operator<<` declaration in the header and its definition in the `.cpp`.

Then build with only `main.cpp` and identify which stage produces the error.

7. Judge an overload

For each, say whether overloading the operator is a good idea, and why:

- `operator+` on a `Money` class, adding amounts
- `operator+` on a `Student` class
- `operator==` on an `Assignment`, comparing name and points
- `operator<` on a `Student`, meaning "higher percentage"
- `operator[]` on a `Gradebook`, retrieving a student
- `operator<<` on anything you want to print

Summary

- **Take objects by `const&`; return by value**. Never return a reference to a local.
- **`this`** points to the object a member function was called on. `return *this;` enables chaining.
- **One class, one header, one implementation file**. Definitions are prefixed with `ClassName::`. Short accessors may stay in the header; logic goes in the `.cpp`.
- **Headers should include as little as possible**. `<iosfwd>` declares `std::ostream` without the whole library.
- **`operator<<`** is the most valuable overload: a free function taking and returning `std::ostream&`, with the object as `const&`. Because it takes any stream, one definition serves the console, files, and string streams.
- **Overload an operator only when its meaning is obvious**. `operator<` meaning "highest first" is a real trap.
- Overload as a **member** when the left operand is your class; as a **free function** when it is not.
- **Static members** belong to the class, not to objects, and must be defined once outside it.
- The compiler generates a **default constructor**, **copy constructor**, **copy assignment**, **and destructor** — correct for classes holding standard types. Declaring any constructor removes the free default one.
- **`friend` breaks encapsulation deliberately**. Prefer public accessors; v2.6 needs no friends.

Key Terms

copy assignment — the operation assigning one existing object to another.

copy constructor — the constructor creating an object as a copy of another.

friend — a function granted access to a class's private members.

operator overloading — defining what an operator means for a user-defined type.

static member — a member belonging to the class rather than to any object.

this — a pointer, available in member functions, to the object the function was called on.

Next: Chapter 20 is where Course II pays off. The requirement you deferred in Chapter 1 finally arrives: weighted-category grading. You will see what a flag-and-branch design would cost, then build a small class hierarchy instead — and discover that the classes you spent two chapters encapsulating need no changes at all. Grade Calculator v3.0.

Chapter 20 — Inheritance

Learning Objectives

When you finish this chapter you will be able to:

- State the weighted-grading requirement precisely and compute a weighted percentage by hand. *(SLO 2.1)*
- Compare a flag-and-branch design with a class hierarchy, and name the specific costs of each. *(SLO 2.3, 2.8)*
- Identify "is-a" relationships and distinguish them from "has-a". *(SLO 2.3)*
- Declare a derived class and call a base constructor. *(SLO 2.3)*
- Use `protected` members appropriately. *(SLO 2.3)*
- Override a base class member function. *(SLO 2.3)*
- Choose between inheritance and composition. *(SLO 2.3, 2.8)*
- Build Grade Calculator v3.0 — weighted-category grading, working.

20.1 A New Requirement: Weighted Grading

Since Chapter 1, your specification has carried a sentence:

> **Out of scope for Course I**: weighted-category grading, in which categories such as exams and homework carry different percentages of the final grade. This calculator computes grades on total points only. Weighted grading is deferred as a planned future enhancement.

Twelve chapters of Course I never needed it. Chapter 13 had you analyze it. This chapter builds it.

What weighted grading is

Assignments belong to **categories**, and each category carries a **weight** — a percentage of the final grade:

Category	Weight
Exams	50%
Homework	30%
Participation	20%

Each category is scored **separately** as a fraction of its own possible points. Those fractions are then combined by weight.

Worked by hand

A student with `Exam 1` at 90/100, `HW1` at 10/10, and `HW2` at 5/10:

Points-based adds everything together:

```
(90 + 10 + 5) / (100 + 10 + 10) = 105 / 120 = 87.5%
```

Weighted scores each category first:

```
Exams:    90/100  = 0.90  ×  50 = 45.0
Homework: 15/20   = 0.75  ×  30 = 22.5
                                  ----
                                  67.5   out of the 80 weight used
67.5 / 80 × 100 = 84.4%
```

```
points-based: 87.5%
weighted:     84.4%
```

Same work, two defensible grades, a 3.1 point difference. Neither is wrong. They answer different questions, and an instructor is entitled to choose.

Note that Participation contributed nothing because the student has no participation assignments yet. The design decision above — **divide by the weight actually used, not by 100** — means a student is not penalized for work that has not been assigned. That was one of the questions Chapter 13 asked you to answer before knowing the implementation.

Why the accumulator pattern cannot do this

Chapter 7 Section 7.9 said points-based grading *is* the accumulator pattern: two running totals, one division. It also said what that implies.

Every point counts the same, regardless of where it came from. Two running totals cannot express "these points are worth more than those." You would need separate totals per category, combined afterward by weight — a fundamentally different computation, not a variation on the same one.

That is why this arrives now. It is not a feature Course I forgot; it is a feature Course I's approach could not express.

20.2 Two Designs Compared

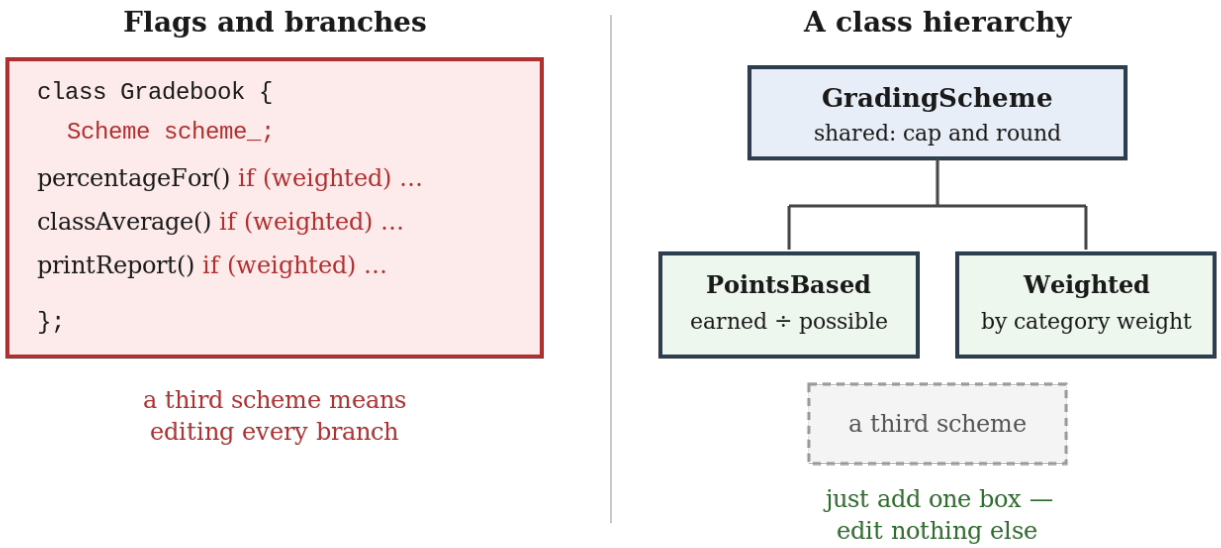
You have a working `Gradebook` with one grading rule. You need two. There are two ways to do it, and the choice matters more than it first appears.

Figure 20.1 — Two designs for supporting more than one grading scheme.

Description of Figure 20.1. The left panel shows the **flag design**: one `Gradebook` class holding a `Scheme scheme_` member, with three operations — `percentageFor`, `classAverage`, `printReport` — each containing an `if (weighted)` branch. A note records that a third scheme means editing every branch.

The right panel shows the **hierarchy design**: an abstract `GradingScheme` at the top holding shared behavior (capping and rounding), with `PointsBased` and `Weighted` below it. A dashed box shows where a third scheme would go, with a note that adding one means adding a box and editing nothing else.

Both compute the same grades. They differ in what a change costs.



Both compute the same grades. They differ in what a change costs.

Figure 15: Two panels: on the left a single Gradebook class holding a scheme flag with an if-else branch inside each of three operations; on the right an abstract GradingScheme with PointsBased and Weighted beneath it.

Design A — a flag and branches

```
class Gradebook {
public:
    enum class Scheme { Points, Weighted };

    double percentageFor(const Student& s) const {
        if (scheme_ == Scheme::Weighted) {
            // ... weighted computation ...
        } else {
            // ... points-based computation ...
        }
    }
private:
    Scheme scheme_;
};
```

This works. It is not stupid, and in a small program it would be entirely reasonable. Its costs are specific:

The branch spreads. Today one function needs it. Tomorrow the class average, the report header, and the file format each need to know which scheme is active. Every new operation on grades brings a new branch.

A third scheme touches every branch. Adding weighted-with-drop-lowest means finding each `if` and extending it. Miss one and you have a silent logic error of exactly the kind Chapter 4 Section 4.9.3 described.

The branches cannot be tested independently. Testing the weighted path means constructing a `Gradebook` with the flag set correctly. There is no way to test the weighted rule on its own.

`Gradebook` accumulates knowledge that is not its job. A gradebook holds students and produces reports. Under this design it also knows the arithmetic of every grading scheme that will ever exist.

Design B — a class hierarchy

Make each scheme a separate type with a shared parent:

```
class GradingScheme { /* shared behavior */ };
class PointsBased : public GradingScheme { /* one rule */ };
class Weighted    : public GradingScheme { /* another rule */ };
```

`Gradebook` holds one of these and asks it for a percentage. It does not know which kind it has.

A third scheme is a new class, and nothing existing changes.

The comparison

	Flag and branches	Class hierarchy
Adding a third scheme	edit every branch	add one class
Testing one rule alone	not possible	construct it directly
Where scheme knowledge lives	inside `Gradebook`	inside each scheme
Risk of missing a branch	real	none — there are none
Lines for two schemes	fewer	more
Lines for five schemes	many more	slightly more

The hierarchy costs more up front. That is honest, and it is why the flag design is right for some programs. It pays off when the number of variants grows or when the variants must be tested and reasoned about separately — which is exactly this case.

You will build Design B. Chapter 21 will show what it makes possible that Design A cannot do at all.

20.3 Modeling "Is-A" Relationships

Inheritance creates a new class from an existing one. The new class gets everything the original had, and may add or change behavior.

The relationship it expresses is "**is-a**":

- A `PointsBased` **is a** `GradingScheme`.
- A `Weighted` **is a** `GradingScheme`.

Test any proposed inheritance by saying it aloud. If "is-a" sounds wrong, inheritance is wrong.

Contrast with "**has-a**", which means **composition** — one object holding another:

- A `Gradebook` **has a** `GradeScale`.
- A `Student` **has** `Assignments`.

A `Gradebook` is not a kind of `GradeScale`, so it holds one rather than inheriting from one. **Getting this backwards is the most common inheritance mistake**, and Section 20.10 returns to it.

20.4 Base and Derived Classes

The **base class** holds what is common:

```
class GradingScheme {
public:
    explicit GradingScheme(const std::string& name) : name_(name) {}

    const std::string& name() const { return name_; }

protected:
    /** Applies the course cap-and-round policy. Shared by every scheme. */
    static double finalize(double rawPercentage);

private:
    std::string name_;
};
```

A **derived class** names its base after a colon:

```
class PointsBased : public GradingScheme {
public:
    PointsBased() : GradingScheme("Points-based") {}
    double computePercentage(const std::vector<Assignment>& work) const;
};
```

`public` inheritance means "is-a" — everything public in the base stays public in the derived class. Other forms exist and are rarely what you want.

A `PointsBased` object contains a complete `GradingScheme` inside it. It has a `name_`, it can be asked its `name()`, and it can call `finalize` — all without those being written twice.

20.5 Constructors Under Inheritance

A derived class must construct its base part first. Do it in the **initializer list**:

```
PointsBased::PointsBased() : GradingScheme("Points-based") {}

Weighted::Weighted() : GradingScheme("Weighted") {
    weights_ = { {"Exam", 50.0}, {"Homework", 30.0}, {"Participation", 20.0} };
}
```

The base constructor runs, then the derived class's members initialize, then the derived constructor body runs. **Construction proceeds from the base outward**, which makes sense: the derived part may depend on the base part being ready.

If you do not name a base constructor, the compiler calls the base's default one. If there is none, the code will not compile — a common error when a base class has only parameterized constructors.

20.6 `protected` Members

Three access levels, now that inheritance exists:

Specifier	Reachable from
`public`	anywhere
`protected`	the class **and its derived classes**
`private`	the class only

Specifier	Reachable from

`finalize` is `protected` because every scheme needs it and no outside code should:

```
protected:
    static double finalize(double rawPercentage);
```

Prefer `private` to `protected` for data. A `protected` data member can be modified by any derived class, present or future, which puts your invariant back at the mercy of code you have not written. `protected` functions are less risky than `protected` data.

`Weighted` does declare `weights_` as `protected` rather than private — deliberately, because Chapter 21 adds a class deriving from `Weighted` that needs it. That is a decision made in anticipation of a known requirement, not a default.

20.7 Overriding Member Functions

A derived class may replace a base class function by declaring one with the same signature. This is **overriding**, and it is different from the overloading in Chapter 10 Section 10.2:

	Overloading	Overriding
Same name	yes	yes
Same parameters	**no**	**yes**
Involves inheritance	no	yes
Resolved	at compile time	see Chapter 21

`PointsBased` and `Weighted` each define `computePercentage`:

```
double PointsBased::computePercentage(const std::vector<Assignment>& work) const {
    double earned = 0.0;
    double possible = 0.0;
    for (const Assignment& a : work) {
        earned   += a.totalEarned();
        possible += a.pointsPossible();
    }
    if (possible <= 0.0) { return 0.0; }
    return finalize(earned / possible * 100.0);
}

double Weighted::computePercentage(const std::vector<Assignment>& work) const {
    double weightedSum = 0.0;
    double weightUsed  = 0.0;

    for (const CategoryWeight& w : weights_) {
        double earned = 0.0;
        double possible = 0.0;
        for (const Assignment& a : work) {
            if (a.category() == w.name) {
                earned   += a.totalEarned();
                possible += a.pointsPossible();
            }
        }
        // A category with no graded work is skipped, and its weight is
```

```
        // redistributed. Otherwise a student would be penalized for
        // assignments not yet given.
        if (possible > 0.0) {
            weightedSum += (earned / possible) * w.weight;
            weightUsed  += w.weight;
        }
    }
    if (weightUsed <= 0.0) { return 0.0; }
    return finalize(weightedSum / weightUsed * 100.0);
}
```

Both take the same argument and return a percentage. Both call the inherited `finalize`. **The cap-and-round policy exists once**, and a third scheme would inherit it too.

> **A limitation, stated now.** These are not yet `virtual`, which means the *type of the variable*, not the type of the object, decides which one runs. Chapter 21 removes this, and it is why v3.0 fixes the scheme at startup. The limitation is real and disclosed rather than hidden.

20.8 Multilevel Hierarchies

A derived class may itself be a base:

```
class GradingScheme { };
class Weighted : public GradingScheme { };
class WeightedDropLowest : public Weighted { };
```

`WeightedDropLowest` is a `Weighted`, which is a `GradingScheme`. It inherits from both levels.

Chapter 21 adds exactly this class, as proof the design is open to extension.

Keep hierarchies **shallow**. Two or three levels is usually plenty; deeper ones become hard to reason about, because behavior may come from any level and you must check them all.

20.9 Composition as an Alternative

Composition — holding an object rather than inheriting from one — is often the better choice.

```
class Gradebook {
private:
    GradeScale scale_;                  // has-a
    std::vector<Student> roster_;       // has-a
};
```

A `Gradebook` is not a kind of `GradeScale`; it holds one.

The general advice is **prefer composition to inheritance**. Composition is more flexible: a held object can be replaced at run time, and it exposes only what you choose. Inheritance is a permanent, visible commitment.

Reach for inheritance when both conditions hold:

1. The relationship is genuinely **is-a**, and
2. You need code to treat different derived types **uniformly**.

Condition 2 is what makes the grading schemes a real case, and Chapter 21 is where it becomes visible.

20.10 When Not to Inherit

Four warning signs:

"Is-a" sounds wrong. A `Gradebook` is not a `std::vector<Student>`, even though it holds one.

You inherit to reuse code, not to model a relationship. If you want the base's functions but the is-a claim is false, use composition.

The derived class must remove behavior. If a derived class has to disable something the base offers, the is-a relationship is false — anything you can do with the base should work with the derived.

The hierarchy is deep and behavior is scattered. Three levels is a signal to reconsider.

Common Errors and Warnings

What you see	Cause	Fix
`error: no matching function for call to 'GradingScheme::GradingScheme()'`	Base has no default constructor and none was named	Call one in the initializer list
`error: 'name_' is private within this context`	Derived class touching base's private data	Use a public accessor, or `protected`
Base's function runs when the derived one was expected	Not `virtual` — Section 20.7	Chapter 21
`error: cannot convert 'PointsBased' to 'GradingScheme'`	Wrong direction — a base is not a derived	Only derived-to-base is automatic
Base constructor runs after derived members initialize	It does not; construction is base-outward	Nothing to fix; check your expectation
`error: invalid use of incomplete type`	Base class not fully declared before use	Include the base's header
Derived class changed base state unexpectedly	`protected` data member	Make it `private` with a `protected` accessor
Duplicate code across derived classes	Common behavior in the wrong place	Move it to the base

---

Design Notes

Say "is-a" aloud before inheriting. If it sounds wrong, use composition.

Prefer composition to inheritance. Inherit when the relationship is genuine *and* code must treat several types uniformly.

Put shared behavior in the base once. `finalize` is the model.

Prefer `private` to `protected` data. A `protected` member is reachable by every future derived class.

Keep hierarchies shallow.

Grade Calculator v3.0 — Weighted Grading

What v3.0 does

Everything v2.6 did, **plus weighted-category grading**. The scheme is chosen at startup and applies for the session.

The deferred requirement from Chapter 1 is now built.

Assignments gain a category

```
Assignment(const std::string& name, double earned, double possible,
           double bonus = 0.0, const std::string& category = "Uncategorized");
```

Both new parameters have defaults, so **every existing call still compiles unchanged** — Chapter 10 Section 10.3's technique for extending an interface without breaking it. Two hundred lines of calling code needed no edits.

Weights are validated

```
double Weighted::weightTotal() const {
    double sum = 0.0;
    for (const CategoryWeight& w : weights_) { sum += w.weight; }
    return sum;
}

bool Weighted::weightsValid() const {
    return std::abs(weightTotal() - 100.0) < 0.001;
}
```

Note the comparison. Weights are `double`, so `weightTotal() == 100.0` is exactly the floating-point equality trap from Chapter 6 Section 6.12. **Comparing against a tolerance** is Chapter 9 Section 9.10's `nearlyEqual` technique, applied where it matters.

The selection, in one place

```
double Gradebook::percentageFor(const Student& s) const {
    // The single branch. One place today; one place per operation as soon as
    // a third scheme or a second calculation is added. See Section 20.2.
    if (scheme_ == Scheme::Weighted) {
        return weighted_.computePercentage(s.assignments());
    }
    return points_.computePercentage(s.assignments());
}
```

This is Design A's branch, deliberately confined to one function. **v3.0 is a hybrid**: it has the hierarchy, and it still selects with an `if`, because the functions are not yet `virtual`.

That is the honest state of the program at the end of this chapter, and the comment says so. Chapter 21 removes the branch entirely.

Expected output

Ada with `Exam 1` 90/100 in category Exam, and `HW1` 10/10 in category Homework:

```
  Grading: Points-based
1001  Ada                    90.9%   A
```

```
  Grading: Weighted
1001  Ada                    93.8%   A
```

Check the weighted figure by hand: exams are 90/100 = 0.90 × 50 = 45; homework is 10/10 = 1.00 × 30 = 30; total 75 out of the 80 weight used; 75/80 × 100 = 93.75%, rounded to 93.8%.

The same scores, two grades. That is not a bug — it is the point.

What did not change

This is the chapter's real result, and it is worth checking rather than taking on trust.

Class	Changes needed
`GradeScale`	**none**
`Student`	**none**
`Assignment`	two parameters added, both defaulted
Report formatting	**none**
Bonus point handling	**none**
Drop-lowest	**none**

The classes you spent Chapters 18 and 19 encapsulating **absorbed a change to the grading model without modification.** `GradeScale` maps a percentage to a letter; it does not care where the percentage came from. `Student` holds work; it does not care how the work is scored.

That is the return on encapsulation, and you can verify it with `diff` rather than believe it.

What v3.0 still cannot do

Switching schemes requires restarting the program. The selection lives in a branch at one point, so the scheme is effectively fixed once chosen.

More seriously: **look at what a third scheme would cost.** With the current design you would add a class — which is cheap — and then extend that `if` into an `if/else if` chain. And when a second operation needs the scheme, it gets its own chain.

The hierarchy is built. The branch is the thing still standing between you and the design in Figure 20.1's right panel. Chapter 21 removes it in one step.

Your StudySite Lab — Add Weighted Grading with Inheritance

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v3.0
- **Starting point**: Your committed Chapter 19 v2.6 program (nine files).

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

The Grade Calculator has always graded by points: total earned ÷ total possible. In this lab you add a second rule — **weighted categories** — using **inheritance**. By the end you will have:

1. A **category** on every `Assignment` (for example `Exam` or `Homework`), added in a way that leaves every existing call compiling unchanged.
2. A base class `GradingScheme` holding the behavior every scheme shares (a name, and the cap-and-round policy), and two **derived classes**: `PointsBased` and `Weighted`.
3. A `Gradebook` that grades each student with whichever scheme was chosen at startup, and shows the scheme in the report.
4. A startup question in `main` that picks the scheme for the session, an optional step to enter your own category weights, and a *Category* prompt for each assignment.

The menu keeps the same five options as Chapter 19.

Files in this checkpoint

File	What you do to it
`assignment.h`, `assignment.cpp`	Edit (Step 1)
`gradingscheme.h`, `gradingscheme.cpp`	Create (Steps 2–3)
`student.h`	Edit — comments only (Step 4)
`gradebook.h`, `gradebook.cpp`	Edit (Steps 5–6)
`main.cpp`	Edit (Steps 7–8)
`gradescale.h`, `gradescale.cpp`, `student.cpp`	No change — keep them open

When you are finished your project has **eleven files**, and all of them must be open in the editor when you click **Run**.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

Build it: step by step

Step 1 — Give `Assignment` a category Weighted grading needs to know which category each assignment belongs to. Add a `category` to `Assignment`, with a **default value** so that every existing call to the constructor still compiles without being edited.

The first line of every class file records the version in which that file last changed. In `assignment.h`, update it. Find:

```
// assignment.h - Grade Calculator v2.6 - Chapter 19
```

Replace it with:

```
// assignment.h - Grade Calculator v3.0 - Chapter 20
```

In `assignment.h`, find the constructor declaration:

```
    Assignment(const std::string& name, double earned, double possible, double bonus = 0.0);
```

Replace it with:

```
    Assignment(const std::string& name, double earned, double possible,
               double bonus = 0.0, const std::string& category = "Uncategorized");
```

Still in `assignment.h`, find the `bonusPoints()` accessor:

```
    double bonusPoints() const      { return bonusPoints_; }
```

Directly below it, add:

```
    const std::string& category() const { return category_; }
```

Near the bottom of the class, find the last data member:

```
    double bonusPoints_    = 0.0;
```

Directly below it, add:

```
    std::string category_  = "Uncategorized";
```

In `assignment.cpp`, update the first line. Find:

```
// assignment.cpp - Grade Calculator v2.6 - Chapter 19
```

Replace it with:

```
// assignment.cpp - Grade Calculator v3.0 - Chapter 20
```

In `assignment.cpp`, find the constructor definition:

```
Assignment::Assignment(const std::string& name, double earned, double possible, double bonus)
    : name_(name), pointsEarned_(earned), pointsPossible_(possible), bonusPoints_(bonus) {}
```

Replace it with:

```
Assignment::Assignment(const std::string& name, double earned, double possible,
                       double bonus, const std::string& category)
    : name_(name), pointsEarned_(earned), pointsPossible_(possible),
      bonusPoints_(bonus), category_(category) {}
```

Still in `assignment.cpp`, near the end of `operator<<`, find:

```
        out << "  (+" << a.bonusPoints() << " bonus)";
    }
```

Directly below it, add:

```
    out << "  [" << a.category() << "]";
```

Why: adding a parameter **with a default** at the end of a parameter list extends a function without breaking any caller — the Chapter 10 technique. `Assignment("HW1", 9, 10)` still compiles and simply gets the category `"Uncategorized"`. The default value appears only in the header declaration.

> ▸ **Run checkpoint**: Click **Run** with all nine files open. The program should compile cleanly and behave exactly like v2.6. Nothing uses the category yet.

Step 2 — Create the `GradingScheme` hierarchy (`gradingscheme.h`) A **base class** holds what every grading scheme has in common. Each **derived class** adds its own rule.

Create a new file named `gradingscheme.h` and enter this code:

```
// gradingscheme.h - Grade Calculator v3.0 - Chapter 20
#ifndef GRADINGSCHEME_H
#define GRADINGSCHEME_H

#include "assignment.h"

#include <string>
#include <vector>

/**
 * Base class for grading schemes.
 *
 * Chapter 20 compares two designs for supporting more than one grading rule:
 * a scheme flag branched on inside every calculation, or a small class
 * hierarchy. This is the hierarchy. Shared behavior - capping and rounding -
 * lives here once, so no derived class repeats it.
 *
 * LIMITATION (removed in Chapter 21): selection still happens through a single
 * branch in Gradebook, because these functions are not yet virtual. The scheme
 * is therefore fixed once the program starts.
 */
class GradingScheme {
public:
    explicit GradingScheme(const std::string& name) : name_(name) {}

    const std::string& name() const { return name_; }

protected:
    /** Applies the course cap-and-round policy. Shared by every scheme. */
    static double finalize(double rawPercentage);

private:
    std::string name_;
};

/** Total points earned divided by total points possible. */
class PointsBased : public GradingScheme {
public:
    PointsBased() : GradingScheme("Points-based") {}
    double computePercentage(const std::vector<Assignment>& work) const;
};

/** Each category scored separately, then combined by weight. */
class Weighted : public GradingScheme {
public:
    struct CategoryWeight {
        std::string name;
        double weight = 0.0;   // percentage points, should total 100
    };

    Weighted();
    explicit Weighted(const std::vector<CategoryWeight>& weights);

    double computePercentage(const std::vector<Assignment>& work) const;

    const std::vector<CategoryWeight>& weights() const { return weights_; }
    double weightTotal() const;
    bool weightsValid() const;
```

```
private:
    std::vector<CategoryWeight> weights_;
};

#endif
```

Read the three classes carefully:

- `GradingScheme` stores a `name_` and provides `finalize()`, the course's cap-and-round policy. `finalize` is **`protected`**: derived classes can call it, but code outside the hierarchy cannot. It is `static` because it does not need any one object's data.
- `class PointsBased : public GradingScheme` reads "a `PointsBased` **is a** `GradingScheme`." Its constructor passes the name `"Points-based"` up to the base-class constructor in its initializer list.
- `Weighted` adds a nested `CategoryWeight` struct (a category name and its weight) and a list of weights. It can tell you whether the weights total 100 (`weightsValid()`).
- Notice that `computePercentage` is **not** `virtual` yet. Each derived class has its own version, but the program must still decide by hand which one to call. Chapter 21 removes that limitation.

Step 3 — Implement the two schemes (`gradingscheme.cpp`) Create a new file named `gradingscheme.cpp` and enter this code:

```
// gradingscheme.cpp - Grade Calculator v3.0 - Chapter 20
#include "gradingscheme.h"

#include <algorithm>
#include <cmath>

namespace {
    const bool CAP_AT_100 = true;
}

double GradingScheme::finalize(double rawPercentage) {
    double reported = CAP_AT_100 ? std::min(rawPercentage, 100.0) : rawPercentage;
    return std::round(reported * 10.0) / 10.0;
}

double PointsBased::computePercentage(const std::vector<Assignment>& work) const {
    double earned = 0.0;
    double possible = 0.0;
    for (const Assignment& a : work) {
        earned   += a.totalEarned();
        possible += a.pointsPossible();
    }
    if (possible <= 0.0) { return 0.0; }
    return finalize(earned / possible * 100.0);
}

Weighted::Weighted() : GradingScheme("Weighted") {
    weights_ = { {"Exam", 50.0}, {"Homework", 30.0}, {"Participation", 20.0} };
}

Weighted::Weighted(const std::vector<CategoryWeight>& weights)
    : GradingScheme("Weighted"), weights_(weights) {}

double Weighted::weightTotal() const {
    double sum = 0.0;
    for (const CategoryWeight& w : weights_) { sum += w.weight; }
    return sum;
```

```
}

bool Weighted::weightsValid() const {
    return std::abs(weightTotal() - 100.0) < 0.001;
}

double Weighted::computePercentage(const std::vector<Assignment>& work) const {
    double weightedSum = 0.0;
    double weightUsed  = 0.0;

    for (const CategoryWeight& w : weights_) {
        double earned = 0.0;
        double possible = 0.0;
        for (const Assignment& a : work) {
            if (a.category() == w.name) {
                earned   += a.totalEarned();
                possible += a.pointsPossible();
            }
        }
        // A category with no graded work yet is skipped, and its weight is
        // redistributed across the categories that do have work. Otherwise a
        // student would be penalized for assignments not yet given.
        if (possible > 0.0) {
            weightedSum += (earned / possible) * w.weight;
            weightUsed  += w.weight;
        }
    }

    if (weightUsed <= 0.0) { return 0.0; }
    return finalize(weightedSum / weightUsed * 100.0);
}
```

- `PointsBased::computePercentage` is the same points math you have used since Chapter 4, now ending in `finalize(...)` from the base class.
- `Weighted::Weighted()` sets the default weights: Exam 50, Homework 30, Participation 20.
- `weightsValid()` compares the total to 100 **with a tolerance** (`< 0.001`), never with ==, because the weights are `doubles` — the floating-point trap from Chapter 6.
- `Weighted::computePercentage` scores each category separately and combines them by weight. A category with **no graded work yet is skipped**, and its weight is spread across the categories that do have work, so a student is not penalized for assignments that have not been given.

▸ **Run checkpoint**: Open both new files and click **Run**. The program should compile cleanly and still behave exactly like v2.6 — the new classes exist but nothing uses them yet.

Step 4 — Note the new role of `Student::percentage()` `Student::percentage()` still computes a points-based percentage, but the course grade will now come from a grading scheme. Record that in a comment so the next reader is not misled.

In `student.h`, update the first line. Find:

```
// student.h - Grade Calculator v2.6 - Chapter 19
```

Replace it with:

```
// student.h - Grade Calculator v3.0 - Chapter 20
```

In `student.h`, find:

```
    double totalPossible() const;
    double percentage() const;
```

Replace it with:

```
    double totalPossible() const;

    /** Points-based percentage. Kept for the report header and for sorting;
     *  the course grade now comes from a GradingScheme instead. */
    double percentage() const;
```

This is a comment-only change; `student.cpp` does not change at all.

Step 5 — Make `Gradebook` scheme-aware (`gradebook.h`) In `gradebook.h`, update the first line. Find:

```
// gradebook.h - Grade Calculator v2.6 - Chapter 19
```

Replace it with:

```
// gradebook.h - Grade Calculator v3.0 - Chapter 20
```

In `gradebook.h`, find the first project include:

```
#include "gradescale.h"
```

Directly above it, add:

```
#include "gradingscheme.h"
```

Find the constructor declaration:

```
    explicit Gradebook(const std::string& courseName = "Untitled Course");
```

Replace it with:

```
    /** Which grading rule this gradebook applies. Fixed at construction in
     *  v3.0; Chapter 21 makes it changeable at any time. */
    enum class Scheme { Points, Weighted };

    explicit Gradebook(const std::string& courseName = "Untitled Course",
                       Scheme scheme = Scheme::Points);

    /** The course grade for one student under the selected scheme. */
    double percentageFor(const Student& s) const;
    std::string schemeName() const;
    const Weighted& weightedScheme() const { return weighted_; }
    void setWeights(const std::vector<Weighted::CategoryWeight>& w) { weighted_ = Weighted(w); }
```

At the bottom of the class, find:

```
    int nextId_ = 1001;
};
```

Replace it with:

```
    int nextId_ = 1001;

    // Both schemes are held as concrete objects, and a single branch in
    // percentageFor() chooses between them. That branch is the limitation
    // Chapter 21 removes with virtual functions.
    Scheme scheme_ = Scheme::Points;
    PointsBased points_;
```

```
    Weighted    weighted_;
};
```

What you added:

- `enum class Scheme { Points, Weighted };` names the two choices. Because it is declared inside `Gradebook`, its full name is `Gradebook::Scheme`.
- The constructor takes a second parameter, the scheme, **defaulting to `Scheme::Points`** — so `Gradebook book("Programming Fundamentals");` still compiles.
- `percentageFor(s)` returns a student's course grade under the selected scheme; `schemeName()` returns its display name.
- `weightedScheme()` and `setWeights()` let `main` read and replace the weights.
- The `Gradebook` **holds one object of each scheme** (`points_` and `weighted_`) plus the `scheme_` flag that says which to use.

Step 6 — Grade through the selected scheme (`gradebook.cpp`) In `gradebook.cpp`, update the first line to name this version. Find:

```
// gradebook.cpp - Grade Calculator v2.6 - Chapter 19
```

Replace it with:

```
// gradebook.cpp - Grade Calculator v3.0 - Chapter 20
```

In `gradebook.cpp`, find the constructor:

```
Gradebook::Gradebook(const std::string& courseName) : courseName_(courseName) {}
```

Replace it with:

```
Gradebook::Gradebook(const std::string& courseName, Scheme scheme)
    : courseName_(courseName), scheme_(scheme) {}

double Gradebook::percentageFor(const Student& s) const {
    // The single branch. One place today; one place per operation as soon as
    // a third scheme or a second calculation is added. See Section 20.2.
    if (scheme_ == Scheme::Weighted) {
        return weighted_.computePercentage(s.assignments());
    }
    return points_.computePercentage(s.assignments());
}

std::string Gradebook::schemeName() const {
    return scheme_ == Scheme::Weighted ? weighted_.name() : points_.name();
}
```

In `classAverage()`, find:

```
    for (const Student& s : roster_) { sum += s.percentage(); }
```

Replace it with:

```
    for (const Student& s : roster_) { sum += percentageFor(s); }
```

In `operator<<`, find the line that prints the scale:

```
    out << "  " << book.scale() << "\n";
```

Directly below it, add:

```
    out << "  Grading: " << book.schemeName() << "\n";
```

Further down in `operator<<`, inside the loop, find:

```
        out << s << "   " << book.scale().letterFor(s.percentage()) << "\n";
```

Replace it with:

```
        double pct = book.percentageFor(s);
        out << std::fixed << std::setprecision(1)
            << std::left << std::setw(6) << s.id()
            << std::setw(20) << s.name()
            << std::right << std::setw(8) << pct << "%   "
            << book.scale().letterFor(pct) << "\n";
```

- `percentageFor` contains **the single branch**: if the scheme is `Weighted`, ask `weighted_`; otherwise ask `points_`. Every grade in the program now flows through this one function.
- The class average and every report row now use `percentageFor(s)` instead of `s.percentage()`, so they honor the chosen scheme.
- The report row is now written out field by field, because the percentage printed must be the *scheme's* percentage, which `Student`'s own `operator<<` knows nothing about.

  ▸ **Run checkpoint**: Click **Run**. The program should compile cleanly. Because the scheme defaults to points, it still behaves like v2.6 — except that the report now shows a `Grading: Points-based` line.

Step 7 — Update the header comment in `main.cpp` At the top of `main.cpp`, find the header comment:

```
// ============================================================================
//  Grade Calculator  v2.6                                        Chapter 19
// ----------------------------------------------------------------------------
//  Change  : One header/implementation pair per class. Reports print through
//            operator<<, so a whole Gradebook is written with a single <<.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp assignment.cpp gradescale.cpp
//            student.cpp gradebook.cpp -o gradecalc
// ============================================================================
```

Replace it with:

```
// ============================================================================
//  Grade Calculator  v3.0                                        Chapter 20
// ----------------------------------------------------------------------------
//  Change  : Weighted-category grading works. A GradingScheme base class with
//            PointsBased and Weighted derived classes replaces the single
//            hard-coded rule. Assignments now carry a category.
//  Chooses : the scheme is selected at STARTUP and fixed for the session.
//  Limit   : switching schemes requires restarting, because the selection is
//            a branch in Gradebook::percentageFor rather than dynamic
//            dispatch. Chapter 21 removes this.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp assignment.cpp gradescale.cpp
//            student.cpp gradingscheme.cpp gradebook.cpp -o gradecalc
// ============================================================================
```

Step 8 — Choose a scheme at startup, and read each assignment's category `main` now asks for a scheme before the menu starts. If the user picks weighted grading, it offers to replace the default weights. The *Add student* loop also asks for a category.

Find the start of `main`:

```
int main() {
```

Delete that line and everything below it, all the way to the final closing `}` at the very end of the file.

In its place, enter the new `main`:

```
int main() {
    std::cout << "=== GRADE CALCULATOR v3.0 (two grading schemes) ===\n\n";
    std::cout << "Grading scheme for this session:\n";
    std::cout << "  1) Points-based   2) Weighted by category\n";
    std::string choice = readLine("Choice: ");

    Gradebook::Scheme scheme = (!choice.empty() && choice[0] == '2')
                             ? Gradebook::Scheme::Weighted
                             : Gradebook::Scheme::Points;
    Gradebook book("Programming Fundamentals", scheme);

    if (scheme == Gradebook::Scheme::Weighted) {
        std::cout << "\nDefault weights: Exam 50, Homework 30, Participation 20.\n";
        if (readLine("Enter your own weights? (y/n): ").substr(0, 1) == "y") {
            std::vector<Weighted::CategoryWeight> weights;
            while (true) {
                std::string name = readLine("  Category name (or 'done'): ");
                if (name == "done" || name.empty()) { break; }
                weights.push_back({name, readNonNegative("    Weight (percent): ")});
            }
            book.setWeights(weights);
            if (!book.weightedScheme().weightsValid()) {
                std::cout << "  Note: weights total "
                          << book.weightedScheme().weightTotal()
                          << ", not 100. Categories are scaled proportionally.\n";
            }
        }
        std::cout << "\nWhen adding assignments, use the category names above.\n";
    }

    bool running = true;
    while (running) {
        std::cout << "\n1) Add student  2) Report  3) Sort by grade  "
                     "4) Custom scale  5) Quit\nChoice: ";
        std::string line;
        if (!std::getline(std::cin, line)) { break; }
        char c = line.empty() ? '?' : line[0];

        if (c == '1') {
            std::string name = readLine("  Student name: ");
            if (name.empty()) { continue; }
            Student s(name, book.claimNextId());
            while (true) {
                std::string an = readLine("    Assignment name (or 'done'): ");
                if (an == "done" || an.empty()) { break; }
                double earned   = readNonNegative("      Points earned  : ");
                double possible = readNonNegative("      Points possible: ");
                double bonus    = readNonNegative("      Bonus points   : ");
                std::string cat = readLine("      Category       : ");
                if (cat.empty()) { cat = "Uncategorized"; }
                s.addAssignment(Assignment(an, earned, possible, bonus, cat));
            }
            book.addStudent(s);
            std::cout << "  Added " << s.name() << " as ID " << s.id() << ".\n";
```

```
        } else if (c == '2') {
            std::cout << book << "\n";
        } else if (c == '3') {
            book.sortByPercentage();
            std::cout << "  Sorted.\n";
        } else if (c == '4') {
            std::vector<GradeScale::Tier> requested;
            std::cout << "  Enter tiers highest first, 'done' to finish.\n";
            while (true) {
                std::string lt = readLine("    Letter (or 'done'): ");
                if (lt == "done" || lt.empty()) { break; }
                requested.push_back({readNonNegative("      Minimum percentage: "), lt[0]});
            }
            book.setScale(GradeScale(requested));
            std::cout << "  " << book.scale() << "\n";
        } else if (c == '5') {
            running = false;
        } else {
            std::cout << "  Please enter 1-5.\n";
        }
    }
    std::cout << "\nGoodbye.\n";
    return 0;
}
```

What changed, from top to bottom:

- **Choosing the scheme.** The answer is read into a `std::string` named `choice`, and a conditional expression turns it into a `Gradebook::Scheme` value, which is passed to the `Gradebook` constructor. The scheme is fixed for the session.
- **Optional custom weights.** For weighted grading, the user can enter their own categories and weights. If they do not total 100, the program prints a note — it does not reject them; categories are scaled proportionally.
- **The menu variable is renamed `c`.** The name `choice` is now taken by the scheme answer, so the menu choice uses `char c`. Every `if` in the loop compares `c`.
- **Add student** asks for a `Category` after the bonus. Pressing Enter with no category stores `"Uncategorized"`. The category is passed as the fifth constructor argument.
- **Sort** prints `Sorted.`, and **Custom scale** reads each letter into `lt` and passes `readNonNegative(...)` straight into the tier — the same steps as before, written more compactly.

▸ **Run checkpoint:** Make sure all eleven files are open and click **Run**. The program should compile with no errors and no warnings and open with the scheme question.

Check your work

Run 1 — points-based. Answer the scheme question with `1`. Add `Ada` with two assignments: `Exam 1` (earned `90`, possible `100`, bonus `0`, category `Exam`) and `HW1` (earned `10`, possible `10`, bonus `0`, category `Homework`). Type `done`, then choose `2`. The report should include:

```
  Grading: Points-based
====================================
1001  Ada                    90.9%   A
```

Check: (90 + 10) ÷ (100 + 10) = 100 ÷ 110 = 90.9%. Choose `5` to quit.

Run 2 — weighted, default weights. Run again, choose `2`, and answer `n` to custom weights. Add the same two assignments for `Ada` and choose `2`:

```
  Grading: Weighted
====================================
1001  Ada                     93.8%   A
```

Check: Exam 0.90 × 50 = 45; Homework 1.00 × 30 = 30. Participation has no work, so its 20 is skipped: 75 ÷ 80 = 93.75%, rounded to 93.8%. **The same scores, two different grades** — that is the point of choosing a scheme.

Run 3 — weights that do not total 100. Run again, choose `2`, answer `y`, and enter `Exam 60`, `Homework 30`, then `done`. You should see:

```
Note: weights total 90, not 100. Categories are scaled proportionally.
```

Add the same two assignments for `Ada` and report: she shows `93.3%` ((0.90 × 60 + 1.00 × 30) ÷ 90).

Then confirm:

- ☐ All eleven files build together with no errors and no warnings.
- ☐ `PointsBased` and `Weighted` both inherit `public`ly from `GradingScheme`, and neither repeats the cap-and-round code — both call `finalize`.
- ☐ `gradescale.h`, `gradescale.cpp`, and `student.cpp` are unchanged from Chapter 19.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click every file in the project (`main.cpp` and the eight class files) to open it. You will create two new files during the lab. Confirm the editor shows the expected file paths before editing.
4. Work through Steps 1–8 above in StudySite's internal editor, in order, clicking **Run** at each ▸ Run checkpoint.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — all eleven files: `main.cpp`, and the `.h` and `.cpp` files for `Assignment`, `GradeScale`, `Student`, `Gradebook`, and `GradingScheme`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 20 Grade Calculator v3.0**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 20 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Two schemes, one hierarchy

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <string>
#include <vector>

class Assignment {
public:
    Assignment(const std::string& n, double e, double p,
               double b = 0.0, const std::string& c = "Uncategorized")
        : name_(n), earned_(e), possible_(p), bonus_(b), category_(c) {}
    double totalEarned() const { return earned_ + bonus_; }
    double pointsPossible() const { return possible_; }
    const std::string& category() const { return category_; }
private:
    std::string name_;
    double earned_ = 0.0;
    double possible_ = 0.0;
    double bonus_ = 0.0;
    std::string category_;
};

class GradingScheme {
public:
    explicit GradingScheme(const std::string& n) : name_(n) {}
    const std::string& name() const { return name_; }
protected:
    static double finalize(double raw) {
        return std::round(std::min(raw, 100.0) * 10.0) / 10.0;
    }
private:
    std::string name_;
};

class PointsBased : public GradingScheme {
public:
    PointsBased() : GradingScheme("Points-based") {}
    double computePercentage(const std::vector<Assignment>& w) const {
        double e = 0.0;
        double p = 0.0;
        for (const Assignment& a : w) { e += a.totalEarned(); p += a.pointsPossible(); }
        return p <= 0.0 ? 0.0 : finalize(e / p * 100.0);
    }
};

class Weighted : public GradingScheme {
public:
```

```
    struct CategoryWeight { std::string name; double weight = 0.0; };
    Weighted() : GradingScheme("Weighted") {
        weights_ = {{"Exam", 50.0}, {"Homework", 30.0}, {"Participation", 20.0}};
    }
    double computePercentage(const std::vector<Assignment>& w) const {
        double sum = 0.0;
        double used = 0.0;
        for (const CategoryWeight& cw : weights_) {
            double e = 0.0;
            double p = 0.0;
            for (const Assignment& a : w) {
                if (a.category() == cw.name) { e += a.totalEarned(); p += a.pointsPossible(); }
            }
            if (p > 0.0) { sum += (e / p) * cw.weight; used += cw.weight; }
        }
        return used <= 0.0 ? 0.0 : finalize(sum / used * 100.0);
    }
protected:
    std::vector<CategoryWeight> weights_;
};

int main() {
    std::vector<Assignment> work = {
        {"Exam 1", 90, 100, 0, "Exam"},
        {"HW1", 10, 10, 0, "Homework"},
        {"HW2", 5, 10, 0, "Homework"}
    };
    PointsBased pb;
    Weighted wt;
    std::cout << "points-based: " << pb.computePercentage(work) << "%\n";
    std::cout << "weighted:     " << wt.computePercentage(work) << "%\n";
    return 0;
}
```

Expected output:

```
points-based: 87.5%
weighted:     84.4%
```

Try: Add a participation assignment worth 10/10. Which percentage changes, and by how much? Predict before running.

2. Constructors run base-first

```
#include <iostream>
#include <string>

class Base {
public:
    explicit Base(const std::string& n) {
        std::cout << "Base constructor for " << n << "\n";
    }
};

class Derived : public Base {
public:
    Derived() : Base("derived object") {
        std::cout << "Derived constructor body\n";
    }
```

```
};

int main() {
    Derived d;
    return 0;
}
```

Expected output:

```
Base constructor for derived object
Derived constructor body
```

Try: Remove `: Base("derived object")` and read the error. What does that tell you about default constructors in base classes?

3. `protected` versus `private`

```
#include <iostream>

class Base {
public:
    Base() : pub_(1), prot_(2), priv_(3) {}
    int pub_;
protected:
    int prot_;
private:
    int priv_;
};

class Derived : public Base {
public:
    void show() {
        std::cout << "pub_  " << pub_  << "\n";
        std::cout << "prot_ " << prot_ << "\n";
        // std::cout << priv_;   // uncomment this line
    }
};

int main() {
    Derived d;
    d.show();
    return 0;
}
```

Expected output:

```
pub_  1
prot_ 2
```

Try: Uncomment the `priv_` line and read the error. Then, in `main`, try `d.prot_`. Why does that also fail?

4. Shared behavior in the base

```
#include <cmath>
#include <iostream>

class Scheme {
public:
    virtual ~Scheme() = default;
```

```
protected:
    static double round1(double x) { return std::round(x * 10.0) / 10.0; }
};

class A : public Scheme {
public:
    double compute() const { return round1(87.4567); }
};

class B : public Scheme {
public:
    double compute() const { return round1(92.9876); }
};

int main() {
    A a;
    B b;
    std::cout << a.compute() << "\n";
    std::cout << b.compute() << "\n";
    return 0;
}
```

Expected output:

```
87.5
93
```

Try: Change `round1` to round to two decimal places. How many classes did you edit? What would the answer be if each class had its own copy?

5. Is-a or has-a?

For each pair, say whether inheritance or composition is right, and why:

- `Car` and `Engine`
- `Square` and `Rectangle`
- `Gradebook` and `GradeScale`
- `WeightedDropLowest` and `Weighted`
- `Student` and `Assignment`
- `Manager` and `Employee`
- `Playlist` and `Song`

6. Cost a change

For each design in Section 20.2, count the edits needed to add a third grading scheme:

- Design A, with the branch in one function
- Design A, with branches in three functions
- Design B, the hierarchy

Now do the same for adding a *fourth* scheme. Which numbers grow?

7. Reason about inheritance

- What does `public` inheritance mean, in one sentence?
- Why must a derived constructor call a base constructor before its own body runs?
- Why prefer `private` data to `protected`?

- What is the difference between overloading and overriding?
- Give one specific case where inheriting to reuse code is the wrong choice.

Summary

- **Weighted-category grading** scores each category separately as a fraction of its own points, then combines by weight. It is a different computation from points-based grading, not a variation.
- **Two running totals cannot express weighting**, which is why Course I could not do this.
- A **flag and branches** works and costs: the branch spreads, a third variant touches every branch, and the rules cannot be tested alone.
- A **class hierarchy** costs more up front and makes a third variant a new class with nothing else edited.
- **Inheritance expresses "is-a."** Say it aloud; if it sounds wrong, use **composition** ("has-a").
- A **derived class** inherits everything from its **base**. `public` inheritance means "is-a".
- Derived constructors call a base constructor in the **initializer list**. **Construction proceeds base-outward**.
- **`protected`** is reachable by derived classes. Prefer `private` for data.
- **Overriding** replaces a base function with the same signature; **overloading** uses a different signature.
- **Prefer composition to inheritance**. Inherit when the relationship is genuine *and* code must treat several types uniformly.
- **v3.0 works and still selects with a branch**, because the functions are not yet `virtual`. That limitation is disclosed, and Chapter 21 removes it.
- The classes encapsulated in Chapters 18 and 19 **needed no changes at all** — the measurable return on that work.

Key Terms

base class — a class from which another inherits.

category weight — the percentage of a final grade a category contributes.

composition — holding an object as a member, expressing "has-a".

derived class — a class inheriting from another.

has-a — a relationship expressed by composition.

inheritance — deriving a new class from an existing one.

is-a — a relationship expressed by inheritance.

multilevel hierarchy — a chain in which a derived class is itself a base.

overriding — replacing a base class function with one of the same signature.

protected — an access level permitting the class and its derived classes.

Next: Chapter 21 removes the branch. `GradingScheme` becomes abstract, `Gradebook` holds a pointer to it and stops asking which scheme it has, and the user can switch schemes mid-session with every grade recomputed. Then you add a third scheme and confirm nothing else changes. Grade Calculator v3.1.

Chapter 21 — Polymorphism

Learning Objectives

When you finish this chapter you will be able to:

- Explain what polymorphism is and the problem it solves. *(SLO 2.3)*
- Declare `virtual` functions and explain dynamic binding. *(SLO 2.3)*
- Use base class pointers and references to hold derived objects. *(SLO 2.3)*
- Declare an abstract class with a pure virtual function. *(SLO 2.3)*
- Explain why a base class needs a virtual destructor. *(SLO 2.3)*
- Recognize and avoid object slicing. *(SLO 2.3)*
- Store mixed types in a polymorphic collection. *(SLO 2.3, 2.6)*
- Explain the costs of polymorphism honestly. *(SLO 2.8)*
- Build Grade Calculator v3.1 — schemes switchable at run time, and a third one added without editing anything.

21.1 The Problem Polymorphism Solves

Chapter 20 built the hierarchy and left one thing standing:

```
double Gradebook::percentageFor(const Student& s) const {
    if (scheme_ == Scheme::Weighted) {
        return weighted_.computePercentage(s.assignments());
    }
    return points_.computePercentage(s.assignments());
}
```

That branch is the problem. `Gradebook` holds both scheme objects, knows both types by name, and selects between them. Add a third scheme and this becomes a chain. Add a second operation needing the scheme and it gets its own chain.

What you want is for `Gradebook` to hold one scheme and ask it for a percentage, without knowing which kind it has.

That is **polymorphism** — from the Greek for "many forms." One call, many possible behaviors, decided by the object rather than by the calling code.

21.2 Virtual Functions

The obstacle in v3.0 was that `computePercentage` was an ordinary function. With ordinary functions, **the type of the variable decides which one runs**, and that is settled at compile time. This is **static binding**.

Mark a function `virtual` and the rule changes:

```
class GradingScheme {
public:
    virtual double computePercentage(const std::vector<Assignment>& work) const;
};

class Weighted : public GradingScheme {
public:
    double computePercentage(const std::vector<Assignment>& work) const override;
};
```

Now **the type of the object decides**, at run time. This is **dynamic binding**, or dynamic dispatch.

The `override` keyword is not required, and you should use it anyway. It asks the compiler to check that you really are overriding something. Misspell the name or get a parameter type wrong and, without `override`, you have quietly written a *new* function that never runs — a silent logic error. With `override`, the compiler stops you.

21.3 Base Class Pointers and References

Dynamic binding needs the object reached through a **base class pointer or reference**:

```
void show(const Shape& s) {
    std::cout << s.describe() << "\n";     // which describe()? depends on the object
}

Circle c;
Square q;
show(c);      // circle
show(q);      // square
```

```
through references:
  circle
  square
```

`show` has one parameter type and produces different behavior for different objects. It has never heard of `Circle` or `Square`.

A derived object can always be used where a base is expected. A `Circle` is a `Shape`, so a `Shape&` may refer to one. The reverse is not true and does not compile.

21.4 Object Slicing

Here is a trap worth meeting deliberately, because it is silent.

Dynamic binding requires a pointer or a reference. Pass a derived object **by value** and something destructive happens:

```
void byValue(Shape s)            { std::cout << s.name() << "\n"; }
void byReference(const Shape& s) { std::cout << s.name() << "\n"; }

Circle c;
byValue(c);
byReference(c);
```

```
  by value:     shape
  by reference: circle
```

Passing by value copies **only the `Shape` part** of the `Circle`. The derived portion is sliced off and discarded, leaving a genuine `Shape` object — so `Shape::name` runs. No error, no warning, wrong answer.

This is called **object slicing**, and it is the single most common polymorphism bug.

The rule: with polymorphism, always use a reference or a pointer, never a value. Chapter 10 Section 10.7 told you to pass large objects by `const&` for efficiency. Here it is a correctness requirement.

21.5 Abstract Classes and Pure Virtual Functions

`GradingScheme` has no sensible default way to compute a percentage. Every scheme must supply its own.

Say so with a **pure virtual function** — `= 0` instead of a body:

```
class GradingScheme {
public:
    explicit GradingScheme(const std::string& name) : name_(name) {}
    virtual ~GradingScheme() = default;

    /** Computes the course percentage. Every scheme defines its own rule. */
    virtual double computePercentage(const std::vector<Assignment>& work) const = 0;

    const std::string& name() const { return name_; }

protected:
    static double finalize(double rawPercentage);

private:
    std::string name_;
};
```

A class with at least one pure virtual function is **abstract**, and cannot be instantiated:

```
GradingScheme s;      // error: cannot instantiate abstract class
```

That refusal is a feature. "A grading scheme" with no rule is not a meaningful object, and the compiler now enforces it. Any derived class failing to supply `computePercentage` is also abstract — so you cannot forget.

An abstract class defines an **interface**: a promise about what every scheme can do, with the how left to each.

Note that it may still hold data and non-virtual functions. `GradingScheme` has a `name_` and a shared `finalize`, inherited by every scheme. Abstract does not mean empty.

21.6 Virtual Destructors

A base class used polymorphically must have a virtual destructor.

```
virtual ~GradingScheme() = default;
```

Here is why. When you delete through a base pointer:

```
GradingScheme* scheme = new Weighted();
delete scheme;
```

Without a virtual destructor, only `~GradingScheme` runs. `~Weighted` never does, so anything `Weighted` owned — its vector of category weights — is never released. That is a memory leak, and Chapter 22 shows how to see one.

The rule is absolute: if a class has any virtual function, give it a virtual destructor. `= default` costs one line and asks the compiler to generate the right one.

21.7 Polymorphic Collections

Virtual functions make a genuinely new thing possible: **one collection holding several different types**, each behaving as itself.

```
std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Circle>());
shapes.push_back(std::make_unique<Square>());

for (const auto& s : shapes) {
    std::cout << s->describe() << "\n";
}
```

```
through a collection of base pointers:
  circle
  square
```

One loop, one call, different behavior per element. **The loop has never heard of `Circle` or `Square`** — add a `Triangle` and the loop is unchanged.

Note that the elements are pointers, not values. A `std::vector<Shape>` would slice every element on insertion. `std::unique_ptr` is Chapter 22's subject; for now, read `std::make_unique<Circle>()` as "create a Circle and hand over ownership."

21.8 Runtime Type Information

C++ can ask what a pointer actually points to:

```
if (Weighted* w = dynamic_cast<Weighted*>(scheme)) {
    // scheme really is a Weighted
}
```

`dynamic_cast` returns `nullptr` when the object is not of the requested type.

Needing this is usually a design smell. If you find yourself writing a chain of `dynamic_cast` tests, you have reinvented the branch that polymorphism was supposed to remove — with worse syntax. The right question is:

what behavior did you want, and could it be a virtual function instead?

It is covered so you recognize it, and so you recognize what it usually indicates.

21.9 I/O Streams as a Case Study

You have used polymorphism since Chapter 15 without noticing.

`std::ostream` is a base class. `std::ofstream` and `std::ostringstream` derive from it. When you write:

```
std::ostream& operator<<(std::ostream& out, const Student& s);
```

that one function works with `std::cout`, with a file, and with a string stream — because the parameter is a base class **reference** and the actual output behavior is virtual.

Chapter 19's promise that `out << book` would write to a file unchanged was polymorphism, working before it was named.

21.10 The Costs

An honest accounting, because polymorphism is not free.

A small run-time cost. A virtual call is an indirect call — the object carries a hidden pointer to a table of its functions, and the call goes through it. This is typically a few nanoseconds and is irrelevant unless the call is in a very hot loop.

A small memory cost. Each polymorphic object carries that hidden pointer, usually 8 bytes.

Real conceptual cost. Reading `scheme_->computePercentage(...)` does not tell you which code runs. You have to know what the pointer holds. The debugger from Chapter 16 answers this — pause and inspect the object's type — and it is a real cost when reading unfamiliar code.

Design cost. A hierarchy is a commitment. Getting the base class interface wrong is expensive to change once several classes depend on it.

When it is not worth it. Two variants that will never become three. A branch confined to one function that no other operation needs. Performance-critical inner loops. Chapter 20 Section 20.2's flag design is genuinely right in those cases.

When it is worth it. Several variants, growing. Behavior that must be selected at run time. Code that should not need editing when a variant is added. The Grade Calculator is all three.

Common Errors and Warnings

What you see	Cause	Fix
The base function runs, not the derived one	Function not `virtual`	Add `virtual` in the base
The base function runs, and it *is* virtual	**Object slicing** — passed by value	Take a reference or pointer

What you see	Cause	Fix
`error: cannot declare variable to be of abstract type`	Instantiating a class with a pure virtual	Instantiate a derived class
`error: 'computePercentage' marked 'override' but does not override`	Signature mismatch — often a missing `const`	Match the base exactly
Memory leaks when deleting through a base pointer	No virtual destructor	Add `virtual ~Base() = default;`
A derived class is unexpectedly abstract	A pure virtual was not implemented	Implement it, or check the signature
`error: no matching function` on `push_back`	Storing objects, not pointers	Store `std::unique_ptr<Base>`
Chains of `dynamic_cast`	Reinventing the branch	Make the behavior virtual instead

---

Design Notes

Always use `override`. It turns a silent logic error into a compile error.

Always give a polymorphic base a virtual destructor.

Never pass polymorphic objects by value. Reference or pointer, always.

Make a function pure virtual when no sensible default exists. The compiler then enforces that every derived class supplies one.

A chain of `dynamic_cast` means you want a virtual function.

Grade Calculator v3.1 — Two Schemes, Chosen at Run Time

What v3.1 does

`GradingScheme` becomes abstract. `Gradebook` holds a `GradingScheme*` and **the branch from v3.0 is gone.** The user can switch schemes mid-session and every grade recomputes. A third scheme is added as proof.

The abstract base

```
/**
 * Abstract base class for grading schemes.
 *
 * computePercentage is pure virtual, so GradingScheme cannot be instantiated
 * and every concrete scheme must supply a rule. Gradebook holds a pointer to
 * this type and never asks which scheme it has - the branch from v3.0 is gone.
 *
 * The destructor is virtual because Gradebook deletes through a base pointer.
 * Without it, only the base part of the object would be destroyed.
 *
 * OWNERSHIP NOTE (revisited in Chapter 22): Gradebook owns a raw pointer.
 * That is correct here, but only because Gradebook also writes a destructor,
 * deletes the old scheme before replacing it, and disables copying. Miss any
 * one of those three and the program leaks or double-frees. Chapter 22 shows
 * how to get the same behavior with none of that hand-written ceremony.
 */
class GradingScheme {
public:
```

```
    explicit GradingScheme(const std::string& name) : name_(name) {}
    virtual ~GradingScheme() = default;

    virtual double computePercentage(const std::vector<Assignment>& work) const = 0;

    const std::string& name() const { return name_; }

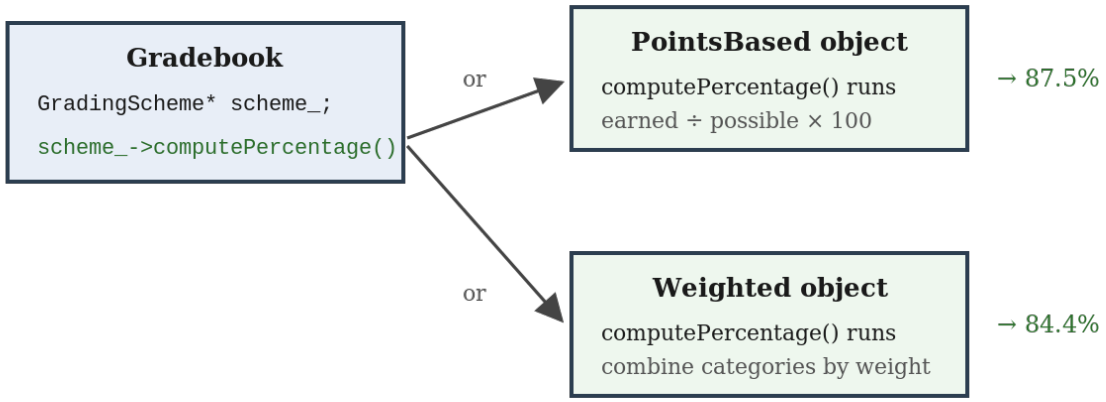
protected:
    static double finalize(double rawPercentage);

private:
    std::string name_;
};
```

Gradebook stops asking

```
double Gradebook::percentageFor(const Student& s) const {
    // Dynamic dispatch: the object decides which computePercentage runs.
    return scheme_->computePercentage(s.assignments());
}

std::string Gradebook::schemeName() const {
    return scheme_->name();
}
```



Gradebook contains no branch and never asks which kind it holds.
The object decides. That is dynamic dispatch.

Figure 16: A Gradebook box holding a scheme pointer, with arrows to a PointsBased object and a Weighted object below, showing that the same call reaches different code depending on which object the pointer refers to.

Figure 21.1 — How a call through a base pointer reaches the right function.

Description of Figure 21.1. A `Gradebook` holds a pointer `scheme_` of type `GradingScheme*`. The call `scheme_->computePercentage()` follows that pointer. When the pointer refers to a `PointsBased` object, the points-based rule runs — earned ÷ possible × 100 — producing 87.5%. When it refers to a `Weighted` object, the weighted rule runs, combining categories by weight, producing 84.4%. `Gradebook` contains no branch and never asks which kind it holds. The object decides.

Ownership, done by hand

Since `Gradebook` owns a raw pointer, three pieces of bookkeeping are required and all three are present:

```
Gradebook::Gradebook(const std::string& courseName)
    : courseName_(courseName), scheme_(new PointsBased()) {}

Gradebook::~Gradebook() {
    delete scheme_;                    // 1. release on destruction
}

void Gradebook::setScheme(GradingScheme* scheme) {
    if (scheme == nullptr || scheme == scheme_) { return; }
    delete scheme_;                    // 2. release before replacing
    scheme_ = scheme;
}
```

```
// 3. disable copying — two Gradebooks must not delete the same scheme
Gradebook(const Gradebook&) = delete;
Gradebook& operator=(const Gradebook&) = delete;
```

v3.1 does not leak. It is correct because all three were remembered. Chapter 22 is about what happens when one is not.

A third scheme, proving the design

```
/**
 * Proof that the design is open to extension: a third scheme.
 *
 * Adding this class required NO change to Gradebook, Student, GradeScale,
 * Assignment, or the report code. Compare with the flag-based design of
 * Section 20.2, where a third scheme means editing every branch.
 */
class WeightedDropLowest : public Weighted {
public:
    WeightedDropLowest();
    explicit WeightedDropLowest(const std::vector<CategoryWeight>& weights);
    double computePercentage(const std::vector<Assignment>& work) const override;
};
```

It drops each category's weakest item before scoring the category. Recall from Chapter 20 Section 20.6 that `Weighted::weights_` was made `protected` specifically so this class could reach it.

Expected output

The same student — `Exam 1` 90/100 in Exam, `HW1` 10/10 and `HW2` 5/10 in Homework — with the scheme switched twice without re-entering anything:

```
  Grading: Points-based
1001  Ada                    87.5%   B

  Grading: Weighted
1001  Ada                    84.4%   B

  Grading: Weighted
1001  Ada                    93.8%   A
```

Three schemes, three defensible grades, from identical data.

The third figure is worth checking. Dropping HW2 (5/10, the weaker homework) leaves homework at 10/10 =

1.00 × 30 = 30, and exams at 0.90 × 50 = 45. That is 75 out of the 80 weight used, giving 93.75%, rounded to 93.8%. **Dropping one assignment moved Ada from a B to an A** — which is why drop-lowest policies matter and why the arithmetic has to be right.

The one place that names concrete types

```
void chooseScheme(Gradebook& book) {
    std::string c = readLine("  Choice: ");
    if      (!c.empty() && c[0] == '2') { book.setScheme(new Weighted()); }
    else if (!c.empty() && c[0] == '3') { book.setScheme(new WeightedDropLowest()); }
    else                                { book.setScheme(new PointsBased()); }
    std::cout << "  Grading is now: " << book.schemeName() << "\n";
}
```

There is still a branch — but look at where it is and what it does. It runs **once**, when the user chooses, and its only job is to construct the right object. It does not compute anything.

That is the difference. In v3.0 the branch was inside the *calculation*, so every calculation needed one. Here it is inside the *creation*, so there is exactly one, forever. Everything downstream sees only `GradingScheme`.

What did not change

File	Changes for v3.1
`gradescale.h` / `.cpp`	**none**
`student.h` / `.cpp`	**none**
`assignment.h` / `.cpp`	**none**
Report `operator<<`	**none**

Adding a third grading scheme to a working application required **one new class and no edits to anything else**. That is the claim Chapter 20 Section 20.2 made about Design B, now verifiable with `diff`.

Your StudySite Lab — Select Schemes with Polymorphism

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v3.1
- **Starting point**: Your committed Chapter 20 v3.0 program (eleven files).

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

In v3.0 the grading scheme was chosen once at startup, and `Gradebook` used an `if` to decide which scheme's `computePercentage` to call. In this lab you make the scheme **polymorphic**. By the end you will have:

1. An **abstract** `GradingScheme` base class with a **pure virtual** `computePercentage` and a **virtual destructor**.
2. A third scheme, `WeightedDropLowest`, that drops the weakest assignment in each category — added **without changing** `Gradebook`, `Student`, `GradeScale`, `Assignment`, or the report code.
3. A `Gradebook` that holds a single `GradingScheme*` and calls through it. The `if` is gone, and because the `Gradebook` owns that pointer, it manages the memory correctly by hand: a destructor, a `delete` before replacing, and copying disabled.

4. A `chooseScheme` function and a new menu option, **Change scheme**, so the user can switch schemes mid-session and every grade recomputes immediately.

What is different about v3.1

- **No startup weights prompt.** v3.0 let you type custom category weights at startup. v3.1 removes that prompt to keep the focus on polymorphism: `Weighted` and `WeightedDropLowest` use the default weights (Exam 50, Homework 30, Participation 20). Custom weights come back in Chapter 24.
- **The menu grows to six options**: Add student, Report, Sort, Custom scale, **Change scheme** (new), and Quit (now option 6).

Files in this checkpoint

File	What you do to it
`gradingscheme.h`, `gradingscheme.cpp`	Edit (Steps 1–2)
`gradebook.h`, `gradebook.cpp`	Edit (Steps 3–4)
`main.cpp`	Edit (Steps 5–7)
`assignment.*`, `gradescale.*`, `student.*`	**No change** — keep them open

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

Build it: step by step

Step 1 — Make `GradingScheme` abstract (`gradingscheme.h`) In `gradingscheme.h`, find the first line:

```
// gradingscheme.h - Grade Calculator v3.0 - Chapter 20
```

Replace it with:

```
// gradingscheme.h - Grade Calculator v3.1 - Chapter 21
```

Next, inside the comment block above `class GradingScheme`, find these lines:

```
 * Base class for grading schemes.
 *
 * Chapter 20 compares two designs for supporting more than one grading rule:
 * a scheme flag branched on inside every calculation, or a small class
 * hierarchy. This is the hierarchy. Shared behavior - capping and rounding -
 * lives here once, so no derived class repeats it.
 *
 * LIMITATION (removed in Chapter 21): selection still happens through a single
 * branch in Gradebook, because these functions are not yet virtual. The scheme
 * is therefore fixed once the program starts.
```

Replace it with:

```
 * Abstract base class for grading schemes.
 *
 * computePercentage is pure virtual, so GradingScheme cannot be instantiated
 * and every concrete scheme must supply a rule. Gradebook holds a pointer to
 * this type and never asks which scheme it has - the branch from v3.0 is gone.
 *
 * The destructor is virtual because Gradebook deletes through a base pointer.
 * Without it, only the base part of the object would be destroyed.
 *
 * OWNERSHIP NOTE (revisited in Chapter 22): Gradebook owns a raw pointer.
 * That is correct here, but only because Gradebook also writes a destructor,
 * deletes the old scheme before replacing it, and disables copying. Miss any
 * one of those three and the program leaks or double-frees. Chapter 22 shows
 * how to get the same behavior with none of that hand-written ceremony.
```

Inside `class GradingScheme`, find the constructor:

```
    explicit GradingScheme(const std::string& name) : name_(name) {}
```

Directly below it, add:

```
    virtual ~GradingScheme() = default;

    /** Computes the course percentage. Every scheme defines its own rule. */
    virtual double computePercentage(const std::vector<Assignment>& work) const = 0;
```

These two lines change what kind of class `GradingScheme` is:

- **`= 0`** makes `computePercentage` a **pure virtual function**. The base class declares that every scheme *has* a rule but supplies none itself. A class with a pure virtual function is **abstract** — you cannot create a plain `GradingScheme` object, only objects of derived classes.
- **`virtual`** means that when code calls `computePercentage` through a `GradingScheme*`, C++ looks at the object the pointer actually points to and runs *that* class's version. This is **dynamic dispatch**.
- The **virtual destructor** matters because `Gradebook` will `delete` a scheme through a `GradingScheme*`. Without `virtual`, only the base part of the object would be destroyed.

Now mark each derived class's version with `override`.

In `class PointsBased`, find:

```
    PointsBased() : GradingScheme("Points-based") {}
    double computePercentage(const std::vector<Assignment>& work) const;
```

Replace it with:

```
    PointsBased() : GradingScheme("Points-based") {}
    double computePercentage(const std::vector<Assignment>& work) const override;
```

In `class Weighted`, find:

```
    explicit Weighted(const std::vector<CategoryWeight>& weights);

    double computePercentage(const std::vector<Assignment>& work) const;
```

Replace it with:

```
    explicit Weighted(const std::vector<CategoryWeight>& weights);

    double computePercentage(const std::vector<Assignment>& work) const override;
```

`override` asks the compiler to confirm that this function really overrides a virtual function in the base class. If you misspell the name or get a parameter wrong, you get a compile error instead of a silent new function.

Finally, let a future derived class reach `Weighted`'s weights, and add the third scheme.

At the bottom of `class Weighted`, find:

```
private:
    std::vector<CategoryWeight> weights_;
};
```

Replace it with:

```
protected:
    std::vector<CategoryWeight> weights_;
};

/**
 * Proof that the design is open to extension: a third scheme.
 *
 * Adding this class required NO change to Gradebook, Student, GradeScale,
 * Assignment, or the report code. Compare with the flag-based design of
 * Section 20.2, where a third scheme means editing every branch.
 */
class WeightedDropLowest : public Weighted {
public:
    WeightedDropLowest();
    explicit WeightedDropLowest(const std::vector<CategoryWeight>& weights);
    double computePercentage(const std::vector<Assignment>& work) const override;
};
```

- `weights_` changes from `private` to **`protected`**, so the derived class `WeightedDropLowest` can read the weights directly.
- `class WeightedDropLowest : public Weighted` inherits everything from `Weighted` — the weights, `weightTotal()`, `weightsValid()`, even its name — and overrides only `computePercentage`.

Step 2 — Implement the third scheme (`gradingscheme.cpp`) In `gradingscheme.cpp`, find the first line:

```
// gradingscheme.cpp - Grade Calculator v3.0 - Chapter 20
```

Replace it with:

```
// gradingscheme.cpp - Grade Calculator v3.1 - Chapter 21
```

Go to the very end of `gradingscheme.cpp`, after the closing `}` of `Weighted::computePercentage`. Leave one blank line and add:

```
WeightedDropLowest::WeightedDropLowest() : Weighted() {}

WeightedDropLowest::WeightedDropLowest(const std::vector<CategoryWeight>& weights)
    : Weighted(weights) {}

double WeightedDropLowest::computePercentage(const std::vector<Assignment>& work) const {
    double weightedSum = 0.0;
    double weightUsed  = 0.0;

    for (const CategoryWeight& w : weights_) {
        // Collect this category's work, then discard its weakest item.
        std::vector<const Assignment*> items;
        for (const Assignment& a : work) {
            if (a.category() == w.name && a.pointsPossible() > 0.0) {
```

```
                items.push_back(&a);
            }
        }
        if (items.size() > 1) {
            std::size_t worst = 0;
            for (std::size_t i = 1; i < items.size(); ++i) {
                if (items[i]->ratio() < items[worst]->ratio()) { worst = i; }
            }
            items.erase(items.begin() + static_cast<long>(worst));
        }

        double earned = 0.0;
        double possible = 0.0;
        for (const Assignment* a : items) {
            earned   += a->totalEarned();
            possible += a->pointsPossible();
        }
        if (possible > 0.0) {
            weightedSum += (earned / possible) * w.weight;
            weightUsed  += w.weight;
        }
    }
    if (weightUsed <= 0.0) { return 0.0; }
    return finalize(weightedSum / weightUsed * 100.0);
}
```

- Both constructors pass straight through to `Weighted`'s constructors, so a `WeightedDropLowest` starts with the same default weights.
- `computePercentage` follows the same plan as `Weighted`'s, with one extra step per category: it collects pointers to that category's assignments, finds the one with the lowest `ratio()`, and erases it — but only if the category has more than one item.

▸ **Run checkpoint**: Click **Run**. The program should compile cleanly and behave exactly like v3.0. `Gradebook` still holds concrete `PointsBased` and `Weighted` objects, which is allowed — only `GradingScheme` itself is abstract.

Step 3 — Hold the scheme through a base-class pointer (`gradebook.h`) In `gradebook.h`, find the first line:

```
// gradebook.h - Grade Calculator v3.0 - Chapter 20
```

Replace it with:

```
// gradebook.h - Grade Calculator v3.1 - Chapter 21
```

Inside the class, find the scheme enum, the constructor, and the scheme accessors:

```
    /** Which grading rule this gradebook applies. Fixed at construction in
     *  v3.0; Chapter 21 makes it changeable at any time. */
    enum class Scheme { Points, Weighted };

    explicit Gradebook(const std::string& courseName = "Untitled Course",
                       Scheme scheme = Scheme::Points);

    /** The course grade for one student under the selected scheme. */
    double percentageFor(const Student& s) const;
    std::string schemeName() const;
    const Weighted& weightedScheme() const { return weighted_; }
    void setWeights(const std::vector<Weighted::CategoryWeight>& w) { weighted_ = Weighted(w); }
```

Replace it with:

```
    explicit Gradebook(const std::string& courseName = "Untitled Course");
    ~Gradebook();

    // Copying is disabled: Gradebook owns a raw pointer, and a default copy
    // would leave two objects deleting the same scheme. Chapter 22 replaces
    // this restriction with smart-pointer ownership.
    Gradebook(const Gradebook&) = delete;
    Gradebook& operator=(const Gradebook&) = delete;

    /**
     * Installs a grading scheme. The Gradebook takes ownership.
     * Can be called at any time; every grade is recomputed on the next report.
     */
    void setScheme(GradingScheme* scheme);

    /** The course grade for one student. No branch: the scheme decides. */
    double percentageFor(const Student& s) const;
    std::string schemeName() const;
```

At the bottom of the class, find:

```
    // Both schemes are held as concrete objects, and a single branch in
    // percentageFor() chooses between them. That branch is the limitation
    // Chapter 21 removes with virtual functions.
    Scheme scheme_ = Scheme::Points;
    PointsBased points_;
    Weighted    weighted_;
```

Replace it with:

```
    // One pointer to an abstract base. Which concrete scheme it refers to is
    // not this class's concern - that is what polymorphism buys.
    GradingScheme* scheme_ = nullptr;
```

- The `Scheme` enum, the two concrete scheme objects, `weightedScheme()`, and `setWeights()` are all gone. `Gradebook` now holds **one pointer to the abstract base**, and never needs to know which concrete scheme it points to.
- `setScheme` installs a new scheme. The comment states the ownership rule: **the `Gradebook` takes ownership** of the object it is given.
- Owning a raw pointer brings three duties, and the header shows all three:
  1. a **destructor** (`~Gradebook();`) to `delete` the scheme;
  2. `setScheme` must `delete` the old scheme before replacing it (Step 4);
  3. **copying is disabled** with `= delete`. A copied `Gradebook` would hold the same pointer as the original, and both would `delete` it.

Step 4 — Dispatch through the pointer (`gradebook.cpp`) In `gradebook.cpp`, find the first line:

```
// gradebook.cpp - Grade Calculator v3.0 - Chapter 20
```

Replace it with:

```
// gradebook.cpp - Grade Calculator v3.1 - Chapter 21
```

Find the constructor, `percentageFor`, and `schemeName`:

```
Gradebook::Gradebook(const std::string& courseName, Scheme scheme)
    : courseName_(courseName), scheme_(scheme) {}
```

```
double Gradebook::percentageFor(const Student& s) const {
    // The single branch. One place today; one place per operation as soon as
    // a third scheme or a second calculation is added. See Section 20.2.
    if (scheme_ == Scheme::Weighted) {
        return weighted_.computePercentage(s.assignments());
    }
    return points_.computePercentage(s.assignments());
}

std::string Gradebook::schemeName() const {
    return scheme_ == Scheme::Weighted ? weighted_.name() : points_.name();
}
```

Replace it with:

```
Gradebook::Gradebook(const std::string& courseName)
    : courseName_(courseName), scheme_(new PointsBased()) {}

Gradebook::~Gradebook() {
    delete scheme_;
}

void Gradebook::setScheme(GradingScheme* scheme) {
    if (scheme == nullptr || scheme == scheme_) { return; }
    delete scheme_;          // release the previous scheme before replacing it
    scheme_ = scheme;
}

double Gradebook::percentageFor(const Student& s) const {
    // Dynamic dispatch: the object decides which computePercentage runs.
    return scheme_->computePercentage(s.assignments());
}

std::string Gradebook::schemeName() const {
    return scheme_->name();
}
```

- The constructor starts every `Gradebook` with a `new PointsBased()`, so `scheme_` is never null.
- The destructor deletes the scheme. `setScheme` ignores a null pointer or the scheme it already holds, deletes the old scheme, then stores the new one.
- **`percentageFor` has no branch.** `scheme_->computePercentage(...)` runs `PointsBased`'s, `Weighted`'s, or `WeightedDropLowest`'s version depending on the object `scheme_` points to. The same is true of `schemeName()`.

The rest of `gradebook.cpp` — `classAverage`, `sortByPercentage`, and `operator<<` — does not change, because it already calls `percentageFor`.

> **Heads-up**: `main.cpp` still uses `Gradebook::Scheme`, which no longer exists, so the program will not compile until the end of Step 7.

Step 5 — Update the header comment in `main.cpp` At the top of `main.cpp`, find the header comment:

```
// ============================================================================
//  Grade Calculator  v3.0                                        Chapter 20
// ----------------------------------------------------------------------------
//  Change  : Weighted-category grading works. A GradingScheme base class with
//            PointsBased and Weighted derived classes replaces the single
//            hard-coded rule. Assignments now carry a category.
//  Chooses : the scheme is selected at STARTUP and fixed for the session.
```

```
//  Limit   : switching schemes requires restarting, because the selection is
//            a branch in Gradebook::percentageFor rather than dynamic
//            dispatch. Chapter 21 removes this.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp assignment.cpp gradescale.cpp
//            student.cpp gradingscheme.cpp gradebook.cpp -o gradecalc
// ============================================================================
```

Replace it with:

```
// ============================================================================
//  Grade Calculator  v3.1                                        Chapter 21
// ----------------------------------------------------------------------------
//  Change  : GradingScheme is now abstract with a pure virtual computePercentage
//            and a virtual destructor. Gradebook holds a GradingScheme* and
//            never asks which scheme it has - the v3.0 branch is gone.
//            The scheme can be switched MID-SESSION and every grade recomputes.
//            A third scheme (WeightedDropLowest) was added without changing
//            Gradebook, Student, GradeScale, Assignment, or the report code.
//  Cost    : correct raw-pointer ownership needs three pieces of hand-written
//            bookkeeping in Gradebook - a destructor, a delete inside
//            setScheme, and deleted copy operations. This build has all three
//            and does not leak. Chapter 22 removes the need for any of them.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp assignment.cpp gradescale.cpp
//            student.cpp gradingscheme.cpp gradebook.cpp -o gradecalc
// ============================================================================
```

Step 6 — Add `chooseScheme`, the one place that names concrete types In `main.cpp`, find the first line of `main`:

```
int main() {
```

Directly above it, add:

```
/** Builds the scheme the user asked for. The only place that names the
 *  concrete types; everything downstream sees only GradingScheme. */
void chooseScheme(Gradebook& book) {
    std::cout << "  1) Points-based\n";
    std::cout << "  2) Weighted by category\n";
    std::cout << "  3) Weighted, lowest dropped per category\n";
    std::string c = readLine("  Choice: ");
    if (!c.empty() && c[0] == '2')      { book.setScheme(new Weighted()); }
    else if (!c.empty() && c[0] == '3') { book.setScheme(new WeightedDropLowest()); }
    else                                { book.setScheme(new PointsBased()); }
    std::cout << "  Grading is now: " << book.schemeName() << "\n";
}
```

This function still has a branch — but look at what it does. It runs only when the user makes a choice, and its only job is to **create** the right object with `new` and hand it to `book.setScheme(...)`, which takes ownership. Nothing downstream ever asks which scheme it has.

Step 7 — Rewrite `main` with a *Change scheme* option Find the start of `main`:

```
int main() {
```

Delete that line and everything below it, all the way to the final closing `}` at the very end of the file.

In its place, enter the new `main`:

```
int main() {
    std::cout << "=== GRADE CALCULATOR v3.1 (switchable schemes) ===\n\n";
```

```
    Gradebook book("Programming Fundamentals");
    std::cout << "Select a grading scheme:\n";
    chooseScheme(book);

    bool running = true;
    while (running) {
        std::cout << "\n1) Add student  2) Report  3) Sort  "
                     "4) Custom scale  5) Change scheme  6) Quit\nChoice: ";
        std::string line;
        if (!std::getline(std::cin, line)) { break; }
        char c = line.empty() ? '?' : line[0];

        if (c == '1') {
            std::string name = readLine("  Student name: ");
            if (name.empty()) { continue; }
            Student s(name, book.claimNextId());
            while (true) {
                std::string an = readLine("    Assignment name (or 'done'): ");
                if (an == "done" || an.empty()) { break; }
                double earned   = readNonNegative("      Points earned  : ");
                double possible = readNonNegative("      Points possible: ");
                double bonus    = readNonNegative("      Bonus points   : ");
                std::string cat = readLine("      Category       : ");
                if (cat.empty()) { cat = "Uncategorized"; }
                s.addAssignment(Assignment(an, earned, possible, bonus, cat));
            }
            book.addStudent(s);
            std::cout << "  Added " << s.name() << " as ID " << s.id() << ".\n";

        } else if (c == '2') {
            std::cout << book << "\n";
        } else if (c == '3') {
            book.sortByPercentage();
            std::cout << "  Sorted.\n";
        } else if (c == '4') {
            std::vector<GradeScale::Tier> requested;
            std::cout << "  Enter tiers highest first, 'done' to finish.\n";
            while (true) {
                std::string lt = readLine("    Letter (or 'done'): ");
                if (lt == "done" || lt.empty()) { break; }
                requested.push_back({readNonNegative("      Minimum percentage: "), lt[0]});
            }
            book.setScale(GradeScale(requested));
            std::cout << "  " << book.scale() << "\n";
        } else if (c == '5') {
            // Every existing student is regraded under the new rule. No data
            // is re-entered, and no other class knows this happened.
            chooseScheme(book);
        } else if (c == '6') {
            running = false;
        } else {
            std::cout << "  Please enter 1-6.\n";
        }
    }
    std::cout << "\nGoodbye.\n";
    return 0;
}
```

- At startup, the `Gradebook` is created first (with its default points scheme), and then `chooseScheme(book)` lets the user pick.

- The menu has six options. **Change scheme** (option 5) simply calls `chooseScheme(book)` again. Every existing student is regraded under the new rule the next time you ask for a report — no data is re-entered.
- **Sort**, **Custom scale**, and **Add student** work exactly as in v3.0.

▸ **Run checkpoint**: Make sure all eleven files are open and click **Run**. The program should compile with no errors and no warnings and open by asking you to select a grading scheme.

Check your work

Run the program and choose `1` (Points-based). Add `Ada` with three assignments:

Assignment	Earned	Possible	Bonus	Category
`Exam 1`	90	100	0	`Exam`
`HW1`	10	10	0	`Homework`
`HW2`	5	10	0	`Homework`

Then report, switch schemes with option `5`, and report again — without re-entering anything:

1. Choose `2`. You should see `Grading: Points-based` and `1001  Ada  87.5%  B` (105 ÷ 120).
2. Choose `5`, then `2` (Weighted). Choose `2` (Report): `Grading: Weighted` and `1001  Ada  84.4%  B` (Exam 0.90 × 50 = 45; Homework 15/20 = 0.75 × 30 = 22.5; 67.5 ÷ 80).
3. Choose `5`, then `3` (Weighted, lowest dropped). Choose `2`: `1001  Ada  93.8%  A`. Dropping HW2 leaves Homework at 10/10, so (45 + 30) ÷ 80 = 93.75%.
4. Choose `6` (Quit).

Notice that in step 3 the report still says `Grading: Weighted`. `WeightedDropLowest` inherits its name from `Weighted`'s constructor and does not set its own — a small reminder that a derived class gets *everything* from its base unless it overrides it.

Then confirm:

- ☐ The program compiles with no errors and no warnings.
- ☐ `Gradebook::percentageFor` contains no `if` — the call goes through `scheme_->computePercentage`.
- ☐ `Gradebook` has a destructor that deletes `scheme_`, `setScheme` deletes the old scheme before storing the new one, and the copy constructor and copy assignment are `= delete`.
- ☐ `assignment.*`, `gradescale.*`, and `student.*` are unchanged from Chapter 20.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click all eleven project files to open them. Confirm the editor shows the expected file paths before editing.
4. Work through Steps 1–7 above in StudySite's internal editor, in order, clicking **Run** at each ▸ Run checkpoint.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — all eleven files. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 21 Grade Calculator v3.1**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 21 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Virtual dispatch through references and a collection

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Shape {
public:
    virtual ~Shape() = default;
    virtual std::string describe() const = 0;
};

class Circle : public Shape {
public:
    std::string describe() const override { return "circle"; }
};

class Square : public Shape {
public:
    std::string describe() const override { return "square"; }
};

void show(const Shape& s) { std::cout << "  " << s.describe() << "\n"; }

int main() {
    Circle c;
    Square q;
    std::cout << "through references:\n";
    show(c);
    show(q);
```

```
    std::cout << "through a collection of base pointers:\n";
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>());
    shapes.push_back(std::make_unique<Square>());
    for (const auto& s : shapes) { std::cout << "  " << s->describe() << "\n"; }
    return 0;
}
```

Expected output:

```
through references:
  circle
  square
through a collection of base pointers:
  circle
  square
```

Try: Add a `Triangle` class and push one into the vector. **How many existing lines did you change?** Then try `Shape s;` and read the error.

2. Object slicing

```
#include <iostream>
#include <string>

class Shape {
public:
    virtual ~Shape() = default;
    virtual std::string name() const { return "shape"; }
};

class Circle : public Shape {
public:
    std::string name() const override { return "circle"; }
};

void byValue(Shape s)            { std::cout << "  by value:     " << s.name() << "\n"; }
void byReference(const Shape& s) { std::cout << "  by reference: " << s.name() << "\n"; }

int main() {
    Circle c;
    byValue(c);
    byReference(c);
    return 0;
}
```

Expected output:

```
  by value:     shape
  by reference: circle
```

Try: No warning was produced. Write one sentence on why that makes slicing dangerous, and state the rule that prevents it.

3. `override` catches a mistake

```
#include <iostream>
```

```
class Base {
public:
    virtual ~Base() = default;
    virtual void greet() const { std::cout << "base\n"; }
};

class Derived : public Base {
public:
    void greet() const override { std::cout << "derived\n"; }
};

int main() {
    Derived d;
    const Base& b = d;
    b.greet();
    return 0;
}
```

Expected output:

```
derived
```

Try: Remove `const` from `Derived::greet` but keep `override`. Read the error. Then remove `override` as well and rerun — which greeting prints now, and why is that worse than a compile error?

4. Virtual destructors matter

```
#include <iostream>

class Base {
public:
    Base()  { std::cout << "Base built\n"; }
    virtual ~Base() { std::cout << "Base destroyed\n"; }
};

class Derived : public Base {
public:
    Derived()  { std::cout << "Derived built\n"; }
    ~Derived() override { std::cout << "Derived destroyed\n"; }
};

int main() {
    Base* p = new Derived();
    delete p;
    return 0;
}
```

Expected output:

```
Base built
Derived built
Derived destroyed
Base destroyed
```

Try: Remove `virtual` from `~Base` and rerun. Which line disappears? Now imagine `Derived` held a `std::vector` — what happened to it?

5. An abstract interface

Write an abstract class `Reporter` with a pure virtual `report(const std::string& text)`. Derive `ConsoleReporter` and `PrefixReporter`, the second adding a prefix.

Then write one function taking `const Reporter&` and call it with both. Confirm the function never names either concrete class.

6. Cost a change, again

Using v3.1, count the edits to add a fourth scheme — say, "points-based with the lowest two dropped."

Compare with the count you produced in Chapter 20 task 7 for v3.0's design. Which grew, and which did not?

7. Reason about polymorphism

- What is the difference between static and dynamic binding?
- Why does dynamic binding require a pointer or reference?
- What makes a class abstract, and why is that useful?
- Why must a polymorphic base class have a virtual destructor?
- Give one situation where the flag design from Chapter 20 Section 20.2 is the better choice.
- You find a function containing four `dynamic_cast` tests. What does that suggest, and what would you do instead?

Summary

- **Polymorphism** means one call producing different behavior depending on the object. It removes the branch that variant-handling would otherwise require.
- **`virtual`** switches from **static binding** (the variable's type decides, at compile time) to **dynamic binding** (the object's type decides, at run time).
- **Always write `override`.** It turns a silent mismatch into a compile error.
- Dynamic binding requires a **base class pointer or reference**. Passing by value causes **object slicing** — the derived part is discarded, silently.
- A **pure virtual function** (`= 0`) makes a class **abstract**: it cannot be instantiated, and every derived class must supply an implementation. An abstract class defines an **interface** and may still hold data and shared functions.
- **A polymorphic base class must have a virtual destructor**, or deleting through a base pointer leaks the derived part.
- A **polymorphic collection** of base pointers holds mixed types, each behaving as itself. Store pointers, never values.
- **`dynamic_cast` in a chain is a design smell** — it usually means a virtual function was wanted.
- Costs are a small run-time indirection, a small memory overhead, and the real conceptual cost that a call site no longer tells you which code runs.
- **v3.1 added a third grading scheme with one new class and no edits to anything else** — the claim Chapter 20 made, now verifiable.

Key Terms

abstract class — a class with at least one pure virtual function; cannot be instantiated.

dynamic binding — deciding at run time which function a call invokes, based on the object's type.

dynamic_cast — a cast checking at run time whether an object is of a given type.

interface — the set of operations an abstract class promises every derived class provides.

object slicing — losing the derived part of an object by copying it into a base-typed variable.

override — a keyword asking the compiler to verify that a function overrides a base virtual.

polymorphic collection — a container of base pointers holding objects of several derived types.

polymorphism — one call producing behavior determined by the object rather than the calling code.

pure virtual function — a virtual function with no implementation, declared `= 0`.

static binding — deciding at compile time which function a call invokes.

virtual destructor — a destructor declared `virtual`, ensuring the full object is destroyed through a base pointer.

virtual function — a member function whose implementation is chosen by the object's actual type.

Next: Chapter 22 examines the raw pointer `Gradebook` has been holding. It does not leak — but only because three separate pieces of bookkeeping were remembered. You will delete one of them, watch the leak appear under a sanitizer, and then replace all three with a smart pointer that cannot be got wrong. Grade Calculator v3.2.

Chapter 22 — Memory Management and Smart Pointers

Learning Objectives

When you finish this chapter you will be able to:

- Describe the four regions of memory a program uses. *(SLO 2.1)*
- Distinguish automatic from dynamic storage. *(SLO 2.1)*
- Allocate and release memory with `new` and `delete`. *(SLO 2.1)*
- Identify memory leaks, dangling pointers, and double frees, and detect them with a sanitizer. *(SLO 2.7)*
- Explain RAII and why destructors make it work. *(SLO 2.1, 2.8)*
- State the Rule of Three and say when it applies. *(SLO 2.2)*
- Use `std::unique_ptr` and `std::shared_ptr`. *(SLO 2.1)*
- Build Grade Calculator v3.2 — the same behavior with none of the ownership bookkeeping.

22.1 Memory Regions

Chapter 1 Section 1.2 described main memory as a sequence of addressed cells. A running program divides that space into four regions with different rules.

Region	Holds	Managed by	Lifetime
Code	the machine instructions	the loader	the whole run
Static	globals and `static` locals	the compiler	the whole run
Stack	local variables, parameters	**automatically**	until the function returns
Heap	anything obtained with `new`	**by you**	until something releases it

The last two are what this chapter is about.

Figure 22.1 — Where a program's data lives.

Description of Figure 22.1. The left column is the **stack**, holding three frames: `percentageOf()` with `earned` and `possible` at the top, `printReport()` with `index` below it, and `main()` at the bottom holding `book` and a pointer `scheme_`. A note records that frames appear and vanish automatically as functions are entered and left.

The right column is the **heap**, holding two blocks: a `Weighted` object obtained with `new`, and storage obtained internally by a `std::vector`. A note records that heap blocks stay until something releases them.

An arrow runs from `scheme_` in the `main` frame to the `Weighted` object on the heap.

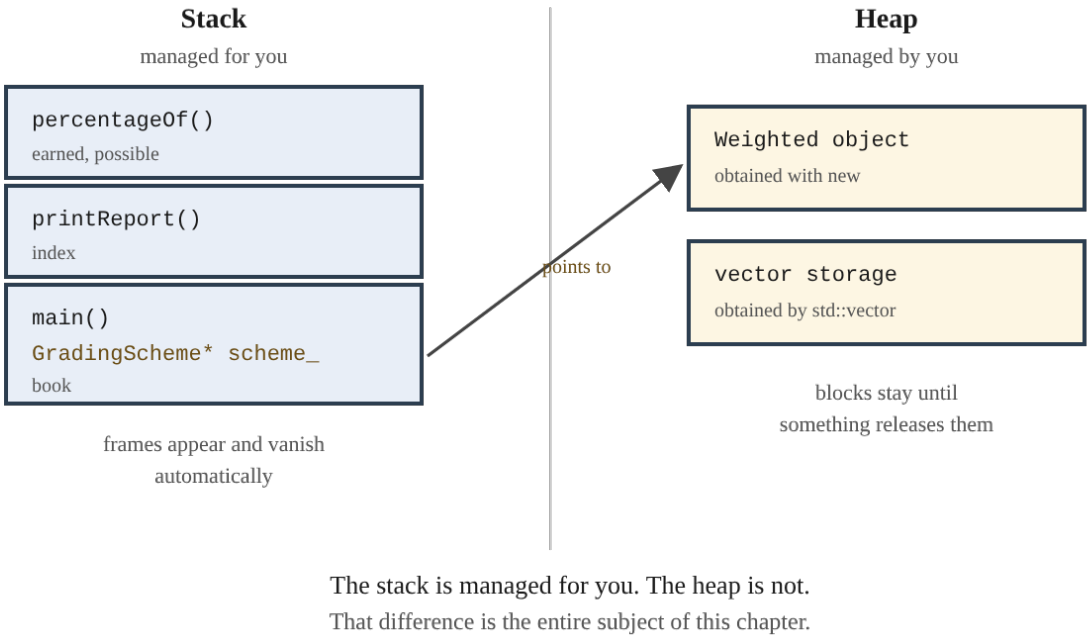


Figure 17: Two columns: the stack on the left holding three function frames, and the heap on the right holding two blocks, with an arrow from a pointer in the main frame to one of the heap blocks.

The stack is managed for you. The heap is not. That difference is the entire subject of this chapter.

22.2 Automatic and Dynamic Storage

Automatic storage is what you have used since Chapter 3:

```
void example() {
    double score = 84.0;          // created here
    Student ada("Ada", 1001);     // created here
}                                 // both destroyed here, automatically
```

The stack frame vanishes when the function returns, and everything in it is destroyed. You cannot forget, because you were never asked.

Dynamic storage is obtained explicitly and stays until released:

```
Student* p = new Student("Ada", 1001);    // on the heap
// ...
delete p;                                 // released, by you
```

Three reasons to need it:

The size is not known at compile time. `std::vector` uses the heap for exactly this.

The object must outlive the function that created it. Returning a pointer to a local is undefined behavior; returning a pointer to a heap object is fine.

Polymorphism. Chapter 21 Section 21.4 showed that a base-typed *value* slices. Holding a `GradingScheme*` avoids that, and the object it points at is on the heap.

The Grade Calculator uses the heap for the third reason and no other.

22.3 `new` and `delete`

```
GradingScheme* scheme = new Weighted();    // allocate and construct
delete scheme;                             // destruct and release
```

`new` finds space, runs the constructor, and returns a pointer. `delete` runs the destructor and returns the space.

Two rules with no exceptions. **Every `new` needs exactly one matching `delete`**. And **`delete` through a base pointer requires a virtual destructor** — Chapter 21 Section 21.6, or only the base part is destroyed.

22.4 Three Ways to Get It Wrong

Memory leak

You allocate and never release:

```
void leak() {
    GradingScheme* s = new Weighted();
    // no delete — the object is unreachable and never freed
}
```

The memory is gone for the rest of the run. One leak is harmless; a leak inside a loop exhausts memory.

Leaks are silent. Nothing crashes, nothing warns, and the program appears to work.

Dangling pointer

You keep using a pointer after releasing what it pointed at:

```
GradingScheme* s = new Weighted();
delete s;
s->computePercentage(work);      // undefined behavior
```

Sometimes it works, sometimes it crashes, sometimes it silently reads garbage — the Chapter 11 out-of-bounds situation again. **Set a pointer to `nullptr` after deleting** if it stays in scope.

Double free

You delete the same object twice:

```
delete s;
delete s;                        // undefined behavior, usually a crash
```

This most often happens when two objects both think they own something — which is exactly why Chapter 21's `Gradebook` disabled copying.

Detecting them

AddressSanitizer finds all three. Add `-fsanitize=address -g`:

```
g++ -std=c++17 -fsanitize=address -g *.cpp -o gradecalc
```

The program runs normally and reports problems as they happen:

```
SUMMARY: AddressSanitizer: 264 byte(s) leaked in 4 allocation(s).
```

This belongs with Chapter 16's debugging toolkit. **A leak you cannot see is a leak you cannot fix**, and a sanitizer makes the invisible visible.

22.5 RAII

Here is the idea that resolves all of this, and it is one of the best in C++.

RAII — Resource Acquisition Is Initialization — means: **tie a resource's lifetime to an object's lifetime**. Acquire it in the constructor, release it in the destructor. Because the destructor runs automatically when the object goes out of scope, the resource is released automatically too.

You have used RAII since Chapter 12 without knowing it:

```
{
    std::vector<double> scores;
    scores.push_back(9.0);          // vector allocates heap memory
}                                   // destructor runs; memory released
```

`std::vector` allocates on the heap. You never call `delete` because its destructor does. Same for `std::string`, and for `std::ofstream` in Chapter 15, which closes the file when it goes out of scope.

The general principle: **never manage a resource by hand if an object can manage it for you**. Files, memory, locks, network connections — the pattern is identical.

22.6 The Rule of Three

If a class manages a resource directly, it usually needs **three** functions:

```
class Owner {
public:
    ~Owner();                              // 1. destructor
    Owner(const Owner& other);             // 2. copy constructor
    Owner& operator=(const Owner& other);  // 3. copy assignment
};
```

The Rule of Three: **if you need any one of these, you almost certainly need all three.**

Chapter 19 Section 19.6 said the compiler generates these for you and that the generated versions are correct for classes holding `std::string` and `std::vector`. The moment a class holds a raw pointer, they stop being correct: the generated copy copies the *pointer*, so two objects point at one thing and both eventually delete it.

Chapter 21's `Gradebook` handled this by **disabling** copying:

```
Gradebook(const Gradebook&) = delete;
Gradebook& operator=(const Gradebook&) = delete;
```

Deleting them is a legitimate choice when copying makes no sense. It is also a symptom: the class needed a decision it should not have had to make.

22.7 Smart Pointers

A **smart pointer** applies RAII to memory. It behaves like a pointer and releases what it holds when destroyed.

`std::unique_ptr`

One owner. When the pointer is destroyed, the object is destroyed.

```
#include <memory>

std::unique_ptr<GradingScheme> scheme = std::make_unique<Weighted>();
scheme->computePercentage(work);     // used exactly like a pointer
// no delete — released automatically
```

```
--- raw pointer, managed by hand ---
  built Points
  destroyed Points
--- unique_ptr, managed for you ---
  built Points
  destroyed Points
```

The behavior is identical; the second required no `delete`.

Assigning to a `unique_ptr` releases what it held:

```
--- reassigning a unique_ptr releases the old one ---
  built Points
  built Weighted
  destroyed Points
  now holding Weighted
  destroyed Weighted
```

That one line replaces the manual `delete scheme_;` that Chapter 21's `setScheme` needed.

A `unique_ptr` **cannot be copied** — that is what "unique" means. It can be **moved** with `std::move`, which transfers ownership:

```
void Gradebook::setScheme(std::unique_ptr<GradingScheme> scheme) {
    scheme_ = std::move(scheme);
}

book.setScheme(std::make_unique<Weighted>());
```

Taking the parameter *by value* as a `unique_ptr` is deliberate: it makes the transfer of ownership visible at every call site, because a caller must hand the pointer over rather than share it.

`std::shared_ptr`

Several owners, with a count. The object is destroyed when the last owner goes away.

```
--- shared_ptr counts owners ---
  built Points
  use_count 1
  use_count 2
  use_count 1
  destroyed Points
```

`shared_ptr` is more flexible and more expensive — it maintains a count, and the count must be updated safely. **Prefer `unique_ptr`.** Reach for `shared_ptr` only when ownership genuinely is shared, which is rarer than people expect.

Choosing

Situation	Use
A value with a clear scope	**no pointer at all** — automatic storage
One owner	`std::unique_ptr`
Genuinely shared ownership	`std::shared_ptr`
Referring without owning	a plain reference or raw pointer

Note the first row. Much code that reaches for `new` did not need the heap at all.

22.8 Linked Lists

A **linked list** is the classic example of manual memory management, and worth seeing once:

```
struct Node {
    double value = 0.0;
    Node* next = nullptr;
};
```

Each node holds a value and a pointer to the next. The list is a chain, ending with `nullptr`.

Unlike a vector, inserting in the middle costs nothing beyond finding the position — no elements move. In exchange, reaching element k means walking k links, so there is no `[]` in any useful sense.

Managing a linked list by hand means every node must be deleted exactly once, which requires walking the chain in the destructor and being careful about the order. This is where the Rule of Three earns its name.

The Grade Calculator uses `std::vector` throughout and never needs this. Linked lists are covered because they are the standard illustration of pointer-based structures, and because `std::list` exists if you ever want one without the bookkeeping.

22.9 Move Semantics, Briefly

Copying a large object is expensive. Sometimes you do not need a copy — you need to *transfer* it, because the source is about to be discarded.

Move semantics does that:

```
std::vector<Student> a = makeRoster();
std::vector<Student> b = std::move(a);    // b takes a's storage; nothing is copied
```

After a move, the source is in a valid but unspecified state. **Do not use a moved-from object except to assign to it or destroy it.**

`std::move` does not move anything by itself — it marks a value as movable, permitting the compiler to choose a transfer instead of a copy. That is the mechanism behind `unique_ptr`'s ownership transfer in Section 22.7.

Common Errors and Warnings

What you see or observe	Cause	Fix
`SUMMARY: AddressSanitizer: N byte(s) leaked`	`new` without a matching `delete`	Use `std::unique_ptr`
`heap-use-after-free`	Dangling pointer	Do not use after delete; set to `nullptr`
`attempting double-free`	Two owners of one object	One owner; use `unique_ptr`
Only the base destructor runs	No virtual destructor	Add `virtual ~Base() = default;`
`error: use of deleted function` on copy	Copying a `unique_ptr`	Use `std::move`, or `shared_ptr`
Memory grows steadily during a run	Leak inside a loop	Build with a sanitizer and find it
`error: 'unique_ptr' was not declared`	Missing `#include <memory>`	Add the header
A moved-from object behaves oddly	Used after `std::move`	Do not use it except to reassign
Two objects delete the same memory	Generated copy constructor copied a raw pointer	Rule of Three, or a smart pointer

Design Notes

Prefer automatic storage. Most objects do not need the heap.

Never write `new` without an owner. If you type `new`, the result should go straight into a smart pointer.

One owner. `unique_ptr` unless ownership is genuinely shared.

Every polymorphic base gets a virtual destructor.

Run a sanitizer before you ship. Leaks are silent; a sanitizer is the only cheap way to see them.

Grade Calculator v3.2 — Safe Ownership

The honest framing

The obvious version of this chapter's lesson would be: *v3.1 leaked, and smart pointers fix it.* That would be false, and it is worth being exact about why.

v3.1 does not leak. Its raw-pointer ownership is correct. Run it under a memory check, switch schemes repeatedly, and it reports nothing.

It is correct because `Gradebook` does **three separate things**:

```
Gradebook::~Gradebook() {
    delete scheme_;                         // 1. release on destruction
}

void Gradebook::setScheme(GradingScheme* scheme) {
    if (scheme == nullptr || scheme == scheme_) { return; }
    delete scheme_;                         // 2. release before replacing
    scheme_ = scheme;
}
```

```
Gradebook(const Gradebook&) = delete;        // 3. prevent two owners
Gradebook& operator=(const Gradebook&) = delete;
```

Eight lines of bookkeeping, each of which had to be remembered.

The lab: remove one line

Take a copy of your working v3.1 and delete **one line** — the `delete scheme_;` inside `setScheme`. Nothing else.

Now switch schemes twice and check it with the memory check available to you.

> **Note — this is the one check StudySite's Run cannot make for you.** Clicking **Run** proves the program still prints correct grades, and it will, which is exactly the problem. A leak is invisible from the program's output. If you have a local C++ toolchain available, build the copy with `-fsanitize=address -g` and switch schemes twice; otherwise ask your instructor which memory check to use. Either way, do the edit, the normal run, and the commit back in the StudySite coding panel.

A leaking build reports something like this:

```
SUMMARY: AddressSanitizer: 264 byte(s) leaked in 4 allocation(s).
```

One line. Four leaked allocations. The program still produced correct grades — the leak is invisible from the outside, which is why Section 22.4 calls leaks silent.

Now put it back and confirm the report is clean again.

The conversion

`unique_ptr` replaces all three pieces:

```
// gradebook.h
#include <memory>

class Gradebook {
public:
    explicit Gradebook(const std::string& courseName = "Untitled Course");

    // No destructor is needed now. unique_ptr releases the scheme when the
    // Gradebook is destroyed, and again whenever the scheme is replaced.
    // The compiler-generated destructor is correct, so none is written here.

    /**
     * Installs a grading scheme. Ownership transfers to the Gradebook.
     * The unique_ptr parameter makes that transfer visible at every call site:
     * a caller must std::move into it, so ownership is never ambiguous.
     */
    void setScheme(std::unique_ptr<GradingScheme> scheme);

    double percentageFor(const Student& s) const;

private:
    // An owning smart pointer to an abstract base. Polymorphism is unchanged;
    // only the ownership question is answered differently.
    std::unique_ptr<GradingScheme> scheme_;
};
```

```
// gradebook.cpp
Gradebook::Gradebook(const std::string& courseName)
```

```
    : courseName_(courseName), scheme_(std::make_unique<PointsBased>()) {}

void Gradebook::setScheme(std::unique_ptr<GradingScheme> scheme) {
    if (!scheme) { return; }
    // Assigning to a unique_ptr destroys whatever it held first. This one line
    // replaces v3.1's manual delete, and the destructor and deleted copy
    // operations that had to accompany it.
    scheme_ = std::move(scheme);
}
```

And at the call site:

```
book.setScheme(std::make_unique<Weighted>());
```

The tally

	v3.1	v3.2
Destructor	hand-written	**not needed**
`delete` in `setScheme`	required	**not needed**
Deleted copy operations	required	**not needed**
Lines of ownership bookkeeping	8	**0**
Leaks	none	none
Behavior	identical	identical

The actual lesson

v3.1 is correct because its author remembered three separate things. v3.2 is correct because there is nothing to remember.

That is a stronger claim than "smart pointers fix leaks," and it is the real argument for RAII. Correctness that depends on vigilance degrades — under deadline, under a new maintainer, under a refactor that moves a line. Correctness that is structural does not.

Notice too that **the polymorphism is untouched**. `GradingScheme` is still abstract, `computePercentage` is still virtual, dispatch still works exactly as in Chapter 21. Only the ownership question was answered differently.

Expected output

Identical to v3.1 in every respect:

```
  Grading: Weighted
1001  Ada                    90.0%   A
```

Your StudySite Lab — Replace Manual Ownership with Smart Pointers

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v3.2
- **Starting point**: Your committed Chapter 21 v3.1 program (eleven files).

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

v3.1 is correct: its `Gradebook` owns a raw `GradingScheme*` and does three separate things by hand to avoid leaks and double deletes — a destructor, a `delete` inside `setScheme`, and deleted copy operations. In this lab you replace that hand-written bookkeeping with **`std::unique_ptr`**. By the end:

1. `Gradebook` holds a `std::unique_ptr<GradingScheme>`.
2. `Gradebook` has **no destructor**, **no `delete`**, and **no deleted copy operations** — `unique_ptr` handles all of it.
3. `setScheme` takes a `std::unique_ptr<GradingScheme>`, so every caller must visibly hand over ownership.
4. `chooseScheme` creates schemes with **`std::make_unique`** instead of `new`.

The program's **behavior does not change at all** — same menu, same grades. What changes is that its correctness no longer depends on anyone remembering three things.

Files in this checkpoint

File	What you do to it
`gradebook.h`, `gradebook.cpp`	Edit (Steps 2–3)
`gradingscheme.h`	Edit — comments only (Step 4)
`main.cpp`	Edit (Steps 5–6)
All other files	**No change** — keep them open

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

Build it: step by step

Step 1 (try it, then undo it) — See why manual ownership is fragile Before you change anything, see how easy it is to break v3.1.

1. In `gradebook.cpp`, find this line inside `Gradebook::setScheme`:

```
    delete scheme_;          // release the previous scheme before replacing it
```

2. Turn it into a comment by typing `//` at the start of the line.
3. Click **Run**. Choose a scheme, then use **Change scheme** (option 5) two or three times, report, and quit.

The program still prints correct grades and exits normally. But every scheme you replaced was never deleted — the program **leaked** that memory. Nothing in the output tells you, which is why leaks are called *silent*.

> **Note — this is the one check StudySite's Run cannot make for you.** A leak is invisible from the program's output. If you have a local C++ toolchain, build the program with `-fsanitize=address`

`-g` and repeat the experiment: AddressSanitizer reports the leaked allocations when the program exits. Otherwise, ask your instructor which memory check to use.

4. **Remove the `// you added`** so the line is restored exactly as it was. Steps 2–6 assume the original v3.1 code.

Step 2 — Own the scheme with `std::unique_ptr` (`gradebook.h`) In `gradebook.h`, find the first line:

```
// gradebook.h - Grade Calculator v3.1 - Chapter 21
```

Replace it with:

```
// gradebook.h - Grade Calculator v3.2 - Chapter 22
```

Find the standard-library include:

```
#include <iosfwd>
```

Directly below it, add:

```
#include <memory>
```

Inside the class, find the constructor, destructor, deleted copy operations, and `setScheme`:

```
    explicit Gradebook(const std::string& courseName = "Untitled Course");
    ~Gradebook();

    // Copying is disabled: Gradebook owns a raw pointer, and a default copy
    // would leave two objects deleting the same scheme. Chapter 22 replaces
    // this restriction with smart-pointer ownership.
    Gradebook(const Gradebook&) = delete;
    Gradebook& operator=(const Gradebook&) = delete;

    /**
     * Installs a grading scheme. The Gradebook takes ownership.
     * Can be called at any time; every grade is recomputed on the next report.
     */
    void setScheme(GradingScheme* scheme);
```

Replace it with:

```
    explicit Gradebook(const std::string& courseName = "Untitled Course");

    // No destructor is needed now. unique_ptr releases the scheme when the
    // Gradebook is destroyed, and again whenever the scheme is replaced.
    // The compiler-generated destructor is correct, so we do not write one.

    /**
     * Installs a grading scheme. Ownership transfers to the Gradebook.
     * The unique_ptr parameter makes that transfer visible at every call site:
     * a caller must std::move into it, so ownership is never ambiguous.
     */
    void setScheme(std::unique_ptr<GradingScheme> scheme);
```

At the bottom of the class, find:

```
    // One pointer to an abstract base. Which concrete scheme it refers to is
    // not this class's concern - that is what polymorphism buys.
    GradingScheme* scheme_ = nullptr;
```

Replace it with:

```
    // An owning smart pointer to an abstract base. Polymorphism is unchanged;
    // only the ownership question is answered differently.
    std::unique_ptr<GradingScheme> scheme_;
```

- **`std::unique_ptr<GradingScheme>`** is an *owning* pointer from `<memory>`. When the `unique_ptr` is destroyed or given a new object, it deletes the object it held. So the `Gradebook`'s compiler-generated destructor is now correct — you do not write one.
- A `unique_ptr` **cannot be copied**, only moved. That makes `Gradebook` non-copyable automatically, so the two `= delete` lines are no longer needed.
- `setScheme` now takes its parameter **by value as a `unique_ptr`**. A caller cannot pass one without giving up ownership, so who owns the scheme is never in doubt.
- **The polymorphism is untouched.** `scheme_` still points to the abstract base, and calls through it still dispatch to the right scheme.

Step 3 — Simplify the implementation (`gradebook.cpp`) In `gradebook.cpp`, find the first line:

```
// gradebook.cpp - Grade Calculator v3.1 - Chapter 21
```

Replace it with:

```
// gradebook.cpp - Grade Calculator v3.2 - Chapter 22
```

Find:

```
#include <algorithm>
```

Directly below it, add:

```
#include <memory>
#include <utility>
```

Find the constructor, the destructor, and `setScheme`:

```
Gradebook::Gradebook(const std::string& courseName)
    : courseName_(courseName), scheme_(new PointsBased()) {}

Gradebook::~Gradebook() {
    delete scheme_;
}

void Gradebook::setScheme(GradingScheme* scheme) {
    if (scheme == nullptr || scheme == scheme_) { return; }
    delete scheme_;          // release the previous scheme before replacing it
    scheme_ = scheme;
}
```

Replace it with:

```
Gradebook::Gradebook(const std::string& courseName)
    : courseName_(courseName), scheme_(std::make_unique<PointsBased>()) {}

void Gradebook::setScheme(std::unique_ptr<GradingScheme> scheme) {
    if (!scheme) { return; }
    // Assigning to a unique_ptr destroys whatever it held first. This one line
    // replaces v3.1's manual delete, and the destructor and deleted copy
    // operations that had to accompany it.
    scheme_ = std::move(scheme);
}
```

- `std::make_unique<PointsBased>()` creates the starting scheme and wraps it in a `unique_ptr` in one step — no `new` appears anywhere.
- The destructor is **gone**.
- In `setScheme`, `scheme_ = std::move(scheme);` transfers ownership from the parameter into the member. Assigning to a `unique_ptr` **first deletes whatever it held**, so this one line replaces v3.1's manual `delete`. `std::move` (from `<utility>`) is required because a `unique_ptr` cannot be copied.

  Heads-up: `main.cpp` still calls `setScheme(new ...)`, and a raw pointer does not convert to a `unique_ptr` automatically. The program will not compile until the end of Step 6.

Step 4 — Update the ownership note (`gradingscheme.h`) The comment above `class GradingScheme` still describes v3.1's hand-written ownership. Make it describe the code you now have.

In `gradingscheme.h`, find the first line:

```
// gradingscheme.h - Grade Calculator v3.1 - Chapter 21
```

Replace it with:

```
// gradingscheme.h - Grade Calculator v3.2 - Chapter 22
```

Then find the ownership note in the comment block:

```
 * OWNERSHIP NOTE (revisited in Chapter 22): Gradebook owns a raw pointer.
 * That is correct here, but only because Gradebook also writes a destructor,
 * deletes the old scheme before replacing it, and disables copying. Miss any
 * one of those three and the program leaks or double-frees. Chapter 22 shows
 * how to get the same behavior with none of that hand-written ceremony.
```

Replace it with:

```
 * OWNERSHIP NOTE (Chapter 22): Gradebook owns its scheme through a
 * std::unique_ptr, so a scheme is released automatically when it is replaced
 * and when the Gradebook is destroyed. No hand-written destructor, delete, or
 * deleted copy operations are needed.
```

Comments are part of the code. A comment that describes code that no longer exists is a defect, even though the compiler never notices it.

Step 5 — Update the header comment and includes in `main.cpp` At the top of `main.cpp`, find the header comment:

```
// ============================================================================
//  Grade Calculator  v3.1                                        Chapter 21
// ----------------------------------------------------------------------------
//  Change  : GradingScheme is now abstract with a pure virtual computePercentage
//            and a virtual destructor. Gradebook holds a GradingScheme* and
//            never asks which scheme it has - the v3.0 branch is gone.
//            The scheme can be switched MID-SESSION and every grade recomputes.
//            A third scheme (WeightedDropLowest) was added without changing
//            Gradebook, Student, GradeScale, Assignment, or the report code.
//  Cost    : correct raw-pointer ownership needs three pieces of hand-written
//            bookkeeping in Gradebook - a destructor, a delete inside
//            setScheme, and deleted copy operations. This build has all three
//            and does not leak. Chapter 22 removes the need for any of them.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp assignment.cpp gradescale.cpp
//            student.cpp gradingscheme.cpp gradebook.cpp -o gradecalc
// ============================================================================
```

Replace it with:

```
// ============================================================================
//  Grade Calculator  v3.2                                        Chapter 22
// ----------------------------------------------------------------------------
//  Change  : Ownership. The raw GradingScheme* becomes std::unique_ptr.
//            Gradebook loses its destructor, its manual delete, and its
//            deleted copy operations - 8 lines of bookkeeping for 0.
//            Behavior is identical to v3.1; both builds are leak-free.
//  Point   : v3.1 is correct because its author remembered three separate
//            things. v3.2 is correct because there is nothing to remember.
//  Verify  : build both with -fsanitize=address, switch schemes, exit.
//            Neither leaks. Then delete Gradebook::~Gradebook from v3.1 and
//            rerun: v3.1 now leaks, v3.2 cannot be broken the same way.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp assignment.cpp gradescale.cpp
//            student.cpp gradingscheme.cpp gradebook.cpp -o gradecalc
// ============================================================================
```

Below it, find:

```
#include <iostream>
```

Directly below it, add:

```
#include <memory>
#include <utility>
```

Step 6 — Create schemes with `std::make_unique` In `chooseScheme`, find the three `setScheme` calls:

```
    if (!c.empty() && c[0] == '2')      { book.setScheme(new Weighted()); }
    else if (!c.empty() && c[0] == '3') { book.setScheme(new WeightedDropLowest()); }
    else                                { book.setScheme(new PointsBased()); }
```

Replace it with:

```
    if (!c.empty() && c[0] == '2') {
        book.setScheme(std::make_unique<Weighted>());
    } else if (!c.empty() && c[0] == '3') {
        book.setScheme(std::make_unique<WeightedDropLowest>());
    } else {
        book.setScheme(std::make_unique<PointsBased>());
    }
```

At the start of `main`, find the banner line:

```
    std::cout << "=== GRADE CALCULATOR v3.1 (switchable schemes) ===\n\n";
```

Replace it with:

```
    std::cout << "=== GRADE CALCULATOR v3.2 (leak-free) ===\n\n";
```

`std::make_unique<Weighted>()` returns a temporary `unique_ptr<Weighted>`, which converts to the `unique_ptr<GradingScheme>` parameter and moves straight into the `Gradebook`. There is no moment when a raw, unowned pointer exists.

Nothing else in `main` changes.

> ▸ **Run checkpoint**: Make sure all eleven files are open and click **Run**. The program should compile with no errors and no warnings and show the `v3.2 (leak-free)` banner.

Check your work

Repeat the Chapter 21 test — it must produce **exactly the same results**. Choose `1` (Points-based) and add `Ada` with `Exam 1` (90/100, bonus 0, `Exam`), `HW1` (10/10, bonus 0, `Homework`), and `HW2` (5/10, bonus 0, `Homework`). Report after each scheme:

Scheme (option 5, then)	Report line
`1` Points-based	`1001 Ada 87.5% B`
`2` Weighted	`1001 Ada 84.4% B`
`3` Weighted, lowest dropped	`1001 Ada 93.8% A`

Then confirm:

- ☐ The program compiles with no errors and no warnings.
- ☐ Searching your project for `delete` and for `new` finds nothing in `gradebook.cpp` or `main.cpp`.
- ☐ `Gradebook` declares no destructor and no `= delete` copy operations.
- ☐ Switching schemes several times and quitting works normally. (If you have a local toolchain, an AddressSanitizer build reports no leaks.)

	v3.1	v3.2
Destructor	hand-written	**not needed**
`delete` in `setScheme`	required	**not needed**
Deleted copy operations	required	**not needed**
Behavior	identical	identical

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click all eleven project files to open them. Confirm the editor shows the expected file paths before editing.
4. Do the Step 1 experiment and undo it, then work through Steps 2–6 above in StudySite's internal editor, in order, and click **Run** at the ▸ Run checkpoint.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — all eleven files. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 22 Grade Calculator v3.2**.

5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 22 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. Raw pointers, smart pointers, and ownership

```
#include <iostream>
#include <memory>
#include <string>

class Scheme {
public:
    explicit Scheme(const std::string& n) : name_(n) {
        std::cout << "  built " << name_ << "\n";
    }
    virtual ~Scheme() { std::cout << "  destroyed " << name_ << "\n"; }
    const std::string& name() const { return name_; }
private:
    std::string name_;
};

class Points   : public Scheme { public: Points()   : Scheme("Points")   {} };
class Weighted : public Scheme { public: Weighted() : Scheme("Weighted") {} };

int main() {
    std::cout << "--- raw pointer, managed by hand ---\n";
    { Scheme* s = new Points(); delete s; }

    std::cout << "--- unique_ptr, managed for you ---\n";
    { std::unique_ptr<Scheme> s = std::make_unique<Points>(); }

    std::cout << "--- reassigning releases the old one ---\n";
    {
        std::unique_ptr<Scheme> s = std::make_unique<Points>();
        s = std::make_unique<Weighted>();
        std::cout << "  now holding " << s->name() << "\n";
    }
    return 0;
}
```

Expected output:

```
--- raw pointer, managed by hand ---
  built Points
  destroyed Points
--- unique_ptr, managed for you ---
  built Points
  destroyed Points
```

```
--- reassigning releases the old one ---
  built Points
  built Weighted
  destroyed Points
  now holding Weighted
  destroyed Weighted
```

Try: Remove the `delete s;` from the first block and rebuild with `-fsanitize=address`. What is reported? Then try removing something from the second block that causes a leak — can you?

2. Shared ownership and counting

```cpp
#include <iostream>
#include <memory>

class Thing {
public:
    Thing()  { std::cout << "  built\n"; }
    ~Thing() { std::cout << "  destroyed\n"; }
};

int main() {
    auto a = std::make_shared<Thing>();
    std::cout << "  use_count " << a.use_count() << "\n";
    {
        auto b = a;
        std::cout << "  use_count " << a.use_count() << "\n";
    }
    std::cout << "  use_count " << a.use_count() << "\n";
    return 0;
}
```

Expected output:

```
  built
  use_count 1
  use_count 2
  use_count 1
  destroyed
```

Try: Note *when* "destroyed" prints. Why not when `b` went out of scope?

3. A leak you can see

```cpp
#include <iostream>

int main() {
    for (int k = 0; k < 5; ++k) {
        double* p = new double(k * 1.5);
        std::cout << *p << " ";
        // no delete
    }
    std::cout << "\n";
    return 0;
}
```

Expected output:

```
0 1.5 3 4.5 6
```

Try: The program looks fine. Build with `-fsanitize=address -g` and rerun. How many allocations leaked? Then fix it two ways — with `delete`, and with no pointer at all. Which fix is better?

4. Ownership transfer with `std::move`

```cpp
#include <iostream>
#include <memory>
#include <string>

int main() {
    std::unique_ptr<std::string> a = std::make_unique<std::string>("Ada");
    std::cout << "a holds: " << *a << "\n";

    std::unique_ptr<std::string> b = std::move(a);
    std::cout << "b holds: " << *b << "\n";
    std::cout << "a is now " << (a ? "non-empty" : "empty") << "\n";
    return 0;
}
```

Expected output:

```
a holds: Ada
b holds: Ada
a is now empty
```

Try: Replace `std::move(a)` with plain `a` and read the error. Why does `unique_ptr` refuse to be copied?

5. RAII with something other than memory

```cpp
#include <fstream>
#include <iostream>

int main() {
    {
        std::ofstream out("raii.txt");
        out << "written inside the block\n";
    }   // out goes out of scope here — the file is closed automatically

    std::ifstream in("raii.txt");
    std::string line;
    std::getline(in, line);
    std::cout << "read back: " << line << "\n";
    return 0;
}
```

Expected output:

```
read back: written inside the block
```

Try: You never called `close()`. Explain in one sentence, using the word destructor, why the file was nevertheless closed and flushed.

6. Rule of Three

This class leaks and can double-free. Identify both problems, then fix it two ways: by following the Rule of Three, and by using `std::unique_ptr`.

```
class Holder {
public:
    Holder() : data_(new double[100]) {}
private:
    double* data_;
};
```

Which fix is shorter? Which is harder to get wrong?

7. Reason about memory

- What is the difference between the stack and the heap, in one sentence each?
- Why is a memory leak harder to notice than a crash?
- What does RAII mean, and which destructor makes it work?
- Why can a `unique_ptr` not be copied?
- You see `new` in unfamiliar code with no smart pointer nearby. What are you looking for?
- Give one case where `shared_ptr` is right and `unique_ptr` is not.

Summary

- A program uses four memory regions: **code**, **static**, **stack**, and **heap**. **The stack is managed for you; the heap is not**.
- **Automatic storage** is destroyed when its scope ends. **Dynamic storage** obtained with `new` stays until `delete`.
- Three failure modes: a **memory leak** (never released), a **dangling pointer** (used after release), and a **double free** (released twice). All are undefined or silent.
- **AddressSanitizer** (`-fsanitize=address -g`) detects all three. Leaks are invisible without it.
- **RAII** ties a resource's lifetime to an object's lifetime: acquire in the constructor, release in the destructor. `std::vector`, `std::string`, and `std::ofstream` all work this way.
- **The Rule of Three**: a class managing a resource directly needs a destructor, a copy constructor, and copy assignment — or must forbid copying.
- **`std::unique_ptr`** has one owner and releases automatically; assigning to one releases what it held. It cannot be copied, only **moved**.
- **`std::shared_ptr`** counts owners and releases when the last goes away. Prefer `unique_ptr`.
- **v3.1 did not leak.** It was correct because three separate pieces of bookkeeping were remembered. **v3.2 is correct because there is nothing to remember** — which is the real argument for RAII.

Key Terms

automatic storage — memory for local variables, released when scope ends.

dangling pointer — a pointer to memory that has been released.

double free — releasing the same memory twice.

dynamic storage — heap memory obtained with `new` and released with `delete`.

heap — the memory region for dynamically allocated objects.

linked list — a chain of nodes, each holding a value and a pointer to the next.

memory leak — allocated memory that is never released.

move semantics — transferring a resource from one object to another instead of copying.

RAII — Resource Acquisition Is Initialization; tying resource lifetime to object lifetime.

Rule of Three — needing any of destructor, copy constructor, or copy assignment implies needing all three.

shared_ptr — a smart pointer with reference counting and shared ownership.

smart pointer — an object that owns a pointer and releases it automatically.

stack — the memory region holding function frames and local variables.

unique_ptr — a smart pointer with sole ownership, releasing on destruction.

Next: Chapter 23 makes your calculator generic. A `Statistics<T>` template computes mean, median, and standard deviation for any numeric type; `std::map` indexes students by ID and counts the grade distribution; and lambdas state a sort's ordering where it is used rather than hiding it in `operator<`. Grade Calculator v3.3.

Chapter 23 — Templates and the Standard Template Library

Learning Objectives

When you finish this chapter you will be able to:

- Write a function template and a class template. *(SLO 2.8)*
- Explain what instantiation means and when it happens. *(SLO 2.8)*
- Choose an appropriate STL container for a purpose. *(SLO 2.5, 2.6)*
- Use iterators and iterator ranges. *(SLO 2.5)*
- Use `std::map` for lookup, indexing, and counting. *(SLO 2.5, 2.6)*
- Write lambda functions and capture values. *(SLO 2.5)*
- Apply standard algorithms rather than hand-written loops. *(SLO 2.5)*
- Explain what namespaces are for. *(SLO 2.8)*
- Build Grade Calculator v3.3 — statistics, grade distribution, and indexed lookup.

23.1 The Case for Generic Code

Suppose you want the mean of some values. For `double`:

```
double mean(const std::vector<double>& v);
```

For `int`, you write it again. For `float`, again. Three functions, one idea, three places for a bug to live.

The computation does not depend on the type. It needs values that can be added and divided, and nothing else.

A template writes the family of functions for you.

You have used templates since Chapter 12. `std::vector<double>` and `std::vector<std::string>` are two instances of one template. This chapter shows how that works and how to write your own.

23.2 Function Templates

```
template <typename T>
T larger(T a, T b) {
    return (a > b) ? a : b;
}
```

`template <typename T>` says: what follows is a pattern, with `T` standing for a type to be supplied later.

```
larger(3, 7);          // T is int
larger(2.5, 1.5);      // T is double
larger('a', 'z');      // T is char
```

The compiler works out `T` from the arguments and **instantiates** a version for each type actually used. This happens at compile time, so there is no run-time cost — unlike the polymorphism in Chapter 21, which decides at run time.

A template is not code until it is instantiated. A template nobody calls is never fully checked, so an error in it may not appear until the first use.

Requirements are implicit

`larger` requires that `T` supports `>`. Nothing states that, and if you instantiate it with a type lacking `>`, you get an error message pointing *inside* the template rather than at your call.

Template error messages have a reputation for being long, and this is why. **Read from the bottom** — the last few lines usually name your call site and the operation that was missing.

23.3 Class Templates

A class can be a template too:

```
template <typename T>
class Statistics {
public:
    void add(T value) { values_.push_back(value); }

    std::size_t count() const { return values_.size(); }

    T mean() const {
        if (values_.empty()) { return T{}; }
        T sum = T{};
        for (T v : values_) { sum += v; }
        return sum / static_cast<T>(values_.size());
    }

private:
    std::vector<T> values_;
};
```

The type is given explicitly, because there are no arguments to deduce it from:

```
Statistics<double> percentages;
Statistics<int> counts;
```

```
double: n=3 mean=75 median=75 stdev=16.3299
int:    n=5 mean=2 median=3
```

Two details worth noticing.

`T{}` is a value-initialized `T` — zero for numeric types. Writing `0` would fail for types where zero is not a valid literal.

`Statistics<int>` reports a mean of 2 for the values 3, 1, 4, 1, 5. The true mean is 2.8. This is **integer division**, from Chapter 4 Section 4.3, arriving inside a template. The template is not wrong; it faithfully does `T` arithmetic, and for `T = int` that truncates.

This is a genuine hazard of generic code: **a template can be correct for every type and still surprise you for a particular one.** The fix is to compute in `double` and return `T`, or to document that integer instantiations truncate. The Grade Calculator uses `Statistics<double>`, so it does not arise — but knowing why is what lets you avoid it elsewhere.

Templates live in headers

A template must be visible where it is instantiated, so **template definitions go in the header**, not in a `.cpp`. This is the one place Appendix D Section D.4's declaration/definition split does not apply. `statistics.h` in v3.3 contains complete definitions.

23.4 The Standard Template Library

The **STL** is the part of the standard library built from templates: containers, iterators, and algorithms.

Container	Header	Good at	Poor at
`std::vector`	`<vector>`	indexing, adding at the end	inserting in the middle
`std::map`	`<map>`	lookup by key, sorted order	nothing much; log-time everything
`std::unordered_map`	`<unordered_map>`	fastest lookup by key	no ordering
`std::set`	`<set>`	membership, uniqueness, sorted	indexing
`std::list`	`<list>`	inserting anywhere	indexing
`std::deque`	`<deque>`	adding at both ends	slightly slower indexing

Use `std::vector` unless you have a reason not to. It is the default for good reasons: contiguous memory, fast traversal, simple mental model. The Grade Calculator uses vectors everywhere and adds one `map` in this chapter.

23.5 Iterators

An **iterator** identifies a position in a container. It generalizes the pointer:

```
std::vector<double> v = {1.0, 2.0, 3.0};

for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << " ";       // * gives the value, like a pointer
}
```

`v.begin()` is the first element. `v.end()` is **one past the last** — not the last element, which is why the test is `!=` rather than `<=`.

That convention makes an empty range natural: `begin() == end()` means nothing is there.

A pair of iterators is a **range**, and it is what standard algorithms take:

```
std::sort(v.begin(), v.end());
```

A range need not be a whole container — `std::sort(v.begin(), v.begin() + 3)` sorts the first three.

Prefer a range-based `for` when you want every element. Iterators matter when you need part of a container, a position, or an algorithm.

23.6 `std::map`

A **map** stores key–value pairs, keeps them sorted by key, and finds by key in logarithmic time.

```
#include <map>

std::map<int, std::string> byId;
byId[1003] = "Alan";
byId[1001] = "Ada";
byId[1002] = "Grace";
```

```
  1001 Ada
  1002 Grace
  1003 Alan
```

Inserted in one order, iterated in another. A map is always sorted by key.

Lookup

```
auto it = byId.find(1002);
if (it != byId.end()) {
    std::cout << it->second;       // "Grace"
}
```

`find` returns an iterator, and `end()` means not found. Each element is a pair: `it->first` is the key, `it->second` is the value.

```
find(1002): Grace
find(9999): not found
```

> **`[]` on a map inserts**. `byId[9999]` creates an entry with a default-constructed value if none exists. That makes `if (byId[id] == "")` quietly grow your map. **Use `find` to look up; use `[]` only when you intend to insert or overwrite.**

Counting

That insert-on-access behavior is exactly right for counting:

```
std::map<char, int> distribution;
for (char c : {'A','B','A','F','B','A'}) {
    ++distribution[c];             // missing keys start at 0
}
```

```
  A=3
  B=2
  F=1
```

Three lines for a complete tally, sorted, with no initialization and no check for whether a letter has been seen. This is one of the most useful small patterns in C++.

`std::unordered_map`

Same interface, faster lookup, **no ordering**. Use it when you never iterate in sorted order and lookup speed matters. For a grade distribution you want the sorted order, so `std::map` is the right choice.

23.7 Lambda Functions

A **lambda** is an unnamed function written where it is used:

```
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
```

```
9 5 2 1
```

The `[]` is the **capture list**, then parameters, then the body.

Capturing

A lambda can use variables from the surrounding scope by capturing them:

```
double threshold = 4.0;
auto passing = std::count_if(v.begin(), v.end(),
                             [threshold](int x) { return x > threshold; });
```

```
above 4: 2
```

Capture	Meaning
`[]`	capture nothing
`[x]`	capture x by value (a copy)
`[&x]`	capture x by reference
`[=]`	capture everything used, by value
`[&]`	capture everything used, by reference

Prefer naming what you capture. `[threshold]` says exactly what the lambda depends on; `[=]` and `[&]` hide it, and `[&]` can leave a dangling reference if the lambda outlives what it captured.

Why this matters for the Grade Calculator

Chapter 19 Section 19.4 raised a concern about `Student::operator<`:

```
std::sort(roster_.begin(), roster_.end());   // sorted by... what?
```

You have to go and read `operator<` to find out — and discover it means "highest percentage first", which is not what < suggests.

A lambda puts the ordering where it is used:

```
std::sort(roster_.begin(), roster_.end(),
          [](const Student& a, const Student& b) {
              return a.percentage() > b.percentage();
          });
```

Longer, and it says what it does. **v3.3 makes this change**, and it is a genuine improvement in readability rather than a use of a new feature for its own sake.

23.8 Standard Algorithms

`<algorithm>` provides tested implementations of common operations.

Algorithm	Does
`std::sort`	sort a range
`std::find`	first element equal to a value
`std::find_if`	first element satisfying a predicate
`std::count` / `std::count_if`	how many match
`std::min_element` / `std::max_element`	smallest / largest
`std::accumulate`	sum a range *(from `<numeric>`)*
`std::for_each`	apply a function to each element
`std::any_of` / `std::all_of`	does any / do all satisfy a predicate
`std::reverse`	reverse in place
`std::transform`	apply a function producing a new range

The pattern is the same throughout: a range, and often a lambda.

```
auto highest = std::max_element(v.begin(), v.end());
auto failing = std::count_if(roster.begin(), roster.end(),
                             [](const Student& s) { return s.percentage() < 60.0; });
```

A named algorithm says what it does; a hand-written loop makes you work it out. `std::count_if` announces its purpose in a way that `for` plus `if` plus `++n` does not.

23.9 Namespaces

A **namespace** groups names so they do not collide:

```
namespace grading {
    double computePercentage(double earned, double possible);
}

double p = grading::computePercentage(84.0, 100.0);
```

`std` is the standard library's namespace, which is why everything from it is written `std::`.

Appendix D Section D.11 forbids `using namespace std;` for two reasons. It pulls thousands of names into scope, inviting collisions with your own — `std::count` and a variable named `count` is a real conflict. And `std::` tells every reader instantly where a name comes from, which is worth five characters.

In a header the rule is absolute: a `using` directive there is forced on every file that includes it.

The Grade Calculator does not define its own namespaces — it is one program with no name conflicts. You will need them as soon as you combine code from several sources.

Common Errors and Warnings

What you see	Cause	Fix
Enormous template error message	A type lacks a required operation	Read from the bottom; find your call site

What you see	Cause	Fix
`undefined reference` to a template function	Definition in a `.cpp`, not the header	Move it to the header
`Statistics<int>` gives a truncated mean	Integer division inside the template	Compute in `double`, or document it
A map grows when you only read from it	`[]` inserts missing keys	Use `find` to look up
`error: no match for 'operator*'` on an iterator	Dereferencing `end()`	Check `it != end()` first
Lambda compiles but sees stale values	Captured by value, expected reference	Capture by reference deliberately
Crash after a lambda outlives its scope	Captured by reference; the referent died	Capture by value
`error: 'map' was not declared`	Missing `#include <map>`	Add the header
Sorting a map fails to compile	A map is already sorted, and by key	Copy into a vector to sort differently

---

Design Notes

Templates go in headers. They must be visible where instantiated.

Use `find` to look up in a map; `[]` only to insert or overwrite.

Name what a lambda captures. `[threshold]` documents the dependency; `[=]` hides it.

Prefer a named algorithm to a hand-written loop. It says what it does.

Put the ordering at the call site. A lambda beats a surprising `operator<`.

Use `std::vector` unless you have a reason not to.

Grade Calculator v3.3 — Generic and Indexed

What v3.3 does

Everything v3.2 did, plus class statistics, a letter-grade distribution, and lookup by student ID — and the sort now states its ordering where it is used.

`Statistics<T>`

A template, in a header, computing over any numeric type:

```
// statistics.h
template <typename T>
class Statistics {
public:
    void add(T value) { values_.push_back(value); }

    std::size_t count() const { return values_.size(); }
    bool empty() const { return values_.empty(); }

    T mean() const;
    T median() const;
    T lowest() const;
    T highest() const;

    /** Population standard deviation. Zero for fewer than two values. */
    double standardDeviation() const;
```

```
private:
    std::vector<T> values_;
};
```

`median` copies before sorting, so the caller's data is untouched:

```
T median() const {
    if (values_.empty()) { return T{}; }
    std::vector<T> sorted = values_;          // copy: do not disturb the caller
    std::sort(sorted.begin(), sorted.end());
    std::size_t mid = sorted.size() / 2;
    if (sorted.size() % 2 == 1) { return sorted[mid]; }
    return (sorted[mid - 1] + sorted[mid]) / static_cast<T>(2);
}
```

Why a template here at all? The Grade Calculator only ever uses `Statistics<double>`. The honest answer is that it demonstrates the technique, and that the class is genuinely reusable elsewhere. A template used at exactly one type is not obviously better than a plain class — this one earns its keep by being independently testable and portable to your next program, not by being generic in this one.

An ID index with `std::map`

```
private:
    // An index from student ID to roster position. std::map keeps its keys
    // sorted and looks up in logarithmic time, replacing the hand-written
    // binary search of Chapter 17 - and it stays correct as students are added.
    std::map<int, std::size_t> idIndex_;
```

```
const Student* Gradebook::findById(int id) const {
    auto it = idIndex_.find(id);
    if (it == idIndex_.end()) { return nullptr; }
    return &roster_[it->second];
}
```

Compare with Chapter 17. The binary search needed the roster sorted by ID, so searching meant sorting a copy first — and any change to the roster invalidated the assumption. **The map maintains itself.**

Returning `nullptr` for "not found" is the same convention Chapter 17 used, with the same weakness: a caller who forgets to check will dereference it. Chapter 24 replaces it.

The index must be maintained

```
void Gradebook::addStudent(const Student& s) {
    roster_.push_back(s);
    idIndex_[s.id()] = roster_.size() - 1;
}
```

And critically:

```
void Gradebook::sortByPercentage() {
    const Gradebook* self = this;
    std::sort(roster_.begin(), roster_.end(),
              [self](const Student& a, const Student& b) {
                  return self->percentageFor(a) > self->percentageFor(b);
              });
    // Sorting moved every student, so the ID index must be rebuilt.
    idIndex_.clear();
```

```
    for (std::size_t i = 0; i < roster_.size(); ++i) {
        idIndex_[roster_[i].id()] = i;
    }
}
```

An index is derived data, and derived data goes stale. Sorting invalidates every stored position, so the index is rebuilt. Forgetting this would produce a silent logic error of exactly the kind Chapter 4 Section 4.9.3 described — lookups returning the wrong student.

That rebuild is the real cost of the index, and it is worth stating plainly: an index is only worth keeping when lookups substantially outnumber the operations that invalidate it.

The lambda captures `this`

Note `[self]` in the sort. The comparison needs `percentageFor`, which is a member function, so the lambda must reach the object. Capturing a pointer to it explicitly — rather than the terser `[this]` or `[=]` — makes the dependency visible, per the Design Notes.

Grade distribution

```
std::map<char, int> Gradebook::letterDistribution() const {
    std::map<char, int> counts;
    for (const Student& s : roster_) {
        ++counts[scale_.letterFor(percentageFor(s))];   // missing keys start at 0
    }
    return counts;
}
```

The counting pattern from Section 23.6, and the result arrives sorted by letter for free.

Expected output

Three students at 95%, 75%, and 55%:

```
  Students : 3
  Mean     : 75.0%
  Median   : 75.0%
  Range    : 55.0% to 95.0%
  Std dev  : 16.3
  Grades   : A=1  C=1  F=1
```

Check the standard deviation by hand: deviations from the mean are +20, 0, −20; squares 400, 0, 400; mean square 800/3 ≈ 266.67; square root ≈ 16.33.

Your StudySite Lab — Add Generic Statistics and an ID Index

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v3.3
- **Starting point**: Your committed Chapter 22 v3.2 program (eleven files).

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

In this lab you put the Standard Template Library to work and write a class template of your own. By the end you will have:

1. **`Statistics<T>`**, a header-only class template that computes count, mean, median, lowest, highest, and standard deviation for any numeric type.
2. An **ID index** — a `std::map<int, std::size_t>` from student ID to roster position — that `Gradebook` keeps up to date, and a `findById` member that uses it.
3. A **letter-grade distribution** counted with a `std::map<char, int>`.
4. `sortByPercentage` rewritten with a **lambda** that states the ordering right where the sort happens.
5. Two new menu options: **Statistics** and **Find by ID**. Quit moves to option 8.

Files in this checkpoint

File	What you do to it
`statistics.h`	Create (Step 1) — a header only; there is no `statistics.cpp`
`gradebook.h`, `gradebook.cpp`	Edit (Steps 2–5)
`main.cpp`	Edit (Step 6)
All other files	**No change** — keep them open

When you are finished your project has **twelve files**.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

Build it: step by step

Step 1 — Write the `Statistics<T>` class template Create a new file named `statistics.h` and enter this code:

```
// statistics.h - Grade Calculator v3.3 - Chapter 23
#ifndef STATISTICS_H
#define STATISTICS_H

#include <algorithm>
#include <cmath>
#include <vector>

/**
 * Descriptive statistics over any numeric type.
 *
 * Written as a template because the computation does not depend on whether the
 * values are double percentages, int point totals, or float weights. One
```

```
 * definition serves them all; the compiler generates a version per type used.
 */
template <typename T>
class Statistics {
public:
    void add(T value) { values_.push_back(value); }

    std::size_t count() const { return values_.size(); }
    bool empty() const { return values_.empty(); }

    T mean() const {
        if (values_.empty()) { return T{}; }
        T sum = T{};
        for (T v : values_) { sum += v; }
        return sum / static_cast<T>(values_.size());
    }

    T median() const {
        if (values_.empty()) { return T{}; }
        std::vector<T> sorted = values_;          // copy: do not disturb the caller
        std::sort(sorted.begin(), sorted.end());
        std::size_t mid = sorted.size() / 2;
        if (sorted.size() % 2 == 1) { return sorted[mid]; }
        return (sorted[mid - 1] + sorted[mid]) / static_cast<T>(2);
    }

    T lowest() const {
        return values_.empty() ? T{} : *std::min_element(values_.begin(), values_.end());
    }

    T highest() const {
        return values_.empty() ? T{} : *std::max_element(values_.begin(), values_.end());
    }

    /** Population standard deviation. Zero for fewer than two values. */
    double standardDeviation() const {
        if (values_.size() < 2) { return 0.0; }
        double m = static_cast<double>(mean());
        double sumSquares = 0.0;
        for (T v : values_) {
            double diff = static_cast<double>(v) - m;
            sumSquares += diff * diff;
        }
        return std::sqrt(sumSquares / static_cast<double>(values_.size()));
    }

private:
    std::vector<T> values_;
};

#endif
```

- `template <typename T>` makes `Statistics` a **blueprint**. When you write `Statistics<double>`, the compiler generates a version of the class with every `T` replaced by `double`.
- The entire template is in the header. The compiler needs the full definition at the point where `Statistics<double>` is used, so a class template normally has **no `.cpp` file**.
- `T{}` is a value-initialized `T` — `0.0` for a `double` — and is returned when there is no data.
- `median()` **copies** the values before sorting, so asking for the median does not rearrange the caller's data.
- `std::min_element` and `std::max_element` from `<algorithm>` return iterators; the `*` in front reads the

value they point to.

Step 2 — Declare the new `Gradebook` members (`gradebook.h`) In `gradebook.h`, find the first line:

```
// gradebook.h - Grade Calculator v3.2 - Chapter 22
```

Replace it with:

```
// gradebook.h - Grade Calculator v3.3 - Chapter 23
```

Find the standard-library include:

```
#include <iosfwd>
```

Directly below it, add:

```
#include <map>
```

Find the declaration of `sortByPercentage`:

```
    void sortByPercentage();
```

Directly below it, add:

```
    /** Finds a student by ID in O(log n) using the id index. Null if absent. */
    const Student* findById(int id) const;

    /** How many students earned each letter grade, in scale order. */
    std::map<char, int> letterDistribution() const;

    /** Prints mean, median, range, and standard deviation for the class. */
    void printStatistics(std::ostream& out) const;
```

In the `private:` section, find:

```
    std::vector<Student> roster_;
```

Directly below it, add:

```
    // An index from student ID to roster position. std::map keeps its keys
    // sorted and looks up in logarithmic time, replacing the hand-written
    // binary search of Chapter 17 - and it stays correct as students are added.
    std::map<int, std::size_t> idIndex_;
```

- `findById` returns a **pointer** to the student, or `nullptr` if there is no such ID. A pointer can say "nothing found"; a reference cannot.
- `letterDistribution` returns a `std::map<char, int>`: each letter grade mapped to how many students earned it.
- `printStatistics` writes to whatever `std::ostream` it is given — the screen today, a file tomorrow.
- `idIndex_` maps an ID (the **key**) to the student's position in `roster_` (the **value**). A `std::map` keeps its keys sorted and finds a key in logarithmic time — the job Chapter 17's binary search did, without needing a sorted copy of the roster.

Step 3 — Keep the index current, and look students up (`gradebook.cpp`) In `gradebook.cpp`, find the first line:

```
// gradebook.cpp - Grade Calculator v3.2 - Chapter 22
```

Replace it with:

```
// gradebook.cpp - Grade Calculator v3.3 - Chapter 23
```

Find the first includes:

```
#include "gradebook.h"

#include <algorithm>
```

Replace it with:

```
#include "gradebook.h"

#include "statistics.h"

#include <algorithm>
#include <map>
```

Find `addStudent`:

```
void Gradebook::addStudent(const Student& s) {
    roster_.push_back(s);
}
```

Replace it with:

```
void Gradebook::addStudent(const Student& s) {
    roster_.push_back(s);
    idIndex_[s.id()] = roster_.size() - 1;
}

const Student* Gradebook::findById(int id) const {
    auto it = idIndex_.find(id);
    if (it == idIndex_.end()) { return nullptr; }
    return &roster_[it->second];
}
```

- In `addStudent`, the new student was just pushed onto the end of the roster, so its position is `roster_.size() - 1`. That position is stored in the index under the student's ID.
- `idIndex_.find(id)` returns an **iterator**. If it equals `idIndex_.end()`, the ID is not in the map. Otherwise `it->second` is the stored position. `auto` lets the compiler spell out the long iterator type for you.

  ▸ **Run checkpoint**: Open `statistics.h` along with the other files and click **Run**. The program should compile cleanly and behave exactly like v3.2. `letterDistribution` and `printStatistics` are declared but not yet defined; that is allowed as long as nothing calls them, and nothing does yet.

Step 4 — Count letter grades and compute statistics (`gradebook.cpp`) Directly below the `findById` function you just added — after this code:

```
    return &roster_[it->second];
}
```

Directly below it, add:

```
std::map<char, int> Gradebook::letterDistribution() const {
    std::map<char, int> counts;
    for (const Student& s : roster_) {
        ++counts[scale_.letterFor(percentageFor(s))];   // missing keys start at 0
    }
```

```
    return counts;
}

void Gradebook::printStatistics(std::ostream& out) const {
    if (roster_.empty()) {
        out << "No students, so there are no statistics.\n";
        return;
    }
    Statistics<double> stats;
    for (const Student& s : roster_) { stats.add(percentageFor(s)); }

    out << std::fixed << std::setprecision(1);
    out << "  Students : " << stats.count() << "\n";
    out << "  Mean     : " << stats.mean() << "%\n";
    out << "  Median   : " << stats.median() << "%\n";
    out << "  Range    : " << stats.lowest() << "% to " << stats.highest() << "%\n";
    out << "  Std dev  : " << stats.standardDeviation() << "\n";

    out << "  Grades   : ";
    for (const auto& entry : letterDistribution()) {
        out << entry.first << "=" << entry.second << "  ";
    }
    out << "\n";
}
```

(The first line in the box is blank on purpose: leave one blank line between `findById` and the new code.)

- `++counts[letter]` is the standard **counting idiom** for a map. If `letter` is not in the map yet, `operator[]` inserts it with the value `0`, and `++` makes it `1`. Because a `std::map` is sorted by key, the counts come back in letter order.
- `printStatistics` fills a `Statistics<double>` with every student's course percentage, then prints the results. It uses `percentageFor`, so the statistics follow the chosen grading scheme.
- `for (const auto& entry : letterDistribution())` walks the map; `entry.first` is the letter and `entry.second` is the count.

Step 5 — Sort with a lambda, and rebuild the index (`gradebook.cpp`) In `sortByPercentage`, find:

```
    std::sort(roster_.begin(), roster_.end());   // uses Student::operator<
```

Replace it with:

```
    // A lambda states the ordering right where it is used, instead of relying
    // on operator< to mean "by percentage, descending" everywhere.
    const Gradebook* self = this;
    std::sort(roster_.begin(), roster_.end(),
              [self](const Student& a, const Student& b) {
                  return self->percentageFor(a) > self->percentageFor(b);
              });
    // Sorting moved every student, so the ID index must be rebuilt.
    idIndex_.clear();
    for (std::size_t i = 0; i < roster_.size(); ++i) {
        idIndex_[roster_[i].id()] = i;
    }
```

- The **lambda** `[self](const Student& a, const Student& b) { ... }` is the comparison, written right where it is used. You no longer have to go read `Student::operator<` to find out what order the sort produces.
- The lambda needs `percentageFor`, a `Gradebook` member, so it **captures** a pointer to the gradebook, `self`,

by listing it in the square brackets.

- **An index is derived data, and derived data goes stale.** Sorting moves every student to a new position, so every position stored in `idIndex_` is now wrong. The loop clears the index and rebuilds it from the sorted roster. Forget this, and *Find by ID* silently returns the wrong student.

Step 6 — Add *Statistics* and *Find by ID* to the menu (`main.cpp`) At the top of `main.cpp`, find these lines of the header comment:

```
//  Grade Calculator  v3.2                                          Chapter 22
// ------------------------------------------------------------------------------
//  Change  : Ownership. The raw GradingScheme* becomes std::unique_ptr.
//            Gradebook loses its destructor, its manual delete, and its
//            deleted copy operations - 8 lines of bookkeeping for 0.
//            Behavior is identical to v3.1; both builds are leak-free.
//  Point   : v3.1 is correct because its author remembered three separate
//            things. v3.2 is correct because there is nothing to remember.
//  Verify  : build both with -fsanitize=address, switch schemes, exit.
//            Neither leaks. Then delete Gradebook::~Gradebook from v3.1 and
//            rerun: v3.1 now leaks, v3.2 cannot be broken the same way.
```

Replace it with:

```
//  Grade Calculator  v3.3                                          Chapter 23
// ------------------------------------------------------------------------------
//  Change  : Templates and the STL. Statistics<T> computes mean, median,
//            range, and standard deviation for any numeric type. std::map
//            indexes students by ID and counts the letter distribution.
//            Sorting is expressed with a lambda instead of operator<.
```

At the start of `main`, find the banner line:

```
    std::cout << "=== GRADE CALCULATOR v3.2 (leak-free) ===\n\n";
```

Replace it with:

```
    std::cout << "=== GRADE CALCULATOR v3.3 (statistics) ===\n\n";
```

Find the menu prompt:

```
        std::cout << "\n1) Add student  2) Report  3) Sort  "
                     "4) Custom scale  5) Change scheme  6) Quit\nChoice: ";
```

Replace it with:

```
        std::cout << "\n1) Add student  2) Report  3) Sort  "
                     "4) Custom scale  5) Change scheme\n"
                     "6) Statistics  7) Find by ID  8) Quit\nChoice: ";
```

Near the end of the menu loop, find:

```
        } else if (c == '6') {
            running = false;
        } else {
            std::cout << "  Please enter 1-6.\n";
```

Replace it with:

```
        } else if (c == '6') {
            std::cout << "\n--- Class statistics ---\n";
            book.printStatistics(std::cout);
        } else if (c == '7') {
```

```
            std::string s = readLine("  Student ID: ");
            int id = 0;
            try { id = std::stoi(s); }
            catch (...) { std::cout << "  Not a valid ID.\n"; continue; }
            const Student* found = book.findById(id);
            if (found != nullptr) {
                std::cout << "  " << found->name() << ": "
                          << book.percentageFor(*found) << "%\n";
            } else {
                std::cout << "  No student with ID " << id << ".\n";
            }
        } else if (c == '8') {
            running = false;
        } else {
            std::cout << "  Please enter 1-8.\n";
```

- **Statistics** (option 6) passes `std::cout` to `printStatistics`.
- **Find by ID** (option 7) converts the typed ID with `std::stoi`, catching bad input, then asks `book.findById(id)`. Always compare the result with `nullptr` before using it — dereferencing a null pointer crashes the program.
- **Quit** moves to option 8.

▸ **Run checkpoint**: Make sure all twelve files are open and click **Run**. The program should compile with no errors and no warnings and show the eight-option menu.

Check your work

Choose 1 (Points-based). Add three students, each with one assignment (category: press Enter):

Student	Assignment	Earned	Possible	Bonus
Ana	Test	95	100	0
Ben	Test	75	100	0
Cy	Test	55	100	0

1. Choose 6 (Statistics). You should see:

```
--- Class statistics ---
  Students : 3
  Mean     : 75.0%
  Median   : 75.0%
  Range    : 55.0% to 95.0%
  Std dev  : 16.3
  Grades   : A=1  C=1  F=1
```

   Check the standard deviation by hand: the deviations from the mean are +20, 0, −20; their squares average $800 \div 3 \approx 266.67$; $\sqrt{266.67} \approx 16.3$.

2. Choose 7 and enter `1002`. You should see `Ben: 75.0%`.
3. Choose 7 and enter `99`. You should see `No student with ID 99.`
4. Choose 3 (Sort), then 7 with `1002` again. It must still find `Ben` — proof that the index was rebuilt after sorting.
5. Choose 8 (Quit).

Then confirm:

- ☐ The program compiles with no errors and no warnings.
- ☐ `statistics.h` has an include guard and there is no `statistics.cpp`.
- ☐ `addStudent` updates `idIndex_`, and `sortByPercentage` rebuilds it.
- ☐ Every use of `findById` checks for `nullptr`.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click all eleven project files to open them. You will create `statistics.h` during the lab. Confirm the editor shows the expected file paths before editing.
4. Work through Steps 1–6 above in StudySite's internal editor, in order, clicking **Run** at each ▸ Run checkpoint.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work:** StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — all twelve files, including the new `statistics.h`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 23 Grade Calculator v3.3**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 23 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

Try It Yourself

1. A class template at two types

```
#include <algorithm>
#include <cmath>
```

```
#include <iostream>
#include <vector>

template <typename T>
class Statistics {
public:
    void add(T v) { values_.push_back(v); }
    std::size_t count() const { return values_.size(); }
    T mean() const {
        if (values_.empty()) { return T{}; }
        T s = T{};
        for (T v : values_) { s += v; }
        return s / static_cast<T>(values_.size());
    }
    T median() const {
        if (values_.empty()) { return T{}; }
        std::vector<T> s = values_;
        std::sort(s.begin(), s.end());
        std::size_t m = s.size() / 2;
        return s.size() % 2 ? s[m] : (s[m - 1] + s[m]) / static_cast<T>(2);
    }
private:
    std::vector<T> values_;
};

int main() {
    Statistics<double> pct;
    for (double v : {95.0, 55.0, 75.0}) { pct.add(v); }
    std::cout << "double: mean=" << pct.mean() << " median=" << pct.median() << "\n";

    Statistics<int> counts;
    for (int v : {3, 1, 4, 1, 5}) { counts.add(v); }
    std::cout << "int:    mean=" << counts.mean() << " median=" << counts.median() << "\n";
    return 0;
}
```

Expected output:

```
double: mean=75 median=75
int:    mean=2 median=3
```

Try: The integer mean should be 2.8. Explain the 2. Then change `mean` to compute in `double` and return `T` — what does the integer version report now?

2. `std::map` for lookup

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<int, std::string> byId;
    byId[1003] = "Alan";
    byId[1001] = "Ada";
    byId[1002] = "Grace";

    for (const auto& entry : byId) {
        std::cout << "  " << entry.first << " " << entry.second << "\n";
    }
```

```
    auto it = byId.find(1002);
    std::cout << "find(1002): " << (it != byId.end() ? it->second : "not found") << "\n";
    std::cout << "find(9999): "
              << (byId.find(9999) != byId.end() ? "found" : "not found") << "\n";
    std::cout << "size after lookups: " << byId.size() << "\n";
    return 0;
}
```

Expected output:

```
  1001 Ada
  1002 Grace
  1003 Alan
find(1002): Grace
find(9999): not found
size after lookups: 3
```

Try: Replace `byId.find(9999) != byId.end()` with `byId[9999] != ""` and print the size again. What happened, and why is `find` the right tool for looking up?

3. Counting with a map

```
#include <iostream>
#include <map>

int main() {
    std::map<char, int> dist;
    for (char c : {'A','B','A','F','B','A'}) { ++dist[c]; }
    for (const auto& e : dist) { std::cout << "  " << e.first << "=" << e.second << "\n"; }
    return 0;
}
```

Expected output:

```
  A=3
  B=2
  F=1
```

Try: Note that no key was initialized and no check for "seen before" was needed. Which map behavior makes that work? Now do the same with `std::unordered_map` — what changes about the output?

4. Lambdas and capture

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {5, 2, 9, 1};

    std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
    for (int x : v) { std::cout << x << " "; }
    std::cout << "\n";

    int threshold = 4;
    auto above = std::count_if(v.begin(), v.end(),
                               [threshold](int x) { return x > threshold; });
    std::cout << "above " << threshold << ": " << above << "\n";
```

```
    return 0;
}
```

Expected output:

```
9 5 2 1
above 4: 2
```

Try: Change `[threshold]` to `[]` and read the error. Then change it to `[&threshold]` and modify `threshold` between the definition and the call — does the lambda see the new value?

5. Iterators explicitly

```
#include <iostream>
#include <vector>

int main() {
    std::vector<double> v = {1.0, 2.0, 3.0};

    for (auto it = v.begin(); it != v.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << "\n";
    std::cout << "first: " << *v.begin() << "\n";
    std::cout << "size:  " << (v.end() - v.begin()) << "\n";
    return 0;
}
```

Expected output:

```
1 2 3
first: 1
size:  3
```

Try: Print `*v.end()` and see what happens. Why is `end()` one past the last element rather than the last?

6. Replace loops with algorithms

Rewrite each using a standard algorithm and a lambda:

```
// count students below 60
int failing = 0;
for (const Student& s : roster) {
    if (s.percentage() < 60.0) { ++failing; }
}
```

```
// find the highest percentage
double highest = roster[0].percentage();
for (const Student& s : roster) {
    if (s.percentage() > highest) { highest = s.percentage(); }
}
```

```
// are all students passing?
bool allPass = true;
for (const Student& s : roster) {
    if (s.percentage() < 60.0) { allPass = false; }
}
```

Which version reads better? Does your answer change if the loop body were three lines longer?

7. Reason about templates and containers

- When is a template instantiated, and what does that mean for run-time cost?
- Why must template definitions live in headers?
- Why does `std::map::operator[]` insert, and when is that helpful rather than dangerous?
- What is the difference between `std::map` and `std::unordered_map`, and when does it matter?
- Why prefer `[threshold]` to `[=]`?
- You need to look up a student by ID 10,000 times on a roster that never changes. Which of a linear scan, a sorted vector with binary search, or a map would you choose?

Summary

- A **template** is a pattern from which the compiler generates code for each type used. **Instantiation** happens at compile time, so there is no run-time cost.
- Template requirements are **implicit**, which is why error messages are long. **Read from the bottom.**
- **Template definitions go in headers**, because they must be visible where instantiated.
- A template can be correct for every type and still surprise you for one — `Statistics<int>` truncates because `T` arithmetic on `int` is integer division.
- The **STL** provides containers, iterators, and algorithms. **Use `std::vector` unless you have a reason not to.**
- An **iterator** identifies a position. `begin()` is the first element; `end()` is **one past the last**. A pair of iterators is a **range**.
- **`std::map`** stores key–value pairs sorted by key, with logarithmic lookup. **`[]` inserts missing keys** — use `find` to look up, `[]` to insert or count.
- The **counting idiom** `++counts[key]` needs no initialization and no membership test.
- A **lambda** is an unnamed function written where it is used. **Name what you capture.**
- Putting a sort's ordering in a **lambda at the call site** is clearer than a surprising `operator<`.
- **Standard algorithms** say what they do. Prefer them to hand-written loops.
- **Namespaces** prevent name collisions. Never write `using namespace std;`.
- **An index is derived data and goes stale.** Sorting the roster invalidates the ID index, so it is rebuilt.

Key Terms

algorithm (STL) — a standard function operating on an iterator range.

capture list — the `[]` of a lambda, naming what it takes from the surrounding scope.

class template — a class parameterized by one or more types.

container — a class holding a collection of objects.

function template — a function parameterized by one or more types.

instantiation — the compiler's generation of concrete code from a template for a given type.

iterator — an object identifying a position in a container.

lambda — an unnamed function defined where it is used.

map — an associative container of key–value pairs, sorted by key.

namespace — a named scope grouping declarations to prevent collisions.

range — a pair of iterators marking the beginning and one-past-the-end of a sequence.

STL — the Standard Template Library: containers, iterators, and algorithms.

template parameter — the placeholder type in a template, conventionally `T`.

unordered_map — a hash-based associative container with faster lookup and no ordering.

Next: Chapter 24 ships the release. Errors become exceptions with a custom hierarchy, so an invalid grade scale is refused rather than quietly repaired. You will revisit the translation pipeline from Chapter 2 with full understanding, produce a release build and a user guide, and reconcile your design document against the specification you wrote in Chapter 5 — closing a requirement traced across two courses. Grade Calculator v4.0.

Chapter 24 — Exceptions, Release, and the Road Ahead

Learning Objectives

When you finish this chapter you will be able to:

- Explain why return codes are insufficient for reporting errors. *(SLO 2.1)*
- Throw and catch exceptions. *(SLO 2.1)*
- Define a custom exception hierarchy and catch by specificity. *(SLO 2.2, 2.3)*
- Explain stack unwinding and why RAII makes exceptions safe. *(SLO 2.1)*
- Decide when an exception is appropriate and when it is not. *(SLO 2.1)*
- Describe every phase of program translation from source to executable. *(SLO 2.9)*
- Distinguish static from dynamic libraries, and debug from release builds. *(SLO 2.9)*
- Produce release documentation and reconcile a design document. *(SLO 2.1, 2.4)*
- Build Grade Calculator v4.0 — the release.

24.1 Why Return Codes Are Not Enough

Your calculator has reported failure three ways, and all three have the same weakness.

A sentinel return value. Chapter 17's `linearSearchById` returns −1 for "not found":

```
int index = linearSearchById(roster, id, comparisons);
const Student& s = roster[index];      // if index is -1, undefined behavior
```

A status flag by reference. Chapter 10's `readNonNegative` returns `bool` alongside the value. Better, and still ignorable.

Silent repair. Chapter 18's `GradeScale` quietly fixes an invalid scale. The user's input was wrong and nothing said so.

The common failure: **nothing forces the caller to notice.** Ignoring a return value is legal, silent, and easy. And a function that computes a percentage has no spare `double` to mean "failed" — every value is a legitimate answer.

An exception cannot be ignored. Unhandled, it terminates the program. That is a strong default, and the right one for a condition that must not pass unnoticed.

24.2 `throw`, `try`, and `catch`

```
double computePercentage(double earned, double possible) {
    if (possible <= 0.0) {
        throw std::invalid_argument("points possible must be greater than zero");
    }
    return earned / possible * 100.0;
}
```

`throw` abandons the function immediately. Execution jumps to the nearest enclosing handler:

```
try {
    double pct = computePercentage(earned, possible);
    std::cout << pct << "%\n";
} catch (const std::invalid_argument& e) {
    std::cout << "Cannot compute: " << e.what() << "\n";
}
```

Three parts. The **`try` block** contains code that might throw. The **`catch` clause** names the type it handles and receives the exception object. **`what()`** returns the message.

Always catch by `const` reference. Catching by value slices the exception — Chapter 21 Section 21.4's problem, in a new place, and it discards exactly the derived information you wanted.

24.3 Standard Exception Types

`<stdexcept>` provides a hierarchy rooted at `std::exception`:

Type	Meaning
`std::exception`	the base of all standard exceptions
`std::logic_error`	a fault in the program's logic
`std::invalid_argument`	an argument was unacceptable
`std::out_of_range`	an index or key was outside the valid range
`std::runtime_error`	a fault detectable only while running

You have already met two. `std::vector::at` throws `std::out_of_range` — Chapter 12 Section 12.4. `std::stod` throws `std::invalid_argument` — Chapter 15 Section 15.6.

24.4 A Custom Exception Hierarchy

Standard types are fine for general faults. For your own application, define types that name your own failures — this is the inheritance of Chapter 20 applied to error reporting.

```
/**
 * Base class for every error this application reports.
 *
 * Deriving from std::runtime_error means these can be caught either
 * specifically, or as GradebookError, or as std::exception - a caller chooses
 * how much it wants to know.
 */
class GradebookError : public std::runtime_error {
```

```
public:
    explicit GradebookError(const std::string& what) : std::runtime_error(what) {}
};

/** A grade scale whose tiers do not descend, or that does not reach 0. */
class InvalidScaleError : public GradebookError {
public:
    explicit InvalidScaleError(const std::string& what)
        : GradebookError("Invalid grade scale: " + what) {}
};

/** Category weights that do not total 100 percent. */
class WeightSumError : public GradebookError {
public:
    explicit WeightSumError(double actual)
        : GradebookError(describe(actual)), actual_(actual) {}
    double actual() const { return actual_; }
private:
    static std::string describe(double actual);
    double actual_;
};
```

Note that `WeightSumError` carries **data**, not just a message. A handler can call `actual()` and offer to scale the weights. An exception is an object, and it may hold whatever the handler needs.

24.5 Catching Multiple Exceptions

Several `catch` clauses may follow one `try`. **The first matching one runs**, so order from most specific to least:

```
try {
    // ...
} catch (const InvalidScaleError& e) {
    std::cout << "  caught InvalidScaleError: " << e.what() << "\n";
} catch (const GradebookError& e) {
    std::cout << "  caught GradebookError:    " << e.what() << "\n";
} catch (const std::exception& e) {
    std::cout << "  caught std::exception:    " << e.what() << "\n";
}
```

```
  caught InvalidScaleError: Invalid grade scale: tier 'B' at 85 is not below the previous tier at 80.
  caught GradebookError:    Category weights total 80.0 percent; they must total 100.
  caught std::exception:    something else entirely
```

Three different throws, three different handlers — each caught at the level of detail it needed. A `WeightSumError` is a `GradebookError`, so the second clause caught it.

Reverse the order and the general clause catches everything, making the specific ones unreachable. That is the same first-match-wins rule as Chapter 6 Section 6.9's `if/else if` chain, with the same consequence for getting the order wrong.

`catch (...)` catches anything. Chapter 15 used it as a blunt instrument. It is appropriate at the top of `main` as a last resort, and rarely elsewhere — you cannot examine what you caught.

24.6 Stack Unwinding and Exception Safety

When an exception is thrown, C++ **unwinds the stack**: it leaves each function between the throw and the handler, **running the destructor of every local object** on the way.

```
--- stack unwinding ---
  built a
  built b
  destroyed b
  destroyed a
  handler: boom
```

Both objects were destroyed, in reverse order, before the handler ran.

This is why RAII matters so much in C++. Chapter 22 argued that `std::unique_ptr` is better than manual `delete` because there is nothing to remember. Exceptions make the argument decisive:

```
void risky() {
    GradingScheme* s = new Weighted();
    somethingThatThrows();              // exception thrown here
    delete s;                           // never reached — leaked
}
```

The `delete` is skipped. With a `unique_ptr`, the destructor runs during unwinding and the memory is released.

Manual resource management and exceptions do not mix. Any function that can throw — which is most of them — leaks if it manages a resource by hand. RAII is not merely tidier; it is what makes exceptions usable.

Leave no operation half-finished

The other half of exception safety is a design rule you have already applied. Chapter 15's `loadGradebook` built a complete `Gradebook` in a temporary and installed it only on success, so a file that failed partway left the previous data untouched.

Build the replacement, then install it. An operation that fails should leave the object as it was.

24.7 When to Use Exceptions

Not for everything.

Use an exception when a function cannot do what its name promises, the caller must not proceed as though it had, and there is no natural return value meaning failure.

Do not use an exception for ordinary control flow, or for expected conditions. A user typing `abc` where a number belongs is **expected** — Chapter 10's validation loop handles it by re-prompting, which is correct. Throwing there would be using an exception as a `goto`.

Situation	Handle it how
User types text where a number belongs	Re-prompt — expected
Menu choice out of range	Re-prompt — expected
Grade scale tiers out of order	**Throw** — the object cannot be built
Category weights do not total 100	**Throw** — the scheme would be wrong
Gradebook file is malformed	**Throw** — loading cannot proceed
A student ID is not found	Return `nullptr` or an optional — a normal outcome

That last row is a judgment worth defending. "Not found" is a normal result of searching, not a failure — so v4.0 keeps returning `nullptr` from `findById` rather than throwing.

24.8 Program Translation, Revisited

Chapter 2 Section 2.2 described four stages before you knew what a linker error meant. This is SLO 2.9, and you can now understand every part of it.

The four stages

Stage	Tool	Input	Output
1	Preprocessor	`main.cpp`	translation unit
2	Compiler	translation unit	assembly
3	Assembler	assembly	object file `main.o`
4	Linker	object files + libraries	executable

Preprocessing carries out `#include` and `#define`. Chapter 2 measured a six-line program expanding to 32,192 lines. You now know why headers should include as little as possible — Chapter 19 Section 19.3's `<iosfwd>` — because every file including yours pays for whatever it pulls in.

Compilation checks syntax and types and emits assembly. Templates are instantiated here, which is why a template must be visible where it is used — Chapter 23 Section 23.3.

Assembly produces machine code with unfilled references.

Linking resolves those references. You have now caused this failure deliberately several times: a prototype with no definition (Chapter 9), a `.cpp` omitted from the build (Chapter 10), a `static` member declared but not defined (Chapter 19). All three are the same thing — the compiler accepted a promise nobody kept.

Separate compilation

Each `.cpp` compiles independently into an object file. That is why changing one implementation file rebuilds only that file, while changing a *header* rebuilds everything that includes it — the measurement from Chapter 19 task 7.

Static and dynamic libraries

A **library** is precompiled code you link against.

A **static library** (`.a`, `.lib`) is copied into your executable at link time. The result is larger and self-contained.

A **dynamic library** (`.so`, `.dll`, `.dylib`) is loaded when the program runs. The executable is smaller, several programs share one copy, and updating the library updates every program — but the library must be present on the machine.

The C++ standard library is usually linked dynamically, which is why your `gradecalc` is a few kilobytes rather than a few megabytes.

Debug and release builds

	Debug	Release
Flag	`-g`	`-O2`
Debug information	yes	no
Optimization	none	aggressive
Assertions	active	removed with `-DNDEBUG`
Size	larger	smaller
Speed	slower	faster

```
g++ -std=c++17 -Wall -Wextra -g main.cpp -o gradecalc          # debug
g++ -std=c++17 -Wall -Wextra -O2 -DNDEBUG main.cpp -o gradecalc # release
```

Test the release build, not only the debug build. Optimization occasionally exposes undefined behavior that happened to work unoptimized — the out-of-bounds access from Chapter 11, or the uninitialized variable from Chapter 3, can behave differently at `-O2`. A program that works in debug and fails in release almost always has undefined behavior somewhere.

A makefile, optionally

Twenty-four chapters and every build has been one command. For larger projects, a build tool records the recipe:

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -g
SRCS = main.cpp assignment.cpp gradescale.cpp student.cpp gradingscheme.cpp gradebook.cpp

gradecalc: $(SRCS)
    $(CXX) $(CXXFLAGS) $(SRCS) -o gradecalc

clean:
    rm -f gradecalc
```

Then `make` builds and `make clean` removes. This is a convenience, not a requirement — the Grade Calculator builds fine without it.

24.9 Preparing a Release

Software that only its author can run is not finished. A release needs four things.

A release build, compiled with optimization, tested with the same cases as the debug build.

A README — what it is, how to build it, how to run it, what it requires.

A user guide — how to use it, what every menu option does, what the file format is, what the error messages mean.

Reconciled design documentation — a design document matching what was actually built.

Common Errors and Warnings

What you see	Cause	Fix
`terminate called after throwing an instance of ...`	Unhandled exception	Add a handler, or fix the cause
A derived exception is caught by the wrong clause	Order runs general to specific	Most specific first
`e.what()` gives a generic message	Caught by value — sliced	Catch by `const&`
Memory leaks when an exception is thrown	Manual `delete` skipped by unwinding	Use RAII
Half-modified object after a failure	Modified in place, then threw	Build a replacement, install on success
The release build behaves differently	Undefined behavior exposed by optimization	Run sanitizers; test both builds
`undefined reference`	Definition missing at link time	Define it; list every `.cpp`
Assertions do nothing in release	`-DNDEBUG` removes them	Do not rely on assertions for user input
Every file rebuilds after a one-line change	A header changed	Keep headers minimal

---

Design Notes

Throw when a function cannot keep its promise. Do not throw for expected conditions.

Catch by `const` reference, most specific first.

Never manage a resource by hand in code that can throw.

Build the replacement, then install it. A failed operation leaves the object as it was.

Exception messages address the user: name what went wrong, where, and what to do.

Grade Calculator v4.0 — Release

What v4.0 does

Everything v3.3 did, with errors reported as exceptions from a `GradebookError` hierarchy — plus a release build, a README, and a user guide.

GradeScale now refuses

Chapter 18 Section 18.8 flagged this. The constructor **repaired** invalid input, and the section noted that rejection is the honest alternative once there is a way to report failure.

```
GradeScale::GradeScale(const std::vector<Tier>& requested) {
    if (requested.empty()) {
        throw InvalidScaleError("no tiers were given.");
    }
    for (const Tier& t : requested) {
        if (t.cutoff < 0.0) {
            throw InvalidScaleError(std::string("tier '") + t.letter
                                    + "' has a negative cutoff.");
        }
        if (!tiers_.empty() && t.cutoff >= tiers_.back().cutoff) {
            throw InvalidScaleError(std::string("tier '") + t.letter + "' at "
                + std::to_string(static_cast<int>(t.cutoff))
                + " is not below the previous tier at "
                + std::to_string(static_cast<int>(tiers_.back().cutoff)) + ".");
```

```
        }
        tiers_.push_back(t);
    }
    if (tiers_.back().cutoff > 0.0) {
        throw InvalidScaleError("the lowest tier must have a cutoff of 0 so that"
                                " every percentage maps to a letter.");
    }
}
```

The invariant did not change — cutoffs descend, the lowest is 0. Only what happens when someone violates it. That the invariant survived a change this large is evidence it was the right thing to state in Chapter 18.

The caller reports and recovers

```
void editScale(Gradebook& book) {
    std::vector<GradeScale::Tier> requested;
    // ... read tiers from the user ...
    try {
        book.setScale(GradeScale(requested));
        std::cout << "  " << book.scale() << "\n";
    } catch (const InvalidScaleError& e) {
        // The old scale is untouched: the new one was never constructed.
        std::cout << "  " << e.what() << "\n";
        std::cout << "  Your previous scale is still in effect.\n";
    }
}
```

That comment is the exception-safety point. `book.setScale(...)` is never reached, because constructing the argument threw first. **The gradebook cannot be left holding a half-built scale.**

Expected behavior

Entering a scale with A at 80 then B at 85 — out of order:

```
  Invalid grade scale: tier 'B' at 85 is not below the previous tier at 80.
  Your previous scale is still in effect.
1001  Ada                    90.0%   A
```

Entering weights of 50 and 30, totalling 80:

```
  Category weights total 80.0 percent; they must total 100.
  The grading scheme was not changed.
```

Both refused, both explained, both leaving the program in its previous working state.

Loading translates library exceptions

```
try {
    // ... std::stod, std::stoi on fields from the file ...
} catch (const std::invalid_argument&) {
    // stod and stoi throw this on non-numeric text. Translate it into
    // an error that names the file and line the user can actually fix.
    throw FileFormatError(filename, lineNumber, "expected a number here.");
}
```

`std::invalid_argument` says a conversion failed. It does not say which file, which line, or what to do. **Catching a low-level exception and rethrowing a meaningful one** is how a library error becomes a user-facing message.

Compare with Chapter 15's `catch (...)`, which knew something went wrong and could say nothing useful. That was appropriate then; this is better now.

Running the release check

Keep every source and header file open, click **Run**, and work through the full test plan in the embedded Terminal — not only the handful of cases you happened to try while writing the code. A release is a claim that you tested the whole thing.

Your StudySite Lab — Add Release-Safe Error Handling

- **Course**: COSC 1437 — Object-Oriented Programming
- **Project checkpoint**: v4.0
- **Starting point**: Your committed Chapter 23 v3.3 program (twelve files).

  One-repository rule: Continue in the same COSC 1437 Grade Calculator repository from Chapter 13 through Chapter 24. Do not create a chapter folder or a new repository. The supplied Chapter 12 solution is the foundation; your COSC 1437 work is what you add in Chapters 13–24.

What you will build

This is the release checkpoint. Until now, bad input was either quietly repaired (an invalid grade scale) or quietly accepted (weights that do not total 100). In this lab you make the program **refuse bad input with an exception, explain why, and carry on**. By the end you will have:

1. An **exception hierarchy** in `gradebookerror.h`: a `GradebookError` base class derived from `std::runtime_error`, and three specific errors — `InvalidScaleError`, `WeightSumError`, and `FileFormatError`.
2. A `GradeScale` constructor that **throws** `InvalidScaleError` instead of repairing the tiers.
3. A `Weighted` constructor that **throws** `WeightSumError` when the weights do not total 100.
4. **Saving and loading** the gradebook to `gradebook.csv`: loaded at startup, saved from a new **Save** option, and saved again on Quit. A malformed file produces a `FileFormatError` that names the file and line.
5. `try`/`catch` blocks in `main` that report each error and keep the program running, plus the return of **custom category weights** when choosing a weighted scheme.

The menu grows to nine options: Add student, Report, Sort, **Edit scale** (renamed), Change scheme, Statistics, Find by ID, **Save** (new), and Quit.

> **Scope**: Complete only the Chapter 24 code checkpoint here. Final-project documentation, finishing touches, and submission instructions will be provided separately.

Files in this checkpoint

File	What you do to it
`gradebookerror.h`	Create (Step 1)
`gradescale.h`, `gradescale.cpp`	Edit (Step 2)
`gradingscheme.h`, `gradingscheme.cpp`	Edit (Step 3)
`gradebook.h`, `gradebook.cpp`	Edit (Step 4)
`main.cpp`	Edit (Steps 5–10)
`assignment.*`, `student.*`, `statistics.h`	**No change** — keep them open

When you are finished your project has **thirteen files**.

How to use these steps

This lab is a guided exercise, not a test. Each step tells you **which file** to open, shows the **exact code to find**, and shows the **exact code to put in its place**. Work through the steps in order and copy the code from the boxes rather than retyping it where you can.

- A **find** box shows code exactly as it appears in your program at the end of the previous chapter. Use the editor's search to locate it. If you cannot find it, your program differs from the previous checkpoint — compare it with your last commit on GitHub, or ask the Tutor to help you line it up, before going further.
- A ▸ **Run checkpoint** marks a point where the program should build and run again. Between checkpoints the program may not compile; that is expected.
- Each step ends with a short explanation of *why*. Read it — it is the chapter's concept, applied to your own program.

Build it: step by step

Step 1 — Write the exception hierarchy Create a new file named `gradebookerror.h` and enter this code:

```
// gradebookerror.h - Grade Calculator v4.0 - Chapter 24
#ifndef GRADEBOOKERROR_H
#define GRADEBOOKERROR_H

#include <sstream>
#include <stdexcept>
#include <string>

/**
 * Base class for every error this application reports.
 *
 * Deriving from std::runtime_error means these can be caught either
 * specifically, or as GradebookError, or as std::exception - a caller chooses
 * how much it wants to know. A function that cannot produce a sensible return
 * value for bad input throws instead of returning a sentinel nobody checks.
 */
class GradebookError : public std::runtime_error {
public:
    explicit GradebookError(const std::string& what) : std::runtime_error(what) {}
};

/** A grade scale whose tiers do not descend, or that does not reach 0. */
class InvalidScaleError : public GradebookError {
public:
    explicit InvalidScaleError(const std::string& what)
        : GradebookError("Invalid grade scale: " + what) {}
};

/** Category weights that do not total 100 percent. */
class WeightSumError : public GradebookError {
public:
    explicit WeightSumError(double actual)
        : GradebookError(describe(actual)), actual_(actual) {}
    double actual() const { return actual_; }
private:
    static std::string describe(double actual) {
        std::ostringstream out;
        out.setf(std::ios::fixed);
```

```
        out.precision(1);
        out << "Category weights total " << actual << " percent; they must total 100.";
        return out.str();
    }
    double actual_;
};

/** A gradebook file that cannot be parsed. */
class FileFormatError : public GradebookError {
public:
    FileFormatError(const std::string& filename, int line, const std::string& why)
        : GradebookError("Cannot read '" + filename + "' at line "
                         + std::to_string(line) + ": " + why),
          filename_(filename), line_(line) {}
    const std::string& filename() const { return filename_; }
    int line() const { return line_; }
private:
    std::string filename_;
    int line_;
};

#endif
```

- `GradebookError` **inherits from `std::runtime_error`**, so every error this program throws can be caught as a `GradebookError`, or more generally as a `std::exception`. The message passed to the constructor is what `what()` returns later.
- Each derived class adds a prefix or builds a more specific message. `WeightSumError` formats the actual total into its message and also keeps it in `actual()`. `FileFormatError` records the file name and line number.
- A thrown exception carries **data**, not just a message, so the code that catches it can decide what to do.

Step 2 — Make `GradeScale` refuse instead of repair In `gradescale.h`, find the first line:

```
// gradescale.h - Grade Calculator v2.6 - Chapter 19
```

Replace it with:

```
// gradescale.h - Grade Calculator v4.0 - Chapter 24
```

Inside the class, find the two constructor declarations:

```
    GradeScale();
    explicit GradeScale(const std::vector<Tier>& requested);
```

Replace it with:

```
    GradeScale();

    /**
     * Builds a scale from user-supplied tiers.
     * @throws InvalidScaleError if the tiers do not strictly descend, if any
     *         cutoff is negative, or if the scale does not reach 0.
     *
     * v2.5 quietly repaired bad input, which meant a user could type a broken
     * scale and get grades computed from something they never asked for.
     * Refusing to construct is the honest behavior; the caller reports why.
     */
    explicit GradeScale(const std::vector<Tier>& requested);
```

In `gradescale.cpp`, find the first lines of the file:

```
// gradescale.cpp - Grade Calculator v2.6 - Chapter 19
#include "gradescale.h"

#include <iomanip>
#include <ostream>
```

Replace it with:

```
// gradescale.cpp - Grade Calculator v4.0 - Chapter 24
#include "gradescale.h"
#include "gradebookerror.h"

#include <iomanip>
#include <string>
#include <ostream>
```

Find the start of the constructor that takes tiers:

```
GradeScale::GradeScale(const std::vector<Tier>& requested) {
```

Delete that line and everything below it (the body of that constructor), down to and including this line:

```
}
```

In its place, enter:

```
GradeScale::GradeScale(const std::vector<Tier>& requested) {
    if (requested.empty()) {
        throw InvalidScaleError("no tiers were given.");
    }
    for (const Tier& t : requested) {
        if (t.cutoff < 0.0) {
            throw InvalidScaleError(std::string("tier '") + t.letter
                                    + "' has a negative cutoff.");
        }
        if (!tiers_.empty() && t.cutoff >= tiers_.back().cutoff) {
            throw InvalidScaleError(std::string("tier '") + t.letter + "' at "
                + std::to_string(static_cast<int>(t.cutoff))
                + " is not below the previous tier at "
                + std::to_string(static_cast<int>(tiers_.back().cutoff)) + ".");
        }
        tiers_.push_back(t);
    }
    if (tiers_.back().cutoff > 0.0) {
        throw InvalidScaleError("the lowest tier must have a cutoff of 0 so that"
                                " every percentage maps to a letter.");
    }
}
```

The invariant did not change — cutoffs still strictly descend and the lowest tier is still 0. Only the response to a violation changed: instead of skipping or patching a bad tier, the constructor **throws**. A constructor that throws never finishes, so **no half-built `GradeScale` can exist**.

The doc comment in the header now says `@throws InvalidScaleError`, so anyone calling the constructor knows to be ready for it.

> ▸ **Run checkpoint**: Click **Run** with every file open, including `gradebookerror.h`. The program should compile cleanly. Do **not** test *Custom scale* yet — nothing catches the new exception until Step 9, so a bad scale would end the program.

Step 3 — Make `Weighted` refuse weights that do not total 100 In `gradingscheme.h`, find the first line:

```
// gradingscheme.h - Grade Calculator v3.2 - Chapter 22
```

Replace it with:

```
// gradingscheme.h - Grade Calculator v4.0 - Chapter 24
```

In `class Weighted`, find the two constructor declarations:

```
    Weighted();
    explicit Weighted(const std::vector<CategoryWeight>& weights);
```

Replace it with:

```
    Weighted();

    /**
     * @throws WeightSumError if the weights do not total 100 percent.
     *         A gradebook that silently normalizes bad weights produces
     *         grades the instructor did not ask for.
     */
    explicit Weighted(const std::vector<CategoryWeight>& weights);
```

In `gradingscheme.cpp`, find the first two lines:

```
// gradingscheme.cpp - Grade Calculator v3.1 - Chapter 21
#include "gradingscheme.h"
```

Replace it with:

```
// gradingscheme.cpp - Grade Calculator v4.0 - Chapter 24
#include "gradingscheme.h"
#include "gradebookerror.h"
```

Find the `Weighted` constructor that takes weights:

```
Weighted::Weighted(const std::vector<CategoryWeight>& weights)
    : GradingScheme("Weighted"), weights_(weights) {}
```

Replace it with:

```
Weighted::Weighted(const std::vector<CategoryWeight>& weights)
    : GradingScheme("Weighted"), weights_(weights) {
    if (!weightsValid()) {
        throw WeightSumError(weightTotal());
    }
}
```

In v3.0 weights that did not total 100 were scaled proportionally, with only a note. Now the constructor uses its own `weightsValid()` check and throws a `WeightSumError` carrying the actual total.

`WeightedDropLowest`'s weights-taking constructor calls this constructor, so it gets the same check **for free** — you do not change `WeightedDropLowest` at all.

Step 4 — Let a `Gradebook` be moved, and keep IDs unique Loading a file will build a complete new `Gradebook` and then move it into place. A `Gradebook` cannot be copied (it holds a `unique_ptr`), but it can be **moved**.

In `gradebook.h`, find the first line:

```
// gradebook.h - Grade Calculator v3.3 - Chapter 23
```

Replace it with:

```
// gradebook.h - Grade Calculator v4.0 - Chapter 24
```

Find the constructor and the comment below it:

```
    explicit Gradebook(const std::string& courseName = "Untitled Course");

    // No destructor is needed now. unique_ptr releases the scheme when the
```

Replace it with:

```
    explicit Gradebook(const std::string& courseName = "Untitled Course");

    // Movable so a fully-built replacement can be swapped in atomically after
    // a successful load. Copying stays disabled: schemes are uniquely owned.
    Gradebook(Gradebook&&) = default;
    Gradebook& operator=(Gradebook&&) = default;

    // No destructor is needed now. unique_ptr releases the scheme when the
```

`= default` asks the compiler to generate the move constructor and move assignment operator. Each member is moved in turn — including the `unique_ptr`, which hands over its scheme.

There is one more thing to fix before students can be loaded from a file. A loaded student keeps the ID it was saved with, but the `Gradebook`'s next-ID counter starts at 1001 every time the program runs. Without a fix, the first student you add after a reload would get an ID that is already taken.

In `gradebook.cpp`, find the first line:

```
// gradebook.cpp - Grade Calculator v3.3 - Chapter 23
```

Replace it with:

```
// gradebook.cpp - Grade Calculator v4.0 - Chapter 24
```

In `addStudent`, find:

```
    roster_.push_back(s);
    idIndex_[s.id()] = roster_.size() - 1;
```

Directly below it, add:

```
    // A student loaded from a file keeps its saved ID. Move the counter past
    // it so the next new student never receives an ID that is already taken.
    if (s.id() >= nextId_) { nextId_ = s.id() + 1; }
```

`addStudent` is the one place every student enters the roster — typed in or loaded — so this is the one place the counter has to be kept ahead of the IDs already in use. **A class protects its own invariants**; here the invariant is "every ID handed out is unused."

Step 5 — Update the header comment and includes in `main.cpp` At the top of `main.cpp`, find the header comment:

```
// ============================================================================
//  Grade Calculator  v3.3                                        Chapter 23
// ----------------------------------------------------------------------------
//  Change  : Templates and the STL. Statistics<T> computes mean, median,
//            range, and standard deviation for any numeric type. std::map
//            indexes students by ID and counts the letter distribution.
//            Sorting is expressed with a lambda instead of operator<.
```

```
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp assignment.cpp gradescale.cpp
//            student.cpp gradingscheme.cpp gradebook.cpp -o gradecalc
// ============================================================================
```

Replace it with:

```
// ============================================================================
//  Grade Calculator  v4.0                                        Chapter 24
//  RELEASE BUILD
// ----------------------------------------------------------------------------
//  Change  : Errors are reported as exceptions from a GradebookError hierarchy
//            instead of silently repaired or returned as sentinel values.
//              InvalidScaleError  tiers that do not descend or reach 0
//              WeightSumError     category weights that do not total 100
//              FileFormatError    a gradebook file that cannot be parsed
//            Bad input now costs the user one clear message and one retry,
//            never a grade computed from something they did not ask for.
//  Build   : g++ -std=c++17 -Wall -Wextra main.cpp assignment.cpp gradescale.cpp
//            student.cpp gradingscheme.cpp gradebook.cpp -o gradecalc
//  Usage   : see README.md
// ============================================================================
```

Below it, find the includes:

```
#include "gradebook.h"

#include <iostream>
#include <memory>
#include <utility>
#include <limits>
#include <string>
```

Replace it with:

```
#include "gradebook.h"
#include "gradebookerror.h"

#include <fstream>
#include <iostream>
#include <limits>
#include <memory>
#include <sstream>
#include <string>
#include <utility>

const std::string DEFAULT_FILE = "gradebook.csv";
```

`<fstream>` and `<sstream>` are back for file input and output, and `DEFAULT_FILE` names the file the gradebook is saved to.

Step 6 — Add `splitCsv` and `saveGradebook` Find the comment just above `chooseScheme` (below `readLine`):

```
/** Builds the scheme the user asked for. The only place that names the
```

Directly above it, add:

```
// ----------------------------------------------------------------------------
//  Persistence
// ----------------------------------------------------------------------------

std::vector<std::string> splitCsv(const std::string& line) {
```

```
    std::vector<std::string> fields;
    std::istringstream in(line);
    std::string field;
    while (std::getline(in, field, ',')) { fields.push_back(field); }
    return fields;
}

void saveGradebook(const Gradebook& book, const std::string& filename) {
    std::ofstream out(filename);
    if (!out) {
        throw GradebookError("Cannot open '" + filename + "' for writing.");
    }
    out << "# Grade Calculator gradebook, format version 2\n";
    for (const GradeScale::Tier& t : book.scale().tiers()) {
        out << "TIER," << t.cutoff << "," << t.letter << "\n";
    }
    for (std::size_t i = 0; i < book.size(); ++i) {
        const Student& s = book[i];
        out << "STUDENT," << s.id() << "," << s.name() << "\n";
        for (const Assignment& a : s.assignments()) {
            out << "WORK," << s.id() << "," << a.name() << "," << a.pointsEarned()
                << "," << a.pointsPossible() << "," << a.bonusPoints()
                << "," << a.category() << "\n";
        }
    }
    if (!out.good()) {
        throw GradebookError("Writing '" + filename + "' failed partway through.");
    }
}
```

- `splitCsv` is the same helper you wrote in Chapter 15.
- The file format is **version 2**: `TIER` lines for the scale, then a `STUDENT` line for each student, followed by one `WORK` line per assignment — including its **category**.
- `saveGradebook` returns nothing. If the file cannot be opened, or the writing fails partway through, it **throws** a `GradebookError` instead of returning `false` for a caller to forget to check.

Step 7 — Add `loadGradebook`, which builds a replacement first Find the same comment line again — it is now just below `saveGradebook`:

```
/** Builds the scheme the user asked for. The only place that names the
```

Directly above it, add:

```
/** @throws FileFormatError if any record is malformed. */
void loadGradebook(Gradebook& book, const std::string& filename) {
    std::ifstream in(filename);
    if (!in) {
        throw GradebookError("No file named '" + filename + "' was found.");
    }

    std::vector<GradeScale::Tier> tiers;
    std::vector<Student> loaded;
    std::string line;
    int lineNumber = 0;

    while (std::getline(in, line)) {
        ++lineNumber;
        if (line.empty() || line[0] == '#') { continue; }
        std::vector<std::string> f = splitCsv(line);
```

```
        if (f.empty()) { continue; }

        try {
            if (f[0] == "TIER") {
                if (f.size() < 3 || f[2].empty()) {
                    throw FileFormatError(filename, lineNumber, "a TIER needs a cutoff and a letter.");
                }
                tiers.push_back({std::stod(f[1]), f[2][0]});
            } else if (f[0] == "STUDENT") {
                if (f.size() < 3) {
                    throw FileFormatError(filename, lineNumber, "a STUDENT needs an ID and a name.");
                }
                loaded.push_back(Student(f[2], std::stoi(f[1])));
            } else if (f[0] == "WORK") {
                if (f.size() < 7) {
                    throw FileFormatError(filename, lineNumber, "a WORK record needs seven fields.");
                }
                int owner = std::stoi(f[1]);
                bool placed = false;
                for (Student& s : loaded) {
                    if (s.id() == owner) {
                        s.addAssignment(Assignment(f[2], std::stod(f[3]),
                                                   std::stod(f[4]), std::stod(f[5]), f[6]));
                        placed = true;
                        break;
                    }
                }
                if (!placed) {
                    throw FileFormatError(filename, lineNumber,
                                          "work belongs to unknown student ID " + f[1] + ".");
                }
            } else {
                throw FileFormatError(filename, lineNumber, "unrecognized record type '" + f[0] + "'.");
            }
        } catch (const std::invalid_argument&) {
            // stod and stoi throw this on non-numeric text. Translate it into
            // an error that names the file and line the user can actually fix.
            throw FileFormatError(filename, lineNumber, "expected a number here.");
        } catch (const std::out_of_range&) {
            throw FileFormatError(filename, lineNumber, "a number is out of range.");
        }
    }

    // Build the scale first: if it is invalid, nothing is loaded.
    Gradebook fresh(book.courseName());
    if (!tiers.empty()) {
        fresh.setScale(GradeScale(tiers));   // may throw InvalidScaleError
    }
    for (const Student& s : loaded) { fresh.addStudent(s); }
    book = std::move(fresh);
}

// ---------------------------------------------------------------------------
```

Read this function in two halves.

Reading. Every line is parsed into local variables — `tiers` and `loaded` — never directly into `book`. Inside the loop:

- A record with too few fields, a `WORK` line for an unknown student, or an unrecognized record type throws

a `FileFormatError` naming the line.

- `std::stod` and `std::stoi` throw `std::invalid_argument` on text that is not a number, and `std::out_of_range` on a number too large. The `catch` blocks **translate** those library exceptions into a `FileFormatError` that names the file and line the user can actually fix.

Committing. Only after the whole file has been read does the function build a `fresh` gradebook, set its scale (which may throw `InvalidScaleError`), add the students, and finally `book = std::move(fresh);`. If anything throws before that last line, `book` is **untouched** — the load either succeeds completely or changes nothing. That is what the move operations from Step 4 make possible.

The `// ----` line at the end separates the persistence section from the menu helpers below it.

▸ **Run checkpoint**: Click **Run**. The program should compile cleanly; the new functions exist but are not called yet.

Step 8 — Rewrite `chooseScheme` to accept custom weights safely Find the comment above `chooseScheme`:

```
/** Builds the scheme the user asked for. The only place that names the
```

Delete that line and everything below it (the whole `chooseScheme` function), down to and including this line:

```
}
```

In its place, enter:

```
void chooseScheme(Gradebook& book) {
    std::cout << "  1) Points-based\n";
    std::cout << "  2) Weighted by category\n";
    std::cout << "  3) Weighted, lowest dropped per category\n";
    std::string c = readLine("  Choice: ");

    if (c.empty() || c[0] == '1') {
        book.setScheme(std::make_unique<PointsBased>());
    } else {
        std::vector<Weighted::CategoryWeight> weights;
        if (readLine("  Enter your own category weights? (y/n): ").substr(0, 1) == "y") {
            while (true) {
                std::string name = readLine("    Category (or 'done'): ");
                if (name == "done" || name.empty()) { break; }
                weights.push_back({name, readNonNegative("      Weight (percent): ")});
            }
        }
        try {
            if (c[0] == '2') {
                book.setScheme(weights.empty() ? std::make_unique<Weighted>()
                                               : std::make_unique<Weighted>(weights));
            } else {
                book.setScheme(weights.empty() ? std::make_unique<WeightedDropLowest>()
                                               : std::make_unique<WeightedDropLowest>(weights));
            }
        } catch (const WeightSumError& e) {
            std::cout << "  " << e.what() << "\n";
            std::cout << "  The grading scheme was not changed.\n";
            return;
        }
    }
    std::cout << "  Grading is now: " << book.schemeName() << "\n";
}
```

- Points-based grading is chosen as before.

- For either weighted scheme, the user may now enter their own categories and weights. If they enter none, the default weights are used.
- Creating the scheme sits inside a `try` block. If the weights do not total 100, the `Weighted` constructor throws, the `catch` prints the error's message, and the function **returns early** — `setScheme` was never reached, so the previous scheme is still in effect.

Step 9 — Add `editScale` Find the first line of `main`:

```
int main() {
```

Directly above it, add:

```
void editScale(Gradebook& book) {
    std::vector<GradeScale::Tier> requested;
    std::cout << "  Enter tiers highest first; the lowest must be 0. 'done' to finish.\n";
    while (true) {
        std::string lt = readLine("    Letter (or 'done'): ");
        if (lt == "done" || lt.empty()) { break; }
        requested.push_back({readNonNegative("      Minimum percentage: "), lt[0]});
    }
    try {
        book.setScale(GradeScale(requested));
        std::cout << "  " << book.scale() << "\n";
    } catch (const InvalidScaleError& e) {
        // The old scale is untouched: the new one was never constructed.
        std::cout << "  " << e.what() << "\n";
        std::cout << "  Your previous scale is still in effect.\n";
    }
}
```

This is the *Custom scale* code from `main`, moved into its own function and protected by `try`/`catch`. Look at the comment in the `catch` block: because `GradeScale(requested)` throws **before** `book.setScale(...)` runs, the old scale is never replaced. The user sees why the scale was refused and keeps the scale they had.

Step 10 — Rewrite `main`: load, catch, save Find the start of `main`:

```
int main() {
```

Delete that line and everything below it, all the way to the final closing `}` at the very end of the file.

In its place, enter the new `main`:

```
int main() {
    std::cout << "=== GRADE CALCULATOR v4.0 ===\n\n";

    Gradebook book("Programming Fundamentals");

    try {
        loadGradebook(book, DEFAULT_FILE);
        std::cout << "Loaded " << DEFAULT_FILE << " (" << book.size() << " students).\n";
    } catch (const GradebookError& e) {
        std::cout << e.what() << "\n";
        std::cout << "Starting a new gradebook.\n";
    }

    std::cout << "\nSelect a grading scheme:\n";
    chooseScheme(book);

    bool running = true;
```

```
    while (running) {
        std::cout << "\n1) Add student  2) Report  3) Sort  4) Edit scale\n"
                     "5) Change scheme  6) Statistics  7) Find by ID  "
                     "8) Save  9) Quit\nChoice: ";
        std::string line;
        if (!std::getline(std::cin, line)) { break; }
        char c = line.empty() ? '?' : line[0];

        try {
            if (c == '1') {
                std::string name = readLine("  Student name: ");
                if (name.empty()) { continue; }
                Student s(name, book.claimNextId());
                while (true) {
                    std::string an = readLine("    Assignment name (or 'done'): ");
                    if (an == "done" || an.empty()) { break; }
                    double earned   = readNonNegative("      Points earned  : ");
                    double possible = readNonNegative("      Points possible: ");
                    double bonus    = readNonNegative("      Bonus points   : ");
                    std::string cat = readLine("      Category       : ");
                    if (cat.empty()) { cat = "Uncategorized"; }
                    s.addAssignment(Assignment(an, earned, possible, bonus, cat));
                }
                book.addStudent(s);
                std::cout << "  Added " << s.name() << " as ID " << s.id() << ".\n";
            } else if (c == '2') {
                std::cout << book << "\n";
            } else if (c == '3') {
                book.sortByPercentage();
                std::cout << "  Sorted.\n";
            } else if (c == '4') {
                editScale(book);
            } else if (c == '5') {
                chooseScheme(book);
            } else if (c == '6') {
                std::cout << "\n--- Class statistics ---\n";
                book.printStatistics(std::cout);
            } else if (c == '7') {
                std::string s = readLine("  Student ID: ");
                int id = 0;
                try { id = std::stoi(s); }
                catch (...) { std::cout << "  Not a valid ID.\n"; continue; }
                const Student* found = book.findById(id);
                if (found != nullptr) {
                    std::cout << "  " << found->name() << ": "
                              << book.percentageFor(*found) << "%\n";
                } else {
                    std::cout << "  No student with ID " << id << ".\n";
                }
            } else if (c == '8') {
                saveGradebook(book, DEFAULT_FILE);
                std::cout << "  Saved to " << DEFAULT_FILE << ".\n";
            } else if (c == '9') {
                running = false;
            } else {
                std::cout << "  Please enter 1-9.\n";
            }
        } catch (const GradebookError& e) {
            // Any error our own code raises is reported and the loop continues.
            // No operation leaves the gradebook half-modified.
```

```
            std::cout << "  " << e.what() << "\n";
        }
    }

    try {
        saveGradebook(book, DEFAULT_FILE);
        std::cout << "\nGradebook saved. Goodbye.\n";
    } catch (const GradebookError& e) {
        std::cout << "\nWarning: " << e.what() << "\n";
        std::cout << "Your work was not saved. Goodbye.\n";
        return 1;
    }
    return 0;
}
```

- **Startup** tries to load `gradebook.csv`. If there is no file, or it is malformed, the error is printed and a new gradebook starts.
- **The whole menu body is inside one `try` block.** Any `GradebookError` raised by an option is caught at the bottom of the loop, its message is printed, and the loop continues. The `catch` names `GradebookError`, so it handles all three specific errors.
- **Edit scale** (option 4) calls `editScale`, and **Change scheme** (option 5) calls the new `chooseScheme`.
- **Save** (option 8) is new. **Quit** moves to option 9.
- **On exit** the gradebook is saved. If that fails, the program says so and returns `1` — a nonzero exit code tells the operating system something went wrong.

  ▸ **Run checkpoint**: Make sure all thirteen files are open and click **Run**. The program should compile with no errors and no warnings. On the first run it prints `No file named 'gradebook.csv' was found.` and `Starting a new gradebook.` — that is correct.

Check your work

Work through this test plan in the Terminal. A release is a claim that you tested the whole program, not only the cases you happened to try while typing.

1. **Fresh start.** Choose `1` (Points-based). Add `Ada` with one assignment: `HW1`, earned `9`, possible `10`, bonus `0`, category Enter. Report with `2`: `1001  Ada  90.0%  A`.
2. **An invalid scale is refused.** Choose `4` (Edit scale). Enter `A` with `80`, then `B` with `85`, then `done`. You should see:

   ```
   Invalid grade scale: tier 'B' at 85 is not below the previous tier at 80.
   Your previous scale is still in effect.
   ```

   Report again: Ada is still an `A` under the default scale.
3. **A scale that does not reach 0 is refused.** Choose `4` and enter only `A` with `90`, then `done`. You should see a message that the lowest tier must have a cutoff of 0, and the previous scale stays in effect.
4. **Bad weights are refused.** Choose `5`, then `2`, answer `y`, and enter `Exam 50` and `Homework 30`, then `done`. You should see:

   ```
   Category weights total 80.0 percent; they must total 100.
   The grading scheme was not changed.
   ```
5. **Save and reload.** Choose `8`. You should see `Saved to gradebook.csv.` Then choose `9` to quit; the program prints `Gradebook saved. Goodbye.` Click **Run** again: the program should print `Loaded gradebook.csv (1 students).` Choose a scheme, then report — Ada is back with 90.0%. Now add a new student, `Bo`, with

no assignments (type `done` right away). The program should print `Added Bo as ID 1002.` — not 1001, which Ada already has.

Then confirm:

- ☐ All thirteen files build together with no errors and no warnings.
- ☐ No input you can type makes the program crash or stop unexpectedly.
- ☐ Every `throw` in the program throws a class derived from `GradebookError`, and every `catch` catches by **`const` reference**.
- ☐ `assignment.*`, `student.*`, and `statistics.h` are unchanged from Chapter 23.

StudySite workflow

1. Confirm that your previous chapter is committed on GitHub, then open this chapter's **coding panel on the StudySite main stage**.
2. Close stale project tabs from an earlier session before loading. This avoids creating files with names such as `_imported` when the same path is already open.
3. Click **Load from GitHub**, select **COSC1437xxx-Grade-Calculator-YourLastName**, and click all twelve project files to open them. You will create `gradebookerror.h` during the lab. Confirm the editor shows the expected file paths before editing.
4. Work through Steps 1–10 above in StudySite's internal editor, in order, clicking **Run** at each ▸ Run checkpoint.
5. Read compiler messages and program output in the embedded Terminal, and type program input there when prompted. Fix every compiler error and warning before moving past a Run checkpoint.
6. Complete the **Check your work** list.
7. Use the Tutor with the current code or Terminal output when you need help.

Save this checkpoint

> **IMPORTANT — commit to save your work**: StudySite autosaves editor tabs locally on this device, but local autosave is not a durable GitHub backup. Your work is not safely saved in your repository until **Save to GitHub** finishes a successful **Commit**.

1. Keep every project file that belongs in this checkpoint open in the editor — all thirteen files, including the new `gradebookerror.h`. You do not need to commit `gradebook.csv`. **Save to GitHub includes every open editor file**, so close scratch files and accidental `_imported` duplicates first.
2. Click **Save to GitHub**.
3. Select **COSC1437xxx-Grade-Calculator-YourLastName** and the existing **main** branch.
4. Enter the commit message **Complete Chapter 24 Grade Calculator v4.0**.
5. Click **Commit** and wait for StudySite's confirmation.
6. Open the commit link, or open the repository on GitHub, and confirm the new commit and expected files are present before leaving StudySite.

Complete when

- Every item in **Check your work** passes.
- **COSC1437xxx-Grade-Calculator-YourLastName** contains the Chapter 24 checkpoint.
- The GitHub commit is visible; StudySite's local autosave alone is not completion.

The Road Ahead

You can now read and write substantial C++. Some directions worth knowing about.

Modern C++ beyond C++17. C++20 adds concepts (stating a template's requirements, so error messages point at your call), ranges (composable algorithms without explicit iterators), and modules (an alternative to headers that would change Chapter 2's translation pipeline considerably).

Testing frameworks. Chapter 16's `check` harness is a small version of Catch2 or GoogleTest, which add fixtures, parameterized tests, and reporting.

Concurrency. `<thread>` and `<mutex>` let a program do several things at once, with a new class of defect that the debugging discipline of Chapter 16 extends to reach.

Data structures and algorithms. Chapter 17 covered searching and sorting. Trees, graphs, and hash tables follow, along with the analysis that makes the choices principled.

Beyond the console. Every program here reads `std::cin` and writes `std::cout`. Graphical interfaces, web services, and databases are the obvious next boundaries.

What transfers. The languages will change. The habits will not: design before coding, name what you decide, verify rather than assume, test at boundaries, encapsulate what should not be reachable, and leave your code readable by whoever inherits it. That last person is often you.

Try It Yourself

1. A custom exception hierarchy

```
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>

class GradebookError : public std::runtime_error {
public:
    explicit GradebookError(const std::string& w) : std::runtime_error(w) {}
};

class InvalidScaleError : public GradebookError {
public:
    explicit InvalidScaleError(const std::string& w)
        : GradebookError("Invalid grade scale: " + w) {}
};

class WeightSumError : public GradebookError {
public:
    explicit WeightSumError(double actual)
        : GradebookError(describe(actual)), actual_(actual) {}
    double actual() const { return actual_; }
private:
    static std::string describe(double a) {
        std::ostringstream o;
        o.setf(std::ios::fixed);
        o.precision(1);
        o << "Category weights total " << a << " percent; they must total 100.";
        return o.str();
```

```
    }
    double actual_;
};

void tryIt(int which) {
    try {
        if (which == 1) { throw InvalidScaleError("tier 'B' at 85 is not below 80."); }
        if (which == 2) { throw WeightSumError(80.0); }
        if (which == 3) { throw std::runtime_error("something else entirely"); }
    }
    catch (const InvalidScaleError& e) { std::cout << "  InvalidScaleError: " << e.what() << "\n"; }
    catch (const GradebookError& e)    { std::cout << "  GradebookError:    " << e.what() << "\n"; }
    catch (const std::exception& e)    { std::cout << "  std::exception:    " << e.what() << "\n"; }
}

int main() {
    for (int i = 1; i <= 3; ++i) { tryIt(i); }
    return 0;
}
```

Expected output:

```
  InvalidScaleError: Invalid grade scale: tier 'B' at 85 is not below 80.
  GradebookError:    Category weights total 80.0 percent; they must total 100.
  std::exception:    something else entirely
```

Try: Move the `std::exception` clause first and rebuild. Read the warning, then predict which handler catches each throw.

2. Stack unwinding

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>

struct Noisy {
    std::string n;
    explicit Noisy(std::string s) : n(std::move(s)) { std::cout << "  built " << n << "\n"; }
    ~Noisy() { std::cout << "  destroyed " << n << "\n"; }
};

int main() {
    try {
        Noisy a("a");
        Noisy b("b");
        throw std::runtime_error("boom");
    } catch (const std::runtime_error& e) {
        std::cout << "  handler: " << e.what() << "\n";
    }
    return 0;
}
```

Expected output:

```
  built a
  built b
  destroyed b
  destroyed a
  handler: boom
```

Try: In what order were they destroyed, and why that order? Now imagine `Noisy` held a `new`ed pointer released in its destructor — what does that tell you about RAII and exceptions?

3. An exception that carries data

```
#include <iostream>
#include <stdexcept>

class TooLarge : public std::runtime_error {
public:
    explicit TooLarge(int value)
        : std::runtime_error("value is too large"), value_(value) {}
    int value() const { return value_; }
private:
    int value_;
};

int main() {
    try {
        throw TooLarge(150);
    } catch (const TooLarge& e) {
        std::cout << e.what() << ": " << e.value() << "\n";
        std::cout << "clamped to " << (e.value() > 100 ? 100 : e.value()) << "\n";
    }
    return 0;
}
```

Expected output:

```
value is too large: 150
clamped to 100
```

Try: Catch it as `const std::runtime_error&` instead. Can you still call `value()`? What does that tell you about catching too generally?

4. Catching by value slices

```
#include <iostream>
#include <stdexcept>
#include <string>

class Specific : public std::runtime_error {
public:
    Specific() : std::runtime_error("specific message") {}
};

int main() {
    try {
        throw Specific();
    } catch (std::runtime_error e) {          // by value — sliced
        std::cout << "by value:     " << e.what() << "\n";
    }
    try {
        throw Specific();
    } catch (const std::runtime_error& e) {   // by reference
        std::cout << "by reference: " << e.what() << "\n";
    }
    return 0;
}
```

Note: this listing produces `warning: catching polymorphic type ... by value`. That warning *is* the lesson — the compiler is telling you the exception is about to be sliced. It is the one listing in this book that is not warning-clean, deliberately.

Try: Add a virtual function to `Specific` that `std::runtime_error` does not have, and call it in each handler. Which one compiles? Which one would have given you the derived behavior?

5. Translate a library exception

Write a function reading a number from a string and throwing your own `ParseError` — naming the line and what was expected — when `std::stod` fails. Catch `std::invalid_argument` and rethrow.

Then compare the two messages. Which would you rather show a user?

6. Debug versus release

Build any Grade Calculator version both ways:

```
g++ -std=c++17 -Wall -Wextra -g main.cpp -o gc_debug
g++ -std=c++17 -Wall -Wextra -O2 -DNDEBUG main.cpp -o gc_release
```

Compare file sizes. Add an `assert` that fails and run both. Then run your full test plan against the release build — does anything behave differently? If so, you have found undefined behavior.

7. Decide: exception or not?

For each, say whether to throw, and why:

- A user types `abc` where a number belongs
- A gradebook file is missing
- A gradebook file exists but line 14 is malformed
- A student ID is not on the roster
- Category weights total 90 percent
- A grade scale has no tier at 0
- The user enters menu choice 9 when options are 1–8
- The program cannot write its save file on exit

Summary

- **Return codes can be ignored.** An exception cannot: unhandled, it terminates the program.
- **`throw`** abandons a function; **`try`**/**`catch`** handles what it throws. **`what()`** gives the message.
- **Always catch by `const` reference.** Catching by value slices the exception.
- `<stdexcept>` provides a hierarchy rooted at `std::exception`. Define your own for your own failures, and let them carry data when a handler could use it.
- Several `catch` clauses may follow one `try`; **the first match wins**, so order most specific to least.
- **Stack unwinding** destroys every local object between the throw and the handler. **This is what makes RAII essential** — manual `delete` is skipped, so any function that can throw leaks if it manages a resource by hand.
- **Build the replacement, then install it**, so a failed operation leaves the object as it was.

- **Throw when a function cannot keep its promise**; do not throw for expected conditions such as invalid typed input.
- **Translation has four stages**: preprocessing, compilation, assembly, and linking. Every "undefined reference" you have met is stage 4 finding a promise nobody kept.
- **Static libraries** are copied in; **dynamic libraries** are loaded at run time.
- **Test the release build**, not only the debug build — optimization can expose undefined behavior.
- A release needs a build, a README, a user guide, and reconciled design documentation.

Key Terms

catch — a clause handling an exception of a stated type.

debug build — a build with debug information and no optimization.

dynamic library — a library loaded when a program runs.

exception — an object thrown to signal a condition the caller must handle.

exception safety — the property that a failed operation leaves objects in a valid state.

release build — an optimized build with assertions removed.

stack unwinding — destroying local objects while leaving functions between a throw and its handler.

static library — a library copied into an executable at link time.

throw — to raise an exception.

try block — a region whose exceptions may be handled by following `catch` clauses.

what() — the member function returning an exception's message.

Afterword

You began at Chapter 1 with a bit pattern that meant nothing until a program said what it was. You end with an application that computes grades three ways, refuses to be built wrong, explains its failures in sentences, and can be handed to someone else with documentation they can follow.

The specific thing worth noticing: **you wrote twenty-three working programs, and every one of them ran**. Not one chapter left you with something broken. That was a constraint on how this book was written, and it is also a description of how software is best built — in small, complete, verified steps.

The weighted-grading requirement you recorded in Chapter 1 and resolved in Chapter 21 is the same idea at a larger scale. It was written down before it could be built, analyzed before it was understood, deferred deliberately rather than forgotten, and delivered twenty chapters later against a specification that still said what it was supposed to do.

That is the whole discipline, and it transfers to every language you will use after this one.

Appendix A — Setting Up Your Development Environment

Every program in this book runs unchanged in either environment below. Neither requires anything to be purchased, and the first requires nothing to be installed.

All instructions here are numbered text steps. Where a screenshot would normally appear, the step names the menu, button, or command by its label instead, so the procedure can be followed by reading alone.

A.1 What This Book Requires

Requirement	Detail
Compiler	Any supporting **C++17** — g++ 9 or later, clang 10 or later
Libraries	**The standard library only.** Nothing to install
Build system	**None.** Every program builds with one command
Terminal	Needed for the debugger in Chapter 16 and the sanitizer in Chapter 22

The compile command used throughout:

```
g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc
```

For multi-file versions, from Chapter 10 onward:

```
g++ -std=c++17 -Wall -Wextra *.cpp -o gradecalc
```

A.2 Option 1 — The StudySite.ai Editor

The fastest way to work through the reading and the Try It Yourself examples. Nothing to install and nothing to configure.

1. Open the chapter you are working on in StudySite.
2. Open the code editor panel.
3. Type your program into the editor. **Type it rather than pasting it** — Chapter 2 Section 2.5 explains why this matters early on.
4. Run it using the editor's Run control. Compilation and execution happen together.
5. Type any input the program prompts for into the terminal panel.

Suits: reading, experimenting, the Try It Yourself examples, and every single-file program in Chapters 2 through 9.

Does not suit: the symbolic debugger in Chapter 16, or AddressSanitizer in Chapter 22. Both need a full toolchain — use Option 2 for those.

A.3 Option 2 — GitHub Codespaces

A complete Linux machine running in your browser, with VS Code, g++, the debugger, and a real terminal. This is the environment Chapter 16 and Chapter 22 assume.

First-time setup

1. Create a free account at `github.com` if you do not have one.
2. Create a new repository for your coursework. Name it something like `cpp-gradecalculator`. Tick **Add a README file** so the repository is not empty.
3. On the repository page, select the green **Code** button.
4. Choose the **Codespaces** tab, then **Create codespace on main**.
5. Wait for the codespace to build. The first time takes a few minutes; later starts are quick.

VS Code opens in your browser with a file explorer on the left, an editor in the centre, and a panel at the bottom for the terminal.

Confirm the toolchain works

1. Open a terminal: **Terminal** → **New Terminal** from the menu bar.
2. Type:
   ```
   g++ --version
   ```
3. You should see a version number. Anything from **g++ 9** onward supports everything in this book.

If that command reports "not found", the codespace has not finished building. Wait and try again.

Create and run your first program

1. In the file explorer, select the **New File** icon and name the file `main.cpp`.
2. Type the program from Chapter 2 Section 2.1.
3. In the terminal:
   ```
   g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc
   ./gradecalc
   ```
4. **If compilation succeeded, nothing is printed.** Silence means success — Chapter 2 Section 2.5.

Saving your work

Codespaces keeps your files, but commit regularly so the work is in your repository:

```
git add .
git commit -m "Grade Calculator v0.1 - first running program"
git push
```

Chapter 13 Section 13.6 explains why the commit history matters, and the capstone in Chapter 24 asks for it.

A.4 Setting Up the Debugger

Chapter 16 needs this. Do it once, in Codespaces.

Install the C++ extension

1. Select the **Extensions** icon in the left sidebar.
2. Search for `C/C++`.
3. Install the extension published by Microsoft.

Create a launch configuration

1. Select the **Run and Debug** icon in the left sidebar.
2. Select **create a launch.json file**.
3. Choose **C++ (GDB/LLDB)**, then **g++ - Build and debug active file**.

VS Code creates `.vscode/launch.json` and `.vscode/tasks.json`.

Make the build include debug information

Open `.vscode/tasks.json` and confirm the `args` list includes `-g`, `-std=c++17`, `-Wall`, and `-Wextra`. Add any that are missing:

```
"args": [
    "-fdiagnostics-color=always",
    "-g",
    "-std=c++17",
    "-Wall",
    "-Wextra",
    "${file}",
    "-o",
    "${fileDirname}/${fileBasenameNoExtension}"
]
```

Without `-g` the debugger cannot show your variable names or line numbers — Chapter 16 Section 16.3.

For multi-file programs

From Chapter 10 onward your program has several `.cpp` files. Replace `"${file}"` in that list with:

```
"${fileDirname}/*.cpp",
```

and change the output name to something fixed, such as `"${fileDirname}/gradecalc"`.

Confirm the debugger works

1. Open `main.cpp`.
2. Click in the narrow gutter immediately left of a line number inside `main`. A red dot appears — that is a breakpoint.
3. Press **F5**.

4. The program builds, starts, and pauses at the red dot. The **Variables** panel on the left shows your local variables, and the **Call Stack** panel shows how execution reached that line.

If it pauses and shows your variables by name, you are set up for Chapter 16.

A.5 Option 3 — VS Code on Your Own Computer

If you prefer to work offline. The debugger instructions in Chapter 16 apply identically, since Codespaces *is* VS Code.

Windows

1. Install **VS Code** from `code.visualstudio.com`.
2. Install a compiler. The simplest route is **MSYS2** from `msys2.org`. After installing, open the MSYS2 terminal and run:

```
pacman -S mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-gdb
```

3. Add the compiler to your PATH. The directory is typically `C:\msys64\ucrt64\bin`.
4. Open a new terminal and confirm with `g++ --version`.

macOS

1. Install VS Code from `code.visualstudio.com`.
2. Install the command line tools by running:

```
xcode-select --install
```

3. Confirm with `g++ --version`. On macOS this reports clang, which is fine — it supports C++17 fully.

Linux

1. Install VS Code from your distribution's package manager or `code.visualstudio.com`.
2. Install the compiler and debugger:

```
sudo apt install g++ gdb          # Debian, Ubuntu
sudo dnf install gcc-c++ gdb      # Fedora
```

3. Confirm with `g++ --version`.

Then, on any platform

Install the Microsoft **C/C++** extension and follow Section A.4 from "Create a launch configuration" onward.

A.6 Verifying Your Setup

Whichever environment you chose, this program confirms everything works. If it builds clean and prints the expected output, you are ready for Chapter 2.

```cpp
// setup-check.cpp - confirms the toolchain is working
// Build: g++ -std=c++17 -Wall -Wextra setup-check.cpp -o setup-check

#include <iostream>
#include <string>
#include <vector>

int main() {
    std::cout << "=== Setup Check ===\n";
    std::cout << "C++ standard: " << __cplusplus << "\n";

    std::vector<std::string> names = {"Ada", "Grace", "Alan"};
    for (const std::string& n : names) {
        std::cout << "  " << n << "\n";
    }

    std::cout << "If you see three names above, your setup is working.\n";
    return 0;
}
```

Expected output:

```
=== Setup Check ===
C++ standard: 201703
  Ada
  Grace
  Alan
If you see three names above, your setup is working.
```

The number `201703` confirms C++17. A smaller number means the `-std=c++17` flag is not reaching the compiler — check your command or your `tasks.json`.

A.7 When Setup Goes Wrong

Symptom	Cause	Fix
`g++: command not found`	Compiler not installed or not on PATH	Section A.5 for your platform
`__cplusplus` is not `201703`	`-std=c++17` missing	Add it to your command or `tasks.json`
`bash: ./gradecalc: No such file`	Compilation failed, so no executable exists	Read the compiler messages above
`Permission denied` running the program	Executable bit not set	`chmod +x gradecalc`
Debugger starts but shows no variable names	Built without `-g`	Add `-g` to `tasks.json`
Breakpoints appear as hollow circles	The build did not include debug information	Add `-g`; rebuild
`undefined reference to ...`	A `.cpp` was left off the build	List every `.cpp`, or use `*.cpp`
Program cannot find `gradebook.csv`	Terminal is in a different directory	`cd` to the directory containing your program
Codespace will not start	Free monthly hours exhausted	Use the StudySite editor, or a local install

---

A.8 Which Environment for Which Chapter

Chapters	Environment	Why
1	Neither — no code	
2–9	StudySite editor or Codespaces	Single-file programs
10–15	Either	Multi-file builds work in both
16	**Codespaces**	The symbolic debugger is required
17–21	Either	
22	**Codespaces**	AddressSanitizer is required
23–24	Either; Codespaces for the release build	

Use the StudySite editor for reading and experimenting; use Codespaces when a chapter needs the toolchain. Nothing prevents you from using Codespaces throughout, and many students do.

Appendix B — Command Line Development

Chapter 2 introduced one command and Chapter 24 explained the four stages it performs. This appendix collects everything you need to build, run, and inspect C++ programs from a terminal.

Everything here works in GitHub Codespaces, macOS, Linux, and Windows with MSYS2.

B.1 Moving Around

Before compiling, you must be in the directory containing your source.

Command	Does
`pwd`	print the current directory
`ls`	list files here
`ls -l`	list with sizes and dates
`cd foldername`	move into a directory
`cd ..`	move up one level
`cd`	return to your home directory
`mkdir foldername`	create a directory

```
cd cpp-gradecalculator/v1.3
ls
g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc
./gradecalc
```

The `./` in `./gradecalc` means "in the current directory." Without it, the shell searches its usual locations and reports "command not found."

B.2 The Compile Command

```
g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc
```

Part	Meaning
`g++`	the compiler
`-std=c++17`	use the C++17 standard
`-Wall`	report all common warnings
`-Wextra`	report additional warnings
`main.cpp`	the source file
`-o gradecalc`	name the executable

Silence means success. A compiler with nothing to report says nothing.

Multi-file programs

List every `.cpp`. Never list a header.

```
g++ -std=c++17 -Wall -Wextra main.cpp assignment.cpp gradescale.cpp \
    student.cpp gradebook.cpp -o gradecalc
```

Or let the shell expand them:

```
g++ -std=c++17 -Wall -Wextra *.cpp -o gradecalc
```

Omitting a `.cpp` produces an `undefined reference` error from the linker — Chapter 24 Section 24.8.

Useful additional flags

Flag	Does	Used in
`-g`	include debug information	Chapter 16
`-O2`	optimize	Chapter 24 release build
`-DNDEBUG`	remove assertions	Chapter 24 release build
`-fsanitize=address`	detect memory errors	Chapter 22
`-Werror`	treat every warning as an error	optional discipline

The three builds this book uses

```
# Ordinary development
g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc

# Debugging (Chapter 16)
g++ -std=c++17 -Wall -Wextra -g main.cpp -o gradecalc

# Release (Chapter 24)
g++ -std=c++17 -Wall -Wextra -O2 -DNDEBUG main.cpp -o gradecalc
```

B.3 Stopping After Each Stage

Chapter 2 Section 2.2 described four stages. Each can be run alone, which makes the pipeline something you can inspect rather than take on trust.

Command	Stops after	Produces
`g++ -E main.cpp -o main.ii`	preprocessing	the expanded translation unit
`g++ -S main.cpp -o main.s`	compilation	assembly
`g++ -c main.cpp -o main.o`	assembly	an object file
`g++ main.cpp -o gradecalc`	linking	the executable

Worth doing once:

```
g++ -std=c++17 -E main.cpp -o main.ii
wc -l main.ii
```

A six-line program expands past 32,000 lines. Chapter 2 measured exactly this.

Compiling separately, then linking

Large projects compile each file once and relink:

```
g++ -std=c++17 -c student.cpp -o student.o
g++ -std=c++17 -c gradebook.cpp -o gradebook.o
g++ -std=c++17 -c main.cpp -o main.o
g++ student.o gradebook.o main.o -o gradecalc
```

Change one `.cpp` and only that file needs recompiling. **Change a header and every file that includes it must be rebuilt** — which is why Chapter 19 Section 19.3 asks you to keep headers small.

B.4 Running Programs

```
./gradecalc
```

Feeding input from a file

Instead of typing the same input repeatedly while testing:

```
printf 'Ada\n1\nHomework 1\n9\n10\n0\n2\n3\n' > testinput.txt
./gradecalc < testinput.txt
```

The `<` redirects the file into the program's standard input, exactly as if you had typed it.

Capturing output

```
./gradecalc < testinput.txt > output.txt
./gradecalc < testinput.txt 2> errors.txt      # error output only
./gradecalc < testinput.txt > all.txt 2>&1     # both together
```

Comparing two versions

This is the verification procedure from Chapter 13 Section 13.6:

```
./gradecalc13 < testinput.txt | tail -9 > out13.txt
./gradecalc20 < testinput.txt | tail -9 > out20.txt
diff out13.txt out20.txt
```

`diff` printing nothing means the files match. That silence is your evidence that a refactor changed no behavior.

The exit code

```
./gradecalc
echo $?
```

`0` means success, anything else means failure — Chapter 2 Section 2.1, and Chapter 24's release build returns 1 when it cannot save.

Stopping a runaway program

Ctrl+C. Useful when Chapter 7's infinite loops arrive.

B.5 Debugging from the Command Line

Chapter 16 uses the VS Code debugger. The same debugger is available directly as `gdb`, and knowing a few commands is useful when no editor is available.

```
g++ -std=c++17 -Wall -Wextra -g main.cpp -o gradecalc
gdb ./gradecalc
```

Command	Does
`break main`	breakpoint at the start of `main`
`break main.cpp:42`	breakpoint at line 42
`run`	start the program
`next`	execute one line, stepping over calls
`step`	execute one line, stepping into calls
`finish`	run until the current function returns
`print x`	show the value of `x`
`backtrace`	show the call stack
`continue`	run to the next breakpoint
`quit`	leave gdb

The concepts are identical to Chapter 16's — `next` is Step Over, `step` is Step Into, `backtrace` is the Call Stack panel.

B.6 Finding Memory Errors

Chapter 22's tool:

```
g++ -std=c++17 -fsanitize=address -g *.cpp -o gradecalc
./gradecalc
```

The program runs normally and reports leaks, use-after-free, and buffer overruns as they occur:

```
SUMMARY: AddressSanitizer: 264 byte(s) leaked in 4 allocation(s).
```

A related tool checks for undefined behavior:

```
g++ -std=c++17 -fsanitize=undefined -g *.cpp -o gradecalc
```

This catches signed overflow, some out-of-bounds accesses, and other undefined behavior — including things Chapter 3 and Chapter 11 warned about but could not demonstrate reliably.

Run both before considering a program finished.

B.7 Makefiles

Every program in this book builds with one command, so a makefile is a convenience rather than a requirement. Chapter 24 Section 24.8 shows one; here is a slightly fuller version.

Create a file named exactly `Makefile`:

```
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -g
SRCS     = main.cpp assignment.cpp gradescale.cpp student.cpp \
```

```
          gradingscheme.cpp gradebook.cpp
TARGET   = gradecalc

$(TARGET): $(SRCS)
    $(CXX) $(CXXFLAGS) $(SRCS) -o $(TARGET)

release: $(SRCS)
    $(CXX) -std=c++17 -Wall -Wextra -O2 -DNDEBUG $(SRCS) -o $(TARGET)

asan: $(SRCS)
    $(CXX) $(CXXFLAGS) -fsanitize=address $(SRCS) -o $(TARGET)

clean:
    rm -f $(TARGET) *.o

.PHONY: release asan clean
```

Then:

```
make            # ordinary build
make release    # optimized build
make asan       # sanitizer build
make clean      # remove build products
```

> **The indented lines in a makefile must begin with a real tab character, not spaces.** This is the single most common makefile error, and the message — `missing separator` — does not say so.

B.8 Version Control

Chapter 13 Section 13.6 explains why this matters. The commands you need:

Command	Does
`git init`	start tracking a directory
`git status`	what has changed
`git add .`	stage all changes
`git commit -m "message"`	record the staged changes
`git log --oneline`	list the history
`git diff`	show unstaged changes
`git checkout -- file`	discard changes to a file

A typical cycle:

```
git add .
git commit -m "v1.2: grade scale is now user-defined rather than hard-coded"
```

Commit each Grade Calculator version separately, with a message saying *why*. By Chapter 24 you will have twelve commits, each a working program — which is the version history the capstone asks for.

Ignoring build products

Create a file named `.gitignore`:

```
gradecalc
*.o
*.ii
```

```
*.s
gradebook.csv
```

Executables and object files are rebuilt from source and should not be committed.

B.9 Command Reference

```
# Build
g++ -std=c++17 -Wall -Wextra main.cpp -o gradecalc
g++ -std=c++17 -Wall -Wextra *.cpp -o gradecalc
g++ -std=c++17 -Wall -Wextra -g *.cpp -o gradecalc              # debug
g++ -std=c++17 -Wall -Wextra -O2 -DNDEBUG *.cpp -o gradecalc    # release
g++ -std=c++17 -fsanitize=address -g *.cpp -o gradecalc         # memory check

# Inspect the pipeline
g++ -std=c++17 -E main.cpp -o main.ii     # after preprocessing
g++ -std=c++17 -S main.cpp -o main.s      # after compiling
g++ -std=c++17 -c main.cpp -o main.o      # after assembling

# Run
./gradecalc
./gradecalc < testinput.txt
./gradecalc < testinput.txt > output.txt
echo $?

# Compare
diff out13.txt out20.txt

# Debug
gdb ./gradecalc

# Version control
git add . && git commit -m "why this changed"
git log --oneline
```

B.10 Common Command Line Problems

Symptom	Cause	Fix
`command not found: g++`	Compiler not installed or not on PATH	Appendix A Section A.5
`bash: ./gradecalc: No such file`	Build failed, so nothing was produced	Read the compiler output
`Permission denied`	Executable bit not set	`chmod +x gradecalc`
`undefined reference to ...`	A `.cpp` was omitted	List every `.cpp`, or use `*.cpp`
`No such file or directory: main.cpp`	Wrong directory	`pwd`, then `cd`
Program cannot find its data file	Terminal is elsewhere	`cd` to the program's directory
`make: *** missing separator`	Spaces instead of a tab in the makefile	Use a real tab
Program will not stop	Infinite loop	Ctrl+C
Nothing printed after compiling	Compilation succeeded	That is what success looks like

Appendix C — Debugger Reference Card

A one-page reference for Chapter 16. The concepts are the same in every debugger; only the gestures differ.

C.1 Before You Start

Build with `-g`. Without it, the debugger cannot show your variable names or line numbers.

```
g++ -std=c++17 -Wall -Wextra -g *.cpp -o gradecalc
```

Never debug an optimized build. `-O2` rearranges code, so stepping jumps unpredictably and variables may appear "optimized out."

C.2 Core Commands

Action	VS Code	gdb	What it does
Start debugging	**F5**	`run`	Run until a breakpoint
Continue	**F5**	`continue`	Run to the next breakpoint
Step Over	**F10**	`next`	Execute this line, not entering calls
Step Into	**F11**	`step`	Execute this line, entering the first call
Step Out	**Shift+F11**	`finish`	Finish this function, return to caller
Restart	**Ctrl+Shift+F5**	`run`	Start again from the beginning
Stop	**Shift+F5**	`kill`	End the debugging session

Step Over versus Step Into is the distinction that matters. Step *over* a call you trust; step *into* one you suspect.

C.3 Breakpoints

Action	VS Code	gdb
Set at a line	Click the gutter left of the line number	`break file.cpp:42`
Set at a function	—	`break functionName`
Remove	Click the red dot again	`delete N`
Disable temporarily	Right-click → Disable Breakpoint	`disable N`

Action	VS Code	gdb
List all	Breakpoints panel	`info breakpoints`
Conditional	Right-click → Edit Breakpoint, enter a condition	`break file.cpp:42 if i > 5`

Useful conditions

```
student.id == 1042
percentage >= 89.0 && percentage <= 91.0
i >= scale.size()
name == "Ada"
```

Break on the impossible. A conditional breakpoint on something you believe cannot happen is free if you are right and decisive if you are wrong.

C.4 Inspecting State

Want	VS Code	gdb
A local variable	Variables panel, or hover over the name	`print x`
An expression	Watch panel → `+`	`print earned / possible`
A container's size	Watch `scale.size()`	`print scale.size()`
A vector element	Expand it in Variables	`print v[2]`
An object's members	Expand it in Variables	`print *this`
Change a value	Right-click in Variables → Set Value	`set var x = 5`

Watch expressions are the underused feature. Watching the exact comparison you suspect — `pct >= 90.0` — and seeing it as `true` or `false` at each step is usually faster than reasoning about it.

C.5 The Call Stack

Want	VS Code	gdb
See how you got here	Call Stack panel	`backtrace`
Inspect a caller's locals	Click that frame	`frame N` then `info locals`
Move up one frame	Click the frame above	`up`
Move down one frame	Click the frame below	`down`

Read the call stack every time you pause. When a crash lands inside library code, walk down to the first frame that is *your* code and look at what it passed.

C.6 Watchpoints

A watchpoint fires when a **variable changes**, rather than when a line is reached. It answers "what changed this?"

VS Code	gdb
Right-click a variable in Variables → **Break on Value Change**	`watch totalEarned`

Useful when a value is wrong and you do not know which of many places wrote it.

C.7 A Debugging Session, Start to Finish

Following Chapter 16 Section 16.2:

1. **Reproduce**. Find input that fails reliably. Save it to a file so it is repeatable.
2. **Narrow**. Find the smallest input that still fails.
3. **Hypothesize**. State specifically what you think is wrong.
4. **Set a breakpoint** where you can first observe it — not at the top of `main`.
5. **Predict** what you expect to see, before you look.
6. **Run**. Compare what you see with what you predicted.
7. **Step** — over to survey, into to investigate.
8. **Read the call stack** to understand how you arrived.
9. **Fix the cause**, not the symptom.
10. **Write a regression test** that would have caught it.

C.8 Debugging Techniques by Symptom

Symptom	Technique
Crash	Run under the debugger; it stops at the failing line with the stack intact
Wrong answer, no crash	**Bisect** between a known-good point and a known-bad one
Fails only for one input	Conditional breakpoint on that input
Fails on iteration 300	Conditional breakpoint on the loop counter
A value changes and you cannot see where	Watchpoint
Crash inside library code	Walk down the call stack to your nearest frame
Works in debug, fails in release	Undefined behavior — run the sanitizers
Cannot reproduce it at all	Log inputs until you can; an unreproducible defect cannot be verified fixed

---

C.9 When Not to Use a Debugger

Instead	When
Read the compiler messages	The program does not build. The compiler already told you
Diagnostic output	You need to see a pattern across many iterations
Assertions	You want a condition checked continuously, not once
AddressSanitizer	Memory leaks, use-after-free, buffer overruns
UndefinedBehaviorSanitizer	Overflow, bad casts, other undefined behavior
A test harness	You want to verify many cases at once, repeatably

```
g++ -std=c++17 -fsanitize=address -g *.cpp -o gradecalc
g++ -std=c++17 -fsanitize=undefined -g *.cpp -o gradecalc
```

C.10 Common Debugger Problems

Symptom	Cause	Fix
Breakpoints show as hollow circles	Built without `-g`	Add `-g`, rebuild
No variable names, only addresses	Built without `-g`	Add `-g`, rebuild
Stepping jumps around unpredictably	Optimized build	Remove `-O2`
"Variable optimized out"	Optimized build	Remove `-O2`
Breakpoint never fires	Line never reached, or condition never true	Check the condition; break earlier
Debugger will not start	No `launch.json`	Appendix A Section A.4
Program exits immediately	It ran to completion before your breakpoint	Break earlier
Stack shows only `??`	Debug information missing from part of the build	Rebuild everything with `-g`

Appendix D — Coding Standard and Style Guide

Every code listing in this book follows the rules in this appendix. So does every version of the Grade Calculator in the companion code repository. When Chapter 13 asks you to bring your Course I code into conformance with a written standard, this is that standard.

A style guide is not a matter of taste dressed up as rules. It exists so that you spend your attention on what a program *does* instead of on what it *looks like*, and so that a reader who has never seen your code can find their way around it. Chapter 13 makes the professional case at length. This appendix is the rules themselves.

How to use this appendix. Read Sections D.1 through D.5 once, early. Refer back to the rest when a specific question comes up. Section D.12 is a one-page checklist you can run through before submitting work.

D.1 Layout

Indentation

Use **four spaces** per level. Never use tab characters — they render at different widths in different editors, and a file that mixes tabs and spaces looks correct to its author and broken to everyone else.

Most editors can insert spaces when you press Tab. In VS Code and the StudySite editor this is on by default.

```
int main() {
    double total = 0.0;
    if (total > 0.0) {
        std::cout << "positive\n";
    }
    return 0;
}
```

Braces

The opening brace goes on the **same line** as the statement that introduces it. The closing brace goes on its own line, aligned with the start of that statement.

```
if (percentage >= 90.0) {
    letter = 'A';
} else {
    letter = 'B';
}
```

Always use braces, even when the body is a single statement. This is not fussiness. Consider:

```
// Do not write this.
if (possible > 0.0)
    percentage = earned / possible * 100.0;
```

The day someone adds a second line to that body — and forgets the braces — the program compiles and silently does the wrong thing. Braces from the start make that impossible.

One exception is permitted, for short guard clauses where the whole statement fits on one line:

```
if (possible <= 0.0) { return 0.0; }
```

The braces are still there. The line is just compact enough to read at a glance.

Line length

Keep lines to **90 characters** or fewer. Long lines force horizontal scrolling, which makes code hard to read on a laptop, in a browser editor, and in a side-by-side diff.

When a line must wrap, break it at a point that makes the structure obvious, and indent the continuation:

```
std::cout << std::left << std::setw(20) << student.name()
          << std::right << std::setw(8) << student.percentage() << "%\n";
```

Blank lines

Use a single blank line to separate logical groups within a function, and between functions. Never use two or more consecutive blank lines inside a function — if a function needs that much visual separation, it probably wants to be two functions.

D.2 Naming

Names are the most valuable documentation in a program, because they appear at every point of use.

Kind	Convention	Examples
Variable	`camelCase`	`pointsEarned`, `assignmentName`, `totalPossible`
Function	`camelCase`, verb phrase	`computePercentage`, `readNonNegative`, `printReport`
Class or struct	`PascalCase`, noun	`Student`, `GradeScale`, `WeightedDropLowest`
Class data member	`camelCase` with trailing `_`	`name_`, `pointsEarned_`, `roster_`
Constant	`SCREAMING_SNAKE_CASE`	`A_CUTOFF`, `MAX_STUDENTS`, `CAP_AT_100`
Enumeration type	`PascalCase`	`Scheme`, `Category`
Enumerator	`PascalCase`	`Scheme::Points`, `Scheme::Weighted`
File	`lowercase`, no separators	`gradescale.h`, `gradingscheme.cpp`
Header guard	`SCREAMING_SNAKE_CASE` of the filename	`GRADESCALE_H`

Why members end in an underscore

The trailing underscore on data members tells you, at every single use, whether you are looking at a member of the object or at a local variable or parameter. Compare:

```
double Student::percentage() const {
    double possible = totalPossible();       // local: no underscore
    if (possible <= 0.0) { return 0.0; }
    return totalEarned() / possible * 100.0;
}
```

You never have to scroll up to the class definition to answer "where does this value come from?" It also lets a constructor parameter share its member's name without ambiguity:

```
Student::Student(const std::string& name, int id) : name_(name), id_(id) {}
```

Choosing good names

Say what the thing is, not what type it is. `pointsEarned` is better than `dblPoints`. The compiler already knows the type; the reader needs to know the meaning.

Prefer clarity to brevity, but do not pad. `computePercentage` is better than both `cp` and `computeTheFinalPercentageValue`.

Single letters are acceptable only for loop counters with an obvious, short scope — `i`, `j`, `a`, `s`. Anything that lives longer than a few lines deserves a real name.

Booleans read as a question or a claim. `capAt100`, `dropLowest`, `weightsValid` — each reads naturally in an `if`.

Avoid abbreviations that are not universal. `pts` saves three characters and costs a moment's thought every time it is read. `max`, `min`, `id`, and `csv` are fine; they are standard.

D.3 Comments

A comment should explain **why**, not **what**. The code already says what it does. If the code is so unclear that it needs a comment to restate it, rewrite the code instead.

```
// Do not write this. It says nothing the code does not.
totalEarned += pointsEarned;   // add points earned to total earned

// Write this. It explains a decision the code cannot express.
// Bonus points are added to points EARNED but not to points POSSIBLE, so a
// percentage above 100 is possible. Course policy caps it; see CAP_AT_100.
totalEarned += pointsEarned + bonusPoints;
```

Documentation comments

Every function that another file can call gets a documentation comment in `/** */` form, placed in the header file, above the declaration. Use these tags:

Tag	Use
`@param name`	What one parameter means, including any restriction on its value
`@return`	What the return value means
`@throws Type`	An exception this function may raise and the condition that raises it
`@pre`	A precondition the caller is responsible for, which the function does not check

```
/**
 * Converts points to a percentage, capped and rounded by course policy.
 *
 * @param  earned    total points earned, including any bonus
 * @param  possible  total points possible
 * @return the percentage, rounded to one decimal place
 * @pre    possible > 0; the caller must check this first
 */
double computePercentage(double earned, double possible);
```

Not every function needs all four tags. A function that takes nothing, returns nothing, throws nothing, and requires nothing needs a single sentence.

Short private helpers used only inside one file may take a one-line `//` comment instead.

File headers

Every file begins with a header giving its identity and purpose. Implementation files that belong to a class need only a single line:

```
// gradescale.cpp - Grade Calculator v4.0 - Chapter 24
```

The file containing `main` gets a fuller block, because it is where a reader starts:

```
// ============================================================================
//  Grade Calculator  v3.1                                        Chapter 21
// ----------------------------------------------------------------------------
//  Change  : GradingScheme is now abstract with a pure virtual computePercentage
//            and a virtual destructor. Gradebook holds a GradingScheme* and
//            never asks which scheme it has.
//  Run     : click Run in StudySite and use the embedded Terminal.
// ============================================================================
```

Section banners

In a long file, separate major groups of functions with a banner. Keep them consistent within a file.

```
// ----------------------------------------------------------------------------
//  Input helpers
// ----------------------------------------------------------------------------
```

Recording known limitations

When you knowingly leave something imperfect, say so in the code, and say what would fix it. A limitation you documented is a decision. A limitation you did not is a bug waiting to be blamed on you.

```
// LIMITATION (removed in Chapter 21): selection happens through a single branch
// here, because computePercentage is not yet virtual. The scheme is therefore
// fixed once the program starts.
```

D.4 Files and Includes

One class per file

Each class gets a header and an implementation file named after it in lowercase: `gradescale.h` and `gradescale.cpp`. Small, tightly related helper types may share their owner's header — `GradeScale::Tier` lives

inside `gradescale.h` because it has no meaning apart from `GradeScale`.

Header guards

Every header is wrapped in a guard so that including it twice is harmless:

```
#ifndef GRADESCALE_H
#define GRADESCALE_H

// ... contents ...

#endif
```

Name the guard after the file, in capitals, with the dot replaced by an underscore. `#pragma once` works on every compiler this book targets and is shorter, but it is not part of the C++ standard, so this book uses guards.

Include order

Group includes, most specific first, with a blank line between groups:

1. The header this file implements, if any
2. Other headers from this project
3. Standard library headers

```
#include "gradebook.h"        // the header this file implements

#include "gradebookerror.h"   // other project headers

#include <algorithm>          // standard library, alphabetical
#include <iostream>
#include <vector>
```

Putting the file's own header first is not arbitrary: if that header is missing an include it needs, this arrangement makes the compiler say so immediately instead of hiding the problem behind whatever came before.

Include what you use

If a file names `std::vector`, it includes `<vector>` — even if some other header it includes happens to provide it already. Relying on an indirect include means your file breaks when an unrelated header changes.

Never write `using namespace std;`

Write `std::cout`, `std::vector`, `std::string` in full. The five extra characters tell every reader exactly where a name comes from, and they prevent collisions between your names and the standard library's. This rule is absolute in headers, where a `using` directive would be forced on every file that includes them, and it is followed in implementation files too for consistency.

D.5 Functions

A function does one thing. If you find yourself writing a comment that says "now we do the second part," that is a second function asking to exist.

Keep functions short enough to see at once. There is no hard limit, but a function longer than about 40 lines is worth a second look.

Use guard clauses to handle the exceptional case first, then let the main logic run unindented:

```
double Student::percentage() const {
    double possible = totalPossible();
    if (possible <= 0.0) { return 0.0; }      // guard: get it out of the way

    double raw = totalEarned() / possible * 100.0;
    return std::round(std::min(raw, 100.0) * 10.0) / 10.0;
}
```

Pass large objects by `const` reference, not by value. Copying a `std::string` or `std::vector` on every call is wasted work, and `const` documents that the function will not modify it.

```
void printReport(const std::string& student, const std::vector<Assignment>& work);
```

Pass small values — `int`, `double`, `char`, `bool` — by value. A reference to a `double` is no cheaper than the `double`.

Use an output parameter only when a function must return two things. Prefer a return value where one will do.

Mark member functions `const` when they do not modify the object. Do this from the beginning; retrofitting `const` onto a finished class is tedious, and until it is done, `const` objects are almost unusable.

D.6 Variables and Types

Declare a variable at the point where you can give it a useful value, not at the top of the function. A variable that exists for twenty lines before it means anything is twenty lines of opportunity to misuse it.

Always initialize. An uninitialized variable holds whatever was in that memory, which makes the resulting bug different every run.

```
double totalEarned = 0.0;    // yes
double totalEarned;          // no
```

Class members are initialized where they are declared:

```
private:
    std::string name_;
    double pointsEarned_   = 0.0;
    double pointsPossible_ = 0.0;
    double bonusPoints_    = 0.0;
```

Use `const` for anything that should not change, including function parameters you do not intend to modify.

Name your magic numbers. A bare `90.0` in the middle of a conditional means nothing to a reader; `A_CUTOFF` means everything. The exceptions are `0`, `1`, and values whose meaning is unmistakable in context, such as `100.0` when converting a ratio to a percentage.

Use `auto` where the type is obvious from the right-hand side and the name still carries the meaning:

```
auto it = idIndex_.find(id);        // yes: the iterator type is noise
auto total = 0;                     // no: is this an int? a double? say so
```

Choose the type that matches the quantity. Points are `double` because partial credit exists. Counts are `int`. A student ID is `int` because it is an identifier, not a quantity you do arithmetic on. Container sizes are `std::size_t`, which is what the containers themselves return.

D.7 Classes

Order the members of a class definition as follows:

1. `public:` — constructors, then member functions
2. `protected:` — anything derived classes need
3. `private:` — helper functions, then data members

Public first, because the interface is what a reader wants and the implementation is what they can skip.

```
class GradeScale {
public:
    struct Tier {
        double cutoff = 0.0;
        char   letter = 'F';
    };

    GradeScale();
    explicit GradeScale(const std::vector<Tier>& requested);

    char letterFor(double percentage) const;
    std::size_t tierCount() const { return tiers_.size(); }

private:
    std::vector<Tier> tiers_;
};
```

Data members are always private. If callers need access, provide an accessor. A public data member is a promise you cannot later take back.

Mark single-argument constructors `explicit` unless you specifically want the implicit conversion. Without it, the compiler will silently convert anything convertible to the parameter type into your class, in places you did not intend.

Very short accessors may be defined inline in the header. Anything with real logic goes in the `.cpp` file.

State class invariants in a comment above the class, and enforce them in the constructor:

```
/**
 * An ordered set of grade cutoffs.
 * INVARIANT: cutoffs strictly descend and the lowest tier is 0, so every
 * percentage from 0 upward maps to exactly one letter.
 */
```

An invariant established in the constructor is a fact the rest of your code can rely on without checking. That is the entire value of encapsulation, and it is why `letterFor` needs no error handling.

D.8 Error Handling

Validate input where it enters the program. Once a value is inside a well-designed class, it should already be known good.

Throw an exception when a function cannot do what its name promises. Do not return a sentinel value that callers will forget to check, and do not silently substitute a default the user did not ask for.

```
// The user typed a scale whose tiers do not descend. Refusing is honest;
// quietly repairing it would compute grades from something they never chose.
throw InvalidScaleError("tier 'B' at 85 is not below the previous tier at 80.");
```

Catch exceptions where you can do something useful about them — usually where you can tell the user what happened and let them try again.

Exception messages address the user, not the programmer. Name what went wrong, where, and what to do about it. `"Cannot read 'gradebook.csv' at line 14: expected a number here."` is useful. `"parse error"` is not.

Leave no operation half-finished. If an operation can fail partway, build the result separately and install it only on success, so a failure leaves the previous state untouched.

D.9 Output Formatting

Console output is this book's entire user interface, so it gets the same care as the code.

Prompts state what is wanted and any restriction on it:

```
Points possible (must be > 0):
```

Align related output in columns using `std::setw` and `std::left` / `std::right`. A ragged report is hard to scan.

Set floating-point formatting once, near the output, not scattered through the function:

```
std::cout << std::fixed << std::setprecision(1);
```

Use `"\n"`, not `std::endl`. `std::endl` also flushes the stream, which is rarely what you want and is slower in a loop. Reach for `std::flush` explicitly on the rare occasion you need it.

Explain a missing result rather than printing a misleading one. An assignment worth zero points has no percentage; say that, do not print `0.0%`.

Use plain text only. No ANSI color codes, no box-drawing characters, no cursor positioning. Output must be readable in the StudySite terminal, in Codespaces, and by a screen reader.

D.10 Portability

Every program in this book must build and run unchanged in the StudySite editor and in GitHub Codespaces. That rules out:

- Platform headers — `<windows.h>`, `<conio.h>`, `<unistd.h>`
- `system()` calls of any kind, including `system("pause")` and `system("cls")`
- Compiler extensions and anything outside the C++17 standard
- Absolute file paths, and backslash path separators
- External libraries — the standard library only

Every file must compile clean under C++17 with all warnings enabled:

```
-std=c++17 -Wall -Wextra
```

StudySite's **Run** applies these settings for you. If you also build locally, pass them to `g++` yourself — the standard is the settings, not the command.

Warnings are errors. A warning is the compiler telling you it understood your program differently than you probably meant. Fix the cause; do not silence the message.

D.11 What Not to Do

A short list of habits that appear in older textbooks and internet examples, with the reason each is excluded.

Avoid	Why
`using namespace std;`	Hides where names come from; invites collisions
`goto`	Makes control flow unfollowable; every use here has a better structured form
`system("pause")`	Not portable, and launches a shell to do nothing
C-style casts, `(int)x`	Silent and unsearchable; use `static_cast<int>(x)`
`char*` string handling	Use `std::string`; see Section 11.9 for why C strings still get covered
Global variables	Any function can change them, so no function can be reasoned about alone
`#define` for constants	No type, no scope; use `const`
Single-letter names outside loops	Costs the reader more than it saves the writer
Commented-out code	Delete it — that is what version control is for
`std::endl` in a loop	Flushes on every iteration for no benefit

D.12 Pre-Submission Checklist

Run through this before you hand in any program.

Builds and runs

- ☐ Compiles under `-std=c++17 -Wall -Wextra` with **no warnings** — in StudySite, click **Run**
- ☐ Runs to a normal exit on valid input
- ☐ Does not crash on invalid input that its own prompts invite
- ☐ Uses only the standard library and no platform-specific calls
- ☐ Saved to your project repository with **Save to GitHub**, and the commit is visible on GitHub

Layout

- ☐ Four-space indentation, no tab characters
- ☐ Braces on every block, opening brace on the same line
- ☐ No line longer than 90 characters

Naming

- ☐ Variables and functions `camelCase`, classes `PascalCase`, constants `SCREAMING_SNAKE_CASE`
- ☐ Class data members end in `_`
- ☐ No magic numbers — every meaningful constant is named
- ☐ No single-letter names outside short loops

Comments

- ☐ File header present and accurate
- ☐ Every public function has a documentation comment with the tags that apply
- ☐ Comments explain *why*, never restate *what*
- ☐ Any known limitation is recorded in the code
- ☐ No commented-out code left behind

Structure

- ☐ Each function does one thing
- ☐ Data members are private
- ☐ Non-modifying member functions are marked `const`
- ☐ Large parameters passed by `const` reference
- ☐ Every variable initialized where declared
- ☐ Headers guarded, includes grouped and ordered
- ☐ No `using namespace std;` anywhere

D.13 A Conforming Example

The following is `gradescale.h` from Grade Calculator v4.0, unmodified. Every rule in this appendix is visible in it: the file header, the guard, grouped includes, the class-invariant comment, public-before-private ordering, `explicit`, a `const` member function, a documentation comment with `@throws`, a trailing-underscore data member, and four-space indentation throughout.

```
// gradescale.h - Grade Calculator v4.0 - Chapter 24
#ifndef GRADESCALE_H
#define GRADESCALE_H

#include <iosfwd>
#include <vector>

/**
 * An ordered set of grade cutoffs.
 * INVARIANT: cutoffs strictly descend and the lowest tier is 0, so every
 * percentage from 0 upward maps to exactly one letter. The constructor
 * establishes this, and no member function can break it.
 */
class GradeScale {
public:
    struct Tier {
        double cutoff = 0.0;
        char   letter = 'F';
    };

    GradeScale();

    /**
     * Builds a scale from user-supplied tiers.
     *
     * @param  requested  tiers in descending order, lowest cutoff 0
     * @throws InvalidScaleError if the tiers do not strictly descend, if any
     *         cutoff is negative, or if the scale does not reach 0.
     */
    explicit GradeScale(const std::vector<Tier>& requested);

    char letterFor(double percentage) const;
```

```
    std::size_t tierCount() const { return tiers_.size(); }
    const std::vector<Tier>& tiers() const { return tiers_; }

private:
    std::vector<Tier> tiers_;
};

std::ostream& operator<<(std::ostream& out, const GradeScale& scale);

#endif
```

Every version of the Grade Calculator in the companion code repository conforms to this standard. When you need to see a rule applied in context, read the reference implementation for the chapter you are working on.

Appendix E — Glossary

Every key term from the book, alphabetized, with the chapter that introduces it.

Terms are defined where they first do real work rather than where they are first mentioned in passing. A cross-reference such as *(Ch 11)* points to the chapter whose Key Terms section defines it, and where the surrounding explanation lives.

A

abstract class — a class with at least one pure virtual function; cannot be instantiated. *(Ch 21)*

access specifier — `public`, `protected`, or `private`, controlling what may reach a member. *(Ch 18)*

accessor — a member function that reads state without modifying it. *(Ch 18)*

accumulator — a variable that builds a running total across loop passes. *(Ch 7)*

adaptive maintenance — changes responding to a changed environment. *(Ch 13)*

address — the number identifying one cell of main memory. *(Ch 1)*

address-of operator — `&`, giving the address of a variable. *(Ch 10)*

aggregate initialization — initializing a struct's members with a brace-enclosed list in declaration order. *(Ch 14)*

agile — a lifecycle model with short cycles, continuous feedback, and changing requirements. *(Ch 13)*

algorithm — a finite sequence of unambiguous, effective steps that solves a problem. *(Ch 5)*

algorithm (STL) — a standard function operating on an iterator range. *(Ch 23)*

analysis — the phase that models the problem precisely. *(Ch 13)*

application software — software that performs a task a user wants done, as opposed to managing the machine. *(Ch 1)*

argument — a value passed to a function at the call site. *(Ch 8)*

array — a collection of values of one type, stored under one name and reached by index. *(Ch 11)*

ASCII — a character encoding assigning the values 0–127 to English letters, digits, punctuation, and control codes. *(Ch 1)*

assembler — a program translating assembly language into machine language. *(Ch 1)*

assembly language — a low-level language with one short mnemonic per machine instruction, specific to a processor family. *(Ch 1)*

assertion — a checked statement of something that must be true, removed in release builds. *(Ch 16)*

assignment — storing a new value in an existing variable, written with `=`. *(Ch 3)*

associativity — the rule deciding the order among operators of equal precedence. *(Ch 4)*

at() — a bounds-checked element access that throws on an invalid index. *(Ch 12)*

automatic storage — memory for local variables, released when scope ends. *(Ch 22)*

B

base case — the condition under which a recursive function returns without recursing. *(Ch 10)*

base class — a class from which another inherits. *(Ch 20)*

binary — base two, using the digits 0 and 1. *(Ch 1)*

binary search — a search that halves the remaining range each step; requires sorted data. *(Ch 17)*

bit — the smallest unit of data, holding one of two states. *(Ch 1)*

bitwise operator — an operator acting on the individual bits of a value. *(Ch 4)*

block — a group of statements enclosed in braces and treated as a unit. *(Ch 2)*

bool — a type holding exactly `true` or `false`. *(Ch 3)*

Boolean expression — an expression producing `true` or `false`. *(Ch 6)*

bounds — the valid range of indices for an array, 0 through size−1. *(Ch 11)*

break — a statement exiting a loop immediately. *(Ch 7)*

breakpoint — an instruction to pause execution at a particular line. *(Ch 16)*

build tool — a program that automates compiling a project of many files. *(Ch 2)*

byte — eight bits; the standard unit of storage; holds 256 distinct patterns. *(Ch 1)*

bytecode — an intermediate form produced by a hybrid language's compiler and executed by a virtual machine. *(Ch 1)*

C

C string — a `char` array terminated by the null character `'\0'`. *(Ch 11)*

call — to invoke a function by name. *(Ch 8)*

call stack — the sequence of function calls leading to the current point. *(Ch 16)*

capture list — the `[]` of a lambda, naming what it takes from the surrounding scope. *(Ch 23)*

cast — an explicit conversion of a value to another type, written `static_cast<Type>(value)`. *(Ch 3)*

catch — a clause handling an exception of a stated type. *(Ch 24)*

category weight — the percentage of a final grade a category contributes. *(Ch 20)*

char — a type holding one character in one byte; also a small integer. *(Ch 3)*

class — a type bundling data with the functions that operate on it. *(Ch 18)*

class template — a class parameterized by one or more types. *(Ch 23)*

code point — the number Unicode assigns to a character. *(Ch 1)*

coding standard — a written agreement on formatting, naming, and documentation. *(Ch 13)*

comment — text in source code ignored by the compiler, written for human readers. *(Ch 2)*

comparison function — a function defining an ordering, passed to a sorting routine. *(Ch 17)*

compile-time error — a fault preventing the compiler from producing a program. *(Ch 4)*

compiler — a program translating an entire source program into machine code before it runs. *(Ch 1)*

composition — holding an object as a member, expressing "has-a". *(Ch 20)*

compound assignment — an operator such as `+=` combining an operation with assignment. *(Ch 4)*

condition — the Boolean expression a selection statement tests. *(Ch 6)*

conditional breakpoint — a breakpoint that pauses only when a condition holds. *(Ch 16)*

conditional operator — `? :`, which selects between two values. *(Ch 6)*

const — a qualifier marking a variable as unmodifiable. *(Ch 3)*

const member function — a member function that does not modify the object. *(Ch 18)*

constructor — a function run when an object is created, establishing its invariant. *(Ch 18)*

container — a class holding a collection of objects. *(Ch 23)*

continue — a statement skipping the rest of the current pass. *(Ch 7)*

copy assignment — the operation assigning one existing object to another. *(Ch 19)*

copy constructor — the constructor creating an object as a copy of another. *(Ch 19)*

corrective maintenance — fixing defects. *(Ch 13)*

counter — a variable counting the number of loop passes. *(Ch 7)*

counter-controlled — repetition running a known number of times. *(Ch 7)*

D

dangling pointer — a pointer to memory that has been released. *(Ch 22)*

debug build — a build with debug information and no optimization. *(Ch 24)*

debugger — a tool for pausing a running program and inspecting its state. *(Ch 2)*

decimal — base ten, using the digits 0 through 9. *(Ch 1)*

decision — a flowchart symbol, drawn as a diamond, with one entry and two labelled exits. *(Ch 5)*

declaration — a statement bringing a variable into existence with a name and type. *(Ch 3)*

decrement — the `--` operator, subtracting one. *(Ch 4)*

default argument — a parameter value used when the caller omits that argument. *(Ch 9)*

defect report — a record of a fault's symptom, cause, fix, and verification. *(Ch 16)*

definition — a function's full declaration together with its body. *(Ch 9)*

delimiter — the character separating fields in a line of text. *(Ch 15)*

dereference operator — `*`, giving the value stored at an address. *(Ch 10)*

derived class — a class inheriting from another. *(Ch 20)*

design — the phase deciding how a system will be built. *(Ch 13)*

desk check — testing an algorithm by hand with specific values. *(Ch 5)*

destructor — a function run when an object is destroyed. *(Ch 18)*

development — the phase in which code is written, integrated, and tested. *(Ch 13)*

distribution — an object shaping raw random values into a desired range. *(Ch 8)*

divide and conquer — solving a problem by splitting it, solving the parts, and combining. *(Ch 17)*

do/while — a loop testing its condition after the body, so the body always runs once. *(Ch 7)*

dot operator — `.`, used to reach a member of a struct or object. *(Ch 14)*

double — the standard floating-point type, 8 bytes, about 15 digits of precision. *(Ch 3)*

double free — releasing the same memory twice. *(Ch 22)*

dynamic binding — deciding at run time which function a call invokes, based on the object's type. *(Ch 21)*

dynamic library — a library loaded when a program runs. *(Ch 24)*

dynamic storage — heap memory obtained with `new` and released with `delete`. *(Ch 22)*

dynamic_cast — a cast checking at run time whether an object is of a given type. *(Ch 21)*

E

editor — the program in which source code is written. *(Ch 2)*

element — one value stored in an array. *(Ch 11)*

encapsulation — bundling data with behavior and restricting access to the data. *(Ch 18)*

encoding — a scheme for storing Unicode code points as bytes; UTF-8 is the most common. *(Ch 1)*

enumerated type — a type whose values are a small fixed named set. *(Ch 3)*

error — a problem that prevents the compiler from producing a program. *(Ch 2)*

escape sequence — a backslash followed by a character, representing one character that cannot be typed directly. *(Ch 3)*

exception — an object thrown to signal a condition the caller must handle. *(Ch 24)*

exception safety — the property that a failed operation leaves objects in a valid state. *(Ch 24)*

executable — a file containing machine code that the operating system can run. *(Ch 2)*

explicit — a qualifier preventing implicit conversion through a constructor. *(Ch 18)*

expression — any construct producing a value. *(Ch 4)*

extraction operator — `>>`, which reads a value from an input stream. *(Ch 3)*

F

fall-through — continuing into the next `case` because `break` was omitted. *(Ch 6)*

file stream — a stream connected to a file. *(Ch 15)*

float — a smaller floating-point type, 4 bytes, about 6 digits of precision. *(Ch 3)*

floating-point — a representation of real numbers as a sign, mantissa, and exponent, offering wide range with limited precision. *(Ch 1)*

flowchart — a diagram showing the steps of an algorithm and the order in which they run. *(Ch 5)*

for — a loop gathering initialization, condition, and update into one line. *(Ch 7)*

friend — a function granted access to a class's private members. *(Ch 19)*

function — a named piece of code performing a task. *(Ch 8)*

function pointer — a variable holding the address of a function. *(Ch 17)*

function template — a function parameterized by one or more types. *(Ch 23)*

G

generator — an object producing a sequence of pseudorandom values. *(Ch 8)*

getline — a function reading an entire line of text, including spaces. *(Ch 3)*

global variable — a variable declared outside every function, visible to all. *(Ch 10)*

H

hardware — the physical components of a computer. *(Ch 1)*

has-a — a relationship expressed by composition. *(Ch 20)*

header — a file, brought in with `#include`, declaring facilities defined elsewhere. *(Ch 2)*

header guard — `#ifndef`/`#define`/`#endif` preventing a header from being processed twice. *(Ch 10)*

heap — the memory region for dynamically allocated objects. *(Ch 22)*

hexadecimal — base sixteen, using the digits 0–9 and letters A–F; each digit corresponds to four bits. *(Ch 1)*

high-level language — a language whose statements describe operations in terms closer to the problem than to the machine. *(Ch 1)*

I

IDE — an integrated development environment, bundling editor, compiler, and debugger. *(Ch 2)*

identifier — a programmer-chosen name. *(Ch 3)*

ifstream — an input file stream. *(Ch 15)*

implicit conversion — an automatic type conversion performed by the compiler. *(Ch 4)*

increment — the `++` operator, adding one. *(Ch 4)*

index — the number identifying an element's position; also called a subscript. *(Ch 11)*

infinite loop — a loop whose condition never becomes false. *(Ch 7)*

inheritance — deriving a new class from an existing one. *(Ch 20)*

initialization — giving a variable a value at the moment it is declared. *(Ch 3)*

initializer list — the `: member(value)` syntax initializing members before the constructor body. *(Ch 18)*

input — data entering the computer from outside. *(Ch 1)*

insertion operator — `<<`, which sends data into an output stream. *(Ch 2)*

insertion sort — a sort that inserts each element into the sorted portion; adapts to input. *(Ch 17)*

instance — one object of a class. *(Ch 18)*

instantiation — the compiler's generation of concrete code from a template for a given type. *(Ch 23)*

int — the standard integer type, 4 bytes. *(Ch 3)*

integer division — division of two integers, discarding the remainder. *(Ch 4)*

interface — the set of operations an abstract class promises every derived class provides. *(Ch 21)*

interpreter — a program translating and executing source code line by line as it runs. *(Ch 1)*

invariant — a property that is always true of an object. *(Ch 18)*

is-a — a relationship expressed by inheritance. *(Ch 20)*

iteration — one pass through a loop body; also, repetition in general. *(Ch 7)*

iterative model — a lifecycle repeating all phases, producing working software each cycle. *(Ch 13)*

iterator — an object identifying a position in a container. *(Ch 23)*

L

lambda — an unnamed function defined where it is used. *(Ch 23)*

lifetime — the period during which a variable exists. *(Ch 9)*

linear search — a search examining elements one at a time in order. *(Ch 17)*

linked list — a chain of nodes, each holding a value and a pointer to the next. *(Ch 22)*

linker — the tool that joins object files and library code into an executable. *(Ch 2)*

literal — a value written directly in source code. *(Ch 3)*

loader — the operating system component that copies an executable into memory and starts it. *(Ch 2)*

local variable — a variable declared inside a function, existing only while it runs. *(Ch 9)*

logic error — a fault in which a valid program produces a wrong result. *(Ch 4)*

logical operator — `&&`, `||`, or `!`, combining Boolean expressions. *(Ch 6)*

loop control variable — the variable a loop tests and updates. *(Ch 7)*

M

machine language — the numeric instruction codes a processor executes directly. *(Ch 1)*

magic number — an unexplained numeric value in code; avoid by naming it. *(Ch 3)*

main — the function where execution of a C++ program begins. *(Ch 2)*

main memory — fast, volatile storage holding data and instructions in current use; also called RAM. *(Ch 1)*

maintenance — the phase in which working software is kept correct and current. *(Ch 13)*

manipulator — a value such as `std::fixed` that changes how a stream formats output. *(Ch 4)*

mantissa — the significant digits of a floating-point value. *(Ch 1)*

map — an associative container of key–value pairs, sorted by key. *(Ch 23)*

member — a variable declared inside a struct or class. *(Ch 14)*

member function — a function belonging to an object, called with a dot. *(Ch 12)*

memory leak — allocated memory that is never released. *(Ch 22)*

merge — combining two sorted sequences into one sorted sequence. *(Ch 17)*

merge sort — a divide-and-conquer sort with $n \log n$ comparisons. *(Ch 17)*

modular design — organizing a program as small parts, each doing one thing. *(Ch 9)*

modulus — the `%` operator, giving the remainder of integer division. *(Ch 4)*

move semantics — transferring a resource from one object to another instead of copying. *(Ch 22)*

multilevel hierarchy — a chain in which a derived class is itself a base. *(Ch 20)*

mutator — a member function that changes state. *(Ch 18)*

N

namespace — a named scope grouping declarations to prevent collisions. *(Ch 23)*

nested conditional — a conditional statement inside another. *(Ch 6)*

nested loop — a loop contained within another loop. *(Ch 7)*

nested struct — a struct used as the type of a member of another struct. *(Ch 14)*

newline — the character `\n`, which moves output to the next line. *(Ch 2)*

non-volatile — retaining contents when power is removed. *(Ch 1)*

npos — the value `find` returns when nothing matches. *(Ch 12)*

null pointer — a pointer holding `nullptr`, pointing at nothing. *(Ch 10)*

null terminator — the `'\0'` character marking the end of a C string. *(Ch 11)*

O

object — an instance of a class, holding its own state. *(Ch 18)*

object file — machine code produced from one source file, not yet runnable. *(Ch 2)*

object slicing — losing the derived part of an object by copying it into a base-typed variable. *(Ch 21)*

object-oriented programming — organizing a program as objects bundling data with operations. *(Ch 13)*

octal — base eight; each digit corresponds to three bits. *(Ch 1)*

off-by-one error — a loop running one pass too many or too few. *(Ch 7)*

ofstream — an output file stream. *(Ch 15)*

operand — a value an operator acts upon. *(Ch 4)*

operator — a symbol combining operands to produce a value. *(Ch 4)*

operator overloading — defining what an operator means for a user-defined type. *(Ch 19)*

out of bounds — an index outside the valid range; undefined behavior. *(Ch 11)*

output — data leaving the computer to the outside world. *(Ch 1)*

overflow — what happens when a value exceeds its type's range; the value wraps around. *(Ch 3)*

overload resolution — the compiler's choice among overloaded functions based on argument types. *(Ch 10)*

overloading — defining several functions with one name and different parameter lists. *(Ch 10)*

override — a keyword asking the compiler to verify that a function overrides a base virtual. *(Ch 21)*

overriding — replacing a base class function with one of the same signature. *(Ch 20)*

P

paradigm — a style of organizing a program, such as procedural, object-oriented, or functional. *(Ch 1)*

parallel arrays — separate arrays in which the same index refers to related values. *(Ch 11)*

parameter — the name a function uses internally for an argument it receives. *(Ch 8)*

parsing — extracting structured values from text. *(Ch 15)*

pass by reference — passing a reference so a function can modify the caller's variable. *(Ch 10)*

pass by value — passing a copy of an argument, so the caller's variable is unaffected. *(Ch 9)*

perfective maintenance — adding capability users want. *(Ch 13)*

persistence — storing data so it survives after a program ends. *(Ch 15)*

planning — the phase deciding whether and what to build. *(Ch 13)*

pointer — a variable holding the address of another variable. *(Ch 10)*

polymorphic collection — a container of base pointers holding objects of several derived types. *(Ch 21)*

polymorphism — one call producing behavior determined by the object rather than the calling code. *(Ch 21)*

pop_back — removes the last element of a vector. *(Ch 12)*

precedence — the rule deciding which operator acts first. *(Ch 4)*

precondition — a requirement the caller must satisfy, stated with `@pre`. *(Ch 9)*

preprocessor — the tool that carries out `#include`, `#define`, and other directives before compilation. *(Ch 2)*

preprocessor directive — a line beginning with `#`, addressed to the preprocessor. *(Ch 2)*

preventive maintenance — changes making future modification easier. *(Ch 13)*

priming read — an input read before a sentinel loop, so the condition has something to test. *(Ch 7)*

procedural programming — organizing a program as procedures operating on separately held data. *(Ch 13)*

process — a flowchart symbol, drawn as a rectangle, representing a computation or assignment. *(Ch 5)*

processor — the component that fetches, decodes, and executes instructions; also called the CPU. *(Ch 1)*

program — a sequence of instructions telling a computer how to accomplish a task. *(Ch 1)*

protected — an access level permitting the class and its derived classes. *(Ch 20)*

prototype — a declaration of a function's signature without its body. *(Ch 9)*

pseudocode — a description of an algorithm in structured English. *(Ch 5)*

pseudorandom — generated deterministically from a seed but statistically resembling randomness. *(Ch 8)*

pure virtual function — a virtual function with no implementation, declared `= 0`. *(Ch 21)*

push_back — adds an element to the end of a vector. *(Ch 12)*

R

RAII — Resource Acquisition Is Initialization; tying resource lifetime to object lifetime. *(Ch 22)*

range — a pair of iterators marking the beginning and one-past-the-end of a sequence. *(Ch 23)*

range-based for — a loop visiting every element of a collection without an index. *(Ch 12)*

record — a group of related values treated as one item; in C++, a struct. *(Ch 14)*

recursion — a function calling itself. *(Ch 10)*

refactoring — changing internal structure without changing behavior. *(Ch 13)*

reference — an alternative name for an existing variable. *(Ch 10)*

regression test — a test written for a specific defect, failing before the fix and passing after. *(Ch 16)*

relational operator — an operator comparing two values: `<`, `>`, `<=`, `>=`, `==`, `!=`. *(Ch 6)*

release build — an optimized build with assertions removed. *(Ch 24)*

repetition — a control structure that performs steps more than once. *(Ch 5)*

requirements statement — a plain-language description of what a program must do. *(Ch 5)*

return — a statement producing a function's result and exiting it. *(Ch 9)*

return value — the result a function hands back to its caller. *(Ch 8)*

row-major order — the layout in which each row of a two-dimensional array is stored contiguously. *(Ch 11)*

Rule of Three — needing any of destructor, copy constructor, or copy assignment implies needing all three. *(Ch 22)*

run-time error — a fault occurring while the program runs. *(Ch 4)*

S

scope — the region of a program in which a name exists. *(Ch 6)*

seed — the starting value determining a pseudorandom sequence. *(Ch 8)*

selection — a control structure that chooses between paths based on a condition. *(Ch 5)*

selection sort — a sort that repeatedly moves the smallest remaining element into place. *(Ch 17)*

sentinel — a special input value signalling the end of data. *(Ch 7)*

sentinel-controlled — repetition running until a sentinel value appears. *(Ch 7)*

sequence — a control structure in which steps happen one after another in order. *(Ch 5)*

shared_ptr — a smart pointer with reference counting and shared ownership. *(Ch 22)*

short-circuit evaluation — stopping evaluation of a compound condition once the result is determined. *(Ch 6)*

signature — a function's return type, name, and parameter list. *(Ch 8)*

size_t — the unsigned type used for container sizes and indices. *(Ch 12)*

smart pointer — an object that owns a pointer and releases it automatically. *(Ch 22)*

software — the instructions a computer carries out. *(Ch 1)*

software development lifecycle — the phases through which software is planned, built, and maintained. *(Ch 13)*

source code — the text of a program as written by a programmer. *(Ch 1)*

specification — a precise, checkable description of inputs, processing, and outputs. *(Ch 5)*

stability — whether a sort preserves the relative order of equal elements. *(Ch 17)*

stack — the memory region holding function frames and local variables. *(Ch 22)*

stack frame — one function call's entry on the call stack, holding its local variables. *(Ch 16)*

stack overflow — a crash caused by recursion that never reaches its base case. *(Ch 10)*

stack unwinding — destroying local objects while leaving functions between a throw and its handler. *(Ch 24)*

standard library — the collection of facilities every C++ implementation provides. *(Ch 2)*

state — the data an object holds. *(Ch 18)*

statement — a single instruction, ending in a semicolon. *(Ch 2)*

static binding — deciding at compile time which function a call invokes. *(Ch 21)*

static library — a library copied into an executable at link time. *(Ch 24)*

static local variable — a local variable retaining its value between calls. *(Ch 10)*

static member — a member belonging to the class rather than to any object. *(Ch 19)*

std — the namespace containing the standard library. *(Ch 2)*

std::cin — the standard input stream, normally the keyboard. *(Ch 3)*

std::string — a type holding text of any length. *(Ch 3)*

std::vector — the standard growable sequence container. *(Ch 12)*

step into — execute the current line, entering any function it calls. *(Ch 16)*

step out — finish the current function and return to its caller. *(Ch 16)*

step over — execute the current line without entering functions it calls. *(Ch 16)*

STL — the Standard Template Library: containers, iterators, and algorithms. *(Ch 23)*

storage — non-volatile, slower memory holding data that must survive power loss. *(Ch 1)*

stream — a destination or source for sequential data, such as `std::cout`. *(Ch 2)*

stream state — the flags recording whether a stream has failed or reached its end. *(Ch 15)*

string literal — text enclosed in double quotes. *(Ch 2)*

stringstream — a stream reading from or writing to a string in memory. *(Ch 15)*

struct — a type grouping related values under one name. *(Ch 14)*

structure chart — a diagram showing what a program is composed of and which part calls which. *(Ch 5)*

structured programming — building programs from sequence, selection, and repetition only. *(Ch 5)*

subscript — the bracketed index used to reach an element. *(Ch 11)*

substr — a function returning part of a string. *(Ch 12)*

switch — a statement comparing one integral value against a list of constants. *(Ch 6)*

symbolic debugger — a debugger presenting a program in terms of its source names. *(Ch 16)*

system software — software managing the computer itself, such as an operating system. *(Ch 1)*

T

table-driven — organized so that behavior is determined by data rather than by branching code. *(Ch 11)*

tag — a leading field identifying what kind of record a line holds. *(Ch 15)*

technical debt — the accumulated future cost of shortcuts taken now. *(Ch 13)*

template — a type parameterized by another type, as in `std::vector<double>`. *(Ch 12)*

template parameter — the placeholder type in a template, conventionally `T`. *(Ch 23)*

terminator — a flowchart symbol, drawn as a rounded rectangle, marking a start or end. *(Ch 5)*

test case — an input paired with an independently determined expected result. *(Ch 16)*

this — a pointer, available in member functions, to the object the function was called on. *(Ch 19)*

throw — to raise an exception. *(Ch 24)*

top-down decomposition — breaking a problem into parts, and those parts into smaller parts. *(Ch 5)*

trace table — a table recording the value of each variable at each step of a desk check. *(Ch 5)*

translation unit — the single expanded file the preprocessor produces from one source file. *(Ch 2)*

truncate — to empty a file when opening it for output. *(Ch 15)*

truncation — discarding a fractional part rather than rounding. *(Ch 4)*

try block — a region whose exceptions may be handled by following `catch` clauses. *(Ch 24)*

two's complement — the standard representation of signed integers, in which the leftmost bit carries a negative place value. *(Ch 1)*

two-dimensional array — an array of arrays, forming a grid. *(Ch 11)*

type — the property of a variable determining what values it can hold, how many bytes it occupies, and how its bits are interpreted. *(Ch 3)*

U

Unicode — a standard assigning a code point to every character in every writing system. *(Ch 1)*

unique_ptr — a smart pointer with sole ownership, releasing on destruction. *(Ch 22)*

unordered_map — a hash-based associative container with faster lookup and no ordering. *(Ch 23)*

unsigned — an integer type with no negative values, using all its bits for magnitude. *(Ch 3)*

UTF-8 — a Unicode encoding storing each character in one to four bytes, compatible with ASCII. *(Ch 1)*

V

variable — a named memory location holding a value of a particular type. *(Ch 3)*

version control — a system recording every change to a codebase. *(Ch 13)*

virtual destructor — a destructor declared `virtual`, ensuring the full object is destroyed through a base pointer. *(Ch 21)*

virtual function — a member function whose implementation is chosen by the object's actual type. *(Ch 21)*

virtual machine — a program executing bytecode, allowing hybrid-language programs to run on many platforms. *(Ch 1)*

void — the return type of a function that returns no value. *(Ch 9)*

volatile — losing contents when power is removed. *(Ch 1)*

W

warning — a report of something suspicious that did not prevent compilation. *(Ch 2)*

watch expression — an expression the debugger evaluates and displays as you step. *(Ch 16)*

watchpoint — a breakpoint that fires when a variable's value changes. *(Ch 16)*

waterfall model — a lifecycle completing each phase before beginning the next. *(Ch 13)*

what() — the member function returning an exception's message. *(Ch 24)*

while — a loop testing its condition before each pass. *(Ch 7)*

word — the amount of data a processor handles most naturally in one operation; commonly 64 bits. *(Ch 1)*

Appendix F — SLO-to-Chapter Coverage Matrix

Every Student Learning Outcome mapped to the chapters that teach it, the Grade Calculator milestones that apply it, and the artifacts that evidence it.

Because exercises are generated by StudySite.ai from the chapter source rather than authored here, this matrix points to the **source material** each outcome is taught from. Generated exercise sets inherit that mapping.

Course I — Programming Fundamentals (Chapters 1–12)

SLO 1.1 — Describe how data are represented, manipulated, and stored in a computer

Where taught	Content
Ch 1 §1.3–1.7	Bits, bytes, words; binary and hexadecimal; two's complement; floating-point; ASCII and Unicode; **a bit pattern has no inherent meaning**
Ch 3 §3.4–3.6	Integer types and ranges; overflow demonstrated; floating-point precision; a `char` *is* a small integer
Ch 4 §4.2–4.4	Integer versus floating-point arithmetic; implicit conversion; casts
Ch 11 §11.3, §11.7	Array memory layout; row-major order

Evidenced by: Ch 1 storage-estimate task; Ch 3 written type justification for every Grade Calculator variable; Ch 4 deliberate integer-division bug and written diagnosis.

SLO 1.2 — Categorize different programming languages and their uses

Where taught	Content
Ch 1 §1.8	Machine, assembly, high-level; compiled, interpreted, hybrid; procedural, object-oriented, functional; choosing by purpose
Ch 1 §1.9	Where C++ sits on all three axes, and why this book uses it
Ch 2 §2.2–2.3	What compilation actually involves; the development toolchain

Evidenced by: Ch 1 Try It Yourself 7 (language category for four stated situations); Ch 2 preprocessor investigation.

SLO 1.3 — Fundamental concepts of data types, structured programming, algorithmic design, and user interface design

Concept	Where taught
Data types	Ch 3 (all), Ch 4 §4.4, Ch 12 §12.8
Structured programming	Ch 5 §5.7 (named), Ch 6 (selection), Ch 7 (repetition)
Algorithmic design	Ch 5 §5.1–5.6
User interface design	Ch 5 §5.8, applied in v0.4

Evidenced by: Ch 5 design document (specification, pseudocode, flowchart, structure chart); v0.4 interface redesign with written justification of three changes.

SLO 1.4 — Software development methodologies: modular design, pseudocode, flowcharting, structure charts, data types, control structures, functions, arrays

Element	Where taught	Applied in
Pseudocode	Ch 5 §5.3	v0.4 design document
Flowcharting	Ch 5 §5.4	v0.4 design document
Structure charts	Ch 5 §5.5	v0.4; converted to functions in Ch 9 §9.7
Control structures	Ch 6, Ch 7	v0.5, v0.6
Functions	Ch 9, Ch 10	v1.0, v1.1
Modular design	Ch 9 §9.6, Ch 10 §10.8	v1.0, v1.1
Arrays	Ch 11	v1.2

Evidenced by: Ch 5 design document; Ch 9's incremental refactor from structure chart to functions, verified unchanged; Ch 10's three-file split.

SLO 1.5 — Develop projects that utilize logical algorithms from specifications and requirements statements

Stage	Chapter	Artifact
Requirements statement	Ch 1	Plain-language requirements, including the deferred weighted-grading scope note
Specification	Ch 5 §5.2	Checkable inputs, processing, outputs, out-of-scope
Implementation from design	Ch 9 §9.7	v1.0, one function per structure-chart box
Full application	Ch 11–12	v1.2, v1.3

Evidenced by: Grade Calculator v1.0 and the Course I capstone v1.3, delivered with a revised design document and a written test plan.

SLO 1.6 — Appropriate design, coding, testing, and documenting of programs that implement specifications

Element	Where taught
Design	Ch 5 (all)
Coding standards	Appendix D, applied from Ch 2 §2.8
Error categories	Ch 4 §4.9
Documentation comments	Ch 9 §9.8
Testing functions individually	Ch 9 §9.9
Boundary testing	Ch 5 §5.6, Ch 6 §6.12

Evidenced by: Ch 9 hand-computed test cases per function; Ch 12 capstone test plan of at least eight cases including boundaries; documentation comments on every public function.

SLO 1.7 — Apply computer programming concepts to new problems or situations

Assessed through transfer rather than a single chapter.

Transfer	From	To
Accumulator pattern	Ch 7 §7.9	Ch 11 array traversal; Ch 20 §20.1 where it demonstrably fails
Replacing code with data	Ch 11 §11.8	Ch 14 struct; Ch 18 class
Library over hand-written	Ch 8 §8.5	Ch 17 §17.10; Ch 23 §23.8
Boundary reasoning	Ch 5 §5.6	Ch 6 §6.12; Ch 16 §16.7

Evidenced by: Try It Yourself variations throughout; Grade Calculator milestones v0.5 through v1.3.

Course II — Object-Oriented Programming (Chapters 13–24)

SLO 2.1 — Identify and explain a programming development lifecycle

Phase	Where taught	Where practiced
Planning	Ch 13 §13.2	Ch 1 requirements statement
Analysis	Ch 13 §13.2	Ch 13 weighted-grading analysis, written before implementation
Design	Ch 13 §13.2	Ch 5 design document; Ch 20 §20.2 design comparison
Development	Ch 13 §13.2	Every Grade Calculator version
Maintenance	Ch 13 §13.2, §13.5	v2.0 onward — the entire second course

Lifecycle models and refactoring discipline: Ch 13 §13.3, §13.5. Version control: Ch 13 §13.6.

Evidenced by: the Ch 13 maintenance plan with a classified backlog; the `diff`-verified behavior-preserving refactors at v2.0, v2.1, v2.5, v2.6, v3.2; the Ch 24 reconciled design document tracing one requirement across both courses; the Git history of twelve versions.

SLO 2.2 — Demonstrate a basic understanding of object-oriented programming by using structs and classes

Element	Where taught	Applied in
`struct`	Ch 14 (all)	v2.1
Classes and encapsulation	Ch 18 §18.1–18.5	v2.5
Constructors	Ch 18 §18.6	v2.5
Class invariants	Ch 18 §18.8	`GradeScale` in v2.5
Operator overloading	Ch 19 §19.4	v2.6
Static members, `friend`	Ch 19 §19.5, §19.7	reference

Evidenced by: v2.1 (every parallel-array pair replaced by a record); v2.5 (`GradeScale` invariant enforced in the constructor, making an invalid scale unconstructable); v2.6 (`operator<<` for four classes).

SLO 2.3 — Use object-oriented techniques including inheritance and polymorphism

Element	Where taught	Applied in
Is-a versus has-a	Ch 20 §20.3, §20.9	design of `GradingScheme`
Base and derived classes	Ch 20 §20.4–20.5	v3.0
`protected`	Ch 20 §20.6	`Weighted::weights_`
Overriding	Ch 20 §20.7	v3.0
Virtual functions, dynamic binding	Ch 21 §21.2	v3.1
Abstract classes	Ch 21 §21.5	v3.1
Virtual destructors, slicing	Ch 21 §21.4, §21.6	v3.1
Polymorphic collections	Ch 21 §21.7	v3.1

Evidenced by: v3.0 (weighted grading working through a hierarchy); v3.1 (a third scheme added with **no edits to any existing class**, verified by `diff`).

SLO 2.4 — Document and format code in a consistent manner

Element	Where
The standard itself	**Appendix D**
Formatting introduced	Ch 2 §2.8
Comment *why*, not *what*	Ch 4 §4.10
Documentation comments	Ch 9 §9.8
File organization	Ch 10 §10.8, Ch 19 §19.3
Conformance as maintenance work	Ch 13 §13.5, performed at v2.0
Defect reports	Ch 16 §16.8
Release documentation	Ch 24 §24.9

Evidenced by: the v2.0 conformance pass, verified behavior-preserving by `diff`; a style audit at every subsequent milestone; the Ch 24 README and user guide.

SLO 2.5 — Apply basic searching and sorting algorithms in software design

Algorithm	Where taught
Linear search	Ch 17 §17.2
Binary search, and its precondition	Ch 17 §17.3
Selection sort	Ch 17 §17.4
Insertion sort	Ch 17 §17.5
Merge sort, divide and conquer	Ch 17 §17.7
Comparing by operation count	Ch 17 §17.8
Library algorithms	Ch 17 §17.10, Ch 23 §23.8
Map-based lookup	Ch 23 §23.6

Evidenced by: v2.4 (sorting by three keys; both searches instrumented with comparison counts); the Ch 17 hand traces; v3.3 (map index replacing the hand-written binary search).

SLO 2.6 — Apply single- and multi-dimensional arrays in software

Element	Where taught	Applied in
One-dimensional arrays	Ch 11 §11.2–11.5	v1.2
Bounds and undefined behavior	Ch 11 §11.3	Ch 16 seeded defect 1
Parallel arrays	Ch 11 §11.6	v1.2, replaced at v2.1
Two-dimensional arrays	Ch 11 §11.7	v1.2 roster
C strings	Ch 11 §11.9	reference
Vectors	Ch 12 §12.2–12.7	v1.3 onward
Vectors of vectors	Ch 12 §12.7	v1.3 roster
Arrays of records	Ch 14 §14.5	v2.1

Evidenced by: v1.2 (two-dimensional roster with parallel name arrays); v1.3 (vector-based, unlimited); v2.1 (records).

SLO 2.7 — Use a symbolic debugger to find and fix runtime and logical errors

Element	Where taught
Systematic debugging process	Ch 16 §16.2
Breakpoints and stepping	Ch 16 §16.3.1–16.3.2
Variables and watch expressions	Ch 16 §16.3.3
The call stack	Ch 16 §16.3.4
Conditional breakpoints, watchpoints	Ch 16 §16.3.5
Debugging run-time errors	Ch 16 §16.4
Debugging logic errors by bisection	Ch 16 §16.5
Assertions	Ch 16 §16.6
Sanitizers	Ch 22 §22.4
Quick reference	**Appendix C**

Evidenced by: the Ch 16 defect hunt (three seeded defects located with the debugger, each producing a defect report and a regression test); the Ch 22 leak investigation under AddressSanitizer.

SLO 2.8 — Basic understanding of programming methodologies: object-oriented, structured, and procedural

Methodology	Where compared	Where practiced
Unstructured	Ch 13 §13.4	excluded — Appendix D §D.11
Procedural	Ch 13 §13.4	**all of Course I**
Structured	Ch 5 §5.7, Ch 13 §13.4	all of Course I
Object-oriented	Ch 13 §13.4	**all of Course II**
Choosing between them	Ch 13 §13.4, Ch 20 §20.2, Ch 21 §21.10	

The comparison is not asserted but demonstrated: **the same application is built procedurally in Course I and converted in Course II**, and Chapter 20 §20.2 costs both designs explicitly.

Evidenced by: Ch 13 Try It Yourself 7 (the same application judged at v1.3 and v3.1, with the reason the answer changes); Ch 20 §20.2 design comparison; Ch 21 §21.10 honest cost accounting.

SLO 2.9 — Describe the phases of program translation from source code to executable code

Phase	Introduced	Revisited with full understanding
Preprocessing	Ch 2 §2.2.1	Ch 24 §24.8
Compilation	Ch 2 §2.2.2	Ch 24 §24.8
Assembly	Ch 2 §2.2.3	Ch 24 §24.8
Linking	Ch 2 §2.2.4	Ch 24 §24.8
Loading	Ch 2 §2.2.4	
Separate compilation	Ch 10 §10.8	Ch 24 §24.8
Static and dynamic libraries	—	Ch 24 §24.8
Debug and release builds	—	Ch 24 §24.8

Evidenced by: the Ch 2 pipeline investigation (measuring a six-line program expanding to 32,192 lines, then to assembly, object file, and executable); three deliberately triggered linker errors in Chapters 9, 10, and 19; the Ch 24 release build.

Where each outcome is assessed

SLO	Primary assessment
1.1	Ch 3 type-justification writeup; Ch 4 integer-division diagnosis
1.2	Ch 1 language-category reasoning
1.3	Ch 5 design document; v0.4 interface redesign
1.4	Ch 5 design document; Ch 9 structure-chart-to-function conversion
1.5	**Course I capstone: v1.3 with revised design document and test plan**
1.6	Ch 9 per-function tests; Ch 12 test plan
1.7	Try It Yourself variations; every milestone
2.1	Ch 13 maintenance plan; **Ch 24 reconciled design document**
2.2	v2.1 records; v2.5 class invariant
2.3	**v3.1: a third scheme added with no edits to existing classes**

SLO	Primary assessment
2.4	v2.0 conformance pass verified by `diff`; Ch 24 release documentation
2.5	v2.4 rankings with instrumented comparison counts
2.6	v1.2 two-dimensional roster; v1.3 vectors
2.7	**Ch 16 defect hunt: three defect reports and three regression tests**
2.8	Ch 13 methodology comparison; Ch 20 design costing
2.9	Ch 2 pipeline investigation; Ch 24 release build

The traced requirement

One artifact evidences SLO 2.1 more directly than any other, and it spans the whole book.

The weighted-grading requirement is recorded, carried, analyzed, designed, and delivered across twenty-one chapters:

Chapter	What happens to it
1	Recorded as an explicit out-of-scope note in the requirements statement
5	Carried into the design document's out-of-scope section
7 §7.9	Explained why the accumulator pattern cannot express it
12	Course I completes without it; the note is still there, unchanged
13	Formally analyzed — before the implementation is known
15	The file format is designed to accommodate it, untested
20 §20.1–20.2	Two designs costed; the hierarchy built; **weighted grading works**
21	Made switchable at run time; a third scheme added with no edits elsewhere
24	Reconciled against the Chapter 5 specification in the capstone

Students do not read about requirements traceability. They produce a two-course example of it.